Erweiterungen

Handbuchseiten (deutsch)

2026-10-26

Inhalt

bwidget **6**
ArrowButton 6
BWidget 9
Button 12
ButtonBox 15
ComboBox 18
Dialog 22
DragSite 26
DropSite 27
DynamicHelp 30
Entry 33
Label 37
LabelEntry 42
LabelFrame 44
ListBox 45
MainFrame 52
MessageDlg 57
NoteBook 59
PagesManager 63
PanedWindow 65
PanelFrame 67
PasswdDlg 69
ProgressBar 71
ProgressDlg 73
ScrollView 75
ScrollableFrame 76
ScrolledWindow 78
SelectColor 80
SelectFont 82
Separator 84
SpinBox 85
StatusBar 88
TitleFrame 89
Tree 91
Widget 100
contents 106
index 108
navtree 108
options 109
expect **117**
expect 118
libexpect 145
itcl **155**
body 155

class 157
code 165
configbody 166
delete 169
ensemble 170
find 172
is 173
itcl 174
itclcomponent 176
itcldelegate 177
itclextendedclass 180
itcloption 189
itclvars 191
itclwidget 192
local 201
scope 202
itk **203**
Archetype 203
Toplevel 208
Widget 211
itk 213
itkvars 214
usual 215
mclistbox **216**
mclistbox 216
mpexpr **221**
mpexpr 221
tclhttpd **229**
httpd 229
tcltls **231**
tls 231
tcludp **246**
udp 246
tclvfs **251**
vfs-filesystems 251
vfs-fsapi 252
vfs 257
vfslib 261
tclws **262**
Calling_a_Web_Service 262
Creating_a_Tcl_Web_Service 275
Creating_a_Web_Service_Type 282
Defining_Types 285
Dictionary_Representation_of_XML_Arrays 288
Embedded_Web_Service 289
Rest_flavor_service_response 291

Tcl_Web_Service_Example 292
Tcl_Web_Service_Math_Example 295
Using_Options 298
index 301
tdbc 302
tdbc 302
tdbc_connection 303
tdbc_mapSqlState 309
tdbc_resultset 309
tdbc_statement 311
tdbc_tokenize 314
tdbcmysql 315
tdbc_mysql 315
tdbcodbc 317
tdbc_odbc 317
tdbcpostgres 320
tdbc_postgres 320
tdbcsqlite3 322
tdbc_sqlite3 322
tdom 323
dom 324
domDoc 333
domNode 342
expat 350
expatapi 361
pullparser 365
schema 367
tdomcmd 388
tnc 389
xpathFunc 391
thread 393
thread 393
tpool 402
tsv 406
ttrace 411
Lizenz 415
Lizenzbedingungen bwidget 415
Lizenzbedingungen expect 416
Lizenzbedingungen itcl 416
Lizenzbedingungen itk 417
Lizenzbedingungen mclistbox 418
Lizenzbedingungen mpexpr 418
Lizenzbedingungen tclhttpd 418
Lizenzbedingungen tcltls 419
Lizenzbedingungen tcludp 420
Lizenzbedingungen tclvfs 420

Lizenzbedingungen tclws 421
Lizenzbedingungen tdbc, tdbcmysql, tdbcodbc 421
Lizenzbedingungen tdbcpostgres 422
Lizenzbedingungen tdbcsqlite3 422
Lizenzbedingungen tdom 423
Lizenzbedingungen thread 429

bwidget

ArrowButton

Stichwörter: ArrowButton

NAME

ArrowButton - Button-Widget in Pfeilform.

ERZEUGUNG

ArrowButton *pathName* ?*option value...*?

STANDARDOPTIONEN

-activebackground	
-activeforeground	
-background oder -bg	-borderwidth oder -bd
-disabledforeground	
-foreground oder -fg	
-highlightbackground	
-highlightcolor	
-highlightthickness	
-relief	
-repeatdelay	
-repeatinterval	
-takefocus	
-troughcolor	

WIDGET-SPEZIFISCHE OPTIONEN

-armcommand
-arrowbd
-arrowrelief
-clean
-command
-dir

-disarmcommand
-height
-helptext
-helptype
-helpvar
-ipadx
-ipady
-state
-type
-width

WIDGET-BEFEHL

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

pathName invoke

BESCHREIBUNG

ArrowButton kann je nach Option **type** von zwei Typen sein: beim Typ **button** ist es eine Standardschaltfläche mit einem darauf gezeichneten Pfeil; beim Typ **arrow** ist es ein Pfeil wie der Pfeil einer Bildlaufleiste.

WIDGET-SPEZIFISCHE OPTIONEN

-armcommand
Legt einen Tcl-Befehl fest, der dem ArrowButton zugeordnet wird, wenn Maustaste 1 über dem ArrowButton gedrückt wird. Wenn die Option **repeatdelay** oder **repeatinterval** eine positive ganze Zahl ist, wird dieser Befehl wiederholt aufgerufen, solange der Mauszeiger über der Schaltfläche ist und bis Maustaste 1 losgelassen wird.

-arrowbd
Wenn der **type** des ArrowButton *arrow* ist, legt das die Randbreite des Pfeils fest. Muss 1 oder 2 sein.

-arrowrelief
Wenn der **type** des ArrowButton *arrow* ist, legt das das Relief des Pfeils fest. Muss **raised** oder **sunken** sein.

-clean
Legt eine Qualitätsstufe zwischen 0 und 2 für den Pfeil fest. Bei 0 wird der Pfeil mit seiner größten Breite und Höhe gezeichnet. Bei 1 wird die Basis des Pfeils ungerade gemacht, damit

die Kanten gleich sind. Bei 2 wird die Basis des Pfeils ungerade gemacht und die Orthogonale zu (base+1)/2, damit die Kanten eine gerade Diagonale haben.

-command
Legt einen Tcl-Befehl fest, der dem ArrowButton zugeordnet wird. Dieser Befehl wird gewöhnlich aufgerufen, wenn Maustaste 1 über dem ArrowButton-Fenster losgelassen wird.

-dir
Legt die Richtung des Pfeils fest: **top**, **bottom**, **left** oder **right**.

-disarmcommand
Legt einen Tcl-Befehl fest, der dem ArrowButton zugeordnet wird, wenn Maustaste 1 losgelassen wird. Dieser Befehl wird auch aufgerufen, wenn der Zeiger nicht über dem ArrowButton ist, und immer vor dem Befehl der Option **command**. Er wird gewöhnlich zusammen mit **armcommand**, **repeatdelay** und **repeatinterval** verwendet.

-height
Legt eine gewünschte Höhe für den ArrowButton fest. Der Wert ist in Bildschirmeinheiten.

-helptext
Text für die dynamische Hilfe. Wenn der Wert leer ist, gibt es für dieses Widget keine Hilfe. Siehe auch DynamicHelp.

-helptype
Typ der dynamischen Hilfe. *balloon* oder *variable* verwenden. Siehe auch DynamicHelp.

-helpvar
Variable, die verwendet wird, wenn die Option **helptype** *variable* ist. Siehe auch DynamicHelp.

-ipadx
Legt einen Mindestabstand zwischen dem Rand des ArrowButton und der rechten und linken Seite des Pfeils fest. Der Wert ist in Bildschirmeinheiten.

-ipady
Legt einen Mindestabstand zwischen dem Rand des ArrowButton und der oberen und unteren Seite des Pfeils fest. Der Wert ist in Bildschirmeinheiten.

-state
Legt einen von drei Zuständen für den ArrowButton fest: **normal**, **active** oder **disabled**.

Wenn der type des ArrowButton *button* ist:
Im Zustand normal wird der ArrowButton mit den Optionen **foreground** und **background** angezeigt. Der Zustand active wird gewöhnlich verwendet, wenn der Zeiger über dem ArrowButton ist. Im Zustand active wird der ArrowButton mit den Optionen **activeforeground** und **activebackground** angezeigt. Im Zustand disabled bestimmen die Optionen **disabledforeground** und **background**, wie der ArrowButton angezeigt wird.

Wenn der type des ArrowButton *arrow* ist:
Nur die Farben des Pfeils ändern sich. Der Hintergrund des ArrowButton wird immer mit der Option **troughcolor** angezeigt. Im Zustand normal wird der ArrowButton mit der Option **background** angezeigt. Der Zustand active wird gewöhnlich verwendet, wenn der Zeiger über dem ArrowButton ist. Im Zustand active wird der ArrowButton mit der Option **activebackground** angezeigt. Im Zustand disabled wird der ArrowButton mit einer dunklen

Schraffur angezeigt.

Der Zustand disabled bedeutet, dass der ArrowButton unempfindlich sein soll: die Vorgabebindungen weigern sich, das Widget zu aktivieren, und ignorieren das Drücken der Maustasten.

-type
Bestimmt den Typ des ArrowButton: **button** für das Aussehen einer Standardschaltfläche oder **arrow** für das Aussehen des Pfeils einer Bildlaufleiste.

-width
Legt eine gewünschte Breite für den ArrowButton fest. Der Wert ist in Bildschirmeinheiten.

WIDGET-BEFEHL

pathName* cget *option
Gibt den aktuellen Wert der Konfigurationsoption zurück, die *option* angibt. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert.

***pathName* configure ?*option*? ?*value option value ...*?**
Fragt die Konfigurationsoptionen des Widgets ab oder ändert sie. Wenn keine *option* angegeben ist, gibt der Befehl eine Liste zurück, die alle verfügbaren Optionen für *pathName* beschreibt. Wenn *option* ohne *value* angegeben ist, gibt der Befehl eine Liste zurück, die die genannte *option* beschreibt (diese Liste ist identisch mit der entsprechenden Teilliste des Wertes, der zurückgegeben wird, wenn keine *option* angegeben ist). Wenn ein oder mehrere Paare aus *option* und *value* angegeben sind, ändert der Befehl die genannten Widget-Optionen auf die genannten Werte; in diesem Fall gibt der Befehl eine leere Zeichenkette zurück. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert. Optionen, die nur gelesen werden können, werden nicht geändert.

***pathName* invoke**
Wenn der **state** des ArrowButton nicht disabled ist, ruft das die Befehle der Schaltfläche auf. Der ArrowButton wird mit der aktiven Farbe und dem Relief sunken neu angezeigt, und **armcommand** wird aufgerufen. Danach wird der ArrowButton mit der normalen Farbe und seinem festgelegten Relief neu angezeigt, und **disarmcommand** und danach **command** werden aufgerufen.

invoke wird aufgerufen, wenn der ArrowButton den Eingabefokus hat und die Leertaste gedrückt wird.

BWidget

Stichwörter: BWidget

Im Aufbau ...

NAME
BWidget - Beschreibungstext

BEFEHL
BWidget::XLFDfont *cmd* ?*arg...*?

BWidget::assert *exp* ?*msg*?

BWidget::badOptionString *type value list*

BWidget::bindMouseWheel *widget*

BWidget::classes *class*

BWidget::clonename *menu*

BWidget::focus *option path*

BWidget::get3dcolor *path bgcolor*

BWidget::getname *name*

BWidget::grab *option path*

BWidget::inuse *class*

BWidget::library *class* ?*class ...*?

BWidget::lreorder *list neworder*

BWidget::parsetext *text*

BWidget::place *path w h* ?*arg...*?

BWidget::write *filename* ?*mode*?

BWidget::wrongNumArgsString *string*

BESCHREIBUNG

Beschreibungstext

BEFEHL

BWidget::XLFDfont *cmd* ?*arg...*?
Beschreibungstext

BWidget::assert *exp* ?*msg*?
Beschreibungstext

BWidget::badOptionString *type value list*
Gibt eine passende Fehlerzeichenkette für einen *value* vom *type* zurück, der nicht zu *list* passt.

BWidget::bindMouseWheel *widget*
Bindet das angegebene *widget* mit den Standardbindungen des Mausrads.

BWidget::classes *class*
Gibt eine Liste der Klassen zurück, die die angegebene *class* braucht.

BWidget::clonename *menu*
Beschreibungstext

BWidget::focus *option path*
Beschreibungstext

BWidget::get3dcolor *path bgcolor*
Beschreibungstext

BWidget::getname *name*
Beschreibungstext

BWidget::grab *option path*
Beschreibungstext

BWidget::inuse *class*
Gibt true oder false zurück, je nachdem, ob die angegebene *class* vom aktuell laufenden Programm verwendet wird.

BWidget::library *class* ?*class ...*?
Gibt eine Zeichenkette mit Code zurück, die alle Bibliotheken enthält, die gebraucht werden, um die durch *class* angegebenen Widgets zu verwenden. Der Code jeder Klasse und der Code ihrer abhängigen Klassen wird an die Zeichenkette angehängt und zurückgegeben. Das ist vor allem nützlich, um BWidgets in ein anderes Projekt zu speichern.

BWidget::lreorder *list neworder*
Beschreibungstext

BWidget::parsetext *text*
Beschreibungstext

BWidget::place *path w h* ?*arg...*?
Dient dazu, das durch *path* angegebene Widget zu positionieren und in der Größe zu ändern. *w* und *h* geben die verlangte Breite und Höhe des Widgets *path* für **wm geometry** an (0 verwendet die aktuellen Werte). Die Platzierung des Widgets relativ zu anderen Widgets oder zum Bildschirm steuern zusätzliche Argumente:

at *x y*
Platziert das durch das Argument *path* angegebene Widget an der Bildschirmposition x,y. Angaben zu den Werten der Fensterplatzierung stehen bei **wm geometry**.

center*?widget?*
Platziert das Widget *path* zentriert gegen *widget* oder gegen das Wurzel-Widget, wenn *widget* nicht angegeben ist.

right*?widget?*
Platziert das Widget *path* links oder rechts vom Bezugs-Widget (oder vom Wurzel-Widget, wenn *widget* nicht angegeben ist). Wenn die Position des Bezugs-Widgets so ist, dass das neu platzierte Fenster verdeckt sein könnte, wird die gegenüberliegende Seite versucht.

below*?widget?*
Wie bei links/rechts oben bewirkt diese Option, dass das Widget entweder über oder unter dem Bezugs-Widget platziert wird; die gegenüberliegende Platzierung wird versucht, wenn

das neu platzierte Widget nicht sichtbar wäre.

BWidget::write *filename* ?*mode*?
Schreibt die aktuell verwendete Menge des Klassencodes von BWidget in den angegebenen *filename*. Der gesamte Code, der nötig ist, um die aktuell verwendeten BWidgets auszuführen, wird in die Datei geschrieben. Das ist vor allem nützlich, um BWidget-Code als eine einzige Datei in ein anderes Projekt zu speichern, anstelle des gesamten BWidget-Pakets.

BWidget::wrongNumArgsString *string*
Gibt eine Standard-Fehlerzeichenkette für die falsche Anzahl von Argumenten zurück. *string* wird an die Standardzeichenkette angehängt.

Button

Stichwörter: Button

NAME
Button - Button-Widget mit erweiterten Optionen

ERZEUGUNG
Button *pathName* ?*option value...*?

STANDARDOPTIONEN

-activebackground	-activeforeground
-anchor	-background oder -bg

WIDGET-SPEZIFISCHE OPTIONEN

-armcommand	-command
-default	-disarmcommand
-height	-helptext
-helptype	-helpvar
-name	-relief
-state	-underline
-width	

WIDGET-BEFEHL
pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

pathName invoke

BESCHREIBUNG

Das Widget Button erweitert den Tk-Button um neue Optionen. Hinzugekommen sind Optionen von DynamicHelp, ein neuer Relief-Stil, eine Rückruffunktion für **arm**/**disarm** und die Optionen **repeatdelay**/**repeatinterval**.

WIDGET-SPEZIFISCHE OPTIONEN

-armcommand
Legt einen Tcl-Befehl fest, der dem Button zugeordnet wird, wenn Maustaste 1 über dem Button gedrückt wird. Wenn die Option **repeatdelay** oder **repeatinterval** eine positive ganze Zahl ist, wird dieser Befehl wiederholt aufgerufen, solange der Mauszeiger über dem Button ist und bis Maustaste 1 losgelassen wird.

-command
Legt einen Tcl-Befehl fest, der dem Button zugeordnet wird. Dieser Befehl wird gewöhnlich aufgerufen, wenn Maustaste 1 über dem Button-Fenster losgelassen wird.

-default
Legt einen von drei Zuständen für den Vorgabering fest: **normal**, **active** oder **disabled**. Im Zustand active wird die Schaltfläche mit dem plattformspezifischen Aussehen einer Vorgabeschaltfläche gezeichnet. Im Zustand normal wird die Schaltfläche mit dem plattformspezifischen Aussehen einer Schaltfläche gezeichnet, die nicht die Vorgabe ist, und es bleibt genug Platz, um das Aussehen der Vorgabeschaltfläche zu zeichnen. Die Zustände normal und active ergeben Schaltflächen derselben Größe. Im Zustand disabled wird die Schaltfläche mit dem Aussehen einer Schaltfläche gezeichnet, die nicht die Vorgabe ist, ohne Platz für das Vorgabeaussehen zu lassen. Der Zustand disabled kann eine kleinere Schaltfläche ergeben als der Zustand active.

-disarmcommand
Legt einen Tcl-Befehl fest, der dem Button zugeordnet wird, wenn Maustaste 1 losgelassen wird. Dieser Befehl wird auch aufgerufen, wenn der Zeiger nicht über dem Button ist, und immer vor dem Befehl der Option **command**. Er wird gewöhnlich zusammen mit **armcommand**, **repeatdelay** und **repeatinterval** verwendet.

-height
Legt eine gewünschte Höhe für den Button fest. Wenn im Button ein Bild oder eine Bitmap angezeigt wird, ist der Wert in Bildschirmeinheiten; bei Text ist er in Textzeilen. Wenn diese Option nicht angegeben ist, wird die gewünschte Höhe des Buttons aus der Größe des angezeigten Bildes, der Bitmap oder des Textes berechnet.

-helptext
Text für die dynamische Hilfe. Wenn der Wert leer ist, gibt es für dieses Widget keine Hilfe. Siehe auch DynamicHelp.

-helptype
Typ der dynamischen Hilfe. *balloon* oder *variable* verwenden. Siehe auch DynamicHelp.

-helpvar
Variable, die verwendet wird, wenn die Option **helptype** *variable* ist. Siehe auch DynamicHelp.

-name

Legt einen Standardnamen für die Schaltfläche fest. Wenn die Option ***nameName** in der Ressourcendatenbank gefunden wird, werden die Optionen **text** und **underline** aus ihrem Wert entnommen.

-relief
Legt den gewünschten 3D-Effekt für das Widget fest. Zulässige Werte sind die Standardwerte für das Button-Relief (**raised**, **sunken**, **flat**, **ridge**, **solid** und **groove**) und **link**, das festlegt, dass das Button-Relief **flat** ist, wenn der Zeiger außerhalb der Schaltfläche ist, und **raised**, wenn der Zeiger innerhalb ist.

-state
Legt einen von drei Zuständen für den Button fest: **normal**, **active** oder **disabled**. Im Zustand normal wird der Button mit den Optionen **foreground** und **background** angezeigt. Der Zustand active wird gewöhnlich verwendet, wenn der Zeiger über dem Button ist. Im Zustand active wird der Button mit den Optionen **activeforeground** und **activebackground** angezeigt. Der Zustand disabled bedeutet, dass der Button unempfindlich sein soll: die Vorgabebindungen weigern sich, das Widget zu aktivieren, und ignorieren das Drücken der Maustasten. In diesem Zustand bestimmen die Optionen **disabledforeground** und **background**, wie der Button angezeigt wird.

-underline
Legt den ganzzahligen Index eines Zeichens fest, das in der Beschriftung der Schaltfläche unterstrichen wird. 0 entspricht dem ersten Zeichen des angezeigten Textes, 1 dem nächsten Zeichen, und so weiter. Die Bindung **<Alt-char>** wird auf dem Toplevel des Buttons automatisch gesetzt, um Button::**setfocus** aufzurufen.

-width
Wenn im Button ein Bild oder eine Bitmap angezeigt wird, ist der Wert in Bildschirmeinheiten; bei Text ist er in Zeichen. Wenn diese Option nicht angegeben ist, wird die gewünschte Breite des Buttons aus der Größe des angezeigten Bildes, der Bitmap oder des Textes berechnet.

WIDGET-BEFEHL

pathName* cget *option
Gibt den aktuellen Wert der Konfigurationsoption zurück, die *option* angibt. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert.

***pathName* configure ?*option*? ?*value option value* ...?**
Fragt die Konfigurationsoptionen des Widgets ab oder ändert sie. Wenn keine *option* angegeben ist, gibt der Befehl eine Liste zurück, die alle verfügbaren Optionen für *pathName* beschreibt. Wenn *option* ohne *value* angegeben ist, gibt der Befehl eine Liste zurück, die die genannte *option* beschreibt (diese Liste ist identisch mit der entsprechenden Teilliste des Wertes, der zurückgegeben wird, wenn keine *option* angegeben ist). Wenn ein oder mehrere Paare aus *option* und *value* angegeben sind, ändert der Befehl die genannten Widget-Optionen auf die genannten Werte; in diesem Fall gibt der Befehl eine leere Zeichenkette zurück. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert. Optionen, die nur gelesen werden können, werden nicht geändert.

***pathName* invoke**
Wenn der **state** des Buttons nicht disabled ist, ruft das die Befehle des Buttons auf. Der

Button wird mit der aktiven Farbe und dem Relief sunken neu angezeigt, und **armcommand** wird aufgerufen. Danach wird der Button mit der normalen Farbe und seinem festgelegten Relief neu angezeigt, und **disarmcommand** und danach **command** werden aufgerufen.

invoke wird aufgerufen, wenn der Button den Eingabefokus hat und die Leertaste gedrückt wird.

ButtonBox

Stichwörter: ButtonBox

NAME

ButtonBox - Menge von Schaltflächen mit waagerechter oder senkrechter Anordnung

ERZEUGUNG

ButtonBox *pathName* ?*option value...*?

WIDGET-SPEZIFISCHE OPTIONEN

-background oder -bg	-default
-homogeneous	
-orient	
-padx	
-pady	
-spacing	
-state	

WIDGET-BEFEHL

pathName add ?*option value...*?

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

pathName delete *index*

pathName index *index*

pathName insert *index* ?*option value...*?

pathName invoke *index*

pathName itemcget *index option*

pathName itemconfigure *index* ?*option*? ?*value option value* ...?

pathName setfocus *index*

BESCHREIBUNG

ButtonBox ordnet Buttons waagerecht oder senkrecht an. Einige Befehle nehmen einen *index* als Argument, der angibt, auf welchen Button sie wirken. Dieser Index kann in jeder der folgenden Formen angegeben werden:

number
Bezeichnet den Button numerisch, wobei 0 dem zuerst hinzugefügten Button entspricht, 1 dem nächsten, und so weiter.

end or last
Bezeichnet den zuletzt hinzugefügten Eintrag.

default
Bezeichnet den Vorgabe-Button.

name
Bezeichnet die Schaltfläche, deren Option **-name** *name* ist.

text
Bezeichnet die Schaltfläche, deren Option **-text** *text* ist.

WIDGET-SPEZIFISCHE OPTIONEN

-background
Legt eine Vorgabe-Hintergrundfarbe für alle hinzugefügten Schaltflächen und für den Rahmen fest.

-default
Legt die Vorgabeschaltfläche der Schaltflächenleiste fest. Der Wert ist eine ganze Zahl, die die n-te hinzugefügte Schaltfläche bezeichnet, beginnend bei 0. Wenn dieser Wert -1 ist (die Vorgabe), werden alle Schaltflächen mit ihrer Option -default auf disabled gezeichnet, und dieser Wert kann nicht geändert werden. Wenn dieser Wert nicht -1 ist, wird die zugehörige Schaltfläche mit der Option -default auf active gezeichnet und die anderen mit der Option -default auf normal. Der Wert kann mit configure geändert werden.

-homogeneous (nur lesbar)
Legt fest, ob die Schaltflächen bei waagerechter Anordnung dieselbe Breite haben müssen.

-orient (nur lesbar)
Legt die Ausrichtung der Schaltflächenleiste fest. Wenn diese Option **horizontal** ist (die Vorgabe), werden Schaltflächen von oben nach unten hinzugefügt. Wenn diese Option **vertical** ist, werden Schaltflächen von links nach rechts hinzugefügt.

-padx
Legt einen Vorgabewert für die Option -padx aller hinzugefügten Schaltflächen fest.

-pady
Legt einen Vorgabewert für die Option -pady aller hinzugefügten Schaltflächen fest.

-spacing
Legt den Vorgabeabstand zwischen den Schaltflächen fest. Dieser Wert kann vor jedem Aufruf von **add** geändert werden.

-state

Legt einen Zustand für alle Schaltflächen in der Schaltflächenleiste fest. Das kann jeder Zustand sein, den Schaltflächen unterstützen.

WIDGET-BEFEHL

***pathName* add ?*option value...*?**
Fügt der Schaltflächenleiste eine Schaltfläche hinzu.

Die Beschreibung der Optionen steht bei Button.

pathName* cget *option
Gibt den aktuellen Wert der Konfigurationsoption zurück, die *option* angibt. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert.

***pathName* configure ?*option*? ?*value option value ...*?**
Fragt die Konfigurationsoptionen des Widgets ab oder ändert sie. Wenn keine *option* angegeben ist, gibt der Befehl eine Liste zurück, die alle verfügbaren Optionen für *pathName* beschreibt. Wenn *option* ohne *value* angegeben ist, gibt der Befehl eine Liste zurück, die die genannte *option* beschreibt (diese Liste ist identisch mit der entsprechenden Teilliste des Wertes, der zurückgegeben wird, wenn keine *option* angegeben ist). Wenn ein oder mehrere Paare aus *option* und *value* angegeben sind, ändert der Befehl die genannten Widget-Optionen auf die genannten Werte; in diesem Fall gibt der Befehl eine leere Zeichenkette zurück. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert. Optionen, die nur gelesen werden können, werden nicht geändert.

pathName* delete *index
Löscht die Schaltfläche an *index* aus der Schaltflächenleiste.

pathName* index *index
Gibt den numerischen Index zurück, der dem Eintrag entspricht.

***pathName* insert *index* ?*option value...*?**
Fügt eine neue Schaltfläche vor dem angegebenen Index in die Schaltflächenleiste ein.

pathName* invoke *index
Ruft den Button auf, den *index* angibt.

pathName* itemcget *index option
Gibt den aktuellen Wert einer Konfigurationsoption für den Eintrag zurück. *Option* kann jeden Wert haben, den der Erzeugungsbefehl des Eintrags akzeptiert.

***pathName* itemconfigure *index* ?*option*? ?*value option value ...*?**
Dieser Befehl entspricht dem Befehl **configure**, er gilt aber für die Optionen eines einzelnen Eintrags, während **configure** für die Optionen des Widgets als Ganzes gilt. **Options** kann jeden Wert haben, den der Widget-Befehl zur Erzeugung des Eintrags akzeptiert. Wenn Optionen angegeben sind, werden sie geändert, wie der Befehl angibt, und der Befehl gibt eine leere Zeichenkette zurück. Wenn keine Optionen angegeben sind, gibt der Befehl eine Liste zurück, die die aktuellen Optionen des Eintrags beschreibt. Optionen, die nur gelesen werden können, werden nicht geändert.

pathName* setfocus *index
Setzt den Fokus auf den Button, den *index* angibt.

ComboBox

Stichwörter: ComboBox

NAME

ComboBox - ComboBox-Widget

ERZEUGUNG

ComboBox *pathName* ?*option value...*?

OPTIONS from ArrowButton

-background oder -bg	-disabledforeground
-foreground oder -fg	-state

OPTIONS from Entry

-command	-disabledforeground
-dragenabled	-dragendcmd
-dragevent	-draginitcmd
-dragtype	-dropcmd
-dropenabled	-dropovercmd
-droptypes	-editable
-entrybg (siehe -background)	-entryfg (see -foreground)
-exportselection	-font
-helptext	-helptype
-helpvar	-highlightbackground
-highlightcolor	-highlightthickness
-insertbackground	-insertborderwidth
-insertofftime	-insertontime
-insertwidth	-justify
-selectbackground	-selectborderwidth
-selectforeground	-show
-state	-takefocus
-text	-textvariable
-width	-xscrollcommand

WIDGET-SPEZIFISCHE OPTIONEN

-autocomplete	-autopost
-bwlistbox	-expand
-height	-hottrack
-images	-listboxwidth
-modifycmd	-postcommand
-values	

WIDGET-BEFEHL

pathName bind ?*arg...*?

pathName cget *option*

pathName clearvalue

pathName configure ?*option*? ?*value option value* ...?

pathName get

pathName getlistbox

pathName getvalue

pathName icursor *index*

pathName post

pathName setvalue *index*

pathName unpost

BESCHREIBUNG

Das Widget ComboBox ermöglicht, einen Wert aus einer Liste zu wählen, die die Option **values** angibt. Die Liste der möglichen Werte kann aufgeklappt werden, indem der ArrowButton gedrückt oder in das Entry geklickt wird, wenn der Wert **editable** der ComboBox **false** ist. Wenn der Wert **editable** der ComboBox **true** ist und das Entry den Fokus hat, können die Pfeiltasten nach oben und unten den Wert ändern. Wenn der aktuelle Wert genau einem Wert in der Liste entspricht, wird der vorherige (Pfeil nach oben) oder der nächste (Pfeil nach unten) Wert der Liste angezeigt. Wenn der aktuelle Wert dem Anfang eines Wertes in der Liste entspricht, wird dieser Wert angezeigt. Wenn der aktuelle Wert zu nichts passt, wird der erste Wert angezeigt.

WIDGET-SPEZIFISCHE OPTIONEN

-autocomplete
Legt fest, ob die Combobox versuchen soll, den Wert im Eingabefeld während der Eingabe zu vervollständigen. Wenn true, füllt die Combobox einen Wert ein, den sie in ihrer Werteliste findet, während in das Entry eingegeben wird.

-autopost
Legt fest, ob die Combobox die Auswahlliste während der Eingabe einblenden soll. Wenn true, blendet die Combobox sie ein und scrollt zu dem Eintrag, der der Eingabe am nächsten kommt.

-bwlistbox
Legt fest, dass die Combobox in der Auswahlliste eine BWidget-Listbox verwendet, anstelle der Tk-Vorgabe. Diese Option ist in der Vorgabe eingeschaltet, wenn die Option **-images** nicht leer ist.

-expand
Legt das Verhalten der Werterweiterung für das Fenster fest. Muss **none** (Vorgabe) oder **tab** sein. Wenn **tab** angegeben ist, wird eine Tab-Bindung hinzugefügt, die versucht, den aktuellen Wert anhand der anderen Werte in **-values** zu erweitern.

-height
Legt die gewünschte Höhe für das Fenster in Zeilen fest. Wenn null oder weniger, wird die gewünschte Höhe des Fensters gerade groß genug gemacht, um alle Elemente der Listbox aufzunehmen.

-modifycmd
Legt einen Tcl-Befehl fest, der aufgerufen wird, wenn der Wert der ComboBox geändert wird, indem er in der Listbox gewählt oder eine Pfeiltaste gedrückt wird.

-postcommand
Legt einen Tcl-Befehl fest, der aufgerufen wird, bevor die Listbox der ComboBox abgebildet wird.

-values
Legt die Werte fest, die in der Listbox der ComboBox angezeigt werden.

WIDGET-BEFEHL

***pathName* bind ?*arg...*?**
Setzt Bindungen auf dem Entry-Widget.

pathName* cget *option
Gibt den aktuellen Wert der Konfigurationsoption zurück, die *option* angibt. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert.

***pathName* clearvalue**
Löscht den aktuellen Text der ComboBox.

***pathName* configure ?*option*? ?*value option value ...*?**
Fragt die Konfigurationsoptionen des Widgets ab oder ändert sie. Wenn keine *option*

angegeben ist, gibt der Befehl eine Liste zurück, die alle verfügbaren Optionen für *pathName* beschreibt. Wenn *option* ohne *value* angegeben ist, gibt der Befehl eine Liste zurück, die die genannte *option* beschreibt (diese Liste ist identisch mit der entsprechenden Teilliste des Wertes, der zurückgegeben wird, wenn keine *option* angegeben ist). Wenn ein oder mehrere Paare aus *option* und *value* angegeben sind, ändert der Befehl die genannten Widget-Optionen auf die genannten Werte; in diesem Fall gibt der Befehl eine leere Zeichenkette zurück. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert. Optionen, die nur gelesen werden können, werden nicht geändert.

***pathName* get**
Gibt den aktuellen Inhalt des Entry zurück.

***pathName* getlistbox**
Gibt den Pfad zur Listbox in der Auswahlliste zurück.

***pathName* getvalue**
Gibt den Index des aktuellen Textes der ComboBox in der Werteliste zurück, oder -1, wenn er zu keinem Wert passt.

pathName* icursor *index
Sorgt dafür, dass der Einfügecursor unmittelbar vor dem Zeichen angezeigt wird, das *index* angibt. Gibt eine leere Zeichenkette zurück.

***pathName* post**
Blendet die Auswahlliste ein.

pathName* setvalue *index
Setzt den Text der ComboBox auf den Wert, den *index* in der Werteliste angibt. *index* kann in jeder der folgenden Formen angegeben werden:

last
Legt das letzte Element der Werteliste fest.

first
Legt das erste Element der Werteliste fest.

next
Legt das Element fest, das auf das aktuelle folgt (das heißt das von **getvalue** zurückgegebene) in der Werteliste.

previous
Legt das Element fest, das dem aktuellen vorausgeht (das heißt das von **getvalue** zurückgegebene) in der Werteliste.

@*number*
Legt den ganzzahligen Index in der Werteliste fest.

***pathName* unpost**
Blendet die Auswahlliste aus.

BINDUNGEN

Wenn das Entry der ComboBox den Eingabefokus hat, hat es die folgenden Bindungen, zusätzlich zu den Vorgabebindungen von Entry:

- Die Taste "Bild auf" setzt den Wert der ComboBox auf den ersten Wert.

- Die Taste "Bild ab" setzt den Wert der ComboBox auf den letzten Wert.
- Die Taste "Pfeil nach oben" setzt den Wert der ComboBox auf den vorherigen Wert.
- Die Taste "Pfeil nach unten" setzt den Wert der ComboBox auf den nächsten Wert.
- Wenn **-autopost** eingeschaltet ist, blendet Escape die Listbox aus.

Wenn die Listbox nicht abgebildet ist und die ComboBox nicht bearbeitbar oder disabled ist, lässt Maustaste 1 auf dem Entry die Listbox aufklappen, als würde der ArrowButton gedrückt.

Dialog

Stichwörter: Dialog

NAME

Dialog - Dialogabstraktion mit eigenen Schaltflächen

ERZEUGUNG

Dialog *pathName* ?*option value...*?

OPTIONEN VON ButtonBox

-background oder -bg	-homogeneous
-padx	-pady
-spacing	

WIDGET-SPEZIFISCHE OPTIONEN

-anchor
-bitmap
-cancel
-default
-geometry
-image
-modal
-parent
-place
-separator
-side
-title
-transient

WIDGET-BEFEHL

pathName add ?*arg...*?

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

pathName draw ?*focus*?

pathName enddialog *result*

pathName getframe

pathName invoke *index*

pathName itemcget *index option*

pathName itemconfigure *index* ?*option*? ?*value option value* ...?

pathName setfocus *index*

pathName withdraw

BESCHREIBUNG

Das Widget Dialog ermöglicht, einen Dialog zu erzeugen. Einige Befehle nehmen einen *index* als Argument, der angibt, auf welchen Button sie wirken. Dieser Index ist derselbe, der für den entsprechenden Befehl von ButtonBox angegeben wird:

number
Bezeichnet den Button numerisch, wobei 0 dem zuerst hinzugefügten Button entspricht, 1 dem nächsten, und so weiter.

end or last
Bezeichnet den zuletzt hinzugefügten Eintrag.

default
Bezeichnet den Vorgabe-Button.

WIDGET-SPEZIFISCHE OPTIONEN

-anchor (nur lesbar)
Legt den Ankerpunkt der ButtonBox fest. Muss einer der Werte **w**, **e**, **n**, **s** oder **c** sein. Wenn die Option **side** auf *top* oder *bottom* gesetzt ist, haben die **anchor**-Werte *n*, *s* und *c* dieselbe Wirkung. Wenn die Option **side** auf *left* oder *right* gesetzt ist, haben die **anchor**-Werte *w*, *e* und *c* dieselbe Wirkung.

-bitmap (nur lesbar)
Legt eine Bitmap fest, die links vom Benutzerrahmen angezeigt wird. Die Option **image** setzt **bitmap** außer Kraft.

-cancel

Legt die Nummer der Abbrechen-Schaltfläche des Dialogs fest. Wenn Esc oder die Schließen-Schaltfläche des Fensters gedrückt wird, wird diese Schaltfläche aufgerufen. Wenn der Wert **-1** ist, ruft Esc nichts auf. Die Schließen-Schaltfläche des Fensters *zerstört* den Dialog in diesem Fall und gibt **-1** zurück.

-default
Legt die Nummer der Vorgabeschaltfläche des Dialogs fest. Wenn im Dialog Return gedrückt wird, wird diese Schaltfläche aufgerufen.

-geometry
Setzt die **geometry**, die verwendet wird, wenn die Methode **draw** aufgerufen wird. Die Beschreibung der Parameter steht bei **wm geometry**.

-image (nur lesbar)
Legt ein Bild fest, das links vom Benutzerrahmen angezeigt wird. Die Option **image** setzt **bitmap** außer Kraft.

-modal
Diese Option muss **none**, **local** oder **global** sein. Der Wert dieser Option legt den Grab-Modus des Dialogs fest und wie Dialog::**draw** arbeitet.

-parent
Elternteil des Dialogs. Der Dialog wird relativ zu seinem Elternteil platziert. Wenn der Wert leer ist, wird er relativ zum Wurzel-Fenster platziert. Siehe auch die Option **place**.

-place
Legt fest, wo das Dialog-Toplevel-Fenster relativ zum Elternteil des Dialogs gezeichnet wird. Muss einer der Werte **none**, **center**, **left**, **right**, **above**, **below** sein. Der Vorgabewert von **place** ist *center*.

-separator (nur lesbar)
Legt fest, ob zwischen dem Benutzerrahmen und der ButtonBox ein Trenner gezeichnet wird.

-side (nur lesbar)
Legt fest, wo die ButtonBox relativ zum Benutzerrahmen gezeichnet wird. Muss einer der Werte **top**, **left**, **bottom** oder **right** sein.

-title
Titel des Dialog-Toplevel-Fensters.

-transient (nur lesbar)
Legt fest, ob das Dialog-Toplevel-Fenster ein transientes Fenster sein soll. Der Vorgabewert von **transient** ist *true*.

WIDGET-BEFEHL

***pathName* add ?*arg...*?**
Fügt der Schaltflächenleiste des Dialogs eine Schaltfläche hinzu. Die Vorgabe der Option -command ist *Dialog::enddialog $path index*, wobei *index* die Nummer der hinzugefügten Schaltfläche ist.

pathName* cget *option
Gibt den aktuellen Wert der Konfigurationsoption zurück, die *option* angibt. *Option* kann jeden

Wert haben, den der Erzeugungsbefehl akzeptiert.

***pathName* configure ?*option*? ?*value option value ...*?**
Fragt die Konfigurationsoptionen des Widgets ab oder ändert sie. Wenn keine *option* angegeben ist, gibt der Befehl eine Liste zurück, die alle verfügbaren Optionen für *pathName* beschreibt. Wenn *option* ohne *value* angegeben ist, gibt der Befehl eine Liste zurück, die die genannte *option* beschreibt (diese Liste ist identisch mit der entsprechenden Teilliste des Wertes, der zurückgegeben wird, wenn keine *option* angegeben ist). Wenn ein oder mehrere Paare aus *option* und *value* angegeben sind, ändert der Befehl die genannten Widget-Optionen auf die genannten Werte; in diesem Fall gibt der Befehl eine leere Zeichenkette zurück. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert. Optionen, die nur gelesen werden können, werden nicht geändert.

***pathName* draw ?*focus*?**
Dieser Befehl zeichnet den Dialog und setzt den Grab darauf, entsprechend der Option **modal** . Wenn die Option **modal** auf *none* gesetzt ist, gibt der Befehl sofort eine leere Zeichenkette zurück. In allen anderen Fällen kehrt der Befehl zurück, wenn Dialog::**enddialog** aufgerufen oder der Dialog zerstört wird. Der Rückgabewert ist das Ergebnisargument von Dialog:: **enddialog** oder -1, wenn er zerstört wird.

In der Vorgabe wird der Fokus auf die Vorgabeschaltfläche gesetzt, auf die die Option **default** zeigt, oder auf das Toplevel des Dialogs, wenn keine Vorgabeschaltfläche gesetzt wurde. Wenn *focus* vorhanden ist, muss es ein Pfadname oder ein Index einer Schaltfläche sein. Der anfängliche Fokus wird auf diesen Pfadnamen oder die entsprechende Schaltfläche gesetzt.

pathName* enddialog *result
Dieser Befehl wird gewöhnlich innerhalb eines Befehls einer Schaltfläche aufgerufen, damit Dialog::**draw** zurückkehrt.

***pathName* getframe**
Gibt den Pfadnamen des Benutzerfensters zurück.

pathName* invoke *index
Ruft den Button auf, den *index* angibt.

pathName* itemcget *index option
Gibt den aktuellen Wert einer Konfigurationsoption für den Eintrag zurück. *Option* kann jeden Wert haben, den der Erzeugungsbefehl des Eintrags akzeptiert.

***pathName* itemconfigure *index* ?*option*? ?*value option value ...*?**
Dieser Befehl entspricht dem Befehl **configure**, er gilt aber für die Optionen eines einzelnen Eintrags, während **configure** für die Optionen des Widgets als Ganzes gilt. **Options** kann jeden Wert haben, den der Widget-Befehl zur Erzeugung des Eintrags akzeptiert. Wenn Optionen angegeben sind, werden sie geändert, wie der Befehl angibt, und der Befehl gibt eine leere Zeichenkette zurück. Wenn keine Optionen angegeben sind, gibt der Befehl eine Liste zurück, die die aktuellen Optionen des Eintrags beschreibt. Optionen, die nur gelesen werden können, werden nicht geändert.

pathName* setfocus *index
Setzt den Fokus auf den Button, den *index* angibt.

***pathName* withdraw**

Dieser Befehl verbirgt den Dialog.

DragSite

Stichwörter: DragSite

NAME

DragSite - Befehlsmenge für Ziehen

BEFEHL

DragSite::include *class type event*

DragSite::register *path* ?*option value...*?

DragSite::setdrag *path subpath initcmd endcmd* ?*force*?

BESCHREIBUNG

Die Befehle dieses Namespace ermöglichen, ein BWidget oder ein Tk-Widget als Ziehort zu definieren.

BEFEHL

DragSite::include *class type event*

Dieser Befehl bietet einen einfachen Weg, Optionen zu einem Ziehort in die Ressourcendefinition von BWidget aufzunehmen. Er nimmt die Optionen auf, die **register** braucht: *-dragevent*, initialisiert auf *event*, *-draginitcmd* und *-dragendcmd*, initialisiert auf die leere Zeichenkette, und zwei neue Optionen:

-dragenabled	Legt fest, ob Ziehen aktiv ist (initialisiert auf 0)

DragSite::register *path* ?*option value...*?

Dieser Befehl erklärt *path* zum Ziehort. Die Optionen sind:

-dragendcmd

Befehl, der aufgerufen wird, wenn das Ziehen endet (das heißt, wenn das Ziehsymbol losgelassen wird). Dieser Befehl wird mit den folgenden Argumenten aufgerufen:
- der Pfadname der Ziehquelle (das Widget selbst),
- der Pfadname des Ablegeziels,
- die Operation,
- der Typ der gezogenen Daten,
- die gezogenen Daten,
- das Ergebnis des Ablegens (das Ergebnis des Aufrufs von **-dropcmd** des Ziels),

Wenn das Ablegen nicht stattfindet, sind das Ziel und die Operation die leere Zeichenkette und das Ergebnis ist 0.

-dragevent

Legt die Nummer der Maustaste fest, die dem Ziehen zugeordnet ist. Muss **1**, **2** oder **3** sein.

-draginitcmd

Befehl, der aufgerufen wird, wenn das Ziehen beginnt. Wenn das Ereignis der Option **dragevent** auf *path* eintritt, wird dieser Befehl mit den folgenden Argumenten aufgerufen:
- Pfadname der Ziehquelle (*path*),
- X-Koordinate des Zeigers im Wurzelkoordinatensystem,
- Y-Koordinate des Zeigers im Wurzelkoordinatensystem,
- ein Toplevel-Fenster, das erzeugt wird, um die gezogenen Daten darzustellen. Wenn der Befehl zurückkehrt und es keine Kinder hat, wird automatisch eine Bitmap angezeigt.

Wenn der Befehl eine leere Zeichenkette zurückgibt, wird das Ziehen unterdrückt. Andernfalls muss der Befehl eine Liste mit drei Elementen zurückgeben:

- der Typ der Daten,
- die Liste der zulässigen Grundoperationen (**copy**, **move** und **link**)
- und die Daten.

Auch wenn **copy** nicht in der Liste der Grundoperationen steht, gilt es als zulässige Operation, weil die Bedeutung von **copy** die Ziehquelle nicht verändert.

DragSite::setdrag *path subpath initcmd endcmd ?force?*
Dieser Befehl bietet einen einfachen Weg, **register** während der Erzeugung oder Konfiguration eines BWidget aufzurufen.

- *path* ist der Pfadname des BWidget,
- *subpath* ist der Pfadname des Tk-Widgets, auf dem das Ziehereignis eintritt,
- *initcmd* BWidget-Befehl für das Ereignis *drag-init*,
- *endcmd* BWidget-Befehl für das Ereignis *drag-end*,
- *force* legt fest, ob **register** aufgerufen wird, auch wenn sich kein Optionswert geändert hat (0 als Vorgabe, für die Konfiguration eines BWidget; 1 für die Erzeugung eines BWidget).

setdrag prüft das Änderungskennzeichen der Optionen **dragenabled** und **dragevent** und ruft **register** auf, wenn das nach den Optionswerten und den Argumenten *initcmd* und *endcmd* nötig ist. **draginitcmd** und **dragendcmd** werden nicht aus den Optionen des Widgets genommen, weil sie als Benutzerbefehl gelten, den die BWidget-Umsetzung der Ereignisse *drag-init* und *drag-end* aufruft.

DropSite

Stichwörter: DropSite

NAME

DropSite - Befehlsmenge für Ablegen

BEFEHL

DropSite::include *class types*

DropSite::register *path* ?*option value...*?

DropSite::setcursor *cursor*

DropSite::setdrop *path subpath dropover drop* ?*force*?

DropSite::setoperation *op*

BESCHREIBUNG

Die Befehle dieses Namespace ermöglichen, ein BWidget oder ein Tk-Widget als Ablegeort zu definieren. Ein Ablegeort besteht aus dem Typ des Objekts, das abgelegt werden kann, und der zugehörigen Operation, einem Befehl, der aufgerufen wird, wenn das Ablegen geschieht, und einem Befehl, wenn ein Objekt über das Widget gezogen wird. Ein Ablegeort muss mindestens einen Typ eines zulässigen Objekts und einen Ablegebefehl haben.

BEFEHL

DropSite::include *class types*

Dieser Befehl bietet einen einfachen Weg, Optionen zu einem Ablegeort in die Ressourcendefinition von BWidget aufzunehmen. Er nimmt die Optionen auf, die **register** braucht, *-dropovercmd* und *-dropcmd*, initialisiert auf die leere Zeichenkette, und *-droptypes*, initialisiert auf *types*, und eine neue Option:

-dropenabled	Legt fest, ob Ablegen aktiv ist (initialisiert auf 0)

DropSite::register *path* ?*option value...*?

Dieser Befehl erklärt *path* zum Ablegeort. Die Optionen sind:

-dropcmd

Dieser Befehl wird aufgerufen, wenn das Ziehsymbol über einem gültigen Ablegeziel-Widget losgelassen wird. Die an den Befehl übergebenen Argumente sind:
- Pfadname des Ablegeziels (das Widget selbst),
- Pfadname der Ziehquelle,
- X-Koordinate des Zeigers im Wurzelkoordinatensystem,
- Y-Koordinate des Zeigers im Wurzelkoordinatensystem,
- Operation,
- Typ der gezogenen Daten,
- gezogene Daten.

Seine Rückgabewerte werden als Ergebnis an den Befehl **-dragendcmd** des Widgets der Ziehquelle übergeben.

-dropovercmd

Dieser Befehl kann verwendet werden, um während der Ereignisse *drag-over* ein dynamisches Ziehen bereitzustellen. Während eines Ziehens werden die Ereignisse <Enter>, <Motion> und <Leave> abgefangen und intern behandelt. Die an den Befehl übergebenen Argumente sind:
- Pfadname des Ablegeziels (das Widget selbst),
- Pfadname der Ziehquelle,
- Ereignis über dem Ablegeziel: *enter*, *motion* oder *leave*,
- X-Koordinate des Zeigers im Wurzelkoordinatensystem,
- Y-Koordinate des Zeigers im Wurzelkoordinatensystem,
- Operation,
- Typ der gezogenen Daten,

- gezogene Daten.

Der Befehl muss den neuen Zustand des Ziehens angeben:

- 0, wenn das Widget dieses Ziehen ablehnt. Der Befehl wird beim Ereignis motion/leave nicht erneut aufgerufen.
- 1, wenn das Widget dieses Ziehen annimmt. Der Befehl wird beim Ereignis motion/leave nicht erneut aufgerufen.
- 2, wenn das Widget dieses Ziehen ablehnt. Der Befehl wird bei jedem Ereignis motion erneut aufgerufen, um neu zu bewerten.
- 3, wenn das Widget dieses Ziehen annimmt. Der Befehl wird bei jedem Ereignis motion erneut aufgerufen, um neu zu bewerten.

Hier ist eine Liste der Ereignisse und der zugehörigen Aktionen an einem DropSite-Widget. Dieses Beispiel nimmt an, dass der Typ der gezogenen Daten für das Ablegeziel gültig ist. **status** ist der Zustand des Ziehens auf einem DropSite. Sein Wert ist:

Ereignis	Alter Zustand	Aktion	Neuer Zustand
<Enter>	-	wenn DropSite dropovercmd hat, mit enter aufrufen	Ergebnis von dropovercmd
sonst	1		
<Motion>	0 oder 1		unverändert
2 oder 3	dropovercmd mit motion aufrufen	Ergebnis von dropovercmd	
<Leave>	0 oder 1		-
2 oder 3	dropovercmd mit leave aufrufen	-	
<Drop>	0	dragendcmd der Ziehquelle aufrufen	-
1	dropcmd aufrufen und dragendcmd der Ziehquelle aufrufen		
2	dropovercmd mit leave aufrufen und dragendcmd der Ziehquelle aufrufen		
3	dropcmd aufrufen und dragendcmd der Ziehquelle aufrufen		

-droptypes

Legt eine Liste {*type oplist* ?*type oplist*? ...} der zulässigen Typen und der zugehörigen

Operationen für das Ablegeziel fest. Für jeden Typ ist *oplist* eine Liste {*descops mod* ?*descops mod*? ...}, die Operationen und Modifikatortasten für diese Operationen beschreibt. *descops* beschreibt eine Operation. Das kann eine vordefinierte Operation sein (**copy**, **move** oder **link**) oder eine neue, vom Benutzer definierte Operation der Form {*subop* **baseop** ?*bitmap*?}. *subop* ist der Name der Unteroperation, *baseop* ist der Name der Basisoperation (**copy**, **move** oder **link**) und *bitmap* ist eine Bitmap, die für die Operation angezeigt wird. Wenn *bitmap* leer ist, wird die Vorgabe-Bitmap der Basisoperation für die Unteroperation verwendet. *subop* kann eine Basisoperation sein, um die Bitmap einer Basisoperation zu ändern. In diesem Fall muss *baseop* leer oder gleich *subop* sein. *mod* ist die Modifikatortaste für die Operation. Sie kann sein:

- **none**, um anzugeben, dass keine Modifikatortaste gedrückt ist. Dieser Modifikator kann nur mit einer Unteroperation namens **default** verwendet werden (und umgekehrt), die keine Operations-Bitmap anzeigt. Für jeden Typ wird der Modifikator **none**, wenn er nicht angegeben ist, automatisch der Unteroperation **default** einer Basisoperation **copy** zugeordnet.
- **program**, um eine Unteroperation anzugeben, die nur über **DropSite::setoperation** erreichbar ist.
- Eine Liste aus **shift**, **control** und **alt**, die die entsprechende Taste bedeutet.

DropSite::setcursor *cursor*

Dieser Befehl kann innerhalb des Skripts **dragovercmd** verwendet werden. Er ist nützlich, um eine sichtbare Wirkung über den Zustand des Ziehens zu geben.

DropSite::setdrop *path subpath dropover drop ?force?*

Dieser Befehl bietet einen einfachen Weg, **register** während der Erzeugung oder Konfiguration eines BWidget aufzurufen.

- *path* ist der Pfadname des BWidget,
- *subpath* ist der Pfadname des Tk-Widgets, auf dem das Ziehereignis eintritt,
- *dropover* ist ein Befehl für das Ereignis *drag-over*,
- *drop* ist ein Befehl für das Ereignis *drop*,
- *force* legt fest, ob **register** aufgerufen wird, auch wenn sich kein Optionswert geändert hat (0 als Vorgabe, für die Konfiguration eines BWidget; 1 für die Erzeugung eines BWidget).

setdrop prüft das Änderungskennzeichen der Optionen **dropenabled** und **droptypes** und ruft **register** auf, wenn das nach den Optionswerten und den Argumenten *dropover* und *drop* nötig ist. **dropovercmd** und **dropcmd** werden nicht aus den Optionen des Widgets genommen, weil sie als Benutzerbefehl gelten, den die BWidget-Umsetzung der Ereignisse *drag-over* und *drop* aufruft.

DropSite::setoperation *op*

Beschreibungstext

DynamicHelp

Stichwörter: DynamicHelp

NAME

DynamicHelp - Hilfe für ein Tk-Widget oder ein BWidget bereitstellen

BEFEHL

DynamicHelp::add *widget* ?*option value ...*?

DynamicHelp::configure ?*option*? ?*value option value* ...?

DynamicHelp::delete *widget*

DynamicHelp::include *class type*

DynamicHelp::register *path type* ?*arg...*?

DynamicHelp::sethelp *path subpath* ?*force*?

BESCHREIBUNG

Beschreibungstext

BEFEHL

DynamicHelp::add ?*option value ...*?
Dieser Befehl fügt dem angegebenen *widget* dynamische Hilfe hinzu.

-command
Wenn angegeben, verweist das auf einen Befehl, der ausgeführt wird, um den anzuzeigenden Hilfetext zu holen. Der Befehl muss eine Zeichenkette zur Anzeige zurückgeben. Wenn der Befehl eine leere Zeichenkette zurückgibt, wird keine Hilfe angezeigt.

-index
Wenn angegeben, verweist das auf einen Menüindex, an den die Hilfe gebunden wird, anstelle eines Widgets. Wenn **-type** nicht menu ist, wird diese Option ignoriert.

-item
Wenn angegeben, verweist das auf ein Item oder ein Tag eines Canvas-Widgets oder auf ein Tag in einem Text-Widget, an das die Hilfe gebunden wird, anstelle eines Widgets.

-type
Legt den Typ der Hilfe fest. Kann sein: **balloon**, **variable** oder **menu**. Vorgabe ist **balloon**.

-text
Der Text, der als Hilfe angezeigt wird.

-variable
Legt einen Variablennamen fest, in den die Hilfezeichenkette geschrieben wird. Ein anderes Widget (zum Beispiel eine Statusleiste) ist dafür zuständig, diese Variable anzuzeigen.

Dynamische Hilfe für ein Menü entsteht in zwei Schritten. Zuerst muss das Menü selbst hinzugefügt werden, danach jeder Menüeintrag einzeln. Hier ist ein kurzes Beispiel.

```
    # Menü erzeugen
    menu .m -type menubar
    # Menüleiste dem Toplevel zuordnen, BEVOR DynamicHelp::register
    # damit es mit dem Klonnamen des Menüs funktioniert
    . configure -menu .m
```

```
    .m add cascade -label "File" -menu .m.file
    menu .m.file
    .m.file add command -label "Open..."
    .m.file add command -label "Quit"
    # Label für die Hilfe erzeugen, mit der Variable varinfo
    label .l -textvariable varinfo
    # alle Einträge des Menüs .m.file der Variable varinfo zuordnen
    DynamicHelp::add .m.file -type menu -variable varinfo
    # dann die Einträge von .m.file deklarieren
    DynamicHelp::add .m.file -type menu -index 0 -text "Detach menu"
    DynamicHelp::add .m.file -type menu -index 1 -text "Open a file"
    DynamicHelp::add .m.file -type menu -index 2 -text "Exit demo"
```

DynamicHelp::configure ?*option*? ?*value option value* ...?
Dieser Befehl konfiguriert die Ballonhilfe.

-borderwidth
Breite des schwarzen Randes um den Ballon.

-delay
Legt die Verzögerung in Millisekunden der Mausruhe fest, bevor der Ballon angezeigt wird.

-state
Legt einen von zwei Zuständen für Hilfeballons fest: **normal** und **disabled**.

Wenn **state disabled** ist, werden Hilfeballons für kein registriertes Widget angezeigt.

-topbackground
Die Hintergrundfarbe des Toplevel-Fensters, das für einen Ballon erzeugt wird.

Weitere Standardoptionen sind:

-background oder -bg	-font
-foreground oder -fg	-justify
-padx	-pady

DynamicHelp::delete *widget*
Löscht die gesamte dynamische Hilfe für das angegebene *widget*.

DynamicHelp::include *class type*
Beschreibungstext

DynamicHelp::register *path type* ?*arg...*?
Die Verwendung ist veraltet. Stattdessen DynamicHelp::add verwenden.

Registriert einen Hilfetext am Widget *path*. *type* bestimmt den Typ der Hilfe oder den Typ des Widgets. Je nach *type* müssen weitere Optionen angegeben werden.

type	options
balloon	?tagOrItem? text
variable	?tagOrItem? varName text
menu	varName
menuentry	index text

Wenn eine der Optionen fehlt oder leer ist, wird die Hilfe für dieses Widget entfernt.

Wenn *tagOrItem* angegeben ist, ist *path* ein canvas oder ein text. Bei einem canvas ist *tagOrItem* der Name eines Tags oder Items auf dem canvas, an das die Hilfe gebunden wird. Bei einem text ist *tagOrItem* der Name eines Tags auf dem text, an das die Hilfe gebunden wird.

Für einen Typ außer *balloon* ist *varName* gewöhnlich eine Variable, die mit einem Label verknüpft ist.
Für ein Menü gibt es keine Hilfe vom Typ balloon. Um eine Hilfe für ein Menü zu deklarieren, wird zuerst das Menü deklariert und danach die Einträge dieses Menüs.
Zum Beispiel:

Menü erzeugen menu .m -type menubar # Menüleiste dem Toplevel zuordnen, BEVOR DynamicHelp::register # damit es mit dem Klonnamen des Menüs funktioniert. configure -menu .m .m add cascade -label "File" -menu .m.file menu .m.file .m.file add command -label "Open..." .m.file add command -label "Quit" # Label für die Hilfe erzeugen, mit der Variable varinfo label .l -textvariable varinfo # alle Einträge des Menüs .m.file der Variable varinfo zuordnen DynamicHelp::register .m.file menu varinfo # dann die Einträge von .m.file deklarieren DynamicHelp::register .m.file menuentry 0 "Detach menu" DynamicHelp::register .m.file menuentry 1 "Open a file" DynamicHelp::register .m.file menuentry 2 "Exit demo"

Wenn ein Popup-Menü zu einer Menüleiste gehört, muss die Menüleiste zuerst ihrem Toplevel zugeordnet werden. In diesem Fall wird beim Erzeugen eines Menü-Popups auch sein Klonfenster erzeugt, und DynamicHelp::register erkennt das Vorhandensein des Klonfensters und bildet die Ereignisse darauf ab.

DynamicHelp::sethelp *path subpath ?force?*
Beschreibungstext

Entry

Stichwörter: Entry

NAME

Entry - Entry-Widget mit der Option **state**, dynamischer Hilfe und Ziehen und Ablegen

ERZEUGUNG

Entry *pathName* ?*option value...*?

STANDARDOPTIONEN

-background oder -bg	-borderwidth oder -bd

-disabledbackground	-disabledforeground
-exportselection	-font
-foreground oder -fg	-highlightbackground
-highlightcolor	-highlightthickness
-insertbackground	-insertborderwidth
-insertofftime	-insertontime
-insertwidth	-justify
-relief	-selectbackground
-selectborderwidth	-selectforeground
-takefocus	-text
-textvariable	-xscrollcommand

WIDGET-SPEZIFISCHE OPTIONEN

-command	-dragenabled
-dragendcmd	-dragevent
-draginitcmd	-dragtype
-dropcmd	-dropenabled
-dropovercmd	-droptypes
-editable	-helptext
-helptype	-helpvar
-show	-state
-width	

WIDGET-BEFEHL

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

pathName invoke

BESCHREIBUNG

Das Widget **Entry** erweitert das Tk-Entry der Vorgabe. Hinzugekommen sind Optionen für eine sichtbare Wirkung je nach Zustand des Entry, Optionen von DynamicHelp sowie Ziehen und

Ablegen. Entry verhält sich weitgehend wie ein Label, mit der Option **text**, die den Inhalt setzt. Der Tk-Befehl entry kann auch auf dem Entry-Widget verwendet werden.

WIDGET-SPEZIFISCHE OPTIONEN

-command
Legt einen Befehl fest, der aufgerufen wird, wenn im Entry <Return> gedrückt wird.

-dragenabled
Ein boolescher Wert, der angibt, ob Ziehen eingeschaltet ist.

-dragendcmd
Legt einen Befehl fest, der aufgerufen wird, wenn das Ziehen endet. **dragendcmd** muss ein Befehl sein, der der Beschreibung der Option **dragendcmd** von **DragSite::register** entspricht. Wenn **dragendcmd** leer ist, aktualisiert der interne Befehl *dragend* das Entry entsprechend der Operation (**move** oder **copy**) und den gezogenen Daten (das ganze Entry oder der ausgewählte Teil).

-dragevent
Legt die Nummer der Maustaste fest, die dem Ziehen zugeordnet ist. Muss **1**, **2** oder **3** sein.

-draginitcmd
Legt einen Befehl fest, der aufgerufen wird, wenn **dragevent** auf dem Widget eintritt. **draginitcmd** muss ein Befehl sein, der der Beschreibung der Option **draginitcmd** von **DragSite::register** entspricht. Wenn **draginitcmd** leer ist, lehnt der Befehl das Ziehen ab, wenn das Entry leer ist oder wenn ein Textabschnitt ausgewählt ist und das Ereignis nicht über der Auswahl eintritt. In allen anderen Fällen gibt der Befehl zurück:

- als Datentyp den Wert der Option **dragtype** oder *TEXT*, wenn er leer ist,
- als Operationen *{copy move}*, wenn **state** normal und **editable** true ist, oder nur *{copy}* in den anderen Fällen,
- als Daten den gesamten Inhalt oder den ausgewählten Teil des Entry.

-dragtype
Legt einen anderen Typ des gezogenen Objekts fest.

-dropcmd
Entry hat einen Befehlsrahmen für Ereignisse *drop*. Dieser Befehl beendet das automatische Scrollen und ermittelt die aktuelle Position. Wenn **dropcmd** nicht leer ist, wird er mit den folgenden Argumenten aufgerufen:

- der Pfadname des Entry,
- der Pfadname der Ziehquelle,
- der numerische Index im Entry, den der Cursor bezeichnet,
- die aktuelle Operation,
- der Datentyp,
- die Daten.

und muss einen Wert zurückgeben, der der Option **dropcmd** entspricht, die in **DropSite::register** beschrieben ist. Wenn **dropcmd** leer ist, aktualisiert der Rahmen das Entry entsprechend dem Typ der Daten:

COLOR oder FGCOLOR	konfiguriert den foreground des Entry neu
BGCOLOR	konfiguriert den background des Entry neu
TEXT, oder jedes andere Tag	konfiguriert das Entry neu, damit es die zugehörige Zeichenkette anzeigt.

und gibt 1 zurück.

-dropenabled
Ein boolescher Wert, der angibt, ob Ablegen eingeschaltet ist.

-dropovercmd
Entry hat einen Befehlsrahmen für Ereignisse *drag-over*. Dieser Befehl schaltet während des *drag-over* das automatische Scrollen und die Ermittlung der Position ein. Wenn **dropovercmd** leer ist, nimmt der Rahmen das Ablegen an, wenn die Option **editable** true und die Option **state** normal ist. Wenn **dropovercmd** nicht leer ist, wird er mit den folgenden Argumenten aufgerufen:

- der Pfadname des Entry,
- der Pfadname der Ziehquelle,
- das Ereignis,
- der numerische Index im Entry, den der Cursor bezeichnet,
- die aktuelle Operation,
- der Datentyp,
- die Daten.

und muss einen Wert zurückgeben, der der Option **dropovercmd** entspricht, die in **DropSite::register** beschrieben ist.

-droptypes
Legt eine Liste der akzeptierten abgelegten Objekte und Operationen fest. Weitere Angaben stehen bei der Option **droptypes** von **DropSite::register**. Die Vorgabe akzeptiert *FGCOLOR*, *COLOR*, *BGCOLOR* und *TEXT*, jeweils mit den Operationen **copy** und **move**.

-editable
Legt fest, ob der Benutzer das Entry bearbeiten kann. Entspricht der Option **state** des Tk-Widgets entry.

-helptext
Text für die dynamische Hilfe. Wenn der Wert leer ist, gibt es für dieses Widget keine Hilfe. Siehe auch DynamicHelp.

-helptype
Typ der dynamischen Hilfe. *balloon* oder *variable* verwenden. Siehe auch DynamicHelp.

-helpvar
Variable, die verwendet wird, wenn die Option **helptype** *variable* ist. Siehe auch DynamicHelp.

-show
Wenn diese Option angegeben ist, wird der wirkliche Inhalt des Entry nicht im Fenster

angezeigt. Stattdessen wird jedes Zeichen des Wertes des Entry als das erste Zeichen des Wertes dieser Option angezeigt, wie ``*''. Das ist nützlich, wenn das Entry zum Beispiel für ein Passwort verwendet wird. Wenn Zeichen im Entry ausgewählt und an eine andere Stelle kopiert werden, ist die kopierte Information das Angezeigte, nicht der wirkliche Inhalt des Entry.

-state
Legt einen von zwei Zuständen für das Entry fest: **normal** oder **disabled**. Im Zustand normal wird der Text des Entry mit der Option **foreground** angezeigt. Im Zustand disabled wird der Text des Entry mit der Option **disabledforeground** angezeigt. Wenn das Entry disabled ist, kann der Wert nicht durch Benutzereingabe geändert werden, und es wird kein Einfügecursor angezeigt, auch wenn der Eingabefokus im Widget liegt. Der Zustand disabled ist dasselbe wie nicht bearbeitbar, mit sichtbarer Wirkung.

-width
Legt eine ganze Zahl fest, die die gewünschte Breite des Entry-Fensters in Zeichen mittlerer Größe der Schriftart des Widgets angibt. Wenn der Wert kleiner oder gleich null ist, wählt das Widget eine Größe, die gerade für den aktuellen Text reicht.

WIDGET-BEFEHL

pathName* cget *option
Gibt den aktuellen Wert der Konfigurationsoption zurück, die *option* angibt. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert.

***pathName* configure ?*option*? ?*value option value* ...?**
Fragt die Konfigurationsoptionen des Widgets ab oder ändert sie. Wenn keine *option* angegeben ist, gibt der Befehl eine Liste zurück, die alle verfügbaren Optionen für *pathName* beschreibt. Wenn *option* ohne *value* angegeben ist, gibt der Befehl eine Liste zurück, die die genannte *option* beschreibt (diese Liste ist identisch mit der entsprechenden Teilliste des Wertes, der zurückgegeben wird, wenn keine *option* angegeben ist). Wenn ein oder mehrere Paare aus *option* und *value* angegeben sind, ändert der Befehl die genannten Widget-Optionen auf die genannten Werte; in diesem Fall gibt der Befehl eine leere Zeichenkette zurück. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert. Optionen, die nur gelesen werden können, werden nicht geändert.

***pathName* invoke**
Ruft den Befehl auf, den die Option **-command** angibt.

Label

Stichwörter: Label

NAME
Label - Label-Widget mit der Option **state**, dynamischer Hilfe und Ziehen und Ablegen

ERZEUGUNG
Label *pathName* ?*option value...*?

STANDARDOPTIONEN

-anchor
-background oder -bg
-bitmap
-borderwidth oder -bd
-cursor
-disabledforeground
-font
-foreground oder -fg
-highlightbackground
-highlightcolor
-highlightthickness
-image
-justify
-padx
-pady
-relief
-takefocus
-text
-textvariable
-wraplength

WIDGET-SPEZIFISCHE OPTIONEN

-dragenabled
-dragendcmd
-dragevent
-draginitcmd
-dragtype
-dropcmd
-dropenabled

-dropovercmd
-droptypes
-focus
-height
-helptext
-helptype
-helpvar
-name
-state
-underline
-width

WIDGET-BEFEHL

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

pathName setfocus

BESCHREIBUNG

Das Widget **Label** erweitert das Tk-Label der Vorgabe. Hinzugekommen sind Optionen für eine sichtbare Wirkung je nach Zustand des Labels, Optionen von DynamicHelp sowie Ziehen und Ablegen.

WIDGET-SPEZIFISCHE OPTIONEN

-dragenabled
Ein boolescher Wert, der angibt, ob Ziehen eingeschaltet ist.

-dragendcmd
Legt einen Befehl fest, der aufgerufen wird, wenn das Ziehen endet. **dragendcmd** muss ein Befehl sein, der der Beschreibung der Option **dragendcmd** von **DragSite::register** entspricht.

-dragevent
Legt die Nummer der Maustaste fest, die dem Ziehen zugeordnet ist. Muss **1**, **2** oder **3** sein.

-draginitcmd
Legt einen Befehl fest, der aufgerufen wird, wenn **dragevent** auf dem Widget eintritt. **draginitcmd** muss ein Befehl sein, der der Beschreibung der Option **draginitcmd** von **DragSite::register** entspricht. Wenn **draginitcmd** leer ist, wird stattdessen der interne

Befehl **draginitcmd** verwendet und gibt zurück:

IMAGE {copy} <image name>	wenn ein Bild angezeigt wird.

Wenn die Option **dragtype** nicht leer ist, wird ihr Wert anstelle der oben genannten verwendet.

-dragtype
Legt einen anderen Typ des gezogenen Objekts fest.

-dropcmd
Legt einen Befehl fest, der aufgerufen wird, wenn auf dem Widget abgelegt wird. **dropcmd** muss ein Befehl sein, der der Beschreibung der Option **dropcmd** von **DropSite::register** entspricht. Wenn **dropcmd** leer ist, aktualisiert der Befehl das Label entsprechend dem Typ der Daten:

COLOR oder FGCOLOR	konfiguriert den foreground des Labels neu.
BGCOLOR	konfiguriert den background des Labels neu.
IMAGE	konfiguriert das Label neu, damit es das zugehörige Bild anzeigt.
BITMAP	konfiguriert das Label neu, damit es die zugehörige Bitmap anzeigt. Die Option image wird auf leer gesetzt.
TEXT, oder jedes andere Tag	konfiguriert das Label neu, damit es die zugehörige Zeichenkette anzeigt. Die Optionen image und bitmap werden auf leer gesetzt.

und gibt 1 zurück.

-dropenabled
Ein boolescher Wert, der angibt, ob Ablegen eingeschaltet ist.

-dropovercmd
Legt einen Befehl fest, der aufgerufen wird, wenn das Ziehsymbol über dem Widget ist. **dropovercmd** muss ein Befehl sein, der der Beschreibung der Option **dropovercmd** von **DropSite::register** entspricht. Wenn **dropovercmd** leer ist, nimmt Label das Ablegen immer an, wenn der Datentyp *FGCOLOR*, *COLOR* oder *BGCOLOR* ist, und nimmt jeden anderen Datentyp nur an, wenn **state** normal ist.

-droptypes
Legt eine Liste der akzeptierten abgelegten Objekte und Operationen fest. Weitere Angaben stehen bei der Option **droptypes** von **DropSite::register**. Die Vorgabe akzeptiert *FGCOLOR*, *COLOR*, *BGCOLOR*, *TEXT*, *BITMAP* und *IMAGE*, jeweils mit den Operationen **copy** und **move**.

-focus
Legt einen Pfadnamen fest, auf den der Befehl Label::**setfocus** den Fokus setzt.

-height

Legt eine gewünschte Höhe für das Label fest. Wenn im Label ein Bild oder eine Bitmap angezeigt wird, ist der Wert in Bildschirmeinheiten, bei Text in Textzeilen. Wenn diese Option nicht angegeben ist, wird die gewünschte Höhe des Labels aus der Größe des angezeigten Bildes, der Bitmap oder des Textes berechnet.

-helptext
Text für die dynamische Hilfe. Wenn der Wert leer ist, gibt es für dieses Widget keine Hilfe. Siehe auch DynamicHelp.

-helptype
Typ der dynamischen Hilfe. *balloon* oder *variable* verwenden. Siehe auch DynamicHelp.

-helpvar
Variable, die verwendet wird, wenn die Option **helptype** *variable* ist. Siehe auch DynamicHelp.

-name
Legt einen Standardnamen für das Label fest. Wenn die Option ***nameName** in der Ressourcendatenbank gefunden wird, werden die Optionen **text** und **underline** aus ihrem Wert entnommen.

-state
Legt einen von zwei Zuständen für das Label fest: **normal** oder **disabled**. Im Zustand normal wird der Text des Labels mit der Option **foreground** angezeigt. Im Zustand disabled wird der Text des Labels mit der Option **disabledforeground** angezeigt.

-underline
Legt den ganzzahligen Index eines Zeichens fest, das im Label unterstrichen wird. 0 entspricht dem ersten Zeichen des angezeigten Textes, 1 dem nächsten Zeichen, und so weiter. Die Bindung **<Alt-char>** wird auf dem Toplevel des Labels automatisch gesetzt, um Label::**setfocus** aufzurufen.

-width
Legt eine gewünschte Breite für das Label fest. Wenn im Label ein Bild oder eine Bitmap angezeigt wird, ist der Wert in Bildschirmeinheiten, bei Text in Zeichen. Wenn diese Option nicht angegeben ist, wird die gewünschte Breite des Labels aus der Größe des angezeigten Bildes, der Bitmap oder des Textes berechnet.

WIDGET-BEFEHL

pathName* cget *option
Gibt den aktuellen Wert der Konfigurationsoption zurück, die *option* angibt. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert.

***pathName* configure ?*option*? ?*value option value ...*?**
Fragt die Konfigurationsoptionen des Widgets ab oder ändert sie. Wenn keine *option* angegeben ist, gibt der Befehl eine Liste zurück, die alle verfügbaren Optionen für *pathName* beschreibt. Wenn *option* ohne *value* angegeben ist, gibt der Befehl eine Liste zurück, die die genannte *option* beschreibt (diese Liste ist identisch mit der entsprechenden Teilliste des Wertes, der zurückgegeben wird, wenn keine *option* angegeben ist). Wenn ein oder mehrere Paare aus *option* und *value* angegeben sind, ändert der Befehl die genannten

Widget-Optionen auf die genannten Werte; in diesem Fall gibt der Befehl eine leere Zeichenkette zurück. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert. Optionen, die nur gelesen werden können, werden nicht geändert.

***pathName* setfocus**
Setzt den Fokus auf den Pfadnamen der Option **-focus**, wenn **-state** *normal* ist.

LabelEntry

Stichwörter: LabelEntry

NAME
LabelEntry - LabelFrame, das ein Entry-Widget enthält.

ERZEUGUNG
LabelEntry *pathName* ?*option value...*?

OPTIONEN VON Entry

-borderwidth oder -bd	-command
-disabledforeground	-dragenabled
-dragendcmd	-dragevent
-draginitcmd	-dragtype
-dropcmd	-dropenabled
-dropovercmd	-droptypes
-editable	-entrybg (siehe -background)
-entryfg (siehe -foreground)	-exportselection
-font	-helptext
-helptype	-helpvar
-highlightbackground	-highlightcolor
-highlightthickness	-insertbackground
-insertborderwidth	-insertofftime
-insertontime	-insertwidth
-justify	-relief
-selectbackground	-selectborderwidth
-selectforeground	-show
-state	-takefocus
-text	-textvariable

-width	-xscrollcommand

OPTIONEN VON LabelFrame

-background oder -bg	-disabledforeground
-foreground oder -fg	-helptext
-helptype	-helpvar
-label (siehe -text)	-labelanchor (siehe -anchor)
-labelfont (siehe -font)	-labelheight (siehe -height)
-labeljustify (siehe -justify)	-labelwidth (siehe -width)
-name	-padx
-pady	-side
-state	-underline
-wraplength	

WIDGET-BEFEHL

pathName bind ?*arg...*?

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

BESCHREIBUNG

LabelEntry ist ein Widget aus einem LabelFrame-Widget, das ein Entry-Widget enthält. Der Tk-Befehl entry kann auch auf dem LabelEntry-Widget verwendet werden.

WIDGET-BEFEHL

***pathName* bind ?*arg...*?**

Setzt Bindungen auf dem Entry-Widget.

pathName* cget *option

Gibt den aktuellen Wert der Konfigurationsoption zurück, die *option* angibt. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert.

***pathName* configure ?*option*? ?*value option value* ...?**

Fragt die Konfigurationsoptionen des Widgets ab oder ändert sie. Wenn keine *option* angegeben ist, gibt der Befehl eine Liste zurück, die alle verfügbaren Optionen für *pathName* beschreibt. Wenn *option* ohne *value* angegeben ist, gibt der Befehl eine Liste zurück, die die genannte *option* beschreibt (diese Liste ist identisch mit der entsprechenden Teilliste des

Wertes, der zurückgegeben wird, wenn keine *option* angegeben ist). Wenn ein oder mehrere Paare aus *option* und *value* angegeben sind, ändert der Befehl die genannten Widget-Optionen auf die genannten Werte; in diesem Fall gibt der Befehl eine leere Zeichenkette zurück. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert. Optionen, die nur gelesen werden können, werden nicht geändert.

LabelFrame

Stichwörter: LabelFrame

NAME

LabelFrame - Rahmen mit einem Label

ERZEUGUNG

LabelFrame *pathName* ?*option value...*?

STANDARDOPTIONEN

-borderwidth oder -bd	-relief

OPTIONEN VON Label

-anchor	-background oder -bg
-bitmap	-disabledforeground
-focus	-font
-foreground oder -fg	-height
-helptext	-helptype
-helpvar	-image
-justify	-name
-padx	-pady
-state	-text
-textvariable	-underline
-width	-wraplength

WIDGET-SPEZIFISCHE OPTIONEN

-side

WIDGET-BEFEHL

LabelFrame::align ?*arg...*?

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

pathName getframe

BESCHREIBUNG

LabelFrame ermöglicht, einen Rahmen mit einem Label zu erzeugen, das an jeder Seite angeordnet ist. LabelFrame wird von ComboBox und SpinBox verwendet.

WIDGET-SPEZIFISCHE OPTIONEN

-side (nur lesbar)
Legt fest, wo das Label relativ zum Benutzerrahmen angeordnet wird: **top**, **bottom**, **left** oder **right**.

WIDGET-BEFEHL

LabelFrame::align ?*arg...*?
Dieser Befehl richtet das Label aller Widgets aus, die *args* angibt und die die Klasse LabelFrame (oder "derived") haben, indem er ihre Breite auf das Maximum plus 1 setzt

pathName* cget *option
Gibt den aktuellen Wert der Konfigurationsoption zurück, die *option* angibt. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert.

***pathName* configure ?*option*? ?*value option value ...*?**
Fragt die Konfigurationsoptionen des Widgets ab oder ändert sie. Wenn keine *option* angegeben ist, gibt der Befehl eine Liste zurück, die alle verfügbaren Optionen für *pathName* beschreibt. Wenn *option* ohne *value* angegeben ist, gibt der Befehl eine Liste zurück, die die genannte *option* beschreibt (diese Liste ist identisch mit der entsprechenden Teilliste des Wertes, der zurückgegeben wird, wenn keine *option* angegeben ist). Wenn ein oder mehrere Paare aus *option* und *value* angegeben sind, ändert der Befehl die genannten Widget-Optionen auf die genannten Werte; in diesem Fall gibt der Befehl eine leere Zeichenkette zurück. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert. Optionen, die nur gelesen werden können, werden nicht geändert.

***pathName* getframe**
Gibt den Rahmen zurück, in dem jedes andere Widget erzeugt werden kann.

ListBox

Stichwörter: ListBox

NAME
ListBox - ListBox-Widget

ERZEUGUNG
ListBox *pathName* ?*option value...*?

STANDARDOPTIONEN

-background oder -bg	-borderwidth oder -bd
-cursor	-highlightbackground
-highlightcolor	-highlightthickness
-relief	-selectbackground
-selectforeground	-takefocus
-xscrollcommand	-yscrollcommand

WIDGET-SPEZIFISCHE OPTIONEN

-autofocus	-deltax	-deltay
-dragenabled	-dragendcmd	
-dragevent	-draginitcmd	
-dragtype	-dropcmd	
-dropenabled	-dropovercmd	
-dropovermode	-droptypes	
-height	-multicolumn	
-padx	-redraw	
-selectfill	-selectmode	-width

WIDGET-BEFEHL

pathName bindImage *event script*

pathName bindText *event script*

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

pathName delete ?*arg...*?

pathName edit *item text* ?*verifycmd*? ?*clickres*? ?*select*?

pathName exists *item*

pathName index *item*

pathName insert *index item* ?*option value...*?

pathName item *first* ?*last*?

pathName itemcget *item option*

pathName itemconfigure *item* ?*option*? ?*value option value* ...?

pathName items ?*first*? ?*last*?

pathName move *item index*

pathName reorder *neworder*

pathName see *item*

pathName selection *cmd* ?*arg...*?

pathName xview ?*arg...*?

pathName yview ?*arg...*?

BESCHREIBUNG

Das Widget **ListBox** verwendet canvas, um eine Liste von Einträgen anzuzeigen. Jeder Eintrag besteht aus einer Beschriftung mit eigener Schriftart und eigenen Vordergrundattributen und einem optionalen Bild oder Fenster. Jeder Eintrag wird in einer einzigen Zeile gezeichnet, deren Höhe die Option **deltay** festlegt, sie dürfen also höchstens diese Höhe haben. Ein Eintrag wird eindeutig durch eine Zeichenkette gekennzeichnet, die bei der Erzeugung (durch den Befehl **insert**) angegeben wird. Die ListBox kann eine oder mehrere Spalten haben, abhängig von der Option **multicolumn**. Die Spalten werden nicht selbst verwaltet; die Anzahl der Spalten ergibt sich aus der Höhe der ListBox, damit jeder Eintrag senkrecht sichtbar ist.

WIDGET-SPEZIFISCHE OPTIONEN

-autofocus
Wenn diese Option true ist, nimmt die Listbox den Fokus, sobald in sie geklickt wird. Ohne Fokus arbeiten die Mausrad-Bindungen der Listbox nicht richtig. Die Vorgabe ist true.

-deltax
Legt den waagerechten Abstand zwischen den Spalten fest.

-deltay
Legt die senkrechte Größe der Einträge fest.

-dragenabled
Ein boolescher Wert, der angibt, ob Ziehen eingeschaltet ist.

-dragendcmd
Legt einen Befehl fest, der aufgerufen wird, wenn das Ziehen endet. **dragendcmd** muss ein

Befehl sein, der der Beschreibung der Option **dragendcmd** von **DragSite::register** entspricht.

-dragevent
Legt die Nummer der Maustaste fest, die dem Ziehen zugeordnet ist. Muss **1**, **2** oder **3** sein.

-draginitcmd
ListBox hat einen Befehlsrahmen für Ereignisse *drag-init*. Dieser Befehl lehnt das Ziehen ab, wenn kein Eintrag bezeichnet ist. In den anderen Fällen: Wenn **draginitcmd** leer ist, gibt er zurück:

- als Datentyp den Wert der Option **dragtype** oder *LISTBOX_ITEM*, wenn er leer ist,
- *{move copy link}* als die Operationen,
- den Bezeichner des Eintrags als die Daten.

Wenn **draginitcmd** nicht leer ist, wird er mit den folgenden Argumenten aufgerufen:

- der Pfadname der Listbox,
- der Bezeichner des gezogenen Eintrags,
- das Toplevel-Fenster, das erzeugt wird, um die gezogenen Daten darzustellen.

und muss einen Wert zurückgeben, der der Option **draginitcmd** entspricht, die in **DragSite::register** beschrieben ist.

-dragtype
Legt einen anderen Typ des gezogenen Objekts fest.

-dropcmd
ListBox hat einen Befehlsrahmen für Ereignisse *drop*. Dieser Befehl beendet das automatische Scrollen und ermittelt Eintrag und Position. Wenn **dropcmd** nicht leer ist, wird er mit den folgenden Argumenten aufgerufen:

- der Pfadname der Listbox,
- der Pfadname der Ziehquelle,
- eine Liste, die beschreibt, wo das Ablegen geschieht. Sie kann sein:
- die aktuelle Operation,
- der Datentyp,
- die Daten.
  - *{***widget***}*,
  - *{***item** *item}* oder
  - *{***position** *index}*.

Der Vorgabe-Ablegebefehl erlaubt Ziehen und Ablegen innerhalb der Listbox, aber nicht zu anderen Widgets oder von ihnen.

-dropenabled
Ein boolescher Wert, der angibt, ob Ablegen eingeschaltet ist.

-dropovercmd
LsitBox hat einen Befehlsrahmen für Ereignisse *drag-over*. Dieser Befehl schaltet während des *drag-over* das automatische Scrollen und die Ermittlung der Position ein. Wenn **dropovercmd** nicht leer ist, wird der Befehl mit den folgenden Argumenten aufgerufen:

- der Pfadname der Listbox,
- der Pfadname der Ziehquelle,

- eine Liste, die beschreibt, wo das Ablegen geschehen kann; ihre Elemente sind:
- die aktuelle Operation,
- der Datentyp,
- die Daten.
  - die Zeichenkette *widget*, wenn die Option **dropovertype** *w* enthält, sonst die leere Zeichenkette.
  - der angezielte Eintrag, wenn das Ziehsymbol auf einen Eintrag zeigt und die Option **dropovertype** *i* enthält, sonst die leere Zeichenkette.
  - ein Index zwischen zwei Einträgen, auf den das Ziehsymbol zeigt, wenn die Option **dropovertype** *p* enthält, sonst die leere Zeichenkette.
  - optional die bevorzugte Methode, wenn das Ablegen sowohl innerhalb eines Eintrags als auch zwischen zwei Einträgen geschehen kann. Der Wert ist *position* oder *item*.

Der Befehl muss eine Liste mit zwei Elementen zurückgeben:

- den Ablegezustand, entsprechend der Beschreibung in der Option **dropovercmd** von **DropSite::register**,
- die gewählte Methode: *widget*, *item* oder *position*.

-dropovermode
Legt den Typ der Wechselwirkung *drop-over* fest. Muss eine Kombination aus **w** sein, das festlegt, dass das Ablegen überall auf dem Widget geschehen kann, **p**, das festlegt, dass das Ablegen zwischen zwei Einträgen geschehen kann, und **i**, das festlegt, dass das Ablegen innerhalb von Einträgen geschieht.

-droptypes
Legt eine Liste der akzeptierten abgelegten Objekte und Operationen fest. Weitere Angaben stehen bei der Option **droptypes** von **DropSite::register**. Die Vorgabe ist *LISTBOX_ITEM* mit den Operationen **copy** und **move**.

-height
Legt die gewünschte Höhe der Listbox in Einheiten von **deltay** Pixeln fest.

-multicolumn
Legt fest, ob die ListBox die Einträge so anordnet, dass jeder senkrecht sichtbar ist.

-padx
Legt den Abstand zwischen Bild oder Fenster und dem Text der Einträge fest.

-redraw
Legt fest, ob die Listbox neu gezeichnet werden soll, wenn der Leerlauf beginnt. Auf false setzen, wenn **update** aufgerufen wird, während die Listbox geändert wird.

-selectfill
Wenn true, zeichnet die Listbox ein Auswahlrechteck, das die Listbox von links nach rechts füllt, anstatt nur einen Rahmen um den ausgewählten Eintrag zu zeichnen. Das entspricht eher der Tk-Listbox der Vorgabe.

-selectmode
Legt den gewünschten Auswahlmodus der Listbox fest. Muss einer der Werte **none**, **single** oder **multiple** sein. **selectmode** *single* erlaubt, 1 Eintrag über seinen Text oder sein Bild auszuwählen. **selectmode** *multiple* erlaubt, mehrere Einträge über ihren Text oder ihr Bild

auszuwählen. Weitere Angaben zu den Auswahlmodi *single* oder *multiple* stehen beim Tk-Befehl **listbox**. Der Vorgabewert für **selectmode** ist *none*.

-width
Legt die gewünschte Breite der Listbox in Einheiten von 8 Pixeln fest.

WIDGET-BEFEHL

pathName* bindImage *event script
Dieser Befehl ordnet einen Befehl zu, der ausgeführt wird, wenn die von *event* angegebene Ereignisfolge auf dem Bild eines Eintrags eintritt. Der Eintragsbezeichner, auf dem das Ereignis eintritt, wird an den Befehl angehängt.

Jedes Vorkommen von **%W** in *script* wird durch den Pfad der Listbox ersetzt.

pathName* bindText *event script
Dieser Befehl ordnet einen Befehl zu, der ausgeführt wird, wenn die von *event* angegebene Ereignisfolge auf der Beschriftung eines Eintrags eintritt. Der Eintragsbezeichner, auf dem das Ereignis eintritt, wird an den Befehl angehängt.

Jedes Vorkommen von **%W** in *script* wird durch den Pfad der Listbox ersetzt.

pathName* cget *option
Gibt den aktuellen Wert der Konfigurationsoption zurück, die *option* angibt. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert.

***pathName* configure ?*option*? ?*value option value ...*?**
Fragt die Konfigurationsoptionen des Widgets ab oder ändert sie. Wenn keine *option* angegeben ist, gibt der Befehl eine Liste zurück, die alle verfügbaren Optionen für *pathName* beschreibt. Wenn *option* ohne *value* angegeben ist, gibt der Befehl eine Liste zurück, die die genannte *option* beschreibt (diese Liste ist identisch mit der entsprechenden Teilliste des Wertes, der zurückgegeben wird, wenn keine *option* angegeben ist). Wenn ein oder mehrere Paare aus *option* und *value* angegeben sind, ändert der Befehl die genannten Widget-Optionen auf die genannten Werte; in diesem Fall gibt der Befehl eine leere Zeichenkette zurück. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert. Optionen, die nur gelesen werden können, werden nicht geändert.

***pathName* delete ?*arg...*?**
Löscht alle Einträge in *arg*. *arg* kann eine Liste von Einträgen oder eine Liste von Listen von Einträgen sein. Um alle Einträge zu löschen, dient *$pathName delete [$pathName items]*.

***pathName* edit *item text* ?*verifycmd*? ?*clickres*? ?*select*?**
Bietet einen Weg, die Beschriftung eines Eintrags an Ort und Stelle zu bearbeiten. Der Befehl nimmt den Anfangstext als Argument und ändert die Beschriftung des bearbeiteten Knotens nicht, gibt aber eine leere Zeichenkette zurück, wenn die Bearbeitung abgebrochen wird, oder den eingegebenen Text, wenn die Bearbeitung angenommen wird. Während der Bearbeitung kann mit Escape abgebrochen und mit Return angenommen werden. *clickres* legt fest, was geschieht, wenn außerhalb des bearbeitbaren Bereichs geklickt wird. Wenn *clickres* 0 ist (die Vorgabe), wird die Bearbeitung abgebrochen. Wenn *clickres* 1 ist, wird die Bearbeitung angenommen. In allen anderen Fällen geht die Bearbeitung weiter. Wenn die Bearbeitung angenommen wird und *modifycmd* nicht leer ist, wird er mit dem neuen Text als Argument aufgerufen und muss 1 zurückgeben, um den neuen Text anzunehmen, oder 0, um ihn

abzulehnen und die Bearbeitung fortzusetzen. *select* legt fest, ob der Anfangstext ausgewählt werden soll. Die Vorgabe ist 1.

pathName* exists *item
Gibt 1 zurück, wenn *item* in der Listbox existiert, sonst 0.

pathName* index *item
Gibt die Position von *item* in der Liste zurück.

***pathName* insert *index item* ?*option value...*?**
Fügt einen neuen Eintrag, der durch *item* gekennzeichnet ist, an der Position *index* in die Liste ein.

Jedes Vorkommen von *#auto* im Namen des Eintrags wird durch die Nummer des Eintrags in der Reihenfolge des Einfügens ersetzt.

-data
Benutzerdaten, die zum Eintrag gehören.

-fill
Legt die Vordergrundfarbe der Beschriftung des Eintrags fest.

-font
Legt eine Schriftart für die Beschriftung des Eintrags fest.

-image
Legt ein Bild fest, das links von der Beschriftung des Eintrags angezeigt wird. Die Option **window** setzt **image** außer Kraft.

-indent
Legt den zusätzlichen Abstand in Pixeln links vom Eintrag fest.

-text
Legt die Beschriftung des Eintrags fest.

-window
Legt einen Pfadnamen fest, der links von der Beschriftung des Eintrags angezeigt wird. Die Option **window** setzt **image** außer Kraft.

***pathName* item *first* ?*last*?**
Die Verwendung ist veraltet. Stattdessen items verwenden. Wenn *last* weggelassen wird, gibt der Befehl den Eintrag am Index *first* in der Liste zurück, oder eine leere Zeichenkette, wenn *first* auf ein nicht vorhandenes Element zeigt. Wenn *last* angegeben ist, gibt der Befehl eine Liste zurück, deren Elemente alle Einträge zwischen *first* und *last* sind, einschließlich der Grenzen. Sowohl *first* als auch *last* können jede der Standardformen für Indizes haben.

pathName* itemcget *item option
Gibt den aktuellen Wert einer Konfigurationsoption für den Eintrag zurück. *Option* kann jeden Wert haben, den der Erzeugungsbefehl des Eintrags akzeptiert.

***pathName* itemconfigure *item* ?*option*? ?*value option value ...*?**
Dieser Befehl entspricht dem Befehl **configure**, er gilt aber für die Optionen eines einzelnen Eintrags, während **configure** für die Optionen des Widgets als Ganzes gilt. **Options** kann jeden Wert haben, den der Widget-Befehl zur Erzeugung des Eintrags akzeptiert. Wenn

Optionen angegeben sind, werden sie geändert, wie der Befehl angibt, und der Befehl gibt eine leere Zeichenkette zurück. Wenn keine Optionen angegeben sind, gibt der Befehl eine Liste zurück, die die aktuellen Optionen des Eintrags beschreibt. Optionen, die nur gelesen werden können, werden nicht geändert.

***pathName* items ?*first*? ?*last*?**
Wenn *first* und *last* weggelassen werden, gibt der Befehl die Liste aller Einträge zurück. Wenn *first* angegeben und *last* weggelassen wird, gibt der Befehl den Eintrag am Index *first* zurück, oder eine leere Zeichenkette, wenn *first* auf ein nicht vorhandenes Element zeigt. Wenn *first* und *last* angegeben sind, gibt der Befehl eine Liste zurück, deren Elemente alle Einträge zwischen *first* und *last* sind, einschließlich der Grenzen. Sowohl *first* als auch *last* können jede der Standardformen für Indizes haben.

pathName* move *item index
Verschiebt *item* an die Position *index* in der Liste.

pathName* reorder *neworder
Ändert die Reihenfolge der Einträge in der Listbox, die *neworder* angibt. Einträge, die nicht in *neworder* vorkommen, werden nicht verschoben.

pathName* see *item
Ordnet den Scrollbereich so an, dass *item* sichtbar wird.

***pathName* selection *cmd* ?*arg...*?**
Ändert die Liste der ausgewählten Einträge entsprechend *cmd*:

clear
entfernt alle Einträge der Auswahl.

set
setzt die Auswahl auf alle Einträge in *arg*

add
nimmt alle Einträge von *arg* in die Auswahl auf

remove
entfernt alle Einträge von *arg* aus der Auswahl

get
gibt die aktuell ausgewählten Einträge zurück

***pathName* xview ?*arg...*?**
Standardbefehl, um das waagerechte Scrollen von *pathName* zu ermöglichen.

***pathName* yview ?*arg...*?**
Standardbefehl, um das senkrechte Scrollen von *pathName* zu ermöglichen.

BINDUNGEN

Ein virtuelles Ereignis **<<ListboxSelect>>** wird jedes Mal erzeugt, wenn sich die Auswahl in der Listbox ändert.

Die Listbox hat alle Standardbindungen des Mausrads, wenn sie den Fokus hat.

MainFrame

Stichwörter: MainFrame

NAME

MainFrame - Toplevel-Fenster mit Menü, Werkzeugleiste und Statusleiste verwalten

ERZEUGUNG

MainFrame *pathName* ?*option value...*?

OPTIONEN VON ProgressBar

-background oder -bg	-progressfg (siehe -foreground)
-progressmax (siehe -maximum)	-progresstype (siehe -type)
-progressvar (siehe -variable)	

WIDGET-SPEZIFISCHE OPTIONEN

-height
-menu
-separator
-textvariable
-width

WIDGET-BEFEHL

pathName addindicator ?*arg...*?

pathName addtoolbar

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

pathName getframe

pathName getindicator *index*

pathName getmenu *menuid*

pathName gettoolbar *index*

pathName setmenustate *tag state*

pathName showstatusbar *name*

pathName showtoolbar *index bool*

BESCHREIBUNG

MainFrame verwaltet ein Toplevel-Fenster, damit es hat:

- einfache Erzeugung eines Menüs, mit automatischen Beschleuniger-Bindungen und Zuordnung von DynamicHelp,
- eine oder mehrere Werkzeugleisten, die verborgen werden können,
- eine Statusleiste, die eine Benutzermeldung oder eine Menübeschreibung anzeigt, und wahlweise eine ProgressBar.

WIDGET-SPEZIFISCHE OPTIONEN

-height
Legt die gewünschte Höhe für den Benutzerrahmen in jeder Form fest, die Tk_GetPixels akzeptiert. Wenn diese Option kleiner oder gleich null ist (die Vorgabe), verlangt das Fenster überhaupt keine Größe.

-menu (nur lesbar)
Diese Option beschreibt das Menü. Das ist eine Liste, deren je fünf Elemente ein Kaskadenmenü beschreiben. Sie hat die folgende Form: {*menuname tags menuId tearoff menuentries*...}, wobei *menuentries* eine Liste ist, deren jedes Element einen Menüeintrag beschreibt, der sein kann:

- für einen Trenner: {**separator**}
- für einen Befehl: {**command** *menuname* ?*tags*? ?*description*? ?*accelerator*? ?*option value*? ...}
- für eine Schaltfläche zum Abhaken: {**checkbutton** *menuname* ?*tags*? ?*description*? ?*accelerator*? ?*option value*? ...}
- für einen Radiobutton: {**radiobutton** *menuname* ?*tags*? ?*description*? ?*accelerator* ?*option value*? ...}
- für ein Kaskadenmenü: {**cascade** *menuname tags menuId tearoff menuentries*}

where:

- *menuname* ist der Name des Menüs. Wenn er ein & enthält, wird das folgende Zeichen automatisch in die entsprechende Option **-underline** des Befehls **menu add** umgesetzt.
- *tags* ist die Tag-Liste für den Eintrag, mit der Menüeinträge über **MainFrame::setmenustate** ein- oder ausgeschaltet werden.
- *menuId* ist eine Kennung für das Menü, aus der **MainFrame::getmenu** den Pfadnamen des Menüs holt.
- *tearoff* legt fest, ob das Menü einen Abreißeintrag hat.
- *description* legt eine Zeichenkette für DynamicHelp fest.
- *accelerator* legt eine Tastenfolge fest. Das ist eine Liste aus zwei Elementen, das erste ist **Ctrl**, **Alt** oder **CtrlAlt**, das zweite ein Buchstabe oder eine Ziffer. Eine Beschleuniger-Zeichenkette wird aufgebaut und die entsprechende Bindung auf dem Toplevel gesetzt, um den Menüeintrag aufzurufen.
- *option value* legt zusätzliche Optionen für den Eintrag fest (siehe den Befehl **menu add**).

Jeder Wert in ? ist optional und in der Vorgabe die leere Zeichenkette, muss aber angegeben werden, wenn eine oder mehrere folgende Optionen nicht leer sind.
Beispiel:

```
set descmenu {
    "&File" {} {} 0 {
        {command "&New"     {} "Create a new document"     {Ctrl n}
    -command Menu::new}
        {command "&Open..." {} "Open an existing document" {Ctrl o}
    -command Menu::open}
        {command "&Save"    open "Save the document" {Ctrl s} -command
    Menu::save}
        {cascade  "&Export"  {} export 0 {
            {command "Format &1" open "Export document to format 1" {}
    -command {Menu::export 1}}
            {command "Format &2" open "Export document to format 2" {}
    -command {Menu::export 2}}
        }}
        {separator}
        {cascade "&Recent files" {} recent 0 {}}
        {separator}
        {command "E&xit" {} "Exit the application" {} -command
    Menu::exit}
    }
    "&Options" {} {} 0 {
        {checkbutton "Toolbar" {} "Show/hide toolbar" {}
            -variable Menu::_drawtoolbar
            -command  {$Menu::_mainframe showtoolbar toolbar
    $Menu::_drawtoolbar}
        }
    }
}
```

-separator (nur lesbar)
Legt fest, ob ein Trenner oben und/oder unten am Benutzerfenster gezeichnet werden soll. Muss einer der Werte **none**, **top**, **bottom** oder **both** sein. Das hängt vom Relief der Unterwidgets des Benutzerfensters ab.

-textvariable
Legt die Option textvariable für die Beschriftung der Statusleiste fest. Die Beschreibung der Menüeinträge von DynamicHelp wird bei der Erzeugung des MainFrame auf diese Variable abgebildet. Wenn diese Variable durch MainFrame::configure geändert wird, steht die Menübeschreibung nicht zur Verfügung. Der Text der Beschriftung ändert sich, indem der Wert der Variable geändert wird.

-width
Legt die gewünschte Breite für den Benutzerrahmen in jeder Form fest, die Tk_GetPixels akzeptiert. Wenn diese Option kleiner oder gleich null ist (die Vorgabe), verlangt das Fenster überhaupt keine Größe.

WIDGET-BEFEHL

***pathName* addindicator ?*arg...*?**
Fügt rechts an der Statusleiste ein Anzeigefeld hinzu. Jede Anzeige wird von links nach rechts hinzugefügt. Eine Anzeige ist ein Tk-Label-Widget, das mit den Options-Wert-Paaren aus ?*arg...*? konfiguriert wird. Die Optionen **-relief** und **-borderwidth** sind in der Vorgabe *sunken* und 1. Gibt den Pfadnamen des erzeugten Labels zurück.

***pathName* addtoolbar**
Fügt dem MainFrame eine Werkzeugleiste hinzu. Gibt den Pfadnamen des neuen Fensters zurück, in das die Einträge der Werkzeugleiste gelegt werden.

pathName* cget *option
Gibt den aktuellen Wert der Konfigurationsoption zurück, die *option* angibt. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert.

***pathName* configure ?*option*? ?*value option value ...*?**
Fragt die Konfigurationsoptionen des Widgets ab oder ändert sie. Wenn keine *option* angegeben ist, gibt der Befehl eine Liste zurück, die alle verfügbaren Optionen für *pathName* beschreibt. Wenn *option* ohne *value* angegeben ist, gibt der Befehl eine Liste zurück, die die genannte *option* beschreibt (diese Liste ist identisch mit der entsprechenden Teilliste des Wertes, der zurückgegeben wird, wenn keine *option* angegeben ist). Wenn ein oder mehrere Paare aus *option* und *value* angegeben sind, ändert der Befehl die genannten Widget-Optionen auf die genannten Werte; in diesem Fall gibt der Befehl eine leere Zeichenkette zurück. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert. Optionen, die nur gelesen werden können, werden nicht geändert.

***pathName* getframe**
Gibt den Pfadnamen des Benutzerfensters zurück.

pathName* getindicator *index
Gibt den Pfadnamen der als *index*-te hinzugefügten Anzeige zurück.

pathName* getmenu *menuid
Gibt den Pfadnamen des Menüs zurück, dessen Kennung *menuid* ist.

pathName* getmenustate *tag state
Gibt den Zustand des angegebenen Menü-*tag* zurück.

pathName* gettoolbar *index
Gibt den Pfadnamen der als *index*-te hinzugefügten Werkzeugleiste zurück.

pathName* setmenustate *tag state
Setzt den Wert der Option **-state** aller Menüeinträge, die das Tag *tag* haben, auf *state*. Ein Menüeintrag ist disabled, wenn eines seiner zugehörigen Tags den Zustand **disabled** hat.

pathName* showstatusbar *name
name ist einer der Werte **none**, **status** oder **progression**. **none** verbirgt die Statusleiste, **status** zeigt nur die Beschriftung, **progression** zeigt die Beschriftung und die ProgressBar.

pathName* showtoolbar *index bool
Verbirgt die als *index*-te hinzugefügte Werkzeugleiste, wenn *bool* 0 ist, oder zeigt sie, wenn *bool* 1 ist. Damit das Toplevel-Fenster beim Verbergen oder Anzeigen der Werkzeugleiste nicht die Größe ändert, dient [wm geometry $top [wm geometry $top]], wenn es verwaltet wird.

MessageDlg

Stichwörter: MessageDlg

NAME

MessageDlg - Meldungsdialog

ERZEUGUNG

MessageDlg *pathName* ?*option value...*?

STANDARDOPTIONEN

-anchor	-font
-foreground oder -fg	-padx
-pady	

OPTIONEN VON Dialog

-background oder -bg	-cancel
-default	-parent

WIDGET-SPEZIFISCHE OPTIONEN

-aspect	-buttons
-buttonwidth	-icon
-justify	-message
-title	-type
-width	

BESCHREIBUNG

MessageDlg bietet einen einfachen Weg, einen Meldungsdialog anzuzeigen. MessageDlg::**create** erzeugt den Meldungsdialog, zeigt ihn an und gibt den Index der gedrückten Schaltfläche zurück, oder -1, wenn er zerstört wird. Beim Zurückkehren existiert der Dialog nicht mehr.

WIDGET-SPEZIFISCHE OPTIONEN

-aspect

Legt eine nichtnegative ganze Zahl fest, die das gewünschte Seitenverhältnis des Textes angibt. Das Seitenverhältnis ist als 100*width/height angegeben. 100 bedeutet, der Text soll so breit wie hoch sein, 200 bedeutet, der Text soll doppelt so breit wie hoch sein, 50 bedeutet, der Text soll doppelt so hoch wie breit sein, und so weiter. Dient dazu, die Zeilenlänge des Textes zu wählen, wenn die Option **width** nicht angegeben ist. Voreingestellt ist 150.

-buttons
Legt eine Liste von Schaltflächen fest, die angezeigt werden, wenn die Option **type** *user* ist. Wenn eine Schaltfläche einen symbolischen Namen hat, wird der zugehörige Text angezeigt.

-buttonwidth
Legt die Standardbreite der Schaltflächen im Dialog fest.

-icon
Legt ein Icon fest, das angezeigt wird. Muss einer der folgenden Werte sein: **error**, **info**, **question** oder **warning**.

-justify
Legt fest, wie Textzeilen ausgerichtet werden. Muss einer der Werte **left**, **center** oder **right** sein. Voreingestellt ist **left**. Diese Option wirkt zusammen mit den Optionen **anchor**, **aspect**, **padx**, **pady** und **width** und ergibt verschiedene Anordnungen des Textes im Fenster. Die Optionen **aspect** und **width** bestimmen, wie viel Bildschirmplatz für die Anzeige des Textes gebraucht wird. Die Optionen **anchor**, **padx** und **pady** bestimmen, wo dieser rechteckige Bereich im Fenster des Widgets angezeigt wird, und die Option **justify** bestimmt, wie jede Zeile in diesem rechteckigen Bereich angezeigt wird. Wenn zum Beispiel **anchor e** und **justify left** ist und das Meldungsfenster viel größer ist, als der Text braucht, wird der Text so angezeigt, dass die linken Kanten aller Zeilen auf einer Linie liegen und die rechte Kante der längsten Zeile **padx** von der rechten Seite des Fensters entfernt ist; der gesamte Textblock wird in der senkrechten Ausdehnung des Fensters zentriert.

-message
Legt die Meldung fest, die in diesem Meldungsfeld angezeigt wird.

-title
Legt eine Zeichenkette fest, die als Titel des Meldungsfeldes angezeigt wird. Wenn der Wert leer ist (die Vorgabe), wird ein Vorgabetitel passend zur Option **icon** gesetzt. Der zugehörige Vorgabetitel ist englisch und kann in eine andere Sprache gesetzt werden, indem die Ressource angegeben wird:

```
    *MessageDlg.nameTitle:   value
```

oder der gleichwertige Tcl-Befehl:

```
    option add *MessageDlg.nameTitle value
```

dabei ist *name* der Name eines Icons, wie er in der Option **icon** definiert ist.
Zum Beispiel kann für die französische Sprache bei einem Warndialog angegeben werden:

```
    option add *MessageDlg.warningTitle  "Attention"
```

-type
Legt eine Menge von Schaltflächen fest, die angezeigt werden. Die folgenden Werte sind möglich:

abortretryignore
Zeigt drei Schaltflächen an, deren symbolische Namen **abort**, **retry** und **ignore** sind.

ok

Zeigt eine Schaltfläche an, deren symbolischer Name **ok** ist.

okcancel

Zeigt zwei Schaltflächen an, deren symbolische Namen **ok** und **cancel** sind.

retrycancel

Zeigt zwei Schaltflächen an, deren symbolische Namen **retry** und **cancel** sind.

yesno

Zeigt zwei Schaltflächen an, deren symbolische Namen **yes** und **no** sind.

yesnocancel

Zeigt drei Schaltflächen an, deren symbolische Namen **yes**, **no** und **cancel** sind.

user

Zeigt die Schaltflächen der Option **-buttons** an.

-width

Legt die Länge der Zeilen im Fenster fest. Wenn diese Option einen Wert größer als null hat, wird die Option **aspect** ignoriert und die Option **width** bestimmt die Zeilenlänge. Wenn diese Option einen Wert kleiner oder gleich null hat, bestimmt die Option **aspect** die Zeilenlänge.

NoteBook

Stichwörter: NoteBook

NAME

NoteBook - Widget zur Notizbuchverwaltung

ERZEUGUNG

NoteBook *pathName* ?*option value...*?

STANDARDOPTIONEN

-font

OPTIONS from ArrowButton

-activebackground	-activeforeground
-background oder -bg	-borderwidth oder -bd
-disabledforeground	-foreground oder -fg
-repeatdelay	-repeatinterval

WIDGET-SPEZIFISCHE OPTIONEN

-arcradius	-height
-homogeneous	-side
-tabbevelsize	-tabpady
-width	

WIDGET-BEFEHL

pathName bindtabs *event script*

pathName cget *option*

pathName compute_size

pathName configure ?*option*? ?*value option value* ...?

pathName delete *page* ?*destroyframe*?

pathName getframe *page*

pathName index *page*

pathName insert *index page* ?*option value...*?

pathName itemcget *page option*

pathName itemconfigure *page* ?*option*? ?*value option value* ...?

pathName move *page index*

pathName page *first* ?*last*?

pathName pages ?*first*? ?*last*?

pathName raise ?*page*?

pathName see *page*

BESCHREIBUNG

Das Widget NoteBook verwaltet eine Menge von Seiten und zeigt eine davon an.

WIDGET-SPEZIFISCHE OPTIONEN

-arcradius
Legt die Stärke der Abrundung an den Ecken eines Reiters fest. Dieser Wert kann von 0 bis 8 Pixel eingestellt werden, die Vorgabe ist 2 Pixel. Gewöhnlich sind kleine Werte vorzuziehen.

-height
Legt die gewünschte Höhe für die Seiten fest. Wenn diese Option gleich null ist (die Vorgabe), verlangt das Fenster überhaupt keine Größe. In diesem Fall kann NoteBook::**compute_size** aufgerufen werden, damit NoteBook groß genug ist, um die größte Seite zu enthalten.

-homogeneous
Legt fest, ob die Beschriftung der Seiten dieselbe Breite haben muss.

-side
Legt die Seite fest, auf der die Beschriftung der Seiten platziert wird. Muss **top** oder **bottom** sein.

-tabbevelsize
Legt die Stärke der Abschrägung der Reiter fest. Dieser Wert kann von 0 bis 8 Pixel eingestellt werden, die Vorgabe ist 0 Pixel. Eine Abschrägung von null Pixel ist im Wesentlichen rechteckig, eine von ungleich null sieht trapezförmig aus.

-tabpady
Legt den Abstand zwischen dem Text im Reiter und dem oberen und unteren Rand des Reiters fest. Der Abstand kann eine Liste aus zwei Werten sein, die den Abstand oben und unten getrennt angeben. Der Vorgabeabstand ist {0 6}.

-width
Legt die gewünschte Breite für die Seiten fest. Wenn diese Option gleich null ist (die Vorgabe), verlangt das Fenster überhaupt keine Größe. In diesem Fall kann NoteBook::**compute_size** aufgerufen werden, damit NoteBook groß genug ist, um die größte Seite zu enthalten.

WIDGET-BEFEHL

pathName* bindtabs *event script
Dieser Befehl ordnet einen Befehl zu, der ausgeführt wird, wenn die von *event* angegebene Ereignisfolge auf einem Reiter eintritt. Der Seitenbezeichner, auf dem das Ereignis eintritt, wird an den Befehl angehängt.

pathName* cget *option
Gibt den aktuellen Wert der Konfigurationsoption zurück, die *option* angibt. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert.

***pathName* compute_size**
Dieser Befehl kann aufgerufen werden, damit das NoteBook groß genug ist, um die größte Seite zu enthalten. Wenn alle Seiten -createcmd verwenden, haben sie keine verlangte Größe.

***pathName* configure ?*option*? ?*value option value ...*?**
Fragt die Konfigurationsoptionen des Widgets ab oder ändert sie. Wenn keine *option* angegeben ist, gibt der Befehl eine Liste zurück, die alle verfügbaren Optionen für *pathName* beschreibt. Wenn *option* ohne *value* angegeben ist, gibt der Befehl eine Liste zurück, die die genannte *option* beschreibt (diese Liste ist identisch mit der entsprechenden Teilliste des Wertes, der zurückgegeben wird, wenn keine *option* angegeben ist). Wenn ein oder mehrere Paare aus *option* und *value* angegeben sind, ändert der Befehl die genannten Widget-Optionen auf die genannten Werte; in diesem Fall gibt der Befehl eine leere Zeichenkette zurück. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert. Optionen, die nur gelesen werden können, werden nicht geändert.

***pathName* delete *page* ?*destroyframe*?**
Löscht die Seite *page*. Wenn *destroyframe* 1 ist (die Vorgabe), wird der zu *page* gehörende Rahmen zerstört. Wenn *destroyframe* 0 ist, wird der Rahmen nicht zerstört und von einem weiteren Aufruf von **insert** mit derselben *page* wiederverwendet.

pathName* getframe *page

Gibt den Pfadnamen der Seite *page* zurück.

pathName* index *page
Gibt den numerischen Index zurück, der dem Eintrag entspricht.

***pathName* insert *index page* ?*option value...*?**
Fügt eine neue Seite ein, die durch *page* gekennzeichnet ist, an der Position *index* in der Seitenliste. *index* muss numerisch oder **end** sein. Der Pfadname der neuen Seite wird zurückgegeben.

-createcmd
Legt einen Befehl fest, der aufgerufen wird, wenn die Seite zum ersten Mal angehoben wird.

-image
Legt ein Bild fest, das für die Seite links von der Beschriftung angezeigt wird

-leavecmd
Legt einen Befehl fest, der aufgerufen wird, wenn eine Seite verlassen werden soll. Der Befehl muss 0 zurückgeben, wenn die Seite nicht verlassen werden kann, oder 1, wenn sie es kann.

-raisecmd
Legt einen Befehl fest, der jedes Mal aufgerufen wird, wenn die Seite angehoben wird.

-state
Legt den Zustand der Seite fest. Muss **normal** oder **disabled** sein.

-text
Legt eine Beschriftung fest, die für die Seite angezeigt wird.

pathName* itemcget *page option
Gibt den aktuellen Wert einer Konfigurationsoption für den Eintrag zurück. *Option* kann jeden Wert haben, den der Erzeugungsbefehl des Eintrags akzeptiert.

***pathName* itemconfigure *page* ?*option*? ?*value option value ...*?**
Dieser Befehl entspricht dem Befehl **configure**, er gilt aber für die Optionen eines einzelnen Eintrags, während **configure** für die Optionen des Widgets als Ganzes gilt. **Options** kann jeden Wert haben, den der Widget-Befehl zur Erzeugung des Eintrags akzeptiert. Wenn Optionen angegeben sind, werden sie geändert, wie der Befehl angibt, und der Befehl gibt eine leere Zeichenkette zurück. Wenn keine Optionen angegeben sind, gibt der Befehl eine Liste zurück, die die aktuellen Optionen des Eintrags beschreibt. Optionen, die nur gelesen werden können, werden nicht geändert.

pathName* move *page index
Verschiebt den Reiter *page* zum Index *index*.

***pathName* page *first* ?*last*?**
Die Verwendung ist veraltet. Stattdessen pages verwenden. Wenn *last* weggelassen wird, gibt der Befehl die Seite am Index *first* zurück, oder eine leere Zeichenkette, wenn *first* auf ein nicht vorhandenes Element zeigt. Wenn *last* angegeben ist, gibt der Befehl eine Liste zurück, deren Elemente alle Seiten zwischen *first* und *last* sind, einschließlich der Grenzen. Sowohl *first* als auch *last* können jede der Standardformen für Indizes haben.

***pathName* pages ?*first*? ?*last*?**
Wenn *first* und *last* weggelassen werden, gibt der Befehl die Liste aller Seiten zurück. Wenn

first angegeben und *last* weggelassen wird, gibt der Befehl die Seite am Index *first* zurück, oder eine leere Zeichenkette, wenn *first* auf ein nicht vorhandenes Element zeigt. Wenn *first* und *last* angegeben sind, gibt der Befehl eine Liste zurück, deren Elemente alle Seiten zwischen *first* und *last* sind, einschließlich der Grenzen. Sowohl *first* als auch *last* können jede der Standardformen für Indizes haben.

***pathName* raise ?*page*?**
Hebt die Seite *page* an oder gibt die angehobene Seite zurück, wenn *page* weggelassen wird.

pathName* see *page
Scrollt die Beschriftungen so, dass die Beschriftung der Seite *page* sichtbar wird.

PagesManager

Stichwörter: PagesManager

NAME
PagesManager - Widget zur Seitenverwaltung

ERZEUGUNG
PagesManager *pathName* ?*option value...*?

STANDARDOPTIONEN
-background

WIDGET-SPEZIFISCHE OPTIONEN
-height

WIDGET-BEFEHL
pathName add *page*

pathName cget *option*

pathName compute_size

pathName configure ?*option*? ?*value option value* ...?

pathName delete *page*

pathName getframe *page*

pathName page *first* ?*last*?

pathName pages ?*first*? ?*last*?

pathName raise ?*page*?

BESCHREIBUNG

Das Widget PagesManager verwaltet eine Menge von Seiten und zeigt eine davon an. PagesManager stellt keine Methode für den Benutzerzugriff bereit, wie es NoteBook tut; das kann also über eine Listbox, ein Menü, einen Radiobutton oder etwas anderes geschehen. Das

Widget zeigt während der Erzeugung keine Seiten; es muss ein ausdrücklicher Aufruf von **raise** erfolgen, um eine anzuzeigen.

WIDGET-SPEZIFISCHE OPTIONEN

-height
Legt die gewünschte Höhe für die Seiten fest. Wenn diese Option gleich null ist (die Vorgabe), verlangt das Fenster überhaupt keine Größe. In diesem Fall kann PagesManager:: **compute_size** aufgerufen werden, damit PagesManager groß genug ist, um die größte Seite zu enthalten.

-width
Legt die gewünschte Breite für die Seiten fest. Wenn diese Option gleich null ist (die Vorgabe), verlangt das Fenster überhaupt keine Größe. In diesem Fall kann PagesManager:: **compute_size** aufgerufen werden, damit PagesManager groß genug ist, um die größte Seite zu enthalten.

WIDGET-BEFEHL

pathName* add *page
Fügt eine neue Seite hinzu, die durch *page* gekennzeichnet ist, einen beliebigen Bezeichner. Der Pfadname der neuen Seite wird zurückgegeben, und die Widgets für die Seite sollten mit diesem als Elternteil oder Vorfahr erzeugt werden. PagesManager verwaltet jedoch seine eigene Geometrie. **pack**, **grid** oder ein gleichwertiger Befehl sollten nicht mit dem von **add** zurückgegebenen Pfadnamen verwendet werden.

pathName* cget *option
Gibt den aktuellen Wert der Konfigurationsoption zurück, die *option* angibt. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert.

***pathName* compute_size**
Dieser Befehl kann aufgerufen werden, damit der PagesManager groß genug ist, um die größte Seite zu enthalten.

***pathName* configure ?*option*? ?*value option value ...*?**
Fragt die Konfigurationsoptionen des Widgets ab oder ändert sie. Wenn keine *option* angegeben ist, gibt der Befehl eine Liste zurück, die alle verfügbaren Optionen für *pathName* beschreibt. Wenn *option* ohne *value* angegeben ist, gibt der Befehl eine Liste zurück, die die genannte *option* beschreibt (diese Liste ist identisch mit der entsprechenden Teilliste des Wertes, der zurückgegeben wird, wenn keine *option* angegeben ist). Wenn ein oder mehrere Paare aus *option* und *value* angegeben sind, ändert der Befehl die genannten Widget-Optionen auf die genannten Werte; in diesem Fall gibt der Befehl eine leere Zeichenkette zurück. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert. Optionen, die nur gelesen werden können, werden nicht geändert.

pathName* delete *page
Löscht die Seite *page*.

pathName* getframe *page
Gibt den Pfadnamen der Seite *page* zurück.

***pathName* page *first* ?*last*?**

Die Verwendung ist veraltet. Stattdessen pages verwenden. Wenn *last* weggelassen wird, gibt der Befehl die Seite am Index *first* zurück, oder eine leere Zeichenkette, wenn *first* auf ein nicht vorhandenes Element zeigt. Wenn *last* angegeben ist, gibt der Befehl eine Liste zurück, deren Elemente alle Seiten zwischen *first* und *last* sind, einschließlich der Grenzen. Sowohl *first* als auch *last* können jede der Standardformen für Indizes haben.

***pathName* pages ?*first*? ?*last*?**

Wenn *first* und *last* weggelassen werden, gibt der Befehl die Liste aller Seiten zurück. Wenn *first* angegeben und *last* weggelassen wird, gibt der Befehl die Seite am Index *first* zurück, oder eine leere Zeichenkette, wenn *first* auf ein nicht vorhandenes Element zeigt. Wenn *first* und *last* angegeben sind, gibt der Befehl eine Liste zurück, deren Elemente alle Seiten zwischen *first* und *last* sind, einschließlich der Grenzen. Sowohl *first* als auch *last* können jede der Standardformen für Indizes haben.

***pathName* raise ?*page*?**

Hebt die Seite *page* an oder gibt die angehobene Seite zurück, wenn *page* weggelassen wird.

PanedWindow

Stichwörter: PanedWindow

NAME

PanedWindow - Widget zur gekachelten Anordnung

ERZEUGUNG

PanedWindow *pathName* ?*option value...*?

STANDARDOPTIONEN

-background oder -bg

WIDGET-SPEZIFISCHE OPTIONEN

-activator	-pad
-side	-weights
-width	

WIDGET-BEFEHL

pathName add ?*option value...*?

pathName cget *option*

pathName configure ?*option*? ?*value option value ...*?

pathName getframe *index*

BESCHREIBUNG

PanedWindow ist ein Widget, das Kinder senkrecht oder waagerecht gekachelt anordnet. Die

Größe der Teilflächen lässt sich über einen Steuersteg einstellen, der zwischen den Kindern erzeugt wird.

WIDGET-SPEZIFISCHE OPTIONEN

-activator (nur lesbar)
Der Steuersteg ist in der Vorgabe je nach Betriebssystem eine Schaltfläche oder eine Linie. Dieser Steg kann mit dieser Option ausdrücklich gesetzt werden. Ist er auf **line** gesetzt, besteht der Steg nur aus einer Trennlinie. Ist er auf **button** gesetzt, besteht er aus einem Knopf, der gezogen werden kann.

-pad (nur lesbar)
Legt zusätzlichen Abstand zwischen der Schaltfläche des Stegs und den Kindern fest.

-side (nur lesbar)
Legt die Seite des Stegs fest und damit die Anordnung: **top** oder **bottom** (waagerechte Anordnung), **left** oder **right** (senkrechte Anordnung).

-weights (nur lesbar)
Legt fest, wie die Gewichte verwendet werden, die beim Hinzufügen von Teilflächen angegeben werden. Muss **extra** oder **available** sein. Bei *extra* wird nur der zusätzliche Platz im Verhältnis zum Gewicht auf die verschiedenen Teilflächen verteilt. Bei *available* wird der gesamte Platz im Verhältnis zum Gewicht auf die verschiedenen Teilflächen verteilt. Der Vorgabewert für **weights** ist *extra*.

-width (nur lesbar)
Legt die Breite der Schaltfläche des Stegs fest. Diese Option wird ignoriert, wenn der Activator auf **line** gesetzt ist.

WIDGET-BEFEHL

***pathName* add ?*option value...*?**
Dieser Befehl fügt eine neue Teilfläche hinzu. Die neue Teilfläche wird bei senkrechter Anordnung unter die vorherige Teilfläche gelegt und bei waagerechter Anordnung rechts davon. Dieser Befehl gibt einen Rahmen zurück, in den das Widget gelegt werden kann. Gültige Optionen sind:

-minsize
Legt die verlangte Mindestgröße der Teilfläche fest. Weitere Angaben stehen beim Befehl **grid** .

-weight
Legt das relative Gewicht fest, nach dem zusätzlicher Platz auf die Teilflächen verteilt wird. Weitere Angaben stehen beim Befehl **grid**.

pathName* cget *option
Gibt den aktuellen Wert der Konfigurationsoption zurück, die *option* angibt. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert.

***pathName* configure ?*option*? ?*value option value ...*?**
Fragt die Konfigurationsoptionen des Widgets ab oder ändert sie. Wenn keine *option* angegeben ist, gibt der Befehl eine Liste zurück, die alle verfügbaren Optionen für *pathName*

beschreibt. Wenn *option* ohne *value* angegeben ist, gibt der Befehl eine Liste zurück, die die genannte *option* beschreibt (diese Liste ist identisch mit der entsprechenden Teilliste des Wertes, der zurückgegeben wird, wenn keine *option* angegeben ist). Wenn ein oder mehrere Paare aus *option* und *value* angegeben sind, ändert der Befehl die genannten Widget-Optionen auf die genannten Werte; in diesem Fall gibt der Befehl eine leere Zeichenkette zurück. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert. Optionen, die nur gelesen werden können, werden nicht geändert.

pathName* getframe *index
Gibt den Pfadnamen der als *index*-te hinzugefügten Teilfläche zurück.

PanelFrame

Stichwörter: PanelFrame

NAME
PanelFrame - Rahmen mit einem umrahmten Titelbereich

ERZEUGUNG
PanelFrame *pathName* ?*option value...*?

STANDARDOPTIONEN

-background oder -bg	-borderwidth oder -bd
-text	
-textvariable	
-font	-relief

WIDGET-SPEZIFISCHE OPTIONEN

-height	
-width	
-panelbackground	-panelforeground
-ipad	

WIDGET-BEFEHL
pathName cget *option*

pathName configure ?*option*? ?*value option value ...*?

BESCHREIBUNG

PanelFrame erzeugt einen Rahmenbereich mit einem umrahmten Titelbereich. Der umrahmte Titelbereich enthält ein Label und erlaubt, weitere Einträge darin zu platzieren. Das kann wie eine kleine Werkzeugleiste dienen.

WIDGET-SPEZIFISCHE OPTIONEN

-height
Legt die gewünschte Höhe für das Widget fest.

-ipad
Der Abstand um die einzelnen Einträge des Panelbereichs.

-panelbackground
Die Farbe für den Hintergrund des Panelbereichs. Voreingestellt ist die Hintergrundfarbe der Auswahlhervorhebung.

-panelforeground
Die Farbe für den Titeltext. Voreingestellt ist die Vordergrundfarbe der Auswahlhervorhebung.

-width
Legt die gewünschte Breite für das Widget fest.

WIDGET-BEFEHL

***pathName* add widget ?*option value option value ...*?**
Fügt dem Panel ein Widget hinzu. Die Widgets werden mit **pack** hineingelegt. Mögliche Optionen sind:

-side
Seite, auf die der Eintrag gelegt wird (Vorgabe **right**).

-fill
Ob der Platz gefüllt wird (Vorgabe **none**).

-expand
Ob der Platz ausgedehnt wird (Vorgabe **0**).

-pad
Überschreibt die Option **-ipad** des Widgets für diesen Eintrag.

pathName* cget *option
Gibt den aktuellen Wert der Konfigurationsoption zurück, die *option* angibt. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert.

***pathName* configure ?*option*? ?*value option value ...*?**
Fragt die Konfigurationsoptionen des Widgets ab oder ändert sie. Wenn keine *option* angegeben ist, gibt der Befehl eine Liste zurück, die alle verfügbaren Optionen für *pathName* beschreibt. Wenn *option* ohne *value* angegeben ist, gibt der Befehl eine Liste zurück, die die genannte *option* beschreibt (diese Liste ist identisch mit der entsprechenden Teilliste des Wertes, der zurückgegeben wird, wenn keine *option* angegeben ist). Wenn ein oder mehrere Paare aus *option* und *value* angegeben sind, ändert der Befehl die genannten Widget-Optionen auf die genannten Werte; in diesem Fall gibt der Befehl eine leere Zeichenkette zurück. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert. Optionen, die nur gelesen werden können, werden nicht geändert.

***pathName* delete widget ?*widget* ...?**
Löscht ein Widget und den zugehörigen Zustand aus dem Panel.

***pathName* getframe**
Holt das Rahmen-Widget der Statusleiste, in dem die Einträge der Statusleiste erzeugt werden

sollen.

***pathName* items**
Liste der Einträge in der Statusleiste.

PasswdDlg

Stichwörter: PasswdDlg

NAME
PasswdDlg - Dialog für Anmeldung und Passwort

ERZEUGUNG
PasswdDlg *pathName* ?*option value...*?

OPTIONEN VON Dialog

-anchor	-background oder -bg
-homogeneous	-modal
-padx	-pady
-parent	-spacing
-title	

OPTIONEN VON LabelEntry

-background oder -bg	-borderwidth oder -bd
-disabledforeground	-entrybg
-entryfg	-exportselection
-font	-foreground oder -fg
-helptype	-highlightbackground
-highlightcolor	-highlightthickness
-insertbackground	-insertborderwidth
-insertofftime	-insertontime
-insertwidth	-labelanchor
-labelfont	-labelheight
-labeljustify	-labelwidth
-loginhelptext (siehe -helptext)	-loginhelpvar (siehe -helpvar)
-loginlabel (siehe -label)	-logintext (siehe -text)

-logintextvariable (siehe -textvariable)	-loginunderline (siehe -underline)
-passwdeditable (siehe -editable)	-passwdhelptext (siehe -helptext)
-passwdhelpvar (siehe -helpvar)	-passwdlabel (siehe -label)
-passwdstate (siehe -state)	-passwdtext (siehe -text)
-passwdtextvariable (siehe -textvariable)	-passwdunderline (siehe -underline)
-relief	-selectbackground
-selectborderwidth	-selectforeground
-wraplength	

WIDGET-SPEZIFISCHE OPTIONEN

-command
-type

WIDGET-BEFEHL

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

BESCHREIBUNG

PasswdDlg bietet einen einfachen Weg, einen Dialog für Anmeldung und Passwort anzuzeigen. PasswdDlg::**create** erzeugt den Dialog, zeigt ihn an und gibt den Wert von Anmeldung und Passwort in einer Liste zurück, oder eine leere Liste, wenn er zerstört wird oder die Schaltfläche cancel gedrückt wird. Beim Zurückkehren existiert der Dialog nicht mehr.
Zusätzliche Ressourcen können gesetzt werden, um weiteren Text zu ändern:

```
    *loginName     Beschriftung für das LabelEntry der Anmeldung
    *passwordName  Beschriftung für das LabelEntry des Passworts
```

WIDGET-SPEZIFISCHE OPTIONEN

-command
Legt einen Befehl fest, der aufgerufen wird, wenn die Schaltfläche ok gedrückt wird.

-type
Legt eine Menge von Schaltflächen fest, die angezeigt werden. Die folgenden Werte sind möglich:

ok
Zeigt eine Schaltfläche an, deren symbolischer Name **ok** ist.

okcancel

Zeigt zwei Schaltflächen an, deren symbolische Namen **ok** und **cancel** sind.

WIDGET-BEFEHL

pathName* cget *option
Gibt den aktuellen Wert der Konfigurationsoption zurück, die *option* angibt. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert.

***pathName* configure ?*option*? ?*value option value ...*?**
Fragt die Konfigurationsoptionen des Widgets ab oder ändert sie. Wenn keine *option* angegeben ist, gibt der Befehl eine Liste zurück, die alle verfügbaren Optionen für *pathName* beschreibt. Wenn *option* ohne *value* angegeben ist, gibt der Befehl eine Liste zurück, die die genannte *option* beschreibt (diese Liste ist identisch mit der entsprechenden Teilliste des Wertes, der zurückgegeben wird, wenn keine *option* angegeben ist). Wenn ein oder mehrere Paare aus *option* und *value* angegeben sind, ändert der Befehl die genannten Widget-Optionen auf die genannten Werte; in diesem Fall gibt der Befehl eine leere Zeichenkette zurück. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert. Optionen, die nur gelesen werden können, werden nicht geändert.

Stephane Lavirotte

(Stephane.Lavirotte@sophia.inria.fr)

ProgressBar

Stichwörter: ProgressBar

NAME
ProgressBar - Widget für eine Fortschrittsanzeige

ERZEUGUNG
ProgressBar *pathName* ?*option value...*?

STANDARDOPTIONEN

-background oder -bg	-borderwidth oder -bd
-foreground oder -fg	-orient
-relief	
-troughcolor	

WIDGET-SPEZIFISCHE OPTIONEN

-height
-maximum
-type

-variable
-width

WIDGET-BEFEHL

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

BESCHREIBUNG

Das Widget ProgressBar zeigt den Fortschritt einer langwierigen Operation an. Es wird von MainFrame und ProgressDlg verwendet.

WIDGET-SPEZIFISCHE OPTIONEN

-height
Legt die gewünschte Höhe der Fortschrittsanzeige fest.

-maximum
Legt den Höchstwert der Variable fest. Dieser Wert muss größer als null sein.

-type
Legt den Typ der ProgressBar fest. Muss einer der Werte **normal**, **incremental**, **infinite** oder **nonincremental_infinite** sein.

Wenn **type** *normal* ist, wird die Fortschrittsanzeige jedes Mal, wenn die Variable gesetzt wird, im Verhältnis zum Wert der Variable und zur Option **maximum** gezeichnet.

Wenn **type** *incremental* ist, wird der Wert der Fortschrittsanzeige intern gehalten und jedes Mal, wenn die Variable gesetzt wird, um ihren Wert erhöht. Die Fortschrittsanzeige wird im Verhältnis zum internen Wert und zur Option **maximum** gezeichnet.

Wenn **type** *infinite* ist, wird der Wert der Fortschrittsanzeige intern gehalten und jedes Mal, wenn die Variable gesetzt wird, um ihren Wert erhöht. Die Fortschrittsanzeige bewegt sich von links nach rechts, wenn der interne Wert (modulo **maximum**) kleiner als **maximum**/2 ist, und von rechts nach links, wenn der interne Wert größer als **maximum**/2 ist.

Wenn **type** *nonincremental_infinite* ist, wird der Wert der Fortschrittsanzeige aus dem Wert der Variable genommen. Die Fortschrittsanzeige bewegt sich von links nach rechts, wenn der Wert der Variable (modulo **maximum**) kleiner als **maximum**/2 ist, und von rechts nach links, wenn der interne Wert größer als **maximum**/2 ist.

Der Sonderfall des Wertes steht bei der Option **-variable**,

Der Vorgabewert für **type** ist *normal*.

-variable
Legt die Variable fest, die an die Fortschrittsanzeige gebunden ist. Die Fortschrittsanzeige wird aktualisiert, wenn sich der Wert der Variable ändert. Wenn der Wert der Variable negativ ist, wird die Fortschrittsanzeige nicht angezeigt (sie wird flach in der Farbe **background** gezeichnet, nützlich für ProgressDlg, um sie unsichtbar zu machen). Wenn ihr Wert 0 ist, wird

die Fortschrittsanzeige neu initialisiert.

-width
Legt die gewünschte Breite der Fortschrittsanzeige fest.

WIDGET-BEFEHL

pathName* cget *option
Gibt den aktuellen Wert der Konfigurationsoption zurück, die *option* angibt. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert.

***pathName* configure ?*option*? ?*value option value ...*?**
Fragt die Konfigurationsoptionen des Widgets ab oder ändert sie. Wenn keine *option* angegeben ist, gibt der Befehl eine Liste zurück, die alle verfügbaren Optionen für *pathName* beschreibt. Wenn *option* ohne *value* angegeben ist, gibt der Befehl eine Liste zurück, die die genannte *option* beschreibt (diese Liste ist identisch mit der entsprechenden Teilliste des Wertes, der zurückgegeben wird, wenn keine *option* angegeben ist). Wenn ein oder mehrere Paare aus *option* und *value* angegeben sind, ändert der Befehl die genannten Widget-Optionen auf die genannten Werte; in diesem Fall gibt der Befehl eine leere Zeichenkette zurück. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert. Optionen, die nur gelesen werden können, werden nicht geändert.

ProgressDlg

Stichwörter: ProgressDlg

NAME
ProgressDlg - Dialog mit Fortschrittsanzeige

ERZEUGUNG
ProgressDlg *pathName* ?*option value...*?

STANDARDOPTIONEN

-font
-textvariable

OPTIONEN VON Dialog

-background oder -bg	-parent
-separator	-title

OPTIONEN VON ProgressBar

-background oder -bg	-borderwidth oder -bd
-foreground oder -fg	-maximum

-relief	-troughcolor
-type	-variable

WIDGET-SPEZIFISCHE OPTIONEN

-command
-height
-stop
-width

WIDGET-BEFEHL

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

BESCHREIBUNG

ProgressDlg bietet einen einfachen Weg, einen Dialog mit Fortschrittsanzeige anzuzeigen. ProgressDlg::**create** erzeugt den Dialog, zeigt ihn an, setzt einen lokalen Grab darauf und kehrt sofort zurück. Der Dialog wird aktualisiert, indem der Wert der Variable der Optionen **-textvariable** und **-variable** geändert wird. Der Dialog muss nach der Verwendung zerstört werden.

WIDGET-SPEZIFISCHE OPTIONEN

-command
Legt einen Befehl fest, der aufgerufen wird, wenn die Stoppschaltfläche gedrückt wird. Das Programm muss regelmäßig **update** aufrufen, damit Ereignisse für das Drücken der Schaltfläche entstehen können.

-height
Legt eine gewünschte Höhe für das Label in Textzeilen fest.

-stop
Legt den Text der Schaltfläche fest, die gewöhnlich zum Anhalten des Prozesses dient. Wenn der Wert leer ist, wird keine Schaltfläche gezeichnet. Das kann ein symbolischer Name sein.

-width
Legt eine gewünschte Breite für das Label in Zeichen fest.

WIDGET-BEFEHL

pathName* cget *option
Gibt den aktuellen Wert der Konfigurationsoption zurück, die *option* angibt. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert.

***pathName* configure ?*option*? ?*value option value ...*?**

Fragt die Konfigurationsoptionen des Widgets ab oder ändert sie. Wenn keine *option* angegeben ist, gibt der Befehl eine Liste zurück, die alle verfügbaren Optionen für *pathName* beschreibt. Wenn *option* ohne *value* angegeben ist, gibt der Befehl eine Liste zurück, die die genannte *option* beschreibt (diese Liste ist identisch mit der entsprechenden Teilliste des Wertes, der zurückgegeben wird, wenn keine *option* angegeben ist). Wenn ein oder mehrere Paare aus *option* und *value* angegeben sind, ändert der Befehl die genannten Widget-Optionen auf die genannten Werte; in diesem Fall gibt der Befehl eine leere Zeichenkette zurück. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert. Optionen, die nur gelesen werden können, werden nicht geändert.

ScrollView

Stichwörter: ScrollView

NAME

ScrollView - den sichtbaren Bereich eines gescrollten Fensters anzeigen

ERZEUGUNG

ScrollView *pathName* ?*option value...*?

STANDARDOPTIONEN

-background oder -bg	-borderwidth oder -bd
-cursor	

WIDGET-SPEZIFISCHE OPTIONEN

-fill
-height

WIDGET-BEFEHL

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

BESCHREIBUNG

ScrollView zeigt den sichtbaren Bereich eines gescrollten Fensters innerhalb seines Scrollbereichs an.

WIDGET-SPEZIFISCHE OPTIONEN

-fill

Legt die Füllfarbe des Rechtecks fest.

-foreground

Legt die Farbe des Randes des Rechtecks fest.

-height
Legt die gewünschte Höhe für das ScrollView fest.

-width
Legt die gewünschte Breite für das ScrollView fest.

-window
Legt das Fenster fest, das angezeigt wird. Dieses Widget muss die Optionen **-xscrollcommand** und **-yscrollcommand** haben und auf die Befehle **xview** und **yview** antworten. Damit ScrollView mit einer anderen Bildlaufleiste zusammenarbeitet, müssen die Optionen **-xscrollcommand** und **-yscrollcommand** des Widgets gesetzt werden, bevor das Widget an die Option **-window** des ScrollView übergeben wird (wenn das Widget zum Beispiel von einem ScrolledWindow verwaltet wird, **setwidget** aufrufen, bevor die Option **-window** gesetzt wird).

WIDGET-BEFEHL

pathName* cget *option
Gibt den aktuellen Wert der Konfigurationsoption zurück, die *option* angibt. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert.

***pathName* configure ?*option*? ?*value option value ...*?**
Fragt die Konfigurationsoptionen des Widgets ab oder ändert sie. Wenn keine *option* angegeben ist, gibt der Befehl eine Liste zurück, die alle verfügbaren Optionen für *pathName* beschreibt. Wenn *option* ohne *value* angegeben ist, gibt der Befehl eine Liste zurück, die die genannte *option* beschreibt (diese Liste ist identisch mit der entsprechenden Teilliste des Wertes, der zurückgegeben wird, wenn keine *option* angegeben ist). Wenn ein oder mehrere Paare aus *option* und *value* angegeben sind, ändert der Befehl die genannten Widget-Optionen auf die genannten Werte; in diesem Fall gibt der Befehl eine leere Zeichenkette zurück. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert. Optionen, die nur gelesen werden können, werden nicht geändert.

BINDUNGEN

ScrollableFrame

Stichwörter: ScrollableFrame

NAME
ScrollableFrame - scrollbarer Rahmen, der ein Widget enthält

ERZEUGUNG
ScrollableFrame *pathName* ?*option value...*?

STANDARDOPTIONEN

-background oder -bg	-xscrollcommand
-yscrollcommand	

WIDGET-SPEZIFISCHE OPTIONEN

-areaheight
-areawidth
-constrainedheight
-constrainedwidth
-height
-width
-xscrollincrement
-yscrollincrement

WIDGET-BEFEHL

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

pathName getframe

pathName see *widget* ?*vert*? ?*horz*?

pathName xview ?*arg...*?

pathName yview ?*arg...*?

BESCHREIBUNG

Das Widget ScrollableFrame, das ein Widget enthält.

WIDGET-SPEZIFISCHE OPTIONEN

-areaheight
Legt die Höhe des scrollbaren Bereichs fest. Wenn null, wird die Höhe des scrollbaren Bereichs gerade groß genug gemacht, um alle seine Kinder aufzunehmen.

-areawidth
Legt die Breite des scrollbaren Bereichs fest. Wenn null, wird die Breite des Fensters des scrollbaren Bereichs gerade groß genug gemacht, um alle seine Kinder aufzunehmen.

-constrainedheight
Legt fest, ob der scrollbare Bereich dieselbe Höhe wie das gescrollte Fenster haben soll. Wenn true, wird keine senkrechte Bildlaufleiste gebraucht.

-constrainedwidth
Legt fest, ob der scrollbare Bereich dieselbe Breite wie das gescrollte Fenster haben soll. Wenn true, wird keine waagerechte Bildlaufleiste gebraucht.

-height
Legt die gewünschte Höhe für das Fenster in Pixeln fest.

-width
Legt die gewünschte Breite für das Fenster in Pixeln fest.

-xscrollincrement
Siehe die Option **xscrollincrement** des Widgets **canvas**.

-yscrollincrement
Siehe die Option **yscrollincrement** des Widgets **canvas**.

WIDGET-BEFEHL

pathName* cget *option
Gibt den aktuellen Wert der Konfigurationsoption zurück, die *option* angibt. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert.

***pathName* configure ?*option*? ?*value option value* ...?**
Fragt die Konfigurationsoptionen des Widgets ab oder ändert sie. Wenn keine *option* angegeben ist, gibt der Befehl eine Liste zurück, die alle verfügbaren Optionen für *pathName* beschreibt. Wenn *option* ohne *value* angegeben ist, gibt der Befehl eine Liste zurück, die die genannte *option* beschreibt (diese Liste ist identisch mit der entsprechenden Teilliste des Wertes, der zurückgegeben wird, wenn keine *option* angegeben ist). Wenn ein oder mehrere Paare aus *option* und *value* angegeben sind, ändert der Befehl die genannten Widget-Optionen auf die genannten Werte; in diesem Fall gibt der Befehl eine leere Zeichenkette zurück. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert. Optionen, die nur gelesen werden können, werden nicht geändert.

***pathName* getframe**
Gibt den Pfadnamen des gescrollten Rahmens zurück, in dem das Widget erzeugt werden sollte.

***pathName* see *widget* ?*vert*? ?*horz*?**
Ordnet den scrollbaren Bereich so an, dass *widget* im Fenster sichtbar ist. *vert* und *horz* geben an, welcher Teil von *widget* vorzugsweise sichtbar sein soll, wenn *widget* zu hoch oder zu breit ist, um ganz sichtbar zu sein. *vert* muss **top** (die Vorgabe) oder **bottom** sein, und *horz* muss **left** (die Vorgabe) oder **right** sein. Wenn *vert* oder *horz* kein gültiger Wert ist, wird der Bereich in dieser Richtung nicht gescrollt.

***pathName* xview ?*arg...*?**
Standardbefehl, um das waagerechte Scrollen von *pathName* zu ermöglichen.

***pathName* yview ?*arg...*?**
Standardbefehl, um das senkrechte Scrollen von *pathName* zu ermöglichen.

ScrolledWindow

Stichwörter: ScrolledWindow

NAME
ScrolledWindow - allgemeines gescrolltes Widget

ERZEUGUNG

ScrolledWindow *pathName* ?*option value...*?

STANDARDOPTIONEN

-background oder -bg	-borderwidth oder -bd
-relief	

WIDGET-SPEZIFISCHE OPTIONEN

-auto
-ipad
-managed
-scrollbar
-sides
-size

WIDGET-BEFEHL

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

pathName getframe

pathName setwidget *widget*

BESCHREIBUNG

ScrolledWindow ermöglicht, leicht ein Widget mit seiner Bildlaufleiste zu erzeugen. Die Bildlaufleisten erzeugt ScrolledWindow, und die Scrollbefehle werden mit **ScrolledWindow::setwidget** automatisch einem scrollbaren Widget zugeordnet.

WIDGET-SPEZIFISCHE OPTIONEN

-auto

Legt die gewünschte automatisch verwaltete Bildlaufleiste fest: none bedeutet, die Bildlaufleisten werden immer gezeichnet, horizontal bedeutet, die waagerechte Bildlaufleiste wird bei Bedarf gezeichnet, vertical bedeutet, die senkrechte Bildlaufleiste wird bei Bedarf gezeichnet, both bedeutet, waagerechte und senkrechte Bildlaufleisten werden bei Bedarf gezeichnet (Vorgabewert)

-ipad (nur lesbar)

Abstand in Pixeln zwischen dem Client-Widget und den Bildlaufleisten. Vorgabewert: **1**.

-managed (nur lesbar)
Wenn true, werden die Bildlaufleisten bei der Erzeugung verwaltet, ihre Größe geht also in die verlangte Größe des ScrolledWindow ein. Wenn false, dann nicht. Vorgabewert: **true**.

-scrollbar
Legt die gewünschte Bildlaufleiste fest: **none**, **horizontal**, **vertical** oder **both** (Vorgabewert).

-sides (nur lesbar)
Seite der Bildlaufleisten. Mögliche Werte sind: **ne**, **en**, **nw**, **wn**, **se** (Vorgabewert), **es**, **sw**, **ws**.

-size (nur lesbar)
Größe der Bildlaufleisten in Pixeln. 0 steht für die Standardgröße (Vorgabewert).

WIDGET-BEFEHL

pathName* cget *option
Gibt den aktuellen Wert der Konfigurationsoption zurück, die *option* angibt. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert.

***pathName* configure ?*option*? ?*value option value* ...?**
Fragt die Konfigurationsoptionen des Widgets ab oder ändert sie. Wenn keine *option* angegeben ist, gibt der Befehl eine Liste zurück, die alle verfügbaren Optionen für *pathName* beschreibt. Wenn *option* ohne *value* angegeben ist, gibt der Befehl eine Liste zurück, die die genannte *option* beschreibt (diese Liste ist identisch mit der entsprechenden Teilliste des Wertes, der zurückgegeben wird, wenn keine *option* angegeben ist). Wenn ein oder mehrere Paare aus *option* und *value* angegeben sind, ändert der Befehl die genannten Widget-Optionen auf die genannten Werte; in diesem Fall gibt der Befehl eine leere Zeichenkette zurück. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert. Optionen, die nur gelesen werden können, werden nicht geändert.

***pathName* getframe**
Gibt den Pfadnamen des Rahmens zurück, in dem das gescrollte Widget erzeugt werden sollte. Dieser Befehl wird nicht mehr gebraucht. Das gescrollte Widget kann unmittelbar als Kind von *pathName* erzeugt werden.

pathName* setwidget *widget
Ordnet *widget* den Bildlaufleisten zu. *widget* wird vom ScrolledWindow verwaltet. *widget* sollte nicht selbst verwaltet werden, solange das ScrolledWindow es verwaltet. *widget* muss ein scrollbares Widget sein, das heißt die Optionen **xscrollcommand**/**yscrollcommand** und die Befehle **xview**/**yview** haben, wie canvas oder text.

SelectColor

Stichwörter: SelectColor

NAME
SelectColor - Widget zur Farbauswahl

ERZEUGUNG
SelectColor *pathName* ?*option value...*?

WIDGET-SPEZIFISCHE OPTIONEN

-color	-parent
-placement	-title
-type	

WIDGET-BEFEHL

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

SelectColor::dialog *pathName?option value ...?*

SelectColor::menu *pathName placement?option value ...?*

SelectColor::setcolor *index color*

BESCHREIBUNG

SelectColor bietet einen einfachen Weg, eine Farbe auszuwählen. Es kann als Dialog oder als Menüschaltfläche angezeigt werden.

WIDGET-SPEZIFISCHE OPTIONEN

-color
Legt den Farbwert des Widgets fest.

-parent
Elternteil des Dialogs. Der Dialog wird in seinem Elternteil zentriert. Wenn der Wert leer ist, wird er im Wurzel-Fenster zentriert.

-placement
Wo der Farbdialog *popup* beim Anzeigen platziert wird. Muss einer der Werte sein: **at**, **center**, **left**, **right**, **above** oder **below**. Wenn *-parent* angegeben ist, gilt die Platzierung relativ zum Eltern-Widget.

-title
Titel des Dialog-Toplevel-Fensters.

-type (nur lesbar)
Legt den Typ des SelectColor-Widgets fest. Muss **dialog** oder **popup** sein. Wenn die Option **type** *dialog* ist, erzeugt SelectColor::**create** den Dialog unmittelbar, zeigt ihn an und gibt eine leere Zeichenkette zurück, wenn die Schaltfläche cancel gedrückt oder der Dialog zerstört wird, und die gewählte Farbe, wenn die Schaltfläche ok gedrückt wird. In allen Fällen wird der Dialog zerstört. Wenn die Option **type** *popup* ist, erzeugt SelectColor::**create** einen kleinen Popup-Dialog mit einer kleinen Menge vordefinierter Farben und einer Schaltfläche, die einen vollständigen Farbdialog öffnet.

WIDGET-BEFEHL

pathName* cget *option
Gibt den aktuellen Wert der Konfigurationsoption zurück, die *option* angibt. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert.

***pathName* configure ?*option*? ?*value option value ...*?**
Fragt die Konfigurationsoptionen des Widgets ab oder ändert sie. Wenn keine *option* angegeben ist, gibt der Befehl eine Liste zurück, die alle verfügbaren Optionen für *pathName* beschreibt. Wenn *option* ohne *value* angegeben ist, gibt der Befehl eine Liste zurück, die die genannte *option* beschreibt (diese Liste ist identisch mit der entsprechenden Teilliste des Wertes, der zurückgegeben wird, wenn keine *option* angegeben ist). Wenn ein oder mehrere Paare aus *option* und *value* angegeben sind, ändert der Befehl die genannten Widget-Optionen auf die genannten Werte; in diesem Fall gibt der Befehl eine leere Zeichenkette zurück. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert. Optionen, die nur gelesen werden können, werden nicht geändert.

SelectColor::dialog *pathName?option value ...?*
Erzeugt einen Dialog, in dem eine eigene Farbe gewählt wird.

SelectColor::menu *pathName placement?option value ...?*
Erzeugt einen kleinen Popup-Dialog, in dem aus einer vordefinierten Farbliste gewählt wird, mit einer zusätzlichen Schaltfläche, die einen vollständigen Farbdialog anzeigt.

placement kann einer der Werte **at**, **center**, **left**, **right**, **above** oder **below** sein. Wenn *-parent* angegeben ist, gilt die Platzierung relativ zum Eltern-Widget.

SelectColor::setcolor *index color*
Setzt den Wert der vom Benutzer vordefinierten Farbe am Index *index* auf *color*. *index* muss zwischen 0 und 10 liegen.

SelectFont

Stichwörter: SelectFont

NAME
SelectFont - Widget zur Schriftauswahl

ERZEUGUNG
SelectFont *pathName* ?*option value...*?

STANDARDOPTIONEN

-background oder -bg	-font

WIDGET-SPEZIFISCHE OPTIONEN
-command

WIDGET-BEFEHL
pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

SelectFont::loadfont

BESCHREIBUNG

SelectFont bietet einen einfachen Weg, eine Schriftart zu wählen. Es kann als Dialog oder als Werkzeugleiste angezeigt werden.
Die Texteinträge im Dialog verwenden die Option **-name**, damit sie in jede Sprache übersetzt werden können. Die verwendeten symbolischen Namen sind **ok**, **cancel**, **font**, **size**, **style**, **bold**, **italic**, **underline** und **overstrike**.

WIDGET-SPEZIFISCHE OPTIONEN

-command
Legt einen Befehl fest, der aufgerufen wird, wenn eine neue Schriftart gewählt wird und die Option **type** von SelectFont *toolbar* ist.

-initialcolor
Wenn angegeben, kommt eine zusätzliche Schaltfläche hinzu, mit der eine Farbe gewählt werden kann. Diese Option wird ignoriert, wenn **type** *toolbar* ist.

-nosizes
Wenn true, wird die Listbox mit den gültigen Schriftgrößen nicht angezeigt. Diese Option wird ignoriert, wenn **type** *toolbar* ist.

-parent
Elternteil des Dialogs. Der Dialog wird in seinem Elternteil zentriert. Wenn der Wert leer ist, wird er im Wurzel-Fenster zentriert.

-sampletext
Legt den Text fest, der im Vorschaubereich angezeigt wird.

-title
Titel des Dialog-Toplevel-Fensters.

-type
Legt den Typ des SelectFont-Widgets fest. Muss **dialog** oder **toolbar** sein. Wenn die Option **type** *dialog* ist, erzeugt SelectFont::**create** den Dialog unmittelbar, zeigt ihn an und gibt eine leere Zeichenkette zurück, wenn die Schaltfläche cancel gedrückt oder der Dialog zerstört wird, und die gewählte Schriftart (und die Farbe, wenn **initialcolor**), wenn die Schaltfläche ok gedrückt wird. In allen Fällen wird der Dialog zerstört. Wenn die Option **type** *toolbar* ist, gibt SelectFont::**create** den Pfadnamen des erzeugten Widgets zurück.

WIDGET-BEFEHL

pathName* cget *option
Gibt den aktuellen Wert der Konfigurationsoption zurück, die *option* angibt. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert.

***pathName* configure ?*option*? ?*value option value ...*?**
Fragt die Konfigurationsoptionen des Widgets ab oder ändert sie. Wenn keine *option* angegeben ist, gibt der Befehl eine Liste zurück, die alle verfügbaren Optionen für *pathName* beschreibt. Wenn *option* ohne *value* angegeben ist, gibt der Befehl eine Liste zurück, die die genannte *option* beschreibt (diese Liste ist identisch mit der entsprechenden Teilliste des

Wertes, der zurückgegeben wird, wenn keine *option* angegeben ist). Wenn ein oder mehrere Paare aus *option* und *value* angegeben sind, ändert der Befehl die genannten Widget-Optionen auf die genannten Werte; in diesem Fall gibt der Befehl eine leere Zeichenkette zurück. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert. Optionen, die nur gelesen werden können, werden nicht geändert.

SelectFont::loadfont
Lädt die auf dem System verfügbaren Schriftarten.

Separator

Stichwörter: Separator

NAME
Separator - 3D-Trenn-Widget

ERZEUGUNG
Separator *pathName* ?*option value...*?

STANDARDOPTIONEN

-background oder -bg	-orient

WIDGET-SPEZIFISCHE OPTIONEN
-relief

WIDGET-BEFEHL
pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

BESCHREIBUNG

Separator ist ein Widget, das eine waagerechte oder senkrechte 3D-Linie anzeigt.

WIDGET-SPEZIFISCHE OPTIONEN

-relief
Legt das Relief des Separators fest. Muss **groove** (die Vorgabe) oder **ridge** sein.

WIDGET-BEFEHL

pathName* cget *option
Gibt den aktuellen Wert der Konfigurationsoption zurück, die *option* angibt. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert.

***pathName* configure ?*option*? ?*value option value* ...?**
Fragt die Konfigurationsoptionen des Widgets ab oder ändert sie. Wenn keine *option* angegeben ist, gibt der Befehl eine Liste zurück, die alle verfügbaren Optionen für *pathName* beschreibt. Wenn *option* ohne *value* angegeben ist, gibt der Befehl eine Liste zurück, die die genannte *option* beschreibt (diese Liste ist identisch mit der entsprechenden Teilliste des

Wertes, der zurückgegeben wird, wenn keine *option* angegeben ist). Wenn ein oder mehrere Paare aus *option* und *value* angegeben sind, ändert der Befehl die genannten Widget-Optionen auf die genannten Werte; in diesem Fall gibt der Befehl eine leere Zeichenkette zurück. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert. Optionen, die nur gelesen werden können, werden nicht geändert.

SpinBox

Stichwörter: SpinBox

NAME

SpinBox - SpinBox-Widget

ERZEUGUNG

SpinBox *pathName* ?*option value...*?

OPTIONEN VON ArrowButton

-background oder -bg	-disabledforeground
-foreground oder -fg	-repeatdelay
-repeatinterval	-state

OPTIONEN VON Entry

-command	-disabledforeground
-dragenabled	-dragendcmd
-dragevent	-draginitcmd
-dragtype	-dropcmd
-dropenabled	-dropovercmd
-droptypes	-editable
-entrybg (siehe -background)	-entryfg (siehe -foreground)
-exportselection	-font
-helptext	-helptype
-helpvar	-highlightbackground
-highlightcolor	-highlightthickness
-insertbackground	-insertborderwidth
-insertofftime	-insertontime
-insertwidth	-justify

-selectbackground	-selectborderwidth
-selectforeground	-show
-state	-takefocus
-text	-textvariable
-width	-xscrollcommand

WIDGET-SPEZIFISCHE OPTIONEN

-modifycmd
-range
-values

WIDGET-BEFEHL

pathName bind ?*arg...*?

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

pathName getvalue

pathName setvalue *index*

BESCHREIBUNG

Das Widget SpinBox ermöglicht, einen Wert aus einer Liste zu wählen, die die Option **values** angibt, oder aus einer Menge von Werten, die durch ein Minimum, ein Maximum und ein Inkrement definiert ist. Die Option **range** definiert eine Liste von Werten, deshalb arbeiten **getvalue** und **setvalue** sowohl mit values als auch mit range.

WIDGET-SPEZIFISCHE OPTIONEN

-modifycmd
Legt einen Tcl-Befehl fest, der aufgerufen wird, wenn der Wert der SpinBox geändert wird.

-range
Legt eine Liste aus drei ganzen Zahlen (oder reellen Zahlen) fest, die Minimum, Maximum und Inkrement der SpinBox beschreiben.

-values
Legt die Werte fest, die die SpinBox akzeptiert. Diese Option hat Vorrang vor der Option **range**.

WIDGET-BEFEHL

***pathName* bind ?*arg...*?**
Setzt Bindungen auf dem Entry-Widget.

pathName* cget *option
Gibt den aktuellen Wert der Konfigurationsoption zurück, die *option* angibt. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert.

***pathName* configure ?*option*? ?*value option value ...*?**
Fragt die Konfigurationsoptionen des Widgets ab oder ändert sie. Wenn keine *option* angegeben ist, gibt der Befehl eine Liste zurück, die alle verfügbaren Optionen für *pathName* beschreibt. Wenn *option* ohne *value* angegeben ist, gibt der Befehl eine Liste zurück, die die genannte *option* beschreibt (diese Liste ist identisch mit der entsprechenden Teilliste des Wertes, der zurückgegeben wird, wenn keine *option* angegeben ist). Wenn ein oder mehrere Paare aus *option* und *value* angegeben sind, ändert der Befehl die genannten Widget-Optionen auf die genannten Werte; in diesem Fall gibt der Befehl eine leere Zeichenkette zurück. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert. Optionen, die nur gelesen werden können, werden nicht geändert.

***pathName* getvalue**
Gibt den Index des aktuellen Textes der SpinBox in der Werteliste zurück, oder -1, wenn er zu keinem Wert passt.

pathName* setvalue *index
Setzt den Text der SpinBox auf den Wert, den *index* in der Werteliste angibt. *index* kann in jeder der folgenden Formen angegeben werden:

last
Legt das letzte Element der Werteliste fest.

first
Legt das erste Element der Werteliste fest.

next
Legt das Element fest, das auf das aktuelle folgt (das heißt das von **getvalue** zurückgegebene) in der Werteliste.

previous
Legt das Element fest, das dem aktuellen vorausgeht (das heißt das von **getvalue** zurückgegebene) in der Werteliste.

@number
Legt den ganzzahligen Index in der Werteliste fest.

BINDUNGEN

Wenn das Entry der SpinBox den Eingabefokus hat, hat es die folgenden Bindungen, zusätzlich zu den Vorgabebindungen von Entry:

- Die Taste "Bild auf" setzt den Wert der SpinBox auf den letzten Wert.
- Die Taste "Bild ab" setzt den Wert der SpinBox auf den ersten Wert.
- Die Taste "Pfeil nach oben" setzt den Wert der SpinBox auf den nächsten Wert.
- Die Taste "Pfeil nach unten" setzt den Wert der SpinBox auf den vorherigen Wert.

StatusBar

Stichwörter: StatusBar

NAME

StatusBar - Statusleisten-Widget

ERZEUGUNG

StatusBar *pathName* ?*option value...*?

STANDARDOPTIONEN

-background oder -bg	-borderwidth oder -bd
-foreground oder -fg	-orient
-relief	
-troughcolor	

WIDGET-SPEZIFISCHE OPTIONEN

-height
-showresize
-ipad
-pad
-width

WIDGET-BEFEHL

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

BESCHREIBUNG

Das Widget StatusBar ist ein einfacher Container mit einer Ecke zur Größenänderung, gedacht für den unteren Rand eines Toplevel-Dialogs.

WIDGET-SPEZIFISCHE OPTIONEN

-height

Legt die gewünschte Höhe für das Widget fest.

-showresize

Legt fest, ob die Ecke zur Größenänderung angezeigt wird.

-pad

Der Abstand um die Statusleiste.

-ipad
Der Abstand um die einzelnen Einträge der Statusleiste.

-width
Legt die gewünschte Breite für das Widget fest.

WIDGET-BEFEHL

***pathName* add widget ?*option value option value ...*?**
Fügt der Statusleiste ein Widget hinzu. Mögliche Optionen sind:

-weight
Gewichtung dieses Eintrags bei der Größenänderung (wird an **grid** übergeben).

-separator
Ob für diesen Eintrag ein Trenner verwendet wird.

-sticky
Wird an grid weitergegeben.

-pad
Überschreibt die Option **-ipad** des Widgets für diesen Eintrag.

pathName* cget *option
Gibt den aktuellen Wert der Konfigurationsoption zurück, die *option* angibt. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert.

***pathName* configure ?*option*? ?*value option value ...*?**
Fragt die Konfigurationsoptionen des Widgets ab oder ändert sie. Wenn keine *option* angegeben ist, gibt der Befehl eine Liste zurück, die alle verfügbaren Optionen für *pathName* beschreibt. Wenn *option* ohne *value* angegeben ist, gibt der Befehl eine Liste zurück, die die genannte *option* beschreibt (diese Liste ist identisch mit der entsprechenden Teilliste des Wertes, der zurückgegeben wird, wenn keine *option* angegeben ist). Wenn ein oder mehrere Paare aus *option* und *value* angegeben sind, ändert der Befehl die genannten Widget-Optionen auf die genannten Werte; in diesem Fall gibt der Befehl eine leere Zeichenkette zurück. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert. Optionen, die nur gelesen werden können, werden nicht geändert.

***pathName* delete widget ?*widget ...*?**
Löscht ein Widget und den zugehörigen Zustand aus der Statusleiste.

***pathName* getframe**
Holt das Rahmen-Widget der Statusleiste, in dem die Einträge der Statusleiste erzeugt werden sollen.

***pathName* items**
Liste der Einträge in der Statusleiste.

TitleFrame

Stichwörter: TitleFrame

NAME
TitleFrame - Rahmen mit einem Titel

ERZEUGUNG

TitleFrame *pathName* ?*option value...*?

STANDARDOPTIONEN

-background oder -bg	-borderwidth oder -bd
-font	
-relief	

WIDGET-SPEZIFISCHE OPTIONEN

-baseline

WIDGET-BEFEHL

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

pathName getframe

BESCHREIBUNG

TitleFrame ermöglicht, einen Rahmen mit einem Titel zu erzeugen, ähnlich dem Motif-Widget XmFrame.

WIDGET-SPEZIFISCHE OPTIONEN

-baseline

Legt die senkrechte Ausrichtung des Titels fest: **top**, **center** oder **bottom**.

-ipad

Legt einen Abstand zwischen dem Rand des Rahmens und dem Benutzerrahmen fest. Der Wert ist in Bildschirmeinheiten.

-side

Legt die waagerechte Ausrichtung des Titels fest: **left**, **center** oder **right**.

WIDGET-BEFEHL

pathName* cget *option

Gibt den aktuellen Wert der Konfigurationsoption zurück, die *option* angibt. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert.

***pathName* configure ?*option*? ?*value option value* ...?**

Fragt die Konfigurationsoptionen des Widgets ab oder ändert sie. Wenn keine *option* angegeben ist, gibt der Befehl eine Liste zurück, die alle verfügbaren Optionen für *pathName* beschreibt. Wenn *option* ohne *value* angegeben ist, gibt der Befehl eine Liste zurück, die die genannte *option* beschreibt (diese Liste ist identisch mit der entsprechenden Teilliste des Wertes, der zurückgegeben wird, wenn keine *option* angegeben ist). Wenn ein oder mehrere Paare aus *option* und *value* angegeben sind, ändert der Befehl die genannten Widget-Optionen auf die genannten Werte; in diesem Fall gibt der Befehl eine leere

Zeichenkette zurück. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert. Optionen, die nur gelesen werden können, werden nicht geändert.

***pathName* getframe**

Gibt den Rahmen zurück, in dem jedes andere Widget erzeugt werden kann.

Tree

Stichwörter: Tree

NAME

Tree - Tree-Widget

ERZEUGUNG

Tree *pathName* ?*option value...*?

STANDARDOPTIONEN

-background oder -bg	-borderwidth oder -bd
-cursor	-highlightbackground
-highlightcolor	-highlightthickness
-relief	-selectbackground
-selectforeground	-takefocus
-xscrollcommand	-yscrollcommand

WIDGET-SPEZIFISCHE OPTIONEN

-closecmd	-crossfill
-crossclosebitmap	-crosscloseimage
-crossopenbitmap	-crossopenimage
-deltax	-deltay
-dragenabled	-dragendcmd
-dragevent	-draginitcmd
-dragtype	-dropcmd
-dropenabled	-dropovercmd
-dropovermode	-droptypes
-height	-linesfill
-linestipple	-opencmd
-padx	-redraw

-selectcommand	-selectfill
-showlines	-width

WIDGET-BEFEHL

pathName bindArea *event script*

pathName bindImage *event script*

pathName bindText *event script*

pathName cget *option*

pathName closetree *node*

pathName configure ?*option*? ?*value option value* ...?

pathName delete ?*arg...*?

pathName edit *node text* ?*verifycmd*? ?*clickres*? ?*select*?

pathName exists *node*

pathName find *findinfo* ?*confine*?

pathName index *node*

pathName insert *index parent node* ?*option value...*?

pathName itemcget *node option*

pathName itemconfigure *node* ?*option*? ?*value option value* ...?

pathName line *node*

pathName move *parent node index*

pathName nodes *node* ?*first*? ?*last*?

pathName opentree *node*

pathName parent *node*

pathName reorder *node neworder*

pathName see *node*

pathName selection *cmd* ?*arg...*?

pathName toggle *node*

pathName visible *node*

pathName xview ?*arg...*?

pathName yview ?*arg...*?

BESCHREIBUNG

Das Widget **Tree** verwendet canvas, um eine hierarchische Liste von Einträgen (Knoten genannt) anzuzeigen. Jeder Knoten besteht aus einer Beschriftung mit eigener Schriftart und eigenen Vordergrundattributen und einem optionalen Bild oder Fenster. Jeder Knoten kann eine Liste von Unterknoten haben, die eingeklappt oder ausgeklappt werden können. Jeder Knoten wird in einer einzigen Zeile gezeichnet, deren Höhe die Option **deltay** festlegt, sie dürfen also höchstens diese Höhe haben. Ein Knoten wird eindeutig durch eine Zeichenkette gekennzeichnet, die bei der Erzeugung (durch den Befehl **insert**) angegeben wird. Der Knoten namens *root* ist die Wurzel des Baums und wird nicht gezeichnet. Die Baumstruktur hält das Widget selbst.

WIDGET-SPEZIFISCHE OPTIONEN

-closecmd
Legt einen Befehl fest, der aufgerufen wird, wenn ein Knoten geschlossen wird. Der geschlossene Knoten wird an den Befehl angehängt.

-crossfill
Legt eine Vordergrundfarbe für die Kreuz-Bitmap fest.

-crossclosebitmap
Legt eine Bitmap fest, die anstelle des Standardkreuzes angezeigt wird, wenn ein Knoten geschlossen ist.

-crosscloseimage
Legt ein Bild fest, das anstelle des Standardkreuzes angezeigt wird, wenn ein Knoten geschlossen ist. Setzt die Option -crossclosebitmap außer Kraft.

-crossopenbitmap
Legt eine Bitmap fest, die anstelle des Standardkreuzes angezeigt wird, wenn ein Knoten offen ist.

-crossopenimage
Legt ein Bild fest, das anstelle des Standardkreuzes angezeigt wird, wenn ein Knoten offen ist. Setzt die Option -crossopenbitmap außer Kraft.

-deltax
Legt die waagerechte Einrückung zwischen einem Knoten und seinen Kindern fest.

-deltay
Legt die senkrechte Größe der Knoten fest.

-dragenabled
Ein boolescher Wert, der angibt, ob Ziehen eingeschaltet ist.

-dragendcmd
Legt einen Befehl fest, der aufgerufen wird, wenn das Ziehen endet. **dragendcmd** muss ein Befehl sein, der der Beschreibung der Option **dragendcmd** von **DragSite::register** entspricht.

-dragevent
Legt die Nummer der Maustaste fest, die dem Ziehen zugeordnet ist. Muss **1**, **2** oder **3** sein.

-draginitcmd
Tree hat einen Befehlsrahmen für Ereignisse *drag-init*. Dieser Befehl lehnt das Ziehen ab, wenn kein Knoten bezeichnet ist. In den anderen Fällen: Wenn **draginitcmd** leer ist, gibt er zurück:

- als Datentyp den Wert der Option **dragtype** oder *TREE_NODE*, wenn er leer ist,
- *{copy move link}* als die Operationen,
- den Bezeichner des Knotens als die Daten.

Wenn **draginitcmd** nicht leer ist, wird er mit den folgenden Argumenten aufgerufen:

- der Pfadname des Baums,
- der Bezeichner des gezogenen Knotens,
- das Toplevel-Fenster, das erzeugt wird, um die gezogenen Daten darzustellen.

und muss einen Wert zurückgeben, der der Option **draginitcmd** entspricht, die in **DragSite::register** beschrieben ist.

-dragtype
Legt einen anderen Typ des gezogenen Objekts fest.

-dropcmd
Tree hat einen Befehlsrahmen für Ereignisse *drop*. Dieser Befehl beendet das automatische Scrollen und ermittelt Knoten und Position. Wenn **dropcmd** nicht leer ist, wird er mit den folgenden Argumenten aufgerufen:

- der Pfadname des Baums,
- der Pfadname der Ziehquelle,
- eine Liste, die beschreibt, wo das Ablegen geschieht. Sie kann sein:
- die aktuelle Operation,
- der Datentyp,
- die Daten.
  - *{***widget***}*,
  - *{***node** *node}* oder
  - *{***position** *node index}*.

-dropenabled
Ein boolescher Wert, der angibt, ob Ablegen eingeschaltet ist.

-dropovercmd

Tree hat einen Befehlsrahmen für Ereignisse *drag-over*. Dieser Befehl schaltet während des *drag-over* das automatische Scrollen und die Ermittlung der Position ein. Wenn **dropovercmd** nicht leer ist, wird der Befehl mit den folgenden Argumenten aufgerufen:

- der Pfadname des Baums,
- der Pfadname der Ziehquelle,
- eine Liste, die beschreibt, wo das Ablegen geschehen kann; ihre Elemente sind:
- die aktuelle Operation,
- der Datentyp,
- die Daten.
  - die Zeichenkette *widget*, wenn die Option **dropovertype** *w* enthält, sonst die leere Zeichenkette.
  - der angezielte Knoten, wenn das Ziehsymbol auf einen Knoten zeigt und die Option **dropovertype** *n* enthält, sonst die leere Zeichenkette.
  - eine Liste mit einem Knoten und der Position innerhalb der Kinder des Knotens, auf die das Ziehsymbol zeigt, wenn die Option **dropovertype** *p* enthält, sonst die leere Zeichenkette.
  - optional die bevorzugte Methode, wenn das Ablegen sowohl innerhalb eines Knotens als auch zwischen zwei Knoten geschehen kann. Der Wert ist *position* oder *node*.

Der Befehl muss eine Liste mit zwei Elementen zurückgeben:

- den Ablegezustand, entsprechend der Beschreibung in der Option **dropovercmd** von **DropSite::register**,
- die gewählte Methode: *widget*, *node* oder *position*.

-dropovermode
Legt den Typ der Wechselwirkung *drop-over* fest. Muss eine Kombination aus **w** sein, das festlegt, dass das Ablegen überall auf dem Widget geschehen kann, **p**, das festlegt, dass das Ablegen zwischen zwei Knoten geschehen kann, und **n**, das festlegt, dass das Ablegen innerhalb von Knoten geschieht.

-droptypes
Legt eine Liste der akzeptierten abgelegten Objekte und Operationen fest. Weitere Angaben stehen bei der Option **droptypes** von **DropSite::register**. Die Vorgabe ist *TREE_NODE* mit den Operationen **copy** und **move**.

-height
Legt die gewünschte Höhe des Baums in Einheiten von **deltay** Pixeln fest.

-linesfill
Legt eine Vordergrundfarbe für die Linien zwischen den Knoten fest.

-linestipple
Legt eine Schraffur-Bitmap für die Linien zwischen den Knoten fest.

-opencmd
Legt einen Befehl fest, der aufgerufen wird, wenn ein Knoten geöffnet wird. Der Name des geöffneten Knotens wird an den Befehl angehängt.

-padx
Legt den Abstand zwischen Bild oder Fenster und dem Text der Knoten fest.

-redraw

Legt fest, ob der Baum neu gezeichnet werden soll, wenn der Leerlauf beginnt. Auf false setzen, wenn **update** aufgerufen wird, während der Baum geändert wird.

-selectcommand
Legt einen Befehl fest, der aufgerufen wird, wenn sich die Auswahl ändert. Der Pfad des Tree-Widgets und die ausgewählten Knoten werden an den Befehl angehängt.

-selectfill
Wenn true, wird der Auswahlrahmen über den gesamten Baum von links nach rechts gezeichnet, anstatt nur um den Text des Eintrags.

-showlines
Legt fest, ob Linien zwischen den Knoten gezeichnet werden sollen.

-width
Legt die gewünschte Breite des Baums in Einheiten von 8 Pixeln fest.

KNOTENNAMEN

Bestimmte besondere Zeichen in Knotennamen ersetzt der Baum während des Betriebs automatisch. Diese Zeichen sind **& | ^!**. Sie werden alle durch ein _ Zeichen ersetzt. Das dient nur dazu, Fehler zu vermeiden, weil die Zeichen für das Tree-Widget besonders sind.

WIDGET-BEFEHL

pathName* bindArea *event script
Dieser Befehl ordnet einen Befehl zu, der ausgeführt wird, wenn die von *event* angegebene Ereignisfolge irgendwo im Bereich des Tree eintritt.

pathName* bindImage *event script
Dieser Befehl ordnet einen Befehl zu, der ausgeführt wird, wenn die von *event* angegebene Ereignisfolge auf dem Bild eines Knotens eintritt. Der Knotenbezeichner, auf dem das Ereignis eintritt, wird an den Befehl angehängt und kann verwendet werden, um den Baum zu bearbeiten (zum Beispiel **%W** nicht verwenden).

Wenn **-selectfill** angegeben ist, wird eine etwaige Bindung des Hintergrundrahmens durch **bindText** überschrieben.

pathName* bindText *event script
Dieser Befehl ordnet einen Befehl zu, der ausgeführt wird, wenn die von *event* angegebene Ereignisfolge auf der Beschriftung eines Knotens eintritt. Der Knotenbezeichner, auf dem das Ereignis eintritt, wird an den Befehl angehängt und kann verwendet werden, um den Baum zu bearbeiten (zum Beispiel **%W** nicht verwenden).

Wenn **-selectfill** angegeben ist, wird eine etwaige Bindung des Hintergrundrahmens durch **bindImage** überschrieben.

pathName* cget *option
Gibt den aktuellen Wert der Konfigurationsoption zurück, die *option* angibt. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert.

***pathName* closetree *node* ?*recurse*?**
Dieser Befehl schließt den gesamten Teilbaum, den *node* angibt. Er geht den Baum von *node* aus durch und setzt die Option **open** auf 0, abhängig von *recurse*. Der Vorgabewert von

recurse ist *true*.

***pathName* configure ?*option*? ?*value option value ...*?**
Fragt die Konfigurationsoptionen des Widgets ab oder ändert sie. Wenn keine *option* angegeben ist, gibt der Befehl eine Liste zurück, die alle verfügbaren Optionen für *pathName* beschreibt. Wenn *option* ohne *value* angegeben ist, gibt der Befehl eine Liste zurück, die die genannte *option* beschreibt (diese Liste ist identisch mit der entsprechenden Teilliste des Wertes, der zurückgegeben wird, wenn keine *option* angegeben ist). Wenn ein oder mehrere Paare aus *option* und *value* angegeben sind, ändert der Befehl die genannten Widget-Optionen auf die genannten Werte; in diesem Fall gibt der Befehl eine leere Zeichenkette zurück. *Option* kann jeden Wert haben, den der Erzeugungsbefehl akzeptiert. Optionen, die nur gelesen werden können, werden nicht geändert.

***pathName* delete ?*arg...*?**
Löscht alle Knoten (und ihre Kinder) in *arg*. *arg* kann eine Liste von Knoten oder eine Liste von Listen von Knoten sein. Um den ganzen Baum zu löschen, dient *$pathName delete [$pathName nodes root]*.

***pathName* edit *node text* ?*verifycmd*? ?*clickres*? ?*select*?**
Bietet einen Weg, die Beschriftung eines Knotens an Ort und Stelle zu bearbeiten. Das ist nur möglich, wenn *node* sichtbar ist (alle seine Eltern sind offen). Der Befehl nimmt den Anfangstext als Argument und ändert die Beschriftung des bearbeiteten Knotens nicht, gibt aber eine leere Zeichenkette zurück, wenn die Bearbeitung abgebrochen wird, oder den eingegebenen Text, wenn die Bearbeitung angenommen wird. Während der Bearbeitung kann mit Escape abgebrochen und mit Return angenommen werden. *clickres* legt fest, was geschieht, wenn außerhalb des bearbeitbaren Bereichs geklickt wird. Wenn *clickres* 0 ist (die Vorgabe), wird die Bearbeitung abgebrochen. Wenn *clickres* 1 ist, wird die Bearbeitung angenommen. In allen anderen Fällen geht die Bearbeitung weiter. Wenn die Bearbeitung angenommen wird und *modifycmd* nicht leer ist, wird er mit dem neuen Text als Argument aufgerufen und muss 1 zurückgeben, um den neuen Text anzunehmen, oder 0, um ihn abzulehnen und die Bearbeitung fortzusetzen. *select* legt fest, ob der Anfangstext ausgewählt werden soll. Die Vorgabe ist 1.

pathName* exists *node
Gibt zurück, ob *node* im Baum existiert.

***pathName* find *findinfo* ?*confine*?**
Gibt den Knoten zurück, den die Position *findinfo* angibt. *findinfo* kann die Form einer Pixelposition *@x,y* oder der Zeilennummer eines gerade sichtbaren Tree-Knotens haben. Die erste Zeile des Tree hat den Wert null.

Wenn *confine* nicht leer ist, wird *findinfo* darauf beschränkt, nur Pixelpositionen in der Fläche der Tree-Beschriftungen zu treffen, nicht irgendwo auf ihren Zeilen. (*confine* hat keine Wirkung, wenn *findinfo* eine Zeilennummer ist.)

pathName* index *node
Gibt die Position von *node* in seinem Elternteil zurück.

***pathName* insert *index parent node* ?*option value...*?**
Fügt einen neuen Knoten, der durch *node* gekennzeichnet ist, in die Kinderliste von *parent* an der Position *index* ein.

Jedes Vorkommen von *#auto* im Knotennamen wird durch die Nummer des Eintrags in der Reihenfolge des Einfügens ersetzt. Die nicht druckbaren Zeichen \1 to \5 sind für den internen Gebrauch reserviert und sollen in Knotennamen nicht vorkommen.

-anchor
Legt den Anker des Bildes oder Fensters des Knotens fest. Voreingestellt ist w.

-data
Benutzerdaten, die zum Knoten gehören.

-deltax
Legt die waagerechte Einrückung des Knotens fest. Wenn der Wert -1 ist, wird der Knoten mit dem deltax des gesamten Baums gezeichnet.

-drawcross
Legt fest, wie das Kreuz gezeichnet wird, mit dem die Kinder eines Knotens ausgeklappt oder eingeklappt werden. Muss einer der Werte **auto**, **always** oder **never** sein. Bei **auto** wird das Kreuz nur gezeichnet, wenn der Knoten Kinder hat. Bei **always** wird das Kreuz immer gezeichnet. Bei **never** wird das Kreuz nie gezeichnet. Zur Kompatibilität mit älteren Versionen dieses Widgets ist **allways** ein veraltetes Synonym für **always**.

-fill
Legt die Vordergrundfarbe der Beschriftung des Knotens fest.

-font
Legt eine Schriftart für die Beschriftung des Knotens fest.

-helpcmd
Wenn angegeben, verweist das auf einen Befehl, der ausgeführt wird, um den anzuzeigenden Hilfetext zu holen. Der Befehl muss eine Zeichenkette zur Anzeige zurückgeben. Wenn der Befehl eine leere Zeichenkette zurückgibt, wird keine Hilfe angezeigt. Siehe auch DynamicHelp.

-helptext
Text für die dynamische Hilfe. Siehe auch DynamicHelp.

-helptype
Typ der dynamischen Hilfe. *balloon* oder *variable* verwenden. Siehe auch DynamicHelp.

-helpvar
Variable, die verwendet wird, wenn die Option **helptype** *variable* ist. Siehe auch DynamicHelp.

-image
Legt ein Bild fest, das links von der Beschriftung des Knotens angezeigt wird. Die Option **window** setzt **image** außer Kraft.

-open
Legt fest, ob die Kinder des Knotens gezeichnet werden sollen.

-padx
Legt den Abstand zwischen Bild oder Fenster und dem Text des Knotens fest. Wenn der Wert -1 ist, wird der Knoten mit dem padx des gesamten Baums gezeichnet.

-selectable

Legt fest, ob der Knoten ausgewählt werden kann.

-text
Legt die Beschriftung des Knotens fest.

-window
Legt einen Pfadnamen fest, der links von der Beschriftung des Knotens angezeigt wird. Die Option **window** setzt **image** außer Kraft.

pathName* itemcget *node option
Gibt den aktuellen Wert einer Konfigurationsoption für den Eintrag zurück. *Option* kann jeden Wert haben, den der Erzeugungsbefehl des Eintrags akzeptiert.

***pathName* itemconfigure *node* ?*option*? ?*value option value ...*?**
Dieser Befehl entspricht dem Befehl **configure**, er gilt aber für die Optionen eines einzelnen Eintrags, während **configure** für die Optionen des Widgets als Ganzes gilt. **Options** kann jeden Wert haben, den der Widget-Befehl zur Erzeugung des Eintrags akzeptiert. Wenn Optionen angegeben sind, werden sie geändert, wie der Befehl angibt, und der Befehl gibt eine leere Zeichenkette zurück. Wenn keine Optionen angegeben sind, gibt der Befehl eine Liste zurück, die die aktuellen Optionen des Eintrags beschreibt. Optionen, die nur gelesen werden können, werden nicht geändert.

pathName* line *node
Gibt die Zeilennummer zurück, in der *node* gezeichnet wurde. Wenn der Knoten nicht sichtbar ist, wird -1 zurückgegeben. Die erste Zeile des Baums hat den Wert 0.

pathName* move *parent node index
Verschiebt *node* in die Kinderliste von *parent* an die Position *index*. *parent* kann kein Nachkomme von *node* sein.

***pathName* nodes *node* ?*first*? ?*last*?**
Gibt Teile der Kinder von *node* zurück, entsprechend *first* und *last*. Wenn *first* und *last* weggelassen werden, gibt der Befehl die Liste aller Kinder zurück. Wenn *first* angegeben und *last* weggelassen wird, gibt der Befehl das Kind am Index *first* zurück, oder eine leere Zeichenkette, wenn *first* auf ein nicht vorhandenes Element zeigt. Wenn *first* und *last* angegeben sind, gibt der Befehl eine Liste zurück, deren Elemente alle Kinder zwischen *first* und *last* sind, einschließlich der Grenzen. Sowohl *first* als auch *last* können jede der Standardformen für Indizes haben.

***pathName* opentree *node* ?*recurse*?**
Dieser Befehl öffnet den gesamten Teilbaum, den *node* angibt. Er geht den Baum von *node* aus durch und setzt die Option **open** auf 1, abhängig vom Wert von *recurse*. Der Vorgabewert von *recurse* ist *true*.

pathName* parent *node
Gibt den Elternteil von *node* zurück.

pathName* reorder *node neworder
Ändert die Reihenfolge der Kinder von *node*, die *neworder* angibt. Kinder von *node*, die nicht in *neworder* vorkommen, werden nicht verschoben.

pathName* see *node
Ordnet den Scrollbereich so an, dass *node* sichtbar wird.

***pathName* selection *cmd* ?*arg...*?**
Ändert die Liste der ausgewählten Knoten entsprechend *cmd*:

add
Nimmt alle Knoten in *arg* in die Auswahl auf.

clear
Entfernt alle Knoten aus der Auswahl.

get
Gibt eine Liste mit den Indizes der aktuell ausgewählten Knoten zurück.

includes
Prüft, ob der angegebene Knoten ausgewählt ist. Gibt true zurück, wenn die Antwort ja ist, und sonst false.

range
Setzt die Auswahl auf alle Knoten zwischen den beiden angegebenen.

remove
Entfernt alle Knoten in *arg* aus der Auswahl.

set
Setzt die Auswahl auf alle Knoten in *arg*.

toggle
Schaltet den Auswahlzustand aller Knoten in *arg* um.

Die Unterbefehle **add**, **range** und **set** ignorieren still Knoten, die als nicht auswählbar deklariert sind. Die Knotenoption -selectable beeinflusst das.

pathName* toggle *node
Schaltet den Zustand offen/geschlossen des angegebenen *node* um.

pathName* visible *node
Gibt zurück, ob *node* sichtbar ist (alle seine Eltern sind offen).

***pathName* xview ?*arg...*?**
Standardbefehl, um das waagerechte Scrollen von *pathName* zu ermöglichen.

***pathName* yview ?*arg...*?**
Standardbefehl, um das senkrechte Scrollen von *pathName* zu ermöglichen.

BINDUNGEN

Ein virtuelles Ereignis **<<TreeSelect>>** wird jedes Mal erzeugt, wenn sich die Auswahl im Baum ändert. Das ist das Vorgabeverhalten eines Eintrags im Baum, es kann aber mit dem Befehl bindText oder bindImage überschrieben werden. Wenn die Bindung der Taste 1 überschrieben wird, wird dieses Ereignis möglicherweise nicht erzeugt.

Der Baum hat alle Standardbindungen des Mausrads, wenn er den Fokus hat.

Widget

Stichwörter: Widget

Im Aufbau ...

NAME
Widget - die Basisklasse Widget

BEFEHL

Widget::addmap *class subclass subpath options*

Widget::bwinclude *class subclass subpath* ?*arg...*?

Widget::cget *path option*

Widget::configure *path options*

Widget::create *class path* ?*rename*?

Widget::declare *class optlist*

Widget::define *class filename* ?*class ...*?

Widget::destroy *path*

Widget::focusNext *w*

Widget::focusOK *w*

Widget::focusPrev *w*

Widget::generate-doc *dir widgetlist*

Widget::generate-widget-doc *class iscmd file*

Widget::getoption *path option*

Widget::getVariable *path varName?myVarName?*

Widget::hasChanged *path option pvalue*

Widget::init *class path options*

Widget::setoption *path option value*

Widget::subcget *path subwidget*

Widget::syncoptions *class subclass subpath options*

Widget::tkinclude *class tkwidget subpath* ?*arg...*?

BESCHREIBUNG

Der Namespace **Widget** verwaltet die Daten, die zu allen BWidgets gehören, und stellt Befehle bereit, mit denen ein BWidget leicht definiert wird.

Zum Definieren eines BWidget können die Befehle **tkinclude**, **bwinclude**, **declare**, **addmap** und **syncoptions** verwendet werden. Hier ist die Definition des Widgets ComboBox:

```
namespace eval ComboBox {
    # ArrowButton, Entry und LabelFrame werden verwendet
    ArrowButton::use
    Entry::use
    LabelFrame::use

    # Ressourcen von LabelFrame aufnehmen
    Widget::bwinclude ComboBox LabelFrame .labf \
        rename     {-text -label} \
        remove     {-focus} \
        prefix     {label -justify -width -anchor -height -font} \
        initialize {-relief sunken -borderwidth 2}

    # Ressourcen von Entry aufnehmen
    Widget::bwinclude ComboBox Entry .e \
        remove {-relief -bd -borderwidth -bg -fg} \
        rename {-foreground -entryfg -background -entrybg}

    # Neue Ressourcen deklarieren
    Widget::declare ComboBox {
        {-height      TkResource 0  0 listbox}
        {-values      String     "" 0}
        {-modifycmd   String     "" 0}
        {-postcommand String     "" 0}
    }

    # Ressourcen auf das Unterwidget abbilden
    Widget::addmap ComboBox "" :cmd {-background {}}
    Widget::addmap ComboBox ArrowButton .a \
        {-foreground {} -background {} -disabledforeground {} -state
    {}}

    # Optionen des Unterwidgets abgleichen
    Widget::syncoptions ComboBox Entry .e {-text {}}
    Widget::syncoptions ComboBox LabelFrame .labf {-label -text
    -underline {}}

    proc use {} {}
}
```

BEFEHL

Widget::addmap *class subclass subpath options*

Dieser Befehl bildet einige Ressourcen auf ein Unterwidget ab. Abgebildete Ressourcen

konfigurieren das Unterwidget automatisch, wenn das Widget konfiguriert wird.

- *class* ist die Klasse des neuen BWidget
- *subclass* ist die Klasse des Unterwidgets (BWidget-Klasse, zum Beispiel Entry, oder leer für ein Tk-Widget)
- *subpath* ist der Pfad des Unterwidgets
- *options* ist die Liste *{option realres ...}* der Optionen, die auf das Unterwidget abgebildet werden

Widget::bwinclude *class subclass subpath ?arg...?*

Dieser Befehl nimmt in ein neues BWidget die Ressourcen eines anderen BWidget auf. Die Argumente sind:

- *class* Klasse des neuen Widgets
- *subclass* Klassenname des aufzunehmenden BWidget
- *subpath* Pfad des Widgets, das konfiguriert wird, wenn das BWidget konfiguriert wird
- *options* ist:
  - *include {option option ...}* Liste der aufzunehmenden Optionen (alle, wenn nicht definiert)
  - *remove {option option ...}* Liste der zu entfernenden Optionen
  - *rename {option name option name ...}* Liste der umzubenennenden Optionen
  - *prefix {prefix option option ...}* stellt allen *option* das *prefix* voran
  - *initialize {option value option value ...}* Vorgabewert der Optionen
  - *readonly {option value option value ...}* neues Kennzeichen nur-lesbar

Widget::cget *path option*

Gibt den Wert von *option* des BWidget *path* zurück. **cget** prüft, ob die Option existiert, und gleicht sie mit dem Unterwidget ab. Wird gewöhnlich vom Befehl **cget** des BWidget aufgerufen.

Widget::configure *path options*

Beschreibungstext

Widget::create *class path ?rename?*

Die Standardmethode, um ein BWidget zu erzeugen. Der wirkliche Widget-Pfad wird in $path:cmd umbenannt, und eine neue Prozedur wird erzeugt, die den Pfad ersetzt und auf die Befehle des BWidget zeigt.

Wenn *rename* false ist, wird der Pfad nicht umbenannt, die Prozedur wird aber trotzdem erzeugt. Das ist nützlich, wenn ein anderes BWidget geerbt wird, das das Widget bereits umbenannt hat.

Der Befehl gibt den Widget-Pfad zurück. Dieser Befehl ist gewöhnlich der letzte Befehl, der im Befehl ::create des Widgets ausgeführt wird.

Widget::declare *class optlist*

Dieser Befehl deklariert neue Ressourcen für ein BWidget.

- *class* ist die Klasse des neuen Widgets
- *options* ist die Liste, die neue Optionen beschreibt. Jede neue Option ist eine Liste **{option type value ro ?args?}**, wobei:
  - *option* der Name der Option ist
  - *type* der Typ der Option ist
  - *value* der Vorgabewert der Option ist

- *ro* das Kennzeichen nur-lesbar der Option ist
- *args* vom Typ abhängt

type kann sein:

TkResource
value von *option* bezeichnet eine Ressource eines Tk-Widgets. *args* muss *class* oder *{class realoption}* sein. *class* ist der Erzeugungsbefehl des Tk-Widgets, zum Beispiel **entry**. Die zweite Form muss verwendet werden, wenn *option* im Tk-Widget nicht denselben Namen hat, sondern *realoption*. Wenn *value* leer ist, wird es mit dem Vorgabewert des Tk-Widgets initialisiert.

BwResource
value von *option* bezeichnet eine Ressource eines BWidget. *args* muss *class* oder *{class realoption}* sein. *class* ist der Name des Namespace des BWidget, zum Beispiel **LabelFrame**. Die zweite Form muss verwendet werden, wenn *option* im BWidget nicht denselben Namen hat, sondern *realoption*. Wenn *value* leer ist, wird es mit dem Vorgabewert des BWidget initialisiert.

Int
value von *option* ist eine ganze Zahl. *args* kann *{?min? ?max?}* sein, um sie in einen Bereich zu zwingen. Die Prüfung ist *[expr $option > $min] && [expr $option < $max]*, wenn args also *{0 10}* ist, muss der Wert zwischen 0 und 10 liegen, die Grenzen ausgeschlossen, wenn *args {=0 =10}* ist, muss der Wert zwischen 0 und 10 liegen, die Grenzen eingeschlossen.

Boolean
value von *option* ist ein boolescher Wert. Wahre Werte können **1**, **true** oder **yes** sein. Falsche Werte können **0**, **false** oder **no** sein. **Widget::cget** gibt immer 0 oder 1 zurück.

Enum
value von *option* ist ein Element einer Aufzählung. *args* muss die Liste der Aufzählung sein, zum Beispiel *{top left bottom right}*.

Flag
value von *option* ist eine Kombination aus einer Menge von Zeichen. *args* muss eine Zeichenkette sein, die die Menge definiert.

String
value von *option* ist eine beliebige ungeprüfte Zeichenkette.

Synonym
option ist ein Synonym der Option *args*. *value* hat hier keine Wirkung.

Widget::define *class filename ?class ...?*
Dieser Befehl definiert eine neue BWidget-Klasse. Er ist gewöhnlich der erste Befehl, der in einer neuen Widget-Definition ausgeführt wird.

- *class* ist der Name der neuen Widget-Klasse.
- *filename* ist der Name der Datei (ohne Erweiterung) in der BWidget-Distribution, die diese Klasse definiert.

Jede Klasse, die nach dem Dateinamen definiert wird, ist eine Klasse, von der dieses Widget abhängt. Der Befehl ::use wird für jede dieser Klassen aufgerufen, nachdem das neue Widget definiert wurde.

Dieser Befehl richtet die neue Klasse in mehreren Schritten ein. Zuerst erzeugt er im globalen

Namespace einen Alias für den Namen der Klasse, der auf den Unterbefehl ::create der Klasse zeigt. Danach definiert er einen Unterbefehl ::use für die Klasse, mit dem andere Klassen diese Klasse bei Bedarf laden können. Zuletzt erzeugt er eine Vorgabebindung an das Ereignis <Destroy> für die Klasse, die Widget::destroy für den Pfad aufruft. Das ist die Vorgabeeinrichtung für fast alle Widgets im Paket BWidget.

Widget::destroy *path*
Beschreibungstext

Widget::focusNext *w*
Beschreibungstext

Widget::focusOK *w*
Beschreibungstext

Widget::focusPrev *w*
Beschreibungstext

Widget::generate-doc *dir widgetlist*
Beschreibungstext

Widget::generate-widget-doc *class iscmd file*
Beschreibungstext

Widget::getoption *path option*
Gibt den Wert von *option* des BWidget *path* zurück. Dieser Befehl prüft nicht, ob die Option existiert, behandelt kein Synonym und gleicht nicht mit dem Unterwidget ab.

Widget::getVariable *path varName?myVarName?*
Macht die Variable *varName*, die zu *path* gehört, in der aktuellen Prozedur zugänglich. Die Variable wird im Widget-Namespace für *path* erzeugt und kann widgetspezifische Angaben speichern. Wenn *path* zerstört wird, wird jede so verwendete Variable mit ihm zerstört.

Wenn *myVarName* angegeben ist, ist die Variable in der aktuellen Prozedur unter diesem Namen zugänglich.

Widget::hasChanged *path option pvalue*
Beschreibungstext

Widget::init *class path options*
Beschreibungstext

Widget::setoption *path option value*
Setzt den Wert von *option* des BWidget *path* ohne Optionsprüfung, ohne Abbildung auf ein Unterwidget, ohne Synonymbehandlung, und setzt das Änderungskennzeichen nicht.

Widget::subcget *path subwidget*
Gibt die Liste aller Optionen und Werte des BWidget *path* zurück, die auf *subwidget* abgebildet sind.

Widget::syncoptions *class subclass subpath options*
Dieser Befehl gleicht die Optionswerte eines Unterwidgets ab. Er wird verwendet, wenn eine Option eines Unterwidgets außerhalb des Befehls **configure** des BWidget geändert wird.

- *class* ist die Klasse des neuen BWidget

- *subclass* ist die Klasse des Unterwidgets (BWidget-Klasse, zum Beispiel Entry, oder leer für ein Tk-Widget)
- *subpath* ist der Pfad des Unterwidgets
- *options* ist die Liste *{option realres ...}* der Optionen, die mit dem Unterwidget abgeglichen werden

Widget::tkinclude *class tkwidget subpath* ?*arg...*?

Dieser Befehl nimmt in ein neues BWidget die Ressourcen eines Tk-Widgets auf. Die Argumente sind:

- *class* Klasse des neuen Widgets
- *tkwidger* Befehlsname des aufzunehmenden Tk-Widgets
- *subpath* Pfad des Widgets, das konfiguriert wird, wenn das BWidget konfiguriert wird
- *options* ist:
  - *include {option option ...}* Liste der aufzunehmenden Optionen (alle, wenn nicht definiert)
  - *remove {option option ...}* list of options to remove
  - *rename {option name option name ...}* Liste der umzubenennenden Optionen
  - *prefix {prefix option option ...}* stellt allen *option* das *prefix* voran
  - *initialize {option value option value ...}* Vorgabewert der Optionen
  - *readonly {option value option value ...}* new readonly flag

contents

Stichwörter: contents

Einfache Widgets

Label	Label-Widget mit der Option state, dynamischer Hilfe und Ziehen und Ablegen
Entry	Entry-Widget mit der Option state, dynamischer Hilfe und Ziehen und Ablegen
Button	Button-Widget mit erweiterten Optionen
ArrowButton	Button-Widget in Pfeilform.
ProgressBar	Widget für eine Fortschrittsanzeige
ScrollView	den sichtbaren Bereich eines gescrollten Fensters anzeigen
Separator	3D-Trenn-Widget

Manager-Widgets

MainFrame	Toplevel-Fenster mit Menü, Werkzeugleiste und Statusleiste verwalten
LabelFrame	Rahmen mit einem Label

TitleFrame	Rahmen mit einem Titel (stattdessen Tk 8.4+ labelframe erwägen)
PanelFrame	Rahmen mit einem umrahmten Titelbereich
ScrolledWindow	Allgemeines gescrolltes Widget
ScrollableFrame	Scrollbarer Rahmen, der ein Widget enthält
PanedWindow	Widget zur gekachelten Anordnung (stattdessen Tk 8.4+ panedwindow erwägen)
ButtonBox	Menge von Schaltflächen mit waagerechter oder senkrechter Anordnung
PagesManager	Widget zur Seitenverwaltung
NoteBook	Widget zur Notizbuchverwaltung
Dialog	Dialogabstraktion mit eigenen Schaltflächen
StatusBar	Statusleisten-Widget mit Steuerung der Größenänderung

Zusammengesetzte Widgets

LabelEntry	LabelFrame, das ein Entry-Widget enthält.
ComboBox	ComboBox-Widget
SpinBox	SpinBox-Widget (stattdessen Tk 8.4+ spinbox erwägen)
Tree	Tree-Widget
ListBox	ListBox-Widget
MessageDlg	Meldungsdialog
ProgressDlg	Dialog mit Fortschrittsanzeige
PasswdDlg	Dialog für Anmeldung und Passwort
SelectFont	Widget zur Schriftauswahl
SelectColor	Widget zur Farbauswahl

Befehlsklassen

Widget	Die Basisklasse Widget
DynamicHelp	Hilfe für ein Tk-Widget oder ein BWidget bereitstellen
DragSite	Befehlsmenge für Ziehen

DropSite	Befehlsmenge für Ablegen
BWidget	Beschreibungstext

index

Stichwörter: index

navtree

Stichwörter: navtree

Kurze Beschreibung

Einfache Widgets

Label

Entry

Button

ArrowButton

ProgressBar

ScrollView

Separator

Manager-Widgets

MainFrame

LabelFrame

TitleFrame

PanelFrame

ScrolledWindow

ScrollableFrame

PanedWindow

ButtonBox

PagesManager

NoteBook

Dialog

StatusBar

Zusammengesetzte Widgets

LabelEntry

ComboBox

SpinBox

Tree

ListBox

MessageDlg
ProgressDlg
PasswdDlg
SelectFont
SelectColor
Befehlsklassen
Widget
DynamicHelp
DragSite
DropSite
BWidget

options

Stichwörter: options

NAME

options - Von Widgets unterstützte Standardoptionen

BESCHREIBUNG

-activebackground, activeBackground, Foreground

-activeborderwidth, activeBorderWidth, BorderWidth

-activeforeground, activeForeground, Background

-anchor, anchor, Anchor

-background oder -bg, background, Background

-bitmap, bitmap, Bitmap

-borderwidth oder -bd, borderWidth, BorderWidth

-cursor, cursor, Cursor

-disabledbackground, disabledBackground, DisabledBackground

-disabledforeground, disabledForeground, DisabledForeground

-exportselection, exportSelection, ExportSelection

-font, font, Font

-foreground oder -fg, foreground, Foreground

-highlightbackground, highlightBackground, HighlightBackground

-highlightcolor, highlightColor, HighlightColor

-highlightthickness, highlightThickness, HighlightThickness

-image, image, Image

-insertbackground, insertBackground, Foreground

-insertborderwidth, insertBorderWidth, BorderWidth

-insertofftime, insertOffTime, OffTime

-insertontime, insertOnTime, OnTime

-insertwidth, insertWidth, InsertWidth

-jump, jump, Jump

-justify, justify, Justify

-orient, orient, Orient

-padx, padX, Pad

-pady, padY, Pad

-relief, relief, Relief

-repeatdelay, repeatDelay, RepeatDelay

-repeatinterval, repeatInterval, RepeatInterval

-selectbackground, selectBackground, Foreground

-selectborderwidth, selectBorderWidth, BorderWidth

-selectforeground, selectForeground, Background

-setgrid, setGrid, SetGrid

-takefocus, takeFocus, TakeFocus

-text, text, Text

-textvariable, textVariable, Variable

-troughcolor, troughColor, Background

-underline, underline, Underline

-wraplength, wrapLength, WrapLength

-xscrollcommand, xScrollCommand, ScrollCommand

-yscrollcommand, yScrollCommand, ScrollCommand

SCHLÜSSELWÖRTER

NAME

options - Von Widgets unterstützte Standardoptionen

BESCHREIBUNG

Dieser Handbucheintrag beschreibt die gemeinsamen Konfigurationsoptionen, die Widgets im Tk-Toolkit unterstützen. Nicht jedes Widget unterstützt notwendigerweise jede Option (die Handbucheinträge der einzelnen Widgets nennen die Standardoptionen, die dieses Widget unterstützt), wenn ein Widget aber eine Option mit einem der unten genannten Namen unterstützt, hat die Option genau die unten beschriebene Wirkung.

In den folgenden Beschreibungen bezeichnet ``Befehlszeilenname'' den Schalter, der in Klassenbefehlen und Widget-Befehlen **configure** verwendet wird, um diesen Wert zu setzen. Wenn zum Beispiel der Befehlszeilenschalter einer Option **-foreground** ist und ein Widget **.a.b.c** existiert, dann kann der Befehl

```
.a.b.c  configure  -foreground black
```

verwendet werden, um den Wert

black

für die Option in dem dem Widget

.a.b.c

. Befehlszeilenschalter können abgekürzt werden, solange die Abkürzung eindeutig ist. ``Datenbankname'' bezeichnet den Namen der Option in der Optionsdatenbank (zum Beispiel in .Xdefaults-Dateien). ``Datenbankklasse'' bezeichnet den Klassenwert der Option in der Optionsdatenbank.

Datenbankklasse: Foreground

Legt die Hintergrundfarbe fest, die beim Zeichnen aktiver Elemente verwendet wird. Ein Element (ein Widget oder ein Teil eines Widgets) ist aktiv, wenn der Mauszeiger über dem Element steht und das Drücken einer Maustaste eine Aktion auslöst. Wenn strenge Motif-Konformität verlangt wurde, indem die Variable **tk_strictMotif** gesetzt wurde, wird diese Option gewöhnlich ignoriert; stattdessen wird die normale Hintergrundfarbe verwendet.

Bei einigen Elementen auf Windows- und Macintosh-Systemen wird die aktive Farbe nur verwendet, solange die Maustaste 1 über dem Element gedrückt ist.

Befehlszeilenname: -activeborderwidth Name der Datenbank: activeBorderWidth Datenbankklasse: BorderWidth

Legt einen nicht negativen Wert fest, der die Breite des 3-D-Rahmens angibt, der um aktive Elemente gezeichnet wird. Die Definition aktiver Elemente steht oben. Der Wert kann jede Form haben, die **Tk_GetPixels** akzeptiert. Diese Option gibt es gewöhnlich nur bei Widgets, die mehr als ein Element gleichzeitig anzeigen (zum Beispiel Menüs, aber keine Schaltflächen).

Befehlszeilenname: -activeforeground Name der Datenbank: activeForeground Datenbankklasse: Background

Legt die Vordergrundfarbe fest, die beim Zeichnen aktiver Elemente verwendet wird. Die Definition aktiver Elemente steht oben.

Befehlszeilenname: -anchor Name der Datenbank: anchor Datenbankklasse: Anchor

Legt fest, wie die Information in einem Widget (zum Beispiel Text oder eine Bitmap) im Widget angezeigt wird. Muss einer der Werte **n**, **ne**, **e**, **se**, **s**, **sw**, **w**, **nw** oder **center** sein. **nw** bedeutet zum Beispiel, die Information so anzuzeigen, dass ihre obere linke Ecke an der oberen linken Ecke des Widgets liegt.

Befehlszeilenname: -background oder -bg Name der Datenbank: background Datenbankklasse: Background

Legt die normale Hintergrundfarbe fest, die beim Anzeigen des Widgets verwendet wird.

Befehlszeilenname: -bitmap Name der Datenbank: bitmap Datenbankklasse: Bitmap

Legt eine Bitmap fest, die im Widget angezeigt wird, in jeder Form, die **Tk_GetBitmap** akzeptiert. Die genaue Art der Anzeige kann von anderen Optionen wie **anchor** oder **justify** beeinflusst werden. Wenn diese Option angegeben ist, setzt sie gewöhnlich andere Optionen außer Kraft, die einen Textwert zur Anzeige im Widget festlegen; die Option **bitmap** kann auf eine leere Zeichenkette zurückgesetzt werden, um die Textanzeige wieder einzuschalten. In Widgets, die sowohl **bitmap** als auch **image** unterstützen, setzt **image** gewöhnlich **bitmap** außer Kraft.

Befehlszeilenname: -borderwidth oder -bd Name der Datenbank: borderWidth Datenbankklasse: BorderWidth

Legt einen nicht negativen Wert fest, der die Breite des 3-D-Rahmens angibt, der um die Außenseite des Widgets gezeichnet wird (wenn ein solcher Rahmen gezeichnet wird; das bestimmt gewöhnlich die Option **relief**). Der Wert kann auch verwendet werden, wenn 3-D-Effekte im Inneren des Widgets gezeichnet werden. Der Wert kann jede Form haben, die **Tk_GetPixels** akzeptiert.

Befehlszeilenname: -cursor Name der Datenbank: cursor Datenbankklasse: Cursor

Legt den Mauszeiger fest, der für das Widget verwendet wird. Der Wert kann jede Form haben, die **Tk_GetCursor** akzeptiert.

Befehlszeilenname: **-disabledforeground**

Befehlszeilenname: -disabledbackground Name der Datenbank: disabledBackground Datenbankklasse: DisabledBackground Legt die Hintergrundfarbe fest, die beim Zeichnen eines deaktivierten Elements verwendet wird. Wenn die Option als leere Zeichenkette angegeben ist

(was auf monochromen Anzeigen gewöhnlich der Fall ist), werden deaktivierte Elemente mit der normalen Hintergrundfarbe gezeichnet, aber durch ein schraffiertes Füllmuster abgeblendet. Name der Datenbank: disabledForeground Datenbankklasse: DisabledForeground

Legt die Vordergrundfarbe fest, die beim Zeichnen eines deaktivierten Elements verwendet wird. Wenn die Option als leere Zeichenkette angegeben ist (was auf monochromen Anzeigen gewöhnlich der Fall ist), werden deaktivierte Elemente mit der normalen Vordergrundfarbe gezeichnet, aber durch ein schraffiertes Füllmuster abgeblendet.

Befehlszeilenname: -exportselection Name der Datenbank: exportSelection Datenbankklasse: ExportSelection

Legt fest, ob eine Auswahl im Widget auch die X-Auswahl sein soll. Der Wert kann jede Form haben, die **Tcl_GetBoolean** akzeptiert, wie **true**, **false**, **0**, **1**, **yes** oder **no**. Wenn die Auswahl exportiert wird, hebt das Auswählen im Widget die aktuelle X-Auswahl auf, das Auswählen außerhalb des Widgets hebt jede Widget-Auswahl auf, und das Widget beantwortet Anforderungen zum Abfragen der Auswahl, wenn es eine Auswahl hat. Die Vorgabe ist gewöhnlich, dass Widgets Auswahlen exportieren.

Befehlszeilenname: -font Name der Datenbank: font Datenbankklasse: Font

Legt die Schriftart fest, die beim Zeichnen von Text im Widget verwendet wird.

Befehlszeilenname: -foreground oder -fg Name der Datenbank: foreground Datenbankklasse: Foreground

Legt die normale Vordergrundfarbe fest, die beim Anzeigen des Widgets verwendet wird.

Befehlszeilenname: -highlightbackground Name der Datenbank: highlightBackground Datenbankklasse: HighlightBackground

Legt die Farbe fest, die im Bereich der Durchlaufhervorhebung angezeigt wird, wenn das Widget nicht den Eingabefokus hat.

Befehlszeilenname: -highlightcolor Name der Datenbank: highlightColor Datenbankklasse: HighlightColor

Legt die Farbe für das Rechteck der Durchlaufhervorhebung fest, das um das Widget gezeichnet wird, wenn es den Eingabefokus hat.

Befehlszeilenname: -highlightthickness Name der Datenbank: highlightThickness Datenbankklasse: HighlightThickness

Legt einen nicht negativen Wert fest, der die Breite des Hervorhebungsrechtecks angibt, das um die Außenseite des Widgets gezeichnet wird, wenn es den Eingabefokus hat. Der Wert kann jede Form haben, die **Tk_GetPixels** akzeptiert. Wenn der Wert null ist, wird keine Fokushervorhebung um das Widget gezeichnet.

Befehlszeilenname: -image Name der Datenbank: image Datenbankklasse: Image

Legt ein Bild fest, das im Widget angezeigt wird; es muss mit dem Befehl **image create** erzeugt worden sein. Wenn die Option **image** angegeben ist, setzt sie gewöhnlich andere Optionen außer Kraft, die eine Bitmap oder einen Textwert zur Anzeige im Widget festlegen; die Option **image** kann auf eine leere Zeichenkette zurückgesetzt werden, um die Anzeige einer Bitmap oder von Text wieder einzuschalten.

Befehlszeilenname: -insertbackground Name der Datenbank: insertBackground Datenbankklasse: Foreground

Legt die Farbe fest, die als Hintergrund in der Fläche der Einfügemarke verwendet wird. Diese Farbe setzt gewöhnlich entweder den normalen Hintergrund des Widgets außer Kraft (oder den Hintergrund der Auswahl, wenn die Einfügemarke in die Auswahl fällt).

Befehlszeilenname: -insertborderwidth Name der Datenbank: insertBorderWidth Datenbankklasse: BorderWidth

Legt einen nicht negativen Wert fest, der die Breite des 3-D-Rahmens angibt, der um die Einfügemarke gezeichnet wird. Der Wert kann jede Form haben, die **Tk_GetPixels** akzeptiert.

Befehlszeilenname: -insertofftime Name der Datenbank: insertOffTime Datenbankklasse: OffTime

Legt einen nicht negativen ganzzahligen Wert fest, der die Anzahl der Millisekunden angibt, die die Einfügemarke in jedem Blinkzyklus ``off'' bleiben soll. Wenn diese Option null ist, blinkt die Einfügemarke nicht: sie ist die ganze Zeit eingeschaltet.

Befehlszeilenname: -insertontime Name der Datenbank: insertOnTime Datenbankklasse: OnTime

Legt einen nicht negativen ganzzahligen Wert fest, der die Anzahl der Millisekunden angibt, die die Einfügemarke in jedem Blinkzyklus ``on'' bleiben soll.

Befehlszeilenname: -insertwidth Name der Datenbank: insertWidth Datenbankklasse: InsertWidth

Legt einen Wert fest, der die Gesamtbreite der Einfügemarke angibt. Der Wert kann jede Form haben, die **Tk_GetPixels** akzeptiert. Wenn für die Einfügemarke ein Rahmen angegeben wurde (mit der Option **insertBorderWidth**), wird der Rahmen innerhalb der Breite gezeichnet, die die Option **insertWidth** angibt.

Befehlszeilenname: -jump Name der Datenbank: jump Datenbankklasse: Jump

Bei Widgets mit einem Schieber, der gezogen werden kann, um einen Wert zu verstellen, wie bei Scrollbars, bestimmt diese Option, wann Benachrichtigungen über Änderungen des Wertes erfolgen. Der Wert der Option muss ein boolescher Wert der Form sein, die **Tcl_GetBoolean** akzeptiert. Wenn der Wert false ist, erfolgen Aktualisierungen fortlaufend, während der Schieber gezogen wird. Wenn der Wert true ist, werden Aktualisierungen aufgeschoben, bis die Maustaste losgelassen wird, um das Ziehen zu beenden; dann erfolgt eine einzige Benachrichtigung (der Wert ``jumps'', anstatt sich gleichmäßig zu ändern).

Befehlszeilenname: -justify Name der Datenbank: justify Datenbankklasse: Justify

Wenn in einem Widget mehrere Textzeilen angezeigt werden, bestimmt diese Option, wie die Zeilen zueinander ausgerichtet werden. Muss einer der Werte **left**, **center** oder **right** sein. **Left** bedeutet, dass die linken Kanten der Zeilen auf einer Linie liegen, **center** bedeutet, dass die Mitten der Zeilen ausgerichtet sind, und **right** bedeutet, dass die rechten Kanten der Zeilen auf einer Linie liegen.

Befehlszeilenname: -orient Name der Datenbank: orient Datenbankklasse: Orient

Bei Widgets, die sich mit waagerechter oder senkrechter Ausrichtung anordnen können, wie Scrollbars, legt diese Option fest, welche Ausrichtung verwendet werden soll. Muss entweder **horizontal** oder **vertical** oder eine Abkürzung eines dieser Werte sein.

Befehlszeilenname: -padx Name der Datenbank: padX Datenbankklasse: Pad

Legt einen nicht negativen Wert fest, der angibt, wie viel zusätzlicher Platz für das Widget in X-Richtung angefordert wird. Der Wert kann jede Form haben, die **Tk_GetPixels** akzeptiert.

Wenn das Widget berechnet, wie groß ein Fenster sein muss, addiert es diesen Betrag zu der Breite, die es gewöhnlich braucht (bestimmt durch die Breite dessen, was im Widget angezeigt wird); wenn der Geometriemanager diese Anforderung erfüllen kann, erhält das Widget zusätzlichen inneren Platz links und/oder rechts von dem, was es innen anzeigt. Die meisten Widgets verwenden diese Option nur für den Abstand um Text: wenn sie eine Bitmap oder ein Bild anzeigen, ignorieren sie Abstandsoptionen gewöhnlich.

Befehlszeilenname: -pady Name der Datenbank: padY Datenbankklasse: Pad

Legt einen nicht negativen Wert fest, der angibt, wie viel zusätzlicher Platz für das Widget in Y-Richtung angefordert wird. Der Wert kann jede Form haben, die **Tk_GetPixels** akzeptiert. Wenn das Widget berechnet, wie groß ein Fenster sein muss, addiert es diesen Betrag zu der Höhe, die es gewöhnlich braucht (bestimmt durch die Höhe dessen, was im Widget angezeigt wird); wenn der Geometriemanager diese Anforderung erfüllen kann, erhält das Widget zusätzlichen inneren Platz oberhalb und/oder unterhalb von dem, was es innen anzeigt. Die meisten Widgets verwenden diese Option nur für den Abstand um Text: wenn sie eine Bitmap oder ein Bild anzeigen, ignorieren sie Abstandsoptionen gewöhnlich.

Befehlszeilenname: -relief Name der Datenbank: relief Datenbankklasse: Relief

Legt den gewünschten 3-D-Effekt des Widgets fest. Zulässige Werte sind **raised**, **sunken**, **flat**, **ridge**, **solid** und **groove**. Der Wert gibt an, wie das Innere des Widgets gegenüber seiner Außenseite erscheinen soll; **raised** bedeutet zum Beispiel, dass das Innere des Widgets gegenüber der Außenseite des Widgets aus dem Bildschirm hervorzutreten scheint.

Befehlszeilenname: -repeatdelay Name der Datenbank: repeatDelay Datenbankklasse: RepeatDelay

Legt die Anzahl der Millisekunden fest, die eine Schaltfläche oder Taste gehalten werden muss, bevor die automatische Wiederholung beginnt. Wird zum Beispiel bei den Auf- und Ab-Pfeilen in Scrollbars verwendet.

Befehlszeilenname: -repeatinterval Name der Datenbank: repeatInterval Datenbankklasse: RepeatInterval

Wird zusammen mit **repeatDelay** verwendet: sobald die automatische Wiederholung beginnt, bestimmt diese Option die Anzahl der Millisekunden zwischen den automatischen Wiederholungen.

Befehlszeilenname: -selectbackground Name der Datenbank: selectBackground Datenbankklasse: Foreground

Legt die Hintergrundfarbe fest, die beim Anzeigen ausgewählter Einträge verwendet wird.

Befehlszeilenname: -selectborderwidth Name der Datenbank: selectBorderWidth Datenbankklasse: BorderWidth

Legt einen nicht negativen Wert fest, der die Breite des 3-D-Rahmens angibt, der um ausgewählte Einträge gezeichnet wird. Der Wert kann jede Form haben, die **Tk_GetPixels** akzeptiert.

Befehlszeilenname: -selectforeground Name der Datenbank: selectForeground Datenbankklasse: Background

Legt die Vordergrundfarbe fest, die beim Anzeigen ausgewählter Einträge verwendet wird.

Befehlszeilenname: -setgrid Name der Datenbank: setGrid Datenbankklasse: SetGrid

Legt einen booleschen Wert fest, der bestimmt, ob dieses Widget das Größenänderungsraster seines Toplevel-Fensters steuert. Diese Option wird gewöhnlich in Text-Widgets verwendet, bei denen die Information im Widget eine natürliche Größe hat (die Größe eines Zeichens) und es sinnvoll ist, dass die Abmessungen des Fensters ganzzahlige Vielfache dieser Einheiten sind. Diese natürlichen Fenstergrößen bilden ein Raster. Wenn die Option **setGrid** auf true gesetzt ist, verständigt sich das Widget mit dem Fenstermanager, sodass beim interaktiven Ändern der Größe des Toplevel-Fensters, das das Widget enthält, die Abmessungen des Fensters in Rastereinheiten angezeigt werden und die Fenstergröße auf ganzzahlige Anzahlen von Rastereinheiten beschränkt wird. Weitere Einzelheiten stehen im Abschnitt GRIDDED GEOMETRY MANAGEMENT im Handbucheintrag **wm**.

Befehlszeilenname: -takefocus Name der Datenbank: takeFocus Datenbankklasse: TakeFocus

Bestimmt, ob das Fenster den Fokus während des Tastaturdurchlaufs annimmt (zum Beispiel Tab und Shift-Tab). Bevor der Fokus auf ein Fenster gesetzt wird, lesen die Durchlaufskripte den Wert der Option **takeFocus**. Ein Wert von **0** bedeutet, dass das Fenster beim Tastaturdurchlauf ganz übersprungen werden soll. **1** bedeutet, dass das Fenster den Eingabefokus erhalten soll, solange es sichtbar ist (es und alle seine Vorfahren sind abgebildet). Ein leerer Wert der Option bedeutet, dass die Durchlaufskripte entscheiden, ob das Fenster den Fokus erhält: der aktuelle Algorithmus überspringt das Fenster, wenn es deaktiviert ist, wenn es keine Tastenbindungen hat oder wenn es nicht sichtbar ist. Wenn der Wert eine andere Form hat, nehmen die Durchlaufskripte den Wert, hängen den Namen des Fensters an (mit einem Leerzeichen als Trenner) und werten die entstehende Zeichenkette als Tcl-Skript aus. Das Skript muss **0**, **1** oder eine leere Zeichenkette zurückgeben: ein Wert **0** oder **1** legt fest, ob das Fenster den Eingabefokus erhält, und eine leere Zeichenkette führt zu der oben beschriebenen Vorgabeentscheidung. Hinweis: diese Deutung der Option legen ausschließlich die Tcl-Skripte fest, die den Durchlauf ausführen: die Widget-Implementierungen ignorieren die Option vollständig, die Bedeutung kann also geändert werden, wenn die Skripte für den Tastaturdurchlauf neu definiert werden.

Befehlszeilenname: -text Name der Datenbank: text Datenbankklasse: Text

Legt eine Zeichenkette fest, die im Widget angezeigt wird. Die Art der Anzeige hängt vom jeweiligen Widget ab und kann von anderen Optionen bestimmt werden, wie **anchor** oder **justify**.

Befehlszeilenname: -textvariable Name der Datenbank: textVariable Datenbankklasse: Variable

Legt den Namen einer Variablen fest. Der Wert der Variablen ist eine Textzeichenkette, die im Widget angezeigt wird; wenn sich der Wert der Variablen ändert, aktualisiert sich das Widget selbst, um den neuen Wert wiederzugeben. Die Art, wie die Zeichenkette im Widget angezeigt wird, hängt vom jeweiligen Widget ab und kann von anderen Optionen bestimmt werden, wie **anchor** oder **justify**.

Befehlszeilenname: -troughcolor Name der Datenbank: troughColor Datenbankklasse: Background

Legt die Farbe für die rechteckigen Muldenflächen in Widgets wie Scrollbars und Scales fest.

Befehlszeilenname: -underline Name der Datenbank: underline Datenbankklasse: Underline

Legt den ganzzahligen Index eines Zeichens fest, das im Widget unterstrichen wird. Die Vorgabebindungen verwenden diese Option, um den Tastaturdurchlauf für Menüschaltflächen und Menüeinträge auszuführen. 0 entspricht dem ersten Zeichen des im Widget angezeigten Textes, 1 dem nächsten Zeichen, und so weiter.

Befehlszeilenname: -wraplength Name der Datenbank: wrapLength Datenbankklasse: WrapLength

Bei Widgets, die einen Wortumbruch ausführen können, legt diese Option die maximale Zeilenlänge fest. Zeilen, die diese Länge überschreiten würden, werden auf die nächste Zeile umbrochen, sodass keine Zeile länger ist als die angegebene Länge. Der Wert kann in jeder der Standardformen für Bildschirmabstände angegeben werden. Wenn dieser Wert kleiner oder gleich 0 ist, wird nicht umbrochen: Zeilen brechen nur an Zeilenumbruchzeichen im Text.

Befehlszeilenname: -xscrollcommand Name der Datenbank: xScrollCommand Datenbankklasse: ScrollCommand

Legt das Präfix für einen Befehl fest, der zur Verständigung mit waagerechten Scrollbars dient. Wenn sich die Ansicht im Fenster des Widgets ändert (oder wenn irgendetwas anderes geschieht, das die Anzeige in einer Scrollbar ändern könnte, wie eine Änderung der Gesamtgröße des Inhalts des Widgets), erzeugt das Widget einen Tcl-Befehl, indem es den Scrollbefehl und zwei Zahlen aneinanderhängt. Jede der Zahlen ist ein Bruch zwischen 0 und 1, der eine Position im Dokument angibt. 0 bezeichnet den Anfang des Dokuments, 1 das Ende, .333 eine Position nach einem Drittel des Dokuments, und so weiter. Der erste Bruch bezeichnet die erste Information im Dokument, die im Fenster sichtbar ist, und der zweite Bruch bezeichnet die Information unmittelbar nach dem letzten sichtbaren Teil. Der Befehl wird dann dem Tcl-Interpreter zur Ausführung übergeben. Gewöhnlich besteht die Option **xScrollCommand** aus dem Pfadnamen eines Scrollbar-Widgets, gefolgt von ``set'', zum Beispiel ``.x.scrollbar set'': dadurch wird die Scrollbar aktualisiert, sobald sich die Ansicht im Fenster ändert. Wenn diese Option nicht angegeben ist, wird kein Befehl ausgeführt.

Befehlszeilenname: -yscrollcommand Name der Datenbank: yScrollCommand Datenbankklasse: ScrollCommand Legt das Präfix für einen Befehl fest, der zur Verständigung mit senkrechten Scrollbars dient. Diese Option wird ebenso behandelt wie die Option xScrollCommand, sie wird aber für senkrechte Scrollbars verwendet und von Widgets bereitgestellt, die senkrechtes Scrollen unterstützen. Wie diese Option verwendet wird, steht in der Beschreibung von xScrollCommand.

SCHLÜSSELWÖRTER
Klasse

,

Name

,

Standardoption

,

Schalter

```
Copyright © 1990-1994 The Regents of the University of California.
Copyright © 1994-1996 Sun Microsystems, Inc.
Copyright © 1995-1997 Roger E. Critchlow Jr.
```

expect

expect

Stichwörter: expect

NAME

expect - programmierter Dialog mit interaktiven Programmen, Version 5

ÜBERSICHT

expect [**-dDinN**] [**-c** *cmds*] [[**-[f|b]**] *cmdfile*] [*args*]

EINLEITUNG

Expect ist ein Programm, das nach einem Skript mit anderen interaktiven Programmen "spricht". Dem Skript folgend weiß **Expect**, was von einem Programm zu erwarten ist und was die passende Antwort sein soll. Eine interpretierte Sprache stellt Verzweigungen und Steuerstrukturen auf hoher Ebene bereit, die den Dialog lenken. Zusätzlich kann der Benutzer die Steuerung übernehmen und bei Bedarf unmittelbar eingreifen und die Steuerung danach an das Skript zurückgeben.

Expectk ist eine Mischung aus **Expect** und **Tk**. Es verhält sich genau wie **Expect** und wie **wish** von **Tk**. **Expect** lässt sich auch unmittelbar in C oder C++ verwenden, also ohne Tcl. Die Beschreibung steht in libexpect(3).

Der Name "Expect" kommt von den Folgen *send/expect*, die uucp, kermit und andere Programme zur Modemsteuerung bekannt gemacht haben. Anders als uucp ist **Expect** so verallgemeinert, dass es als Befehl auf Benutzerebene mit jedem Programm und jeder Aufgabe laufen kann. **Expect** kann tatsächlich mit mehreren Programmen gleichzeitig sprechen.

Zum Beispiel kann **Expect** Folgendes:

- Sich vom eigenen Rechner zurückrufen lassen, um sich anzumelden, ohne das Gespräch zu bezahlen.
- Ein Spiel starten, zum Beispiel rogue, und wenn die günstigste Konfiguration nicht erscheint, es wieder und wieder neu starten, bis sie erscheint, und dann die Steuerung übergeben.
- fsck laufen lassen und auf seine Fragen mit "yes" oder "no" antworten oder die Steuerung nach festgelegten Kriterien zurückgeben.
- Eine Verbindung zu einem anderen Netz oder einem BBS aufbauen, zum Beispiel MCI Mail oder CompuServe, und die Post automatisch holen, sodass es aussieht, als wäre sie ursprünglich an das lokale System geschickt worden.
- Umgebungsvariablen, das aktuelle Verzeichnis oder beliebige Angaben über rlogin, telnet, tip, su, chgrp und Ähnliches hinübertragen.

Es gibt verschiedene Gründe, warum die Shell diese Aufgaben nicht ausführen kann. Ein Versuch zeigt es. Mit **Expect** sind sie alle möglich.

Im Allgemeinen eignet sich **Expect**, um jedes Programm laufen zu lassen, das eine Wechselwirkung zwischen Programm und Benutzer verlangt. Nötig ist nur, dass sich die

Wechselwirkung programmgesteuert beschreiben lässt. **Expect** kann die Steuerung auf Wunsch an den Benutzer zurückgeben, ohne das gesteuerte Programm anzuhalten. Ebenso kann der Benutzer die Steuerung jederzeit an das Skript zurückgeben.

VERWENDUNG

Expect liest *cmdfile* als Liste auszuführender Befehle. Auf Systemen, die die Notation #! unterstützen, lässt sich **Expect** auch implizit aufrufen, indem das Skript ausführbar markiert wird und die erste Zeile des Skripts lautet:

#!/usr/local/bin/expect -f

Der Pfad muss natürlich genau beschreiben, wo **Expect** liegt. /usr/local/bin ist nur ein Beispiel.

Die Markierung **-c** stellt einen Befehl voran, der vor jedem Befehl im Skript ausgeführt wird. Der Befehl sollte in Anführungszeichen stehen, damit die Shell ihn nicht zerlegt. Diese Option darf mehrmals verwendet werden. Mehrere Befehle lassen sich mit einem einzelnen **-c** ausführen, getrennt durch Semikolons. Die Befehle werden in der Reihenfolge ausgeführt, in der sie stehen. Bei Expectk heißt diese Option **-command**.

Die Markierung **-d** schaltet eine Diagnoseausgabe ein, die vor allem die interne Tätigkeit von Befehlen wie **expect** und **interact** meldet. Diese Markierung wirkt wie "exp_internal 1" am Anfang eines Expect-Skripts, und zusätzlich wird die Version von **Expect** ausgegeben. Der Befehl **strace** eignet sich, um Anweisungen zu verfolgen, der Befehl **trace**, um Variablenzuweisungen zu verfolgen. Bei Expectk heißt diese Option **-diag**.

Die Markierung **-D** schaltet einen interaktiven Debugger ein. Es muss ein ganzzahliger Wert folgen. Der Debugger übernimmt die Steuerung vor der nächsten Tcl-Prozedur, wenn der Wert nicht null ist oder ^C gedrückt wird oder ein Haltepunkt getroffen wird oder ein anderer passender Debugger-Befehl im Skript erscheint. Weitere Angaben zum Debugger stehen in der Datei README oder unter SEE ALSO weiter unten. Bei Expectk heißt diese Option **-Debug**.

Die Markierung **-f** stellt eine Datei voran, aus der Befehle gelesen werden. Die Markierung selbst ist optional, denn sie nützt nur bei der Notation #!, siehe oben, damit weitere Argumente auf der Befehlszeile angegeben werden können. Bei Expectk heißt diese Option **-file**.

In der Vorgabe wird die Befehlsdatei in den Speicher gelesen und vollständig ausgeführt. Gelegentlich sollen Dateien zeilenweise gelesen werden. stdin wird zum Beispiel so gelesen. Sollen beliebige Dateien so behandelt werden, dient die Markierung **-b**. Bei Expectk heißt diese Option **-buffer**. Die Pufferung von stdio kann trotzdem stattfinden. Beim Lesen aus einem Fifo oder von stdin sollte das keine Probleme verursachen.

Ist als Dateiname die Zeichenkette "-" angegeben, wird stattdessen die Standardeingabe gelesen. "./-" liest eine Datei, die tatsächlich "-" heißt.

Die Markierung **-i** lässt **Expect** interaktiv nach Befehlen fragen, statt sie aus einer Datei zu lesen. Die Abfrage endet mit dem Befehl **exit** oder bei EOF. Weitere Angaben stehen bei **interpreter** weiter unten. **-i** gilt als gesetzt, wenn weder eine Befehlsdatei noch **-c** verwendet wird. Bei Expectk heißt diese Option **-interactive**.

-- kann das Ende der Optionen markieren. Das ist nützlich, wenn ein optionsartiges Argument an das Skript gehen soll, ohne dass **Expect** es auswertet. In der Zeile #! verhindert das jede flagartige Auswertung durch Expect. Das folgende Beispiel lässt die ursprünglichen Argumente, einschließlich des Skriptnamens, in der Variable *argv*.

#!/usr/local/bin/expect --

Beim Ergänzen von Argumenten in der Zeile #! sind die üblichen Konventionen von getopt(3) und execve(2) einzuhalten.

Die Datei $exp_library/expect.rc wird automatisch eingelesen, wenn sie vorhanden ist, sofern nicht die Markierung **-N** verwendet wird. Bei Expectk heißt diese Option **-NORC**. Unmittelbar danach wird die Datei ~/.expect.rc automatisch eingelesen, sofern nicht die Markierung **-n** verwendet wird. Ist die Umgebungsvariable DOTDIR definiert, gilt sie als Verzeichnis, und .expect.rc wird von dort gelesen. Bei Expectk heißt diese Option **-norc**. Dieses Einlesen geschieht erst nach der Ausführung aller Markierungen **-c**.

-v lässt Expect seine Versionsnummer ausgeben und sich beenden. Die entsprechende Markierung in Expectk, das lange Markierungsnamen verwendet, ist -version.

Optionale *args* werden zu einer Liste zusammengebaut und in der Variable *argv* gespeichert. *argc* wird auf die Länge von argv gesetzt.

argv0 ist der Name des Skripts, oder der Binärdatei, wenn kein Skript verwendet wird. Das folgende Beispiel gibt den Namen des Skripts und die ersten drei Argumente aus:

```
    send_user "$argv0 [lrange $argv 0 2]\n"
```

BEFEHLE

Expect verwendet *Tcl* (Tool Command Language). Tcl stellt den Steuerfluss bereit, zum Beispiel if, for, break, die Auswertung von Ausdrücken und mehrere weitere Eigenschaften wie Rekursion, Prozedurdefinition und Ähnliches. Befehle, die hier verwendet, aber nicht definiert werden, zum Beispiel **set**, **if**, **exec**, sind Tcl-Befehle, siehe tcl(3). **Expect** unterstützt zusätzliche Befehle, die unten beschrieben sind. Wenn nichts anderes angegeben ist, geben Befehle die leere Zeichenkette zurück.

Die Befehle stehen alphabetisch, damit sie sich schnell finden lassen. Neue Benutzer finden den Einstieg leichter, wenn sie die Beschreibungen von **spawn**, **send**, **expect** und **interact** in dieser Reihenfolge lesen.

Die beste Einführung in die Sprache, sowohl Expect als auch Tcl, steht im Buch "Exploring Expect", siehe SEE ALSO weiter unten. Diese Handbuchseite enthält Beispiele, aber nur sehr wenige, denn sie ist vor allem als Nachschlagewerk gedacht.

Im Text dieser Handbuchseite meint "Expect" mit großem "E" das Programm **Expect**, während "expect" mit kleinem "e" den Befehl **expect** innerhalb des Programms **Expect** meint.

close *[-slave] [-onexec 0|1] [-i spawn_id]*

Schließt die Verbindung zum aktuellen Prozess. Die meisten interaktiven Programme erkennen EOF auf ihrer Standardeingabe und beenden sich. **close** genügt deshalb gewöhnlich auch, um den Prozess zu beenden. Die Markierung **-i** nennt den zu schließenden Prozess anhand der angegebenen spawn_id. Sowohl **expect** als auch **interact** erkennen, wenn der aktuelle Prozess endet, und führen implizit **close** aus. Wird der Prozess zum Beispiel mit "exec kill $pid" beendet, ist **close** ausdrücklich aufzurufen. Die Markierung **-onexec** bestimmt, ob die Spawn-Id in jedem neu gestarteten Prozess geschlossen wird oder wenn der Prozess überlagert wird. Der Wert 0 lässt eine Spawn-Id offen. Ein ganzzahliger Wert ungleich null erzwingt das Schließen des Spawns in jedem neuen Prozess, das ist die Vorgabe. Die Markierung **-slave** schließt den zur Spawn-Id gehörenden Slave. Siehe dazu "spawn -pty".

Wird die Verbindung geschlossen, wird der Slave automatisch mitgeschlossen, falls er noch offen ist. Ob die Verbindung implizit oder ausdrücklich geschlossen wird, **wait** ist aufzurufen, um den zugehörigen Prozessplatz im Kernel freizugeben. **close** ruft **wait** nicht auf, denn es ist nicht gesichert, dass das Schließen einer Prozessverbindung den Prozess beendet. Weitere Angaben stehen unten bei **wait**.

debug *[[-now] 0|1]*

Steuert einen Tcl-Debugger, der Anweisungen schrittweise ausführt, Haltepunkte setzt und Ähnliches. Ohne Argumente kommt 1 zurück, wenn der Debugger nicht läuft, sonst 0. Mit dem Argument 1 wird der Debugger gestartet. Mit dem Argument 0 wird der Debugger angehalten. Steht vor dem Argument 1 die Markierung **-now**, startet der Debugger sofort, also mitten im Befehl **debug** selbst. Sonst startet der Debugger mit der nächsten Tcl-Anweisung. Der Befehl **debug** ändert keine Traps. Zum Vergleich der Start von Expect mit der Markierung **-D**, siehe oben. Weitere Angaben zum Debugger stehen in der Datei README oder unter SEE ALSO weiter unten.

disconnect

Trennt einen abgespaltenen Prozess vom Terminal. Er läuft im Hintergrund weiter. Der Prozess bekommt eine eigene Prozessgruppe, wenn das möglich ist. Die Standard-Ein-/Ausgabe wird nach /dev/null umgelenkt.

Das folgende Fragment nutzt **disconnect**, damit das Skript im Hintergrund weiterläuft.

```
if {[fork]!=0} exit
disconnect
. . .
```

Das folgende Skript liest ein Passwort und lässt dann stündlich ein Programm laufen, das bei jedem Lauf ein Passwort verlangt. Das Skript liefert das Passwort, sodass es nur einmal einzugeben ist. Der Befehl **stty** zeigt, wie sich das Echo des Passworts abschalten lässt.

```
send_user "password?\ "
expect_user -re "(.*)\n"
for {} 1 {} {
    if {[fork]!=0} {sleep 3600;continue}
    disconnect
    spawn priv_prog
    expect Password:
    send "$expect_out(1,string)\r"
    . . .
    exit
}
```

Ein Vorteil von **disconnect** gegenüber der asynchronen Prozessfunktion der Shell (&) ist, dass **Expect** die Terminalparameter vor dem Trennen speichern und sie später auf neue Ptys anwenden kann. Mit & kann **Expect** die Terminalparameter nicht lesen, denn das Terminal ist schon getrennt, wenn **Expect** die Steuerung erhält.

exit *[-opts] [status]*

Lässt **Expect** sich beenden oder sich darauf vorbereiten. Die Markierung **-onexit** verwendet das nächste Argument als Exit-Handler. Ohne Argument kommt der aktuelle Exit-Handler

zurück. Die Markierung **-noexit** lässt **Expect** das Beenden vorbereiten, ohne die Steuerung tatsächlich an das Betriebssystem zurückzugeben. Der benutzerdefinierte Exit-Handler läuft ebenso wie die eigenen internen Handler von Expect. Weitere Expect-Befehle sollten nicht ausgeführt werden. Das ist nützlich, wenn Expect zusammen mit anderen Tcl-Erweiterungen läuft. Der aktuelle Interpreter und, in der Tk-Umgebung, das Hauptfenster bleiben, damit andere Tcl-Erweiterungen aufräumen können. Wird **exit** von Expect erneut aufgerufen, wie auch immer das geschieht, laufen die Handler nicht noch einmal. Beim Beenden werden alle Verbindungen zu gestarteten Prozessen geschlossen. Gestartete Prozesse erkennen das Schließen als EOF. **exit** tut nichts weiter als die gewöhnliche Prozedur _exit(2). Gestartete Prozesse, die nicht auf EOF prüfen, können deshalb weiterlaufen. Für die Frage, welche Signale ein gestarteter Prozess erhält, sind verschiedene Bedingungen maßgeblich. Diese Bedingungen hängen vom System ab und stehen gewöhnlich unter exit(3). Gestartete Prozesse, die weiterlaufen, werden von init geerbt. *status*, oder 0, wenn nichts angegeben ist, kommt als Exit-Status von **Expect** zurück. **exit** wird implizit ausgeführt, wenn das Ende des Skripts erreicht ist.

exp_continue [-continue_timer]
Der Befehl **exp_continue** lässt **expect** selbst weiter ausführen, statt wie gewöhnlich zurückzukehren. In der Vorgabe setzt **exp_continue** den Timeout-Zeitgeber zurück. Die Markierung *-continue_timer* verhindert, dass der Zeitgeber neu gestartet wird. Weitere Angaben stehen bei **expect**.

exp_internal *[-f file] value*
Lässt weitere Befehle Diagnoseangaben aus dem Inneren von **Expect** nach stderr senden, wenn *value* ungleich null ist. Diese Ausgabe ist aus, wenn *value* 0 ist. Die Diagnoseangaben umfassen jedes empfangene Zeichen und jeden Versuch, die aktuelle Ausgabe gegen die Muster zu prüfen.

Ist die optionale *file* angegeben, wird die gesamte gewöhnliche Ausgabe und die Ausgabe zur Fehlersuche in diese Datei geschrieben, unabhängig vom Wert von *value*. Eine frühere Datei für die Diagnoseausgabe wird geschlossen. Die Markierung **-info** lässt exp_internal eine Beschreibung der zuletzt angegebenen Argumente zurückgeben, die nicht info sind.

exp_open *[args] [-i spawn_id]*
Gibt einen Tcl-Dateibezeichner zurück, der der ursprünglichen Spawn-Id entspricht. Der Dateibezeichner lässt sich dann verwenden, als hätte ihn der Tcl-Befehl **open** geöffnet. Die Spawn-Id sollte nicht mehr verwendet werden. **wait** sollte nicht ausgeführt werden. Die Markierung **-leaveopen** lässt die Spawn-Id für den Zugriff über Expect-Befehle offen. Für die Spawn-Id ist **wait** auszuführen.

exp_pid *[-i spawn_id]*
Gibt die Prozesskennung zurück, die zum aktuell gestarteten Prozess gehört. Ist die Markierung **-i** verwendet, gehört die zurückgegebene pid zur angegebenen Spawn-Id.

exp_send
Ist ein Alias für **send**.

exp_send_error
Ist ein Alias für **send_error**.

exp_send_log
Ist ein Alias für **send_log**.

exp_send_tty
Ist ein Alias für **send_tty**.

exp_send_user
Ist ein Alias für **send_user**.

exp_version *[[-exit] version]*
Dient dazu, sicherzustellen, dass das Skript zur aktuellen Version von Expect passt.

Ohne Argumente kommt die aktuelle Version von **Expect** zurück. Diese Version lässt sich dann im Skript festhalten. Ist bekannt, dass keine Eigenschaften neuerer Versionen verwendet werden, kann eine frühere Version angegeben werden.

Versionen bestehen aus drei durch Punkte getrennten Zahlen. Die erste ist die Hauptnummer. Skripte, die für Versionen von **Expect** mit einer anderen Hauptnummer geschrieben sind, arbeiten mit ziemlicher Sicherheit nicht. **exp_version** gibt einen Fehler zurück, wenn die Hauptnummern nicht übereinstimmen.

Die zweite ist die Nebennummer. Skripte, die für eine Version mit einer größeren Nebennummer als der aktuellen Version geschrieben sind, können von einer neuen Eigenschaft abhängen und möglicherweise nicht laufen. **exp_version** gibt einen Fehler zurück, wenn die Hauptnummern übereinstimmen, die Nebennummer des Skripts aber größer ist als die des laufenden **Expect**.

Die dritte Zahl spielt beim Versionsvergleich keine Rolle. Diese Zahl wird aber erhöht, wenn die Software-Ausgabe von **Expect** irgendwie geändert wird, zum Beispiel durch zusätzliche Dokumentation oder eine Optimierung. Bei jeder neuen Nebenversion wird sie auf 0 gesetzt.

Mit der Markierung **-exit** gibt **Expect** einen Fehler aus und beendet sich, wenn die Version veraltet ist.

expect *[[-opts] pat1 body1] ... [-opts] patn [bodyn]*
Wartet, bis eines der Muster zur Ausgabe eines gestarteten Prozesses passt, eine angegebene Zeitspanne vergangen ist oder ein Dateiende gesehen wird. Ist der letzte Aktionsblock leer, darf er entfallen.

Muster des letzten Befehls **expect_before** werden implizit vor allen anderen Mustern verwendet. Muster des letzten Befehls **expect_after** werden implizit nach allen anderen Mustern verwendet.

Brauchen die Argumente der ganzen Anweisung **expect** mehr als eine Zeile, dürfen alle Argumente in eine geschweifte Klammer gefasst werden, damit nicht jede Zeile mit einem Rückwärtsschrägstrich enden muss. In diesem einen Fall finden die üblichen Tcl-Ersetzungen trotz der Klammern statt.

Ist ein Muster das Schlüsselwort **eof**, wird der zugehörige Aktionsblock bei Dateiende ausgeführt. Ist ein Muster das Schlüsselwort **timeout**, wird der zugehörige Aktionsblock bei Zeitüberschreitung ausgeführt. Wird kein Schlüsselwort timeout verwendet, wird bei Zeitüberschreitung eine implizite leere Aktion ausgeführt. Die Vorgabe der Zeitspanne ist 10 Sekunden, sie lässt sich zum Beispiel mit dem Befehl "set timeout 30" auf 30 setzen. Eine

unendliche Zeitspanne lässt sich mit dem Wert -1 bezeichnen. Ist ein Muster das Schlüsselwort **default**, wird der zugehörige Aktionsblock bei Zeitüberschreitung oder bei Dateiende ausgeführt.

Passt ein Muster, wird der zugehörige Aktionsblock ausgeführt. **expect** gibt das Ergebnis des Aktionsblocks zurück, oder die leere Zeichenkette, wenn kein Muster gepasst hat. Passen mehrere Muster, wird das zuerst stehende verwendet, um einen Aktionsblock auszuwählen.

Jedes Mal, wenn neue Ausgabe eintrifft, wird sie mit jedem Muster in der aufgeführten Reihenfolge verglichen. Das Fehlen einer Übereinstimmung lässt sich also prüfen, indem das letzte Muster etwas ist, das mit Sicherheit erscheint, zum Beispiel eine Eingabeaufforderung. Gibt es keine Eingabeaufforderung, ist **timeout** zu verwenden, so wie bei einer unmittelbaren Bedienung von Hand.

Muster werden auf drei Arten angegeben. In der Vorgabe werden Muster wie beim Tcl-Befehl **string match** angegeben. Solche Muster ähneln auch den regulären Ausdrücken der C-Shell, die gewöhnlich "glob"-Muster heißen. Die Markierung **-gl** kann Muster schützen, die sonst auf Markierungen von **expect** passen würden. Jedes Muster, das mit "-" beginnt, sollte so geschützt werden. Alle Zeichenketten, die mit "-" beginnen, sind für künftige Optionen reserviert.

Das folgende Fragment sucht zum Beispiel eine erfolgreiche Anmeldung. **abort** gilt als Prozedur, die an anderer Stelle im Skript definiert ist.

```
expect {
    busy               {puts busy\n ; exp_continue}
    failed             abort
    "invalid password" abort
    timeout            abort
    connected
}
```

Beim vierten Muster sind Anführungszeichen nötig, weil es ein Leerzeichen enthält, das sonst das Muster von der Aktion trennen würde. Muster mit derselben Aktion, wie das dritte und das vierte, verlangen, dass die Aktionen erneut aufgeführt werden. Das lässt sich vermeiden, indem Muster im Regexp-Stil verwendet werden, siehe unten. Weitere Angaben zum Bilden von Mustern im Glob-Stil stehen im Tcl-Handbuch.

Muster im Regexp-Stil folgen der Syntax, die der Tcl-Befehl **regexp** definiert, die Kurzform für "regular expression". Regexp-Muster werden mit der Markierung **-re** eingeleitet. Das vorige Beispiel lässt sich mit einem regulären Ausdruck so umschreiben:

```
expect {
    busy       {puts busy\n ; exp_continue}
    -re "failed|invalid password" abort
    timeout    abort
    connected
}
```

Beide Arten von Mustern sind ohne Anker. Das heißt, Muster müssen nicht auf die ganze Zeichenkette passen, sondern können die Übereinstimmung irgendwo in der Zeichenkette beginnen und beenden, solange alles Übrige passt. ^ passt auf den Anfang einer Zeichenkette, $ auf das Ende. Wird nicht auf das Ende einer Zeichenkette gewartet, können die Antworten leicht mitten in der Zeichenkette landen, weil der gestartete Prozess sie zurücksendet. Das Ergebnis bleibt richtig, die Ausgabe kann aber unnatürlich aussehen. $ ist deshalb angebracht, wenn sich die Zeichen am Ende einer Zeichenkette genau beschreiben lassen.

In vielen Editoren passen ^ und $ auf den Anfang beziehungsweise das Ende von Zeilen. Weil expect nicht zeilenorientiert ist, passen diese Zeichen auf den Anfang und das Ende der Daten, die gerade im Vergleichspuffer von expect stehen, nicht auf Zeilen. Dazu auch der Hinweis weiter unten zur "Systemüberlastung".

Die Markierung **-ex** lässt das Muster als "genaue" Zeichenkette passen. *, ^ und Ähnliches werden nicht ausgewertet, die üblichen Tcl-Konventionen gelten trotzdem. Genaue Muster sind immer ohne Anker.

Die Markierung **-nocase** lässt Großbuchstaben der Ausgabe so vergleichen, als wären sie Kleinbuchstaben. Das Muster bleibt unberührt.

Beim Lesen der Ausgabe können mehr als 2000 Bytes frühere Bytes zum "Vergessen" zwingen. Das lässt sich mit der Funktion **match_max** ändern. Übermäßig große Werte können den Mustervergleich verlangsamen. Ist *patlist* **full_buffer**, wird der zugehörige Aktionsblock ausgeführt, wenn *match_max* Bytes empfangen wurden und kein anderes Muster gepasst hat. Ob das Schlüsselwort **full_buffer** verwendet wird oder nicht, die vergessenen Zeichen werden nach expect_out(buffer) geschrieben. Ist *patlist* das Schlüsselwort **null** und sind Nullen erlaubt, über den Befehl **remove_nulls**, wird der zugehörige Aktionsblock ausgeführt, wenn ein einzelnes ASCII 0 passt. Über Glob- oder Regexp-Muster lassen sich 0 Bytes nicht treffen. Passt ein Muster, oder eof oder full_buffer, wird jede passende und zuvor nicht passende Ausgabe in der Variable *expect_out(buffer)* gespeichert. Bis zu 9 Übereinstimmungen von Regexp-Teilzeichenketten werden in den Variablen *expect_out(1,string)* bis *expect_out(9,string)* gespeichert. Wird vor einem Muster die Markierung **-indices** verwendet, werden Anfangs- und Endindex der 10 Zeichenketten, in einer für **lrange** geeigneten Form, in den Variablen *expect_out(X,start)* und *expect_out(X,end)* gespeichert. X ist eine Ziffer und entspricht der Position der Teilzeichenkette im Puffer. 0 meint Zeichenketten, die auf das ganze Muster gepasst haben, und wird für Glob-Muster ebenso erzeugt wie für Regexp-Muster. Hat ein Prozess zum Beispiel die Ausgabe "abcdefgh\n" erzeugt, ist das Ergebnis von:

```
expect "cd"
```

so, als wären die folgenden Anweisungen ausgeführt worden:

```
set expect_out(0,string) cd
set expect_out(buffer) abcd
```

und "efgh\n" bleibt im Ausgabepuffer. Hat ein Prozess die Ausgabe "abbbcabkkkka\n" erzeugt, ist das Ergebnis von:

```
expect -indices -re "b(b*).*(k+)"
```

so, als wären die folgenden Anweisungen ausgeführt worden:

```
    set expect_out(0,start) 1
    set expect_out(0,end) 10
    set expect_out(0,string) bbbcabkkkk
    set expect_out(1,start) 2
    set expect_out(1,end) 3
    set expect_out(1,string) bb
    set expect_out(2,start) 10
    set expect_out(2,end) 10
    set expect_out(2,string) k
    set expect_out(buffer) abbbcabkkkk
```

und "a\n" bleibt im Ausgabepuffer. Das Muster "*" und -re ".*" leeren den Ausgabepuffer, ohne weitere Ausgabe vom Prozess zu lesen.

Gewöhnlich wird die passende Ausgabe aus den internen Puffern von Expect verworfen. Das lässt sich verhindern, indem einem Muster die Markierung **-notransfer** vorangestellt wird. Diese Markierung ist beim Ausprobieren besonders nützlich und lässt sich dabei der Kürze halber als "-not" schreiben. Die zur passenden Ausgabe gehörende Spawn-Id, oder eof oder full_buffer, wird in *expect_out(spawn_id)* gespeichert. Die Markierung **-timeout** lässt den aktuellen Befehl expect den folgenden Wert als Zeitspanne verwenden, statt des Werts der Variable timeout. In der Vorgabe werden Muster gegen die Ausgabe des aktuellen Prozesses geprüft. Die Markierung **-i** bestimmt, dass die Ausgabe der genannten spawn_id-Liste gegen alle folgenden Muster geprüft wird, bis zum nächsten **-i**. Die spawn_id-Liste sollte entweder eine durch Leerraum getrennte Liste von spawn_ids sein oder eine Variable, die auf eine solche Liste von spawn_ids verweist. Das folgende Beispiel wartet auf "connected" vom aktuellen Prozess oder auf "busy", "failed" oder "invalid password" von der spawn_id, die $proc2 nennt.

```
   expect {
       -i $proc2 busy {puts busy\n ; exp_continue}
       -re "failed|invalid password" abort
       timeout abort
       connected
   }
```

Der Wert der globalen Variable *any_spawn_id* kann Muster auf alle spawn_ids passen lassen, die mit allen anderen Markierungen **-i** im aktuellen Befehl **expect** genannt sind. Die spawn_id einer Markierung **-i** ohne zugehöriges Muster, also unmittelbar gefolgt von einem weiteren **-i**, steht allen anderen Mustern im selben Befehl **expect** zur Verfügung, die zu *any_spawn_id.* gehören.

Die Markierung **-i** kann auch eine globale Variable nennen. Dann wird die Variable als Liste von Spawn-Ids gelesen. Die Variable wird neu gelesen, sobald sie sich ändert. So lässt sich die Ein-/Ausgabe-Quelle ändern, während der Befehl läuft. Auf diese Weise angegebene Spawn-Ids heißen "indirekte" Spawn-Ids.

Aktionen wie **break** und **continue** lassen Steuerstrukturen, also **for** und **proc**, sich wie üblich verhalten. Der Befehl **exp_continue** lässt **expect** selbst weiter ausführen, statt wie gewöhnlich zurückzukehren.

Das vermeidet ausdrückliche Schleifen oder wiederholte expect-Anweisungen. Das folgende Beispiel ist Teil eines Fragments, das rlogin automatisiert. **exp_continue** erspart eine zweite Anweisung **expect**, die erneut nach der Eingabeaufforderung sucht, wenn rlogin nach einem Passwort fragt.

```
expect {
    Password: {
        stty -echo
        send_user "password (for $user) on $host: "
        expect_user -re "(.*)\n"
        send_user "\n"
        send "$expect_out(1,string)\r"
        stty echo
        exp_continue
    } incorrect {
        send_user "invalid password or account\n"
        exit
    } timeout {
        send_user "connection to $host timed out\n"
        exit
    } eof {
        send_user \
            "connection to host failed: $expect_out(buffer)"
        exit
    } -re $prompt
}
```

Das folgende Fragment kann zum Beispiel helfen, eine schon vollständig automatisierte Wechselwirkung zu lenken. Das Terminal wird dabei in den Rohmodus gesetzt. Wird "+" gedrückt, wird eine Variable erhöht. Wird "p" gedrückt, gehen mehrere Wagenrückläufe an den Prozess, vielleicht um ihn anzustoßen, und "i" lässt den Benutzer unmittelbar mit dem Prozess arbeiten und entzieht dem Skript die Steuerung. In jedem Fall lässt **exp_continue** das aktuelle **expect** nach der aktuellen Aktion weiter Muster vergleichen.

```
stty raw -echo
expect_after {
    -i $user_spawn_id
    "p" {send "\r\r\r"; exp_continue}
    "+" {incr foo; exp_continue}
    "i" {interact; exp_continue}
    "quit" exit
}
```

In der Vorgabe setzt **exp_continue** den Timeout-Zeitgeber zurück. Der Zeitgeber wird nicht neu gestartet, wenn **exp_continue** mit der Markierung **-continue_timer** aufgerufen wird.

expect_after ***[expect_args]***

Arbeitet genauso wie **expect_before**, außer dass das Muster von **expect** verwendet wird, wenn Muster von **expect** und von **expect_after** passen können. Weitere Angaben stehen

beim Befehl **expect_before**.

expect_background ***[expect_args]***
Nimmt dieselben Argumente wie **expect**, kehrt aber sofort zurück. Muster werden geprüft, sobald neue Eingabe eintrifft. Die Muster **timeout** und **default** haben für **expect_background** keine Bedeutung und werden still verworfen. Sonst verwendet der Befehl **expect_background** Muster von **expect_before** und **expect_after** genau wie **expect**. Während Aktionen von **expect_background** ausgewertet werden, ist die Hintergrundverarbeitung für dieselbe Spawn-Id gesperrt. Die Hintergrundverarbeitung wird freigegeben, wenn die Aktion endet. Solange die Hintergrundverarbeitung gesperrt ist, ist ein **expect** im Vordergrund auf derselben Spawn-Id möglich. Ein **expect** lässt sich nicht ausführen, solange ein **expect_background** freigegeben ist. **expect_background** für eine bestimmte Spawn-Id wird gelöscht, indem ein neues expect_background mit derselben Spawn-Id erklärt wird. **expect_background** ohne Muster nimmt der angegebenen Spawn-Id die Fähigkeit, Muster im Hintergrund zu prüfen.

expect_before ***[expect_args]***
Nimmt dieselben Argumente wie **expect**, kehrt aber sofort zurück. Paare aus Muster und Aktion des letzten **expect_before** mit derselben Spawn-Id werden implizit jedem folgenden Befehl **expect** hinzugefügt. Passt ein Muster, wird es behandelt, als wäre es im Befehl **expect** selbst angegeben worden, und der zugehörige Aktionsblock wird im Zusammenhang des Befehls **expect** ausgeführt. Können Muster von **expect_before** und von **expect** passen, wird das Muster von **expect_before** verwendet. Ist kein Muster angegeben, wird die Spawn-Id auf keine Muster geprüft. Überschreibt eine Markierung **-i** das nicht, passen Muster von **expect_before** gegen die Spawn-Id, die zum Zeitpunkt der Ausführung des Befehls **expect_before** definiert war, nicht zum Zeitpunkt der Musterübereinstimmung. Die Markierung -info lässt **expect_before** die aktuellen Angaben zurückgeben, welche Muster es prüfen wird. In der Vorgabe berichtet es über die aktuelle Spawn-Id. Eine optionale Spawn-Id-Angabe kann Auskunft über diese Spawn-Id verlangen. Zum Beispiel

```
expect_before -info -i $proc
```

Höchstens eine Spawn-Id-Angabe darf angegeben werden. Die Markierung -indirect unterdrückt direkte Spawn-Ids, die nur aus indirekten Angaben stammen.

Statt einer Spawn-Id-Angabe lässt die Markierung "-all" die Markierung "-info" über alle Spawn-Ids berichten.

Die Ausgabe der Markierung -info lässt sich erneut als Argument von expect_before verwenden.

expect_tty ***[expect_args]***
Arbeitet wie **expect**, liest die Zeichen aber von /dev/tty, also die Tastenanschläge des Benutzers. In der Vorgabe wird im kanonischen Modus gelesen. Zeilen müssen deshalb mit einem Return enden, damit **expect** sie sieht. Das lässt sich mit **stty** ändern. Der Befehl **stty** steht weiter unten.

expect_user ***[expect_args]***
Arbeitet wie **expect**, liest die Zeichen aber von stdin, also die Tastenanschläge des Benutzers. In der Vorgabe wird im kanonischen Modus gelesen. Zeilen müssen deshalb mit einem Return enden, damit **expect** sie sieht. Das lässt sich mit **stty** ändern. Der Befehl **stty** steht weiter unten.

fork

Legt einen neuen Prozess an. Der neue Prozess ist eine genaue Kopie des aktuellen **Expect** -Prozesses. Bei Erfolg gibt **fork** dem neuen Kindprozess 0 zurück und dem Elternprozess die Prozesskennung des Kindprozesses. Bei einem Fehlschlag, ausnahmslos wegen fehlender Betriebsmittel wie Auslagerungsspeicher oder Speicher, gibt **fork** dem Elternprozess -1 zurück, und es entsteht kein Kindprozess.

Abgespaltene Prozesse beenden sich über den Befehl **exit**, genau wie der ursprüngliche Prozess. Abgespaltene Prozesse dürfen in die Protokolldateien schreiben. Wird die Fehlersuche oder das Protokollieren in den meisten Prozessen nicht ausgeschaltet, kann das Ergebnis verwirrend sein.

Manche Pty-Umsetzungen geraten durch mehrere Leser und Schreiber durcheinander, auch wenn das nur einen Augenblick dauert. Am sichersten ist deshalb **fork**, bevor Prozesse gestartet werden.

interact ***[string1 body1] ... [stringn [bodyn]]***

Gibt dem Benutzer die Steuerung über den aktuellen Prozess. Tastenanschläge gehen an den aktuellen Prozess, und stdout und stderr des aktuellen Prozesses kommen zurück.

Als Argumente dürfen Paare aus Zeichenkette und Aktionsblock angegeben werden. Dann wird der Aktionsblock ausgeführt, wenn die zugehörige Zeichenkette eingegeben wird. In der Vorgabe geht die Zeichenkette nicht an den aktuellen Prozess. Fehlt der letzte Aktionsblock, gilt der Befehl **interpreter**.

Brauchen die Argumente der ganzen Anweisung **interact** mehr als eine Zeile, dürfen alle Argumente in eine geschweifte Klammer gefasst werden, damit nicht jede Zeile mit einem Rückwärtsschrägstrich enden muss. In diesem einen Fall finden die üblichen Tcl-Ersetzungen trotz der Klammern statt.

Der folgende Befehl lässt zum Beispiel interact mit diesen Paaren aus Zeichenkette und Aktionsblock laufen. Wird ^Z gedrückt, wird **Expect** angehalten. Die Markierung **-reset** stellt die Terminalmodi wieder her. Wird ^A gedrückt, sieht der Benutzer "you typed a control-A", und der Prozess erhält ein ^A. Wird $ gedrückt, sieht der Benutzer das Datum. Wird ^C gedrückt, beendet sich **Expect**. Wird "foo" eingegeben, sieht der Benutzer "bar". Wird ~~ gedrückt, läuft der Interpreter von **Expect** interaktiv.

```
set CTRLZ \032
interact {
    -reset $CTRLZ {exec kill -STOP [pid]}
    \001   {send_user "you typed a control-A\n";
            send "\001"
           }
    $      {send_user "The date is [clock format [clock
seconds]]."}
    \003   exit
    foo    {send_user "bar"}
    ~~
}
```

In Paaren aus Zeichenkette und Aktionsblock werden die Zeichenketten in der Reihenfolge geprüft, in der sie als Argumente stehen. Zeichenketten, die teilweise passen, gehen nicht an den aktuellen Prozess, weil der Rest noch kommen kann. Werden dann Zeichen eingegeben, die eine Übereinstimmung unmöglich machen, geht nur der Teil der Zeichenkette an den Prozess, der unmöglich eine andere Übereinstimmung beginnen kann. Zeichenketten, die Teilzeichenketten teilweiser Übereinstimmungen sind, können also später passen, wenn die ursprüngliche Zeichenkette, die passen sollte, am Ende fehlschlägt.

In der Vorgabe ist der Zeichenkettenvergleich genau und ohne Platzhalter. Der Befehl **expect** verwendet in der Vorgabe dagegen Muster im Glob-Stil. Die Markierung **-ex** kann Muster schützen, die sonst auf Markierungen von **interact** passen würden. Jedes Muster, das mit "-" beginnt, sollte so geschützt werden. Alle Zeichenketten, die mit "-" beginnen, sind für künftige Optionen reserviert. Die Markierung **-re** erzwingt, dass die Zeichenkette als Muster im Regexp-Stil gelesen wird. Passende Teilzeichenketten werden dann in der Variable *interact_out* gespeichert, so wie **expect** seine Ausgabe in der Variable **expect_out** speichert. Die Markierung **-indices** wird ebenso unterstützt. Das Muster **eof** leitet eine Aktion ein, die bei Dateiende ausgeführt wird. Ein eigenes Muster **eof** darf auch auf die Markierung **-output** folgen. Dann passt es, wenn beim Schreiben der Ausgabe ein eof erkannt wird. Die Vorgabeaktion für **eof** ist "return", sodass **interact** bei jedem EOF einfach zurückkehrt. Das Muster **timeout** leitet eine Zeitspanne in Sekunden und eine Aktion ein, die ausgeführt wird, nachdem eine bestimmte Zeit lang keine Zeichen gelesen wurden. Das Muster **timeout** gilt für den zuletzt angegebenen Prozess. Eine Vorgabezeitspanne gibt es nicht. Die besondere Variable "timeout", die der Befehl **expect** verwendet, hat auf diese Zeitspanne keine Wirkung. Die folgende Anweisung kann zum Beispiel Benutzer abmelden, die eine Stunde lang nichts getippt haben, aber weiterhin häufig Systemmeldungen erhalten:

```
interact -input $user_spawn_id timeout 3600 return -output \
    $spawn_id
```

Ist das Muster das Schlüsselwort **null** und sind Nullen erlaubt, über den Befehl **remove_nulls**, wird der zugehörige Aktionsblock ausgeführt, wenn ein einzelnes ASCII 0 passt. Über Glob- oder Regexp-Muster lassen sich 0 Bytes nicht treffen.

Wird einem Muster die Markierung **-iwrite** vorangestellt, wird die Variable *interact_out(spawn_id)* auf die spawn_id gesetzt, die auf das Muster oder auf eof gepasst hat.

Aktionen wie **break** und **continue** lassen Steuerstrukturen, also **for** und **proc**, sich wie üblich verhalten. **return** lässt interact dagegen zu seinem Aufrufer zurückkehren, während **inter_return interact** eine Rückkehr in seinem Aufrufer auslösen lässt. Hat zum Beispiel "proc foo" **interact** aufgerufen und dieses dann die Aktion **inter_return** ausgeführt, würde **proc foo** zurückkehren. Ruft **interact** interaktiv **interpreter** auf, lässt eingetipptes **return** das interact weiterlaufen, während **inter_return** das interact zu seinem Aufrufer zurückkehren lässt.

Während **interact** gilt der Rohmodus, damit alle Zeichen an den aktuellen Prozess gehen können. Fängt der aktuelle Prozess Jobsteuerungs-Signale nicht ab, hält er an, wenn er ein Stoppsignal erhält, in der Vorgabe ^Z. Zum Weiterlaufen dient ein Continue-Signal, zum Beispiel "kill -CONT <pid>". Soll ein solcher Prozess wirklich ein SIGSTOP erhalten, durch ^Z, wird zuerst csh gestartet und dann das Programm. Soll **Expect** selbst ein SIGSTOP erhalten, wird zuerst interpreter aufgerufen, vielleicht über ein Fluchtzeichen, und dann ^Z gedrückt.

Paare aus Zeichenkette und Aktionsblock können als Kurzform dienen, damit der Interpreter nicht betreten und Befehle nicht interaktiv ausgeführt werden müssen. Während der Aktionsblock eines solchen Paares ausgeführt wird, gilt der vorige Terminalmodus.

Der Geschwindigkeit wegen laufen Aktionen in der Vorgabe im Rohmodus. Die Markierung **-reset** setzt das Terminal auf den Modus zurück, den es vor **interact** hatte, ausnahmslos den kanonischen Modus. Zeichen, die beim Umschalten des Modus eingegeben werden, können verloren gehen, eine unglückliche Eigenschaft des Terminaltreibers auf manchen Systemen. **-reset** ist nur dann begründet, wenn die Aktion den kanonischen Modus braucht.

Die Markierung **-echo** sendet Zeichen, die auf das folgende Muster passen, an den Prozess zurück, der sie erzeugt hat, sobald jedes Zeichen gelesen wird. Das kann nützlich sein, wenn der Benutzer eine Rückmeldung zu teilweise getippten Mustern braucht.

Wird ein Muster zurückgesendet, passt am Ende aber nicht, gehen die Zeichen an den gestarteten Prozess. Sendet der gestartete Prozess sie dann zurück, sieht der Benutzer die Zeichen zweimal. **-echo** passt vermutlich nur, wenn es unwahrscheinlich ist, dass das Muster nicht zu Ende getippt wird. Der folgende Ausschnitt stammt aus rftp, dem Skript für rekursives ftp. Der Benutzer wird aufgefordert, ~g, ~p oder ~l einzugeben, um das aktuelle Verzeichnis rekursiv zu holen, zu senden oder aufzulisten. Das liegt so weit von den gewöhnlichen ftp-Befehlen entfernt, dass nach ~ kaum etwas anderes getippt wird, außer aus Versehen. Dann wird das Ergebnis vermutlich übergangen.

```
interact {
    -echo ~g {getcurdirectory 1}
    -echo ~l {getcurdirectory 0}
    -echo ~p {putcurdirectory}
}
```

Die Markierung **-nobuffer** sendet Zeichen, die auf das folgende Muster passen, an den Ausgabeprozess, während die Zeichen gelesen werden.

Das ist nützlich, wenn ein Programm das Muster zurücksenden soll. Das Folgende kann zum Beispiel überwachen, wohin jemand wählt, ein Modem im Hayes-Stil. Jedes Mal, wenn "atd" gesehen wird, protokolliert das Skript den Rest der Zeile.

```
proc lognumber {} {
    interact -nobuffer -re "(.*)\r" return
    puts $log "[clock format [clock seconds]]: dialed
$interact_out(1,string)"
}

interact -nobuffer "atd" lognumber
```

Während **interact** wird eine frühere Verwendung von **log_user** übergangen. **interact** erzwingt insbesondere, dass seine Ausgabe protokolliert wird, also an die Standardausgabe geht, weil angenommen wird, dass der Benutzer nicht blind arbeiten will.

Die Markierung **-o** wendet alle folgenden Paare aus Taste und Aktionsblock auf die Ausgabe des aktuellen Prozesses an. Das kann zum Beispiel nützlich sein bei Rechnern, die während einer Telnet-Sitzung unerwünschte Zeichen senden.

In der Vorgabe erwartet **interact**, dass der Benutzer die Standardeingabe des **Expect** -Prozesses selbst schreibt und dessen Standardausgabe liest. Die Markierung **-u**, für "user", lässt **interact** den Benutzer als den Prozess suchen, den ihr Argument nennt. Das muss eine gestartete Id sein.

So lassen sich zwei unabhängige Prozesse verbinden, ohne eine ausdrückliche Schleife zu benutzen. Zur Fehlersuche geht die Diagnose von Expect immer nach stderr, oder nach stdout bei bestimmten Angaben zum Protokoll und zur Fehlersuche. Aus demselben Grund liest der Befehl **interpreter** interaktiv von stdin.

Das folgende Fragment legt zum Beispiel einen Login-Prozess an. Dann wählt es den Benutzer an, das ist nicht gezeigt, und verbindet die beiden schließlich. An die Stelle von login kann jeder Prozess treten. Eine Shell würde zum Beispiel die Arbeit erlauben, ohne ein Konto und ein Passwort anzugeben.

```
spawn login
set login $spawn_id
spawn tip modem
# dial back out to user
# connect user to login
interact -u $login
```

Sollen mehrere Prozesse Ausgabe erhalten, wird jede Spawn-Id-Liste aufgeführt, der eine Markierung **-output** voransteht. Die Eingabe für eine Gruppe von Ausgabe-Spawn-Ids kann eine Spawn-Id-Liste bestimmen, der eine Markierung **-input** voransteht. **-input** und **-output** dürfen Listen in derselben Form annehmen wie die Markierung **-i** im Befehl **expect**, nur ist any_spawn_id in **interact** ohne Bedeutung. Alle folgenden Markierungen und Zeichenketten oder Muster gelten für diese Eingabe, bis eine weitere Markierung -input erscheint. Erscheint kein **-input**, bedeutet **-output** "-input $user_spawn_id -output". Ebenso bei Mustern, die kein **-input** haben. Ist ein **-input** angegeben, überschreibt es $user_spawn_id. Ist ein zweites **-input** angegeben, überschreibt es $spawn_id. Weitere Markierungen **-input** dürfen angegeben werden.

Die beiden mitgemeinten Eingabeprozesse haben in der Vorgabe ihre Ausgaben als $spawn_id und $user_spawn_id angegeben, in umgekehrter Reihenfolge. Erscheint eine Markierung **-input** ohne Markierung **-output**, werden die Zeichen dieses Prozesses verworfen.

Die Markierung **-i** führt einen Ersatz für die aktuelle spawn_id ein, wenn keine anderen Markierungen **-input** oder **-output** verwendet werden. Eine Markierung -i bedeutet eine Markierung -o.

Die Prozesse, mit denen gearbeitet wird, lassen sich über indirekte Spawn-Ids wechseln. Indirekte Spawn-Ids sind im Abschnitt zum Befehl expect beschrieben. Indirekte Spawn-Ids dürfen mit den Markierungen -i, -u, -input oder -output angegeben werden.

interpreter [args]

Fordert den Benutzer interaktiv zur Eingabe von Befehlen für **Expect** und Tcl auf. Das Ergebnis jedes Befehls wird ausgegeben.

Aktionen wie **break** und **continue** lassen Steuerstrukturen, also **for** und **proc**, sich wie üblich verhalten. **return** lässt interpreter dagegen zu seinem Aufrufer zurückkehren, während **inter_return interpreter** eine Rückkehr in seinem Aufrufer auslösen lässt. Hat zum Beispiel "proc foo" **interpreter** aufgerufen und dieses dann die Aktion **inter_return** ausgeführt, würde **proc foo** zurückkehren. Jeder andere Befehl lässt **interpreter** weiter nach neuen Befehlen fragen.

In der Vorgabe enthält die Eingabeaufforderung zwei ganze Zahlen. Die erste beschreibt die Tiefe des Auswertungsstapels, also wie oft Tcl_Eval aufgerufen wurde. Die zweite ist der Tcl-Verlaufskennzeichner. Die Eingabeaufforderung lässt sich setzen, indem eine Prozedur "prompt1" definiert wird, deren Rückgabewert die nächste Eingabeaufforderung wird. Hat eine Anweisung offene Anführungszeichen, Klammern, geschweifte oder eckige Klammern, erscheint bei einem Zeilenumbruch eine zweite Eingabeaufforderung, in der Vorgabe "+> ". Die zweite Eingabeaufforderung lässt sich setzen, indem eine Prozedur "prompt2" definiert wird.

Während **interpreter** gilt der kanonische Modus, auch wenn sein Aufrufer den Rohmodus verwendet hat.

Ist stdin geschlossen, kehrt **interpreter** zurück, sofern nicht die Markierung **-eof** verwendet wird. Dann wird das folgende Argument aufgerufen.

log_file ***[args] [[-a] file]***

Ist ein Dateiname angegeben, zeichnet **log_file** ab diesem Punkt ein Protokoll der Sitzung in der Datei auf. Ohne Argument beendet **log_file** das Protokoll. Eine frühere Protokolldatei wird geschlossen. Statt eines Dateinamens kann mit den Markierungen **-open** oder **-leaveopen** ein Tcl-Dateibezeichner angegeben werden. Das ähnelt dem Befehl **spawn**. Weitere Angaben stehen bei **spawn**. Die Markierung **-a** erzwingt, dass Ausgabe protokolliert wird, die der Befehl **log_user** unterdrückt hat. In der Vorgabe *hängt* der Befehl **log_file** an alte Dateien an, statt sie abzuschneiden, damit sich das Protokollieren in einer Sitzung mehrmals aus- und einschalten lässt. Zum Abschneiden dient die Markierung **-noappend**. Die Markierung **-info** lässt log_file eine Beschreibung der zuletzt angegebenen Argumente zurückgeben, die nicht info sind.

log_user ***-info|0|1***

In der Vorgabe wird der Dialog send/expect nach stdout protokolliert und in eine Protokolldatei, wenn eine offen ist. Der Befehl "log_user 0" schaltet das Protokollieren nach stdout aus, "log_user 1" schaltet es wieder ein. Das Protokollieren in die Protokolldatei bleibt unverändert. Die Markierung **-info** lässt log_user eine Beschreibung der zuletzt angegebenen Argumente zurückgeben, die nicht info sind.

match_max ***[-d] [-i spawn_id] [size]***

Legt die Größe des Puffers in Bytes fest, den **expect** intern verwendet. Ohne Argument *size* kommt die aktuelle Größe zurück.

Mit der Markierung **-d** wird die Vorgabegröße gesetzt. Die anfängliche Vorgabe ist 2000. Mit der Markierung **-i** wird die Größe für die genannte Spawn-Id gesetzt, sonst für den aktuellen

Prozess.

overlay *[-# spawn_id] [-# spawn_id] [...] program [args]*
Führt *program args* anstelle des aktuellen Programms **Expect** aus, das sich beendet. Ein bloßes Bindestrich-Argument erzwingt einen Bindestrich vor dem Befehlsnamen, als wäre es eine Login-Shell. Alle spawn_ids werden geschlossen, außer den als Argumente genannten. Diese werden auf die genannten Dateibezeichner abgebildet.

Spawn-Ids werden auf Dateibezeichner abgebildet, die das neue Programm erbt. Die folgende Zeile lässt zum Beispiel chess laufen und vom aktuellen Prozess steuern, etwa einem Schachmeister.

```
overlay -0 $spawn_id -1 $spawn_id -2 $spawn_id chess
```

Das ist effizienter als "interact -u", gibt aber die Möglichkeit auf, den Dialog weiter per Programm zu steuern, weil der Prozess **Expect** nicht mehr die Steuerung hat.

Ein steuerndes Terminal wird nicht bereitgestellt. Wird die Verbindung getrennt oder die Standardeingabe umgelegt, arbeiten Programme mit Jobsteuerung, also Shells, login und Ähnliches, nicht richtig.

parity *[-d] [-i spawn_id] [value]*
Legt fest, ob die Parität in der Ausgabe gestarteter Prozesse erhalten oder entfernt wird. Ist *value* null, wird die Parität entfernt, sonst nicht. Ohne Argument *value* kommt der aktuelle Wert zurück.

Mit der Markierung **-d** wird der Vorgabewert der Parität gesetzt. Die anfängliche Vorgabe ist 1, die Parität wird also nicht entfernt. Mit der Markierung **-i** wird der Paritätswert für die genannte Spawn-Id gesetzt, sonst für den aktuellen Prozess.

remove_nulls *[-d] [-i spawn_id] [value]*
Legt fest, ob Nullen in der Ausgabe gestarteter Prozesse erhalten oder entfernt werden, bevor Muster geprüft oder in der Variable *expect_out* oder *interact_out* gespeichert werden. Ist *value* 1, werden Nullen entfernt. Ist *value* 0, werden Nullen nicht entfernt. Ohne Argument *value* kommt der aktuelle Wert zurück.

Mit der Markierung **-d** wird der Vorgabewert gesetzt. Die anfängliche Vorgabe ist 1, Nullen werden also entfernt. Mit der Markierung **-i** wird der Wert für die genannte Spawn-Id gesetzt, sonst für den aktuellen Prozess. Ob Nullen entfernt werden oder nicht, zeichnet **Expect** Nullbytes im Protokoll und auf stdout auf.

send *[-flags] string*
Sendet *string* an den aktuellen Prozess. Zum Beispiel der Befehl

```
send "hello world\r"
```

sendet die Zeichen h e l l o <blank> w o r l d <return> an den aktuellen Prozess. Tcl enthält einen printf-ähnlichen Befehl, **format**, der beliebig verwickelte Zeichenketten bauen kann.

Die Zeichen werden sofort gesendet, obwohl Programme mit zeilenweise gepufferter Eingabe die Zeichen erst lesen, wenn ein Return-Zeichen gesendet wird. Ein Return-Zeichen wird als "\r" geschrieben. Die Markierung **--** erzwingt, dass das nächste Argument als Zeichenkette gelesen wird und nicht als Markierung. Jeder Zeichenkette darf "--" voranstehen, ob sie wie

eine Markierung aussieht oder nicht. Das ist ein verlässlicher Weg, veränderliche Zeichenketten anzugeben, ohne an solchen zu scheitern, die zufällig wie Markierungen aussehen. Alle Zeichenketten, die mit "-" beginnen, sind für künftige Optionen reserviert. Die Markierung **-i** legt fest, dass die Zeichenkette an die genannte spawn_id gesendet wird. Ist die spawn_id *user_spawn_id* und steht das Terminal im Rohmodus, werden Zeilenumbrüche in der Zeichenkette in Folgen aus Return und Zeilenumbruch übersetzt, sodass sie aussehen, als stünde das Terminal im kanonischen Modus. Die Markierung **-raw** schaltet diese Übersetzung aus. Die Markierung **-null** sendet Nullzeichen, 0 Bytes. In der Vorgabe wird eine Null gesendet. Auf **-null** darf eine ganze Zahl folgen, die angibt, wie viele Nullen gesendet werden. Die Markierung **-break** erzeugt eine Break-Bedingung. Das ist nur sinnvoll, wenn die Spawn-Id ein tty-Gerät meint, das über "spawn -open" geöffnet wurde. Wurde ein Prozess wie tip gestartet, gilt zum Erzeugen eines Breaks die Konvention von tip. Die Markierung **-s** erzwingt, dass die Ausgabe "langsam" gesendet wird, und vermeidet so den häufigen Fall, dass ein Rechner einen Eingabepuffer schneller füllt, als ein Mensch denselben Puffer tippen würde. Diese Ausgabe steuert der Wert der Variable "send_slow", die eine Liste aus zwei Elementen annimmt. Das erste Element ist eine ganze Zahl, die angibt, wie viele Bytes unteilbar gesendet werden. Das zweite Element ist eine reelle Zahl, die angibt, wie viele Sekunden die unteilbaren Sendungen trennen müssen. "set send_slow {10 .001}" würde zum Beispiel "send -s" zwingen, Zeichenketten mit 1 Millisekunde Abstand zwischen jeweils 10 gesendeten Zeichen zu senden. Die Markierung **-h** erzwingt, dass die Ausgabe einigermaßen so gesendet wird, wie ein Mensch wirklich tippt. Zwischen den Zeichen erscheinen menschenähnliche Verzögerungen. Der Algorithmus beruht auf einer Weibull-Verteilung, mit Änderungen für diese Anwendung. Diese Ausgabe steuert der Wert der Variable "send_human", die eine Liste aus fünf Elementen annimmt. Die ersten beiden Elemente sind die mittlere Zwischenankunftszeit der Zeichen in Sekunden. Das erste gilt in der Vorgabe. Das zweite gilt an Wortenden, um die feinen Pausen nachzubilden, die an solchen Übergängen gelegentlich auftreten. Der dritte Parameter misst die Streuung: 0,1 ist sehr veränderlich, 1 ist einigermaßen veränderlich, und 10 ist sehr gleichmäßig. Die Extreme sind 0 bis unendlich. Die letzten beiden Parameter sind eine kleinste und eine größte Zwischenankunftszeit. Minimum und Maximum werden zuletzt angewendet und schneiden die endgültige Zeit ab. Der endgültige Mittelwert kann sich deutlich vom angegebenen Mittelwert unterscheiden, wenn Minimum und Maximum genug Werte abschneiden. Der folgende Befehl bildet zum Beispiel einen schnellen und gleichmäßigen Tipper nach:

```
set send_human {.1 .3 1 .05 2}
send -h "I'm hungry.  Let's do lunch."
```

Das Folgende passt eher nach einem Kater:

```
set send_human {.4 .4 .2 .5 100}
send -h "Goodd party lash night!"
```

Fehler werden nicht nachgebildet. Situationen zur Fehlerkorrektur lassen sich selbst anlegen, indem Tippfehler und Korrekturen in ein send-Argument eingebettet werden.

Die Markierungen zum Senden von Nullzeichen, zum Senden von Breaks, zum Erzwingen langsamer Ausgabe und zur Ausgabe im menschlichen Stil schließen einander aus. Nur die zuletzt angegebene wird verwendet. Außerdem darf bei den Markierungen zum Senden von Nullzeichen oder Breaks kein Argument *string* angegeben werden.

Dem ersten **send** an einen Prozess sollte ein **expect** vorangehen. **expect** wartet, bis der Prozess startet, **send** kann das nicht. Endet das erste **send** insbesondere, bevor der Prozess läuft, können die Daten übergangen werden. Bieten interaktive Programme keine anfängliche Eingabeaufforderung, kann **send** eine Verzögerung vorangehen, wie hier:

```
# To avoid giving hackers hints on how to break in,
# this system does not prompt for an external password.
# Wait for 5 seconds for exec to complete
spawn telnet very.secure.gov
sleep 5
send password\r
```

exp_send ist ein Alias für **send**. Läuft Expectk oder eine andere Variante von Expect in der Tk-Umgebung, definiert Tk **send** für einen ganz anderen Zweck. **exp_send** sorgt für die Verträglichkeit zwischen den Umgebungen. Ähnliche Aliase gibt es für die anderen send-Befehle von Expect.

send_error ***[-flags] string***
: Arbeitet wie **send**, nur geht die Ausgabe nach stderr und nicht an den aktuellen Prozess.

send_log ***[--] string***
: Arbeitet wie **send**, nur geht die Zeichenkette allein in die Protokolldatei. Siehe dazu **log_file**. Ist keine Protokolldatei offen, werden die Argumente übergangen.

send_tty ***[-flags] string***
: Arbeitet wie **send**, nur geht die Ausgabe nach /dev/tty und nicht an den aktuellen Prozess.

send_user ***[-flags] string***
: Arbeitet wie **send**, nur geht die Ausgabe nach stdout und nicht an den aktuellen Prozess.

sleep ***seconds***
: Lässt das Skript die angegebene Zahl von Sekunden schlafen. Sekunden dürfen eine Dezimalzahl sein. Während Expect schläft, werden Unterbrechungen verarbeitet, und Tk-Ereignisse, wenn Expectk läuft.

spawn ***[args] program [args]***
: Legt einen neuen Prozess an, der *program args* ausführt. Seine Standardeingabe, Standardausgabe und Standardfehlerausgabe sind mit Expect verbunden, sodass andere Befehle von **Expect** sie lesen und schreiben können. Die Verbindung bricht durch **close** oder wenn der Prozess selbst einen der Dateibezeichner schließt.

  Startet **spawn** einen Prozess, wird die Variable *spawn_id* auf einen Deskriptor gesetzt, der diesen Prozess meint. Der Prozess, den *spawn_id* beschreibt, gilt als *aktueller Prozess*. *spawn_id* darf gelesen oder geschrieben werden und bietet so eine Jobsteuerung.

  user_spawn_id ist eine globale Variable mit einem Deskriptor, der den Benutzer meint. Wird *spawn_id* zum Beispiel auf diesen Wert gesetzt, verhält sich **expect** wie **expect_user**. *error_spawn_id* ist eine globale Variable mit einem Deskriptor, der die Standardfehlerausgabe meint. Wird *spawn_id* zum Beispiel auf diesen Wert gesetzt, verhält sich **send** wie **send_error**.

  tty_spawn_id ist eine globale Variable mit einem Deskriptor, der /dev/tty meint. Gibt es /dev/tty nicht, etwa in einem cron-, at- oder batch-Skript, ist *tty_spawn_id* nicht definiert. Das

lässt sich so prüfen:

```
if {[info vars tty_spawn_id]} {
    # /dev/tty exists
} else {
    # /dev/tty doesn't exist
    # probably in cron, batch, or at script
}
```

spawn gibt die UNIX-Prozesskennung zurück. Wird kein Prozess gestartet, kommt 0 zurück. Die Variable *spawn_out(slave,name)* wird auf den Namen des Pty-Slave-Geräts gesetzt.

In der Vorgabe gibt **spawn** den Befehlsnamen und die Argumente aus. Die Markierung **-noecho** hält **spawn** davon ab.

Die Markierung **-console** leitet die Konsolenausgabe an den gestarteten Prozess um. Das wird nicht auf allen Systemen unterstützt. Intern verwendet **spawn** eine Pty, die wie das tty des Benutzers eingerichtet wird. Diese Pty wird weiter so eingerichtet, dass alle Einstellungen "sane" sind, laut stty(1). Ist die Variable *stty_init* definiert, wird sie im Stil von stty-Argumenten als weitere Einrichtung gelesen. "set stty_init raw" lässt zum Beispiel die Terminals später gestarteter Prozesse im Rohmodus beginnen. **-nottycopy** überspringt die Einrichtung nach dem tty des Benutzers. **-nottyinit** überspringt die Einrichtung "sane".

Gewöhnlich braucht **spawn** wenig Zeit. Dauert spawn auffällig lange, trifft es vermutlich auf festgefahrene Ptys. Auf Ptys laufen mehrere Prüfungen, damit es keine Verwicklung mit fehlgeleiteten Prozessen gibt. Das dauert 10 Sekunden je festgefahrener Pty. Läuft Expect mit der Option **-d**, zeigt sich, ob **Expect** viele Ptys in merkwürdigen Zuständen trifft. Lassen sich die Prozesse nicht beenden, an denen diese Ptys hängen, bleibt als einziger Ausweg ein Neustart. Kann *program* nicht erfolgreich gestartet werden, weil exec(2) fehlschlägt, etwa weil *program* nicht existiert, gibt der nächste Befehl **interact** oder **expect** eine Fehlermeldung zurück, als wäre *program* gelaufen und hätte die Fehlermeldung als Ausgabe erzeugt. Dieses Verhalten folgt aus der Umsetzung von **spawn**. Intern spaltet spawn ab. Danach kann der gestartete Prozess mit dem ursprünglichen Prozess **Expect** nur noch über die spawn_id kommunizieren. Die Markierung **-open** lässt das nächste Argument als Tcl-Dateibezeichner lesen, also als Rückgabe von **open**. Die Spawn-Id lässt sich dann verwenden, als wäre sie ein gestarteter Prozess. Der Dateibezeichner sollte nicht mehr verwendet werden. So lassen sich Rohgeräte, Dateien und Pipelines als gestartete Prozesse behandeln, ohne eine Pty. 0 kommt zurück und zeigt, dass kein Prozess zugehört. Wird die Verbindung zum gestarteten Prozess geschlossen, wird auch der Tcl-Dateibezeichner geschlossen. Die Markierung **-leaveopen** ähnelt **-open**, nur lässt **-leaveopen** den Dateibezeichner offen, auch nachdem die Spawn-Id geschlossen ist. Die Markierung **-pty** öffnet eine Pty, startet aber keinen Prozess. 0 kommt zurück und zeigt, dass kein Prozess zugehört. Die Spawn-Id wird wie üblich gesetzt. Die Variable *spawn_out(slave,fd)* wird auf einen Dateibezeichner gesetzt, der dem Pty-Slave entspricht. Er lässt sich mit "close -slave" schließen. Die Markierung **-ignore** nennt ein Signal, das im gestarteten Prozess übergangen wird. Sonst erhalten Signale das Vorgabeverhalten. Signale werden wie im Befehl **trap** benannt, nur braucht jedes Signal eine eigene Markierung.

strace *level*

Lässt die folgenden Anweisungen ausgeben, bevor sie ausgeführt werden. Der trace-Befehl von Tcl verfolgt Variablen. *level* gibt an, wie weit im Aufrufstapel verfolgt wird. Der folgende Befehl lässt zum Beispiel **Expect** laufen und verfolgt die ersten 4 Aufrufebenen, aber keine darunter.

```
expect -c "strace 4" script.exp
```

Die Markierung **-info** lässt strace eine Beschreibung der zuletzt angegebenen Argumente zurückgeben, die nicht info sind.

stty *args*

Ändert die Terminalmodi ähnlich wie der externe Befehl stty. In der Vorgabe wird das steuernde Terminal angesprochen. Andere Terminals lassen sich ansprechen, indem "< /dev/tty..." an den Befehl angehängt wird. Die Argumente sollten nicht zu einem einzigen Argument zusammengefasst werden. Statusanfragen geben den Status als Ergebnis des Befehls zurück. Wird kein Status angefragt und das steuernde Terminal angesprochen, kommt der vorige Status der Attribute raw und echo in einer Form zurück, die der Befehl später verwenden kann. Die Argumente **raw** oder **-cooked** setzen das Terminal zum Beispiel in den Rohmodus. Die Argumente **-raw** oder **cooked** setzen das Terminal in den kanonischen Modus. Die Argumente **echo** und **-echo** setzen das Terminal in den Echo- beziehungsweise den Noecho-Modus.

Das folgende Beispiel zeigt, wie das Echo vorübergehend ausgeschaltet wird. Das kann in sonst automatischen Skripten verhindern, dass Passwörter darin stehen. Weitere Erörterung dazu steht unter den Expect-Hinweisen weiter unten.

```
stty -echo
send_user "Password: "
expect_user -re "(.*)\n"
set password $expect_out(1,string)
stty echo
```

system *args*

Gibt *args* an sh(1) als Eingabe, als wäre es als Befehl von einem Terminal getippt worden. **Expect** wartet, bis die Shell endet. Der Rückgabestatus von sh wird so behandelt, wie **exec** seinen Rückgabestatus behandelt.

exec leitet stdin und stdout an das Skript um. **system** nimmt keine Umleitung vor, außer der, die die Zeichenkette selbst angibt. So lassen sich Programme verwenden, die unmittelbar mit /dev/tty sprechen müssen. Aus demselben Grund werden die Ergebnisse von **system** nicht im Protokoll aufgezeichnet.

timestamp *[args]*

Gibt einen Zeitstempel zurück. Ohne Argumente kommt die Zahl der Sekunden seit der Epoche zurück. Die Markierung **-format** leitet eine Zeichenkette ein, die zurückgegeben wird, aber mit Ersetzungen nach den POSIX-Regeln für strftime. %a wird zum Beispiel durch einen abgekürzten Wochentagsnamen ersetzt, also Sat. Weitere sind:

```
    %a      abbreviated weekday name
    %A      full weekday name
    %b      abbreviated month name
    %B      full month name
    %c      date-time as in: Wed Oct  6 11:45:56 1993
    %d      day of the month (01-31)
    %H      hour (00-23)
    %I      hour (01-12)
    %j      day (001-366)
    %m      month (01-12)
    %M      minute (00-59)
    %p      am or pm
    %S      second (00-61)
    %u      day (1-7, Monday is first day of week)
    %U      week (00-53, first Sunday is first day of week one)
    %V      week (01-53, ISO 8601 style)
    %w      day (0-6)
    %W      week (00-53, first Monday is first day of week one)
    %x      date-time as in: Wed Oct  6 1993
    %X      time as in: 23:59:59
    %y      year (00-99)
    %Y      year as in: 1993
    %Z      timezone (or nothing if not determinable)
    %%      a bare percent sign
```

Andere %-Angaben sind undefiniert. Andere Zeichen gehen unverändert durch. Nur die Locale C wird unterstützt.

Die Markierung **-seconds** leitet eine Zahl von Sekunden seit der Epoche ein, die als Quelle für die Formatierung dient. Sonst gilt die aktuelle Zeit.

Die Markierung **-gmt** erzwingt, dass die Ausgabe von timestamp die Zeitzone GMT verwendet. Ohne Markierung gilt die lokale Zeitzone.

trap *[[command] signals]*

Lässt den angegebenen *command* bei künftigem Empfang eines der angegebenen Signale ausführen. Der Befehl wird im globalen Geltungsbereich ausgeführt. Fehlt *command*, kommt die Signalaktion zurück. Ist *command* die Zeichenkette SIG_IGN, werden die Signale übergangen. Ist *command* die Zeichenkette SIG_DFL, werden die Signale auf die Systemvorgabe zurückgesetzt. *signals* ist entweder ein einzelnes Signal oder eine Liste von Signalen. Signale dürfen numerisch oder symbolisch angegeben werden, wie in signal(3). Die Vorsilbe "SIG" darf entfallen. Ohne Argumente, oder mit dem Argument -number, gibt **trap** die Signalnummer des trap-Befehls zurück, der gerade ausgeführt wird. Die Markierung **-code** verwendet den Rückgabecode des Befehls anstelle des Codes, den Tcl zurückgeben wollte, als der Befehl ursprünglich zu laufen begann. Die Markierung **-interp** lässt den Befehl mit dem Interpreter auswerten, der aktiv war, als der Befehl zu laufen begann, und nicht mit dem, der galt, als der Trap vereinbart wurde. Die Markierung **-name** lässt den Befehl **trap** den Signalnamen des trap-Befehls zurückgeben, der gerade ausgeführt wird. Die Markierung **-max** lässt den Befehl **trap** die größte Signalnummer zurückgeben, die gesetzt werden kann.

Der Befehl "trap {send_user "Ouch!"} SIGINT" gibt zum Beispiel jedes Mal "Ouch!" aus, wenn der Benutzer ^C drückt. In der Vorgabe lassen SIGINT, gewöhnlich durch ^C erzeugt, und SIGTERM Expect enden. Das liegt an folgendem Trap, der in der Vorgabe angelegt wird, wenn Expect startet.

```
trap exit {SIGINT SIGTERM}
```

Wird der Debugger mit der Markierung -D gestartet, wird SIGINT umdefiniert, sodass es den interaktiven Debugger startet. Das liegt an folgendem Trap:

```
trap {exp_debug 1} SIGINT
```

Der Debugger-Trap lässt sich ändern, indem die Umgebungsvariable EXPECT_DEBUG_INIT auf einen neuen trap-Befehl gesetzt wird.

Beide lassen sich überschreiben, indem trap-Befehle in das Skript kommen. Ein eigenes "trap exit SIGINT" überschreibt insbesondere den Debugger-Trap. Das ist nützlich, wenn Benutzer den Debugger gar nicht erreichen sollen.

Soll für SIGINT ein eigener Trap gelten, bei laufendem Debugger aber weiter der Debugger angesprungen werden, dient:

```
if {![exp_debug]} {trap mystuff SIGINT}
```

Stattdessen lässt sich der Debugger über ein anderes Signal anspringen.

trap lässt die Aktion für SIGALRM nicht überschreiben, weil **Expect** das Signal intern verwendet. Der Befehl disconnect setzt SIGALRM auf SIG_IGN (ignore). Das lässt sich wieder einschalten, solange es bei den folgenden spawn-Befehlen ausgeschaltet wird.

Weitere Angaben stehen in signal(3).

wait *[args]*

Wartet, bis ein gestarteter Prozess endet, oder der aktuelle Prozess, wenn keiner genannt ist.

wait gibt gewöhnlich eine Liste aus vier ganzen Zahlen zurück. Die erste ist die pid des Prozesses, auf den gewartet wurde. Die zweite ist die zugehörige Spawn-Id. Die dritte ist -1, wenn ein Fehler des Betriebssystems auftrat, sonst 0. War die dritte Zahl 0, ist die vierte der Status, den der gestartete Prozess zurückgegeben hat. War die dritte Zahl -1, ist die vierte der Wert von errno, den das Betriebssystem gesetzt hat. Die globale Variable errorCode wird ebenfalls gesetzt. Am Ende des Rückgabewerts von **wait** können weitere Elemente stehen. Ein optionales fünftes Element nennt eine Klasse von Angaben. Derzeit ist der einzige mögliche Wert dieses Elements CHILDKILLED. Dann sind die nächsten beiden Werte der Signalname im C-Stil und eine kurze Textbeschreibung.

Die Markierung **-i** nennt den Prozess, auf den gewartet wird, über die genannte spawn_id, nicht über die Prozesskennung. In einem SIGCHLD-Handler lässt sich mit der Spawn-Id -1 auf jeden gestarteten Prozess warten. Die Markierung **-nowait** lässt wait sofort zurückkehren und ein erfolgreiches Warten melden. Endet der Prozess später, verschwindet er von selbst, ohne ein ausdrückliches wait. Der Befehl **wait** kann mit den Argumenten "-i -1" auch auf einen abgespaltenen Prozess warten. Anders als bei gestarteten Prozessen darf dieser Befehl jederzeit ausgeführt werden. Welcher Prozess eingesammelt wird, lässt sich nicht steuern. Der Rückgabewert kann aber auf die Prozesskennung geprüft werden.

BIBLIOTHEKEN

Expect kennt von selbst zwei eingebaute Bibliotheken für Expect-Skripte. Die Bibliotheken sind durch die Verzeichnisse bestimmt, die in den Variablen exp_library und exp_exec_library genannt sind. Beide sollen Hilfsdateien enthalten, die andere Skripte verwenden können.

exp_library enthält architekturunabhängige Dateien. exp_exec_library enthält architekturabhängige Dateien. Je nach System können beide Verzeichnisse völlig leer sein. Ob die Datei $exp_exec_library/cat-buffers existiert, beschreibt, ob /bin/cat in der Vorgabe puffert.

FORMATIERTE AUSGABE

Für die formatierte Ausgabe von **Expect**-Skripten gibt es eine vgrind-Definition. Ist die vgrind-Definition aus der **Expect**-Distribution richtig installiert, lässt sie sich so verwenden:

```
    vgrind -lexpect file
```

BEISPIELE

Es ist nicht ohne Weiteres zu sehen, wie alles zusammengehört, was die Handbuchseite beschreibt. Die Beispiele im Beispielverzeichnis der **Expect**-Distribution sollten gelesen und ausprobiert werden. Manche sind echte Programme. Andere veranschaulichen nur bestimmte Techniken, und ein paar sind nur schnelle Behelfslösungen. Die Datei INSTALL gibt einen kurzen Überblick über diese Programme.

Die Aufsätze zu **Expect** (siehe SEE ALSO) sind ebenfalls nützlich. Manche Aufsätze verwenden die Syntax früherer Expect-Versionen. Die zugehörigen Begründungen gelten weiter und gehen viel mehr ins Einzelne als diese Handbuchseite.

EINSCHRÄNKUNGEN

Erweiterungen können mit den Befehlsnamen von Expect zusammenstoßen. **send** ist zum Beispiel von Tk für einen ganz anderen Zweck definiert. Deshalb stehen die meisten Befehle von **Expect** auch als "exp_XXXX" zur Verfügung. Befehle und Variablen, die mit "exp", "inter", "spawn" und "timeout" beginnen, haben keine Aliase. Die erweiterten Befehlsnamen dienen der Verträglichkeit zwischen den Umgebungen.

Expect fasst Geltungsbereiche recht großzügig. Variablen, die Befehle des Programms **Expect** lesen, werden insbesondere zuerst im lokalen Geltungsbereich gesucht und, wenn sie dort fehlen, im globalen. Damit entfällt "global timeout" in jeder Prozedur, die **expect** verwendet. Geschriebene Variablen liegen dagegen immer im lokalen Geltungsbereich, sofern kein Befehl "global" abgesetzt wurde. Das häufigste Problem entsteht, wenn spawn in einer Prozedur ausgeführt wird. Außerhalb der Prozedur gibt es *spawn_id* nicht mehr, und der gestartete Prozess ist allein wegen des Geltungsbereichs nicht mehr erreichbar. In eine solche Prozedur kommt "global spawn_id".

Lässt sich die Fähigkeit nicht einschalten, mehrere Prozesse zu starten, weil das System weder select (BSD *.*), poll (SVR>2) noch etwas Gleichwertiges unterstützt, kann **Expect** nur einen Prozess zur Zeit steuern. In diesem Fall darf *spawn_id* nicht gesetzt werden, und über exec dürfen keine Prozesse ausgeführt werden, solange ein gestarteter Prozess läuft. Außerdem lässt sich nicht gleichzeitig von mehreren Prozessen **expect** ausführen, den Benutzer als einen davon eingeschlossen.

Terminalparameter können Skripte stark beeinflussen. Ist ein Skript zum Beispiel auf Echo

geschrieben, verhält es sich falsch, wenn das Echo ausgeschaltet ist. Deshalb erzwingt Expect in der Vorgabe Terminalparameter im Zustand sane. Das kann andere Programme stören. Die emacs-Shell will zum Beispiel die "üblichen" Zuordnungen ändern: Zeilenumbrüche werden auf Zeilenumbrüche abgebildet statt auf Zeilenumbrüche mit Wagenrücklauf, und das Echo ist ausgeschaltet. So lässt sich die Eingabezeile mit emacs bearbeiten. Expect kann das unmöglich erraten.

Expect kann angewiesen werden, seine Vorgabe für die Terminalparameter nicht zu überschreiben. Dann ist beim Schreiben von Skripten für solche Umgebungen große Vorsicht nötig. Im Fall von emacs wird nicht auf Echo und Zuordnungen am Zeilenende gebaut.

Die Befehle, die Argumente in einer einzigen Liste mit geschweiften Klammern annehmen, die Varianten von **expect** und **interact**, verwenden eine Heuristik, um zu entscheiden, ob die Liste tatsächlich ein Argument oder viele ist. Die Heuristik kann nur fehlschlagen, wenn die Liste wirklich ein einziges Argument darstellt, das mehrere eingebettete \n mit Zeichen dazwischen enthält, die kein Leerraum sind. Das erscheint hinreichend unwahrscheinlich. Das Argument "-nobrace" kann trotzdem erzwingen, dass ein einziges Argument als ein einziges Argument behandelt wird. Das kann bei maschinell erzeugtem Expect-Code nützlich sein. Ebenso erzwingt -brace, dass ein einziges Argument als mehrere Muster und Aktionen behandelt wird.

BUGS

Es war sehr verlockend, das Programm "sex" zu nennen, für "Smart EXec" oder "Send-EXpect". Die Vernunft, oder vielleicht nur der Puritanismus, hat sich durchgesetzt.

Auf manchen Systemen beklagt sich eine gestartete Shell, das tty nicht erreichen zu können, läuft aber trotzdem. Das System hat dann einen Mechanismus, das steuernde tty zu erlangen, den **Expect** nicht kennt. Ist er bekannt, soll die Angabe an den Autor zurückgehen.

Ultrix 4.1, zumindest die letzten Versionen hier, behandelt Zeitspannen über 1000000 als gleichwertig mit 0.

Digital UNIX 4.0A, und vermutlich andere Versionen, verweigert die Zuteilung von Ptys, wenn ein SIGCHLD-Handler definiert wird. Weitere Angaben stehen auf der Seite grantpt.

IRIX 6.0 behandelt Pty-Rechte falsch. Versucht Expect, eine Pty zuzuteilen, die vorher jemand anders benutzt hat, schlägt das fehl. Abhilfe schafft ein Wechsel auf IRIX 6.1.

Telnet, nur unter SunOS 4.1.2 geprüft, hängt, wenn TERM nicht gesetzt ist. Das ist ein Problem unter cron, at und in cgi-Skripten, die TERM nicht definieren. Es muss also ausdrücklich gesetzt werden. Welcher Typ, ist meist unerheblich. Es muss nur irgendein Wert sein. Das Folgende genügt vermutlich in den meisten Fällen.

```
    set env(TERM) vt100
```

Tip, nur unter BSDI BSD/OS 3.1 i386 geprüft, hängt, wenn SHELL und HOME nicht gesetzt sind. Das ist ein Problem unter cron, at und in cgi-Skripten, die diese Umgebungsvariablen nicht definieren. Die Variablen müssen also ausdrücklich gesetzt werden. Welcher Wert, ist meist unerheblich. Es muss nur irgendein Wert sein. Das Folgende genügt vermutlich in den meisten Fällen.

```
    set env(SHELL) /bin/sh
    set env(HOME) /usr/local/bin
```

Manche Pty-Umsetzungen sind so gebaut, dass der Kern ungelesene Ausgabe 10 bis 15 Sekunden nach dem Schließen des Dateideskriptors durch den Prozess verwirft. Die genaue Zahl hängt von der Umsetzung ab. **Expect**-Programme wie diese schlagen deshalb fehl:

```
    spawn date
    sleep 20
    expect
```

Das lässt sich vermeiden, indem nicht interaktive Programme mit **exec** aufgerufen werden und nicht mit **spawn**. Solche Situationen sind denkbar. In der Praxis ist dem Autor kein Fall begegnet, in dem die letzte Ausgabe eines wirklich interaktiven Programms durch dieses Verhalten verloren ging.

Ptys von Cray UNICOS verwerfen ungelesene Ausgabe dagegen sofort, nachdem der Prozess den Dateideskriptor geschlossen hat. Das ist an Cray gemeldet, und dort wird an einer Korrektur gearbeitet.

Manchmal ist zwischen einer Eingabeaufforderung und einer Antwort eine Verzögerung nötig, etwa wenn eine tty-Schnittstelle UART-Einstellungen ändert oder Baudraten angleicht, indem sie nach Start- und Stoppbits sucht. Gewöhnlich genügt es, ein oder zwei Sekunden zu schlafen. Robuster ist, es zu wiederholen, bis die Hardware Eingabe annehmen kann. Das folgende Beispiel verwendet beide Wege:

```
    send "speed 9600\r";
    sleep 1
    expect {
        timeout {send "\r"; exp_continue}
        $prompt
    }
```

trap -code arbeitet mit keinem Befehl, der in der Ereignisschleife von Tcl sitzt, etwa sleep. In der Ereignisschleife verwirft Tcl die Rückgabecodes asynchroner Ereignisbehandler. Ein Ausweg ist, im Trap-Code eine Variable zu setzen und sie unmittelbar nach dem Befehl zu prüfen, also nach sleep.

Der Befehl expect_background übergeht Argumente -timeout und kennt Zeitüberschreitungen allgemein nicht.

EXPECT-HINWEISE

Einiges an **Expect** ist nicht naheliegend. Dieser Abschnitt geht auf einiges davon mit ein paar Vorschlägen ein.

Ein häufiges Problem von expect ist, Shell-Eingabeaufforderungen zu erkennen. Diese Eingabeaufforderungen werden von verschiedenen Personen und verschiedenen Shells unterschiedlich angepasst. rlogin lässt sich deshalb ohne Kenntnis der Eingabeaufforderung schwer so automatisieren, dass es auf anderen Systemen ebenso läuft. Eine vernünftige Vereinbarung ist, dass Benutzer einen regulären Ausdruck, der ihre Eingabeaufforderung beschreibt, insbesondere ihr Ende, in der Umgebungsvariable EXPECT_PROMPT ablegen. Code wie der folgende kann verwendet werden. Fehlt EXPECT_PROMPT, hat der Code trotzdem gute Aussicht, richtig zu arbeiten.

```
    set prompt "(%|#|\\$) $"          ;# default prompt
    catch {set prompt $env(EXPECT_PROMPT)}

    expect -re $prompt
```

Muster für **expect** sollten das Ende dessen enthalten, was erwartet wird. So wird vermieden, eine Frage zu beantworten, bevor sie ganz gesehen wurde. Fragen lassen sich zwar beantworten, bevor sie ganz sichtbar sind. Wird früh geantwortet, kann die Antwort mitten in der Frage zurückgesendet erscheinen. Der entstehende Dialog ist dann richtig, sieht aber durcheinander aus.

Die meisten Eingabeaufforderungen enden mit einem Leerzeichen. Die Eingabeaufforderung von ftp ist zum Beispiel 'f', 't', 'p', '>' und <blank>. Damit dieses Muster passt, muss jedes dieser Zeichen berücksichtigt werden. Ein häufiger Fehler ist, das Leerzeichen wegzulassen. Das Leerzeichen wird ausdrücklich ins Muster geschrieben.

Wird ein Muster der Form X* verwendet, passt das * auf die ganze Ausgabe vom Ende von X bis zum zuletzt Empfangenen. Das klingt naheliegend, kann aber verwirren, weil "zuletzt Empfangenes" je nach Geschwindigkeit des Rechners und der Ein-/Ausgabe-Verarbeitung durch Kern und Gerätetreiber verschieden sein kann.

Menschen sehen Programm-Ausgabe insbesondere gern in großen Stücken ankommen, unteilbar, während die meisten Programme in Wirklichkeit eine Zeile nach der anderen erzeugen. Trifft das zu, passt das * im Muster des vorigen Absatzes vielleicht nur auf das Ende der aktuellen Zeile, obwohl mehr zu kommen scheint, weil zum Zeitpunkt der Übereinstimmung nur diese Ausgabe empfangen worden war.

expect kann nicht wissen, dass weitere Ausgabe kommt, wenn das Muster das nicht ausdrücklich berücksichtigt.

Sich auf zeilenorientiertes Puffern zu verlassen, ist unklug. Programme machen selten Zusagen über die Art ihres Pufferns, und Systemüberlastung kann Ausgabezeilen so zerteilen, dass die Zeilen an scheinbar zufälligen Stellen umbrechen. Lassen sich beim Schreiben von Mustern die letzten Zeichen einer Eingabeaufforderung ausdrücken, ist das ratsam.

Wird auf ein Muster in der letzten Ausgabe eines Programms gewartet und das Programm gibt stattdessen etwas anderes aus, lässt sich das mit dem Schlüsselwort **timeout** nicht erkennen. **expect** läuft dann nicht in die Zeitüberschreitung, sondern erhält eine Anzeige **eof**. Die wird verwendet. Besser ist, beide zu verwenden. Wird die Zeile später verschoben, muss die Zeile selbst nicht geändert werden.

Zeilenumbrüche werden vom Terminaltreiber gewöhnlich in Folgen aus Wagenrücklauf und Zeilenvorschub umgewandelt. Soll ein Muster die beiden Zeilen ausdrücklich treffen, etwa aus printf("foo\nbar"), dient das Muster "foo\r\nbar".

Eine ähnliche Übersetzung geschieht beim Lesen vom Benutzer, über **expect_user**. Wird Return gedrückt, wird das in einen Zeilenumbruch übersetzt. Gibt **Expect** das dann an ein Programm weiter, das sein Terminal in den Rohmodus setzt, wie telnet, entsteht ein Problem, weil das Programm ein echtes Return erwartet. Manche Programme sind nachsichtig und übersetzen Zeilenumbrüche von selbst in Returns, die meisten nicht. Es gibt leider keinen Weg zu erkennen, dass ein Programm sein Terminal in den Rohmodus gesetzt hat.

Statt Zeilenumbrüche von Hand durch Returns zu ersetzen, dient der Befehl "stty raw", der die

Übersetzung anhält. Damit entfallen allerdings die Funktionen zur Zeilenbearbeitung im kanonischen Modus.

interact setzt das Terminal implizit in den Rohmodus, sodass dieses Problem dann nicht auftritt.

Es ist oft nützlich, Passwörter oder andere private Angaben in **Expect**-Skripten zu speichern. Das ist nicht zu empfehlen, weil alles, was auf einem Rechner liegt, von jedem gelesen werden kann. Passwörter interaktiv aus einem Skript abzufragen, ist klüger, als sie wörtlich einzubetten. Trotzdem ist das Einbetten manchmal die einzige Möglichkeit.

Das UNIX-Dateisystem hat keinen unmittelbaren Weg, Skripte anzulegen, die ausführbar, aber nicht lesbar sind. Systeme, die setgid-Shellskripte unterstützen, können das mittelbar so nachbilden:

Das **Expect**-Skript mit den geheimen Daten wird wie üblich angelegt. Seine Rechte werden 750 (-rwxr-x---), und es gehört einer vertrauenswürdigen Gruppe, also einer Gruppe, die es lesen darf. Nötigenfalls wird dafür eine neue Gruppe angelegt. Danach wird ein /bin/sh-Skript mit den Rechten 2751 (-rwxr-s--x) angelegt, das derselben Gruppe gehört.

Das Ergebnis ist ein Skript, das jeder ausführen und lesen darf. Beim Aufruf führt es das **Expect** -Skript aus.

AUTOREN

Don Libes, National Institute of Standards and Technology

DANKSAGUNGEN

Dank an John Ousterhout für Tcl und an Scott Paisley für die Anregung. Dank an Rob Savoye für den Autokonfigurations-Code von Expect.

Die Datei HISTORY dokumentiert einen großen Teil der Entwicklung von **expect**. Die Datei ist lesenswert und kann weiteren Einblick in diese Software geben. Dank an die dort genannten Personen, die Fehlerkorrekturen geschickt und sonst geholfen haben.

Entwurf und Umsetzung von **Expect** wurden zum Teil von der Regierung der USA bezahlt und sind deshalb gemeinfrei. Der Autor und das NIST wünschen eine Nennung, wenn dieses Programm und diese Dokumentation oder Teile davon verwendet werden.

libexpect

Stichwörter: libexpect

NAME

libexpect - Bibliothek für programmierten Dialog mit interaktiven Programmen

BESCHREIBUNG

Diese Bibliothek enthält Funktionen, mit denen Expect als Tcl-Erweiterung oder unmittelbar aus C oder C++ verwendet werden kann, ohne Tcl. Expect als Tcl-Erweiterung hinzuzufügen, ist sehr kurz und einfach. Das kommt zuerst.

ÜBERSICHT

#include expect_tcl.h
Expect_Init(interp);

cc files... -lexpect5.20 -ltcl7.5 -lm

Hinweis: Die Bibliotheksversionen können in der tatsächlichen Ausgabe abweichen.

Die Funktion Expect_Init fügt dem genannten Interpreter expect-Befehle hinzu. Vorhandene Befehle werden nicht überschrieben. Aliase, die mit "exp_" beginnen, werden für expect-Befehle aber immer angelegt. "send" kann zum Beispiel als "exp_send" verwendet werden.

Expect-Befehle werden nur über Tcl_Eval aufgerufen. Bestimmte Hilfsfunktionen dürfen unmittelbar aufgerufen werden. Die Hilfsfunktionen sind unten zusammengefasst. Das kann nützlich sein, um eine eigene main zu bauen. Die Datei exp_main_exp.c in der Expect-Distribution ist eine main als Vorbild. Ein weiteres Vorbild ist tclAppInit.c in der Tcl-Distribution. Ein Vorbild für die Arbeit mit Tk steht in exp_main_tk.c in der Expect-Distribution.

```
int exp_cmdlinecmds;
int exp_interactive;
FILE *exp_cmdfile;
char *exp_cmdfilename;
int exp_tcl_debugger_available;

void    exp_parse_argv(Tcl_Interp *,int argc,char **argv);
int exp_interpreter(Tcl_Interp *);
void    exp_interpret_cmdfile(Tcl_Interp *,FILE *);
void    exp_interpret_cmdfilename(Tcl_Interp *,char *);
void    exp_interpret_rcfiles(Tcl_Interp *,int my_rc,int sys_rc);
char *  exp_cook(char *s,int *len);
void    (*exp_app_exit)EXP_PROTO((Tcl_Interp *);
void    exp_exit(Tcl_Interp *,int status);
void    exp_exit_handlers(Tcl_Interp *);
void    exp_error(Tcl_Interp,char *,...);
```

exp_cmdlinecmds ist 1, wenn Expect mit Befehlen auf der Programm-Befehlszeile aufgerufen wurde, zum Beispiel mit "-c". **exp_interactive** ist 1, wenn Expect mit der Markierung -i aufgerufen wurde oder wenn keine Befehle und kein Skript aufgerufen werden. **exp_cmdfile** ist ein Strom, aus dem Expect Befehle liest. **exp_cmdfilename** ist der Name einer Datei, die Expect öffnet und aus der es Befehle liest. **exp_tcl_debugger_available** ist 1, wenn der Debugger scharf gemacht wurde.

exp_parse_argv liest die Darstellung der Befehlszeile. Je nachdem, was gefunden wird, werden die anderen hier aufgeführten Variablen passend eingerichtet. **exp_interpreter** fragt den Benutzer interaktiv nach Befehlen und wertet sie aus. **exp_interpret_cmdfile** liest den angegebenen Strom und wertet gefundene Befehle aus. **exp_interpret_cmdfilename** öffnet die genannte Datei und wertet gefundene Befehle aus. **exp_interpret_rcfiles** liest die .rc-Dateien und wertet sie aus. Ist my_rc null, wird ~/.expectrc übersprungen. Ist sys_rc null, wird die systemweite expectrc-Datei übersprungen. **exp_cook** gibt einen statischen Puffer zurück, der das Argument enthält, wobei Zeilenumbrüche durch Folgen aus Wagenrücklauf und Zeilenvorschub ersetzt sind. Der Hauptzweck ist, Meldungen zu erzeugen, ohne darauf zu achten, ob das Terminal im Rohmodus oder im kanonischen Modus steht. Ist die Länge null, wird sie über strlen berechnet. **exp_error** ist eine printf-ähnliche Funktion, die das Ergebnis nach interp->result schreibt.

ÜBERSICHT

#include <expect.h>

int
exp_spawnl(file, arg0 [, arg1, ..., argn] (char *)0);
char *file;
char *arg0, *arg1, ... *argn;

int
exp_spawnv(file,argv);
char *file, *argv[ ];

int
exp_spawnfd(fd);
int fd;

FILE *
exp_popen(command);
char *command;

extern int exp_pid;
extern int exp_ttyinit;
extern int exp_ttycopy;
extern int exp_console;
extern char *exp_stty_init;
extern void (*exp_close_in_child)();
extern void (*exp_child_exec_prelude)();
extern void **exp_close_tcl_files**();

cc files... -lexpect -ltcl -lm

BESCHREIBUNG

exp_spawnl und **exp_spawnv** spalten einen neuen Prozess ab, sodass der aktuelle Prozess dessen stdin, stdout und stderr schreiben und lesen kann. *file* ist der Name einer Datei, die ausgeführt werden soll. Die Zeiger *arg* sind nullterminierte Zeichenketten. Nach dem Stil von execve() ist *arg0*, oder *argv[0]*, üblicherweise ein Doppel des Dateinamens.

Es gibt vier Schnittstellen. **exp_spawnl** ist nützlich, wenn die Zahl der Argumente zur Übersetzungszeit bekannt ist. **exp_spawnv** ist nützlich, wenn die Zahl der Argumente zur Übersetzungszeit nicht bekannt ist. **exp_spawnfd** ist nützlich, wenn ein offener Dateideskriptor schon als Quelle vorliegt. **exp_popen** wird weiter unten erklärt.

Wird der Prozess erfolgreich angelegt, kommt ein Dateideskriptor zurück, der stdin, stdout und stderr des Prozesses entspricht. Mit fdopen() kann dem Dateideskriptor ein Strom zugeordnet werden. Darauf sollte fast sicher setbuf() folgen, um die Ein-/Ausgabe zu entpuffern.

Das Schließen des Dateideskriptors erkennt der Prozess gewöhnlich als EOF. Ist ein solcher Prozess beendet, sollte mit wait auf ihn gewartet werden, damit der Prozessplatz im Kern frei wird. Manche Systeme erlauben, das zu unterlassen, wenn das Signal SIGCHLD übergangen wird.

exp_popen ist eine weitere Schnittstelle, nach dem Vorbild von popen(). Die Funktion nimmt eine Befehlszeile der Bourne-Shell und gibt einen Strom zurück, der stdin, stdout und stderr des

Prozesses entspricht. Die Umsetzung von **exp_popen** unten zeigt **exp_spawnl**.

```
FILE *
exp_popen(program)
char *program;
{
    FILE *fp;
    int ec;

    if (0 > (ec = exp_spawnl("sh","sh","-c",program,(char *)0)))
        return(0);
    if (NULL == (fp = fdopen(ec,"r+")) return(0);
    setbuf(fp,(char *)0);
    return(fp);
}
```

Nachdem ein Prozess gestartet ist, wird die Variable **exp_pid** auf die Prozesskennung des neuen Prozesses gesetzt. Die Variable **exp_pty_slave_name** wird auf den Namen der Slave-Seite der Pty gesetzt.

Die spawn-Funktionen verwenden eine Pty, um mit dem Prozess zu kommunizieren. In der Vorgabe wird die Pty wie das tty des Benutzers eingerichtet, wenn das möglich ist, also wenn die Umgebung ein steuerndes Terminal hat. Diese Einrichtung lässt sich überspringen, indem exp_ttycopy auf 0 gesetzt wird.

Ist exp_ttyinit ungleich null, wird die Pty weiter auf einige systemweite Vorgaben eingerichtet. Die Vorgabe entspricht im Allgemeinen "stty sane".

Die tty-Einstellung lässt sich weiter ändern, indem die Variable **exp_stty_init** gesetzt wird. Diese Variable wird im Stil von stty-Argumenten gelesen. exp_stty_init = "sane"; wiederholt zum Beispiel die Vorgabeeinrichtung.

Auf manchen Systemen lässt sich die Konsolenausgabe auf Ptys umleiten. Wird das unterstützt, erhält der nächste spawn die Konsolenausgabe, wenn die Variable **exp_console** auf 1 gesetzt wird.

Zwischen dem Start eines Prozesses und der Übergabe der Steuerung an das neue Programm können die spawn-Funktionen die Umgebung aufräumen, indem sie Dateideskriptoren schließen. In der Vorgabe werden nur Dateideskriptoren geschlossen, die intern zu Expect gehören, und solche mit der Markierung "close-on-exec".

Weitere Dateideskriptoren lassen sich schließen, indem eine passende Funktion angelegt und exp_close_in_child zugewiesen wird. Die Funktion wird nach dem fork und vor dem exec aufgerufen. Das ändert auch das Verhalten des Befehls spawn in Expect.

Wird auch Tcl verwendet, kann die Funktion exp_close_tcl_files bequem sein. Die Funktion schließt alle Dateien zwischen den vorgegebenen Standard-Dateideskriptoren und dem höchsten Deskriptor, den Tcl kennt. Expect tut das.

Die Funktion exp_child_exec_prelude ist die letzte Funktion, die im Kind vor dem eigentlichen exec aufgerufen wird. Die Funktion lässt sich neu definieren, etwa um die uid oder die Signale zu verändern.

EIGENE PTY ZUTEILEN

```
extern int exp_autoallocpty;
extern int exp_pty[2];
```

Die spawn-Funktionen verwenden eine Pty, um mit dem Prozess zu kommunizieren. In der Vorgabe wird bei jedem Start eines Prozesses automatisch eine Pty zugeteilt. Sollen die Ptys selbst zugeteilt werden, wird vor dem Aufruf einer spawn-Funktion **exp_autoallocpty** auf 0 gesetzt, **exp_pty[0]** auf den Dateideskriptor der Master-Pty und **exp_pty[1]** auf den Dateideskriptor der Slave-Pty. Die expect-Bibliothek nimmt dann keine Pty-Einrichtung vor, exp_stty_init wird zum Beispiel nicht verwendet. Der Dateideskriptor der Slave-Pty wird automatisch geschlossen, wenn der Prozess gestartet wird. Nach dem Start des Prozesses läuft die weitere Kommunikation über den Dateideskriptor der Master-Pty.

exp_spawnl und **exp_spawnv** bilden das Suchen der Shell nach einer ausführbaren Datei in einer Verzeichnisliste nach. Die Verzeichnisliste kommt aus der Umgebung.

EXPECT-VERARBEITUNG

Aus einem Prozess, den **exp_spawnl** oder **exp_spawnv** gestartet hat, lässt sich mit read() lesen. Bequemere Funktionen stehen bereit. Es folgen:

```
int
exp_expectl(fd,type1,pattern1,[re1,],value1,type2,...,exp_end);
int fd;
enum exp_type type;
char *pattern1, *pattern2, ...;
regexp *re1, *re2, ...;
int value1, value2, ...;

int
exp_fexpectl(fp,type1,pattern1,[re1,]value1,type2,...,exp_end);
FILE *fp;
enum exp_type type;
char *pattern1, *pattern2, ...;
regexp *re1, *re2, ...;
int value1, value2, ...;

enum exp_type {
exp_end,
exp_glob,
exp_exact,
exp_regexp,
exp_compiled,
exp_null,
};

struct exp_case {
char *pattern;
```

```
regexp *re;
enum exp_type type;
int value;
};

int
exp_expectv(fd,cases);
int fd;
struct exp_case *cases;

int
exp_fexpectv(fp,cases);
FILE *fp;
struct exp_case *cases;

extern int exp_timeout;
extern char *exp_match;
extern char *exp_match_end;
extern char *exp_buffer;
extern char *exp_buffer_end;
extern int exp_match_max;
extern int exp_full_buffer;
extern int exp_remove_nulls;
```

Die Funktionen warten, bis die Ausgabe eines Prozesses auf eines der Muster passt, eine angegebene Zeitspanne vergangen ist oder ein EOF gesehen wird.

Das erste Argument jeder Funktion ist entweder ein Dateideskriptor oder ein Strom. Die folgenden Argumentgruppen beschreiben Muster und zugehörige ganze Zahlen, die zurückgegeben werden, wenn das Muster passt.

Das Argument type ist einer von mehreren Werten. exp_end zeigt, dass keine weiteren Muster folgen. exp_glob zeigt, dass das Muster eine Zeichenkette im Glob-Stil ist. exp_exact zeigt, dass das Muster eine genaue Zeichenkette ist. exp_regexp zeigt, dass das Muster eine Zeichenkette im Regexp-Stil ist. exp_compiled zeigt, dass das Muster eine Zeichenkette im Regexp-Stil ist und dass seine kompilierte Form ebenfalls angegeben wird. exp_null zeigt, dass das Muster eine Null ist. Zur Fehlersuche muss danach auch ein Zeichenkettenmuster folgen.

Wird die kompilierte Form bei den Funktionen **exp_expectl** und **exp_fexpectl** nicht angegeben, wird jede intern vorgenommene Musterkompilierung verworfen, nachdem die Funktion zurückgekehrt ist. Die Funktionen **exp_expectv** und **exp_fexpectv** kompilieren Muster automatisch und verwerfen sie nicht. Stattdessen werden sie verworfen, indem für jedes Muster free aufgerufen wird. Verwerfen ist nur beim letzten Verwenden der Fälle nötig.

Passende Regexp-Teilmuster werden in der kompilierten regexp gespeichert. Enthält "re" eine kompilierte regexp, steht die passende Zeichenkette in re->startp[0]. Die passenden Teilzeichenketten, nach den Klammern im ursprünglichen Muster, stehen in re->startp[1], re->startp[2] und so weiter bis re->startp[9]. Die zugehörigen Enden der Zeichenketten sind re->endp[x], wobei x derselbe Index ist wie beim Anfang der Zeichenkette.

Der Typ exp_null passt, wenn in der Eingabe eine Null erscheint. Die Variable exp_remove_nulls muss auf 0 gesetzt werden, damit Nullen nicht automatisch entfernt werden. In der Vorgabe ist exp_remove_nulls 1, und Nullen werden automatisch entfernt.

exp_expectv und **exp_fexpectv** sind nützlich, wenn die Zahl der Muster nicht im Voraus bekannt ist. Dann stehen die Gruppen in einem Feld. Das Ende des Feldes bezeichnet eine struct exp_case mit dem Typ exp_end. Im Rest dieser Erörterung werden diese Funktionen allgemein *expect* genannt.

Passt ein Muster, kommt die zugehörige ganze Zahl zurück. Die Werte müssen nicht eindeutig sein, sollten aber positiv sein, damit sie nicht mit EXP_EOF, EXP_TIMEOUT oder EXP_FULLBUFFER verwechselt werden. Bei EOF oder Zeitüberschreitung kommt der Wert **EXP_EOF** oder **EXP_TIMEOUT** zurück. Die Vorgabezeitspanne ist 10 Sekunden und lässt sich durch Setzen der Variable **exp_timeout** ändern. Der Wert -1 schaltet eine Zeitüberschreitung aus. Der Wert 0 lässt die expect-Funktion nach einem read() sofort zurückkehren, also pollen. Davor muss aber eine Funktion wie select, poll oder eine Rückruffunktion einer Ereignisverwaltung stehen, damit Daten zum Lesen da sind.

Ist die Variable exp_full_buffer 1, kommt EXP_FULLBUFFER zurück, wenn exp_buffer voll wird und kein Muster gepasst hat.

Kehrt die expect-Funktion zurück, zeigt **exp_buffer** auf den Puffer der Zeichen, die für den Vergleich betrachtet wurden. **exp_buffer_end** zeigt auf die Stelle ein Zeichen hinter dem letzten Zeichen in exp_buffer. Ist eine Übereinstimmung entstanden, zeigt **exp_match** in **exp_buffer** auf den Anfang der Übereinstimmung. **exp_match_end** zeigt auf das Zeichen hinter dem Ende der Übereinstimmung.

Kommt neue Eingabe, wird sie mit jedem Muster in der aufgeführten Reihenfolge verglichen. Das Fehlen einer Übereinstimmung lässt sich prüfen, indem das letzte Muster etwas ist, das sicher erscheint, etwa eine Eingabeaufforderung. Gibt es keine Eingabeaufforderung, ist auf **EXP_TIMEOUT** zu prüfen, so wie bei der Arbeit von Hand. Weitere Überlegungen und Vorgehensweisen zum Angeben von Mustern für **expect** stehen in der Dokumentation des Programms **expect** selbst. Weitere Angaben stehen unter SEE ALSO weiter unten.

Muster sind die üblichen regulären Ausdrücke im C-Shell-Stil. Das folgende Fragment sucht zum Beispiel eine erfolgreiche Anmeldung, etwa aus einem Telnet-Dialog.

```
    switch (exp_expectl(
        exp_glob,"connected",CONN,
        exp_glob,"busy",BUSY,
        exp_glob,"failed",ABORT,
        exp_glob,"invalid password",ABORT,
        exp_end)) {
    case CONN:  /* logged in successfully */
        break;
    case BUSY:  /* couldn't log in at the moment */
        break;
    case EXP_TIMEOUT:
    case ABORT: /* can't log in at any moment! */
        break;
    default: /* problem with expect */
    }
```

Sternchen, wie im Beispiel oben, sind eine nützliche Kurzform, um Zeichen am Zeilenende und anderes Detail wegzulassen. Muster müssen die ganze Ausgabe des aktuellen Prozesses treffen, seit dem vorigen read auf dem Deskriptor oder Strom. Mehr als 2000 Bytes Ausgabe können frühere Bytes zum "Vergessen" zwingen. Das lässt sich durch Setzen der Variable **exp_match_max** ändern. Übermäßig große Werte können den Mustervergleich verlangsamen.

LAUF IM HINTERGRUND

```
extern int exp_disconnected;
int exp_disconnect();
```

Ein Prozess lässt sich in den Hintergrund legen, nachdem er zu laufen begonnen hat. Ein typischer Gebrauch ist, Passwörter zu lesen und dann im Hintergrund zu schlafen, bevor die Passwörter für die eigentliche Arbeit verwendet werden.

Um einen Prozess in den Hintergrund zu legen, wird fork aufgerufen, im Kindprozess exp_disconnect() und im Elternprozess exit(). Das löst den Prozess vom steuernden Terminal. Soll ein Prozess auf andere Weise in den Hintergrund, muss die Variable exp_disconnected auf 1 gesetzt werden. Prozesse, die danach gestartet werden, starten dann richtig.

MEHRFACHZUGRIFF

In der Vorgabe blockieren die expect-Funktionen in einem read auf einem einzigen Dateideskriptor. Soll auf Muster von mehreren Dateideskriptoren gewartet werden, dienen select, poll oder eine Ereignisverwaltung. Daran ist abzulesen, welcher Dateideskriptor zum Lesen bereit ist.

Ist ein Dateideskriptor zum Lesen bereit, können die expect-Funktionen genau ein read ausführen, indem timeout auf 0 gesetzt wird.

STEUERUNG DES SLAVE

```
void
exp_slave_control(fd,enable)
int fd;
int enable;
```

Das Einfangen der Pty geschieht gewöhnlich automatisch durch die expect-Funktionen. Soll zum Beispiel ein ioctl unmittelbar auf dem Slave-Gerät abgesetzt werden, wird das Einfangen vorübergehend ausgeschaltet.

Das Einfangen der Pty lässt sich mit exp_slave_control steuern. Das erste Argument ist der Dateideskriptor des gestarteten Prozesses. Das zweite Argument ist 0, wenn das Einfangen ausgeschaltet werden soll, und 1, wenn es eingeschaltet werden soll.

FEHLER

Alle Funktionen zeigen Fehler, indem sie -1 zurückgeben und errno setzen.

Fehler, die auftreten, nachdem die spawn-Funktionen abgespalten haben, etwa der Versuch, ein nicht vorhandenes Programm zu starten, werden auf stderr des Prozesses geschrieben und vom ersten **expect** gelesen.

SIGNALE

```
extern int exp_reading;
extern jmp_buf exp_readenv;
```

expect verwendet alarm() für die Zeitüberschreitung. Werden während **expect** Alarme erzeugt, läuft die Zeit vorzeitig ab.

Intern ruft **expect** read() auf, das von Signalen unterbrochen werden kann. Werden Signalbehandler definiert, lässt sich das interne read von **expect** neu starten oder abbrechen. Die Variable **exp_reading** ist wahr genau dann, wenn das read von **expect** unterbrochen wurde. longjmp(exp_readenv,EXP_ABORT) bricht das read ab. longjmp(exp_readenv,EXP_RESTART) startet das read neu.

PROTOKOLL

```
extern int exp_loguser;
extern int exp_logfile_all
extern FILE *exp_logfile;
```

Ist **exp_loguser** ungleich null, sendet **expect** jede Ausgabe des gestarteten Prozesses nach stdout. Interaktive Programme senden ihre Eingabe gewöhnlich zurück, deshalb genügt das meist, um beide Seiten des Gesprächs zu zeigen. Ist **exp_logfile** ebenfalls ungleich null, wird dieselbe Ausgabe in den Strom geschrieben, den **exp_logfile** bezeichnet. Ist **exp_logfile_all** ungleich null, wird **exp_logfile** geschrieben, unabhängig vom Wert von **exp_loguser**.

FEHLERSUCHE

Die Bibliothek ist leicht zu verwenden. Das eigenständige Programm expect ist aber sehr viel leichter zu verwenden als die Arbeit mit dem C-Compiler und seinem üblichen Kreis aus Bearbeiten, Übersetzen und Fehlersuchen. Anders als bei typischen C-Programmen besteht die Fehlersuche meist nicht darin, den C-Compiler zur Annahme der Programme zu bringen, sondern den Dialog richtig zu bekommen. Skripte von expect nach C zu übertragen, ist gewöhnlich nicht

nötig. Die Geschwindigkeit interaktiver Dialoge ist zum Beispiel so gut wie nie das Problem. Zuerst sollte das eigenständige Programm 'expect' versucht werden. Für die meisten ist es eine passendere Lösung als die Bibliothek.

Soll trotzdem in C nach Fehlern gesucht werden, helfen die folgenden Werkzeuge.

```
extern int exp_is_debugging;
extern FILE *exp_debugfile;
```

Expect-Dialoge wirken sehr naheliegend. Dialoge in einem Programm festzuhalten, kann viele Überraschungen in der Schnittstelle eines Programms zeigen. Deshalb gibt es verschiedene Hilfen zur Fehlersuche. Die oben genannten Variablen steuern sie, in der Vorgabe alle 0.

Interne Angaben zur Fehlersuche von **expect** gehen nach stderr, wenn **exp_is_debugging** ungleich null ist. Die Angaben umfassen jedes empfangene Zeichen und jeden Versuch, die aktuelle Eingabe mit den Mustern zu vergleichen. Nicht druckbare Zeichen werden zusätzlich in eine druckbare Form übersetzt. Ein Control-C erscheint zum Beispiel als Zirkumflex, gefolgt von einem C. Ist **exp_logfile** ungleich null, werden diese Angaben auch in diesen Strom geschrieben.

Ist **exp_debugfile** ungleich null, werden alle gewöhnlichen Angaben und alle Angaben zur Fehlersuche in diesen Strom geschrieben, unabhängig vom Wert von **exp_is_debugging**.

EINSCHRÄNKUNGEN

Die Stromfassungen der Funktionen **expect** sind viel langsamer als die Fassungen mit Dateideskriptor, weil sich eine unbekannte Zahl von Bytes nicht übertragbar lesen lässt, ohne dass eine Zeitüberschreitung droht. Die Zeichen werden deshalb einzeln gelesen. Die Fassungen von **expect** mit Dateideskriptor sind deshalb vorzuziehen, auch wenn automatisierte interaktive Programme gewöhnlich keine hohe Geschwindigkeit verlangen.

Beides zusammen geht: Schreiben mit den üblichen Stromfunktionen und Lesen mit den Dateideskriptor-Fassungen von **expect**, solange keine anderen Eingabefunktionen für Ströme dazwischen gemischt werden, etwa fgetc. Dazu wird jedes Mal fileno(stream) als Dateideskriptor übergeben. Glücklicherweise gibt es wenig Grund, beim Lesen von interaktiven Programmen etwas anderes als die Funktionen **expect** zu verwenden.

Zu exp_popen gibt es kein passendes exp_pclose, anders als bei popen und pclose. Eine Verbindung zu schließen, braucht nur zwei Funktionen, fclose() und danach das Warten auf die pid. Diese beiden Schritte liegen aber nicht selten weit auseinander, sodass die Funktion wenig Wert zu haben scheint.

Läuft das auf einem Cray unter Unicos, das ist das, was aus Erfahrung sicher ist, muss das kompilierte Programm als root oder setuid laufen. Das Problem ist, dass der Cray nur root-Prozessen erlaubt, Ptys zu öffnen. Die Vorsicht sollte so groß wie möglich sein. Werden die Rechte nicht gebraucht, steht setuid(0) nur unmittelbar vor dem Aufruf einer spawn-Funktion, und unmittelbar danach wird es zurückgesetzt.

Gewöhnlich braucht **spawn** wenig Zeit. Dauert spawn auffällig lange, trifft es vermutlich auf festgefahrene Ptys. Auf Ptys laufen mehrere Prüfungen, damit es keine Verwicklung mit fehlgeleiteten Prozessen gibt. Das dauert 10 Sekunden je festgefahrener Pty. Läuft expect mit der Option -d, zeigt sich, ob **expect** viele Ptys in merkwürdigen Zuständen trifft. Lassen sich die Prozesse nicht beenden, an denen diese Ptys hängen, bleibt als einziger Ausweg ein Neustart.

BUGS
Die Funktionen **exp_fexpect** arbeiten unter HP-UX gar nicht. Das scheint ein Fehler in getc zu sein. Es gilt der Rat weiter oben, die Funktionen **exp_expect** zu verwenden. Die brauchen getc nicht. Wird der Fehler behoben, und die letzte Ausgabe ist geprüft, soll das dem Autor mitgeteilt werden.

AUTOREN
Don Libes, libes@nist.gov, Nationales Institut für Standards und Technologie

DANKSAGUNGEN
Dank an John Ousterhout (UCBerkeley) für den Mustervergleicher.

Entwurf und Umsetzung der Bibliothek **expect** wurden von der Regierung der USA bezahlt und sind deshalb gemeinfrei. Der Autor und das NIST wünschen eine Nennung, wenn dieses Programm und diese Dokumentation oder Teile davon verwendet werden.

itcl

body
Stichwörter: body, class, object, procedure

NAME
itcl::body - den Rumpf einer Klassenmethode oder eines Klassen-proc ändern

ÜBERSICHT
itcl::body *className::function args body*

BESCHREIBUNG
Der Befehl **body** wird außerhalb einer Klassendefinition von **[incr\ Tcl]** verwendet, um den Rumpf einer Klassenmethode oder eines proc zu definieren oder neu zu definieren. Diese Einrichtung erlaubt einer Klassendefinition getrennte Teile für Schnittstelle und Umsetzung. Der Teil der Schnittstelle ist ein Befehl **class** mit Deklarationen für methods, procs, Instanzvariablen und common-Variablen. Der Teil der Umsetzung ist eine Folge der Befehle **body** und **configbody**. Wenn der Teil der Umsetzung in einer eigenen Datei liegt, lässt sie sich immer wieder einlesen, während Fehler behoben werden, um die interaktive Entwicklung zu unterstützen. Im Modus "tcl" des Editors **emacs** können Schnittstelle und Umsetzung in derselben Datei liegen; während Fehler behoben werden, lassen sich einzelne Rümpfe markieren und an die Testanwendung schicken.

Der Name "*className*::*function*" bezeichnet die Methode oder das proc, das geändert wird.

Wenn bei der Definition der *function* in der Klassendefinition eine Liste *args* angegeben wurde, muss die Liste *args* des Befehls **body** in der Bedeutung übereinstimmen. Variablennamen dürfen sich ändern, die Argumentlisten müssen aber dieselben Pflichtargumente und dieselben Vorgabewerte für optionale Argumente haben. Das besondere Argument **args** wirkt als Platzhalter, wenn es in der Liste *args* der Klassendefinition steht; es passt auf null oder mehr Argumente beliebigen Typs, wenn der Rumpf neu definiert wird.

Wenn die Zeichenkette *body* mit "@" beginnt, wird sie als symbolischer Name einer C-Prozedur

behandelt. Die Liste *args* hat für die C-Prozedur wenig Bedeutung, außer die erwartete Verwendung zu dokumentieren. (Es ist nicht garantiert, dass die C-Prozedur die Argumente auf diese Weise verwendet.) Wenn *body* nicht mit "**@**" beginnt, wird es als Tcl-Befehlsskript behandelt. Wenn die Funktion aufgerufen wird, werden die Argumente der Befehlszeile mit der Liste *args* abgeglichen, und für jedes Argument werden lokale Variablen erzeugt. Das ist das übliche Verhalten einer Prozedur im Tcl-Stil.

Symbolische Namen für C-Prozeduren werden festgelegt, indem Prozeduren über **Itcl_RegisterObjC()** registriert werden. Das geschieht gewöhnlich in der Prozedur **Tcl_AppInit()**, die automatisch aufgerufen wird, wenn der Interpreter startet. Im folgenden Beispiel wird die Prozedur \ffCMy_FooObjCmd() unter dem symbolischen Namen "foo" registriert. Im Befehl **body** lässt sich diese Prozedur als "\ffC@foo" ansprechen.

```
int
Tcl_AppInit(interp)
    Tcl_Interp *interp;     /* Interpreter für die Anwendung. */
{
    if (Itcl_Init(interp) == TCL_ERROR) {
        return TCL_ERROR;
    }

    if (Itcl_RegisterObjC(interp, "foo", My_FooObjCmd) != TCL_OK) {
        return TCL_ERROR;
    }
}
```

BEISPIEL

Im folgenden Beispiel wird eine Klasse "File" definiert, die offene Dateien darstellt. Die Methodenrümpfe stehen unter der Klassendefinition, über den Befehl **body**. Die Rümpfe von Konstruktor und Destruktor müssen in der Klassendefinition stehen, sie lassen sich aber ebenfalls über den Befehl **body** neu definieren.

```
itcl::class File {
    private variable fid ""
    constructor {name access} {
        set fid [open $name $access]
    }
    destructor {
        close $fid
    }

    method get {}
    method put {line}
    method eof {}
}

itcl::body File::get {} {
```

```
    return [gets $fid]
}
itcl::body File::put {line} {
    puts $fid $line
}
itcl::body File::eof {} {
    return [::eof $fid]
}

#
# Die Klasse File in Aktion:
#
File x /etc/passwd "r"
while {![x eof]} {
    puts "=> [x get]"
}
itcl::delete object x
```

Copyright (c) 1993-1998 Lucent Technologies, Inc.

class

Stichwörter: class, object, object-oriented

NAME

itcl::class - eine Klasse von Objekten erzeugen

ÜBERSICHT

itcl::class *className* {
inherit *baseClass* ?*baseClass* ...?
constructor *args* ?*init*? *body*
destructor *body*
method *name* ?*args*? ?*body*?
proc *name* ?*args*? ?*body*?
variable *varName* ?*init*? ?*config*?
common *varName* ?*init*?

public *command* ?*arg arg* ...?
protected *command* ?*arg arg* ...?
private *command* ?*arg arg* ...?

set *varName* ?*value*?
array *option* ?*arg arg* ...?
}

className objName ?*arg arg ...*?

objName method ?*arg arg ...*?

className::proc ?*arg arg ...*?

BESCHREIBUNG

Das grundlegende Konstrukt in **[incr\ Tcl]** ist die Klassendefinition. Jede Klasse dient als Vorlage für tatsächliche Objekte, die erzeugt werden können. Die Klasse selbst ist ein Namespace, der enthält, was allen Objekten gemeinsam ist. Jedes Objekt hat sein eigenes Datenbündel, das Instanzen der in der Klassendefinition festgelegten "variables" enthält. Jedes Objekt hat außerdem eine eingebaute Variable namens "this", die den Namen des Objekts enthält. Klassen können auch "common"-Datenelemente haben, die allen Objekten einer Klasse gemeinsam sind.

In die Klassendefinition können zwei Arten von Funktionen aufgenommen werden. "Methods" sind Funktionen, die auf ein bestimmtes Objekt wirken und deshalb Zugriff auf "variables" und auf "common"-Datenelemente haben. "Procs" sind gewöhnliche Prozeduren im Namespace der Klasse und haben nur Zugriff auf "common"-Datenelemente.

Wenn der Rumpf einer Methode oder eines proc mit "@" beginnt, wird er als symbolischer Name einer C-Prozedur behandelt. Sonst wird er als Tcl-Codeskript behandelt. Unten steht, wie C-Prozeduren registriert und verwendet werden.

Eine Klasse kann nur einmal definiert werden, die Rümpfe von Klassenmethoden und procs lassen sich zur interaktiven Fehlersuche aber immer wieder definieren. Einzelheiten stehen bei den Befehlen **body** und **configbody**.

Jeder Namespace kann seine eigene Sammlung von Objekten und Klassen haben. Die Liste der im aktuellen Kontext verfügbaren Klassen lässt sich mit dem Befehl "**itcl::find classes**" abfragen, die Liste der Objekte mit dem Befehl "**itcl::find objects**".

Eine Klasse lässt sich mit dem Befehl "**delete class**" löschen. Einzelne Objekte lassen sich mit dem Befehl "**delete object**" löschen.

KLASSENDEFINITIONEN

class *className definition*

Liefert die Definition für eine Klasse namens *className*. Wenn die Klasse *className* bereits existiert oder im aktuellen Namespace-Kontext ein Befehl namens *className* existiert, gibt dieser Befehl einen Fehler zurück. Wenn die Klassendefinition erfolgreich zerlegt wird, wird *className* ein Befehl im aktuellen Kontext und übernimmt das Erzeugen von Objekten dieser Klasse. Die Klassen*definition* wird als Folge von Tcl-Anweisungen ausgewertet, die Elemente innerhalb der Klasse definieren. Die folgenden Befehle zur Klassendefinition werden erkannt:

inherit *baseClass* ?*baseClass*...?

Bewirkt, dass die aktuelle Klasse Eigenschaften von einer oder mehreren Basisklassen erbt. Die Klassen müssen durch einen vorherigen Befehl **class** definiert worden sein oder der Einrichtung zum automatischen Laden zur Verfügung stehen (siehe unten **AUTOMATISCHES LADEN**). Eine einzelne Klassendefinition darf höchstens einen Befehl **inherit** enthalten.

Die Reihenfolge der Namen *baseClass* in der Liste **inherit** beeinflusst die Namensauflösung für Klassenmitglieder. Wenn derselbe Mitgliedsname in zwei oder mehr Basisklassen vorkommt, hat die Basisklasse Vorrang, die in der Liste **inherit** zuerst steht. Wenn die Klassen "Foo" und "Bar" zum Beispiel beide das Mitglied "x" enthalten und eine andere Klasse die Anweisung "**inherit**" hat:

```
inherit Foo Bar
```

dann bedeutet der Name "x" "Foo::x". Andere geerbte Mitglieder namens "x" müssen mit ihrem

ausdrücklichen Namen angesprochen werden, wie "Bar::x".

constructor *args* ?*init*? *body*
Deklariert die Argumentliste *args* und den *body*, die für den Konstruktor verwendet werden. Er wird automatisch aufgerufen, sobald ein Objekt erzeugt wird.

Bevor der *body* ausgeführt wird, dient die optionale Anweisung *init* dazu, Basisklassen-Konstruktoren aufzurufen, die Argumente verlangen. Variablen in der Angabe *args* sind im Codefragment *init* zugänglich und können an Basisklassen-Konstruktoren übergeben werden. Nach der Auswertung der Anweisung *init* werden alle noch nicht ausgeführten Basisklassen-Konstruktoren automatisch ohne Argumente aufgerufen. So sind alle Basisklassen vollständig konstruiert, bevor der *body* des Konstruktors ausgeführt wird. Voreingestellt bewirkt dieses Schema, dass Konstruktoren der Reihe nach vom allgemeinsten zum spezifischsten aufgerufen werden. Das ist genau die umgekehrte Reihenfolge, in der der Befehl **info heritage** die Klassen meldet.

Wenn die Konstruktion erfolgreich ist, gibt der Konstruktor immer den Objektnamen zurück, unabhängig davon, wie der *body* definiert ist, und der Objektname wird ein Befehl im aktuellen Namespace-Kontext. Wenn die Konstruktion scheitert, wird eine Fehlermeldung zurückgegeben.

destructor *body*
Deklariert den *body* für den Destruktor, der automatisch aufgerufen wird, wenn ein Objekt gelöscht wird. Wenn der Destruktor erfolgreich ist, werden die Objektdaten zerstört und der Objektname als Befehl aus dem Interpreter entfernt. Wenn die Zerstörung scheitert, wird eine Fehlermeldung zurückgegeben und das Objekt bleibt bestehen.

Wenn ein Objekt zerstört wird, werden alle Destruktoren seiner Klassenhierarchie der Reihe nach vom spezifischsten zum allgemeinsten aufgerufen. Das ist die Reihenfolge, in der der Befehl "**info heritage**" die Klassen meldet, und genau die Umkehrung der voreingestellten Konstruktorreihenfolge.

method *name* ?*args*? ?*body*?
Deklariert eine Methode namens *name*. Wenn der *body* der Methode ausgeführt wird, hat er automatischen Zugriff auf objektspezifische Variablen und gemeinsame Datenelemente.

Wenn die Liste *args* angegeben ist, legt sie die Verwendungsangabe für diese Methode fest. Mit dem Befehl **body** lässt sich der Methodenrumpf neu definieren, die Liste *args* muss aber zu dieser Angabe passen.

Innerhalb des Rumpfes einer anderen Klassenmethode lässt sich eine Methode wie jeder andere Befehl aufrufen, einfach über ihren Namen. Außerhalb des Klassenkontexts muss dem Methodennamen ein Objektname vorangestellt werden, der den Kontext für die Daten liefert, die sie bearbeitet. Methoden einer Basisklasse, die in der aktuellen Klasse neu definiert oder von einer anderen Basisklasse verdeckt sind, lassen sich mit der Syntax "*className*::*method*" qualifizieren.

proc *name* ?*args*? ?*body*?
Deklariert ein proc namens *name*. Ein proc ist eine gewöhnliche Prozedur im Namespace der Klasse. Anders als eine Methode wird ein proc aufgerufen, ohne sich auf ein bestimmtes Objekt zu beziehen. Wenn der *body* des proc ausgeführt wird, hat er automatischen Zugriff nur auf gemeinsame Datenelemente.

Wenn die Liste *args* angegeben ist, legt sie die Verwendungsangabe für dieses proc fest. Mit

dem Befehl **body** lässt sich der Rumpf des proc neu definieren, die Liste *args* muss aber zu dieser Angabe passen.

Innerhalb des Rumpfes einer anderen Klassenmethode oder eines proc lässt sich ein proc wie jeder andere Befehl aufrufen, einfach über seinen Namen. In jedem anderen Namespace-Kontext wird das proc mit einem qualifizierten Namen wie "*className*::*proc*" aufgerufen. Procs einer Basisklasse, die in der aktuellen Klasse neu definiert oder von einer anderen Basisklasse verdeckt sind, sind ebenfalls über ihren qualifizierten Namen zugänglich.

variable *varName* ?*init*? ?*config*?
: Definiert eine objektspezifische Variable namens *varName*. Alle objektspezifischen Variablen stehen in Klassenmethoden automatisch zur Verfügung. Sie müssen nicht mit etwas wie dem Befehl **global** deklariert werden.

Wenn die optionale Zeichenkette *init* angegeben ist, dient sie als Anfangswert der Variablen, wenn ein neues Objekt erzeugt wird. Die Initialisierung erzwingt, dass die Variable ein einfacher skalarer Wert ist; nicht initialisierte Variablen lassen sich dagegen im Konstruktor setzen und als Arrays verwenden.

Das optionale Skript *config* ist nur für öffentliche Variablen erlaubt. Wenn es angegeben ist, wird dieses Codefragment jedes Mal ausgeführt, wenn eine öffentliche Variable durch die eingebaute Methode "configure" geändert wird. Das Skript *config* lässt sich auch außerhalb der Klassendefinition mit dem Befehl **configbody** angeben.

common *varName* ?*init*?
: Deklariert eine common-Variable namens *varName*. Common-Variablen liegen im Namespace der Klasse und werden von allen Objekten geteilt, die zur Klasse gehören. Sie sind wie globale Variablen, nur dass sie nicht mit dem gewöhnlichen Befehl **global** deklariert werden müssen. Sie sind in allen Klassenmethoden und procs automatisch sichtbar.

Wenn die optionale Zeichenkette *init* angegeben ist, dient sie als Anfangswert der Variablen. Die Initialisierung erzwingt, dass die Variable ein einfacher skalarer Wert ist; nicht initialisierte Variablen lassen sich dagegen mit nachfolgenden Befehlen **set** und **array** setzen und als Arrays verwenden.

Sobald ein gemeinsames Datenelement definiert ist, lässt es sich mit den Befehlen **set** und **array** innerhalb der Klassendefinition setzen. Dadurch lassen sich gemeinsame Datenelemente als Arrays initialisieren. Zum Beispiel:

```
itcl::class Foo {
    common boolean
    set boolean(true) 1
    set boolean(false) 0
}
```

Wenn gemeinsame Datenelemente im Konstruktor initialisiert werden, werden sie jedes Mal erneut initialisiert, wenn neue Objekte erzeugt werden.

public *command* ?*arg arg ...*?
: ...siehe nächstes...

protected *command* ?*arg arg ...*?
: ...siehe nächstes...

private *command* ?*arg arg ...*?
Diese Befehle setzen die Schutzstufe für Klassenmitglieder, die erzeugt werden, wenn *command* ausgewertet wird. Das *command* ist gewöhnlich **method**, **proc**, **variable** oder **common**, und die übrigen *arg* vervollständigen die Mitgliedsdefinition. *command* kann aber auch ein Skript mit vielen verschiedenen Mitgliedsdefinitionen sein, und die Schutzstufe gilt dann für alle erzeugten Mitglieder.

VERWENDUNG DER KLASSE

Sobald eine Klasse definiert ist, kann der Klassenname als Befehl verwendet werden, um neue Objekte zu erzeugen, die zur Klasse gehören.

***className objName* ?*args...*?**
Erzeugt ein neues Objekt der Klasse *className* mit dem Namen *objName*. Die übrigen Argumente werden an den Konstruktor der spezifischsten Klasse übergeben. Der gibt seinerseits Argumente an die Konstruktoren der Basisklassen weiter, bevor er seinen eigenen Befehlsumpf aufruft. Wenn die Konstruktion erfolgreich ist, wird im aktuellen Namespace-Kontext ein Befehl namens *objName* erzeugt, und *objName* wird als Ergebnis dieser Operation zurückgegeben. Wenn während der Konstruktion ein Fehler auftritt, werden die Destruktoren automatisch aufgerufen, um belegte Ressourcen freizugeben, das Objekt wird gelöscht, und ein Fehler wird zurückgegeben.

Wenn *objName* die Zeichenkette "**#auto**" enthält, wird diese Zeichenkette durch einen automatisch erzeugten Namen ersetzt. Namen haben die Form *className*<*number*>, wobei der Teil *className* so geändert wird, dass er mit einem Kleinbuchstaben beginnt. In der Klasse "Toaster" würde die Angabe "**#auto**" zum Beispiel Namen wie toaster0, toaster1 und so weiter erzeugen. "**#auto**" kann auch mitten in einem Objektnamen stehen:

```
fileselectiondialog .foo.bar.#auto -background red
```

Das würde ein Objekt namens ".foo.bar.fileselectiondialog0" erzeugen.

VERWENDUNG DES OBJEKTS

Sobald ein Objekt erzeugt ist, kann der Objektname als Befehl verwendet werden, um Methoden aufzurufen, die auf das Objekt wirken.

***objName method* ?*args...*?**
Ruft eine Methode namens *method* auf einem Objekt namens *objName* auf. Die übrigen Argumente werden an die Argumentliste der Methode übergeben. Der Methodenname kann "constructor", "destructor", jeder Methodenname aus der Klassendefinition oder eine der folgenden eingebauten Methoden sein.

EINGEBAUTE METHODEN

***objName* cget option**
Erlaubt den Zugriff auf öffentliche Variablen als Konfigurationsoptionen. Das ahmt das Verhalten der gewöhnlichen Operation "cget" für Tk-Widgets nach. Das Argument *option* ist eine Zeichenkette der Form "**-***varName*", und diese Methode gibt den aktuellen Wert der öffentlichen Variablen *varName* zurück.

***objName* configure ?*option*? ?*value option value ...*?**
Erlaubt den Zugriff auf öffentliche Variablen als Konfigurationsoptionen. Das ahmt das

Verhalten der gewöhnlichen Operation "configure" für Tk-Widgets nach. Ohne Argumente gibt diese Methode eine Liste von Listen zurück, die alle öffentlichen Variablen beschreibt. Jede Liste hat drei Elemente: den Variablennamen, seinen Anfangswert und seinen aktuellen Wert.

Wenn eine einzelne *option* der Form "**-***varName*" angegeben ist, gibt diese Methode die Angaben für diese eine Variable zurück.

Sonst werden die Argumente als Paare *option*/*value* behandelt, die öffentlichen Variablen neue Werte zuweisen. Jede Variable erhält ihren neuen Wert, und wenn ihr "config"-Code zugeordnet ist, wird er im Kontext der Klasse ausgeführt, in der sie definiert wurde. Wenn der "config"-Code einen Fehler erzeugt, wird die Variable auf ihren vorherigen Wert zurückgesetzt, und die Methode **configure** gibt einen Fehler zurück.

objName* isa *className
Gibt einen Wert ungleich null zurück, wenn der gegebene *className* in der Abstammung des Objekts zu finden ist, und sonst null.

***objName* info *option* ?*args...*?**
Gibt Angaben zu einem bestimmten Objekt namens *objName* oder zu seiner Klassendefinition zurück. Der Parameter *option* umfasst die folgenden Dinge sowie die Optionen, die der gewöhnliche Tcl-Befehl "info" erkennt:

***objName* info class**
Gibt den Namen der spezifischsten Klasse des Objekts *objName* zurück.

***objName* info inherit**
Gibt die Liste der Basisklassen zurück, wie sie im Befehl "**inherit**" definiert wurden, oder eine leere Zeichenkette, wenn diese Klasse keine Basisklassen hat.

***objName* info heritage**
Gibt den aktuellen Klassennamen und die gesamte Liste der Basisklassen in der Reihenfolge zurück, in der sie bei der Mitgliedersuche und bei der Objektzerstörung durchlaufen werden.

***objName* info function ?*cmdName*? ?-protection? ?-type? ?-name? ?-args? ?-body?**
Ohne Argumente gibt dieser Befehl eine Liste aller Klassenmethoden und procs zurück. Wenn *cmdName* angegeben ist, gibt er Angaben zu einer bestimmten Methode oder einem bestimmten proc zurück. Wenn keine Flags angegeben sind, gibt dieser Befehl eine Liste mit den folgenden Elementen zurück: der Schutzstufe, dem Typ (method/proc), dem qualifizierten Namen, der Argumentliste und dem Rumpf. Mit Flags lassen sich bestimmte Elemente aus dieser Liste anfordern.

***objName* info variable ?*varName*? ?-protection? ?-type? ?-name? ?-init? ?-value? ?-config? ?-scope?**
Ohne Argumente gibt dieser Befehl eine Liste aller objektspezifischen Variablen und gemeinsamen Datenelemente zurück. Wenn *varName* angegeben ist, gibt er Angaben zu einem bestimmten Datenelement zurück. Flags können zusammen mit *varName* in beliebiger Reihenfolge angegeben werden. Das Ergebnis ist eine Liste der genannten Angaben in genau der Reihenfolge der Flags. Wenn keine Flags angegeben sind, gibt dieser Befehl eine Liste zurück, als wären die folgenden Flags angegeben worden:

-protection -type -name -init -value ?**-config**? Das Ergebnis **-config** ist nur vorhanden, wenn *varName* eine öffentliche Variable ist. Es enthält den Code, der bei der Initialisierung von *varName* ausgeführt wird. Das Flag **-scope** gibt den Namespace-Kontext von *varName*. Damit lässt sich die Variable von außerhalb des Objekts wie jede andere Variable ansprechen.

Es ähnelt dem Ergebnis des Befehls **itcl::scope**.

VERKETTEN VON METHODEN UND PROCS

Manchmal hat eine Basisklasse eine Methode oder ein proc, das in einer abgeleiteten Klasse unter demselben Namen neu definiert wird. So kann die abgeleitete Klasse dieselben Operationen wie die Basisklasse behandeln, aber mit ihrem eigenen spezialisierten Verhalten. Angenommen, eine Klasse Toaster sieht so aus:

```
itcl::class Toaster {
    variable crumbs 0
    method toast {nslices} {
        if {$crumbs > 50} {
            error "== FIRE! FIRE! =="
        }
        set crumbs [expr {$crumbs+4*$nslices}]
    }
    method clean {} {
        set crumbs 0
    }
}
```

Eine weitere Klasse wie SmartToaster könnte die Methode "toast" neu definieren. Wenn die Methode der Basisklasse zugänglich sein soll, lässt sie sich mit dem Namen der Basisklasse qualifizieren, um Mehrdeutigkeit zu vermeiden:

```
itcl::class SmartToaster {
    inherit Toaster
    method toast {nslices} {
        if {$crumbs > 40} {
            clean
        }
        return [Toaster::toast $nslices]
    }
}
```

Statt den Namen der Basisklasse fest einzutragen, lässt sich der Befehl "chain" so verwenden:

```
itcl::class SmartToaster {
    inherit Toaster
    method toast {nslices} {
        if {$crumbs > 40} {
            clean
        }
        return [chain $nslices]
    }
}
```

Der Befehl chain sucht in der Klassenhierarchie nach einer etwas allgemeineren Umsetzung (Basisklasse) einer Methode oder eines proc und ruft sie mit den angegebenen Argumenten auf.

Er beginnt beim aktuellen Klassenkontext und durchsucht die Basisklassen in der Reihenfolge, die der Befehl "info heritage" meldet. Wenn keine andere Umsetzung gefunden wird, tut dieser Befehl nichts und gibt die leere Zeichenkette zurück.

AUTOMATISCHES LADEN

Klassendefinitionen müssen nicht ausdrücklich geladen werden; sie können bei Bedarf von der gewöhnlichen Tcl-Einrichtung zum automatischen Laden geladen werden. Jedes Verzeichnis mit Dateien von Klassendefinitionen sollte eine begleitende Datei "tclIndex" haben. Jede Zeile dieser Datei nennt eine Tcl-Prozedur oder eine Klassendefinition von **[incr\ Tcl]** und die Datei, in der die Definition zu finden ist.

Angenommen, ein Verzeichnis enthält die Definitionen der Klassen "Toaster" und "SmartToaster". Dann sähe die Datei "tclIndex" für dieses Verzeichnis so aus:

```
# Tcl-Autoload-Indexdatei, Version 2.0 für [incr Tcl]
# Diese Datei wird vom Befehl "auto_mkindex" erzeugt
# und eingelesen, um Indexangaben für einen oder
# mehrere Befehle einzurichten. Gewöhnlich ist jede Zeile ein Befehl,
# der ein Element im Array auto_index setzt, wobei der
# Elementname der Name eines Befehls ist und der Wert
# ein Skript ist, das den Befehl lädt.

set auto_index(::Toaster) "source $dir/Toaster.itcl"
set auto_index(::SmartToaster) "source $dir/SmartToaster.itcl"
```

Der Befehl **auto_mkindex** erzeugt Dateien "tclIndex" automatisch.

Der automatische Lader muss dieses Verzeichnis kennen, indem der Verzeichnisname an die Variable "auto_path" angehängt wird. Wenn das eingerichtet ist, werden Klassen bei der Verwendung in einer Anwendung bei Bedarf automatisch geladen.

C-PROZEDUREN

C-Prozeduren lassen sich in eine Klassendefinition von **[incr\ Tcl]** einbinden, um Methoden, procs und den "config"-Code öffentlicher Variablen umzusetzen. Jeder Rumpf, der mit "**@**" beginnt, wird als symbolischer Name einer C-Prozedur behandelt.

Symbolische Namen werden festgelegt, indem Prozeduren über **Itcl_RegisterObjC()** registriert werden. Das geschieht gewöhnlich in der Prozedur **Tcl_AppInit()**, die automatisch aufgerufen wird, wenn der Interpreter startet. Im folgenden Beispiel wird die Prozedur \ffCMy_FooObjCmd() unter dem symbolischen Namen "foo" registriert. Im Befehl **body** lässt sich diese Prozedur als "\ffC@foo" ansprechen.

```
int
Tcl_AppInit(interp)
    Tcl_Interp *interp;     /* Interpreter für die Anwendung. */
{
    if (Itcl_Init(interp) == TCL_ERROR) {
        return TCL_ERROR;
    }
```

```
    if (Itcl_RegisterObjC(interp, "foo", My_FooObjCmd) != TCL_OK) {
        return TCL_ERROR;
    }
}
```

C-Prozeduren werden genau wie gewöhnliche Tcl-Befehle umgesetzt. Einzelheiten stehen auf der Handbuchseite von **CrtCommand**. Innerhalb der Prozedur sind Klassendatenelemente wie gewöhnliche Variablen über **Tcl_SetVar()**, **Tcl_GetVar()**, **Tcl_TraceVar()** und so weiter zugänglich. Klassenmethoden und procs lassen sich wie gewöhnliche Befehle über **Tcl_Eval()** ausführen. **[incr\ Tcl]** macht das möglich, indem es den Kontext automatisch einrichtet, bevor die C-Prozedur ausgeführt wird.

Dieses Schema bietet einen natürlichen Weg, den Code weiterzuentwickeln. Klassen lassen sich schnell entwickeln, indem die Rümpfe in Tcl-Code umgesetzt werden. Eine ganze Anwendung lässt sich bauen und prüfen. Wenn nötig, lassen sich einzelne Rümpfe in C-Code umsetzen, um die Leistung zu verbessern.

Copyright (c) 1993-1998 Lucent Technologies, Inc.

code

Stichwörter: code, scope, callback, namespace, public, protected, private

NAME

itcl::code - den Namespace-Kontext eines Codefragments festhalten

ÜBERSICHT

itcl::code ?*-namespace name*? *command* ?*arg arg* ...?

BESCHREIBUNG

Erzeugt einen geltungsbezogenen Wert für den angegebenen *command* und die zugehörigen Argumente *arg*. Ein geltungsbezogener Wert ist eine Liste mit drei Elementen: dem Schlüsselwort "\ffC@scope", einem Namespace-Kontext und einer Wert-Zeichenkette. Der Befehl

```
namespace foo {
    code puts "Hello World!"
}
```

erzeugt zum Beispiel den geltungsbezogenen Wert:

```
@scope ::foo {puts {Hello World!}}
```

Der Befehl **code** hält den aktuellen Namespace-Kontext fest. Wenn die Option **-namespace** angegeben ist, wird der aktuelle Kontext ignoriert, und die Zeichenkette *name* wird als Namespace-Kontext verwendet.

Erweiterungen wie Tk führen gewöhnliche Codefragmente im globalen Namespace aus. Ein geltungsbezogener Wert hält ein Codefragment samt seinem Namespace-Kontext so fest, dass es sich später richtig ausführen lässt. Er wird zum Beispiel gebraucht, um Codefragmente einzupacken, wenn ein Tk-Widget innerhalb eines Namespace verwendet wird:

```
namespace foo {
    private proc report {mesg} {
        puts "click: $mesg"
    }

    button .b1 -text "Push Me" \
        -command [code report "Hello World!"]
    pack .b1
}
```

Das Codefragment der Schaltfläche \ffC.b1 ist nur im Kontext des Namespace "foo" sinnvoll. Außerdem ist die Prozedur "report" private und nur innerhalb dieses Namespace zugänglich. Der Befehl **code** packt das Codefragment so ein, dass es richtig ausgeführt wird, wenn die Schaltfläche gedrückt wird.

Außerdem bewahrt der Befehl **code** die Argumente auf der Befehlszeile unversehrt. Dadurch ist er ein natürlicher Ersatz für den Befehl **list**, mit dem Tcl-Codefragmente oft formatiert werden. Mit anderen Worten, statt den Befehl **list** so zu verwenden:

```
after 1000 [list puts "Hello $name!"]
```

wird der Befehl **code** so verwendet:

```
after 1000 [code puts "Hello $name!"]
```

Das formatiert nicht nur den Befehl richtig, sondern hält auch seinen Namespace-Kontext fest.

Geltungsbezogene Befehle lassen sich wie gewöhnliche Codefragmente aufrufen, mit oder ohne den Befehl **eval**. Die folgenden Anweisungen funktionieren zum Beispiel richtig:

```
set cmd {@scope ::foo .b1}
$cmd configure -background red

set opts {-bg blue -fg white}
eval $cmd configure $opts
```

Geltungsbezogene Befehle umgehen die üblichen Schutzmechanismen; der Befehl

```
@scope ::foo {report {Hello World!}}
```

kann dazu dienen, aus jedem Namespace-Kontext auf das proc "foo::report" zuzugreifen, obwohl es private ist.

Copyright (c) 1993-1998 Lucent Technologies, Inc.

configbody

Stichwörter: configbody, class, object, variable, configure

NAME

itcl::configbody - den "config"-Code einer öffentlichen Variablen ändern

ÜBERSICHT

itcl::configbody *className::varName body*

BESCHREIBUNG

Der Befehl **configbody** wird außerhalb einer Klassendefinition von **[incr\ Tcl]** verwendet, um den Konfigurationscode zu definieren oder neu zu definieren, der einer öffentlichen Variablen zugeordnet ist. Öffentliche Variablen wirken wie Konfigurationsoptionen eines Objekts. Sie lassen sich außerhalb des Klassengeltungsbereichs mit der eingebauten Methode **configure** ändern. Jede Variable kann ein Stück "config"-Code haben, der automatisch ausgeführt wird, wenn die Variable konfiguriert wird. Mit dem Befehl **configbody** lässt sich dieser Coderumpf definieren oder neu definieren.

Wie der Befehl **body** erlaubt diese Einrichtung einer Klassendefinition getrennte Teile für Schnittstelle und Umsetzung. Der Teil der Schnittstelle ist ein Befehl **class** mit Deklarationen für methods, procs, Instanzvariablen und common-Variablen. Der Teil der Umsetzung ist eine Folge der Befehle **body** und **configbody**. Wenn der Teil der Umsetzung in einer eigenen Datei liegt, lässt sie sich immer wieder einlesen, während Fehler behoben werden, um die interaktive Entwicklung zu unterstützen. Im Modus "tcl" des Editors **emacs** können Schnittstelle und Umsetzung in derselben Datei liegen; während Fehler behoben werden, lassen sich einzelne Rümpfe markieren und an die Testanwendung schicken.

Der Name "*className*::*varName*" bezeichnet die öffentliche Variable, die aktualisiert wird. Wenn die Zeichenkette *body* mit "@" beginnt, wird sie als symbolischer Name einer C-Prozedur behandelt. Sonst wird sie als Tcl-Befehlsskript behandelt.

Symbolische Namen für C-Prozeduren werden festgelegt, indem Prozeduren über **Itcl_RegisterObjC()** registriert werden. Das geschieht gewöhnlich in der Prozedur **Tcl_AppInit()**, die automatisch aufgerufen wird, wenn der Interpreter startet. Im folgenden Beispiel wird die Prozedur \ffCMy_FooObjCmd() unter dem symbolischen Namen "foo" registriert. Im Befehl **configbody** lässt sich diese Prozedur als "\ffC@foo" ansprechen.

```
int
Tcl_AppInit(interp)
    Tcl_Interp *interp;     /* Interpreter für die Anwendung. */
{
    if (Itcl_Init(interp) == TCL_ERROR) {
        return TCL_ERROR;
    }

    if (Itcl_RegisterObjC(interp, "foo", My_FooObjCmd) != TCL_OK) {
        return TCL_ERROR;
    }
}
```

BEISPIEL

Im folgenden Beispiel wird eine Klasse "File" definiert, die offene Dateien darstellt. Immer wenn die Option "-name" konfiguriert wird, wird die vorhandene Datei geschlossen und eine neue Datei geöffnet. Der "config"-Code einer öffentlichen Variablen ist optional. Die Option "-access" hat ihn zum Beispiel nicht.

```
itcl::class File {
    private variable fid ""

    public variable name ""
    public variable access "r"

    constructor {args} {
        eval configure $args
    }
    destructor {
        if {$fid != ""} {
            close $fid
        }
    }

    method get {}
    method put {line}
    method eof {}
}

itcl::body File::get {} {
    return [gets $fid]
}
itcl::body File::put {line} {
    puts $fid $line
}
itcl::body File::eof {} {
    return [::eof $fid]
}

itcl::configbody File::name {
    if {$fid != ""} {
        close $fid
    }
    set fid [open $name $access]
}

#
# Die Klasse File in Aktion:
#
File x

x configure -name /etc/passwd
while {![x eof]} {
    puts "=> [x get]"
}
```

```
itcl::delete object x
```

Copyright (c) 1993-1998 Lucent Technologies, Inc.

delete

Stichwörter: delete, namespace, proc, variable, ensemble

NAME

itcl::delete - Dinge im Interpreter löschen

ÜBERSICHT

itcl::delete *option* ?*arg arg* ...?

BESCHREIBUNG

Der Befehl **delete** löscht Dinge im Interpreter. Er ist als Ensemble umgesetzt, sodass Erweiterungen eigene Optionen hinzufügen und das Verhalten dieses Befehls erweitern können. Voreingestellt übernimmt der Befehl **delete** das Zerstören von Namespaces.

Das Argument *option* legt fest, welche Aktion der Befehl ausführt. Die zulässigen *options* (die abgekürzt werden dürfen) sind:

delete class *name* ?*name...*?
: Löscht eine oder mehrere Klassen von **[incr\ Tcl]** namens *name*. Dabei werden alle Objekte der Klasse und ebenso alle abgeleiteten Klassen gelöscht.

Wenn beim Zerstören eines Objekts ein Fehler auftritt, verhindert das die Zerstörung der Klasse und aller übrigen Objekte. Um die gesamte Klasse ohne Rücksicht auf Fehler zu zerstören, dient der Befehl "**delete namespace**".

delete object *name* ?*name...*?
: Löscht ein oder mehrere Objekte von **[incr\ Tcl]** namens *name*. Ein Objekt wird gelöscht, indem alle Destruktoren seiner Klassenhierarchie aufgerufen werden, der Reihe nach vom spezifischsten zum allgemeinsten. Wenn alle Destruktoren erfolgreich sind, werden die dem Objekt zugeordneten Daten gelöscht, und der *name* wird als Befehl aus dem Interpreter entfernt.

Wenn der Zugriffsbefehl eines Objekts in einem anderen Namespace liegt, kann sein qualifizierter Name verwendet werden:

```
itcl::delete object foo::bar::x
```

Wenn beim Zerstören eines Objekts ein Fehler auftritt, wird der Befehl **delete** abgebrochen, und das Objekt bleibt bestehen. Um ein Objekt ohne Rücksicht auf Fehler zu zerstören, dient der Befehl "**rename**" dazu, den Zugriffsbefehl des Objekts zu zerstören.

delete namespace *name* ?*name...*?
: Löscht einen oder mehrere Namespaces namens *name*. Dabei werden alle Befehle und Variablen im Namespace gelöscht und ebenso alle Kind-Namespaces. Wenn ein Namespace gelöscht wird, wird er automatisch aus den Importlisten aller anderen Namespaces entfernt.

Copyright (c) 1993-1998 Lucent Technologies, Inc.

ensemble

Stichwörter: ensemble, part, info

NAME

itcl::ensemble - einen zusammengesetzten Befehl erzeugen oder ändern

ÜBERSICHT

itcl::ensemble *ensName* ?*command arg arg* ...?
ensemble *ensName* {
part *partName args body*
...
ensemble *partName* {
part *subPartName args body*
part *subPartName args body*
...
}
}

oder

BESCHREIBUNG

Der Befehl **ensemble** erzeugt einen zusammengesetzten Befehl oder ändert ihn. Ein kurzer Überblick über Ensembles steht unten im Abschnitt **WAS IST EIN ENSEMBLE?**.

Wenn der Befehl **ensemble** ein vorhandenes Ensemble namens *ensName* findet, aktualisiert er dieses Ensemble. Sonst erzeugt er ein Ensemble namens *ensName*. Wenn der *ensName* ein einfacher Name wie "foo" ist, wird dem aktuellen Namespace-Kontext ein Ensemble-Befehl namens "foo" hinzugefügt. Wenn in diesem Kontext bereits ein Befehl namens "foo" existiert, wird er gelöscht. Wenn der *ensName* Namespace-Qualifizierer wie "a::b::foo" enthält, wird der Namespace-Pfad aufgelöst, und der Ensemble-Befehl wird diesem Namespace-Kontext hinzugefügt. Übergeordnete Namespaces wie "a" und "b" werden bei Bedarf automatisch erzeugt.

Wenn der *ensName* Leerzeichen enthält wie "a::b::foo bar baz", werden weitere Wörter wie "bar" und "baz" als Unter-Ensembles behandelt. Unter-Ensembles sind lediglich Teile innerhalb eines Ensembles; ihnen ist kein Tcl-Befehl zugeordnet. Ein Ensemble wie "foo" kann ein Unter-Ensemble "foo bar" haben, das seinerseits ein Unter-Ensemble "foo bar baz" haben kann. In diesem Fall muss das Unter-Ensemble "foo bar" erzeugt werden, bevor das Unter-Ensemble "foo bar baz" erzeugt wird, das in ihm liegt.

Wenn auf *ensName* Argumente folgen, werden sie als Befehle behandelt und ausgeführt, um das Ensemble zu aktualisieren. In diesem Zusammenhang werden die folgenden Befehle erkannt: **part** und **ensemble**.

Der Befehl **part** definiert einen neuen Teil des Ensembles. Seine Syntax ist mit dem gewöhnlichen Befehl **proc** identisch, er definiert aber einen Teil innerhalb eines Ensembles statt eines Tcl-Befehls. Wenn im Ensemble bereits ein Teil namens *partName* existiert, gibt der Befehl **part** einen Fehler zurück.

Der Befehl **ensemble** kann in einem anderen Befehl **ensemble** verschachtelt werden, um ein Unter-Ensemble zu definieren.

WAS IST EIN ENSEMBLE?

Der gewöhnliche Befehl "info" ist ein zusammengesetzter Befehl: auf den Befehlsnamen **info** muss ein Unterbefehl wie **body** oder **globals** folgen. Ein Befehl wie **info** wird hier *Ensemble* genannt, Unterbefehle wie **body** oder **globals** seine *Teile*.

Ensembles können verschachtelt sein. Der Befehl **info** hat zum Beispiel ein Ensemble **info namespace** in sich. Dieses Ensemble hat Teile wie **info namespace all** und **info namespace children**.

Mit Ensembles lassen sich zusammengesetzte Befehle automatisch erzeugen und erweitern. Jedes Paket kann ein vorhandenes Ensemble finden und ihm neue Teile hinzufügen. Autoren von Erweiterungen können zum Beispiel dem Befehl **info** eigene Teile hinzufügen.

Die Ensemble-Einrichtung verwaltet alle Teilnamen und merkt sich eindeutige Abkürzungen. Gewöhnlich lässt sich **info complete** zu **info comp** abkürzen. Wenn eine Erweiterung aber den Teil **info complexity** hinzufügt, wird die kürzeste Abkürzung für **info complete** zu **info complet**.

Die Ensemble-Einrichtung automatisiert nicht nur den Bau zusammengesetzter Befehle, sondern auch die Fehlerbehandlung. Wird ein Ensemble-Befehl ohne Teilnamen aufgerufen, entsteht eine automatisch erzeugte Fehlermeldung, die die Verwendung zusammenfasst. Wenn der Befehl **info** zum Beispiel ohne Argumente aufgerufen wird, erzeugt er die folgende Fehlermeldung:

```
wrong # args: should be one of...
  info args procname
  info body procname
  info cmdcount
  info commands ?pattern?
  info complete command
  info context
  info default procname arg varname
  info exists varName
  info globals ?pattern?
  info level ?number?
  info library
  info locals ?pattern?
  info namespace option ?arg arg ...?
  info patchlevel
  info procs ?pattern?
  info protection ?-command? ?-variable? name
  info script
  info tclversion
  info vars ?pattern?
  info which ?-command? ?-variable? ?-namespace? name
```

Auch die Reaktion eines Ensembles auf Fehler lässt sich anpassen. Wenn ein Ensemble auf einen nicht angegebenen oder mehrdeutigen Teilnamen trifft, sucht es einen Teil namens **@error**. Wenn der existiert, behandelt er den Fehler. Dieser Teil erhält alle Argumente der Befehlszeile, beginnend mit dem beanstandeten Teilnamen. Er kann den Befehl auf einem anderen Weg auflösen oder eine eigene Fehlermeldung erzeugen.

BEISPIEL

Ein Ensemble könnte die Syntax der verschiedenen "wait"-Befehle in Tcl/Tk aufräumen. Statt einer Reihe eigenartiger Befehle wie dieser:

```
vwait x
tkwait visibility .top
tkwait window .
```

ließen sich Befehle mit einheitlicher Syntax verwenden, so:

```
wait variable x
wait visibility .top
wait window .
```

Das Paket Tcl könnte das folgende Ensemble definieren:

```
itcl::ensemble wait part variable {name} {
    uplevel vwait $name
}
```

Das Paket Tk könnte diesem Ensemble einige Optionen hinzufügen, mit einem Befehl wie diesem:

```
itcl::ensemble wait {
    part visibility {name} {
        tkwait visibility $name
    }
    part window {name} {
        tkwait window $name
    }
}
```

Andere Erweiterungen könnten dem Befehl **wait** ebenfalls eigene Teile hinzufügen.

Copyright (c) 1993-1998 Lucent Technologies, Inc.

find

Stichwörter: find, class, object, search, import

NAME

itcl::find - Klassen und Objekte suchen

ÜBERSICHT

itcl::find *option* ?*arg arg* ...?

BESCHREIBUNG

Der Befehl **find** sucht Klassen und Objekte, die im aktuellen Interpreter verfügbar sind. Klassen und Objekte werden zuerst im aktiven Namespace gemeldet, dann in allen anderen Namespaces des Interpreters.

Das Argument *option* legt fest, welche Aktion der Befehl ausführt. Die zulässigen *options* (die abgekürzt werden dürfen) sind:

find classes ?*pattern*?
Gibt eine Liste der Klassen von [incr Tcl] zurück. Klassen im aktuellen Namespace stehen zuerst, danach die Klassen in allen anderen Namespaces des Interpreters. Wenn das optionale *pattern* angegeben ist, werden die gemeldeten Namen nach den Regeln des Befehls "**string match**" verglichen, und nur passende Namen werden gemeldet.

Wenn eine Klasse im aktuellen Namespace-Kontext liegt, meldet dieser Befehl ihren einfachen Namen, ohne Qualifizierer. Wenn das *pattern* jedoch Qualifizierer **::** enthält oder die Klasse in einem anderen Kontext liegt, meldet dieser Befehl ihren voll qualifizierten Namen. Mit dem folgenden Befehl erhält man deshalb eine Liste, in der alle Namen voll qualifiziert sind:

```
itcl::find classes ::*
```

find objects ?*pattern*? ?-class *className*? ?-isa *className*?
Gibt eine Liste der Objekte von [incr Tcl] zurück. Objekte im aktuellen Namespace stehen zuerst, danach die Objekte in allen anderen Namespaces des Interpreters. Wenn das optionale *pattern* angegeben ist, werden die gemeldeten Namen nach den Regeln des Befehls "**string match**" verglichen, und nur passende Namen werden gemeldet. Wenn der optionale Parameter "**-class**" angegeben ist, beschränkt sich diese Liste auf Objekte, deren spezifischste Klasse *className* ist. Wenn der optionale Parameter "**-isa**" angegeben ist, beschränkt sich die Liste weiter auf Objekte, die das gegebene *className* irgendwo in ihrer Abstammung haben.

Wenn ein Objekt im aktuellen Namespace-Kontext liegt, meldet dieser Befehl seinen einfachen Namen, ohne Qualifizierer. Wenn das *pattern* jedoch Qualifizierer **::** enthält oder das Objekt in einem anderen Kontext liegt, meldet dieser Befehl seinen voll qualifizierten Namen. Mit dem folgenden Befehl erhält man deshalb eine Liste, in der alle Namen voll qualifiziert sind:

```
itcl::find objects ::*
```

Copyright (c) 1993-1998 Lucent Technologies, Inc.

is

Stichwörter: is, class, object

NAME

itcl::is - Argument prüfen, ob es eine Klasse oder ein Objekt ist

ÜBERSICHT

itcl::is *option* ?*arg arg* ...?

BESCHREIBUNG

Der Befehl **is** prüft, ob das gegebene Argument eine Klasse oder ein Objekt ist, je nach der gegebenen Option. Wenn das Argument eine Klasse oder ein Objekt ist, wird 1 zurückgegeben. Sonst wird 0 zurückgegeben. Der Befehl **is** erkennt auch die Befehle, die im itcl-Befehl **code** verpackt sind.

Das Argument *option* legt fest, welche Aktion der Befehl ausführt. Die zulässigen *options* (die abgekürzt werden dürfen) sind:

is class *command*

Gibt 1 zurück, wenn command eine Klasse ist, und gibt sonst 0 zurück.

Der voll qualifizierte Name der Klasse muss als Argument *command* angegeben werden. Wenn eine Klasse in einem Namespace liegt, muss also auch der Namespace angegeben werden. Wenn eine Klasse **C** in einem Namespace **N** liegt, sollte der Befehl so aufgerufen werden:

```
is N::C
    oder
is ::N::C
```

is object ?-class *className*? *command*

Gibt 1 zurück, wenn *command* ein Objekt ist, und gibt sonst 0 zurück.

Wenn der optionale Parameter "**-class**" angegeben ist, wird das *command* im Kontext der gegebenen Klasse geprüft. *className* muss existieren. Sonst wird ein Fehler gemeldet. Wenn unsicher ist, ob *className* eine Klasse ist, muss die Existenz vorher geprüft werden, oder der Aufruf wird in eine catch-Anweisung gepackt.

Wenn also **c** ein Objekt der Klasse **C** im Namespace N ist, sind dies die Möglichkeiten (alle geben 1 zurück):

```
set obj [N::C c]
itcl::is object N::c
itcl::is object c
itcl::is object $obj
itcl::is object [itcl::code c]
```

Copyright (c) 1993-1998 Lucent Technologies, Inc.

itcl

Stichwörter: itcl, class, object, object-oriented, namespace, mega-widget

NAME

itcl - objektorientierte Erweiterungen für Tcl

BESCHREIBUNG

[incr\ Tcl] stellt objektorientierte Erweiterungen für Tcl bereit, so wie C++ objektorientierte Erweiterungen für C bereitstellt. Der Schwerpunkt dieser Arbeit liegt jedoch nicht darin, eine aufsehenerregende Umgebung für objektorientierte Programmierung zu schaffen. Es geht vielmehr darum, in Tcl stärker strukturierte Programmierweisen zu unterstützen, ohne den Charakter der Sprache zu ändern. Vor allem anderen bietet **[incr\ Tcl]** ein Mittel, zusammengehörige Prozeduren samt ihren gemeinsamen Daten in einem Namespace zu kapseln, der vor der Außenwelt verborgen ist. Es fördert besseres Programmieren, indem es die objektorientierte Denkweise einer "Bibliothek" nahelegt. Außerdem erlaubt es die Wiederverwendung von Code durch Vererbung.

KLASSEN

Das grundlegende Konstrukt in **[incr\ Tcl]** ist die Klassendefinition. Jede Klasse dient als Vorlage für tatsächliche Objekte, die erzeugt werden können. Jedes Objekt hat sein eigenes Datenbündel, das Instanzen der in der Klasse definierten "variables" enthält. Spezielle

Prozeduren, "methods" genannt, dienen dazu, einzelne Objekte zu bearbeiten. Methoden gleichen den Operationen, mit denen Tk-Widgets bedient werden. Das Widget "**button**" hat zum Beispiel Methoden wie "flash" und "invoke", die eine bestimmte Schaltfläche blinken lassen und ihren Befehl auslösen.

Innerhalb des Rumpfes einer Methode stehen die in der Klasse definierten "variables" automatisch zur Verfügung. Sie müssen nicht mit etwas wie dem Befehl **global** deklariert werden. Innerhalb einer anderen Klassenmethode lässt sich eine Methode wie jeder andere Befehl aufrufen, einfach über ihren Namen. Aus jedem anderen Zusammenhang muss dem Methodennamen ein Objektname vorangestellt werden, der den Zusammenhang für die Daten liefert, auf die die Methode zugreifen kann.

Jede Klasse hat ihren eigenen Namespace mit dem, was allen Objekten gemeinsam ist, die zur Klasse gehören. "common"-Datenelemente werden zum Beispiel von allen Objekten der Klasse geteilt. Es sind globale Variablen, die im Namespace der Klasse existieren; da sie in der Klassendefinition stehen, müssen sie nicht mit dem Befehl **global** deklariert werden. Sie stehen jedem Code automatisch zur Verfügung, der im Klassenkontext ausgeführt wird. Eine Klasse kann auch gewöhnliche globale Variablen anlegen, diese müssen aber bei jeder Verwendung mit dem Befehl **global** deklariert werden.

Klassen können auch gewöhnliche Prozeduren haben, die als "procs" deklariert sind. Innerhalb einer anderen Klassenmethode oder eines proc lässt sich ein proc wie jeder andere Befehl aufrufen, einfach über seinen Namen. Aus jedem anderen Zusammenhang sollte der Prozedurname mit dem Namespace der Klasse qualifiziert werden, wie "*className***::***proc*". Klassen-procs werden im Klassenkontext ausgeführt und haben deshalb automatischen Zugriff auf alle "common"-Datenelemente. Auf objektspezifische "variables" können sie jedoch nicht zugreifen, weil sie ohne Bezug auf ein bestimmtes Objekt aufgerufen werden. Sie dienen gewöhnlich dazu, allgemeine Operationen auszuführen, die alle zur Klasse gehörenden Objekte betreffen.

Jedes der Elemente einer Klasse kann als "public", "protected" oder "private" deklariert werden. Auf öffentliche Elemente können die Klasse, abgeleitete Klassen (andere Klassen, die diese Klasse erben) und externe Aufrufer zugreifen, die die Klasse verwenden. Auf geschützte Elemente können die Klasse und abgeleitete Klassen zugreifen. Private Elemente sind nur in der Klasse zugänglich, in der sie definiert sind.

Die "public"-Elemente einer Klasse legen ihre Schnittstelle zur Außenwelt fest. Öffentliche Methoden legen die Operationen fest, mit denen sich ein Objekt bedienen lässt. Öffentliche Variablen erkennen die Methoden "configure" und "cget", die in jede Klasse eingebaut sind, als Konfigurationsoptionen. Die öffentliche Schnittstelle sagt, *was* ein Objekt tun wird, aber nicht, *wie* es das tun wird. Geschützte und private Mitglieder liefern zusammen mit den Rümpfen der Klassenmethoden und procs die Einzelheiten der Umsetzung. Wenn der Anwendungsentwickler von diesen Einzelheiten abgeschirmt ist, kann der Entwerfer der Klasse sie jederzeit ohne Vorwarnung ändern, ohne Programme zu beeinträchtigen, die sich auf die Klasse stützen. Genau diese Kapselung macht objektorientierte Programme leichter zu verstehen und zu warten.

Dass Objekte von **[incr\ Tcl]** wie Tk-Widgets aussehen, ist kein Zufall. **[incr\ Tcl]** wurde so entworfen, dass es sich natürlich in eine Tcl/Tk-Anwendung einfügt. **[incr\ Tcl]** erweitert das Paradigma von Tk aber davon, nur objektbasiert zu sein, dahin, vollständig objektorientiert zu sein. Ein objektorientiertes System unterstützt die Vererbung und erlaubt Klassen, gemeinsames Verhalten zu teilen, indem sie es von einer Vorfahrenklasse oder Basisklasse erben. Eine

Basisklasse als gemeinsame Abstraktion erlaubt es, verwandte Klassen auf ähnliche Weise zu behandeln. Ein Toaster und ein Mixer erfüllen zum Beispiel verschiedene (spezialisierte) Funktionen, teilen aber die Abstraktion, Geräte zu sein. Wenn gemeinsames Verhalten in eine Basisklasse gezogen wird, lässt sich Code *teilen* statt *kopieren*. Die entstehende Anwendung ist leichter zu verstehen und zu warten, und abgeleitete Klassen (zum Beispiel spezialisierte Geräte) lassen sich leichter hinzufügen oder entfernen.

Diese Beschreibung war nur ein kurzer Überblick über objektorientierte Programmierung und **[incr\ Tcl]**. Eine stärker Lehrbuch-artige Einführung steht in dem Papier, das dieser Distribution beiliegt. Weitere Einzelheiten zum Erzeugen und Verwenden von Klassen stehen beim Befehl **class**.

NAMESPACES

[incr\ Tcl] enthält inzwischen eine vollständige Einrichtung für Namespaces. Ein Namespace ist eine Sammlung von Befehlen und globalen Variablen, die vom gewöhnlichen globalen Geltungsbereich getrennt gehalten wird. Dadurch lassen sich Bibliotheken mit Tcl-Code auf festgelegte Weise als Paket fassen, und unerwünschte Wechselwirkungen mit anderen Bibliotheken werden verhindert. Ein Namespace kann auch Kind-Namespaces in sich enthalten, sodass eine Bibliothek ihre eigene private Kopie vieler anderer Bibliotheken enthalten kann. Ein Namespace kann auch eine Gruppe verwandter Klassen zusammenfassen. Der globale Geltungsbereich (namens "\ffC::") ist der Wurzel-Namespace eines Interpreters; alle anderen Namespaces sind in ihm enthalten.

Einzelheiten zum Erzeugen und Verwenden von Namespaces stehen beim Befehl **namespace**.

MEGAWIDGETS

Megawidgets sind Widgets auf hoher Ebene, die aus Tk-Widgets als Komponenten gebaut werden, gewöhnlich ohne C-Code. Eine Dateiauswahlbox kann zum Beispiel einige Listboxen, einige Entry-Widgets und einige Steuerschaltflächen enthalten. Diese einzelnen Widgets werden so zusammengesetzt, dass sie wie ein großes Widget wirken.

[incr\ Tk] ist ein Rahmen für den Bau von Megawidgets. Es verwendet **[incr\ Tcl]**, um das Objektparadigma zu unterstützen, und fügt Basisklassen hinzu, die das voreingestellte Widget-Verhalten liefern. Weitere Einzelheiten stehen auf der Handbuchseite von **itk**.

[incr\ Widgets] ist eine Bibliothek von Megawidgets, die mit **[incr\ Tk]** gebaut ist. Sie enthält mehr als 30 verschiedene Widget-Klassen, die sich sofort einsetzen lassen, um Tcl/Tk-Anwendungen zu bauen. Jede Widget-Klasse hat eine eigene Handbuchseite, die die verfügbaren Merkmale beschreibt.

Copyright (c) 1993-1998 Lucent Technologies, Inc.

itclcomponent

Stichwörter: itclcomponent, component, widget, widgetadaptor, extendedclass

NAME

itcl::component - Komponenten für extendedclass, widget oder widgetadaptor definieren

Teile dieser Beschreibung sind von der Tcl-Erweiterung [snit] "entliehen", weil die Funktion weitgehend gleich ist.

WARNUNG
Das ist neue Funktionalität in [incr Tcl], bei der sich die API noch ändern kann!!

ÜBERSICHT
public component *comp* ?**-inherit**?
protected component *comp* ?**-inherit**?
private component *comp* ?**-inherit**?

BESCHREIBUNG
Der Befehl **component** wird innerhalb einer Definition extendedclass/widget/widgetadaptor von **[incr\ Tcl]** verwendet, um Komponenten zu definieren.

Deklariert ausdrücklich eine Komponente namens comp und definiert automatisch die Instanzvariable der Komponente.

Wenn die Option *-inherit* angegeben ist, werden alle unbekannten Methoden und Optionen an diese Komponente weitergeleitet. Der Name -inherit besagt, dass Instanzen dieses neuen Typs in gewissem Sinn die Methoden und Optionen der Komponente erben. Das heißt, -inherit yes entspricht:

```
component mycomp
delegate option * to mycomp
delegate method * to mycomp
```

Copyright (c) 2008 Arnulf Wiedemann

itcldelegate
Stichwörter: itcldelegate, delegation, option, method, proc

NAME
itcl::delegation - Methoden, procs oder Optionen an andere Objekte weiterleiten

Teile dieser Beschreibung sind von der Tcl-Erweiterung [snit] "entliehen", weil die Funktion weitgehend gleich ist.

WARNUNG
Das ist neue Funktionalität in [incr Tcl], bei der sich die API noch ändern kann!!

ÜBERSICHT
delegate method *methodName* **to** *componentName* ?*as targetName*?
delegate method *methodName* ?*to componentName*? **using** *pattern*
delegate method * ?*to componentName*? ?*using pattern*? ?*except methodName methodName* ...?

delegate proc *procName* **to** *componentName* ?*as targetName*?
delegate proc *procName* ?*to componentName*? **using** *pattern*
delegate proc * ?*to componentName*? ?*using pattern*? ?*except procName procName* ...?

delegate option *optionSpec* **to** *componentName*
delegate option *optionSpec* **to** *componentName* **as** *targetname?*
delegate option * **to** *componentName*

delegate option * **to** *componentName* **except** *optionName optionname ...*

BESCHREIBUNG

Der Befehl **delegate** wird innerhalb einer Definition extendedclass/widget/widgetadaptor von **[incr\ Tcl]** verwendet, um Methoden, procs oder Optionen zur Behandlung an andere Objekte weiterzuleiten.

delegate method *methodName* to *componentName* ?as *targetName*?
Diese Form von delegate method leitet die Methode methodName an die Komponente componentName weiter. Wenn also die Methode methodName auf einer Instanz dieses Typs aufgerufen wird, werden die Methode und ihre Argumente stattdessen an den Befehl der genannten Komponente übergeben. Die folgende Anweisung

```
delegate method wag to tail
```

ist ungefähr gleichwertig zu dieser ausdrücklich definierten Methode:

```
method wag {args} {
    uplevel $tail wag $args
}
```

Die optionale Klausel **as** erlaubt es, den Namen der weitergeleiteten Methode anzugeben und möglicherweise einige Argumente hinzuzufügen:

```
delegate method wagtail to tail as "wag briskly"
```

Eine Methode kann nicht zugleich lokal definiert und weitergeleitet sein.

delegate method *methodName* ?to *componentName*? using *pattern*
In dieser Form der Anweisung delegate gibt die Klausel **using** die genaue Form des Befehls an, an den der Methodenname weitergeleitet wird. Die Klausel **to** ist optional, weil der gewählte Befehl keine bestimmte Komponente betreffen muss.

Der Wert der Klausel using ist eine Liste, die einige oder alle der folgenden Ersetzungscodes enthalten darf; diese Codes werden durch den beschriebenen Wert ersetzt, um das Präfix des weitergeleiteten Befehls zu bauen. Die folgenden beiden Anweisungen sind gleichwertig:

```
delegate method wag to tail
delegate method wag to tail using "%c %m"
```

Jedes Element der Liste wird ein einzelnes Element des weitergeleiteten Befehls; es wird nie erneut als Zeichenkette zerlegt.

Ersetzungen:

%%
Das wird durch ein einzelnes "%" ersetzt. Um die Zeichenkette "%c" als Argument an den Befehl zu geben, steht hier "%%c".

%c
Das wird durch den Befehl der genannten Komponente ersetzt.

%j
Das wird durch den Methodennamen ersetzt; besteht der Name aus mehreren Token, werden sie durch Unterstriche verbunden ("").

%m
Das wird durch das letzte Token des Methodennamens ersetzt; hat der Methodenname ein Token, ist das identisch mit **%M**.

%M
Das wird durch den Methodennamen ersetzt; besteht der Name aus mehreren Token, werden sie durch Leerzeichen verbunden.

%n
Das wird durch den Namen des privaten Namespace der Instanz ersetzt.

%s
Das wird durch den Namen des Instanzbefehls ersetzt.

%t
Das wird durch den voll qualifizierten Typnamen ersetzt.

%w
Das wird durch den ursprünglichen Namen des Instanzbefehls ersetzt; bei Itcl-Widgets und Widget-Adaptoren ist das der Tk-Fenstername. Er bleibt gleich, auch wenn der Instanzbefehl umbenannt wird.

delegate method * ?to *componentName*? ?using *pattern*? ?except *methodName methodName ...*?
In dieser Form werden alle unbekannten Methodennamen an die angegebene Komponente weitergeleitet. Die Klausel except kann eine Liste von Ausnahmen angeben, also Methodennamen, die nicht so weitergeleitet werden. Die Klausel using ist wie oben definiert. In dieser Form muss die Anweisung die Klausel to, die Klausel using oder beide enthalten.

Tatsächlich kann das "\" eine Liste aus zwei oder mehr Token sein, deren letztes Element "\" ist, wie im folgenden Beispiel:

```
delegate method {tail *} to tail
```

Das definiert implizit die Methode tail, deren Unterbefehle an die Komponente tail weitergeleitet werden.

Die Definitionen für **delegate proc** ... sind dieselben wie für method, der einzige Unterschied ist, dass es hier um procs geht.

delegate option *namespec* to *comp*
...siehe nächstes...

delegate option namespec to comp as target
...siehe nächstes...

delegate option \ to *comp*
...siehe nächstes...

delegate option \ to *comp* except *exceptions*
Definiert eine weitergeleitete Option; die namespec ist wie bei der Anweisung option definiert. Wenn die Instanzmethode configure, configurelist oder cget den Wert der Option setzt oder abfragt, wird der gleichwertige Befehl configure oder cget auf die Komponente angewendet, als wäre die Option mit der folgenden **-configuremethod** und **-cgetmethod** definiert worden:

```
method ConfigureMethod {option value} {
    $comp configure $option $value
}

method CgetMethod {option} {
    return [$comp cget $option]
}
```

Weitergeleitete Optionen erscheinen nie im Array **itcl_options**. Wenn die Klausel as angegeben ist, wird der Zieloptionsname anstelle von name verwendet.

delegate *option* \ ?except *optionName optionName ...*?
: Diese Form leitet alle unbekannten Optionen an die angegebene Komponente weiter. Die Klausel except kann eine Liste von Ausnahmen angeben, also Optionsnamen, die nicht so weitergeleitet werden.

Warnung: Optionen lassen sich nur dann an eine Komponente weiterleiten, wenn sie die Instanzmethoden **configure** und **cget** unterstützt.

Eine Option kann nicht zugleich lokal definiert und weitergeleitet sein. Offen: hier weitermachen.

Copyright (c) 2008 Arnulf Wiedemann

itclextendedclass

Stichwörter: itclextendedclass, extendedclass, object, object-oriented

NAME

itcl::extendedclass - eine Extendedclass von Objekten erzeugen

WARNUNG

Das ist neue Funktionalität in [incr Tcl], bei der sich die API noch ändern kann!!

ÜBERSICHT

itcl::extendedclass *extendedclassName* {
inherit *baseExtendedclass* ?*baseExtendedclass* ...?
constructor *args* ?*init*? *body*
destructor *body*
public method *name* ?*args*? ?*body*?
protected method *name* ?*args*? ?*body*?
private method *name* ?*args*? ?*body*?
public proc *name* ?*args*? ?*body*?
protected proc *name* ?*args*? ?*body*?
private proc *name* ?*args*? ?*body*?
public variable *varName* ?*init*? ?*config*?
protected variable *varName* ?*init*? ?*config*?
private variable *varName* ?*init*? ?*config*?
public common *varName* ?*init*?
protected common *varName* ?*init*?
private common *varName* ?*init*?

public *command* ?*arg arg* ...?
protected *command* ?*arg arg* ...?
private *command* ?*arg arg* ...?

<delegation info> see delegation page

<option info> see option page

set *varName* ?*value*?
array *option* ?*arg arg* ...?
}

extendedclassName objName ?*arg arg ...*?

objName method ?*arg arg ...*?

extendedclassName::proc ?*arg arg ...*?

BESCHREIBUNG

Das grundlegende Konstrukt in **[incr\ Tcl]** ist die Extendedclass-Definition. Jede Extendedclass dient als Vorlage für tatsächliche Objekte, die erzeugt werden können. Die Extendedclass selbst ist ein Namespace, der enthält, was allen Objekten gemeinsam ist. Jedes Objekt hat sein eigenes Datenbündel, das Instanzen der in der Extendedclass-Definition festgelegten "variables" enthält. Jedes Objekt hat außerdem eine eingebaute Variable namens "this", die den Namen des Objekts enthält. Extendedclasses können auch "common"-Datenelemente haben, die allen Objekten einer Extendedclass gemeinsam sind.

In die Extendedclass-Definition können zwei Arten von Funktionen aufgenommen werden. "Methods" sind Funktionen, die auf ein bestimmtes Objekt wirken und deshalb Zugriff auf "variables" und auf "common"-Datenelemente haben. "Procs" sind gewöhnliche Prozeduren im Namespace der Extendedclass und haben nur Zugriff auf "common"-Datenelemente.

Wenn der Rumpf einer Methode oder eines proc mit "**@**" beginnt, wird er als symbolischer Name einer C-Prozedur behandelt. Sonst wird er als Tcl-Codeskript behandelt. Unten steht, wie C-Prozeduren registriert und verwendet werden.

Eine Extendedclass kann nur einmal definiert werden, die Rümpfe von Extendedclass-Methoden und procs lassen sich zur interaktiven Fehlersuche aber immer wieder definieren. Einzelheiten stehen bei den Befehlen **body** und **configbody**.

Jeder Namespace kann seine eigene Sammlung von Objekten und Extendedclasses haben. Die Liste der im aktuellen Kontext verfügbaren Extendedclasses lässt sich mit dem Befehl "**itcl::find extendedclasses**" abfragen, die Liste der Objekte mit dem Befehl "**itcl::find objects**".

Eine Extendedclass lässt sich mit dem Befehl "**delete extendedclass**" löschen. Einzelne Objekte lassen sich mit dem Befehl "**delete object**" löschen.

EXTENDEDCLASS-DEFINITIONEN

extendedclass *extendedclassName definition*

Liefert die Definition für eine Extendedclass namens *extendedclassName*. Wenn die Extendedclass *extendedclassName* bereits existiert oder im aktuellen Namespace-Kontext ein Befehl namens *extendedclassName* existiert, gibt dieser Befehl einen Fehler zurück. Wenn die Extendedclass-Definition erfolgreich zerlegt wird, wird *extendedclassName* ein Befehl im aktuellen Kontext und übernimmt das Erzeugen von Objekten dieser Extendedclass. Die *definition* der Extendedclass wird als Folge von Tcl-Anweisungen ausgewertet, die Elemente

innerhalb der Extendedclass definieren. Die folgenden Befehle zur Extendedclass-Definition werden erkannt:

inherit *baseExtendedclass ?baseExtendedclass...?*
Bewirkt, dass die aktuelle Extendedclass Eigenschaften von einer oder mehreren Basis-Extendedclasses erbt. Die Extendedclasses müssen durch einen vorherigen Befehl **extendedclass** definiert worden sein oder der Einrichtung zum automatischen Laden zur Verfügung stehen (siehe unten **AUTOMATISCHES LADEN**). Eine einzelne Extendedclass-Definition darf höchstens einen Befehl **inherit** enthalten.

Die Reihenfolge der Namen *baseExtendedclass* in der Liste **inherit** beeinflusst die Namensauflösung für Extendedclass-Mitglieder. Wenn derselbe Mitgliedsname in zwei oder mehr Basis-Extendedclasses vorkommt, hat die Basis-Extendedclass Vorrang, die in der Liste **inherit** zuerst steht. Wenn die Extendedclasses "Foo" und "Bar" zum Beispiel beide das Mitglied "x" enthalten und eine andere Extendedclass die Anweisung "**inherit**" hat:

```
inherit Foo Bar
```

dann bedeutet der Name "x" "Foo::x". Andere geerbte Mitglieder namens "x" müssen mit ihrem ausdrücklichen Namen angesprochen werden, wie "Bar::x".

constructor *args ?init? body*
Deklariert die Argumentliste *args* und den *body*, die für den Konstruktor verwendet werden. Er wird automatisch aufgerufen, sobald ein Objekt erzeugt wird.

Bevor der *body* ausgeführt wird, dient die optionale Anweisung *init* dazu, Basis-Extendedclass-Konstruktoren aufzurufen, die Argumente verlangen. Variablen in der Angabe *args* sind im Codefragment *init* zugänglich und können an Basis-Extendedclass-Konstruktoren übergeben werden. Nach der Auswertung der Anweisung *init* werden alle noch nicht ausgeführten Basis-Extendedclass-Konstruktoren automatisch ohne Argumente aufgerufen. So sind alle Basis-Extendedclasses vollständig konstruiert, bevor der *body* des Konstruktors ausgeführt wird. Voreingestellt bewirkt dieses Schema, dass Konstruktoren der Reihe nach vom allgemeinsten zum spezifischsten aufgerufen werden. Das ist genau die umgekehrte Reihenfolge, in der der Befehl **info heritage** die Extendedclasses meldet.

Wenn die Konstruktion erfolgreich ist, gibt der Konstruktor immer den Objektnamen zurück, unabhängig davon, wie der *body* definiert ist, und der Objektname wird ein Befehl im aktuellen Namespace-Kontext. Wenn die Konstruktion scheitert, wird eine Fehlermeldung zurückgegeben.

destructor *body*
Deklariert den *body* für den Destruktor, der automatisch aufgerufen wird, wenn ein Objekt gelöscht wird. Wenn der Destruktor erfolgreich ist, werden die Objektdaten zerstört und der Objektname als Befehl aus dem Interpreter entfernt. Wenn die Zerstörung scheitert, wird eine Fehlermeldung zurückgegeben und das Objekt bleibt bestehen. Wenn ein Objekt zerstört wird, werden alle Destruktoren seiner Extendedclass-Hierarchie der Reihe nach vom spezifischsten zum allgemeinsten aufgerufen. Das ist die Reihenfolge, in der der Befehl "**info heritage**" die Extendedclasses meldet, und genau die Umkehrung der voreingestellten Konstruktorreihenfolge.

method *name ?args? ?body?*
Deklariert eine Methode namens *name*. Wenn der *body* der Methode ausgeführt wird, hat er

automatischen Zugriff auf objektspezifische Variablen und gemeinsame Datenelemente.

Wenn die Liste *args* angegeben ist, legt sie die Verwendungsangabe für diese Methode fest. Mit dem Befehl **body** lässt sich der Methodenrumpf neu definieren, die Liste *args* muss aber zu dieser Angabe passen.

Innerhalb des Rumpfes einer anderen Extendedclass-Methode lässt sich eine Methode wie jeder andere Befehl aufrufen, einfach über ihren Namen. Außerhalb des Extendedclass-Kontexts muss dem Methodennamen ein Objektname vorangestellt werden, der den Kontext für die Daten liefert, die sie bearbeitet. Methoden einer Basis-Extendedclass, die in der aktuellen Extendedclass neu definiert oder von einer anderen Basis-Extendedclass verdeckt sind, lassen sich mit der Syntax "*extendedclassName*::*method*" qualifizieren.

proc *name* ?*args*? ?*body*?

Deklariert ein proc namens *name*. Ein proc ist eine gewöhnliche Prozedur im Namespace der Extendedclass. Anders als eine Methode wird ein proc aufgerufen, ohne sich auf ein bestimmtes Objekt zu beziehen. Wenn der *body* des proc ausgeführt wird, hat er automatischen Zugriff nur auf gemeinsame Datenelemente.

Wenn die Liste *args* angegeben ist, legt sie die Verwendungsangabe für dieses proc fest. Mit dem Befehl **body** lässt sich der Rumpf des proc neu definieren, die Liste *args* muss aber zu dieser Angabe passen.

Innerhalb des Rumpfes einer anderen Extendedclass-Methode oder eines proc lässt sich ein proc wie jeder andere Befehl aufrufen, einfach über seinen Namen. In jedem anderen Namespace-Kontext wird das proc mit einem qualifizierten Namen wie "*extendedclassName***::***proc*" aufgerufen. Procs einer Basis-Extendedclass, die in der aktuellen Extendedclass neu definiert oder von einer anderen Basis-Extendedclass verdeckt sind, sind ebenfalls über ihren qualifizierten Namen zugänglich.

variable *varName* ?*init*? ?*config*?

Definiert eine objektspezifische Variable namens *varName*. Alle objektspezifischen Variablen stehen in Extendedclass-Methoden automatisch zur Verfügung. Sie müssen nicht mit etwas wie dem Befehl **global** deklariert werden.

Wenn die optionale Zeichenkette *init* angegeben ist, dient sie als Anfangswert der Variablen, wenn ein neues Objekt erzeugt wird. Die Initialisierung erzwingt, dass die Variable ein einfacher skalarer Wert ist; nicht initialisierte Variablen lassen sich dagegen im Konstruktor setzen und als Arrays verwenden.

Das optionale Skript *config* ist nur für öffentliche Variablen erlaubt. Wenn es angegeben ist, wird dieses Codefragment jedes Mal ausgeführt, wenn eine öffentliche Variable durch die eingebaute Methode "configure" geändert wird. Das Skript *config* lässt sich auch außerhalb der Extendedclass-Definition mit dem Befehl **configbody** angeben.

common *varName* ?*init*?

Deklariert eine common-Variable namens *varName*. Common-Variablen liegen im Namespace der Extendedclass und werden von allen Objekten geteilt, die zur Extendedclass gehören. Sie sind wie globale Variablen, nur dass sie nicht mit dem gewöhnlichen Befehl **global** deklariert werden müssen. Sie sind in allen Extendedclass-Methoden und procs automatisch sichtbar.

Wenn die optionale Zeichenkette *init* angegeben ist, dient sie als Anfangswert der Variablen. Die Initialisierung erzwingt, dass die Variable ein einfacher skalarer Wert ist; nicht initialisierte

Variablen lassen sich dagegen mit nachfolgenden Befehlen **set** und **array** setzen und als Arrays verwenden.

Sobald ein gemeinsames Datenelement definiert ist, lässt es sich mit den Befehlen **set** und **array** innerhalb der Extendedclass-Definition setzen. Dadurch lassen sich gemeinsame Datenelemente als Arrays initialisieren. Zum Beispiel:

```
itcl::extendedclass Foo {
    common boolean
    set boolean(true) 1
    set boolean(false) 0
}
```

Wenn gemeinsame Datenelemente im Konstruktor initialisiert werden, werden sie jedes Mal erneut initialisiert, wenn neue Objekte erzeugt werden.

public *command* ?*arg arg ...*?
...siehe nächstes...

protected *command* ?*arg arg ...*?
...siehe nächstes...

private *command* ?*arg arg ...*?
Diese Befehle setzen die Schutzstufe für Extendedclass-Mitglieder, die erzeugt werden, wenn *command* ausgewertet wird. Das *command* ist gewöhnlich **method**, **proc**, **variable** oder **common**, und die übrigen *arg* vervollständigen die Mitgliedsdefinition. *command* kann aber auch ein Skript mit vielen verschiedenen Mitgliedsdefinitionen sein, und die Schutzstufe gilt dann für alle erzeugten Mitglieder.

VERWENDUNG DER EXTENDEDCLASS

Sobald eine Extendedclass definiert ist, kann der Extendedclass-Name als Befehl verwendet werden, um neue Objekte zu erzeugen, die zur Extendedclass gehören.

extendedclassName objName ?args...?
Erzeugt ein neues Objekt der Extendedclass *extendedclassName* mit dem Namen *objName*. Die übrigen Argumente werden an den Konstruktor der spezifischsten Extendedclass übergeben. Der gibt seinerseits Argumente an die Konstruktoren der Basis-Extendedclasses weiter, bevor er seinen eigenen Befehlsumpf aufruft. Wenn die Konstruktion erfolgreich ist, wird im aktuellen Namespace-Kontext ein Befehl namens *objName* erzeugt, und *objName* wird als Ergebnis dieser Operation zurückgegeben. Wenn während der Konstruktion ein Fehler auftritt, werden die Destruktoren automatisch aufgerufen, um belegte Ressourcen freizugeben, das Objekt wird gelöscht, und ein Fehler wird zurückgegeben.

Wenn *objName* die Zeichenkette "**#auto**" enthält, wird diese Zeichenkette durch einen automatisch erzeugten Namen ersetzt. Namen haben die Form *extendedclassName*<*number*>, wobei der Teil *extendedclassName* so geändert wird, dass er mit einem Kleinbuchstaben beginnt. In der Extendedclass "Toaster" würde die Angabe "**#auto**" zum Beispiel Namen wie toaster0, toaster1 und so weiter erzeugen. "**#auto**" kann auch mitten in einem Objektnamen stehen:

```
fileselectiondialog .foo.bar.#auto -background red
```

Das würde ein Objekt namens ".foo.bar.fileselectiondialog0" erzeugen.

VERWENDUNG DES OBJEKTS

Sobald ein Objekt erzeugt ist, kann der Objektname als Befehl verwendet werden, um Methoden aufzurufen, die auf das Objekt wirken.

***objName method* ?*args...*?**
Ruft eine Methode namens *method* auf einem Objekt namens *objName* auf. Die übrigen Argumente werden an die Argumentliste der Methode übergeben. Der Methodenname kann "constructor", "destructor", jeder Methodenname aus der Extendedclass-Definition oder eine der folgenden eingebauten Methoden sein.

EINGEBAUTE METHODEN

***objName* cget option**
Erlaubt den Zugriff auf öffentliche Variablen als Konfigurationsoptionen. Das ahmt das Verhalten der gewöhnlichen Operation "cget" für Tk-Widgets nach. Das Argument *option* ist eine Zeichenkette der Form "**-***varName*", und diese Methode gibt den aktuellen Wert der öffentlichen Variablen *varName* zurück.

***objName* configure ?*option*? ?*value option value ...*?**
Erlaubt den Zugriff auf öffentliche Variablen als Konfigurationsoptionen. Das ahmt das Verhalten der gewöhnlichen Operation "configure" für Tk-Widgets nach. Ohne Argumente gibt diese Methode eine Liste von Listen zurück, die alle öffentlichen Variablen beschreibt. Jede Liste hat drei Elemente: den Variablennamen, seinen Anfangswert und seinen aktuellen Wert.

Wenn eine einzelne *option* der Form "**-***varName*" angegeben ist, gibt diese Methode die Angaben für diese eine Variable zurück.

Sonst werden die Argumente als Paare *option*/*value* behandelt, die öffentlichen Variablen neue Werte zuweisen. Jede Variable erhält ihren neuen Wert, und wenn ihr "config"-Code zugeordnet ist, wird er im Kontext der Extendedclass ausgeführt, in der sie definiert wurde. Wenn der "config"-Code einen Fehler erzeugt, wird die Variable auf ihren vorherigen Wert zurückgesetzt, und die Methode **configure** gibt einen Fehler zurück.

objName* isa *extendedclassName
Gibt einen Wert ungleich null zurück, wenn der gegebene *extendedclassName* in der Abstammung des Objekts zu finden ist, und sonst null.

***objName* info *option* ?*args...*?**
Gibt Angaben zu einem bestimmten Objekt namens *objName* oder zu seiner Extendedclass-Definition zurück. Der Parameter *option* umfasst die folgenden Dinge sowie die Optionen, die der gewöhnliche Tcl-Befehl "info" erkennt:

***objName* info extendedclass**
Gibt den Namen der spezifischsten Extendedclass des Objekts *objName* zurück.

***objName* info inherit**
Gibt die Liste der Basis-Extendedclasses zurück, wie sie im Befehl "**inherit**" definiert wurden, oder eine leere Zeichenkette, wenn diese Extendedclass keine Basis-Extendedclasses hat.

***objName* info heritage**
Gibt den aktuellen Extendedclass-Namen und die gesamte Liste der Basis-Extendedclasses in der Reihenfolge zurück, in der sie bei der Mitgliedersuche und bei der Objektzerstörung durchlaufen werden.

***objName* info function ?*cmdName*? ?-protection? ?-type? ?-name? ?-args? ?-body?**

Ohne Argumente gibt dieser Befehl eine Liste aller Extendedclass-Methoden und procs zurück. Wenn *cmdName* angegeben ist, gibt er Angaben zu einer bestimmten Methode oder einem bestimmten proc zurück. Wenn keine Flags angegeben sind, gibt dieser Befehl eine Liste mit den folgenden Elementen zurück: der Schutzstufe, dem Typ (method/proc), dem qualifizierten Namen, der Argumentliste und dem Rumpf. Mit Flags lassen sich bestimmte Elemente aus dieser Liste anfordern.

***objName* info variable ?*varName*? ?-protection? ?-type? ?-name? ?-init? ?-value? ?-config?**

Ohne Argumente gibt dieser Befehl eine Liste aller objektspezifischen Variablen und gemeinsamen Datenelemente zurück. Wenn *varName* angegeben ist, gibt er Angaben zu einem bestimmten Datenelement zurück. Wenn keine Flags angegeben sind, gibt dieser Befehl eine Liste mit den folgenden Elementen zurück: der Schutzstufe, dem Typ (variable/common), dem qualifizierten Namen, dem Anfangswert und dem aktuellen Wert. Wenn *varName* eine öffentliche Variable ist, steht der "config"-Code auf dieser Liste. Mit Flags lassen sich bestimmte Elemente aus dieser Liste anfordern.

VERKETTEN VON METHODEN UND PROCS

Manchmal hat eine Basis-Extendedclass eine Methode oder ein proc, das in einer abgeleiteten Extendedclass unter demselben Namen neu definiert wird. So kann die abgeleitete Extendedclass dieselben Operationen wie die Basis-Extendedclass behandeln, aber mit ihrem eigenen spezialisierten Verhalten. Angenommen, eine Extendedclass Toaster sieht so aus:

```
itcl::extendedclass Toaster {
    variable crumbs 0
    method toast {nslices} {
        if {$crumbs > 50} {
            error "== FIRE! FIRE! =="
        }
        set crumbs [expr {$crumbs+4*$nslices}]
    }
    method clean {} {
        set crumbs 0
    }
}
```

Eine weitere Extendedclass wie SmartToaster könnte die Methode "toast" neu definieren. Wenn die Methode der Basis-Extendedclass zugänglich sein soll, lässt sie sich mit dem Namen der Basis-Extendedclass qualifizieren, um Mehrdeutigkeit zu vermeiden:

```
itcl::extendedclass SmartToaster {
    inherit Toaster
    method toast {nslices} {
        if {$crumbs > 40} {
            clean
        }
        return [Toaster::toast $nslices]
    }
}
```

Statt den Namen der Basis-Extendedclass fest einzutragen, lässt sich der Befehl "chain" so verwenden:

```
itcl::extendedclass SmartToaster {
    inherit Toaster
    method toast {nslices} {
        if {$crumbs > 40} {
            clean
        }
        return [chain $nslices]
    }
}
```

Der Befehl chain sucht in der Extendedclass-Hierarchie nach einer etwas allgemeineren Umsetzung (Basis-Extendedclass) einer Methode oder eines proc und ruft sie mit den angegebenen Argumenten auf. Er beginnt beim aktuellen Extendedclass-Kontext und durchsucht die Basis-Extendedclasses in der Reihenfolge, die der Befehl "info heritage" meldet. Wenn keine andere Umsetzung gefunden wird, tut dieser Befehl nichts und gibt die leere Zeichenkette zurück.

AUTOMATISCHES LADEN

Extendedclass-Definitionen müssen nicht ausdrücklich geladen werden; sie können bei Bedarf von der gewöhnlichen Tcl-Einrichtung zum automatischen Laden geladen werden. Jedes Verzeichnis mit Dateien von Extendedclass-Definitionen sollte eine begleitende Datei "tclIndex" haben. Jede Zeile dieser Datei nennt eine Tcl-Prozedur oder eine Extendedclass-Definition von **[incr\ Tcl]** und die Datei, in der die Definition zu finden ist.

Angenommen, ein Verzeichnis enthält die Definitionen der Extendedclasses "Toaster" und "SmartToaster". Dann sähe die Datei "tclIndex" für dieses Verzeichnis so aus:

```
# Tcl-Autoload-Indexdatei, Version 2.0 für [incr Tcl]
# Diese Datei wird vom Befehl "auto_mkindex" erzeugt
# und eingelesen, um Indexangaben für einen oder
# mehrere Befehle einzurichten. Gewöhnlich ist jede Zeile ein Befehl,
# der ein Element im Array auto_index setzt, wobei der
# Elementname der Name eines Befehls ist und der Wert
# ein Skript ist, das den Befehl lädt.

set auto_index(::Toaster) "source $dir/Toaster.itcl"
set auto_index(::SmartToaster) "source $dir/SmartToaster.itcl"
```

Der Befehl **auto_mkindex** erzeugt Dateien "tclIndex" automatisch.

Der automatische Lader muss dieses Verzeichnis kennen, indem der Verzeichnisname an die Variable "auto_path" angehängt wird. Wenn das eingerichtet ist, werden Extendedclasses bei der Verwendung in einer Anwendung bei Bedarf automatisch geladen.

C-PROZEDUREN

C-Prozeduren lassen sich in eine Extendedclass-Definition von **[incr\ Tcl]** einbinden, um Methoden, procs und den "config"-Code öffentlicher Variablen umzusetzen. Jeder Rumpf, der mit "**@**" beginnt, wird als symbolischer Name einer C-Prozedur behandelt.

Symbolische Namen werden festgelegt, indem Prozeduren über **Itcl_RegisterObjC()** registriert werden. Das geschieht gewöhnlich in der Prozedur **Tcl_AppInit()**, die automatisch aufgerufen wird, wenn der Interpreter startet. Im folgenden Beispiel wird die Prozedur \ffCMy_FooObjCmd() unter dem symbolischen Namen "foo" registriert. Im Befehl **body** lässt sich diese Prozedur als "\ffC@foo" ansprechen.

```
int
Tcl_AppInit(interp)
    Tcl_Interp *interp;     /* Interpreter für die Anwendung. */
{
    if (Itcl_Init(interp) == TCL_ERROR) {
        return TCL_ERROR;
    }

    if (Itcl_RegisterObjC(interp, "foo", My_FooObjCmd) != TCL_OK) {
        return TCL_ERROR;
    }
}
```

C-Prozeduren werden genau wie gewöhnliche Tcl-Befehle umgesetzt. Einzelheiten stehen auf der Handbuchseite von **CrtCommand**. Innerhalb der Prozedur sind Extendedclass-Datenelemente wie gewöhnliche Variablen über **Tcl_SetVar()**, **Tcl_GetVar()**, **Tcl_TraceVar()** und so weiter zugänglich. Extendedclass-Methoden und procs lassen sich wie gewöhnliche Befehle über **Tcl_Eval()** ausführen. **[incr\ Tcl]** macht das möglich, indem es den Kontext automatisch einrichtet, bevor die C-Prozedur ausgeführt wird.

Dieses Schema bietet einen natürlichen Weg, den Code weiterzuentwickeln. Extendedclasses lassen sich schnell entwickeln, indem die Rümpfe in Tcl-Code umgesetzt werden. Eine ganze

Anwendung lässt sich bauen und prüfen. Wenn nötig, lassen sich einzelne Rümpfe in C-Code umsetzen, um die Leistung zu verbessern.

Copyright (c) 2008 Arnulf Wiedemann

itcloption

Stichwörter: itcloption, option, widget, widgetadaptor, extendedclass

NAME

itcl::option - Optionen für extendedclass, widget oder widgetadaptor definieren

Teile dieser Beschreibung sind von der Tcl-Erweiterung [snit] "entliehen", weil die Funktion weitgehend gleich ist.

WARNUNG

Das ist neue Funktionalität in [incr Tcl], bei der sich die API noch ändern kann!!

ÜBERSICHT

option *optionSpec* ?*defaultValue*?
option *optionSpec* ?*options*?

BESCHREIBUNG

Der Befehl **option** wird innerhalb einer Definition extendedclass/widget/widgetadaptor von **[incr \ Tcl]** verwendet, um Optionen zu definieren.

Die erste Form definiert eine Option für Instanzen dieses Typs und gibt ihr optional einen Anfangswert. Der Anfangswert ist die leere Zeichenkette, wenn kein defaultValue angegeben ist.

Eine so definierte Option gilt als lokal definiert. Die optionSpec ist eine Liste, die den Namen der Option, den Ressourcennamen und den Klassennamen definiert, zum Beispiel:

```
    option {-font font Font} {Courier 12}
```

Der Optionsname muss mit einem Bindestrich beginnen und darf keine Großbuchstaben enthalten. Ressourcenname und Klassenname sind optional; fehlen sie, ist der Ressourcenname voreingestellt der Optionsname ohne den Bindestrich, und der Klassenname ist voreingestellt der Ressourcenname mit großem Anfangsbuchstaben. Die folgende Anweisung entspricht deshalb dem vorherigen Beispiel:

```
    option -font {Courier 12}
```

Weitere Angaben zu Ressourcen- und Klassennamen stehen in der Optionsdatenbank von Tk.

Optionen werden gewöhnlich mit den Standard-Instanzmethoden configure und cget gesetzt und abgefragt; im Instanzcode (Methodenrümpfe und so weiter) stehen die Optionswerte über das Array options zur Verfügung:

```
    set myfont $itcl_options(-font)
```

In der zweiten Form lassen sich Optionsbehandler definieren (zum Beispiel -configuremethod); der Code sollte dann wahrscheinlich configure und cget verwenden, um auf seine Optionen zuzugreifen und subtile Fehler zu vermeiden.

Die Anweisung option kann die folgenden Optionen enthalten:

-default *defvalue*
Legt den Vorgabewert der Option fest; sonst ist der Vorgabewert der Option "".

-readonly
Die Option wird nur lesbar behandelt: sie lässt sich mit configure nur zur Erzeugungszeit setzen, also im Konstruktor des Typs.

-cgetmethod *methodName*
Jede lokal definierte Option darf eine -cgetmethod definieren; sie wird aufgerufen, wenn der Wert der Option mit der Methode cget abgefragt wird. Was der Rumpf der Methode zurückgibt, ist der Rückgabewert des Aufrufs von cget.

Die genannte Methode muss ein Argument nehmen, den Optionsnamen. Dieser Code entspricht zum Beispiel der voreingestellten cget-Behandlung von Itcl, ist aber langsamer:

```
option -font -cgetmethod GetOption
method GetOption {option} {
    return $itcl_options($option)
}
```

Beliebig viele Optionen können sich eine -cgetmethod teilen.

-cgetmethodvar *varName*
Das ist -cgetmethod sehr ähnlich, der einzige Unterschied ist, dass sich eine Variable angeben lässt, in der die cgetmethod zur Laufzeit zu finden ist.

-configuremethod *methodName*
Jede lokal definierte Option darf eine -configuremethod definieren; sie wird aufgerufen, wenn der Wert der Option mit den Methoden configure oder configurelist gesetzt wird. Die genannte Methode muss den Wert der Option selbst speichern; der Wert wird also nur dann im Array itcl_options() gespeichert, wenn die Methode ihn dort speichert.

Die genannte Methode muss zwei Argumente nehmen, den Optionsnamen und seinen neuen Wert. Dieser Code entspricht zum Beispiel der voreingestellten configure-Behandlung von Itcl, ist aber langsamer:

```
option -font -configuremethod SetOption
method SetOption {option value} {
    set itcl_options($option) $value
}
```

Beliebig viele Optionen können sich eine einzelne -configuremethod teilen.

-configuremethodvar *varName*
Das ist -configuremethod sehr ähnlich, der einzige Unterschied ist, dass sich eine Variable angeben lässt, in der die configuremethod zur Laufzeit zu finden ist.

-validatemethod *methodName*
Jede lokal definierte Option darf eine -validatemethod definieren; sie wird aufgerufen, wenn der Wert der Option mit den Methoden configure oder configurelist gesetzt wird, unmittelbar vor der -configuremethod (falls vorhanden). Die genannte Methode muss den neuen Wert der Option prüfen und einen Fehler melden, wenn der Wert ungültig ist.

Die genannte Methode muss zwei Argumente nehmen, den Optionsnamen und seinen neuen Wert. Dieser Code prüft zum Beispiel, dass der Wert von -flag ein gültiger boolescher Wert ist:

```
        option -font -validatemethod CheckBoolean
        method CheckBoolean {option value} {
            if {![string is boolean -strict $value]} {
                error "option $option must have a boolean value."
            }
        }
```

Beliebig viele Optionen können sich eine einzelne -validatemethod teilen.

-validatemethodvar *varName*
Das ist -validatemethod sehr ähnlich, der einzige Unterschied ist, dass sich eine Variable angeben lässt, in der die validatemethod zur Laufzeit zu finden ist.

Copyright (c) 2008 Arnulf Wiedemann

itclvars

Stichwörter: itclvars, itcl, variables

NAME

itclvars - Variablen, die [incr\ Tcl] verwendet

BESCHREIBUNG

Die folgenden globalen Variablen werden von der Bibliothek **[incr\ Tcl]** automatisch erzeugt und verwaltet. Soweit unten nichts anderes vermerkt ist, sollten anwendungsspezifischer Code und Benutzer diese Variablen gewöhnlich als nur lesbar behandeln.

itcl::library
Wenn ein Interpreter erzeugt wird, initialisiert **[incr\ Tcl]** diese Variable mit dem Namen eines Verzeichnisses, das die Systembibliothek der Skripte von **[incr\ Tcl]** enthält. Der Anfangswert von **itcl::library** wird aus der Umgebungsvariable ITCL_LIBRARY gesetzt, falls sie existiert, sonst aus einem einkompilierten Wert.

itcl::patchLevel
Wenn ein Interpreter erzeugt wird, initialisiert **[incr\ Tcl]** diese Variable mit dem aktuellen Patch-Stand von **[incr\ Tcl]**. Der Wert "**2.0p1**" steht zum Beispiel für Version 2.0 von **[incr\ Tcl]** mit dem ersten Satz angewendeter Patches.

itcl::purist
Wenn ein Interpreter erzeugt wird, der Tcl/Tk und die Namespace-Einrichtung von **[incr\ Tcl]** enthält, steuert diese Variable einen Modus der "Rückwärtskompatibilität" für den Widget-Zugriff.

Im gewöhnlichen Tcl/Tk gibt es einen einzigen Vorrat von Befehlen, sodass der Zugriffsbefehl eines Widgets derselbe ist wie der Fenstername. Wenn ein Widget jedoch innerhalb eines Namespace erzeugt wird, wird sein Zugriffsbefehl in diesem Namespace installiert und sollte außerhalb des Namespace über einen qualifizierten Namen angesprochen werden. Zum Beispiel

```
namespace foo {
    namespace bar {
        button .b -text "Testing"
    }
```

```
}
foo::bar::.b configure -background red
pack .b
```

Der Fenstername "\ffC.b" wird weiterhin zusammen mit Befehlen wie **pack** und **destroy** verwendet. Der Zugriffsbefehl des Widgets (also der Name, der als *erstes* Argument auf einer Befehlszeile erscheint) muss jedoch genauer sein.

Mit dem Befehl "**winfo command**" lässt sich der voll qualifizierte Zugriffsbefehl jedes Widgets abfragen, sodass sich schreiben lässt:

```
[winfo command .b] configure -background red
```

und das ist gute Praxis beim Schreiben von Bibliotheksprozeduren. Zusammen mit dem Befehl **bind** kann das Feld "%q" anstelle von "%W" als Zugriffsbefehl verwendet werden:

```
bind Button <Key-Return> {%q flash; %q invoke}
```

Dieses Verhalten ist aus Sicht der Kapselung sinnvoll, verursacht aber Probleme mit vorhandenen Tcl/Tk-Anwendungen. Viele vorhandene Anwendungen sind mit Bindungen geschrieben, die "%W" verwenden. Viele Bibliotheksprozeduren nehmen an, dass der Fenstername der Zugriffsbefehl ist.

Die Variable **itcl::purist** steuert einen Modus der Rückwärtskompatibilität. Voreingestellt ist diese Variable "0", und der Fenstername lässt sich in jedem Kontext als Zugriffsbefehl verwenden. Immer wenn die Prozedur **unknown** auf einen Widgetnamen stößt, verwendet sie einfach "**winfo command**", um den passenden Befehlsnamen zu ermitteln. Wenn diese Variable auf "1" gesetzt ist, ist dieser Modus der Rückwärtskompatibilität abgeschaltet. Das ergibt eine bessere Kapselung, die Verwendung des Fensternamens als Zugriffsbefehl kann aber zu Fehlern "invalid command" führen.

itcl::version
: Wenn ein Interpreter erzeugt wird, initialisiert **[incr\ Tcl]** diese Variable mit der Versionsnummer der Form *x.y*. Änderungen an *x* stehen für große Änderungen mit wahrscheinlichen Inkompatibilitäten, Änderungen an *y* stehen für kleine Erweiterungen und Fehlerbehebungen, die die Rückwärtskompatibilität bewahren.

Copyright (c) 1993-1998 Lucent Technologies, Inc.

itclwidget

Stichwörter: itclwidget, widget, object, object-oriented

NAME

itcl::widget - eine Widget-Klasse von Objekten erzeugen

WARNUNG

Das ist neue Funktionalität in [incr Tcl], bei der sich die API noch ändern kann!!

ÜBERSICHT

itcl::widget *widgetName* {
inherit *baseWidget* ?*baseWidget* ...?

constructor *args* ?*init*? *body*
destructor *body*
public method *name* ?*args*? ?*body*?
protected method *name* ?*args*? ?*body*?
private method *name* ?*args*? ?*body*?
public proc *name* ?*args*? ?*body*?
protected proc *name* ?*args*? ?*body*?
private proc *name* ?*args*? ?*body*?
public variable *varName* ?*init*? ?*config*?
protected variable *varName* ?*init*? ?*config*?
private variable *varName* ?*init*? ?*config*?
public common *varName* ?*init*?
protected common *varName* ?*init*?
private common *varName* ?*init*?

public *command* ?*arg arg* ...?
protected *command* ?*arg arg* ...?
private *command* ?*arg arg* ...?

<delegation info> see delegation page

<option info> see option page

set *varName* ?*value*?
array *option* ?*arg arg* ...?
}

widgetName objName ?*arg arg ...*?

objName method ?*arg arg ...*?

widgetName::proc ?*arg arg ...*?

BESCHREIBUNG

Eines der grundlegenden Konstrukte in **[incr\ Tcl]** ist die Widget-Definition. Ein Widget ist wie eine Klasse mit einigen zusätzlichen Merkmalen. Jede Widget dient als Vorlage für tatsächliche Objekte, die erzeugt werden können. Das Widget selbst ist ein Namespace, der enthält, was allen Objekten gemeinsam ist. Jedes Objekt hat sein eigenes Datenbündel, das Instanzen der in der Widget-Definition festgelegten "variables" enthält. Jedes Objekt hat außerdem eine eingebaute Variable namens "this", die den Namen des Objekts enthält. Widgets können auch "common"-Datenelemente haben, die allen Objekten eines Widgets gemeinsam sind.

In die Widget-Definition können zwei Arten von Funktionen aufgenommen werden. "Methods" sind Funktionen, die auf ein bestimmtes Objekt wirken und deshalb Zugriff auf "variables" und auf "common"-Datenelemente haben. "Procs" sind gewöhnliche Prozeduren im Namespace des Widgets und haben nur Zugriff auf "common"-Datenelemente.

Wenn der Rumpf einer Methode oder eines proc mit "@" beginnt, wird er als symbolischer Name einer C-Prozedur behandelt. Sonst wird er als Tcl-Codeskript behandelt. Unten steht, wie C-Prozeduren registriert und verwendet werden.

Ein Widget kann nur einmal definiert werden, die Rümpfe von Widget-Methoden und procs lassen sich zur interaktiven Fehlersuche aber immer wieder definieren. Einzelheiten stehen bei den Befehlen **body** und **configbody**.

Jeder Namespace kann seine eigene Sammlung von Objekten und Widgets haben. Die Liste der im aktuellen Kontext verfügbaren Widgets lässt sich mit dem Befehl "**itcl::find widgets**" abfragen, die Liste der Objekte mit dem Befehl "**itcl::find objects**".

Ein Widget lässt sich mit dem Befehl "**delete widget**" löschen. Einzelne Objekte lassen sich mit dem Befehl "**delete object**" löschen.

WIDGET-DEFINITIONEN

widget *widgetName definition*

Liefert die Definition für ein Widget namens *widgetName*. Wenn das Widget *widgetName* bereits existiert oder im aktuellen Namespace-Kontext ein Befehl namens *widgetName* existiert, gibt dieser Befehl einen Fehler zurück. Wenn die Widget-Definition erfolgreich zerlegt wird, wird *widgetName* ein Befehl im aktuellen Kontext und übernimmt das Erzeugen von Objekten dieses Widgets. Die *definition* des Widgets wird als Folge von Tcl-Anweisungen ausgewertet, die Elemente innerhalb des Widgets definieren. Die folgenden Befehle zur Widget-Definition werden erkannt:

inherit *baseWidget* ?*baseWidget*...?

Bewirkt, dass das aktuelle Widget Eigenschaften von einer oder mehreren Basis-Widgets erbt. Die Widgets müssen durch einen vorherigen Befehl **widget** definiert worden sein oder der Einrichtung zum automatischen Laden zur Verfügung stehen (siehe unten **AUTOMATISCHES LADEN**). Eine einzelne Widget-Definition darf höchstens einen Befehl **inherit** enthalten.

Die Reihenfolge der Namen *baseWidget* in der Liste **inherit** beeinflusst die Namensauflösung für Widget-Mitglieder. Wenn derselbe Mitgliedsname in zwei oder mehr Basis-Widgets vorkommt, hat das Basis-Widget Vorrang, die in der Liste **inherit** zuerst steht. Wenn die Widgets "Foo" und "Bar" zum Beispiel beide das Mitglied "x" enthalten und ein anderes Widget die Anweisung "**inherit**" hat:

```
inherit Foo Bar
```

dann bedeutet der Name "x" "Foo::x". Andere geerbte Mitglieder namens "x" müssen mit ihrem ausdrücklichen Namen angesprochen werden, wie "Bar::x".

constructor *args* ?*init*? *body*

Deklariert die Argumentliste *args* und den *body*, die für den Konstruktor verwendet werden. Er wird automatisch aufgerufen, sobald ein Objekt erzeugt wird.

Bevor der *body* ausgeführt wird, dient die optionale Anweisung *init* dazu, Basis-Widget-Konstruktoren aufzurufen, die Argumente verlangen. Variablen in der Angabe *args* sind im Codefragment *init* zugänglich und können an Basis-Widget-Konstruktoren übergeben werden. Nach der Auswertung der Anweisung *init* werden alle noch nicht ausgeführten Basis-Widget-Konstruktoren automatisch ohne Argumente aufgerufen. So sind alle Basis-Widgets vollständig konstruiert, bevor der *body* des Konstruktors ausgeführt wird. Voreingestellt bewirkt dieses Schema, dass Konstruktoren der Reihe nach vom allgemeinsten zum spezifischsten aufgerufen werden. Das ist genau die umgekehrte Reihenfolge, in der der Befehl **info heritage** die Widgets meldet.

Wenn die Konstruktion erfolgreich ist, gibt der Konstruktor immer den Objektnamen zurück, unabhängig davon, wie der *body* definiert ist, und der Objektname wird ein Befehl im aktuellen Namespace-Kontext. Wenn die Konstruktion scheitert, wird eine Fehlermeldung zurückgegeben.

destructor *body*
Deklariert den *body* für den Destruktor, der automatisch aufgerufen wird, wenn ein Objekt gelöscht wird. Wenn der Destruktor erfolgreich ist, werden die Objektdaten zerstört und der Objektname als Befehl aus dem Interpreter entfernt. Wenn die Zerstörung scheitert, wird eine Fehlermeldung zurückgegeben und das Objekt bleibt bestehen.

Wenn ein Objekt zerstört wird, werden alle Destruktoren seiner Widget-Hierarchie der Reihe nach vom spezifischsten zum allgemeinsten aufgerufen. Das ist die Reihenfolge, in der der Befehl "**info heritage**" die Widgets meldet, und genau die Umkehrung der voreingestellten Konstruktorreihenfolge.

method *name* ?*args*? ?*body*?
Deklariert eine Methode namens *name*. Wenn der *body* der Methode ausgeführt wird, hat er automatischen Zugriff auf objektspezifische Variablen und gemeinsame Datenelemente.

Wenn die Liste *args* angegeben ist, legt sie die Verwendungsangabe für diese Methode fest. Mit dem Befehl **body** lässt sich der Methodenrumpf neu definieren, die Liste *args* muss aber zu dieser Angabe passen.

Innerhalb des Rumpfes einer anderen Widget-Methode lässt sich eine Methode wie jeder andere Befehl aufrufen, einfach über ihren Namen. Außerhalb des Widget-Kontexts muss dem Methodennamen ein Objektname vorangestellt werden, der den Kontext für die Daten liefert, die sie bearbeitet. Methoden einer Basis-Widget, die in der aktuellen Widgets neu definiert oder von einer anderen Basis-Widget verdeckt sind, lassen sich mit der Syntax "*widgetName*::*method*" qualifizieren.

proc *name* ?*args*? ?*body*?
Deklariert ein proc namens *name*. Ein proc ist eine gewöhnliche Prozedur im Namespace des Widgets. Anders als eine Methode wird ein proc aufgerufen, ohne sich auf ein bestimmtes Objekt zu beziehen. Wenn der *body* des proc ausgeführt wird, hat er automatischen Zugriff nur auf gemeinsame Datenelemente.

Wenn die Liste *args* angegeben ist, legt sie die Verwendungsangabe für dieses proc fest. Mit dem Befehl **body** lässt sich der Rumpf des proc neu definieren, die Liste *args* muss aber zu dieser Angabe passen.

Innerhalb des Rumpfes einer anderen Widget-Methode oder eines proc lässt sich ein proc wie jeder andere Befehl aufrufen, einfach über seinen Namen. In jedem anderen Namespace-Kontext wird das proc mit einem qualifizierten Namen wie "*widgetName***::***proc*" aufgerufen. Procs einer Basis-Widget, die in der aktuellen Widgets neu definiert oder von einer anderen Basis-Widget verdeckt sind, sind ebenfalls über ihren qualifizierten Namen zugänglich.

variable *varName* ?*init*? ?*config*?
Definiert eine objektspezifische Variable namens *varName*. Alle objektspezifischen Variablen stehen in Widget-Methoden automatisch zur Verfügung. Sie müssen nicht mit etwas wie dem Befehl **global** deklariert werden.

Wenn die optionale Zeichenkette *init* angegeben ist, dient sie als Anfangswert der Variablen, wenn ein neues Objekt erzeugt wird. Die Initialisierung erzwingt, dass die Variable ein einfacher skalarer Wert ist; nicht initialisierte Variablen lassen sich dagegen im Konstruktor setzen und als Arrays verwenden.

Das optionale Skript *config* ist nur für öffentliche Variablen erlaubt. Wenn es angegeben ist, wird

dieses Codefragment jedes Mal ausgeführt, wenn eine öffentliche Variable durch die eingebaute Methode "configure" geändert wird. Das Skript *config* lässt sich auch außerhalb der Widget-Definition mit dem Befehl **configbody** angeben.

common *varName* ?*init*?
Deklariert eine common-Variable namens *varName*. Common-Variablen liegen im Namespace des Widgets und werden von allen Objekten geteilt, die zum Widget gehören. Sie sind wie globale Variablen, nur dass sie nicht mit dem gewöhnlichen Befehl **global** deklariert werden müssen. Sie sind in allen Widget-Methoden und procs automatisch sichtbar.

Wenn die optionale Zeichenkette *init* angegeben ist, dient sie als Anfangswert der Variablen. Die Initialisierung erzwingt, dass die Variable ein einfacher skalarer Wert ist; nicht initialisierte Variablen lassen sich dagegen mit nachfolgenden Befehlen **set** und **array** setzen und als Arrays verwenden.

Sobald ein gemeinsames Datenelement definiert ist, lässt es sich mit den Befehlen **set** und **array** innerhalb der Widget-Definition setzen. Dadurch lassen sich gemeinsame Datenelemente als Arrays initialisieren. Zum Beispiel:

```
itcl::widget Foo {
    protected common boolean
    set boolean(true) 1
    set boolean(false) 0
}
```

Wenn gemeinsame Datenelemente im Konstruktor initialisiert werden, werden sie jedes Mal erneut initialisiert, wenn neue Objekte erzeugt werden.

public *command* ?*arg arg ...*?
...siehe nächstes...

protected *command* ?*arg arg ...*?
...siehe nächstes...

private *command* ?*arg arg ...*?
Diese Befehle setzen die Schutzstufe für Widget-Mitglieder, die erzeugt werden, wenn *command* ausgewertet wird. Das *command* ist gewöhnlich **method**, **proc**, **variable** oder **common**, und die übrigen *arg* vervollständigen die Mitgliedsdefinition. *command* kann aber auch ein Skript mit vielen verschiedenen Mitgliedsdefinitionen sein, und die Schutzstufe gilt dann für alle erzeugten Mitglieder.

VERWENDUNG DES WIDGETS

Sobald ein Widget definiert ist, kann der Widget-Name als Befehl verwendet werden, um neue Objekte zu erzeugen, die zum Widget gehören.

widgetName objName ?args...?
Erzeugt ein neues Objekt des Widgets *widgetName* mit dem Namen *objName*. Die übrigen Argumente werden an den Konstruktor des spezifischsten Widgets übergeben. Der gibt seinerseits Argumente an die Konstruktoren der Basis-Widgets weiter, bevor er seinen eigenen Befehlsumpf aufruft. Wenn die Konstruktion erfolgreich ist, wird im aktuellen Namespace-Kontext ein Befehl namens *objName* erzeugt, und *objName* wird als Ergebnis dieser Operation zurückgegeben. Wenn während der Konstruktion ein Fehler auftritt, werden

die Destruktoren automatisch aufgerufen, um belegte Ressourcen freizugeben, das Objekt wird gelöscht, und ein Fehler wird zurückgegeben.

Wenn *objName* die Zeichenkette "**#auto**" enthält, wird diese Zeichenkette durch einen automatisch erzeugten Namen ersetzt. Namen haben die Form *widgetName*<*number*>, wobei der Teil *widgetName* so geändert wird, dass er mit einem Kleinbuchstaben beginnt. In des Widgets "Toaster" würde die Angabe "**#auto**" zum Beispiel Namen wie toaster0, toaster1 und so weiter erzeugen. "**#auto**" kann auch mitten in einem Objektnamen stehen:

```
fileselectiondialog .foo.bar.#auto -background red
```

Das würde ein Objekt namens ".foo.bar.fileselectiondialog0" erzeugen.

VERWENDUNG DES OBJEKTS

Sobald ein Objekt erzeugt ist, kann der Objektname als Befehl verwendet werden, um Methoden aufzurufen, die auf das Objekt wirken.

***objName method* ?*args...*?**

Ruft eine Methode namens *method* auf einem Objekt namens *objName* auf. Die übrigen Argumente werden an die Argumentliste der Methode übergeben. Der Methodenname kann "constructor", "destructor", jeder Methodenname aus der Widget-Definition oder eine der folgenden eingebauten Methoden sein.

EINGEBAUTE METHODEN

***objName* cget option**

Erlaubt den Zugriff auf öffentliche Variablen als Konfigurationsoptionen. Das ahmt das Verhalten der gewöhnlichen Operation "cget" für Tk-Widgets nach. Das Argument *option* ist eine Zeichenkette der Form "**-***varName*", und diese Methode gibt den aktuellen Wert der öffentlichen Variablen *varName* zurück.

***objName* configure ?*option*? ?*value option value ...*?**

Erlaubt den Zugriff auf öffentliche Variablen als Konfigurationsoptionen. Das ahmt das Verhalten der gewöhnlichen Operation "configure" für Tk-Widgets nach. Ohne Argumente gibt diese Methode eine Liste von Listen zurück, die alle öffentlichen Variablen beschreibt. Jede Liste hat drei Elemente: den Variablennamen, seinen Anfangswert und seinen aktuellen Wert.

Wenn eine einzelne *option* der Form "**-***varName*" angegeben ist, gibt diese Methode die Angaben für diese eine Variable zurück.

Sonst werden die Argumente als Paare *option*/*value* behandelt, die öffentlichen Variablen neue Werte zuweisen. Jede Variable erhält ihren neuen Wert, und wenn ihr "config"-Code zugeordnet ist, wird er im Kontext des Widgets ausgeführt, in der sie definiert wurde. Wenn der "config"-Code einen Fehler erzeugt, wird die Variable auf ihren vorherigen Wert zurückgesetzt, und die Methode **configure** gibt einen Fehler zurück.

objName* isa *widgetName

Gibt einen Wert ungleich null zurück, wenn der gegebene *widgetName* in der Abstammung des Objekts zu finden ist, und sonst null.

***objName* info *option* ?*args...*?**

Gibt Angaben zu einem bestimmten Objekt namens *objName* oder zu seiner Widget-Definition zurück. Der Parameter *option* umfasst die folgenden Dinge sowie die Optionen, die der

gewöhnliche Tcl-Befehl "info" erkennt:

***objName* info widget**
Gibt den Namen des spezifischsten Widgets des Objekts *objName* zurück.

***objName* info inherit**
Gibt die Liste der Basis-Widgets zurück, wie sie im Befehl "**inherit**" definiert wurden, oder eine leere Zeichenkette, wenn dieses Widget keine Basis-Widgets hat.

***objName* info heritage**
Gibt den aktuellen Widget-Namen und die gesamte Liste der Basis-Widgets in der Reihenfolge zurück, in der sie bei der Mitgliedersuche und bei der Objektzerstörung durchlaufen werden.

***objName* info function ?*cmdName*? ?-protection? ?-type? ?-name? ?-args? ?-body?**
Ohne Argumente gibt dieser Befehl eine Liste aller Widget-Methoden und procs zurück. Wenn *cmdName* angegeben ist, gibt er Angaben zu einer bestimmten Methode oder einem bestimmten proc zurück. Wenn keine Flags angegeben sind, gibt dieser Befehl eine Liste mit den folgenden Elementen zurück: der Schutzstufe, dem Typ (method/proc), dem qualifizierten Namen, der Argumentliste und dem Rumpf. Mit Flags lassen sich bestimmte Elemente aus dieser Liste anfordern.

***objName* info variable ?*varName*? ?-protection? ?-type? ?-name? ?-init? ?-value? ?-config?**
Ohne Argumente gibt dieser Befehl eine Liste aller objektspezifischen Variablen und gemeinsamen Datenelemente zurück. Wenn *varName* angegeben ist, gibt er Angaben zu einem bestimmten Datenelement zurück. Wenn keine Flags angegeben sind, gibt dieser Befehl eine Liste mit den folgenden Elementen zurück: der Schutzstufe, dem Typ (variable/common), dem qualifizierten Namen, dem Anfangswert und dem aktuellen Wert. Wenn *varName* eine öffentliche Variable ist, steht der "config"-Code auf dieser Liste. Mit Flags lassen sich bestimmte Elemente aus dieser Liste anfordern.

VERKETTEN VON METHODEN UND PROCS

Manchmal hat eine Basis-Widget eine Methode oder ein proc, das in einer abgeleiteten Widgets unter demselben Namen neu definiert wird. So kann die abgeleitetes Widget dieselben Operationen wie die Basis-Widget behandeln, aber mit ihrem eigenen spezialisierten Verhalten. Angenommen, ein Widget Toaster sieht so aus:

```
itcl::widget Toaster {
    variable crumbs 0
    method toast {nslices} {
        if {$crumbs > 50} {
            error "== FIRE! FIRE! =="
        }
        set crumbs [expr {$crumbs+4*$nslices}]
    }
    method clean {} {
        set crumbs 0
    }
}
```

Eine weitere Widget wie SmartToaster könnte die Methode "toast" neu definieren. Wenn die

Methode der Basis-Widget zugänglich sein soll, lässt sie sich mit dem Namen der Basis-Widget qualifizieren, um Mehrdeutigkeit zu vermeiden:

```
itcl::widget SmartToaster {
    inherit Toaster
    method toast {nslices} {
        if {$crumbs > 40} {
            clean
        }
        return [Toaster::toast $nslices]
    }
}
```

Statt den Namen der Basis-Widget fest einzutragen, lässt sich der Befehl "chain" so verwenden:

```
itcl::widget SmartToaster {
    inherit Toaster
    method toast {nslices} {
        if {$crumbs > 40} {
            clean
        }
        return [chain $nslices]
    }
}
```

Der Befehl chain sucht in der Widget-Hierarchie nach einer etwas allgemeineren Umsetzung (Basis-Widget) einer Methode oder eines proc und ruft sie mit den angegebenen Argumenten auf. Er beginnt beim aktuellen Widget-Kontext und durchsucht die Basis-Widgets in der Reihenfolge, die der Befehl "info heritage" meldet. Wenn keine andere Umsetzung gefunden wird, tut dieser Befehl nichts und gibt die leere Zeichenkette zurück.

AUTOMATISCHES LADEN

Widget-Definitionen müssen nicht ausdrücklich geladen werden; sie können bei Bedarf von der gewöhnlichen Tcl-Einrichtung zum automatischen Laden geladen werden. Jedes Verzeichnis mit Dateien von Widget-Definitionen sollte eine begleitende Datei "tclIndex" haben. Jede Zeile dieser Datei nennt eine Tcl-Prozedur oder eine Widget-Definition von **[incr\ Tcl]** und die Datei, in der die Definition zu finden ist.

Angenommen, ein Verzeichnis enthält die Definitionen des Widgetsn "Toaster" und "SmartToaster". Dann sähe die Datei "tclIndex" für dieses Verzeichnis so aus:

```
# Tcl-Autoload-Indexdatei, Version 2.0 für [incr Tcl]
# Diese Datei wird vom Befehl "auto_mkindex" erzeugt
# und eingelesen, um Indexangaben für einen oder
# mehrere Befehle einzurichten. Gewöhnlich ist jede Zeile ein Befehl,
# der ein Element im Array auto_index setzt, wobei der
# Elementname der Name eines Befehls ist und der Wert
# ein Skript ist, das den Befehl lädt.

set auto_index(::Toaster) "source $dir/Toaster.itcl"
set auto_index(::SmartToaster) "source $dir/SmartToaster.itcl"
```

Der Befehl **auto_mkindex** erzeugt Dateien "tclIndex" automatisch.

Der automatische Lader muss dieses Verzeichnis kennen, indem der Verzeichnisname an die Variable "auto_path" angehängt wird. Wenn das eingerichtet ist, werden Widgets bei der Verwendung in einer Anwendung bei Bedarf automatisch geladen.

C-PROZEDUREN

C-Prozeduren lassen sich in eine Widget-Definition von **[incr\ Tcl]** einbinden, um Methoden, procs und den "config"-Code öffentlicher Variablen umzusetzen. Jeder Rumpf, der mit "**@**" beginnt, wird als symbolischer Name einer C-Prozedur behandelt.

Symbolische Namen werden festgelegt, indem Prozeduren über **Itcl_RegisterObjC()** registriert werden. Das geschieht gewöhnlich in der Prozedur **Tcl_AppInit()**, die automatisch aufgerufen wird, wenn der Interpreter startet. Im folgenden Beispiel wird die Prozedur \ffCMy_FooObjCmd() unter dem symbolischen Namen "foo" registriert. Im Befehl **body** lässt sich diese Prozedur als "\ffC@foo" ansprechen.

```
int
Tcl_AppInit(interp)
    Tcl_Interp *interp;     /* Interpreter für die Anwendung. */
{
    if (Itcl_Init(interp) == TCL_ERROR) {
        return TCL_ERROR;
    }

    if (Itcl_RegisterObjC(interp, "foo", My_FooObjCmd) != TCL_OK) {
        return TCL_ERROR;
    }
}
```

C-Prozeduren werden genau wie gewöhnliche Tcl-Befehle umgesetzt. Einzelheiten stehen auf der Handbuchseite von **CrtCommand**. Innerhalb der Prozedur sind Widget-Datenelemente wie gewöhnliche Variablen über **Tcl_SetVar()**, **Tcl_GetVar()**, **Tcl_TraceVar()** und so weiter zugänglich. Widget-Methoden und procs lassen sich wie gewöhnliche Befehle über **Tcl_Eval()** ausführen. **[incr\ Tcl]** macht das möglich, indem es den Kontext automatisch einrichtet, bevor die C-Prozedur ausgeführt wird.

Dieses Schema bietet einen natürlichen Weg, den Code weiterzuentwickeln. Widgets lassen sich schnell entwickeln, indem die Rümpfe in Tcl-Code umgesetzt werden. Eine ganze Anwendung

lässt sich bauen und prüfen. Wenn nötig, lassen sich einzelne Rümpfe in C-Code umsetzen, um die Leistung zu verbessern.

Copyright (c) 2008 Arnulf Wiedemann

local

Stichwörter: local, class, object, procedure

NAME

itcl::local - ein Objekt erzeugen, das lokal zu einer Prozedur ist

ÜBERSICHT

itcl::local *className objName* ?*arg arg* ...?

BESCHREIBUNG

Der Befehl **local** erzeugt ein Objekt von **[incr\ Tcl]**, das lokal zum aktuellen Aufruf-Frame ist. Wenn der Aufruf-Frame verschwindet, wird das Objekt automatisch gelöscht. Dieser Befehl eignet sich, um Objekte zu erzeugen, die lokal zu einer Prozedur sind.

Als Nebenwirkung erzeugt dieser Befehl eine Variable namens "**itcl-local-***xxx*", wobei *xxx* der Name des erzeugten Objekts ist. Diese Variable erkennt, wenn der Aufruf-Frame zerstört wird, und löscht das zugeordnete Objekt automatisch.

BEISPIEL

Im folgenden Beispiel wird ein einfaches Objekt "counter" innerhalb der Prozedur "test" verwendet. Der Zähler wird als lokales Objekt erzeugt und deshalb bei jedem Verlassen der Prozedur automatisch gelöscht. Die Anweisungen **puts** in Konstruktor und Destruktor zeigen, wie das Objekt beim Aufruf der Prozedur entsteht und wieder verschwindet.

```
itcl::class counter {
    private variable count 0
    constructor {} {
        puts "created: $this"
    }
    destructor {
        puts "deleted: $this"
    }

    method bump {{by 1}} {
        incr count $by
    }
    method get {} {
        return $count
    }
}

proc test {val} {
    local counter x
```

```
    for {set i 0} {$i < $val} {incr i} {
        x bump
    }
    return [x get]
}

set result [test 5]
puts "test: $result"

set result [test 10]
puts "test: $result"

puts "objects: [itcl::find objects *]"
```

Copyright (c) 1993-1998 Lucent Technologies, Inc.

scope

Stichwörter: scope, code, namespace, variable

NAME

itcl::scope - den Namespace-Kontext einer Variablen festhalten

ÜBERSICHT

itcl::scope *name*

BESCHREIBUNG

Erzeugt einen geltungsbezogenen Wert für den angegebenen *name*, der ein Variablenname sein muss. Wenn der *name* eine Instanzvariable ist, gibt der Befehl scope einen Namen zurück, der sich in jedem Kontext als Instanzvariable auflöst, die zu *object* gehört. Das genaue Format dieses Namens ist eine interne Einzelheit von Itcl. Ein solcher geltungsbezogener Wert macht es möglich, Instanzvariablen zusammen mit Widgets zu verwenden. Wenn ein Objekt eine private Variable \ffCx hat, lässt sich \ffCx zusammen mit der Option \ffC-textvariable eines Entry-Widgets verwenden. Vor itcl3.0 ließen sich auf diese Weise nur common-Variablen verwenden.

Wenn der *name* keine Instanzvariable ist, muss es eine common-Variable oder eine globale Variable sein. In diesem Fall gibt der Befehl scope den voll qualifizierten Namen der Variablen zurück, zum Beispiel \ffC::foo::bar::x.

Wenn der *name* nicht als Variable erkannt wird, gibt der Befehl scope einen Fehler zurück.

Gewöhnliche Variablennamen bezeichnen Variablen im globalen Namespace. Ein geltungsbezogener Wert hält einen Variablennamen samt seinem Namespace-Kontext so fest, dass später richtig darauf Bezug genommen werden kann. Er wird zum Beispiel gebraucht, um Variablennamen einzupacken, wenn ein Tk-Widget innerhalb eines Namespace verwendet wird:

```
namespace foo {
    private variable mode 1

    radiobutton .rb1 -text "Mode #1" \
```

```
        -variable [scope mode] -value 1
    pack .rb1

    radiobutton .rb2 -text "Mode #2" \
        -variable [scope mode] -value 2
    pack .rb2
}
```

Die Radiobuttons \ffC.rb1 und \ffC.rb2 wirken über die Variable "mode" zusammen, die im Namespace "foo" enthalten ist. Der Befehl **scope** stellt das sicher, indem er den voll qualifizierten Variablennamen \ffC::foo::mode zurückgibt.

Eigene geltungsbezogene Variablennamen sollten nie selbst gebastelt werden, auch wenn die Kodierung fehlerfrei nachgebaut scheint. Stattdessen sollte immer der Befehl scope den Variablennamen dynamisch erzeugen. Dieser Name lässt sich dann an ein Widget oder an jedes andere Stück Code im Programm übergeben.

Copyright (c) 1993-1998 Lucent Technologies, Inc.

itk

Archetype

Stichwörter: Archetype, itk, Widget, Toplevel, mega-widget

NAME

Archetype - Basisklasse für alle Megawidgets von [incr\ Tk]

VERERBUNG

keine

WIDGET-SPEZIFISCHE OPTIONEN

```
Name:           clientData
Class:          ClientData
Command-Line Switch: -clientdata
```

Das beeinflusst den Betrieb des Widgets in keiner Weise. Es ist lediglich ein Haken, über den Aufrufer bei jedem Widget ein paar Daten ablegen können. Das ist praktisch, wenn mit Widgets Anwendungen gebaut werden.

BESCHREIBUNG

Die Klasse **Archetype** ist die Grundlage aller Megawidgets von **[incr\ Tk]**. Sie verfolgt Komponenten-Widgets und stellt Methoden wie "configure" und "cget" bereit, mit denen auf die zusammengesetzten Konfigurationsoptionen zugegriffen wird. Jedes Komponenten-Widget muss mit der Methode "**itk_component add**" bei der Basisklasse **Archetype** registriert werden. Wenn die Komponente registriert wird, gehen ihre Konfigurationsoptionen in die zusammengesetzte Optionsliste ein. Das Konfigurieren einer zusammengesetzten Option wie "-background" bewirkt, dass alle internen Komponenten ihre Hintergrundfarbe ändern.

Sie wird nicht selbst als Widget verwendet, sondern als Basisklasse für stärker spezialisierte

Widgets. Die Basisklasse **Widget** erbt von **Archetype** und fügt einen Tk-Frame hinzu, der als "hull" des Megawidgets dient. Die Basisklasse **Toplevel** erbt von **Archetype**, fügt aber ein Toplevel-Fenster von Tk hinzu, das als "hull" dient.

Jede abgeleitete Klasse muss die Methode **itk_initialize** in ihrem Konstruktor aufrufen, damit alle Optionen richtig in die zusammengesetzte Liste eingehen und initialisiert werden.

ÖFFENTLICHE METHODEN

Die folgenden Methoden unterstützen die öffentliche Schnittstelle des Megawidgets.

pathName* cget *option

Gibt den aktuellen Wert der durch *option* gegebenen Konfigurationsoption zurück. *option* bezeichnet hier eine zusammengesetzte Konfigurationsoption des Megawidgets. Einzelne Komponenten nehmen ihre eigenen Konfigurationsoptionen in die zusammengesetzte Liste auf, wenn sie mit der Methode "**itk_component add**" registriert werden.

***pathName* component ?*name*? ?*command arg arg ...*?**

Dient dazu, Komponenten-Widgets innerhalb eines Megawidgets abzufragen oder anzusprechen. Ohne Argumente gibt die Methode eine Liste der symbolischen Namen der Komponenten-Widgets zurück, die im aktuellen Geltungsbereich zugänglich sind. Der symbolische Name einer Komponente wird festgelegt, wenn sie mit der Methode "**itk_component add**" registriert wird. Komponenten-Widgets gehorchen jeder Zugriffseinschränkung public/protected/private, die beim Erzeugen der Komponente gilt. Wenn ein symbolischer *name* angegeben ist, gibt diese Methode den Fensterpfadnamen dieser Komponente zurück. Andernfalls werden *command* und die übrigen Argumente *arg* als Methode der Komponente mit dem symbolischen Namen *name* aufgerufen. Das bietet einen festgelegten Weg, interne Komponenten anzusprechen, ohne sich auf bestimmte Fensterpfadnamen zu stützen, die eigentlich Einzelheiten der Umsetzung sind.

***pathName* configure ?*option*? ?*value option value ...*?**

Fragt die Konfigurationsoptionen des Widgets ab oder ändert sie. Wenn keine *option* angegeben ist, gibt der Befehl eine Liste zurück, die alle für *pathName* verfügbaren Optionen beschreibt (das Format dieser Liste steht bei **Tk_ConfigureInfo**). Wenn *option* ohne *value* angegeben ist, gibt der Befehl eine Liste zurück, die die eine genannte Option beschreibt (diese Liste ist mit der entsprechenden Teilliste des Wertes identisch, der zurückgegeben wird, wenn keine *option* angegeben ist). Wenn ein oder mehrere Paare *option-value* angegeben sind, ändert der Befehl die genannten Widget-Optionen auf die genannten Werte; in diesem Fall gibt der Befehl eine leere Zeichenkette zurück. Die *options* bezeichnen hier zusammengesetzte Konfigurationsoptionen des Megawidgets. Einzelne Komponenten nehmen ihre eigenen Konfigurationsoptionen in die zusammengesetzte Liste auf, wenn sie mit der Methode "**itk_component add**" registriert werden.

GESCHÜTZTE METHODEN

Die folgenden Methoden werden in abgeleiteten Klassen als Teil der Umsetzung eines Megawidgets verwendet.

itk_component add ?-protected? ?-private? ?--? *name createCmds* ?*optionCmds*?

Erzeugt ein Komponenten-Widget, indem das Argument *createCmds* ausgeführt wird, und registriert die neue Komponente unter dem symbolischen Namen *name*. Die Optionen **-protected** und **-private** halten die Komponente vor der Außenwelt verborgen. Diese

Optionen wirken auf die Sichtbarkeit einer Komponente ähnlich wie auf Klassenmitglieder. Der Code *createCmds* kann beliebig viele Befehle enthalten, muss aber den Fensterpfadnamen des neuen Komponenten-Widgets zurückgeben. Das Skript *optionCmds* enthält Befehle, die beschreiben, wie die Konfigurationsoptionen der neuen Komponente in die zusammengesetzte Liste des Megawidgets eingehen sollen. Es kann jeden der folgenden Befehle enthalten:

ignore *option* ?*option option ...*?
Entfernt eine oder mehrere Konfigurationsoptionen aus der zusammengesetzten Liste. Alle Optionen werden voreingestellt ignoriert, der Befehl **ignore** hebt also nur die Wirkung eines vorherigen Befehls **keep** oder **rename** auf. Das ist zum Beispiel nützlich, wenn einige der vom Befehl **usual** hinzugefügten Optionen auf eine bestimmte Komponente nicht zutreffen sollen und ignoriert werden müssen.

keep *option* ?*option option ...*?
Nimmt eine oder mehrere Konfigurationsoptionen in die zusammengesetzte Liste auf und behält den Namen bei. Immer wenn die Option des Megawidgets konfiguriert wird, wird der neue Wert auch auf die aktuelle Komponente angewendet. Optionen wie "-background" und "-cursor" stehen häufig auf der Liste **keep**.

rename *option switchName resourceName resourceClass*
Nimmt die Konfigurationsoption unter einem anderen Namen in die zusammengesetzte Liste auf. Die Option heißt auf der zusammengesetzten Liste *switchName*. Sie lässt sich außerdem ändern, indem Werte für *resourceName* und *resourceClass* in der X11-Ressourcendatenbank gesetzt werden. Die Option "-highlightbackground" wird häufig in "-background" umbenannt, damit sich beim Ändern des Hintergrunds des Megawidgets auch der Hintergrund des Fokusrahmens ändert.

usual ?*tag*?
Sucht die üblichen Befehle zur Optionsbehandlung für den angegebenen Namen *tag* und führt sie aus. Wenn der *tag* nicht angegeben ist, wird der Name der Widget-Klasse als Name *tag* verwendet. Die üblichen Befehle zur Optionsbehandlung werden über den Befehl **usual** registriert.

Wenn das Skript *optionCmds* nicht angegeben ist, werden voreingestellt die üblichen Befehle zur Optionsbehandlung verwendet, die der Klasse des Komponenten-Widgets zugeordnet sind.

itk_component delete *name* ?*name name ...*?
Entfernt das Komponenten-Widget mit dem symbolischen Namen *name* aus dem Megawidget. Das Komponenten-Widget existiert weiter, ist aber nicht mehr als Komponente des Megawidgets zugänglich. Außerdem werden alle der Komponente zugeordneten Optionen aus der zusammengesetzten Optionsliste entfernt. Eine Komponente lässt sich mit dem Befehl **destroy** zerstören, genau wie jedes andere Tk-Widget. Komponenten lösen sich beim Zerstören automatisch von ihrem übergeordneten Megawidget, deshalb wird "**itk_component delete**" selten verwendet.

itk_initialize ?*option value option value...*?
Diese Methode muss im Konstruktor jeder Klasse einer Megawidget-Hierarchie aufgerufen werden. Sie stellt sicher, dass alle Optionen richtig in die zusammengesetzte Optionsliste eingehen, und gleicht alle Komponenten mit den Anfangswerten der Optionen ab. Sie wird gewöhnlich nahe am Ende des Konstruktors aufgerufen, nachdem alle Komponenten-Widgets hinzugefügt wurden. Wenn Paare *option*/*value* angegeben sind, überschreiben sie

Einstellungen aus der X11-Ressourcendatenbank. Die Argumente des Konstruktors werden gewöhnlich so an diese Methode weitergereicht:

```
itcl::class MyWidget {
    inherit Widget

    constructor {args} {
        .
        .
        .
        eval itk_initialize $args
    }
}
```

itk_option add *optName* ?*optName optName ...*?

Fügt der zusammengesetzten Optionsliste eines Megawidgets eine oder mehrere Optionen hinzu. *optName* kann hier eine der folgenden Formen haben:

component*.*option

Greift auf eine *option* zu, die zu einer Komponente mit dem symbolischen Namen *component* gehört. Der Name der *option* wird ohne führendes Zeichen "**-**" angegeben.

className*::*option

Greift auf eine *option* zu, die der Befehl "**itk_option define**" in der Klasse *className* definiert. Der Name der *option* wird ohne führendes Zeichen "**-**" angegeben.

Optionen gehen gewöhnlich in die zusammengesetzte Optionsliste ein, wenn ein Komponenten-Widget zuerst erzeugt wird. Mit dieser Methode lassen sich Optionen zu einem späteren Zeitpunkt hinzufügen. Die Basisklassen **Widget** und **Toplevel** behalten zum Beispiel für ihre Komponente "hull" nur das Nötigste: -background und -cursor. Eine abgeleitete Klasse kann diese Entscheidung aufheben und zusätzlich Optionen hinzufügen, die den Rahmen der Komponente "hull" steuern:

```
itcl::class MyWidget {
    inherit Widget

    constructor {args} {
        itk_option add hull.borderwidth hull.relief

        itk_component add label {
            label $itk_interior.l1 -text "Hello World!"
        }
        pack $itk_component(label)

        eval itk_initialize $args
    }
}
```

itk_option define *switchName resourceName resourceClass init* ?*config*?

Dieser Befehl wird auf der Ebene der Klassendefinition verwendet, um eine synthetische

Option des Megawidgets zu definieren. Innerhalb der Methoden **configure** und **cget** wird diese Option über *switchName* angesprochen, das mit einem Zeichen "**-**" beginnen muss. Sie lässt sich außerdem ändern, indem Werte für *resourceName* und *resourceClass* in der X11-Ressourcendatenbank gesetzt werden. Die Zeichenkette *init* dient als letzter Ausweg, die Option zu initialisieren, wenn kein anderer Wert aus einer vorhandenen Option verwendet oder aus der X11-Ressourcendatenbank abgefragt werden kann. Wenn Code *config* angegeben ist, wird er ausgeführt, sobald die Option über die Methode **configure** geändert wird. Der Code *config* lässt sich auch außerhalb der Klassendefinition über den Befehl **configbody** angeben. Im folgenden Beispiel wird der Klasse eine synthetische Option "-background" hinzugefügt, sodass bei jeder Änderung des Hintergrunds der neue Wert auf die Standardausgabe gemeldet wird. Diese synthetische Option wird mit den übrigen Optionen "-background" zusammengeführt, die von Komponenten-Widgets behalten wurden:

```
itcl::class MyWidget {
    inherit Widget
    constructor {args} {
        itk_component add label {
            label $itk_interior.l1 -text "Hello World!"
        }
        pack $itk_component(label)

        eval itk_initialize $args
    }
    itk_option define -background background Background #d9d9d9 {
        puts "new background: $itk_option(-background)"
    }
}
```

itk_option remove *optName ?optName optName ...?*

Entfernt eine oder mehrere Optionen aus der zusammengesetzten Optionsliste eines Megawidgets. *optName* kann eine der Formen haben, die oben beim Befehl "**itk_option add**" beschrieben sind. Optionen gehen gewöhnlich in die zusammengesetzte Optionsliste ein, wenn ein Komponenten-Widget zuerst erzeugt wird. Mit dieser Methode lassen sich Optionen zu einem späteren Zeitpunkt entfernen. Eine abgeleitete Klasse kann zum Beispiel eine in einer Basisklasse definierte Option aufheben, indem sie die Option entfernt und neu definiert:

```
itcl::class Base {
    inherit itk::Widget
    constructor {args} {
        eval itk_initialize $args
    }

    itk_option define -foo foo Foo "" {
        puts "Base: $itk_option(-foo)"
    }
}
```

```
itcl::class Derived {
    inherit Base

    constructor {args} {
        itk_option remove Base::foo
        eval itk_initialize $args
    }
    itk_option define -foo foo Foo "" {
        puts "Derived: $itk_option(-foo)"
    }
}
```

Ohne den Befehl "**itk_option remove**" würden die Codefragmente beider Optionen "-foo" jedes Mal ausgeführt, wenn die zusammengesetzte Option "-foo" konfiguriert wird. Im Beispiel oben wird die Option \ffCBase::foo in allen Widgets der Klasse Derived unterdrückt, sodass nur die Option \ffCDerived::foo bleibt.

GESCHÜTZTE VARIABLEN

Abgeleitete Klassen finden in den folgenden geschützten Variablen nützliche Angaben.

itk_component(*name*)
: Das Array "itk_component" gibt den echten Fensterpfadnamen eines Komponenten-Widgets mit dem symbolischen Namen *name* zurück. Dieselbe Angabe lässt sich mit der Methode **component** abfragen, der Zugriff auf dieses Array ist aber schneller und bequemer.

itk_interior
: Diese Variable enthält den Namen des Fensters, das als Elternfenster für interne Komponenten dient. Sie wird mit dem Namen der Komponente "hull" initialisiert, den die Klassen **Widget** und **Toplevel** bereitstellen. Abgeleitete Klassen können die Anfangseinstellung überschreiben und auf ein anderes inneres Fenster zeigen, das für weiter abgeleitete Klassen verwendet werden soll.

itk_option(*option*)
: Das Array "itk_option" gibt den aktuellen Optionswert der zusammengesetzten Widget-Option namens *option* zurück. Der Name *option* soll hier ein führendes Zeichen "**-**" enthalten. Dieselbe Angabe lässt sich mit der Methode **cget** abfragen, der Zugriff auf dieses Array ist aber schneller und bequemer.

Copyright (c) 1993-1998 Lucent Technologies, Inc.

Toplevel

Stichwörter: Toplevel, itk, Archetype, Widget, mega-widget

NAME

Toplevel - Basisklasse für Megawidgets in einem Toplevel-Fenster

VERERBUNG

itk::Archetype <- itk::Toplevel

STANDARDOPTIONEN

```
background cursor
```

Einzelheiten zu den Standardoptionen stehen im Handbuchabschnitt "options".

WIDGET-SPEZIFISCHE OPTIONEN

```
Name:   title
Class:  Title
Command-Line Switch:    -title
```

Legt den Titel fest, den der Fenstermanager in der Titelleiste über dem Fenster anzeigt. Der voreingestellte Titel ist die leere Zeichenkette.

BESCHREIBUNG

Die Klasse **Toplevel** erbt alles von der Klasse **Archetype** und fügt ein Tk-Toplevel namens Komponente "hull" hinzu, das den Rumpf des Megawidgets darstellt. Der Name der Fensterklasse des Hull wird auf den spezifischsten Klassennamen des Megawidgets gesetzt. Die geschützte Variable **itk_interior** enthält den Fensterpfadnamen der Komponente "hull". Abgeleitete Klassen spezialisieren dieses Widget, indem sie weitere Widget-Komponenten in den Hull packen.

Da der Hull der Klasse **Toplevel** mit einem Tk-Toplevel umgesetzt ist, haben Megawidgets der Klasse **Toplevel** ein eigenes Toplevel-Fenster. Diese Klasse wird verwendet, um Dialogfenster und andere eingeblendete Fenster zu erzeugen.

KOMPONENTEN

```
Name:           hull
Class:          Toplevel
```

Die Komponente "hull" dient als Rumpf des gesamten Megawidgets. Weitere Komponenten werden in den Hull gepackt, um das Widget weiter zu spezialisieren.

BEISPIEL

Das folgende Beispiel setzt ein Megawidget **MessageInfo** um. Es erzeugt eine eingeblendete Meldung, die der Benutzer mit der Schaltfläche "Dismiss" schließen kann.

```
option add *MessageInfo.title "Notice" widgetDefault

itcl::class MessageInfo {
    inherit itk::Toplevel

    constructor {args} {
        itk_component add dismiss {
            button $itk_interior.dismiss -text "Dismiss" \
                -command "destroy $itk_component(hull)"
        }
        pack $itk_component(dismiss) -side bottom -pady 4

        itk_component add separator {
```

```
            frame $itk_interior.sep -height 2 -borderwidth 1 -relief
    sunken
        }
        pack $itk_component(separator) -side bottom -fill x -padx 4

        itk_component add icon {
            label $itk_interior.icon -bitmap info
        }
        pack $itk_component(icon) -side left -padx 8 -pady 8

        itk_component add infoFrame {
            frame $itk_interior.info
        }
        pack $itk_component(infoFrame) -side left -expand yes \
            -fill both -padx 4 -pady 4

        itk_component add message {
            label $itk_interior.mesg -width 20
        } {
            usual
            rename -text -message message Text
        }
        pack $itk_component(message) -expand yes -fill both

        eval itk_initialize $args

        after idle [code $this centerOnScreen]
    }

    protected method centerOnScreen {} {
        update idletasks
        set wd [winfo reqwidth $itk_component(hull)]
        set ht [winfo reqheight $itk_component(hull)]
        set x [expr ([winfo screenwidth $itk_component(hull)]-$wd)/2]
        set y [expr ([winfo screenheight $itk_component(hull)]-$ht)/2]
        wm geometry $itk_component(hull) +$x+$y
    }
}

itk::usual MessageInfo {
    keep -background -cursor -foreground -font
    keep -activebackground -activeforeground -disabledforeground
    keep -highlightcolor -highlightthickness
}

#
```

```
# BEISPIEL: Ein Hinweisfenster erzeugen:
#
MessageInfo .m -message "File not found:\n/usr/local/bin/foo"
```

Copyright (c) 1993-1998 Lucent Technologies, Inc.

Widget

Stichwörter: Widget, itk, Archetype, mega-widget

NAME

Widget - Basisklasse für Megawidgets innerhalb eines Frames

VERERBUNG

itk::Archetype <- itk::Widget

STANDARDOPTIONEN

```
background cursor
```

Einzelheiten zu den Standardoptionen stehen im Handbuchabschnitt "options".

BESCHREIBUNG

Die Klasse **Widget** erbt alles von der Klasse **Archetype** und fügt einen Tk-Frame namens Komponente "hull" hinzu, der den Rumpf des Megawidgets darstellt. Der Name der Fensterklasse des Hull wird auf den spezifischsten Klassennamen des Megawidgets gesetzt. Die geschützte Variable **itk_interior** enthält den Fensterpfadnamen der Komponente "hull". Abgeleitete Klassen spezialisieren dieses Widget, indem sie weitere Widget-Komponenten in den Hull packen.

Da der Hull der Klasse **Widget** mit einem Tk-Frame umgesetzt ist, lassen sich Megawidgets der Klasse **Widget** in andere Frames und Toplevel-Fenster packen.

KOMPONENTEN

```
Name:           hull
Class:          Frame
```

Die Komponente "hull" dient als Rumpf des gesamten Megawidgets. Weitere Komponenten werden in den Hull gepackt, um das Widget weiter zu spezialisieren.

BEISPIEL

Das folgende Beispiel setzt ein einfaches Megawidget **TextDisplay** um. Es erzeugt eine nur lesbare Anzeige von Text mit einem Text-Widget und einem Rollbalken.

```
option add *TextDisplay.wrap none widgetDefault
option add *TextDisplay.textBackground ivory widgetDefault
option add *TextDisplay.width 40 widgetDefault
option add *TextDisplay.height 10 widgetDefault

itcl::class TextDisplay {
```

```
    inherit itk::Widget

    constructor {args} {
        itk_component add text {
            text $itk_interior.info -state disabled \
                -yscrollcommand [code $itk_interior.sbar set]
        } {
            usual
            keep -tabs -wrap -width -height
            rename -background -textbackground textBackground
    Background
        }
        pack $itk_component(text) -side left -expand yes -fill both

        itk_component add scrollbar {
            scrollbar $itk_interior.sbar \
                -command [code $itk_interior.info yview]
        }
        pack $itk_component(scrollbar) -side right -fill y

        eval itk_initialize $args
    }

    public method display {info}
    public method append {info}
}

itcl::body TextDisplay::display {info} {
    $itk_component(text) configure -state normal
    $itk_component(text) delete 1.0 end
    $itk_component(text) insert 1.0 $info
    $itk_component(text) configure -state disabled
}

itcl::body TextDisplay::append {info} {
    $itk_component(text) configure -state normal
    $itk_component(text) insert end $info
    $itk_component(text) configure -state disabled
}

itk::usual TextDisplay {
    keep -background -cursor -foreground -font
    keep -activebackground -activerelief
    keep -highlightcolor -highlightthickness
    keep -insertbackground -insertborderwidth -insertwidth
    keep -insertontime -insertofftime
```

```
    keep -selectbackground -selectborderwidth -selectforeground
    keep -textbackground -troughcolor
}

#
# BEISPIEL: Die Datei /etc/passwd anzeigen
#
TextDisplay .file -background red
pack .file

.file display [exec cat /etc/passwd]
```

Copyright (c) 1993-1998 Lucent Technologies, Inc.

itk

Stichwörter: itk, class, object, object-oriented, mega-widget

NAME

itk - Rahmen für den Bau von Megawidgets in Tcl/Tk

BESCHREIBUNG

Megawidgets sind Widgets auf hoher Ebene, die aus Tk-Widgets als Komponenten gebaut werden, gewöhnlich ohne C-Code. Eine Dateiauswahlbox kann zum Beispiel einige Listboxen, einige Entry-Widgets und einige Steuerschaltflächen enthalten. Diese einzelnen Widgets werden so zusammengesetzt, dass sie wie ein großes Widget wirken. Ein Megawidget für eine Dateiauswahlbox lässt sich mit einem Befehl wie diesem erzeugen:

```
fileselectionbox .fsb -background blue -foreground white
```

Sobald es erzeugt ist, lässt es sich mit einem Befehl wie diesem neu konfigurieren:

```
.fsb configure -background green -foreground black
```

und alle seine internen Komponenten ändern die Farbe. Jedes Megawidget hat eine Menge von Methoden, mit denen es sich bedienen lässt. Die aktuelle Auswahl lässt sich aus einer Dateiauswahlbox zum Beispiel so abfragen:

```
set fileName [.fsb get]
```

Im Ergebnis sieht ein Megawidget aus und verhält sich genau wie ein Tk-Widget, lässt sich aber erheblich leichter umsetzen.

[incr\ Tk] ist ein Rahmen für den Bau von Megawidgets. Es verwendet **[incr\ Tcl]**, um das Objektparadigma zu unterstützen, und fügt Basisklassen hinzu, die das voreingestellte Widget-Verhalten liefern.

Alle Widgets von **[incr\ Tk]** sind von der Basisklasse **Archetype** abgeleitet. Diese Klasse verwaltet interne Komponenten-Widgets und stellt Methoden wie "configure" und "cget" bereit, mit denen sich auf Konfigurationsoptionen zugreifen lässt.

Die Basisklasse **Widget** erbt alles von **Archetype** und fügt einen Tk-Frame hinzu, der als Container für das Megawidget dient. Sie wird verwendet, um Megawidgets zu bauen, die in

anderen Frames und Toplevel-Fenstern sitzen. Abgeleitete Klassen erzeugen weitere interne Komponenten und packen sie in den Frame "hull", den die Basisklasse **Widget** erzeugt.

Die Basisklasse **Toplevel** erbt alles von **Archetype**, fügt aber ein Tk-Toplevel hinzu, das als Container für das Megawidget dient. Sie wird verwendet, um Megawidgets wie Dialogfenster zu bauen, die ein eigenes Toplevel-Fenster haben. Abgeleitete Klassen erzeugen weitere interne Komponenten und packen sie in das Toplevel "hull", das die Basisklasse **Toplevel** erzeugt.

Bibliothek [incr Widgets]

[incr\ Widgets] ist eine Megawidget-Bibliothek, die mit **[incr\ Tk]** gebaut ist. Sie lässt sich sofort einsetzen und enthält mehr als 30 verschiedene Widget-Klassen, darunter:

- fileselectiondialog
- tabnotebook
- panedwindow
- combobox
- optionmenu
- scrolledlistbox
- scrolledframe
- messagedialog
- und viele weitere ... Die Demo **catalog** im Verzeichnis "iwidgets/demos" zeigt alle verfügbaren Widgets in Aktion. Jede Widget-Klasse hat eine eigene Handbuchseite, die die verfügbaren Merkmale beschreibt.

Copyright (c) 1993-1998 Lucent Technologies, Inc.

itkvars

Stichwörter: itkvars, itcl, itk, variables

NAME

itkvars - Variablen, die [incr\ Tk] verwendet

BESCHREIBUNG

Die folgenden globalen Variablen werden von der Bibliothek **[incr\ Tk]** automatisch erzeugt und verwaltet. Soweit unten nichts anderes vermerkt ist, sollten anwendungsspezifischer Code und Benutzer diese Variablen gewöhnlich als nur lesbar behandeln.

itk::library
: Wenn ein Interpreter erzeugt wird, initialisiert **[incr\ Tk]** diese Variable mit dem Namen eines

Verzeichnisses, das die Systembibliothek der Skripte von **[incr\ Tk]** enthält. Der Anfangswert von **itk::library** wird aus der Umgebungsvariable ITK_LIBRARY gesetzt, falls sie existiert, sonst aus einem einkompilierten Wert. Wenn **[incr\ Tk]** einem Interpreter hinzugefügt wird, führt es das Skript "\ffCinit.itk" in diesem Verzeichnis aus. Dieses Skript sucht seinerseits nach weiteren Skriptdateien mit dem Namen "\ffCinit.*xxx*". Megawidget-Bibliotheken werden automatisch initialisiert, wenn sie in diesem Verzeichnis ein Skript namens "\ffCinit.*xxx*" installieren, wobei "*xxx*" der Name der Megawidget-Bibliothek ist. Die Bibliothek **[incr\ Widgets]** installiert zum Beispiel das Skript "\ffCinit.iwidgets" in diesem Verzeichnis. Dieses Skript richtet den Namespace "iwidgets" ein und bereitet das automatische Laden aller Befehle von **[incr\ Widgets]** vor.

Copyright (c) 1993-1998 Lucent Technologies, Inc.

usual

Stichwörter: usual, itk, Archetype, component, mega-widget

NAME

usual - Zugriff auf die voreingestellten Befehle zur Optionsbehandlung

für eine Megawidget-Komponente

ÜBERSICHT

usual ?*tag*? ?*commands*?

BESCHREIBUNG

Der Befehl **usual** wird außerhalb einer Klassendefinition von **[incr\ Tcl]** verwendet, um die übliche Menge der Befehle zur Optionsbehandlung für ein Komponenten-Widget festzulegen. Befehle zur Optionsbehandlung werden verwendet, wenn eine Komponente über die Methode "**itk_component add**" bei der Basisklasse **Archetype** registriert wird. Sie legen fest, wie die Konfigurationsoptionen der Komponente in die zusammengesetzte Optionsliste des Megawidgets eingehen. Optionen können behalten, umbenannt oder ignoriert werden, wie auf der Handbuchseite von **Archetype** beschrieben.

Es ist mühsam, dieselben Deklarationen jedes Mal erneut einzufügen, wenn Komponenten hinzugefügt werden. Der Befehl **usual** erlaubt es, für jede Widget-Klasse ein Standard-Codefragment zu registrieren, das voreingestellt die Optionen behandelt. Alle Standard-Widgets von Tk haben **usual**-Deklarationen, die in der Bibliothek **[incr\ Tk]** definiert sind. Ähnliche **usual**-Deklarationen sollten angelegt werden, sobald eine neue Megawidget-Klasse entworfen wird. In die Deklaration **usual** gehören nur die allgemeinsten Optionen.

Der Name *tag* ist gewöhnlich der Name einer Widget-Klasse und beginnt mit einem Großbuchstaben; jede hier registrierte Zeichenkette lässt sich später jedoch mit dem Befehl **usual** verwenden, der auf der Handbuchseite von **Archetype** beschrieben ist.

Wenn das Argument *commands* angegeben ist, wird es der Zeichenkette *tag* zugeordnet und lässt sich später über **itk_component add** abrufen.

Wenn nur das Argument *tag* angegeben ist, sucht dieser Befehl einen vorhandenen Namen *tag* und gibt die ihm zugeordneten Befehle zurück. Wenn dem Namen *tag* keine Befehle zugeordnet sind, gibt dieser Befehl die leere Zeichenkette zurück.

Wenn keine Argumente angegeben sind, gibt dieser Befehl eine Liste aller zuvor registrierten Namen *tag* zurück.

BEISPIEL

Es folgt die Deklaration **usual** für das Standard-Widget Button von Tk:

```
itk::usual Button {
    keep -background -cursor -foreground -font
    keep -activebackground -activeforeground -disabledforeground
    keep -highlightcolor -highlightthickness
    rename -highlightbackground -background background Background
}
```

Behalten oder umbenannt werden nur die Optionen, die allen Schaltflächen in einem einzelnen Megawidget gemeinsam wären. Optionen wie "-text", die einer bestimmten Schaltfläche eigen wären, werden ignoriert.

Copyright (c) 1993-1998 Lucent Technologies, Inc.

mclistbox

mclistbox

Stichwörter: mclistbox

NAME

mclistbox::mclistbox - Eine mehrspaltige Listbox anlegen und bedienen

ÜBERSICHT

package require mclistbox ?1.02

mclistbox::mclistbox *pathName?options?*

EXPORTE

mclistbox

STANDARDOPTIONEN

-background -borderwidth -cursor -exportselection -font -foreground -height -highlightbackground -highlightcolor -highlightthickness -relief -selectbackground -selectborderwidth -selectforeground -setgrid -takefocus -width -xscrollcommand -yscrollcommand

Die ausführliche Beschreibung der Optionen oben steht im Handbucheintrag *options*.

WIDGET-SPEZIFISCHE OPTIONEN

Befehlszeilen-Name: **-columnborderwidth** Datenbank-Name: **columnBorderWidth** Datenbank-Klasse: **BorderWidth**

Legt einen nicht negativen Wert für die Breite des 3D-Rands um die Außenseite der Spalte fest, sofern ein solcher Rand gezeichnet wird. Das bestimmt in der Regel die Option **columnrelief**. Der Wert kann auch für 3D-Effekte im Inneren des Widgets verwendet werden. Zulässig sind alle Formen, die Tk_GetPixels annimmt.

Befehlszeilen-Name: **-columnrelief** Datenbank-Name: **columnRelief** Datenbank-Klasse: **Relief**

Legt den gewünschten 3D-Effekt der Spalte fest. Zulässige Werte sind raised, **sunken**, **flat**, **ridge**, **solid** und **groove**. Der Wert sagt, wie das Innere der Spalte gegenüber dem Äußeren erscheinen soll. raised bedeutet zum Beispiel, dass das Innere der Spalte gegenüber dem Äußeren aus dem Bildschirm hervortritt.

Befehlszeilen-Name: **-fillcolumn** Datenbank-Name: **fillColumn** Datenbank-Klasse: **FillColumn**

Legt fest, welche Spalte wächst oder schrumpft, sodass alle Spalten das Listbox-Widget genau füllen.

Befehlszeilen-Name: **-labelanchor** Datenbank-Name: **labelAnchor** Datenbank-Klasse: **Anchor**

Legt fest, wie der Inhalt der Spaltenbeschriftung angezeigt wird. Einer der Werte **n**, **ne**, **e**, **se**, **s**, **sw**, **w**, **nw** oder **center**. nw zeigt den Inhalt so, dass seine linke obere Ecke in der linken oberen Ecke der Spaltenbeschriftung liegt.

Befehlszeilen-Name: **-labelbackground** oder **-labelbg** Datenbank-Name: **labelBackground** Datenbank-Klasse: **Background**

Legt die normale Hintergrundfarbe der Beschriftung fest.

Befehlszeilen-Name: **-labelborderwidth or -labelbd** Datenbank-Name: **labelBorderWidth** Datenbank-Klasse: **BorderWidth**

Legt einen nicht negativen Wert für die Breite des 3D-Rands um die Außenseite der Spaltenbeschriftung fest, sofern ein solcher Rand gezeichnet wird. Das bestimmt in der Regel die Option relief. Der Wert kann auch für 3D-Effekte im Inneren der Spaltenbeschriftung verwendet werden. Zulässig sind alle Formen, die Tk_GetPixels annimmt.

Befehlszeilen-Name: **-labelfont** Datenbank-Name: **labelFont** Datenbank-Klasse: **Font**

Legt die Schrift fest, mit der Text in der Spaltenbeschriftung gezeichnet wird.

Befehlszeilen-Name: **-labelforeground** Datenbank-Name: **labelForeground** Datenbank-Klasse: **Foreground**

Legt die normale Vordergrundfarbe der Spaltenbeschriftung fest.

Befehlszeilen-Name: **-labelheight** Datenbank-Name: **labelHeight** Datenbank-Klasse: **Height**

Legt die gewünschte Höhe der Beschriftung fest. Wird in der Beschriftung ein Bild oder eine Bitmap angezeigt, gilt der Wert in Bildschirmeinheiten, also in jeder Form, die Tk_GetPixels annimmt. Für Text gilt er in Textzeilen. Fehlt die Option, wird die gewünschte Höhe aus der Größe des angezeigten Bildes, der Bitmap oder des Textes berechnet.

Befehlszeilen-Name: **-labelrelief** Datenbank-Name: **labelRelief** Datenbank-Klasse: **Relief**

Legt den gewünschten 3D-Effekt der Spaltenbeschriftung fest. Zulässige Werte sind **raised**, **sunken**, **flat**, **ridge**, **solid** und **groove**. Der Wert sagt, wie das Innere der Spaltenbeschriftung gegenüber dem Äußeren erscheinen soll. raised bedeutet zum Beispiel, dass das Innere der Spaltenbeschriftung gegenüber dem Äußeren aus dem Bildschirm hervortritt.

Befehlszeilen-Name: **-labels** Datenbank-Name: **labels** Datenbank-Klasse: **Labels**

Ein boolescher Wert. Er legt fest, ob Spaltenbeschriftungen angezeigt werden.

Befehlszeilen-Name: **-selectcommand** *command* Datenbank-Name: **selectCommand** Datenbank-Klasse: **Command**

Legt einen Tcl-Befehl fest, der läuft, sobald sich die Auswahl in der mclistbox ändert. Der Befehl

wird mit dem Ergebnis des Befehls curselection aufgerufen.

Befehlszeilen-Name: **-resizablecolumns** Datenbank-Name: **resizableColumns** Datenbank-Klasse: **ResizableColumns**

Legt fest, ob die Spalten interaktiv in der Größe geändert werden können. Ist der Wert true, werden Mausbindungen definiert, die das Ändern erlauben. Mehr dazu unter **VORGABEBINDUNGEN**.

Befehlszeilen-Name: **-selectmode** Datenbank-Name: **selectMode** Datenbank-Klasse: **SelectMode**

Legt einen von mehreren Stilen für die Auswahl fest. Der Wert der Option darf beliebig sein. Die Vorgabebindungen erwarten **single**, **browse**, **multiple** oder **extended**. Der Vorgabewert ist browse.

BESCHREIBUNG

Der Befehl **mclistbox** erzeugt ein neues Fenster, gegeben durch das Argument *pathName*, und macht es zu einem mclistbox-Widget. Weitere Optionen, oben beschrieben, können auf der Befehlszeile Aspekte der mclistbox festlegen, etwa Farben, Schrift, Text und Relief. Der Befehl mclistbox gibt sein Argument pathName zurück. Beim Aufruf darf noch kein Fenster namens pathName bestehen. Das Elternfenster von pathName muss bestehen.

Eine mclistbox zeigt eine Liste von Zeichenketten, eine je Zeile. Eine neue mclistbox hat anfangs keine Elemente und keine Spalten. Spalten und Elemente werden mit den Widget-Befehlen unten hinzugefügt oder gelöscht. Ein oder mehrere Elemente lassen sich auswählen, wie unten beschrieben. Wird ein Element in einer Spalte ausgewählt, werden die Elemente in allen Spalten ausgewählt.

Nicht alle Elemente müssen gleichzeitig im Fenster der mclistbox sichtbar sein. Die Befehle unten ändern die Sicht im Fenster. Mclistboxen lassen sich in beide Richtungen schieben, über die Standardoptionen xScrollCommand und yScrollCommand. Sie unterstützen auch das Scannen, wie unten beschrieben.

WIDGET-BEFEHL

Der Widget-Befehl nimmt weitgehend dieselben Argumente an wie der Befehl der gewöhnlichen Listbox. Die folgenden Befehle verhalten sich genau wie die Befehle der gewöhnlichen Listbox:

activate bbox cget configure curselection delete index nearest scan see selection size xview yview

Dazu kommen einige Befehle, die es nur bei der mclistbox gibt:

pathName column option args

pathName column add name options

Fügt eine neue Spalte mit dem angegebenen *name* hinzu. *options* dürfen alle gültigen Spaltenoptionen sein.

pathName column cget name option

Gibt den Wert der genannten *option* zurück.

pathName column configure name options

Konfiguriert Attribute der genannten Spalte oder fragt sie ab.

Spalten lassen sich mit den Standardoptionen der Listbox konfigurieren (-background, -font und

so weiter). Zusätzlich stehen diese Optionen zur Verfügung:

-background
Legt die Hintergrundfarbe dieser Spalte fest. Ist der Wert die leere Zeichenkette, erhält die Spalte die voreingestellte Hintergrundfarbe.

-bitmap

Legt eine Bitmap fest, die in der Spaltenbeschriftung angezeigt wird. Ist die Option bitmap angegeben, verdrängt sie die Option -label. Die Option bitmap lässt sich auf die leere Zeichenkette zurücksetzen, damit wieder Text angezeigt wird.

-font

Legt die Schrift dieser Spalte fest. Ist der Wert die leere Zeichenkette, erhält die Spalte die voreingestellte Schrift. Werden für getrennte Spalten Schriften mit verschiedener Zeilenhöhe verwendet, stehen die Daten der Spalten nicht auf einer Linie.

-foreground

Legt die Vordergrundfarbe dieser Spalte fest. Ist der Wert die leere Zeichenkette, erhält die Spalte die voreingestellte Vordergrundfarbe.

-image

Legt ein Bild fest, das in der Spaltenbeschriftung angezeigt wird. Ist die Option image angegeben, verdrängt sie andere Optionen, die eine Bitmap oder einen Text für die Spaltenbeschriftung nennen. Die Option image lässt sich auf die leere Zeichenkette zurücksetzen, damit wieder eine Bitmap oder Text angezeigt wird.

-label

Legt den Text fest, der in der Spaltenbeschriftung stehen soll.

-position

Legt die Position der Spalte fest. 0 (null) ist die erste Spalte, 1 (eins) die zweite, und so weiter.

-resizable

Legt fest, ob die Spalte in der Größe geändert werden kann. Das wirkt nur, wenn die globale Option **-resizablecolumns** true ist.

-visible

Legt fest, ob die Spalte sichtbar ist.

-width

Legt die gewünschte Breite der Spalte in Zeichen fest. Hat die Schrift keine einheitliche Breite, wird die Breite des Zeichens ``0'' verwendet, um von Zeicheneinheiten auf Bildschirmeinheiten umzurechnen. Ist der Wert null oder kleiner, wird die gewünschte Breite des Fensters gerade so groß, dass alle Elemente der Listbox hineinpassen.

pathName column delete name

Löscht die Spalte mit dem Namen *name*.

pathName column names

Gibt eine Liste aller Spalten-*names* zurück. Die Reihenfolge der Liste ist die sichtbare Reihenfolge der Spalten und dieselbe Reihenfolge, in der der Befehl get die Daten zurückgibt.

pathName column nearext x

Zu einer x-Koordinate im Fenster der mclistbox gibt dieser Befehl den Namen der Spalte zurück, die dieser x-Koordinate am nächsten liegt.

pathName get first ?last?

Fehlt last, kommt eine Liste zurück mit dem Element jeder Spalte des Listbox-Elements, das *first* bezeichnet, oder eine leere Liste, wenn first auf ein nicht vorhandenes Element zeigt. Ist *last* angegeben, gibt der Befehl eine Liste von Listen zurück. Jede Teilliste enthält ein Element jeder Spalte des mclistbox-Elements. first und last dürfen jede der üblichen Indexformen haben.

Die Daten kommen in der sichtbaren Reihenfolge der Spalten zurück, derselben Reihenfolge, die der Befehl **column names** meldet.

pathName insert index list ?list ...?

Fügt null oder mehr neue Elemente in die mclistbox ein, unmittelbar vor dem Element, das index bezeichnet. Ist index als end angegeben, kommen die neuen Elemente an das Ende der Liste. Der Befehl gibt die leere Zeichenkette zurück. Jedes Argument *list* wird eine Zeile der mclistbox. Reichen die Elemente nicht für alle Spalten, füllen leere Werte die fehlenden Spalten.

pathName label bind name sequence command

Dieser Befehl verbindet *command* mit einer Spaltenbeschriftung. Tritt die Ereignisfolge *sequence* über der Beschriftung der Spalte *name* ein, wird *command* aufgerufen. Der Widget-Befehl ähnelt dem Befehl bind, wirkt aber auf Spaltenbeschriftungen und nicht auf das ganze Widget. Der Handbucheintrag bind beschreibt die Syntax von *sequence* und die Ersetzungen in *command* vor dem Aufruf.

BEISPIEL

Das folgende Beispiel erzeugt eine Listbox mit drei Spalten:

```
    mclistbox .listbox -width 55
```

```
    .listbox column add name -label "First Name"    -width 20
    .listbox column add size -label "Last Name"     -width 20
    .listbox column add mod  -label "Organization"  -width 15
```

Einträge kommen so in die Liste:

```
    .listbox insert end [list "Bryan" "Oakley" "ChannelPoint"]
    .listbox insert end [list "John"  "Ousterhout" "Ajuba Solutions"]
```

... oder kürzer:

```
    .listbox insert end \
            [list "Bryan" "Oakley" "ChannelPoint"] \
            [list "John"  "Ousterhout" "Ajuba Solutions"]
```

Damit eine Prozedur aufgerufen wird, wenn auf die Spaltenbeschriftung der Spalte ``date'' geklickt wird, dient ``label bind'' wie im folgenden Beispiel:

```
    .listbox label bind date <ButtonPress-1> "sortByDate %W"
```

```
    proc sortByDate {w} {
        # code to sort the contents of the mclistbox named $w
    }
```

VORGABEBINDUNGEN

Die Bindungen des mclistbox-Widgets sind dieselben wie bei einem gewöhnlichen Listbox-Widget. Ist **-resizableColumns** true, kommen diese Bindungen dazu:

ButtonPress-1 beginnt eine Größenänderung der Spalte, wenn der Zeiger über einem Spaltenkopf und wenige Pixel vom linken oder rechten Rand der Beschriftung entfernt ist. **B1-Motion** ändert die Spalten beiderseits des Zeigers entsprechend der Bewegung des Zeigers. **ButtonRelease-1** beendet die Größenänderung.

Ist über **label bind** eine Bindung auf einer Beschriftung gesetzt und **-resizablecolumns** ist gesetzt, feuert die Bindung nicht, wenn das Ereignis in den wenigen Pixeln am linken und rechten Rand der Beschriftung liegt, in denen eine Größenänderung möglich ist. So lösen eine Beschriftungsbindung und eine Größenänderung nicht denselben Tastendruck aus.

FEHLER

Werden für verschiedene Spalten verschiedene Schriften verwendet, stehen die Daten nicht auf einer Linie, wenn diese Schriften nicht dieselbe Zeilenhöhe haben. Derzeit hilft nur, Schriften mit derselben Zeilenhöhe zu verwenden.

Beschriftungen und Bilder gemischt in den Spaltenköpfen ergeben ungleich hohe Köpfe. Dadurch stehen die Daten in den Spalten nicht auf einer Linie.

Ereignisbindungen, die feuern, während der Mauszeiger über einer Spalte ist, liefern falsche Werte für %W, %x und %y. Die Werte beziehen sich auf die internen Widgets unter dem Klick und nicht auf das mclistbox-Widget. In Tk gibt es dafür bis 8.1b2 keinen Ausweg. Ein Wert zum Schlüsselwort %W lässt sich mit der Funktion ::mclistbox::convert umrechnen. Eine eigene Beschreibung fehlt. Das Beispiel example.tcl dieser Verteilung zeigt den Aufruf.

Der Befehl ``<pathName> xview index'' funktioniert nicht.

Weitere sind sicher noch da.

mpexpr

mpexpr

Stichwörter: mpexpr

Inhaltsverzeichnis

NAME

mpexpr - Einen Ausdruck mit Mehrfachgenauigkeit auswerten

ÜBERSICHT

package require Mpexpr
mpexpr arg ?*arg arg ...* ?
mpformat formatString ?*arg arg ...* ?
global mp_precision

BESCHREIBUNG

Mpexpr beruht auf dem Tcl-Befehl *expr* und hat mit *expr* viel gemeinsam. Alle Rechnungen laufen über ein Paket für Arithmetik mit beliebiger Genauigkeit.

Mpexpr hängt die Argumente *arg* aneinander, mit Leerzeichen dazwischen, wertet das Ergebnis als Tcl-Ausdruck aus und gibt den Wert zurück. Die Operatoren in Tcl-Ausdrücken sind eine Teilmenge der Operatoren in C-Ausdrücken, mit derselben Bedeutung und derselben Vorrangstufe. Ausdrücke liefern fast immer Zahlen, ganze Zahlen oder Gleitkommawerte. Zum Beispiel der Ausdruck

mpexpr 8.2 + 6

ergibt 14.2. Tcl-Ausdrücke unterscheiden sich von C-Ausdrücken darin, wie Operanden angegeben werden. Tcl-Ausdrücke erlauben außerdem Operanden, die keine Zahlen sind, und Vergleiche von Zeichenketten.

OPERANDEN

Ein Tcl-Ausdruck besteht aus Operanden, Operatoren und Klammern. Zwischen Operanden, Operatoren und Klammern darf Leerraum stehen. Der Ausdrucksprozessor ignoriert ihn. Wo es geht, werden Operanden als ganze Zahlen gelesen. Ganze Zahlen stehen dezimal, das ist der Normalfall, oktal, wenn das erste Zeichen des Operanden **0** ist, oder hexadezimal, wenn die ersten beiden Zeichen **0x** sind. Passt keines dieser Formate, wird der Operand als Gleitkommazahl gelesen, sofern das möglich ist. Gleitkommazahlen dürfen in jeder Form stehen, die ein ANSI-C-Compiler annimmt. Die Endungen ``f'', ``F'', ``l'' und ``L'' sind in den meisten Installationen nicht zulässig. Gültige Gleitkommazahlen sind zum Beispiel 2.1, 3., 6e4 und 7.91e+16. Ist keine numerische Lesart möglich, bleibt der Operand eine Zeichenkette, und nur wenige Operatoren gelten für ihn.

Operanden lassen sich auf eine der folgenden Arten angeben:

1. Als Zahl, ganze Zahl oder Gleitkommazahl.
2. Als Tcl-Variable in der üblichen Schreibweise **$**. Der Wert der Variablen ist der Operand.
3. Als Zeichenkette in doppelten Anführungszeichen. Der Ausdrucksprozessor führt zwischen den Anführungszeichen Backslash-, Variablen- und Befehlsersetzung aus und nimmt den entstandenen Wert als Operanden.
4. Als Zeichenkette in geschweiften Klammern. Die Zeichen zwischen der öffnenden Klammer und der passenden schließenden Klammer sind der Operand, ohne Ersetzung.
5. Als Tcl-Befehl in eckigen Klammern. Der Befehl wird ausgeführt, sein Ergebnis ist der Operand.
6. Als mathematische Funktion, deren Argumente eine der genannten Operandenformen haben, zum Beispiel ``**sin($x)** ''. Die definierten Funktionen stehen unten.

Wo oben Ersetzungen vorkommen, etwa in Zeichenketten in Anführungszeichen, führt sie der Ausdrucksprozessor aus. Eine weitere Ersetzung kann der Befehlsparser schon ausgeführt haben, bevor der Ausdrucksprozessor aufgerufen wird. Wie unten beschrieben, gehören Ausdrücke in der Regel in geschweifte Klammern, damit der Befehlsparser den Inhalt nicht ersetzt.

Für einfache Beispiele habe die Variable **a** den Wert 3 und die Variable **b** den Wert 6. Der Befehl links in den folgenden Zeilen liefert den Wert der nächsten Zeile:

mpexpr 3.1 + $a
6.1

mpexpr 2 + "$a.$b"
5.6
mpexpr 4*[llength "6 2"]
8
mpexpr {{word one} < "word $a"}
0

OPERATOREN

Die gültigen Operatoren stehen unten, nach abnehmender Vorrangstufe:

- + ~!
Unäres Minus, unäres Plus, bitweises NOT, logisches NOT. Keiner dieser Operatoren gilt für Zeichenketten, und bitweises NOT gilt nur für ganze Zahlen.

*** / %**
Multiplizieren, dividieren, Rest. Keiner dieser Operatoren gilt für Zeichenketten, und der Rest gilt nur für ganze Zahlen. Der Rest hat immer das Vorzeichen des Divisors und einen Absolutbetrag, der kleiner ist als der Divisor.

+ -
Addieren und subtrahieren. Gilt für alle numerischen Operanden.

<< >>
Schieben nach links und nach rechts. Gilt nur für ganzzahlige Operanden. Ganze Zahlen in mpexpr sind nicht auf ein Maschinenwort beschränkt und stehen nicht im Zweierkomplement. Beim Schieben gibt es daher kein Vorzeichenbit.

< > <= >=
Boolesches Kleiner, Größer, Kleiner-oder-gleich und Größer-oder-gleich. Jeder Operator liefert 1, wenn die Bedingung wahr ist, sonst 0. Die Operatoren gelten für Zeichenketten und für Zahlen. Bei Zeichenketten wird als Zeichenkette verglichen.

==!=
Boolesches Gleich und Ungleich. Jeder Operator liefert null oder eins. Gilt für alle Arten von Operanden.

&
Bitweises UND. Gilt nur für ganzzahlige Operanden.

^
Bitweises exklusives ODER. Gilt nur für ganzzahlige Operanden.

|
Bitweises ODER. Gilt nur für ganzzahlige Operanden.

&&
Logisches UND. Liefert 1, wenn beide Operanden ungleich null sind, sonst 0. Gilt nur für numerische Operanden, ganze Zahlen oder Gleitkommazahlen.

||
Logisches ODER. Liefert 0, wenn beide Operanden null sind, sonst 1. Gilt nur für numerische Operanden, ganze Zahlen oder Gleitkommazahlen.

x?y:z
Wenn-dann-sonst, wie in C. Wertet *x* zu ungleich null aus, ist das Ergebnis der Wert von *y*. Sonst ist das Ergebnis der Wert von *z*. Der Operand *x* muss eine Zahl sein.

Das C-Handbuch beschreibt die Ergebnisse der Operatoren genauer. Alle zweistelligen Operatoren derselben Vorrangstufe binden von links nach rechts. Zum Beispiel der Befehl

mpexpr 4*2 < 7

liefert 0.

Die Operatoren **&&**, **||** und**?:** haben wie in C eine ``verzögerte Auswertung''. Operanden werden nicht ausgewertet, wenn sie für das Ergebnis nicht nötig sind. Zum Beispiel im Befehl

mpexpr {$v ? [a]: [b]}

wird nur eines von **[a]** oder **[b]** wirklich ausgewertet, je nach dem Wert von **$v**. Das gilt nur, wenn der ganze Ausdruck in geschweiften Klammern steht. Sonst wertet der Tcl-Parser **[a]** und **[b]** aus, bevor er den Befehl **expr** aufruft.

MATHEMATISCHE FUNKTIONEN

Mpexpr kennt in Ausdrücken die folgenden mathematischen Funktionen. *x* und *y* sind ganze Zahlen oder Gleitkommawerte. *i*, *j* und *c* sind ganze Zahlen.

Mathematische Funktionen, die mit *expr* übereinstimmen:

acos(x)
Arkuskosinus von *x*.

asin(x)
Arkussinus von *x*.

atan(x)
Arkustangens von *x*.

atan2(x,y)
Arkustangens von *x* / *y*.

ceil(x)
Kleinste ganze Zahl, die größer oder gleich *x* ist.

cos(x)
Kosinus von *x*.

cosh(x)
Hyperbelkosinus von *x*.

exp(x)
Exponentialfunktion e ** *x*.

floor(x)
Größte ganze Zahl, die kleiner oder gleich *x* ist.

fmod(x,y)
Rest von *x* geteilt durch *y*.

hypot(x,y)
Euklidischer Abstand sqrt(*x* * *x* + *y* * *y*).

log(x)
Natürlicher Logarithmus von *x*.

log10(x)
Zehnerlogarithmus von *x*.

pow(x,y)
x hoch *y*.

sin(x)
Sinus von *x*.

sinh(x)
Hyperbelsinus von *x*.

sqrt(x)
Quadratwurzel von *x*.

tan(x)
Tangens von *x*.

tanh(x)
Hyperbeltangens von *x*.

abs(x)
Gibt den Absolutbetrag von *x* zurück. *x* darf eine ganze Zahl oder eine Gleitkommazahl sein. Das Ergebnis hat dieselbe Form.

double(x)
Ist *x* eine Gleitkommazahl, kommt *x* zurück. Sonst wird *x* in eine Gleitkommazahl umgewandelt und dieser Wert zurückgegeben.

int(x)
Ist *x* eine ganze Zahl, kommt *x* zurück. Sonst wird *x* durch Abschneiden in eine ganze Zahl umgewandelt und dieser Wert zurückgegeben.

round(x)
Ist *x* eine ganze Zahl, kommt *x* zurück. Sonst wird *x* durch Runden in eine ganze Zahl umgewandelt und dieser Wert zurückgegeben.

Weitere Funktionen von *mpexpr*:

root(x,y)
Die *y*-te Wurzel von *x*.

frem(x,y)
Entfernt jedes Vorkommen des Faktors *y* aus der Zahl *x*.

minv(x,y)
Inverses von *x* modulo *y*.

gcd(x,y)
Größter gemeinsamer Teiler von *x* und *y*.

lcm(x,y)
Kleinstes gemeinsames Vielfaches von *x* und *y*.

max(x,y)
Maximum von *x* und *y*.

min(x,y)
Minimum von *x* und *y*.

pi()
Wert von pi.

fib(i)
Fibonacci-Zahl der ganzen Zahl *i*.

fact(i)
Fakultät der ganzen Zahl *i*.

pfact(i)
Produkt der Primzahlen bis zur ganzen Zahl *i*.

lfactor(i,c)
Kleinster Primfaktor der ganzen Zahl *i*, mit *c* Versuchen.

iroot(i,j)
Ganzzahlige *j*-te Wurzel der ganzen Zahl *i*.

gcdrem(i,j)
Relativ prim zum größten gemeinsamen Teiler von *i*, geteilt durch *j*.

perm(i,j)
Permutationen von *i* zu je *j*: *i*! / (*i* - *j*)!.

comb(i,j)
Kombinationen von *i* zu je *j*: *i*! / (*j*! * (*i* - *j*)!).

prime(i,c)
Gibt 0 zurück, wenn *i* nicht prim ist, und 1, wenn *i* wahrscheinlich prim ist. Der Primzahltest läuft *c*-mal. Die Chance, dass eine Nicht-Primzahl den Test besteht, ist kleiner als (1/4)^count. Bei einem Zähler von 100 besteht eine Nicht-Primzahl den Test nur in 1 von 10^60 Fällen.

relprime(i,j)
Gibt 1 zurück, wenn *i* und *j* teilerfremd sind, sonst 0.

TYPEN, ÜBERLAUF UND GENAUIGKEIT

Gerechnet wird mit Festkomma- und Gleitkommawerten beliebiger Größe. Native Maschinenwerte (*int, long, IEEE 754 Gleitkomma und so weiter*) und Maschinenbefehle werden nicht verwendet. Die Umwandlung zwischen den internen Darstellungen für ganze Zahlen, Gleitkomma und Zeichenketten geschieht bei Bedarf von selbst. In arithmetischen Rechnungen bleiben die Werte ganzzahlig, bis eine Gleitkommazahl vorkommt. Danach wird mit Gleitkomma gerechnet. Zum Beispiel

mpexpr 5 / 4

liefert 1, während

mpexpr 5 / 4.0
mpexpr 5 / ([string length "abcd"] + 0.0)

liefern beide 1.25. Gleitkommawerte kommen immer mit einem ``.'' oder einem ``e'' zurück, damit sie nicht wie ganze Zahlen aussehen. Zum Beispiel

mpexpr 20.0/5.0

liefert ``4.0'' und nicht ``4''.

Die globale Variable **mp_precision** legt fest, wie viele signifikante Stellen bei der Auswertung bleiben. Ist **mp_precision** nicht gesetzt, werden 17 Stellen verwendet. Der größte Wert von **mp_precision** ist 10000. Größere Werte von **mp_precision** brauchen zunehmend längere Laufzeiten. Ein unzulässiger Wert von **mp_precision** erzeugt einen Fehler.

ZEICHENKETTENOPERATIONEN

Zeichenketten dürfen Operanden der Vergleichsoperatoren sein. Der Auswerter vergleicht trotzdem ganzzahlig oder als Gleitkomma, wo er kann. Ist ein Operand eines Vergleichs eine Zeichenkette und der andere eine Zahl, wird die Zahl mit dem *sprintf*-Format **%d** (ganze Zahlen) bzw. **%g** (Gleitkommawerte) zurück in eine Zeichenkette gewandelt. Zum Beispiel die Befehle

mpexpr {"0x03" > "2"} mpexpr {"0y" < "0x12"}

liefern beide 1. Der erste Vergleich ist ein Vergleich ganzer Zahlen. Der zweite ist ein Zeichenkettenvergleich, nachdem der zweite Operand in die Zeichenkette ``18'' gewandelt wurde. Tcl behandelt Werte, wo es geht, als Zahlen. Operatoren wie **==** sind deshalb ungeeignet, wenn wirklich Zeichenketten verglichen werden sollen und die Operanden beliebige Werte haben können. Dann ist der Befehl **string compare** die bessere Wahl.

mpformat formatiert eine Zeichenkette im Stil des Tcl-Befehls *format*. Mpformat liest numerische Argumente als Zahlen beliebiger Genauigkeit. Mpformat kennt nur einen Teil der %-Angaben. Angegeben werden kann:

% [-] [width[.precision]] formatChar

-
Legt linksbündig fest. Voreingestellt ist rechtsbündig.

width.precision
Legt die optionale Breite und Genauigkeit fest. Die voreingestellte Genauigkeit ist 8. Breite und Genauigkeit dürfen als * stehen. Dann liefert das nächste Argument den Wert für Breite oder Genauigkeit.

Formatzeichen und Ergebnis

d
Formatiert das nächste Argument als ganze Zahl und schneidet nach dem Dezimalpunkt ab.

f
Formatiert das nächste Argument als dezimale Gleitkommazahl.

e
Formatiert das nächste Argument in wissenschaftlicher Schreibweise.

r, R
Formatiert das nächste Argument als rationalen Bruch x / y.

N
Formatiert das nächste Argument nur als Zähler des rationalen Bruchs x / y.

D
Formatiert das nächste Argument nur als Nenner des rationalen Bruchs x / y.

o
Formatiert das nächste Argument oktal, mit führender '0'. Ein Gleitkommaargument wird als oktaler rationaler Bruch x / y formatiert.

x
Formatiert das nächste Argument hexadezimal, mit führendem '0x'. Ein Gleitkommaargument wird als hexadezimaler rationaler Bruch x / y formatiert.

b
Formatiert das nächste Argument binär, mit führendem '0b'. Ein Gleitkommaargument wird als binärer rationaler Bruch x / y formatiert.

s
Formatiert das nächste Argument als Zeichenkette.

c
Formatiert das nächste Argument als einzelnes Zeichen.

%

Formatiert ein einzelnes wörtliches %.

Weitere Zeichen in der Formatzeichenkette

\n
Formatiert einen ASCII-Zeilenumbruch.

\r
Formatiert einen ASCII-Wagenrücklauf.

\t
Formatiert einen ASCII-Tabulator.

\f
Formatiert einen ASCII-Seitenvorschub.

\v
Formatiert einen vertikalen ASCII-Tabulator.

\b
Formatiert ein ASCII-Rückschrittzeichen.

ANMERKUNGEN

Mpexpr beruht auf 'tclExpr.c' aus Tcl 7.6 und auf dem Programm 'Calc' von David Bell. Diese Handbuchseite und die Testsuite von mpexpr sind zum großen Teil ebenfalls aus Tcl 7.6 übernommen.

Weitere Angaben stehen in den Dateien README und INSTALL.

Tcl 7.6 steht unter Copyright (c) 1987-1994 The Regents of the University of California und Copyright (c) 1994 Sun Microsystems, Inc.

Calc steht unter Copyright (c) 1994 David I. Bell.

AUTOR

Tom Poindexter, tpoindex@nyx.net, Talus Technologies, Inc., Highlands Ranch, CO.
http://www.nyx.net/~tpoindex

Version 1.0 erschien im November 1998.

Copyright 1998 Tom Poindexter. Weitere Copyright- und Lizenzbedingungen stehen in der Datei 'LICENSE.TERMS'.

Inhaltsverzeichnis

- NAME
- ÜBERSICHT
- BESCHREIBUNG
- OPERANDEN
- OPERATOREN
- MATHEMATISCHE FUNKTIONEN
- TYPEN, ÜBERLAUF UND GENAUIGKEIT
- ZEICHENKETTENOPERATIONEN
- ANMERKUNGEN
- AUTOR

tclhttpd

httpd

Stichwörter: httpd, Web Server, HTTP, TclHttpd

NAME

tclhttpd - Tcl-Webserver

ÜBERSICHT

tclsh httpd.tcl ?*options*?

OPTIONEN

-help
Zeigt die Aufrufhilfe an und beendet sich, ohne etwas zu tun.

-config *filename*
Name der Konfigurationsdatei (zum Beispiel tclpro/bin/tclhttpd.rc).

-main *filename*
Name des Haupt-Skripts je Thread (zum Beispiel tclpro/bin/httpdthread.tcl).

-docRoot *directory*
Wurzelverzeichnis der Webseiten (zum Beispiel tclpro/tclhttpd/htdocs).

-port *value*
HTTP-Port, an dem der Server lauscht. Voreingestellt 8015.

-host *value*
Rechnername des HTTP-Sockets, an dem der Server lauscht.

-ipaddr *value*
Schnittstelle, an die der Server bindet.

-webmaster *email*
E-Mail-Adresse des Webmasters.

-uid *userid*
Benutzername oder Kennung für die Benutzerkennung des Serverprozesses.

-gid *groupid*
Gruppenname oder Kennung für die Gruppenkennung des Serverprozesses.

-threads *num*
Läuft mit *num* Arbeits-Threads. Dafür ist eine thread-sichere Tcl-Shell nötig.

-library *directory*
Verzeichnis, das an den auto_path angehängt wird.

-verbose
Gibt beim Start zusätzliche Meldungen aus.

BESCHREIBUNG

TclHttpd ist ein einfacher, erweiterbarer Webserver, der sich einbetten lässt. Die ausführliche Dokumentation liegt als HTML beim Server.

Zum Starten wird das Skript httpd.tcl mit *tclsh* oder *wish* ausgeführt. Das folgende Beispiel startet den Server auf dem üblichen Webserver-Port 80: tclsh <installdir>/bin/httpd.tcl -port 80.

Auf UNIX-Systemen muss der Server als root starten, wenn die Portnummer kleiner als 1024 ist. Soll der Prozess danach unter einem anderen Benutzer als root laufen, was dringend zu empfehlen ist, dienen die Optionen **-uid** und **-gid**. So startet der Server als root, öffnet den Socket und wechselt dann auf ein Konto mit weniger Rechten.

KONFIGURATION UND ANPASSUNG

Das Haupt-Skript hängt von einem Tcl-Skript je Thread, httpdthread.tcl, und von einer Konfigurationsdatei tclhttpd.rc ab. Darin stehen die Einstellungen und der Startcode des Webservers.

Die Konfigurationsdatei setzt Port, Benutzerkennung und die übrigen Werte aus der Optionsliste oben. Weitere Merkmale, etwa der Ort der Protokolldatei, stehen in derselben Datei. Zu jeder Option gibt es eine Erklärung. Eine Kopie der Datei nimmt neue Einstellungen auf.

```
tclsh httpd.tcl -config myserver.rc
```

Wer Tcl Httpd mit eigenem Code erweitert, ergänzt den Initialisierungscode in bin/httpd.tcl und bin/httpdthread.tcl. Das ist in der Regel ein "package require" des Moduls und ein oder zwei Aufrufe, die es initialisieren. Der folgende Code in httpdthread.tcl schaltet zum Beispiel eine URL /debug frei, über die sich der Zustand des Servers ansehen lässt.

```
package require httpd::debug
Debug_Url /debug Debug
```

Der Webserver erreicht jedes Tcl-Paket, das mit der Tcl-Installation liegt. Die mitgelieferte HTML-Dokumentation beschreibt das Programmieren des Servers ausführlicher.

SEITENVORLAGEN

TclHttpd hat ein Vorlagensystem, das Tcl-Code in die HTML-Seiten einbettet. Der Webserver wertet das Tcl aus. Es erzeugt in der Regel Teile der HTML-Seite, und das Ergebnis geht unverändert an den Client. Die Ergebnisse der Vorlagen lassen sich zwischenspeichern, oder die Seiten werden bei jedem Zugriff neu verarbeitet.

Jede Seite, die auf ".tml" endet, gilt als HTML+Tcl-Vorlage. Der Webserver ersetzt mit dem Tcl-Befehl **subst** Befehle in Klammern, [und], und Variablenverweise wie $Phone durch ihren Wert. Auch der Backslash wird verarbeitet. Wörtliche Dollarbeträge in den Vorlagen brauchen einen Backslash vor dem $:

```
The price is \$10.00.
```

Die ".tml"-Dateien in der Beispielstruktur unter htdocs sind die Vorlagen zum Weiterarbeiten.

Der Tcl-Code in den Seiten bleibt am besten bei einfachen Prozeduraufrufen. Die Prozedurdefinitionen stehen in Dateien je Verzeichnis mit dem Namen ".tml". Der Name verwirrt: jedes Verzeichnis kann eine Datei "dot-t-m-l" (.tml) enthalten, und die enthält Tcl-Code. Diese Dateien werden geladen, bevor eine Vorlage in diesem Verzeichnis oder in Unterverzeichnissen verarbeitet wird.

Zuerst entsteht zum Beispiel ein neues Verzeichnis unter dem htdocs-Verzeichnis von TclHttpd.

```
mkdir htdocs/mystuff
```

Danach kommt Folgendes in die Datei htdocs/mystuff/.tml.

```
package require htmlutils

# A procedure to format the date the way you like it
proc MyDate {{seconds {}}} {
    if {[string length $seconds] == 0} {
    set seconds [clock seconds]
    }
    return [clock format $seconds -format "%B %m, %Y"]
}
# Some page settings
set bgcolor pink
```

Danach kann jede Seite im Verzeichnis htdocs/mystuff die Prozedur **MyDate** in einer Vorlage verwenden. Zum Schluss kommt Folgendes in die Datei htdocs/mystuff/index.tml.

```
<title>My Stuff</title>
<body text=black bgcolor=$bgcolor>
<h2>My Stuff</h2>
[MyDate]
<br>
Page content here.
<p>
Send email to [Mailto [Doc_Webmaster]].
```

Die Variable **bgcolor** wird in der Datei .tml gesetzt und im BODY-Tag verwendet. **Mailto** gehört zum Paket **htmlutils**, das die Datei .tml anfordert. Die Prozedur **Doc_Webmaster** ist in TclHttpd eingebaut. Die Prozedur MyDate kommt dazu und gilt für jede Seite in htdocs/mystuff und darunter.

Copyright (c) 1998-2000 by Ajuba Solutions
All rights reserved.

tcltls

tls

Stichwörter: tls, I/O, IP Address, OpenSSL, SSL, TCP, TclTLS, asynchronous I/O, bind, certificate, channel, connection, domain name, host, https, network, network address, socket

NAME

tls - Anbindung an die Bibliothek OpenSSL für verschlüsselte Socket- und E/A-Kanal-Kommunikation

ÜBERSICHT

package require Tcl 8.5-
package require tls 2.0.1
tls::init ?*-option*? ?*value*? ?*-option value* ...?
tls::socket ?*-option*? ?*value*? ?*-option value* ...? *host port*
tls::socket -server *command* ?*-option*? ?*value*? ?*-option value* ...? *port*

tls::import *channel* ?*-option*? ?*value*? ?*-option value* ...?
tls::starttls *channel* ?*-option*? ?*value*? ?*-option value* ...?
tls::handshake *channel*
tls::shutdown *channel*
tls::unimport *channel*
tls::unstack *channel*
tls::status ?**-local**? *channel*
tls::connection *channel*
tls::ciphers ?*protocol*? ?*verbose*? ?*supported*?
tls::protocols
tls::version

BESCHREIBUNG

Diese Erweiterung gibt Tcl-Skripten Zugang zu Secure Socket Layer (SSL) über das Protokoll Transport Layer Security (TLS). Die Erweiterung bindet allgemein an OpenSSL und nutzt die API **Tcl_StackChannel** ab Tcl 8.4. Diese Sockets verhalten sich genau wie Kanäle des eingebauten Befehls **socket** und bieten zusätzliche Optionen für die SSL/TLS-Sitzung.

KOMPATIBILITÄT

Diese Erweiterung läuft mit OpenSSL 1.1.1 und neuer, einschließlich der Ausgaben 3.x und 4.0. Sie braucht Tcl 8.5 oder neuer und läuft mit Tcl 9.0.

BEFEHLE

Die folgenden Befehle stellt das Paket TclTLS bereit. Beispiele stehen unter **Beispiele** und im Verzeichnis "*demos*".

tls::init ?*-option*? ?*value*? ?*-option value ...*?
Setzt optional die Vorgaben, die **tls::socket** verwendet. Ein direkter Aufruf von **tls::import** lässt die Werte dieses Befehls ohne Wirkung. Der Befehl kennt dieselben Optionen wie **tls::socket**. Die Optionen sollten auf die TLS-bezogenen beschränkt bleiben.

tls::socket ?*-option*? ?*value*? ?*-option value ...*? *host port*
Hilfsfunktion. Die Funktion stellt die Verbindung über die Befehle **socket** und **tls::import** her, verhält sich wie der Tcl-Befehl **socket** und kennt zusätzlich die Optionen von **tls::import** sowie eine weitere Option. Zurück kommt die Kanal-Kennung des neuen Sockets. Die zusätzliche Option:

-autoservername *bool*
Bei **true** wird das Argument **-servername** automatisch auf das Argument *host* gesetzt. Vor TclTLS 2.0 ist die Vorgabe **false**. Ab TclTLS 2.0 ist die Vorgabe **true**, sofern **-servername** nicht ebenfalls angegeben ist.

tls::socket -server *command* ?*-option*? ?*value*? ?*-option value ...*? *port*
Wie der vorige Befehl, legt aber einen Server-Socket an, an den sich Klienten verbinden, analog zu **socket -server**. Zurück kommt die Kanal-Kennung des neuen Sockets.

tls::import *channel* ?*-option*? ?*value*? ?*-option value ...*?
... siehe den folgenden Eintrag ...

tls::starttls *channel* ?*-option*? ?*value*? ?*-option value ...*?
Startet die TLS-Verschlüsselung auf dem Tcl-Kanal *channel* über einen gestapelten Kanal. Ein

Socket ist dafür nicht nötig. Der Kanal muss Daten in beide Richtungen führen. Außerdem werden die Sitzungsparameter für den SSL-Handshake gesetzt. Gültige Optionen:

-alpn *list*
Liste der Protokolle, die bei der Application-Layer Protocol Negotiation (ALPN) angeboten werden. Zum Beispiel **h2** und **http/1.1**, nicht aber **h3** oder **quic**. Diese Option ist neu in TclTLS 1.8.

-cadir *directory*
Legt das Verzeichnis fest, in dem die Zertifikate der Zertifizierungsstelle (CA) liegen. Die Vorgabe hängt von der Plattform ab und kann zur Übersetzungszeit gesetzt werden. Die Umgebungsvariable **SSL_CERT_DIR** setzt den Vorgabeort außer Kraft. Einzelheiten stehen unter **Zertifikatsprüfung**.

-cafile *filename*
Legt die Datei mit den Zertifikaten der Zertifizierungsstelle (CA) im Dateiformat **PEM** fest. Die Vorgabe ist "*cert.pem*" im OpenSSL-Verzeichnis. Die Umgebungsvariable **SSL_CERT_FILE** setzt die Vorgabedatei außer Kraft. Einzelheiten stehen unter **Zertifikatsprüfung**.

-castore *URI*
Legt den Uniform Resource Identifier (URI) des Speichers der Zertifizierungsstelle (CA) fest. Das kann ein einzelner Container oder ein Katalog von Containern sein. Ab OpenSSL 3.2 unter MS Windows steht hier "**org.openssl.winstore://**", um den eingebauten Zertifikatsspeicher von MS Windows zu nutzen. Einzelheiten stehen unter **Zertifikatsprüfung**. Diese Option ist neu in TclTLS 1.8.

-certfile *filename*
Legt den Namen der Datei fest, deren Zertifikat im PEM-Format als lokales Zertifikat (Klient oder Server) dient. Die Datei enthält auch den öffentlichen Schlüssel.

-cert *string*
Legt das Zertifikat als DER-kodierte Zeichenkette fest (X.509 DER).

-cipher *string*
Legt die Liste der Ciphers für Verbindungen mit TLS 1.2 und älter fest. Die Zeichenkette ist eine mit Doppelpunkt "**:**" getrennte Liste von Ciphers. Ciphers lassen sich mit dem Zeichen "**+**" verbinden. Vorsilben entfernen eine Cipher dauerhaft ("**!**"), löschen sie ("**-**") oder stellen sie ans Ende ("**+**"). Zulässig sind auch die Schlüsselwörter **@STRENGTH** (Sortierung nach der Schlüssellänge des Algorithmus), **@SECLEVEL=***n* (Sicherheitsstufe n) und **DEFAULT** (Vorgabeliste der Ciphers, nur am Anfang). Die vollständige Liste gültiger Werte steht in der Dokumentation von OpenSSL.

-ciphersuites *string*
Legt die Liste der Cipher-Suites für TLS 1.3 fest, als mit Doppelpunkt "**:**" getrennte Liste von Suite-Namen. Die vollständige Liste gültiger Werte steht in der Dokumentation von OpenSSL. Diese Option ist neu in TclTLS 1.8.

-command *callback*
Legt den Rückrufbefehl fest, der während des Handshakes an mehreren Stellen aufgerufen wird und Fehler, Ablaufinformationen und Protokollnachrichten übergibt. Einzelheiten stehen unter **Rückrufoptionen**.

-dhparams *filename*
Legt die Datei mit den Parametern für Diffie-Hellman (DH) fest.

-keyfile *filename*
Legt die Datei mit dem privaten Schlüssel fest. Voreingestellt ist die Datei aus der Option **-certfile**.

-key *string*
Legt den privaten Schlüssel als DER-kodierte Zeichenkette fest (PKCS#1 DER).

-model *channel*
Erzwingt, dass dieser Kanal dieselbe Struktur *SSL_CTX* nutzt wie der angegebene Kanal *channel* und damit Konfiguration, Rückruffunktionen und Weiteres teilt.

-password *callback*
Legt den Rückrufbefehl fest, den OpenSSL aufruft, wenn ein Passwort gebraucht wird. Üblich ist das zum Entsperren des privaten Schlüssels eines Zertifikats. Die Rückruffunktion gibt eine Passwort-Zeichenkette zurück. Diese Option hat sich in TclTLS 1.8 geändert. Einzelheiten stehen unter **Rückrufoptionen**.

-post_handshake *bool*
Erlaubt Aktualisierungen des Sitzungstickets nach dem Handshake. Diese Option ist neu in TclTLS 1.8.

-request *bool*
Fordert während des SSL-Handshakes ein Zertifikat von der Gegenstelle an. Das ist für die Zertifikatsprüfung nötig. Ab TclTLS 1.8 ist die Vorgabe für Klientenverbindungen **true**. Ab TclTLS 2.0 wird der Wert auf **true** gesetzt, wenn hier **false** steht und **-require true** ist. Einzelheiten stehen unter **Zertifikatsprüfung**.

-require *bool*
Verlangt während des SSL-Handshakes ein gültiges Zertifikat von der Gegenstelle. Steht die Option auf wahr, muss auch **-request** auf wahr stehen, und zur Prüfung muss **-cadir**, **-cafile**, **-castore** oder eine Plattformvorgabe vorhanden sein. In TclTLS 1.8 und älter ist die Vorgabe **false**, weil nicht jede Plattform Zertifikate in einer mit OpenSSL verträglichen Form bereithält. Ab TclTLS 2.0 ist die Vorgabe für Klientenverbindungen **true**. Einzelheiten stehen unter **Zertifikatsprüfung**.

-security_level *integer*
Legt die Sicherheitsstufe fest, ein Wert von 0 bis 5. Die Stufe bestimmt die erlaubten Verschlüsselungsalgorithmen der Cipher-Suites, die unterstützten ECC-Kurven, die unterstützten Signaturalgorithmen, die Größen der DH-Parameter, die Schlüsselgrößen der Zertifikate und die Signaturalgorithmen. Vor OpenSSL 3.2 ist die Vorgabe 1, danach 2. Stufe 3 und höher schalten Sitzungstickets ab und nehmen nur Cipher-Suites mit vorwärts gerichteter Geheimhaltung an. Diese Option ist neu in TclTLS 1.8.

-server *bool*
Legt fest, ob die Seite als Server auftritt und auf den Handshake eines Klienten mit einem Server-Handshake antwortet. Die Vorgabe ist **false**.

-servername *hostname*
Gibt den Hostnamen der Gegenstelle an. Daraus wird die Erweiterung TLS Server Name Indication (SNI) gesetzt. Der Wert ist der erwartete Servername im Zertifikat des Servers oder einer der Subject Alternate Names (SAN). Ab TclTLS 2.0 ist die Vorgabe der Host aus dem Befehl **tls::socket**.

-session_id *binary_string*
Legt die Sitzungskennung fest, mit der eine Sitzung fortgesetzt wird. Noch nicht unterstützt.

Diese Option ist neu in TclTLS 1.8.

-ssl2 ***bool***
Schaltet SSL v2 ein. Die Vorgabe ist **false**. OpenSSL 1.1 und neuer unterstützt SSL v2 nicht mehr, die Option kann also ohne Wirkung bleiben. Die unterstützten Protokolle liefert der Befehl **tls::protocols**.

-ssl3 ***bool***
Schaltet SSL v3 ein. Die Vorgabe ist **false**. Ab TclTLS 1.8 steht SSL v3 nur über eine Option zur Übersetzungszeit zur Verfügung. OpenSSL 4.0 und neuer unterstützt SSL v3 nicht mehr, die Option kann also ohne Wirkung bleiben. Die unterstützten Protokolle liefert der Befehl **tls::protocols**.

-tls1 ***bool***
Schaltet TLS v1 ein. Ab TclTLS 2.0 ist die Vorgabe **false**. TLS 1.0 braucht zum Betrieb SHA1, und das steht bei OpenSSL 3.0 und neuer nur auf Sicherheitsstufe 0 bereit. Siehe die Option **-security_level**.

-tls1.1 ***bool***
Schaltet TLS v1.1 ein. Ab TclTLS 2.0 ist die Vorgabe **false**. TLS 1.1 braucht zum Betrieb SHA1, und das steht bei OpenSSL 3.0 und neuer nur auf Sicherheitsstufe 0 bereit. Siehe die Option **-security_level**.

-tls1.2 ***bool***
Schaltet TLS v1.2 ein. Die Vorgabe ist **true**.

-tls1.3 ***bool***
Schaltet TLS v1.3 ein. Die Vorgabe ist **true**. Verfügbar erst ab OpenSSL 1.1.1 und TclTLS 1.7.

-validatecommand ***callback***
Legt den Rückrufbefehl fest, der während der Protokollverhandlung die Zertifikate der Gegenstelle und weitere Konfiguration prüft. Tcl-Skripte können damit eine eigene Zertifikatsprüfung zusätzlich zur Vorgabe von OpenSSL ausführen. Das Skript gibt den booleschen Wert wahr zurück, damit die Verhandlung weiterläuft. Einzelheiten stehen unter **Rückrufoptionen**. Diese Option ist neu in TclTLS 1.8.

tls::handshake ***channel***
Erzwingt den Handshake der TLS-Verhandlung sofort. Zurück kommt 0, wenn der Handshake noch läuft (nicht blockierend), oder 1, wenn er gelungen ist. Schlägt der Handshake fehl, kommt ein Fehler zurück.

tls::shutdown ***channel***
… siehe den folgenden Eintrag …

tls::unimport ***channel***
… siehe den folgenden Eintrag …

tls::unstack ***channel***
Beendet die SSL/TLS-Sitzung durch die Nachricht "close_notify" und nimmt den obersten gestapelten Kanal von *channel*, ohne den Socket zu schließen. Das ist das Gegenstück zu **tls::import** und beendet die Verschlüsselung eines Tcl-Kanals. Ist TLS nicht der oberste gestapelte Kanaltyp, wird ein Fehler ausgelöst.

tls::status ?-local? ***channel***
Gibt den aktuellen Status eines SSL-Kanals zurück. Das Ergebnis ist eine Liste von Schlüssel-Wert-Paaren zu SSL, Zertifikat und Zertifikatsprüfung. Ist der SSL-Handshake noch

nicht abgeschlossen, kommt eine leere Liste zurück. Mit der Option **-local** wird das lokale Zertifikat verwendet. Zurückgegebene Werte, darunter der SSL-Status:

alpn *protocol*
Das nach der Application-Layer Protocol Negotiation (ALPN) gewählte Protokoll. Dieser Wert ist neu in TclTLS 1.8.

cipher *cipher*
Die Cipher, die die Sitzung gerade verwendet.

peername *name*
Der Name der Gegenstelle aus dem Zertifikat. Dieser Wert ist neu in TclTLS 1.8.

protocol *version*
Die Protokollversion der Verbindung: SSL2, SSL3, TLS1, TLS1.1, TLS1.2, TLS1.3 oder unknown. Dieser Wert ist neu in TclTLS 1.8.

sbits *n*
Die Anzahl der Bits des Sitzungsschlüssels.

signatureHashAlgorithm *algorithm*
Der Hash-Algorithmus der Signatur. Dieser Wert ist neu in TclTLS 1.8.

signatureType *type*
Der Wert des Signaturtyps. Dieser Wert ist neu in TclTLS 1.8.

verifyDepth *n*
Größte Tiefe bei der Prüfung der Zertifikatskette. Die Vorgabe ist -1, dann wird alles geprüft. Dieser Wert ist neu in TclTLS 1.8.

verifyMode *list*
Liste der Modi der Zertifikatsprüfung. Dieser Wert ist neu in TclTLS 1.8.

verifyResult *result*
Ergebnis der Zertifikatsprüfung. Dieser Wert ist neu in TclTLS 1.8.

ca_names *list*
Liste der Zertifizierungsstellen, mit denen das Zertifikat erzeugt wurde. Dieser Wert ist neu in TclTLS 1.8.

Zertifikatsstatus

all *string*
Ausgabe aller Zertifikatsangaben. Dieser Wert ist neu in TclTLS 1.8.

version *value*
Die Version des Zertifikats.

serialNumber *string*
Die Seriennummer des Zertifikats als Hexadezimalzeichenkette. Dieser Wert hieß vor TclTLS 1.8 serial.

signature *algorithm*
Cipher-Algorithmus der Zertifikatssignatur. Dieser Wert ist neu in TclTLS 1.8.

issuer *string*
Der Distinguished Name (DN) des Ausstellers des Zertifikats.

notBefore *date*
Das Anfangsdatum der Gültigkeit des Zertifikats.

notAfter *date*

Das Ablaufdatum der Gültigkeit des Zertifikats.

subject *string*
Der Distinguished Name (DN) des Inhabers des Zertifikats. Felder sind Common Name (CN), Organization (O), Locality oder City (L), State oder Province (S) und Country Name (C).

issuerUniqueID *string*
Die eindeutige Kennung des Ausstellers. Dieser Wert ist neu in TclTLS 1.8.

subjectUniqueID *string*
Die eindeutige Kennung des Inhabers. Dieser Wert ist neu in TclTLS 1.8.

num_extensions *n*
Anzahl der Zertifikatserweiterungen. Dieser Wert ist neu in TclTLS 1.8.

extensions *list*
Liste der Namen der Zertifikatserweiterungen. Dieser Wert ist neu in TclTLS 1.8.

authorityKeyIdentifier *string*
Authority Key Identifier (AKI) des CA-Zertifikats, das das SSL-Zertifikat signiert hat, als Hexadezimalzeichenkette. Der Wert stimmt mit dem SKI des Zwischen-CA-Zertifikats überein. Dieser Wert ist neu in TclTLS 1.8.

subjectKeyIdentifier *string*
Subject Key Identifier (SKI), der Hash des öffentlichen Schlüssels im Zertifikat als Hexadezimalzeichenkette. Damit lassen sich Zertifikate erkennen, die einen bestimmten öffentlichen Schlüssel enthalten. Dieser Wert ist neu in TclTLS 1.8.

subjectAltName *list*
Liste aller Subject Alternative Names (SAN), einschließlich Domänennamen, Unterdomänen und IP-Adressen, die das Zertifikat absichert. Dieser Wert ist neu in TclTLS 1.8.

ocsp *list*
Liste aller URLs des Online Certificate Status Protocol (OCSP), mit denen sich die Gültigkeit dieses Zertifikats prüfen lässt. Dieser Wert ist neu in TclTLS 1.8.

certificate *cert*
Das PEM-kodierte Zertifikat.

signatureAlgorithm *algorithm*
Cipher-Algorithmus der Signatur des Zertifikats. Dieser Wert ist neu in TclTLS 1.8.

signatureValue *string*
Zertifikatssignatur als Hexadezimalzeichenkette. Dieser Wert ist neu in TclTLS 1.8.

signatureDigest *version*
Signaturprüfsumme des Zertifikats als Hexadezimalzeichenkette. Dieser Wert ist neu in TclTLS 1.8.

publicKeyAlgorithm *algorithm*
Algorithmus des öffentlichen Schlüssels der Zertifikatssignatur. Dieser Wert ist neu in TclTLS 1.8.

publicKey *string*
Öffentlicher Schlüssel der Zertifikatssignatur als Hexadezimalzeichenkette. Dieser Wert ist neu in TclTLS 1.8.

bits *n*
Anzahl der Bits des Signaturschlüssels des Zertifikats. Dieser Wert ist neu in TclTLS 1.8.

self_signed *boolean*

Ob die Zertifikatssignatur selbst signiert ist. Dieser Wert ist neu in TclTLS 1.8.

sha1_hash *hash*
Der SHA1-Hash des Zertifikats als Hexadezimalzeichenkette. Dieser Wert ist neu in TclTLS 1.8.

sha256_hash *hash*
Der SHA256-Hash des Zertifikats als Hexadezimalzeichenkette. Dieser Wert ist neu in TclTLS 1.8.

tls::connection *channel*
Gibt den aktuellen Verbindungsstatus eines SSL-Kanals zurück. Das Ergebnis ist eine Liste von Schlüssel-Wert-Paaren zur Verbindung. Dieser Befehl ist neu in TclTLS 1.8. Zurückgegebene Werte, darunter der SSL-Status:

state *state*
Zustand der Verbindung.

servername *name*
Der Name des Servers, zu dem die Verbindung besteht.

protocol *version*
Die Protokollversion der Verbindung: SSL2, SSL3, TLS1, TLS1.1, TLS1.2, TLS1.3 oder unknown.

renegotiation_allowed *boolean*
Ob eine Neuverhandlung des Protokolls unterstützt wird.

security_level *level*
Die Sicherheitsstufe für die Wahl der Ciphers, der Schlüsselgröße und Weiteres.

session_reused *boolean*
Ob die Sitzung wiederverwendet wurde.

is_server *boolean*
Ob die Verbindung als Server (1) oder als Klient (0) eingerichtet ist.

compression *mode*
Kompressionsverfahren.

expansion *mode*
Expansionsverfahren.

caList *list*
Liste der Zertifizierungsstellen (CA) für das X.509-Zertifikat.

Angaben zur Cipher

cipher *cipher*
Die Cipher, die die Verbindung gerade verwendet.

standard_name *name*
Der Standardname der Cipher nach RFC.

algorithm_bits *n*
Die Anzahl der verarbeiteten Bits der Cipher.

secret_bits *n*
Die Anzahl der geheimen Bits der Cipher.

min_version *version*
Die kleinste Protokollversion der Cipher.

cipher_is_aead *boolean*

Ob die Cipher Authenticated Encryption with Associated Data (AEAD) ist.

cipher_id *id*
Die Cipher-Kennung von OpenSSL.

description *string*
Eine Textbeschreibung der Cipher.

handshake_digest *boolean*
Prüfsumme, die während des Handshakes verwendet wird.

Angaben zur Sitzung

alpn *protocol*
Das nach der Application-Layer Protocol Negotiation (ALPN) gewählte Protokoll.

resumable *boolean*
Ob die Sitzung fortgesetzt werden kann.

start_time *seconds*
Zeit seit dem Beginn der Sitzung, in Sekunden seit der Epoche.

timeout *seconds*
Höchstdauer der Sitzung in Sekunden bis zur Zeitüberschreitung.

lifetime *seconds*
Hinweis auf die Lebensdauer des Sitzungstickets in Sekunden.

session_id *binary_string*
Eindeutige Sitzungskennung zum Fortsetzen der Sitzung.

session_ticket *binary_string*
Eindeutiges Sitzungsticket zum Fortsetzen der Sitzung.

ticket_app_data *binary_string*
Eindeutige Anwendungsdaten des Sitzungstickets.

master_key *binary_string*
Eindeutiger Master-Schlüssel der Sitzung.

session_cache_mode *mode*
Cache-Modus des Servers (client, server oder both).

tls::ciphers ?*protocol*? ?*verbose*? ?*supported*?
Ohne Optionen kommt eine Liste aller symmetrischen Ciphers für die Option *-cipher* zurück. Mit *protocol* kommen nur die Ciphers, die dieses Protokoll unterstützt. Die unterstützten Protokolle liefert der Befehl **tls::protocols**. Ist *verbose* wahr, kommt eine ausführliche, lesbare Liste mit weiteren Angaben zur Cipher zurück. Ist *supported* wahr, werden nur die Ciphers aufgeführt, die das Protokoll unterstützt. Das Argument *supported* ist neu in TclTLS 1.8.

tls::protocols
Gibt eine Liste der unterstützten SSL/TLS-Protokolle zurück. Gültige Werte sind **ssl2**, **ssl3**, **tls1**, **tls1.1**, **tls1.2** und **tls1.3**. Die genaue Liste hängt von der OpenSSL-Version und den Schaltern zur Übersetzungszeit ab. Dieser Befehl ist neu in TclTLS 1.8.

tls::version
Gibt die Versionszeichenkette von OpenSSL zurück.

ZERTIFIKATSPRÜFUNG

PKI und Zertifikate

Mit der Public Key Infrastructure (PKI) erzeugt jeder Benutzer einen privaten Schlüssel, den nur er kennt, und einen öffentlichen Schlüssel, den er mit anderen zum Ver- und Entschlüsseln austauscht. Der Ablauf ist: der Absender verschlüsselt seine Daten mit seinem privaten Schlüssel und dem öffentlichen Schlüssel des Empfängers. Die Daten gehen dann an den Empfänger. Entsprechend entschlüsselt der Empfänger mit seinem privaten Schlüssel und dem öffentlichen Schlüssel des Absenders. Das sichert die Unversehrtheit der Daten, damit sie unterwegs nicht gelesen oder verändert werden können. Der private Schlüssel steht in den Optionen **-key** und **-keyfile**. Das Passwort steht in der Option **-password**.

Für die Authentisierung, also dafür, dass jemand der ist, der er zu sein behauptet, liegen der öffentliche Schlüssel und die Identitätsangaben in einem X.509-Zertifikat, und eine Zertifizierungsstelle (CA) authentisiert dieses Zertifikat, das heißt, sie signiert es. Die Zertifikate werden beim TLS-Start ausgetauscht und gegen die Wurzelzertifikate der CA geprüft. Das erledigt OpenSSL über die Optionen **-request** und **-require**. Wo die CA-Zertifikate liegen, steht in den Optionen **-cadir**, **-cadir** und **-castore**. In einer späteren Ausgabe können sie zusätzlich gegen die Certificate Revocation List (CRL) widerrufener Zertifikate geprüft werden. Zertifikate können auch selbst signiert sein. Voreingestellt gelten sie nicht als vertrauenswürdig, solange sie nicht in den Zertifikatsspeicher aufgenommen sind.

Beim Besuch von Webseiten muss in der Regel nur der Klient das Zertifikat des Servers prüfen. Der Server prüft das Klientenzertifikat nur, wenn eine Anmeldung eine Authentisierung verlangt. Ein eigenes Zertifikat steht in den Optionen **-cert** und **-certfile**.

Zusammenfassung der Befehlszeilenoptionen

Die folgenden Optionen dienen der Prüfung des Zertifikats der Gegenstelle:

-cadir *directory*
: Legt das Verzeichnis fest, in dem die Zertifikate der Zertifizierungsstelle (CA) liegen. Die Vorgabe hängt von der Plattform ab, auf Linux- und Unix-Systemen ist das meist "*/etc/ssl/certs*". Die Umgebungsvariable **SSL_CERT_DIR** setzt den Vorgabeort außer Kraft.

-cafile *filename*
: Legt die Datei mit den Zertifikaten der Zertifizierungsstelle (CA) im Dateiformat **PEM** fest. Die Vorgabe ist "*cert.pem*" im OpenSSL-Verzeichnis. Auf Linux- und Unix-Systemen ist das meist "*/etc/ssl/ca-bundle.pem*". Die Umgebungsvariable **SSL_CERT_FILE** setzt die Vorgabedatei außer Kraft.

-castore *URI*
: Legt den Uniform Resource Identifier (URI) des Speichers der Zertifizierungsstelle (CA) fest. Das kann ein einzelner Container oder ein Katalog von Containern sein. Ab OpenSSL 3.2 unter MS Windows steht hier "**org.openssl.winstore://**", um den eingebauten Zertifikatsspeicher von MS Windows zu nutzen. Ab TclTLS 2.0 ist das die Vorgabe, wenn **-cadir**, **-cadir** und **-castore** nicht angegeben sind. Dieser Speicher kennt nur Wurzelzertifikatsspeicher.

-request *bool*
: Fordert während des SSL-Handshakes ein Zertifikat von der Gegenstelle an. Das ist für die Zertifikatsprüfung nötig. Ab TclTLS 1.8 ist die Vorgabe für Klientenverbindungen **true**. Ab TclTLS 2.0 wird der Wert auf **true** gesetzt, wenn hier **false** steht und **-require true** ist. Zusätzlich kann der Klient jedes Zertifikat mit der Option **-validatecommand** selbst prüfen und annehmen oder ablehnen.

-require *bool*
Verlangt während des SSL-Handshakes ein gültiges Zertifikat von der Gegenstelle. Steht die Option auf wahr, muss auch **-request** auf wahr stehen, und zur Prüfung muss **-cadir**, **-cafile**, **-castore** oder eine Plattformvorgabe vorhanden sein. In TclTLS 1.8 und älter ist die Vorgabe **false**, weil nicht jede Plattform Zertifikate in einer mit OpenSSL verträglichen Form bereithält. Ab TclTLS 2.0 ist die Vorgabe für Klientenverbindungen **true**.

Wann Befehlszeilenoptionen nötig sind

In TclTLS 1.8 und älter ist die Zertifikatsprüfung voreingestellt *nicht* eingeschaltet. Das liegt daran, dass es keine gemeinsame, plattformübergreifende Datenbank der von einer Zertifizierungsstelle (CA) bereitgestellten Zertifikate gibt. Viele Linux-Systeme unterstützen OpenSSL von Haus aus und haben diese Zertifikate als Teil des Systems installiert, MacOS und MS Windows nicht. Ab TclTLS 2.0 verlangen Klientenverbindungen die Zertifikatsprüfung voreingestellt. Für die Option **-require** muss eines der Folgenden zutreffen:

- Auf Linux- und Unix-Systemen mit bereits installiertem OpenSSL, oder wenn die CA-Zertifikate im PEM-Format vorliegen und an den üblichen Orten liegen oder die Umgebungsvariablen **SSL_CERT_DIR** oder **SSL_CERT_FILE** gesetzt sind, werden **-cadir**, **-cadir** und **-castore** nicht gebraucht.
- Liegt OpenSSL nicht am Vorgabeort, oder ist OpenSSL unter Mac OS oder MS Windows installiert, müssen die Umgebungsvariablen **SSL_CERT_DIR** und/oder **SSL_CERT_FILE** oder eine der Optionen **-cadir**, **-cadir** oder **-castore** gesetzt sein.
- Unter MS Windows kann OpenSSL ab Version 3.2 den eingebauten Windows-Zertifikatsspeicher ansprechen. Dafür steht die Option **-castore** auf "**org.openssl.winstore://**". In TclTLS 2.0 ist das der Vorgabewert, wenn **-cadir**, **-cadir** und **-castore** nicht angegeben sind.
- Ist OpenSSL nicht installiert oder liegen die CA-Zertifikate nicht im PEM-Format vor, müssen die CA-Zertifikate geladen und mit der Software installiert werden. Das CURL-Team stellt sie unter CA-Zertifikate aus Mozilla in der Datei "*cacert.pem*" bereit. Danach zeigen die Umgebungsvariablen **SSL_CERT_DIR** und/oder **SSL_CERT_FILE** oder die Optionen **-cadir** oder **-cafile** auf den Installationsort dieser Datei. Die Datei aktuell zu halten, liegt beim Anwender.

RÜCKRUFOPTIONEN

Wie oben beschrieben, kann jeder Kanal eigene Rückruffunktionen für die Zwischenverarbeitung durch die Bibliothek OpenSSL bekommen, über die Optionen **-command**, **-password** und **-validate_command** an **tls::socket** oder **tls::import**. Anders als in älteren TclTLS-Versionen wird der Befehl **bgerror** nur aufgerufen, wenn die Rückruffunktion einen Fehler erzeugt, und dann mit den Fehlerangaben.

Werte für den Befehlsrückruf

Die Rückruffunktion der Option **-command** wird an mehreren Stellen während des OpenSSL-Handshakes und im laufenden Betrieb aufgerufen. Die möglichen Argumente des

Rückrufskripts stehen unten. Rückgabewerte der Rückruffunktion werden ignoriert.

error *channelId message*
Diese Form wird aufgerufen, wenn während der ersten Verbindung, des Handshakes oder der E/A ein Fehler auftritt. Das Argument *message* kann von Tcl_ErrnoMsg, von der OpenSSL-Funktion **ERR_reason_error_string()** oder eine eigene Nachricht sein. Diese Form ist neu in TclTLS 1.8.

info *channelId major minor message type*
Diese Form wird von der OpenSSL-Funktion **SSL_set_info_callback()** während der ersten Verbindung und des Handshakes aufgerufen. Die Argumente:

major
Hauptkategorie des Fehlers. Gültige Werte: **handshake**, **alert**, **connect**, **accept**.

minor
Nebenkategorie des Fehlers. Gültige Werte: **start**, **done**, **read**, **write**, **loop**, **exit**.

message
Beschreibende Nachricht, je nach Zusammenhang von **SSL_state_string_long()** oder **SSL_alert_desc_string_long()**.

type
Bei Alarmen sind die Werte **warning**, **fatal** und **unknown**. Sonst steht **info**. Dieses Argument ist neu in TclTLS 1.8.

message *channelId direction version content_type message*
Diese Form wird von der OpenSSL-Funktion **SSL_set_msg_callback()** aufgerufen, wenn während der ersten Verbindung, des Handshakes oder der E/A eine Nachricht gesendet oder empfangen wird. Verfügbar nur, wenn OpenSSL mit der Option **enable-ssl-trace** übersetzt wurde. Diese Form ist neu in TclTLS 1.8. Die Argumente:

direction
Die Richtung ist **Sent** oder **Received**.

version
Die Version ist die Protokollversion.

content_type
Der Inhaltstyp ist der Inhaltstyp der Nachricht.

message
Die Nachricht sind weitere Angaben aus der API **SSL_trace**.

session *channelId session_id session_ticket lifetime*
Diese Form wird von der OpenSSL-Funktion **SSL_CTX_sess_set_new_cb()** aufgerufen, wenn der Server während der ersten Verbindung und des Handshakes eine neue Sitzungskennung sendet, und auch während der Sitzung, wenn **-post_handshake** auf wahr steht. Diese Form ist neu in TclTLS 1.8. Die Argumente:

session_id
Die Sitzungskennung ist der aktuelle Bezeichner der Sitzung.

session_ticket
Das Ticket sind die Angaben zum Sitzungsticket.

lifetime
Die Lebensdauer ist die Lebensdauer des Tickets in Sekunden.

verify *channelId depth cert status error*
Diese Rückruffunktion liegt seit TclTLS 1.8 bei **-validatecommand**.

Werte für den Passwortrückruf

Die Rückruffunktion der Option **-password** ruft TclTLS auf, wenn OpenSSL ein Passwort braucht. Die möglichen Argumente des Rückrufskripts stehen unten. Die Rückruffunktion gibt das eingegebene Passwort zurück.

password *rwflag size*
Wird beim Laden oder Speichern eines verschlüsselten PEM-Zertifikats aufgerufen. Die Argumente:

rwflag
Die Lese-/Schreibmarke ist 0 für Lesen und Entschlüsseln oder 1 für Schreiben und Verschlüsseln. Der Wert 1 zeigt, wann eine Bestätigung erfragt wird. Dieses Argument ist neu in TclTLS 1.8.

size
Die Größe ist die größte Länge des Passworts in Bytes. Dieses Argument ist neu in TclTLS 1.8.

Werte für den Prüfrückruf

Die Rückruffunktion der Option **-validatecommand** wird während des Handshakes aufgerufen, damit die Anwendung die übergebenen Werte prüft. Die möglichen Argumente des Rückrufskripts stehen unten. Ist keiner angegeben, nimmt OpenSSL alle gültigen Zertifikate und Erweiterungen an. Zum Ablehnen des Wertes und Abbrechen der Verbindung gibt die Rückruffunktion 0 zurück. Zum Annehmen und Fortsetzen gibt er 1 zurück. Zum Ablehnen bei fortgesetzter Verbindung gibt er 2 zurück. Diese Form ist neu in TclTLS 1.8.

alpn *channelId protocol match*
Auf Servern wird diese Form aufgerufen, wenn die ALPN-Erweiterung des Klienten ankommt. Ist *match* wahr, ist *protocol* die erste Option **-alpn**, die Klient und Server gemeinsam haben. Sonst wird das erste vom Klienten angegebene Protokoll verwendet. Diese Form kommt nach den Formen **hello** und **sni**.

hello *channelId servername session_id*
Auf Servern wird diese Form bei der Verarbeitung der Client-Hello-Nachricht aufgerufen. Der Server kann damit das passende Zertifikat für den Klienten wählen und die Konfiguration zu diesem Servernamen anpassen. Diese Form kommt vor den Formen **sni** und **alpn**.

sni *channelId servername*
Auf Servern wird diese Form aufgerufen, wenn die Erweiterung Server Name Indication (SNI) ankommt. Das Argument *servername* ist der vom Klienten angegebene Servername aus der Option **-servername**. Unterstützt ein Server mehrere Namen, kann so das richtige Zertifikat verwendet werden. Diese Form kommt nach **hello** und vor **alpn**.

verify *channelId depth cert status error*
Diese Form wird von OpenSSL aufgerufen, wenn von der Gegenstelle ein neues Zertifikat ankommt. Der Klient kann das Prüfergebnis ansehen und entscheiden, ob es weitergeht. Die Rückruffunktion läuft für jedes Zertifikat der Kette. Diese Form lag vor TclTLS 1.8 bei **-command**. Die Argumente:

depth
Die Tiefe ist die ganzzahlige Tiefe des Zertifikats in der Zertifikatskette. 0 ist das Zertifikat der

Gegenstelle, höhere Werte führen hinauf zur Zertifizierungsstelle (CA).

cert
Das Argument cert ist eine Liste von Schlüssel-Wert-Paaren, ähnlich der Rückgabe von **tls::status**.

status
Das Argument status ist die boolesche Gültigkeit des aktuellen Zertifikats. 0 ist ungültig, 1 ist gültig.

error
Das Argument error ist die Fehlermeldung, sofern vorhanden, von **X509_STORE_CTX_get_error()**.

Referenzumsetzungen dieser Rückruffunktionen stehen in "*tls.tcl*" als **tls::callback**, **tls::password** und **tls::validate_command**. Das sind nur Beispielumsetzungen. In einem wirklichen Einsatz bekommen die TLS-Kanäle eigene Rückrufskripte über die Optionen **-command**, **-password** und **-validate_command**.

Sind **-command** und **-validate_command** nicht angegeben, verarbeitet TclTLS die zugehörigen Bibliotheksrückrufe intern. Ist **-password** nicht angegeben, versucht TclTLS, **tls::password** aufzurufen. Der Unterschied folgt aus der Verträglichkeit mit älteren Umsetzungen.

Von den Referenzrückrufen **tls::callback**, **tls::password** und **tls::validate_command** wird abgeraten. In späteren Ausgaben können sie entfallen.

FEHLERSUCHE

Für die meisten Fälle der Fehlersuche genügt die Option **-callback**, um den TLS-Handshake und den Fortschritt zu sehen. Wird mehr gebraucht, liefert die Option zur Übersetzungszeit **--enable-debug** einen genauen Ablauf.

Die Protokollierung von TLS-Schlüsseln wird eingeschaltet, indem die Umgebungsvariable **SSLKEYLOGFILE** auf den Namen der Protokolldatei zeigt. Erzeugtes oder empfangenes TLS-Schlüsselmaterial wird dann in diese Datei geschrieben. Netz-Protokollwerkzeuge können die Daten damit entschlüsseln.

Die Variable **tls::debug** steuert die Diagnoseprotokolle der Vorgaben **tls::callback**, **tls::password** und **tls::validate_command** in "*tls.tcl*" zusätzlich. Die Vorgabe ist 0. Höhere Werte erzeugen mehr Diagnose und zwingen die Prüfmethode in **tls::callback**, das Zertifikat anzunehmen, auch wenn es ungültig ist, sofern **-validatecommand** auf **tls::validate_command** steht.

Von der Variablen **tls::debug** wird abgeraten. In späteren Ausgaben kann sie entfallen.

BEISPIELE

Die folgenden Skripte laden mit dem Paket http eine Webseite und eine Datei. Wann zusätzlich **-cadir**, **-cafile** und **-castore** nötig sind, steht unter **Zertifikatsprüfung**. Weitere Beispielskripte liegen im Verzeichnis "*demos*".

Beispiel 1: Eine Webseite laden

```
package require http
package require tls

set url "https://www.tcl.tk/"
http::register https 443 [list ::tls::socket -autoservername 1
    -require 1]

# Get URL
set token [http::geturl $url]

# Check for error
if {[http::status $token] ne "ok"} {
    puts [format "Error %s" [http::status $token]]
}

# Save web page to file
set ch [open example.html wb]
puts $ch [http::data $token]
close $ch

# Cleanup
::http::cleanup $token
```

Beispiel 2: Eine Datei laden

```
package require http
package require tls

set url "https://wiki.tcl-lang.org/sitemap.xml"
http::register https 443 [list ::tls::socket -autoservername 1
    -require 1]

# Open output file
set filename [file tail $url]
set ch [open $filename wb]

# Get file
set token [::http::geturl $url -blocksize 65536 -channel $ch]

# Check for error
if {[http::status $token] ne "ok"} {
    puts [format "Error %s" [http::status $token]]
}

# Cleanup
close $ch
::http::cleanup $token
```

BESONDERE HINWEISE

Was dieses Paket kann, hängt stark davon ab, wie die angebundene OpenSSL-Bibliothek konfiguriert und gebaut wurde. Neue Versionen können ältere Protokollversionen ablösen, Ciphers hinzufügen oder entfernen, Vorgaben ändern. Die unterstützten Protokollversionen liefert der Befehl **tls::protocols**.

FEHLERMELDUNGEN

Manche Fehlermeldungen von OpenSSL sind schwer zu lesen. Die Liste nennt die Meldung und ihre Bedeutung.

handshake failed: certificate verify failed due to "unable to get local issuer certificate"

In der CA-Datei oder im Zertifikatsspeicher fehlt mindestens ein Aussteller der geprüften Zertifikate, oder die Zertifikate sind abgelaufen. In der Regel wird eine aktuelle CAcert-Datei gebraucht.

packet length too long

Der Klient hat einen HTTP-Server auf dem Klartext-Port angesprochen, nicht auf dem SSL/TLS-Port.

unexpected eof while reading

Die Gegenstelle hat die Verbindung geschlossen, ohne den Abschlusshinweis "close notify" zu senden. Manche Server beenden die Verbindung nach dem Senden der Datei oder Webseite ohne diese Nachricht. Dann sollten keine Daten fehlen.

wrong version number

Der Klient hat einen Server, der kein HTTP spricht, auf einem Port ohne TLS angesprochen, also im Klartext.

KATEGORIE

tls

COPYRIGHT


```
Copyright (c) 1999 Matt Newman
Copyright (c) 2004 Starfish Systems
Copyright (c) 2024 Brian O'Hagan
```


tcludp

udp

Stichwörter: udp, I/O, IP Address, TclUDP, asynchronous I/O, bind, channel, connection, domain, host, name, network, network address, networking, socket

NAME

udp - UDP-Sockets in Tcl anlegen

ÜBERSICHT

package require Tcl 8.5-
package require udp 1.0.14

udp_open ?*localport*? ?**reuse**? ?**ipv6**?
udp_conf *channel host port*
udp_conf *channel* ?*optionName*? ?*value*? ?*optionName value* ...?
udp_peek *channel* ?*buffersize*?
::udp::build-info
::udp::getaddrinfo -hostname *name* ?*optionName value* ...?
::udp::getnameinfo *address* ?**ipv6**?

BESCHREIBUNG

Dieses Paket ermöglicht UDP aus Tcl. Es stellt einen neuen Kanaltyp bereit und erlaubt paketorientiertes UDP über die stromorientierten Kanäle von Tcl. Das Paket definiert drei Befehle. **udp_conf** gilt als veraltet; an seine Stelle tritt der Tcl-Befehl **fconfigure**.

BEFEHLE

udp_open ?*localport*? ?reuse? ?ipv6?
: **udp_open** öffnet einen UDP-Socket. Ist *localport* angegeben, wird der Socket an diesen Port gebunden. Sonst wählt das System einen Port. Die Portnummer liefert bei Bedarf **udp_conf**. Die folgenden Schlüsselwörter setzen Optionen des geöffneten Sockets.

reuse
: Dieses Schlüsselwort setzt die Socket-Option SO_REUSEADDR. Damit dürfen mehrere Sockets an dieselbe Kombination aus Adresse und Port gebunden werden.

ipv6
: Voreingestellt entsteht ein IPv4-Socket. Das Schlüsselwort **ipv6** öffnet einen IPv6-Socket.

udp_conf *channel host port*
: *Veraltet*, zugunsten der Tcl-Befehle **fconfigure** und **chan configure**. **udp_conf** in dieser Form legt das entfernte Ziel der Pakete fest, die auf diesen *channel* geschrieben werden. Der Befehl ist aufzurufen, bevor Daten auf den UDP-Socket geschrieben werden.

udp_conf *channel?optionName??value??optionName value ...?*
: *Veraltet*, zugunsten der Tcl-Befehle **fconfigure** und **chan configure**. Neben dem entfernten Rechner liefert **udp_conf** Angaben zum UDP-Socket. Alle diese Optionen stehen inzwischen über **fconfigure** und **chan configure** zur Verfügung.

-myport
: Gibt die lokale Portnummer des Sockets zurück. Option nur zum Lesen.

-remote*?address port?*
: Legt den entfernten Rechnernamen und die Portnummer fest oder gibt sie zurück. Dieselbe Angabe setzt **udp_conf** *channel host port*.

-peer
: Gibt Rechnername und Portnummer des zuletzt auf diesem Socket empfangenen Pakets zurück. Option nur zum Lesen.

-family
: Gibt zurück, ob der Socket für IPv4 oder IPv6 eingerichtet ist. Option nur zum Lesen.

-broadcast*?boolean?*
: Legt fest oder gibt zurück, ob der Socket auf der Broadcast-Adresse empfangen und senden kann. Auf manchen Systemen braucht der Socket dafür ein Flag. Mit IPv6 ist die Option nicht

zulässig; dort ist Multicast zu verwenden.

-ttl*?count?*
Die Lebensdauer ist die Zahl der Router-Sprünge, die das Paket ausführen darf. Bei Multicast-Paketen bestimmt sie die Verteilung. Die Systemvorgabe für Multicast ist 1 und beschränkt das Paket auf das lokale Subnetz. Sollen Pakete Router passieren, ist der TTL-Wert zu erhöhen. 31 bleibt in der Regel innerhalb eines Standorts, 255 lässt das Paket das gesamte Netz durchqueren.

-mcastadd groupaddr
... siehe den folgenden Eintrag ...

-mcastadd "groupaddr netwif"
... siehe den folgenden Eintrag ...

-mcastdrop groupaddr
... siehe den folgenden Eintrag ...

-mcastdrop "groupaddr netwif"
... siehe den folgenden Eintrag ...

-mcastgroups
Sockets von **tcludp** unterstützen Multicast mit IPv4 und IPv6. Damit Multicast-Pakete ankommen, muss die Anwendung dem Betriebssystem mitteilen, dass sie einer bestimmten Multicast-Gruppe beitritt. Bei IPv4 sind das Adressen von 224.0.0.0 bis 239.255.255.255. Steht nur *groupaddr*, wählt das System die Netzschnittstelle. *netwif* tritt einer Multicast-Gruppe auf einer bestimmten Schnittstelle bei. Das ist auf einem System mit mehreren Netzschnittstellen sinnvoll. Unter Windows ist der Index der Netzschnittstelle anzugeben. Auf anderen Plattformen kann der Name der Schnittstelle stehen, zum Beispiel 'eth0'. Die aktuellen Multicast-Gruppen eines Kanals zeigt *-mcastgroups*.

-mcastif
Gibt zurück, welche Schnittstelle ausgehende Multicast-Pakete verwendet. Nur UNIX.

-mcastloop*?boolean?*
Bei Multicast-UDP kann das System Pakete empfangen, die es selbst gesendet hat, oder sie verwerfen. Das ist die Multicast-Schleife und lässt sich mit dieser Option steuern. Voreingestellt ist der Wert true, und die Anwendung empfängt ihre eigenen Sendungen.

udp_peek *channel* ?*buffersize*?
Sieht ein Paket an, ohne es aus dem Puffer zu nehmen. Die Option *buffersize* legt die größte Puffergröße fest. Der Wert liegt zwischen 0 und 16. Unter Windows gibt es diese Funktion nicht.

::udp::build-info
Gibt Informationen zur Bauumgebung zurück.

::udp::getaddrinfo -hostname *name?optionName value ...?*
Gibt eine Liste mit Angaben zu der IP-Adresse zurück, die zu den Parametern passt. Gültige Optionen:

-hostname *name*
Legt fest, welche IP-Adresse nachgeschlagen wird. Zulässig sind Adresse oder Name. Pflichtoption.

-port *number*
Legt fest, welche Portnummer nachgeschlagen wird.

-service *name*
Legt fest, welcher bekannte Dienst (also http, https und so weiter) beim Nachschlagen den Port bestimmt.

-ipv4
Gibt nur IPv4-Adressen zurück.

-ipv6
Gibt nur IPv6-Adressen zurück.

-server
Gibt nur Dienste zurück, für die der Rechnername ein Server ist.

-tcp
Gibt nur TCP-Dienste zurück.

-udp
Gibt nur UDP-Dienste zurück.

::udp::getnameinfo *address*?ipv6?
::udp::getnameinfo gibt die Namen zur IP-Adresse *address* zurück. Mit *ipv6* ist die Adresse eine IPv6-Adresse.

BEISPIELE

```
# Send data to a remote UDP socket
proc udp_puts {host port} {
    set s [udp_open]
    fconfigure $s -remote [list $host $port]
    puts $s "Hello, World"
    close $f
}
```

```
# A simple UDP server
package require udp

proc udpEventHandler {sock} {
    set pkt [read $sock]
    set peer [fconfigure $sock -peer]
    puts "$peer: [string length $pkt] {$pkt}"
    return
}

proc udp_listen {port} {
    set srv [udp_open $port]
    fconfigure $srv -buffering none -translation binary
    fileevent $srv readable [list ::udpEventHandler $srv]
    puts "Listening on udp port: [fconfigure $srv -myport]"
    return $srv
}

set sock [udp_listen 53530]
vwait forever
close $sock
```

```
# A multicast demo.
proc udpEvent {chan} {
    set data [read $chan]
    set peer [fconfigure $chan -peer]
    puts "$peer [string length $data] '$data'"
    if {[string match "QUIT*" $data]} {
        close $chan
        set ::forever 1
    }
    return
}

set group 224.5.1.21
set port  7771
set s [udp_open $port]
fconfigure $s -buffering none -blocking 0
fconfigure $s -mcastadd $group -remote [list $group $port]
fileevent $s readable [list udpEvent $s]
puts -nonewline $s "hello, world"
set ::forever 0
vwait ::forever
exit
```

GESCHICHTE

Ein Teil des Codes dieser Erweiterung stammt aus dem Paket tcludp von Michael Miller (http://www.neosoft.com/tcl/ftparchive/sorted/comm/tcludp-1.0/). Gegenüber Michaels UDP-Erweiterung bietet diese Erweiterung Windows und das Lesen und Schreiben des Sockets mit 'gets/puts'. Dazu kommen weitere Einstellungen.

Erweiterungen für Binärdaten und die Einrichtung des Pakets für die Tcl Extension Architecture stammen von Pat Thoyts.

Die Unterstützung für IPv6 und den Beitritt zu einer Multicast-Gruppe auf einer bestimmten Netzschnittstelle hat Huub Eikens ergänzt.

COPYRIGHT

```
Copyright (c) 1999-2000 Columbia University; all rights reserved
```

tclvfs

vfs-filesystems

Stichwörter: vfs-filesystems, file, filesystem, ftp, http, metakit, namespace, tar, vfs, vfslib, webdav, zip

NAME

vfs-filesystems - Dateisysteme von tclvfs

ÜBERSICHT

package require Tcl 8.4
package require vfs ?**1.2.1**?
package require vfs::zip ?**1.0**?
package require vfs::mk4 ?**1.6**?
package require vfs::tar ?**0.9**?
package require vfs::ftp ?**1.0**?
package require vfs::ns ?**1.0**?
package require vfs::webdav ?**0.1**?
package require vfs::http ?**0.5**?
package require vfs::urltype ?**1.0**?
vfs::zip::Mount *path to*
vfs::mk4::Mount *path to*
vfs::tar::Mount *path to*
vfs::ftp::Mount *path to*
vfs::ns::Mount *path to*
vfs::webdav::Mount *path to*
vfs::http::Mount *path to*
vfs::urltype::Mount *path to*

BESCHREIBUNG

Das Paket **vfs** liefert nicht nur die Mittel, ein virtuelles Dateisystem auf Tcl-Ebene zu schreiben, sondern auch eine Reihe fertiger Dateisysteme.

Jedes dieser Dateisysteme ist ein eigenes Paket und wird mit **package require vfs::NAME**

geladen. Die ganze Sammlung heißt informell *vfslib*.

UNTERSTÜTZTE VFS-ARTEN

Derzeit unterstützt sind ftp, tar, http, zip, mk4, ns und webdav. Dazu lässt sich jeder 'urltype' als neues Verzeichnis einhängen, sofern ein passendes vfs vorhanden ist. Damit lassen sich URLs mit den Schemata *ftp://*, *http://* und *file://* wie Dateien behandeln. Dafür wird zum Beispiel der Befehl **vfs::urltype::Mount ftp** ausgewertet. Jeder Zugriff im neuen Volume versucht, ein Paket mit **package require vfs::${type}** zu laden. Das Paket muss vorhanden sein, sonst entsteht ein Fehler. Ist ein Dateisystem geladen, hängt der zugehörige Befehl unten eine Quelle dieses Dateisystems als Tcl-Verzeichnis ein.

vfs::zip::Mount *path to*
: Hängt die Zip-Datei *path* als Verzeichnis *to* ein.

vfs::mk4::Mount *path to*
: Hängt die Metakit-Datenbankdatei *path* als Verzeichnis *to* ein.

vfs::tar::Mount *path to*
: Hängt die Tar-Datei *path* als Verzeichnis *to* ein.

vfs::ftp::Mount *path to*
: Hängt die FTP-URL *path* als Verzeichnis *to* ein.

vfs::ns::Mount *path to*
: Hängt den Tcl-Namespace *path* als Verzeichnis *to* ein.

vfs::webdav::Mount *path to*
: Hängt die WebDAV-URL *path* als Verzeichnis *to* ein.

vfs::http::Mount *path to*
: Hängt die HTTP-URL *path* als Verzeichnis *to* ein.

vfs::urltype::Mount *path to*
: Hängt die URL *path* vom Typ *urltype* als Verzeichnis *to* ein.

EINSCHRÄNKUNGEN

Die meisten der genannten vfs-Arten sind noch wenig geprüft. Sie sind zu testen.

COPYRIGHT


```
Copyright (c) 2001-2003 Vince Darley
    <vincentdarley@users.sourceforge.net>
Copyright (c) 2003 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


vfs-fsapi

Stichwörter: vfs-fsapi, file, filesystem, vfs

NAME

vfs-fsapi - API für die Umsetzung eines Dateisystems in Tcl

ÜBERSICHT

package require Tcl 8.4
package require vfs ?**1.2.1**?

vfshandler *subcmd root relative actualpath args...*
vfshandler access *root relative actualpath mode*
vfshandler createdirectory *root relative actualpath*
vfshandler deletefile *root relative actualpath*
vfshandler fileattributes *root relative actualpath* ?*index*? ?*value*?
vfshandler matchindirectory *root relative actualpath pattern types*
vfshandler open *root relative actualpath mode permissions*
vfshandler removedirectory *root relative actualpath recursive*
vfshandler stat *root relative actualpath*
vfshandler utime *root relative actualpath actime mtime*
vfs::accessMode *mode*
vfs::matchDirectories *types*
vfs::matchFiles *types*
vfs::matchCorrectTypes *types filelist* ?*inDir*?

BESCHREIBUNG

Dieses Dokument beschreibt die API, mit der das Paket **vfs** mit Dateisystem-Umsetzungen in Tcl spricht.

ÜBERBLICK ÜBER DEN HANDLER

Das Paket **vfs** fängt jede Dateisystemoperation ab, die in einen Einhängepunkt fällt, und gibt sie an den Befehl **vfshandler** des Einhängepunkts in dem Interpreter weiter, der ihn eingetragen hat.

Behandelt der Handler jeden Fall, für den er aufgerufen wird, passend, entsteht ein vollständiges virtuelles Dateisystem, das Tcl vom nativen Dateisystem nicht unterscheidet. (Einschränkung: derzeit reicht **vfs** nicht alle berechtigungsbezogenen Flags von **glob** an Tcl durch.)

vfshandler *subcmd root relative actualpath args...*

Das erste Argument nennt die Operation, die der Handler für den Dateisystemcode im Tcl-Kern ausführt. Der Rest nennt den Dateipfad, auf dem sie arbeitet, in verschiedenen Formen und Teilen, und weitere Argumente, die die Aktion braucht. Als Beispiel für die Behandlung eines Pfads durch die allgemeine Schicht dient "*C:/foo/bar/mount.zip/xxx/yyy*". Zusätzlich gelten die folgenden Bedingungen:

[1]
"*mount.zip*" ist ein Zip-Archiv, das auf sich selbst eingehängt ist,

[2]
dieses Zip-Archiv enthält eine Datei mit dem Pfad "*xxx/yyy*",

[3]
das Arbeitsverzeichnis der Anwendung liegt im Verzeichnis "*xxx*",

[4]
und der ausgeführte Befehl ist **file exists yyy**.

Das Trennzeichen zwischen *root* und *relative* entfällt. Die meisten Dateisystemoperationen brauchen für ein richtiges Ergebnis nur das Argument *relative*. Manche brauchen die anderen Teile des Pfads.

subcmd

Dieses Argument des Handlers ist eines von **access**, **createdirectory**, **deletefile**, **fileattributes**, **matchindirectory**, **open**, **removedirectory**, **stat** oder **utime**. Die allgemeine Schicht erwartet, dass die Unterbefehle eines Handlers Fehler mit **vfs::filesystem posixerror** und dem passenden POSIX-Fehlercode melden, statt einen Tcl-Fehler zu werfen. Geschieht das dennoch, gilt er als unbekannter POSIX-Fehler. Drei Ausnahmen: Werfen **open** (wenn ein Interpreter übergeben wird), **matchindirectory** oder **fileattributes** (nur bei einer set- oder get-Operation) einen Tcl-Fehler, geht dieser Fehler an den Aufrufer des Dateisystembefehls, der den Handler aufgerufen hat. Diese Unterbefehle dürfen trotzdem **vfs::filesystem posixerror** verwenden, um gewöhnliche Dateisystemfehler zu melden.

root
Teil der Angabe des Pfads, auf dem die Operation arbeitet. Enthalten ist der Teil des Pfads außerhalb des Einhängepunkts dieses Dateisystems. Im Beispiel oben ist der Wert "*C:/foo/bar/mount.zip*".

relative
Teil der Angabe des Pfads, auf dem die Operation arbeitet. Enthalten ist der Teil des Pfads innerhalb dieses Dateisystems. Im Beispiel oben ist der Wert "*xxx/yyy*".

actualpath
Teil der Angabe des Pfads, auf dem die Operation arbeitet. Enthalten ist der ursprüngliche, nicht normalisierte Name des Pfads, wie ihn der aktuelle Befehl verwendet hat, egal ob er aus Tcl oder aus C kam. Im Beispiel oben ist der Wert "*yyy*".

HANDLER-METHODEN

vfshandler access *root relative actualpath mode*
Meldet einen POSIX-Fehler, wenn der angegebene Zugriffsmodus *mode* (eine ganze Zahl) nicht zur Datei oder zum Verzeichnis des Pfads passt. Die allgemeine Schicht ignoriert jeden nicht leeren Rückgabewert. Der Befehl **vfs::accessMode** (siehe Abschnitt **HANDLER-UMGEBUNG**) wandelt die ganze Zahl *mode* in eine leichter zu prüfende Zeichenkette um.

vfshandler createdirectory *root relative actualpath*
Legt ein Verzeichnis mit dem angegebenen Namen an. Der Befehl darf annehmen, dass alle Unterverzeichnisse im Pfad vorhanden und gültig sind und dass der gewünschte Pfad noch nicht existiert. Darum kümmert sich Tcl.

vfshandler deletefile *root relative actualpath*
Löscht die angegebene Datei.

vfshandler fileattributes *root relative actualpath* ?*index*? ?*value*?
Der Befehl gibt eine Liste mit den Namen aller zulässigen Attribute zurück, wenn weder *index* noch *value* angegeben sind. Er gibt den Wert des *index*-ten Attributs zurück, wenn *index* angegeben ist, *value* aber nicht. Gezählt wird in der Reihenfolge der Namen aus dem Aufruf ohne *index* und ohne *value*. Das erste Attribut hat den Index **0**. Sind *index* und *value* angegeben, setzt der Befehl das *index*-te Attribut auf *value*.

vfshandler matchindirectory *root relative actualpath pattern types*
Gibt die Liste der Dateien oder Verzeichnisse im angegebenen Pfad zurück, die zum Glob-Muster *pattern* passen und zur angegebenen Liste *types*. Der Pfad ist immer der Name eines vorhandenen Verzeichnisses. *Hinweis:* Bei jedem rekursiven Globbing erzeugt Tcl bei

den beteiligten Dateisystemen Anforderungen, die nur Verzeichnisse treffen. Dieser Unterbefehl muss solche Anforderungen und solche nur für Dateien richtig behandeln, sonst geht es schief. Die Befehle **vfs::matchDirectories** und **vfs::matchFiles** (siehe Abschnitt **HANDLER-UMGEBUNG**) helfen dabei erheblich.

vfshandler open *root relative actualpath mode permissions*

Gibt entweder eine Liste zur erfolgreich geöffneten Datei zurück oder wirft einen Fehler, der das Misslingen beschreibt. Die Liste bei Erfolg hat mindestens ein und höchstens zwei Elemente. Das erste Element ist Pflicht und das Handle des Kanals, der den Zugriff auf den Inhalt der Datei ermöglicht. Ist ein zweites Element angegeben, gilt es als Rückruffunktion, also als Befehlspräfix. Dieses Präfix wird so ausgeführt, wie es dasteht, ohne zusätzliche Argumente. Nötige Argumente gehören zum Ergebnis des Aufrufs von **open**. Ist die Rückruffunktion vorhanden, wird sie unmittelbar vor dem Schließen des Kanals *durch die allgemeine Dateisystemschicht* ausgewertet. Die Rückruffunktion selbst darf **close** nicht aufrufen. Der Kanal ist noch lebendig genug für **seek** und **read**. Alle vorhandenen Daten sind bereits hineingeschrieben. Die Rückruffunktion kann also an den Anfang dieses Kanals gehen, den Inhalt lesen und die gesammelten Daten anderswo ablegen. Sie räumt damit die Ressourcen einer geöffneten Datei auf und macht ein beschreibbares Dateisystem überhaupt erst möglich. Im Normalfall werden Rückgabecode und Fehler der Rückruffunktion ignoriert. Fehler sind dann asynchron zu melden, zum Beispiel mit **bgerror**. Ist über den Unterbefehl **internalerror** ein Skript zur Fehlerbehandlung für das Dateisystem angegeben, gehen alle hier geworfenen Fehler an dieses Skript.

mode

ist einer der Werte **r**, **w**, **a**, **w+** oder **a+**.

permissions

bestimmt den nativen Modus, mit dem die geöffnete Datei angelegt wird. Relevant nur, wenn der Öffnungsmodus *mode* das Anlegen einer noch nicht vorhandenen Datei verlangt, also nicht **r** ist.

vfshandler removedirectory *root relative actualpath recursive*

Löscht das angegebene Verzeichnis. Das Argument *recursive* ist ein boolescher Wert. Ist der Wert **true**, wird auch ein nicht leeres Verzeichnis mitsamt Inhalt rekursiv gelöscht. Ist der Wert **false** und das Verzeichnis nicht leer, ist ein POSIX-Fehler (**EEXIST**) zu werfen.

vfshandler stat *root relative actualpath*

Das Ergebnis ist eine Liste aus Schlüsseln und Werten in einer Form, die der eingebaute Befehl **array set** annimmt. Sie beschreibt den Inhalt einer stat-Struktur. Die Reihenfolge der Schlüssel ist gleichgültig. Der Unterbefehl endet daher etwa so:

```
return [list dev 0 type file mtime 1234 ...].
```

als letzten Befehl seiner Umsetzung. Das Dateisystem liefert die folgenden Schlüssel und ihre Werte:

dev

Eine lange ganze Zahl, die Gerätenummer des Pfads, für den stat aufgerufen wurde.

ino

Eine lange ganze Zahl, die Inode-Nummer des Pfads, für den stat aufgerufen wurde. Jeder Pfad, den das Dateisystem behandelt, soll durch die Kombination aus Gerätenummer und

Inode-Nummer eindeutig sein. Wird das verletzt, scheitern Algorithmen höherer Ebenen, die Gerät und Inode verfolgen müssen, auf jede erdenkliche Weise. Ein Beispiel ist ein Verzeichnisdurchlauf, der sich mit Gerät und Inode aus Endlosschleifen durch symbolische Links heraushält. Nicht eindeutige Angaben zu Gerät und Inode lassen einen solchen Durchlauf Pfade überspringen, weil er sie fälschlich schon gesehen hat.

mode
Eine ganze Zahl, der Zugriffsmodus des Pfads. Diesen Modus prüft der Unterbefehl **access**.

nlink
Eine lange ganze Zahl, die Zahl der harten Links auf den angegebenen Pfad.

uid
Eine lange ganze Zahl, die Kennung des Benutzers, dem der virtuelle Pfad gehört.

gid
Eine lange ganze Zahl, die Kennung der Benutzergruppe, zu der der virtuelle Pfad gehört.

size
Eine lange ganze Zahl, die wirkliche Größe des virtuellen Pfads in Bytes.

atime
Eine lange ganze Zahl, der Zeitpunkt des letzten Zugriffs auf den Pfad, in Sekunden seit der Epoche. Der Befehl **clock format** macht daraus ein lesbares Datum.

mtime
Eine lange ganze Zahl, der Zeitpunkt der letzten Änderung des Pfads, in Sekunden seit der Epoche. Der Befehl **clock format** macht daraus ein lesbares Datum.

ctime
Eine lange ganze Zahl, der Zeitpunkt, zu dem der Pfad angelegt wurde, in Sekunden seit der Epoche. Der Befehl **clock format** macht daraus ein lesbares Datum.

type
Eine Zeichenkette, entweder **directory** oder **file**, die den Typ des Pfads beschreibt.

vfshandler utime *root relative actualpath actime mtime*
Setzt Zugriffs- und Änderungszeit der angegebenen Datei. Gelesen werden sie mit **stat**.

HANDLER-UMGEBUNG

Die Umsetzung eines Dateisystem-Handlers kann sich auf die folgenden Hilfsbefehle stützen:

vfs::accessMode *mode*
Dieser Befehl wandelt einen Zugriffsmodus *mode* als ganze Zahl in eine Zeichenkette um, eine von **F**, **X**, **W**, **XW**, **R**, **RX** und **RW**.

vfs::matchDirectories *types*
Prüft, ob die Glob-Typangabe Verzeichnisse einschließen will. Das Ergebnis ist boolesch. **true** kommt zurück, wenn types Verzeichnisse verlangt, sonst **false**.

vfs::matchFiles *types*
Prüft, ob die Glob-Typangabe Dateien einschließen will. Das Ergebnis ist boolesch. **true** kommt zurück, wenn types Dateien verlangt, sonst **false**.

vfs::matchCorrectTypes *types filelist* ?*inDir*?
Gibt den Teil der *filelist* zurück, der zu den angegebenen *types* passt. Die Elemente von *filelist* sind absolute Pfade oder Namen von Dateien im Verzeichnis *indir*. Die zweite Lesart gilt genau dann, wenn das Argument *indir* angegeben ist.

FEHLERSUCHE IM DATEISYSTEM

Zum Untersuchen eines Problems in der Umsetzung eines Dateisystems dient der Code unten. Er trägt den Befehl **report** als Fehlerbehandler des Dateisystems ein. Der gibt den Fehlerstapel aus, den Tcl liefert.

```
vfs::filesystem internalerror report

proc report {} {
    puts stderr $::errorInfo
}
```

COPYRIGHT

```
Copyright (c) 2001-2003 Vince Darley
    <vincentdarley@users.sourceforge.net>
Copyright (c) 2003 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

vfs

Stichwörter: vfs, filesystem, file

NAME

::vfs - Befehle und Prozeduren zum Anlegen virtueller Dateisysteme

ÜBERSICHT

package require Tcl

package require vfs ?**1.4**?

vfs::filesystem *info*

vfs::filesystem *mount*

vfs::filesystem *unmount*

vfs::accessMode *mode*

vfs::matchDirectories *types*

vfs::matchFiles *types*

vfs::matchCorrectTypes *types filelist* ?*inDir*?

BESCHREIBUNG

Das Paket **::vfs** liefert Befehle, um virtuelle Dateisysteme abzufragen, einzuhängen und auszuhängen, und als Tcl-Bibliotheken Hilfen zum Schreiben neuer virtueller Dateisysteme in Tcl. Sobald ein virtuelles Dateisystem steht, gelten darin die Tcl-Befehle **file**, **glob**, **cd**, **pwd** und **open**, einschließlich ihrer C-Schnittstellen in der Tcl-Bibliothek (zum Beispiel **Tcl_FSOpenFileChannel**, **Tcl_FSMatchInDirectory**, ...). Sauber geschriebene Erweiterungen wie Tk, die Dateien öffnen oder lesen, erreichen das virtuelle Dateisystem ebenfalls. Weil die gesamte Dateisystemarbeit von Tcl durch eine Schicht läuft, lässt sie sich abfangen. Genau das tut dieses Paket. Das ist etwas anderes, als den Befehl **file** in Tcl zu überladen. Ersetzt werden C-Befehle wie **access** und **stat**. Wenige Befehle auf dieser niedrigen Ebene genügen, damit alle

Befehle der höheren Ebenen arbeiten, unabhängig davon, was in der Dateisystemschicht geschieht.

Die Dateisystem-Hooks von Tcl gelten je Prozess. Jeder Tcl-Interpreter im selben Prozess sieht dasselbe Dateisystem, einschließlich der virtuellen Dateisysteme.

Der Befehl **package require vfs** lädt diese Bibliothek. Er trägt die vfs-Hooks in das Dateisystem von Tcl ein, und sie bleiben, bis Tcl endet (eine Steuerung dafür könnte später an Tcl durchgereicht werden). Zu diesem Zeitpunkt ist noch kein neues Dateisystem eingehängt, die Wirkung bleibt also klein. **package require vfs** hat zwei Wirkungen. Erstens sorgt der Aufruf in **jedem** Tcl-Interpreter dafür, dass die vfs-Hooks genau einmal im Kern von Tcl eingetragen werden. Werden Interpreter später gelöscht, bleiben die Hooks eingetragen, bis Tcl endet. Zweitens stellt der Aufruf den Befehl **vfs::filesystem** bereit. Damit fängt der Interpreter Dateisystembefehle ab und behandelt sie mit Tcl-Code in diesem Interpreter.

Drei wenig unterstützte Unterbefehle von **vfs::filesystem** sind **fullynormalize path**, **posixerror int** und **internalerror ?script?**. Sie normalisieren einen Pfad einschließlich eines symbolischen Links am Ende, ordnen einen POSIX-Fehlercode einem Tcl-Fehler zu und fangen interne Fehler in tclvfs-Umsetzungen ab.

vfs::filesystem *mount?-volume? path command*
: Ein virtuelles Dateisystem muss eingehängt werden. Dabei erklärt man dem Paket vfs, dass alle Unterverzeichnisse eines *path* im Dateisystem von dem angegebenen *command* behandelt werden. Das ist ein Tcl-Befehl oder eine Prozedur in dem Interpreter, in dem **vfs::filesystem** ausgeführt wird. Mit dem Flag*?-volume?* wird der Einhängepunkt bei Tcl zusätzlich als neues Volume eingetragen, wie ein neues Laufwerk, das in *file volumes* erscheint. Das ist sinnvoll und für einen brauchbaren Betrieb nötig bei Einhängungen wie *ftp://*. Für Pfade innerhalb des nativen Dateisystems wird das Flag nicht angegeben. Die neuen Einhängungen gelten sofort in allen Interpretern des Prozesses. Wird der Interpreter später gelöscht, fallen alle Einhängungen weg, die er abfängt, und damit ändert sich die Sicht aller Interpreter auf das Dateisystem.

vfs::filesystem *unmount path*
: Hängt das virtuelle Dateisystem aus, das unter *path* eingehängt war, und nimmt es damit aus dem Dateisystem von Tcl. War dort nichts eingehängt, entsteht ein Fehler.

vfs::filesystem *info?path?*
: Ohne Argumente gibt der Befehl die Liste aller eingehängten Dateisysteme zurück, über alle Interpreter. Mit einem Pfadargument kommt das *command* für diesen Pfad zurück. Ist für den Pfad kein vfs eingehängt, entsteht ein Fehler. Derzeit lässt sich nicht feststellen, in welchem Interpreter jeder Befehl ausgewertet wird.

vfs::filesystem *fullynormalize path*
: Expandiert *path* vollständig, wie 'file normalize', und folgt dabei auch Links im letzten Pfadbestandteil.

EIN VIRTUELLES DATEISYSTEM NUR IN TCL

Das Paket vfs fängt jede Dateisystemoperation ab, die in einen Einhängepunkt fällt, und gibt sie an das *command* des Einhängepunkts in dem Interpreter weiter, der ihn eingetragen hat. Das entspricht auf C-Ebene einer Auswertung wie *eval $command [list $subcmd $root $relative $actualpath] $args*.

subcmd ist einer der Werte *access*, *createdirectory*, *deletefile*, *fileattributes*, *matchindirectory*, *open*, *removedirectory*, *stat*, *utime*. Behandelt *command* jeden dieser Fälle passend, entsteht ein vollständiges virtuelles Dateisystem, das Tcl vom nativen Dateisystem nicht unterscheidet. (Einschränkung: derzeit werden nicht alle berechtigungsbezogenen Flags von 'glob' an Tcl durchgereicht.)

Die übrigen Argumente nennen den Dateipfad, auf dem die Operation arbeitet (jeder Befehl arbeitet auf einem solchen Pfad), und weitere Argumente, die die Aktion braucht. Der Dateipfad besteht aus drei Argumenten: *root* ist der Teil des Pfads außerhalb des Einhängepunkts dieses Dateisystems, *relative* ist der Teil innerhalb dieses Dateisystems, und *actualpath* ist der ursprüngliche, nicht normalisierte Name des Pfads, wie ihn der aktuelle Befehl verwendet hat, egal ob er aus Tcl oder aus C kam. Ist zum Beispiel *C:/foo/bar/mount.zip/xxx/yyy* ein Pfad im Dateisystem, *mount.zip* ein Zip-Archiv, das auf sich selbst eingehängt ist und *xxx/yyy* enthält, das Arbeitsverzeichnis liegt in *xxx*, und ausgewertet wird ein Befehl wie *file exists yyy*, dann ist *root* gleich C:/foo/bar/mount.zip, *relative* ist *xxx/yyy* und *actualpath* ist *yyy*. Das Trennzeichen zwischen *root* und *relative* entfällt.

Die meisten Dateisystemoperationen brauchen für ein richtiges Ergebnis nur das Argument *relative*. Für manche Unterbefehle sind die anderen Argumente nötig.

Fast alle diese Befehle kehren entweder ordnungsgemäß zurück (auf C-Ebene mit dem Ergebnis TCL_OK) oder sie melden den passenden POSIX-Fehlercode mit vfs::filesystem posixerror. Ein geworfener Tcl-Fehler gilt als Fehler der Umsetzung und wird als unbekannter POSIX-Fehler des Dateisystemaufrufs gelesen. Ausgenommen sind die Dateisystembefehle, die eine Tcl-Fehlermeldung direkt angeben dürfen: open (wenn ein Interpreter übergeben wird), matchindirectory und fileattributes (nur bei einer set- oder get-Operation). Diese drei Befehle dürfen jede Tcl-Fehlermeldung werfen; sie geht an den Aufrufer. Sie dürfen auch einen POSIX-Fehler werfen, der passend behandelt wird.

Die Befehle im Einzelnen (*r-r-a* steht für das übliche Argumenttripel *root*, *relative* und *actualpath*):

command access r-r-a mode
: Gibt TCL_OK zurück oder wirft einen POSIX-Fehler, je nachdem, ob der angegebene Zugriffsmodus (eine ganze Zahl) zur Datei passt.

command createdirectory r-r-a
: Legt ein Verzeichnis mit dem angegebenen Namen an. Der Befehl darf annehmen, dass alle Unterverzeichnisse im Pfad vorhanden und gültig sind und dass der gewünschte Pfad noch nicht existiert. Darum kümmert sich Tcl.

command deletefile r-r-a
: Löscht die angegebene Datei.

command fileattributes r-r-a?index??value?
: Sind weder index noch value angegeben, kommt die Liste aller zulässigen Attributnamen zurück. Ist *index* angegeben, aber kein Wert, liefert der Befehl den Wert des *index*-ten Attributs der Datei, gezählt in der Reihenfolge der Liste ohne Argumente. Ist auch ein Wert angegeben, setzt der Befehl das *index*-te Attribut der Datei auf diesen Wert.

command matchindirectory r-r-a pattern types
: Gibt die Liste der Dateien oder Verzeichnisse im angegebenen Pfad zurück. Der Pfad ist immer der Name eines vorhandenen Verzeichnisses. Die Einträge passen zum *pattern* und zu

den angegebenen *types*. Wichtig ist, dass der Befehl *types*-Anforderungen richtig behandelt, die nur Verzeichnisse (oder nur Dateien) verlangen: Für jedes rekursive Globbing erzeugt Tcl selbst beim Dateisystem Anforderungen, die nur Verzeichnisse treffen. Hilfe dazu steht unten bei **vfs::matchDirectories**.

command open r-r-a mode permissions
mode ist einer der Werte "r", "w", "a", "w+", "a+". Wird dabei eine Datei angelegt, bestimmen *permissions* die Modi dafür. Schlägt das Öffnen fehl, ist ein Fehler zu werfen. Gelingt es, gibt der Befehl eine Liste mit einem oder zwei Elementen zurück. Das erste Element ist Pflicht und der Name des angelegten Kanals. Das zweite Element, falls vorhanden, ist eine Tcl-Rückruffunktion für das Schließen des Kanals, damit das vfs aufräumen kann. Tcl wertet diese Rückruffunktion unmittelbar vor dem Schließen des Kanals aus. Der Kanal besteht noch, und alle vorhandenen Daten sind hineingeschrieben. Die Rückruffunktion kann zum Beispiel an den Anfang des Kanals gehen, den Inhalt lesen und ihn anderswo ablegen, etwa komprimiert oder auf einem entfernten FTP-Server. Rückgabecode und Fehler der Rückruffunktion werden ignoriert, es sei denn, das Skript 'internalerror' ist angegeben; dann gehen sie an dieses Skript. Will die Rückruffunktion einen Fehler melden, muss sie das asynchron tun, zum Beispiel mit bgerror.

command removedirectory r-r-a recursive
Löscht das angegebene Verzeichnis. *recursive* ist 0 oder 1. Bei 1 wird auch ein nicht leeres Verzeichnis mitsamt Inhalt rekursiv gelöscht. Bei 0 und einem nicht leeren Verzeichnis ist ein POSIX-Fehler (EEXIST) zu werfen.

command stat r-r-a
Gibt eine Liste gerader Länge mit Paaren aus Feldname und Wert für den Inhalt einer stat-Struktur zurück. Die Reihenfolge ist gleichgültig. Die Optionsnamen sind dev (long), ino (long), mode (int), nlink (long), uid (long), gid (long), size (long), atime (long), mtime (long), ctime (long), type (eine Zeichenkette, entweder "directory" oder "file"). Der Typ jedes Arguments steht in Klammern. Die Prozedur kehrt daher etwa so zurück: *return [list dev 0 type file mtime 1234 ...]*.

command utime r-r-a actime mtime
Setzt Zugriffs- und Änderungszeit der angegebenen Datei. Gelesen werden sie mit 'stat'.

VFS-HILFEN

Die vfslib stellt Tcl-Prozeduren bereit, die beim Schreiben der Befehlsprozeduren für die genannten Fälle helfen. Das sind:

vfs::accessMode *mode*
Wandelt einen ganzzahligen Zugriffsmodus *access* in eine handlichere Zeichenkette um, eine von F, X, W, XW, R, RX, RW.

vfs::matchDirectories *types*
Sollen laut *types* Verzeichnisse enthalten sein?

vfs::matchFiles *types*
Sollen laut *types* Dateien enthalten sein?

vfs::matchCorrectTypes *types filelist?inDir?*
Gibt den Teil der *filelist* zurück, der zu den angegebenen *types* passt. Die Elemente sind absolute Pfade oder Namen von Dateien in *inDir*.

VFS-FEHLERSUCHE

Zum Untersuchen von Problemen in der Umsetzung dient zum Beispiel: vfs::filesystem internalerror report; proc report {} { puts stderr $::errorInfo }

EINSCHRÄNKUNGEN

Der vfs-Code hat sehr wenige Einschränkungen. Eine Feinheit: wird ein virtuelles Dateisystem, das Groß- und Kleinschreibung unterscheidet, in ein System eingehängt, das sie nicht unterscheidet (das übliche Dateisystem von Windows oder MacOS), und der eigene Code verlässt sich auf die Gleichbehandlung, dann läuft er im virtuellen Dateisystem nicht richtig. Code, der sich darauf verlässt, liefe unter Tcl auf Unix ebenfalls nicht. Der Ausweg ist, den Code zu korrigieren.

Später können die Befehle *link* und *lstat* dazukommen, damit virtuelle Dateisysteme Links lesen und schreiben. Die C-Schnittstelle kann das. Diese Erweiterung reicht es an Tcl noch nicht durch.

Die Tcl-Funktion 'Tcl_FSMatchInDirectory' nimmt verschiedene Typangaben in einer Struktur Tcl_GlobTypeData entgegen. Derzeit wird daraus nur das Feld 'type' durchgereicht. Die Felder 'permissions' und der MacOS-Typ mit Creator bleiben unbeachtet.

Copyright (c) 2001-2003, Vince Darley

vfslib

Stichwörter: vfslib, vfs, filesystem, zip, tar, webdav, namespace, ftp, http, file

NAME

::vfslib - Prozeduren für virtuelle Dateisysteme

ÜBERSICHT

package require Tcl

package require vfs ?**1.2.1**?

package require vfs::zip ?**1.0**?

package require vfs::mk4 ?**1.6**?

package require vfs::tar ?**0.9**?

package require vfs::ftp ?**1.0**?

package require vfs::ns ?**1.0**?

package require vfs::webdav ?**0.1**?

package require vfs::http ?**0.5**?

package require vfs::urltype ?**1.0**?

vfs::zip::Mount *path to*

vfs::ftp::Mount *path to*

vfs::tar::Mount *path to*

vfs::http::Mount *path to*

vfs::mk4::Mount *path to*

vfs::ns::Mount *path to*

vfs::urltype::Mount *type*

BESCHREIBUNG

Das Paket **::vfs** enthält eine Bibliothek aus Tcl-Code mit mehreren virtuellen Dateisystemen. Jedes davon ist ein eigenes Paket und wird mit **package require vfs::NAME** geladen. Die ganze Sammlung heißt informell 'vfslib'.

UNTERSTÜTZTE VFS-ARTEN

Derzeit unterstützt sind ftp, tar, http, zip, mk4, ns und webdav. Dazu lässt sich jeder 'urltype' als neues Volume einhängen, sofern ein passendes vfs vorhanden ist. Damit lassen sich URLs der Form 'ftp://', 'http://' und 'file://' wie Dateien behandeln. Dafür wird zum Beispiel der Befehl

vfs::urltype::Mount ftp

ausgewertet. Jeder Zugriff im neuen Volume versucht, ein Paket mit 'package require vfs::${type}' zu laden. Das Paket muss vorhanden sein, sonst entsteht ein Fehler.

EINSCHRÄNKUNGEN

Die meisten der genannten vfs-Arten sind noch wenig geprüft. Sie sind zu testen.

Copyright (c) 2001, Vince Darley

tclws

Calling_a_Web_Service

Stichwörter: Calling_a_Web_Service

Einen Webdienst aus Tcl aufrufen

INHALT

- 1 Überblick
- 2 Das Paket Webservices Client laden
- 3 WSDL eines entfernten Webdienst-Servers abfragen und zerlegen
- 4 Eine gespeicherte WSDL zerlegen
- 5 Eine vorzerlegte WSDL laden
- 6 Einen REST-Dienst von Hand definieren
- 7 Transformationen definieren
- 8 Synchroner Aufruf, der ein Dictionary-Objekt zurückgibt
- 9 Asynchroner Aufruf mit getrennten Rückruffunktionen für Erfolg und Fehler
- 10 Erzeugen von Stub-Prozeduren in Tcl für synchrone Aufrufe
- 11 Synchroner Aufruf, der das rohe XML zurückgibt
- 12 Dienste konfigurieren
- 13 Abstrakte Typen in konkrete Typen umwandeln

ÜBERBLICK

Das Paket Webservices Client bietet mehrere Wege, festzulegen, welche Operationen ein entfernter Webdienst-Server bereitstellt und wie diese Operationen aufgerufen werden. Es enthält außerdem mehrere Wege, eine Operation aufzurufen.

Die folgenden Wege stehen bereit, um entfernte Operationen zu definieren:

- Eine vorzerlegte WSDL laden
- WSDL eines entfernten Webdienst-Servers abfragen und zerlegen
- Eine gespeicherte WSDL zerlegen
- Einen REST-Dienst von Hand definieren

Das zerlegte Format ist deutlich kompakter als das XML einer WSDL.

Die folgenden Wege stehen bereit, um eine Operation eines Webdienst-Servers direkt aufzurufen:

- Synchroner Aufruf, der ein Dictionary-Objekt zurückgibt
- Synchroner Aufruf, der das rohe XML zurückgibt
- Asynchroner Aufruf mit getrennten Rückruffunktionen für Erfolg und Fehler
- Erzeugen von Stub-Prozeduren in Tcl für synchrone Aufrufe

Dieses Paket verwendet das Paket **log** aus **TclLib**. Insbesondere werden die folgenden Stufen verwendet:

- error/warning -- Fehler, die beim Zerlegen einer WSDL auftreten. Die tatsächliche Stufe hängt von den Optionen ab, die im Paket **::WS::Utils** gesetzt sind.
- info -- HTTP-Aufrufe einschließlich des XML, mit denen Operationen aufgerufen werden, und die empfangenen Antworten. *Eingeführt in 2.2.8.*
- debug -- ausführliche interne Angaben. Das sollte nur verwendet werden, wenn die Interna des Pakets TclWs im Code nachvollzogen werden sollen.

DAS PAKET WEBSERVICES CLIENT LADEN

Das Serverpaket für Webservices wird so geladen:

```
package require WS::Client
```

Dieser Befehl lädt die Hilfsprogramme nur beim ersten Mal, es schadet also nicht, ihn in jede Datei zu setzen, die die Hilfsprogramme verwendet.

WSDL EINES ENTFERNTEN WEBDIENST-SERVERS ABFRAGEN UND ZERLEGEN

Prozedurname: ::WS::Client::GetAndParseWsdl

Beschreibung: Die WSDL-Datei von der angegebenen URL holen, sie zerlegen und den Dienst erzeugen.

Argumente:

```
    url     - Die URL der WSDL
    headers - Zusätzliche Header für die HTTP-Anfrage. Das
              ist ein Schlüssel-Wert-Listenargument. Es muss eine
   Liste mit
              einer geraden Anzahl von Elementen sein, die abwechseln
   zwischen
              Schlüsseln und Werten. Die Schlüssel werden zu
   Headerfeldnamen.
              Zeilenumbrüche werden aus den Werten entfernt, damit
   der Header
```

```
            nicht verfälscht werden kann.
            Das ist ein optionales Argument und ist voreingestellt
   {}.
    serviceAlias - Aliasname (eindeutig) für den Dienst.
                   Das ist ein optionales Argument und ist
   voreingestellt auf den Namen des
                   Dienstes in serviceInfo.
    serviceNumber - Nummer des Dienstes innerhalb der WSDL, dem der
                    serviceAlias zugewiesen wird. Wird nur zusammen
   mit einem serviceAlias erkannt.
                    Der erste Dienst (Vorgabe) wird mit dem Wert "1"
   angesprochen.
```

Der folgende WSDL-Ausschnitt definiert zwei Dienste:

```
>definitions ...<
  >service name="service1"<
    ...
  >/service<
  >service name="service1"<
    ...
  >/service<
>/definitions<
```

Ein leerer oder fehlender *serviceAlias* würde die Dienste "service1" und "service2" erzeugen.

Mit *serviceAlias*="SE" und *serviceNumber*=2 entstünde der Dienst "SE", der "service2" aus der WSDL enthält.

Gibt zurück: Die zerlegte Dienstdefinition

Nebenwirkungen: Keine

Ausnahmebedingungen: Keine

Voraussetzungen: Keine

EINE GESPEICHERTE WSDL ZERLEGEN

Prozedurname: ::WS::Client::ParseWsdl

Beschreibung: Eine WSDL zerlegen und den Dienst erzeugen. Die Stubs ebenfalls erzeugen, wenn das angegeben ist.

Argumente:

```
    wsdlXML - XML der WSDL
```

Optionale Argumente:

```
    -createStubs 0|1 - Stub-Routinen für den Dienst erzeugen
    -headers         - Zusätzliche Header für die HTTP-Anfrage. Das
                       ist ein Schlüssel-Wert-Listenargument. Es muss
   eine Liste mit
```

```
                      einer geraden Anzahl von Elementen sein, die
    abwechseln zwischen
                      Schlüsseln und Werten. Die Schlüssel werden zu
    Headerfeldnamen.
                      Zeilenumbrüche werden aus den Werten entfernt,
    damit der Header
                      nicht verfälscht werden kann.
                      Das ist ein optionales Argument und ist
    voreingestellt {}.
     -serviceAlias    - Aliasname (eindeutig) für den Dienst.
                      Das ist ein optionales Argument und ist
    voreingestellt auf den Namen des
                      Dienstes in serviceInfo.
     serviceNumber - Nummer des Dienstes innerhalb der WSDL, dem der
                     serviceAlias zugewiesen wird. Wird nur zusammen
    mit einem serviceAlias erkannt.
                     Der erste Dienst (Vorgabe) wird mit dem Wert "1"
    angesprochen.
```

Die Argumente sind positionsunabhängig.

Ein Beispiel für *serviceAlias* und *serviceNumber* steht in dem Kapitel oben.

Gibt zurück: Die zerlegte Dienstdefinition

Nebenwirkungen: Keine

Ausnahmebedingungen: Keine

Voraussetzungen: Keine

EINE VORZERLEGTE WSDL LADEN

Prozedurname: ::WS::Client::LoadParsedWsdl

Beschreibung: Eine gespeicherte Dienstdefinition laden in

Argumente:

```
     serviceInfo - zerlegte Dienstdefinition, wie zurückgegeben von
                   ::WS::Client::ParseWsdl oder
    ::WS::Client::GetAndParseWsdl
     headers     - Zusätzliche Header für die HTTP-Anfrage. Das
                     ist ein Schlüssel-Wert-Listenargument. Es muss
    eine Liste mit
                     einer geraden Anzahl von Elementen sein, die
    abwechseln zwischen
                     Schlüsseln und Werten. Die Schlüssel werden zu
    Headerfeldnamen.
                     Zeilenumbrüche werden aus den Werten entfernt,
    damit der Header
                     nicht verfälscht werden kann.
```

```
                  Das ist ein optionales Argument und ist
voreingestellt {}.
 serviceAlias - Aliasname (eindeutig) für den Dienst.
                  Das ist ein optionales Argument und ist
voreingestellt auf den Namen des
                  Dienstes in serviceInfo.
```

Gibt zurück: Der Name des geladenen Dienstes

Nebenwirkungen: Keine

Ausnahmebedingungen: Keine

Voraussetzungen: Keine

EINEN REST-DIENST VON HAND DEFINIEREN

Dienstdefinition
Prozedurname: ::WS::Client::CreateService

Beschreibung: Einen REST-Dienst definieren

Argumente:

```
 serviceName - Dienstname, dem der Namespace hinzugefügt wird
 type        - Der Typ des Dienstes, derzeit wird nur REST
unterstützt.
 url          - URL des Namespace.
 args         - Optionale Argumente:
                        Das ist ein optionales Argument und ist
voreingestellt auf den Namen des
                             -header httpHeaderList.
```

Gibt zurück: Der lokale Alias (tns)

Nebenwirkungen: Keine

Ausnahmebedingungen: Keine

Voraussetzungen: Keine

Methodendefinition
Prozedurname: ::WS::Client::DefineRestMethod

Beschreibung: Eine Methode eines REST-Dienstes definieren

Argumente:

```
 serviceName - Dienstname, dem der Namespace hinzugefügt wird
 methodName  - Der Name der hinzuzufügenden Methode.
 returnType  - Der Typ, den die Prozedur gegebenenfalls
zurückgibt. Das Format ist:
                       xmlTag typeInfo.
 inputArgs   - Liste von Definitionen der Eingabeargumente, wobei
jede
```

```
                          Definition das Format hat: name typeInfo.

    dabei hat typeInfo das Format {type typeName comment commentString}
```

Gibt zurück: Die aktuelle Dienstdefinition

Nebenwirkungen: Keine

Ausnahmebedingungen: Keine

Voraussetzungen: Keine

TRANSFORMATIONEN DEFINIEREN

Prozedurname: ::WS::Client::SetServiceTransforms

Beschreibung: Die Transformationen eines Dienstes definieren

```
            Die Signatur der Transformation ist:
                cmd serviceName operationName transformType xml
    {url {}} {argList {}}
            dabei ist transformType REQUEST oder REPLY
            und url und argList sind nur vorhanden, wenn
    transformType == REQUEST
```

Argumente:

```
     serviceName  - Der Name des Webdienstes
     inTransform  - Befehl der Eingabetransformation, voreingestellt
    {}.
     inTransform ist die Prozedur, die die ausgehende SOAP-Nachricht
    umformt, die die Eingabe des Servers wird.
     outTransform - Befehl der Ausgabetransformation, voreingestellt
    {}.
     outTransform ist die Prozedur, die die eingehende SOAP-Nachricht
    umformt, die die Antwort des Servers ist.
```

Gibt zurück: Nichts

Nebenwirkungen: Keine

Ausnahmebedingungen: Keine

Voraussetzungen: Der Dienst muss definiert worden sein.

SYNCHRONER AUFRUF, DER EIN DICTIONARY-OBJEKT ZURÜCKGIBT

Prozedurname: ::WS::Client::DoCall

Beschreibung: Eine Operation eines Webdienstes aufrufen

Argumente:

```
 serviceName     - Der Name des Webdienstes
 operationName   - Der Name der aufzurufenden Operation
 argList         - Die Argumente der Operation als
Dictionary-Objekt
                   Das gilt für die Nachrichten von Soap-Header
und Body.
 headers         - Zusätzliche Header für die HTTP-Anfrage. Das
                   ist ein Schlüssel-Wert-Listenargument. Es muss
eine Liste mit
                   einer geraden Anzahl von Elementen sein, die
abwechseln zwischen
                   Schlüsseln und Werten. Die Schlüssel werden zu
Headerfeldnamen.
                   Zeilenumbrüche werden aus den Werten entfernt,
damit der Header
                   nicht verfälscht werden kann.
                   Das ist ein optionales Argument und ist
voreingestellt {}.
```

Gibt zurück:

```
 Der Rückgabewert der Operation als Dictionary-Objekt.
   Das umfasst das Rückgabeergebnis und alle Rückgabeheader.
```

Nebenwirkungen: Keine

Ausnahmebedingungen:

```
 WSCLIENT HTTPERROR      - wenn ein HTTP-Fehler aufgetreten ist
 others                  - wie von der aufgerufenen Operation
gemeldet
```

Voraussetzungen: Der Dienst muss definiert worden sein.

ASYNCHRONER AUFRUF MIT GETRENNTEN RÜCKRUFFUNKTIONEN FÜR ERFOLG UND FEHLER

Prozedurname: ::WS::Client::DoAsyncCall

Beschreibung: Eine Operation eines Webdienstes asynchron aufrufen

Argumente:

```
 serviceName     - Der Name des Webdienstes
 operationName   - Der Name der aufzurufenden Operation
 argList         - Die Argumente der Operation als
Dictionary-Objekt
                   Das gilt für die Nachrichten von Soap-Header
und Body.
 succesCmd       - Ein Befehlspräfix, das aufgerufen wird, wenn
die Operationen
```

```
                    keinen Fehler melden. Die Ergebnisse werden als
    Dictionary-
                    Objekt an das Präfix angehängt. Das umfasst
                    das Rückgabeergebnis und alle Rückgabeheader.
    Leer lassen,
                    damit keine Funktion aufgerufen wird.

     errorCmd       - Ein Befehlspräfix, das aufgerufen wird, wenn
    die Operationen
                    einen Fehler melden. Der Fehlercode, die
    Stapelverfolgung und
                    die Fehlermeldung werden an das Präfix
    angehängt. Leer lassen,
                    damit keine Funktion aufgerufen wird.
     headers        - Zusätzliche Header für die HTTP-Anfrage. Das
                    ist ein Schlüssel-Wert-Listenargument. Es muss
    eine Liste mit
                    einer geraden Anzahl von Elementen sein, die
    abwechseln zwischen
                    Schlüsseln und Werten. Die Schlüssel werden zu
    Headerfeldnamen.
                    Zeilenumbrüche werden aus den Werten entfernt,
    damit der Header
                    nicht verfälscht werden kann.
                    Das ist ein optionales Argument und ist
    voreingestellt {}.
```

Gibt zurück: Nichts.

Nebenwirkungen: Keine

Ausnahmebedingungen:

```
     WSCLIENT HTTPERROR      - wenn ein HTTP-Fehler aufgetreten ist
     others                  - wie von der aufgerufenen Operation
    gemeldet
```

Voraussetzungen: Der Dienst muss definiert worden sein.

ERZEUGEN VON STUB-PROZEDUREN IN TCL FÜR SYNCHRONE AUFRUFE

Prozedurname: ::WS::Client::CreateStubs

Beschreibung: Stub-Routinen erzeugen, um Webdienst-Operationen aufzurufen.

```
             Alle Routinen werden in einem Namespace erzeugt, der
    gleich
             dem Dienstnamen ist. Der Prozedurname ist gleich
             dem Operationsnamen.
```

```
          HINWEIS -- Argumente des Webdienstes sind
positionsunabhängig, deshalb
                werden die Argumente der Prozedur in
alphabetischer Reihenfolge definiert.
```

Argumente:

```
 serviceName     - Der Dienst, für den Stubs erzeugt werden
```

Gibt zurück: Eine Zeichenkette, die die erzeugten Prozeduren beschreibt.

Nebenwirkungen: Der vorhandene Namespace wird gelöscht.

Ausnahmebedingungen: Keine

Voraussetzungen: Der Dienst muss definiert worden sein.

SYNCHRONER AUFRUF, DER DAS ROHE XML ZURÜCKGIBT

Prozedurname: ::WS::Client::DoRawCall

Beschreibung: Eine Operation eines Webdienstes aufrufen

Argumente:

```
 serviceName     - Der Name des Webdienstes
 operationName   - Der Name der aufzurufenden Operation
 argList         - Die Argumente der Operation als
Dictionary-Objekt
                   Das gilt für die Nachrichten von Soap-Header
und Body.
 headers         - Zusätzliche Header für die HTTP-Anfrage. Das
                   ist ein Schlüssel-Wert-Listenargument. Es muss
eine Liste mit
                   einer geraden Anzahl von Elementen sein, die
abwechseln zwischen
                   Schlüsseln und Werten. Die Schlüssel werden zu
Headerfeldnamen.
                   Zeilenumbrüche werden aus den Werten entfernt,
damit der Header
                   nicht verfälscht werden kann.
                   Das ist ein optionales Argument und ist
voreingestellt {}.
```

Gibt zurück:

```
 Das XML der Operation.
```

Nebenwirkungen: Keine

Ausnahmebedingungen:

```
 WSCLIENT HTTPERROR      - wenn ein HTTP-Fehler aufgetreten ist
```

Voraussetzungen: Der Dienst muss definiert worden sein.

EIN VORLAGEN-DICTIONARY ERZEUGEN

Prozedurname: ::WS::Utils::GenerateTemplateDict

Beschreibung: Ein Vorlagen-Dictionary-Objekt für einen gegebenen Typ erzeugen.

Argumente:

```
     mode            - Der Modus, Client oder Server
     serviceName     - Der Name des Webdienstes
     type            - Der Name des Typs
     arraySize       - Anzahl der Elemente, die in einem Array erzeugt
    werden. Vorgabe ist 2
```

Gibt zurück:

```
Ein Dictionary-Objekt für einen gegebenen Typ. Wenn zyklische Verweise
    vorkommen, haben sie den Wert <** Circular Reference **>
Nebenwirkungen : Keine
Ausnahmebedingungen : Keine
Voraussetzungen : Der Dienst muss definiert worden sein.
```

EINEN DIENST KONFIGURIEREN

Es gibt zwei Prozeduren, um einen Dienst zu konfigurieren:

- ::WS::Client::SetOption
- ::WS::Client::Config

Die erste Prozedur enthält die Vorgabeoptionen des Pakets. Die Vorgabeoptionen werden beim Erzeugen des Dienstes verwendet und dann in den Dienst kopiert.

Die zweite Prozedur enthält die Optionen jedes Dienstes. Sie werden beim Erzeugen des Dienstes aus den Vorgabeoptionen kopiert.

Die meisten Optionseinträge sind über beide Funktionen erreichbar. Einige Optionen werden nur in der Phase des Erzeugens verwendet und existieren nicht als Dienstoption. Andere Optionen existieren nicht als Vorgabeoption, weil sie aus der WSDL-Datei initialisiert werden.

Im Folgenden werden zuerst die beiden Zugriffsroutinen beschrieben. Danach folgt eine Liste der Optionen für beide Funktionen, mit Hinweisen, wenn sie nur für eine der beiden Prozeduren gelten.

Prozedurname: ::WS::Client::SetOption

Beschreibung: Die Vorgabeoptionen des Pakets abfragen oder setzen

Argumente:

```
     -globalonly   - Gibt eine Liste der nur globalen Optionen und
    ihrer Werte zurück.
                    Nur globale Optionen werden nicht in den Dienst
    kopiert.
     -defaultonly  - Gibt eine Liste der nur als Vorgabe geltenden
    Optionen und ihrer Werte zurück.
```

```
                  Nur als Vorgabe geltende Optionen werden in den
    Dienst kopiert.
     --           - Ende der Optionen
     item         - Der Optionseintrag, der abgefragt oder
    konfiguriert wird.
                  Gibt eine Liste aller Paare aus Eintrag und Wert
    zurück, wenn er weggelassen wird.
     value        - Der Wert, auf den der Optionseintrag gesetzt wird.
                  Gibt den aktuellen Wert zurück, wenn er
    weggelassen wird.
```

Prozedurname: ::WS::Client::Config

Beschreibung: Die Optionen abfragen oder setzen, die zu einer Dienstdefinition gehören

Argumente:

```
     serviceName - Der Name des Webdienstes.
                   Gibt eine Liste der Vorgabepaare aus Eintrag und
    Wert zurück, wenn er nicht angegeben ist.
     item        - Der Optionseintrag, der abgefragt oder konfiguriert
    wird.
                   Gibt eine Liste aller Paare aus Eintrag und Wert
    zurück, wenn er nicht angegeben ist.
     value       - Der Wert, auf den der Optionseintrag gesetzt wird.
                   Gibt den aktuellen Wert zurück, wenn er weggelassen
    wird.
```

Optionsliste:

-

`allowOperOverloading`

Eine überladene Operation ist eine Operation mit demselben Namen, die mit unterschiedlichen Sätzen von Eingabeparametern vorkommen kann.

Diese Option meldet einen Fehler, wenn eine WSDL mit einer überladenen Operation zerlegt wird.

Vorgabe: 1

-

`contentType`

Der HTTP-Inhaltstyp der HTTP-Anfrage, die zum Aufruf des Webdienstes gesendet wird. Vorgabe: "text/xml;charset=utf-8"

-

`errorOnRedefine`

Meldet einen Fehler, wenn ein Dienst (CreateService und so weiter) für einen bereits vorhandenen Dienst erzeugt wird.

Vorgabewert: 0

Dieser Eintrag darf nicht mit *::WS::Client::Config* verwendet werden.

-

`genOutAttr`

erzeugt Attribute an ausgehenden Tags, Einzelheiten stehen hier

-

`inlineElementNS`

Namespace-Präfixe für Typen können im Wurzelelement der WSDL definiert sein.

Dieser Eintrag darf nicht mit *::WS::Client::Config* verwendet werden.

Diese Option einschalten, damit Namespace-Präfixe auch in der Typdefinition gesucht werden. Weil sie als globale Präfixe gelten, kann ein doppelt verwendetes Präfix einen Verarbeitungsfehler verursachen, wenn unterschiedliche URIs zugewiesen sind.

Den Fehler würde eine WSDL wie folgt verursachen

```
<wsdl:definitions targetNamespace="http://www.webserviceX.NET/"
xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/ xmlns:q1="myURI1" ...> ...
<xs:element xmlns:q1="myURI2" type="q1:MessageQ1"/>
```

-

`location`

Die URL des Dienstes. Sie wird mit dem Wert aus der WSDL-Datei initialisiert, wenn die WSDL-Datei zerlegt wird. Der Wert kann durch Setzen dieser Option überschrieben werden.

Dieser Eintrag darf nicht mit *::WS::Client::SetOption* verwendet werden.

-

`noTargetNs`

Die URI des Ziel-Namespace steht im Envelope des Webdienst-Aufrufs normalerweise zweimal:

```
<SOAP-ENV:Envelope ... xmlns="http://targeturi.org/"
xmlns:tns1="http://targeturi.org/" ...>
```

Wird diese Option auf 1 gesetzt, entfällt die Zeile mit "xmlns=".
Diese Option wurde gesetzt, um einen von SAP veröffentlichten Dienst aufzurufen.
Vorgabewert: 0

-

`nsOnChangeOnly`

setzt das Namespace-Präfix nur, wenn sich Namespaces ändern

-

`parseInAttr`

zerlegt Attribute an eingehenden Tags, Einzelheiten stehen hier

-

`queryTimeout`

Zeitlimit für jede Netzabfrage in ms. Vorgabewert: 60000 (1 Minute). Das Hilfspaket hat eine Option mit derselben Funktion, die verwendet wird, wenn kein Kontext einer Aufrufoption vorliegt.

-

`skipHeaderLevel`

boolescher Wert, der angibt, dass die erste Ebene des XML in einem Anfrageheader *übersprungen werden soll*. Abgeleitet aus den Optionen. Vorgabe ist 0 (nicht überspringen). *(Eingeführt in 2.2.8)*

-

`skipLevelOnReply`

boolescher Wert, der angibt, dass die erste Ebene des XML in einer Antwort *übersprungen werden darf*. Abgeleitet aus den Optionen. Vorgabe ist 0 (nicht überspringen). *(Eingeführt in 2.2.8)*

-

`skipLevelWhenActionPresent`

boolescher Wert, der angibt, ob die erste Ebene des XML übersprungen wird. Abgeleitet aus den Optionen. Vorgabe ist 0 (nicht überspringen).

-

`suppressNS (default: empty string)`

ein bestimmtes Namespace-Präfix nicht setzen

-

`suppressTargetNS`

Das URI-Präfix "tns1" des Ziel-Namespace nicht an alle Parameter im XML des Webdienst-Aufrufs anhängen.

Als Beispiel wird das XML geändert von (Option nicht gesetzt):

```
<SOAP-ENV:Envelope ... xmlns:tns1="http://targeturi.org/" ... >
<SOAP-ENV:Body> <tns1:CalledMethod>
<tns1:Parameter1>Value;/tns1:Parameter1> </tns1:CalledMethod>
</SOAP-ENV:Body> </SOAP-ENV:Envelope>
```

zu (Option gesetzt)

```
<SOAP-ENV:Envelope ... xmlns:tns1="http://targeturi.org/" ... >
<SOAP-ENV:Envelope ... <SOAP-ENV:Body> <tns1:CalledMethod>
<Parameter1>Value;/Parameter1> </tns1:CalledMethod> </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Abgeleitet aus den Optionen.
Intern setzt diese Option die Option "suppressNS" auf "tns1".
Diese Option wurde gesetzt, um einen von SAP veröffentlichten Dienst aufzurufen.
Diese Option ließ einen Aufruf eines bestimmten MS-Webdienstes mit der Fehlermeldung scheitern: "Input parameter 'Parameter1' can not be NULL or Empty.".
Vorgabe ist 0 (nicht unterdrücken).

-

`targetNamespace (default: empty string)`

der Ziel-Namespace des Dienstes, abgeleitet aus der WSDL.

Dieser Eintrag darf nicht mit *::WS::Client::SetOption* verwendet werden.

-

`UseNS (default: empty string)`

Siehe hier

-

`useTypeNS (default: empty string)`

das Namespace-Präfix des Typs als Präfix der Elemente verwenden

-

`valueAttrCompatiblityMode (default: 1)`

Wenn dies und genOutAttr/parseInAttr gesetzt sind, werden Werte im Dictionary als {} angegeben. Wenn dagegen genOutAttr/parseInAttr gesetzt ist und dies nicht gesetzt ist, werden die Werte im Dictionary als ::value angegeben.

```
     value          - Optional, der neue Wert.
```

Gibt zurück:

```
     Der Wert des Eintrags.
Nebenwirkungen : Keine
Ausnahmebedingungen : Keine
Voraussetzungen : Der Dienst muss definiert worden sein.
```

ABSTRAKTE TYPEN IN KONKRETE TYPEN UMWANDELN

Wenn *parseInAttr* und *genOutAttr* eingeschaltet sind, behandelt das System halbautomatisch die Umwandlung von Elementen, die als Typ deklariert sind, der ein abstrakter Typ ist, in den konkreten Typ, der in einer Nachricht tatsächlich verwendet wird, und zurück. Bei einem Element, das als Typ deklariert ist, der ein abstrakter Typ ist, gibt der Wert des Schlüssels **::type** im Dictionary den konkreten Typ an, der tatsächlich verwendet werden soll (oder bei einer Antwortnachricht den konkreten Typ, der tatsächlich verwendet wurde).

HINWEIS: In der WSDL muss der konkrete Typ eine Erweiterung des abstrakten Typs sein, das Paket erzwingt diese Einschränkung jedoch nicht, deshalb ist Vorsicht geboten.

Creating_a_Tcl_Web_Service

Stichwörter: Creating_a_Tcl_Web_Service

Einen Tcl-Webdienst erzeugen

INHALT

- 1 Das Paket Webservices Server laden
- 2 Einen Dienst definieren
- 3 Eine Operation definieren (auch eine Dienstprozedur)
- 4 Komplexe Typen deklarieren

DAS PAKET WEBSERVICES SERVER LADEN

Das Serverpaket für Webservices wird so geladen:

```
package require WS::Server
```

Dieser Befehl lädt den Server nur beim ersten Mal, es schadet also nicht, ihn in jede Datei zu setzen, die einen Dienst oder eine Dienstprozedur deklariert.

Verwendung als Teil von TclHttpd
Das Paket Web Services, WS::Server, ist keine eigenständige Anwendung, sondern als "Modul" von TclHttpd gedacht. Der folgende Befehl steht gewöhnlich in httpdthread.tcl:

Einbetten in eine eigenständige Anwendung
Um einen Webdienst in eine Anwendung einzubetten, muss die Anwendung ereignisgesteuert sein, und das Paket **WS::Embeded** wird gebraucht. Der Dienst muss außerdem mit der Option *-mode=embedded* definiert werden.

Siehe auch Einen Webdienst in eine Anwendung einbetten.

Verwendung mit Apache Rivet
Apache Rivet ist ein Modul (mod_rivet), das der Apache-httpd-Server laden kann, damit Webseiten eingebettete Tcl-Befehle ausführen, ähnlich wie PHP. Um einen Webdienst in Rivet zu erzeugen, dient das Beispiel EchoRivetService.rvt als Ausgangspunkt: es wird einfach in ein Verzeichnis kopiert, das die Apache-Instanz ausliefert. Der neue Ort sollte sofort unter den folgenden URLs erreichbar sein:

```
                /path/to/EchoRivetService.rvt/doc
                      Zeigt eine HTML-Seite, die den Dienst beschreibt
                /path/to/EchoRivetService.rvt/wsdl
                      Gibt eine WSDL zurück, die den Dienst beschreibt
                /path/to/EchoRivetService.rvt/op
                      Ruft eine Operation auf
```

Wenn die veröffentlichten URLs des Dienstes anders erscheinen sollen, können die Standardmodule Apache mod_rewrite oder mod_alias jede andere URL transparent auf diese Orte abbilden.

EINEN DIENST DEFINIEREN

Der Code, der einen Dienst definiert, steht gewöhnlich in einer oder mehreren Dateien im Verzeichnis custom.

Prozedurname: ::WS::Server::Service

Beschreibung: Einen Webdienst deklarieren, die folgenden URLs werden existieren

```
                /service/<ServiceName>
                      Zeigt eine HTML-Seite, die den Dienst beschreibt
                /service/<ServiceName>/wsdl
                      Gibt eine WSDL zurück, die den Dienst beschreibt
                /service/<ServiceName>/op
                      Ruft eine Operation auf
```

Argumente: diese Prozedur verwendet positionsunabhängige Argumente, und zwar:

```
         -hostcompatibility32 bool - Aktiviert den
Kompatibilitätsmodus
                            der Version 3.2.0 für den Parameter
-host.
                            Voreingestellt ist true.
         -host           - Die Host-Angabe innerhalb der
XML-Namespaces
                            der übertragenen XML-Dateien.
                            Sie sollte eindeutig sein.
                            Voreingestellt ist localhost.
                            Wenn die Kompatibilität 3.2 aktiv ist,
ändert sich
                            der Vorgabewert im eingebetteten Modus
zu ip:port.
         -hostlocation   - Der Hostname, der in der erzeugten
                            WSDL-Datei bekanntgegeben wird.
Voreingestellt ist localhost.
                            Wenn die Kompatibilität 3.2 aktiv ist,
ist
                            der Vorgabewert gleich dem Parameter
-host.
         -hostlocationserver bool - Wenn true, setzt der aktuelle
Server
                            die Host-Lage.
                            Bei einem httpd-Server wird dieser Wert
importiert.
                            Bei anderen Servern oder wenn das
scheitert, ist der Wert
                            das aktuelle ip:port.
                            Der Vorgabewert ist true.
                            Bei Kompatibilität 3.2 ist der
Vorgabewert
                            für tclhttpd true, sonst false.
         -hostProtocol   - Legt das Host-Protokoll (http, https)
für die
                            URL der WSDL-Lage fest. Der besondere
Wert "server"
                            (Vorgabe) folgt der Angabe des
TCP/IP-Servers.
                            Das ist für den eingebetteten Server
und tclhttpd umgesetzt.
                            Das Protokoll für XML-Namespaces
                            ist immer "http".
         -description    - Die HTML-Beschreibung für diesen Dienst
         -htmlhead       - Die Titelzeichenkette der
Dienstbeschreibung
```

```
         -author         - Die Autoreneigenschaft in der
Dienstbeschreibung
         -xmlnamespace   - Zusätzliche XML-Namespaces, die der
Dienst verwendet
         -service        - Der Dienstname (er wird auch für
                           den Tcl-Namespace der Prozeduren
verwendet, die
                           die Operationen umsetzen.
         -premonitor     - Das ist ein Befehlspräfix, das
aufgerufen wird, bevor
                           eine Operation aufgerufen wird. Die
folgenden Argumente werden
                           dem Befehlspräfix hinzugefügt:
                              PRE serviceName operationName
operArgList
         -postmonitor    - Das ist ein Befehlspräfix, das
aufgerufen wird, nachdem
                           eine Operation aufgerufen wurde. Die
folgenden Argumente werden
                           dem Befehlspräfix hinzugefügt:
                              POST serviceName operationName
OK|ERROR results
         -inheaders      - Liste der Typen von Eingabeheadern.
         -outheaders     - Liste der Typen von Ausgabeheadern.
         -intransform    - Transformation für eingehende
Anforderungen (2.0.3 und später).
                          Die Signatur des Befehls muss sein:
                               cmd \
                                   mode (REQUEST) \
                                   xml \
                                   notUsed_1 \
                                   notUsed_2
         -outtransform   - Transformation für ausgehende Antworten
(2.0.3 und später).
                          Die Signatur des Befehls muss sein:
                               cmd \
                                   mode (REPLY) \
                                   xml \
                                   operation \
                                   resultDict
         -checkheader    - Befehlspräfix, um Header zu prüfen.
                             Wenn der Aufruf nicht erlaubt sein
soll, sollte dieser Befehl
                             einen Fehler melden.
                             Die Signatur des Befehls muss sein:
                               cmd \
```

```
                                        service \
                                        operation \
                                        caller_ipaddr \
                                        http_header_list \
                                        soap_header_list
        -mode           - Modus, in dem der Dienst läuft. Muss
einer der folgenden sein:
                               tclhttpd  -- läuft innerhalb von
tclhttpd oder einer
                                            Umgebung, die passende
Befehle
                                            Url_PrefixInstall
                                            und Httpd_ReturnData
bereitstellt
                               embedded  -- verwendet das Paket
::WS::Embedded
                               aolserver -- verwendet das Paket
::WS::AolServer
                               wub       -- verwendet das Paket
::WS::Wub
                               wibble    -- läuft innerhalb von
wibble
                               rivet     -- läuft innerhalb von
Apache Rivet (mod_rivet)
        -ports          - Liste der Ports für den eingebetteten
Modus. Vorgabe: 80
                                HINWEIS -- für jeden Port in
dieser Liste sollte
                                        ::WS::Embedded::Listen
aufgerufen werden,
                                       bevor ::WS::Embeded::Start
aufgerufen wird
        -prefix         - Pfadpräfix für den Namespace und den
Endpunkt
                          Voreingestellt ist "/service/" plus der
Dienstname
        -traceEnabled   - Boolescher Wert, der steuert, ob ein
Trace in der Ausnahme zurückgegeben wird
                          Voreingestellt ist "Y"
        -docFormat      - Format der Dokumentation für Operationen
("text" oder "html").
                          Voreingestellt ist "text"
        -stylesheet     - Die URL des CSS-Stylesheets in der
HTML-Dokumentation

        -errorCallback  - Rückruffunktion, die aufgerufen wird,
```

```
    wenn ein Fehler entsteht
            -verifyUserArgs - Boolescher Wert, der die Prüfung der vom
    Benutzer gelieferten Argumente steuert
                             Voreingestellt ist "N"
            -enforceRequired - Meldet einen Fehler, wenn ein
    Pflichtfeld in der
                              Antwort fehlt.
                              Voreingestellt ist "N"
```

Gibt zurück: Nichts

Nebenwirkungen: Keine

Ausnahmebedingungen:

```
     MISSREQARG -- Fehlende Pflichtargumente
```

Voraussetzungen: Keine

EINE OPERATION DEFINIEREN (AUCH EINE DIENSTPROZEDUR)

Prozedurname: ::WS::Server::ServiceProc

Beschreibung: Eine Operation für einen Dienst registrieren und die Prozedur deklarieren, die die Operationen behandelt.

Argumente:

```
     ServiceName     -- Name des Dienstes, zu dem diese Operation
    gehört
     NameInfo        -- Liste aus zwei Elementen:
                             1) OperationName -- der Name der Operation
                             2) ReturnType    -- der Typ der Rückgabe
    der Prozedur,
                                                 das kann ein
    einfacher oder komplexer Typ sein
     Arglist         -- Liste von Argumentdefinitionen,
                         jedes Listenelement muss die Form haben:
                             1) ArgumentName -- der Name des Arguments
                             2) ArgumentTypeInfo -- -- Eine Liste aus:
                                    {type typeName comment
    commentString}
                                          typeName kann jeder einfache
    oder definierte Typ sein.
                                          commentString ist eine
    Zeichenkette in Anführungszeichen, die das Feld beschreibt
     Documentation   -- HTML, das beschreibt, was diese Operation tut
     Body            -- Der Tcl-Code, der aufgerufen wird, wenn die
    Operation aufgerufen wird. Dieser
                           Code sollte ein Dictionary zurückgeben,
    mit <OperationName>Result als
                           Schlüssel und dem Ergebnis der Operation
    als Wert.
```

Verfügbare einfache Typen sind:

- anyType, string, boolean, decimal, float, double, duration, dateTime, time, date, gYearMonth, gYear, gMonthDay, gDay, gMonth, hexBinary, base64Binary, anyURI, QName, NOTATION, normalizedString, token, language, NMTOKEN, NMTOKENS, Name, NCName, ID, IDREF, IDREFS, ENTITY, ENTITIES, integer, nonPositiveInteger, negativeInteger, long, int, short, byte, nonNegativeInteger, unsignedLong, unsignedInt, unsignedShort, unsignedByte, positiveInteger

Das

typeName

darf die folgenden Suffixe enthalten:

- *()*: der Typ ist ein Array
- *?*: der Typ ist ein optionaler Parameter

Gibt zurück: Nichts

Nebenwirkungen:

```
   Eine Prozedur namens "<ServiceName>::<OperationName>" ist definiert
   Ein Typname mit dem Namen <OperationName>Result ist definiert.
```

Ausnahmebedingungen: Keine

Voraussetzungen: ::WS::Server::Server muss für den ServiceName aufgerufen worden sein

KOMPLEXE TYPEN DEKLARIEREN

Siehe: Einen Webdienst-Typ aus Tcl erzeugen

Creating_a_Web_Service_Type

Stichwörter: Creating_a_Web_Service_Type

Einen Webdienst-Typ aus Tcl erzeugen

INHALT

- 1 Überblick
- 2 Das Paket Webservices Utility laden
- 3 Einen Typ definieren
- 3 Einen abgeleiteten Typ definieren
- 4 Eine Typdefinition abfragen
- 5 Ein Vorlagen-Dictionary für eine Typdefinition erzeugen

ÜBERBLICK

Die Deklaration von Webdienst-Typen gehört zum Paket Webservices Utility.

Beim Schreiben eines Webdienstes ist oft eine komplexe Typdefinition für ein Argument mit strukturierten Daten nötig.

Beim Aufruf einer Operation eines Webdienstes ist es manchmal bequem, einen komplexen Typ zu definieren, um strukturierte Daten als XML-Fragment zurückzugeben, auch wenn der Dienst angibt, nur eine Zeichenkette zu erwarten.

DAS PAKET WEBSERVICES UTILITY LADEN

Das Serverpaket für Webservices wird so geladen:

```
package require WS::Utils
```

Dieser Befehl lädt die Hilfsprogramme nur beim ersten Mal, es schadet also nicht, ihn in jede Datei zu setzen, die die Hilfsprogramme verwendet.

EINEN TYP DEFINIEREN

Prozedurname: *::WS::Utils::ServiceTypeDef*

Beschreibung: Einen Typ für einen Dienst definieren.

Argumente:

```
    mode            - Client oder Server
    service         - Name des Dienstes, für den diese Typdefinition
   gilt
    type            - Der zu definierende oder neu zu definierende Typ
    definition      - Die Definition der Felder des Typs. Sie besteht
   aus einem
                          oder mehreren Vorkommen einer
   Felddefinition. Jede Felddefinition
```

```
                          besteht aus:  fieldName fieldInfo
                          Dabei ist die Feldinfo: {type typeName
    comment commentString}
                             typeName kann jeder einfache oder
    definierte Typ sein.
                             commentString ist eine Zeichenkette in
    Anführungszeichen, die das Feld beschreibt.
```

Gibt zurück: Nichts

Nebenwirkungen: Keine

Ausnahmebedingungen: Keine

Voraussetzungen: Keine

EINEN ABGELEITETEN TYP DEFINIEREN

Prozedurname: *::WS::Utils::ServiceSimpleTypeDef*

Beschreibung: Einen abgeleiteten Typ für einen Dienst definieren.

Argumente:

```
     mode            - Client oder Server
     service         - Name des Dienstes, für den diese Typdefinition
    gilt
     type            - Der zu definierende oder neu zu definierende Typ
     definition      - Die Definition der Felder des Typs. Sie besteht
    aus einem
                          oder mehreren Vorkommen einer
    Felddefinition. Jede Felddefinition
                          besteht aus:  fieldName fieldInfo
                          Dabei: {type typeName comment commentString}
                             baseType typeName - jeder einfache oder
    definierte Typ.
                             comment commentString - eine
    Zeichenkette in Anführungszeichen, die das Feld beschreibt.
                             pattern value
                             length value
                             fixed "true"|"false"
                             maxLength value
                             minLength value
                             minInclusive value
                             maxInclusive value
                             enumeration value
```

Gibt zurück: Nichts

Nebenwirkungen: Keine

Ausnahmebedingungen: Keine

Voraussetzungen: Keine

EINE TYPDEFINITION ABFRAGEN

Prozedurname: *::WS::Utils::GetServiceTypeDef*

Beschreibung: Typdefinitionen abfragen.

Argumente:

```
 mode            - Client oder Server
 service         - Name des Dienstes, für den diese Abfrage gilt
 type            - Der abzurufende Typ (optional)
```

Gibt zurück:

```
 Wenn type nicht angegeben ist, ein Dictionary-Objekt, das alle
Typen
 des Dienstes beschreibt.
 Wenn type angegeben ist, ein Dictionary-Objekt, das den Typ
beschreibt.
   Eine Definition besteht aus einem Dictionary-Objekt mit den
folgenden Schlüsseln und Werten:
     xns         - Der Namespace für diesen Typ.
     definition  - Die Definition der Felder des Typs. Sie besteht
aus einem
                   oder mehreren Vorkommen einer Felddefinition.
Jede Felddefinition
                   besteht aus:  fieldName fieldInfo
                   Dabei ist die Feldinfo: {type typeName comment
commentString}
                     typeName kann jeder einfache oder definierte
Typ sein.
                     commentString ist eine Zeichenkette in
Anführungszeichen, die das Feld beschreibt.
```

Nebenwirkungen: Keine

Ausnahmebedingungen: Keine

Voraussetzungen: Der Dienst muss definiert sein.

EIN VORLAGEN-DICTIONARY FÜR EINE TYPDEFINITION ERZEUGEN

Prozedurname: *::WS::Utils::GenerateTemplateDict*

Beschreibung: Ein Vorlagen-Dictionary-Objekt für einen gegebenen Typ erzeugen.

Argumente:

```
    mode            - Client oder Server
    serviceName     - Name des Dienstes, in dem der Typ definiert ist
    type            - Der Name des Typs
    arraySize       - Anzahl der Elemente, die in einem Array erzeugt
   werden. Voreingestellt ist 2.
```

Gibt zurück:

```
     Ein Dictionary-Objekt für einen gegebenen Typ. Wenn Ringverweise
   existieren, haben sie den Wert <** Circular Reference **>
```

Nebenwirkungen: Keine

Ausnahmebedingungen: Keine

Voraussetzungen: Typ und Dienst müssen definiert sein.

Defining_Types

Stichwörter: Defining_Types

INHALT

- 1 Überblick
- 2 Das Paket Webservices Utility laden
- 3 Einen Typ definieren
- 3 Einen abgeleiteten Typ definieren
- 4 Eine Typdefinition abfragen
- 5 Ein Vorlagen-Dictionary für eine Typdefinition erzeugen

ÜBERBLICK

Die Deklaration von Webdienst-Typen gehört zum Paket Webservices Utility.

Beim Schreiben eines Webdienstes ist oft eine komplexe Typdefinition für ein Argument mit strukturierten Daten nötig.

Beim Aufruf einer Operation eines Webdienstes ist es manchmal bequem, einen komplexen Typ zu definieren, um strukturierte Daten als XML-Fragment zurückzugeben, auch wenn der Dienst angibt, nur eine Zeichenkette zu erwarten.

DAS PAKET WEBSERVICES UTILITY LADEN

Das Serverpaket für Webservices wird so geladen:

```
package require WS::Utils
```

Dieser Befehl lädt die Hilfsprogramme nur beim ersten Mal, es schadet also nicht, ihn in jede Datei zu setzen, die die Hilfsprogramme verwendet.

EINEN TYP DEFINIEREN

Prozedurname: *::WS::Utils::ServiceTypeDef*

Beschreibung: Einen Typ für einen Dienst definieren.

Argumente:

```
 mode            - Client oder Server
 service         - Name des Dienstes, für den diese Typdefinition
gilt
 type            - Der zu definierende oder neu zu definierende Typ
 definition      - Die Definition der Felder des Typs. Sie besteht
aus einem
                       oder mehreren Vorkommen einer
Felddefinition. Jede Felddefinition
                       besteht aus:  fieldName fieldInfo
                       Dabei ist die Feldinfo: {type typeName
comment commentString}
                          typeName kann jeder einfache oder
definierte Typ sein.
                          commentString ist eine Zeichenkette in
Anführungszeichen, die das Feld beschreibt.
```

Gibt zurück: Nichts

Nebenwirkungen: Keine

Ausnahmebedingungen: Keine

Voraussetzungen: Keine

EINEN ABGELEITETEN TYP DEFINIEREN

Prozedurname: *::WS::Utils::ServiceSimpleTypeDef*

Beschreibung: Einen abgeleiteten Typ für einen Dienst definieren.

Argumente:

```
 mode            - Client oder Server
 service         - Name des Dienstes, für den diese Typdefinition
gilt
 type            - Der zu definierende oder neu zu definierende Typ
 definition      - Die Definition der Felder des Typs. Sie besteht
aus einem
                       oder mehreren Vorkommen einer
Felddefinition. Jede Felddefinition
                       besteht aus:  fieldName fieldInfo
                       Dabei: {type typeName comment commentString}
                          baseType typeName - jeder einfache oder
definierte Typ.
                          comment commentString - eine
Zeichenkette in Anführungszeichen, die das Feld beschreibt.
                          pattern value
                          length value
                          fixed "true"|"false"
                          maxLength value
                          minLength value
```

```
                              minInclusive value
                              maxInclusive value
                              enumeration value
```

Gibt zurück: Nichts

Nebenwirkungen: Keine

Ausnahmebedingungen: Keine

Voraussetzungen: Keine

EINE TYPDEFINITION ABFRAGEN

Prozedurname: *::WS::Utils::GetServiceTypeDef*

Beschreibung: Typdefinitionen abfragen.

Argumente:

```
     mode            - Client oder Server
     service         - Name des Dienstes, für den diese Abfrage gilt
     type            - Der abzurufende Typ (optional)
```

Gibt zurück:

```
     Wenn type nicht angegeben ist, ein Dictionary-Objekt, das alle
    Typen
     des Dienstes beschreibt.
     Wenn type angegeben ist, ein Dictionary-Objekt, das den Typ
    beschreibt.
       Eine Definition besteht aus einem Dictionary-Objekt mit den
    folgenden Schlüsseln und Werten:
         xns         - Der Namespace für diesen Typ.
         definition  - Die Definition der Felder des Typs. Sie besteht
    aus einem
                       oder mehreren Vorkommen einer Felddefinition.
    Jede Felddefinition
                       besteht aus:  fieldName fieldInfo
                       Dabei ist die Feldinfo: {type typeName comment
    commentString}
                         typeName kann jeder einfache oder definierte
    Typ sein.
                         commentString ist eine Zeichenkette in
    Anführungszeichen, die das Feld beschreibt.
```

Nebenwirkungen: Keine

Ausnahmebedingungen: Keine

Voraussetzungen: Der Dienst muss definiert sein.

EIN VORLAGEN-DICTIONARY FÜR EINE TYPDEFINITION ERZEUGEN

Prozedurname: *::WS::Utils::GenerateTemplateDict*

Beschreibung: Ein Vorlagen-Dictionary-Objekt für einen gegebenen Typ erzeugen.

Argumente:

```
 mode            - Client oder Server
 serviceName     - Name des Dienstes, in dem der Typ definiert ist
 type            - Der Name des Typs
 arraySize       - Anzahl der Elemente, die in einem Array erzeugt
werden. Voreingestellt ist 2.
```

Gibt zurück:

```
  Ein Dictionary-Objekt für einen gegebenen Typ. Wenn Ringverweise
existieren, haben sie den Wert <** Circular Reference **>
```

Nebenwirkungen: Keine

Ausnahmebedingungen: Keine

Voraussetzungen: Typ und Dienst müssen definiert sein.

Dictionary_Representation_of_XML_Arrays

Stichwörter: Dictionary_Representation_of_XML_Arrays

XML-Arrays werden im Dictionary-Format als Liste von Werten dargestellt. Sehen wir uns an, wie das für einen einfachen Typ und für einen komplexen Typ; aussieht.

Array eines einfachen Typs

Nehmen wir an, ein Element hat die folgende Definition:

```
<xs:element minOccurs="0" maxOccurs="unbounded" name="Primes"
type="xs:integer" />
```

Nehmen wir außerdem an, dass dieses Element in unserem Dictionary die ersten vier Primzahlen enthält. Die Dictionary-Darstellung dieses Elements sähe dann so aus:

```
Primes {2 3 5 7}
```

Oder, wenn Attribute verwendet werden (das heißt, parseInAttr und/oder genOutAttr sind gesetzt), sähe es so aus:

```
Primes {{} {2 3 5 7}}
```

Array eines komplexen Typs

Nehmen wir an, die Typdefinition lautet:

```
<xs:element name="Person">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="FristName" type="xs:string"/>
      <xs:element name="LastName" type="xs:integer"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

Nehmen wir die folgende Definition an:

```
    <xs:element minOccurs="0" maxOccurs="unbounded" name="Attendees"
    type="Person" />
```

Nehmen wir nun an, die folgenden Personen nehmen teil:

- John Doe
- Jane Doe

Die Dictionary-Darstellung dieses Elements sähe dann so aus:

```
    Attendees {
        {FirstName {John} LastName {Doe}}
        {FirstName {Jane} LastName {Doe}}
    }
```

Oder, wenn Attribute verwendet werden (das heißt, parseInAttr und/oder genOutAttr sind gesetzt), sähe es so aus:

```
    Attendees {
        {{} {FirstName {{} {John}} LastName {{} {Doe}}}}
        {{} {FirstName {{} {Jane}} LastName {{} {Doe}}}}
    }
```

Embedded_Web_Service

Stichwörter: Embedded_Web_Service

Einen Webdienst einbetten

Inhalt

- 1 Das eingebettete Paket Webservices laden
- 2 Einen Port angeben, auf dem Anforderungen empfangen werden
- 3 Die Ereigniswarteschlange ausführen
- 4 Einen Port schließen
- 4 Alle Ports schließen

Das Paket Webservices Server laden

Das Serverpaket für Webservices wird so geladen:

```
 package require WS::Embeded
```

Dieser Befehl lädt den Server nur beim ersten Mal. Es schadet also nicht, ihn in jede Datei zu

setzen, die einen Dienst oder eine Dienstprozedur deklariert.

Einen Port angeben, auf dem Anforderungen empfangen werden

Der folgende Befehl öffnet einen Listener-Socket auf dem angegebenen Port. Die Webdienst-Funktion lässt sich durch einen Aufruf von *::WS::Server::Service* hinzufügen, bei dem der Parameter *-mode* auf *embedded* gesetzt ist.

Prozedurname: ::WS::Embeded::Listen

Beschreibung: Weist das Modul an, auf einem Port zu lauschen, mit Sicherheitsangaben.

Argumente: diese Prozedur verwendet positionsabhängige Argumente, und zwar:

```
 port     -- Port number to listen on.
 certfile -- Name of the certificate file or a pfx archive for
twapi.
             Defaults to {}.
 keyfile  -- Name of the key file. Defaults to {}.
             To use twapi TLS, specify a list with the following
elements:
             -- "-twapi": Flag, that TWAPI TLS should be used
             -- password: password of PFX file passed by
                [::twapi::conceal]. The concealing makes sure that
the
                password is not readable in the error stack trace
             -- ?subject?: optional search string in pfx file, if
                multiple certificates are included.
 userpwds -- A list of username:password. Defaults to {}.
 realm    -- The seucrity realm. Defaults to {}.
 timeout  -- A time in ms the sender may use to send the request.
             If a sender sends wrong data (Example: TLS if no TLS
is
             used), the process will just stand and a timeout is
required
             to clear the connection. Set to 0 to not use a
timeout.
             Default: 60000 (1 Minuit).
```

Gibt zurück: Handle des Sockets

Nebenwirkungen: Keine

Ausnahmebedingungen:: Keine

Voraussetzungen: Keine

Die Ereigniswarteschlange ausführen

Um Anforderungen zu bedienen, muss der Interpreter die Ereigniswarteschlange ausführen. Ist das nicht ohnehin der Fall (etwa weil Tk geladen ist), genügt der folgende Aufruf:

```
    vwait waitVariable
```

Um die Ereigniswarteschlange nach dem Herunterfahren des Servers zu beenden, dient:

```
    set waitVariable 1
```

Einen Port schließen
Prozedurname: ::WS::Embeded::Close

Beschreibung: Schließt einen zuvor geöffneten Listener-Port und beendet alle laufenden Anforderungen auf diesem Port.

Argumente: diese Prozedur verwendet positionsabhängige Argumente, und zwar:

```
    port     -- Port number to close.
```

Gibt zurück: Nichts

Nebenwirkungen: Keine

Ausnahmebedingungen:: Keine

Voraussetzungen: Keine

Alle Ports schließen
Prozedurname: ::WS::Embeded::CloseAll

Beschreibung: Schließt alle zuvor geöffneten Listener-Ports und beendet alle laufenden Anforderungen.

Argumente: diese Prozedur verwendet keine Argumente

Gibt zurück: Nichts

Nebenwirkungen: Keine

Ausnahmebedingungen:: Keine

Voraussetzungen: Keine

Rest_flavor_service_response
Stichwörter: Rest_flavor_service_response

Antwort im Rest-Stil

INHALT

- 1 Überblick
- 2 Rivet-Beispiel

ÜBERBLICK

Seit TCLWS 2.4 kann eine Antwort im *REST-Stil* zurückgegeben werden. Das heißt, statt eines XML-Dokuments wird eine JSON-Antwort zurückgegeben.

Unser Anwendungsfall verlangte nur, FORM-Argumente anzunehmen und für alles JSON-Antworten zurückzugeben. Logik, die Eingabeargumente zerlegt, die als serialisierte

JSON-Daten übergeben werden, ist deshalb nicht umgesetzt. Das könnte jemand später untersuchen.

RIVET-BEISPIEL

Ein Stück Code zeigt, wie dieser Modus in Apache Rivet anfangs gestartet wird. Das ähnelt tatsächlich stark der Verwendung von tclws im SOAP-Modus aus Apache Rivet:

```
        # Die Angaben der Anfrage in ein Array übernehmen.
        load_headers hdrArray
        set sock [pid];         # ein beliebiger Wert
        array unset ::Httpd$sock

        # Die Argumente im CGI-Stil in eine Liste bringen.
        load_response formArray
        set opname $formArray(call)
        unset formArray(call)
        set queryarg [list $opname [array get formArray]]

        # Die Methode aufrufen.
        array set ::Httpd$sock [list query $queryarg ipaddr [env
    REMOTE_ADDR] headerlist [array get hdrArray]]

        # Die Methode im REST-Modus aufrufen.
        set result [catch {::WS::Server::callOperation $svcname $sock
    -rest} error]
        array unset ::Httpd$sock
        if {$result} {
                headers numeric 500
                puts "Operation failed: $error"
                abort_page
        }
```

Tcl_Web_Service_Example

Stichwörter: Tcl_Web_Service_Example

Beispiel eines Tcl-Webdienstes

SERVERSEITE

Das Folgende steht in httpdthread.tcl:

```
    package require WS::Server
    package require WS::Utils
```

Das Folgende steht in eine Datei im Verzeichnis custom:

```
##
## Den Dienst definieren
##
::WS::Server::Service \
    -service wsExamples \
    -description  {Tcl Example Web Services} \
    -host         $::Config(host):$::Config(port)
```

```
##
## Besondere Typen definieren
##
::WS::Utils::ServiceTypeDef Server wsExamples echoReply {
    echoBack     {type string}
    echoTS       {type dateTime}
}
```

```
##
## Die verfügbaren Operationen definieren
##
::WS::Server::ServiceProc \
    wsExamples \
    {SimpleEcho {type string comment {Requested Echo}}} \
    {
        TestString      {type string comment {The text to echo
 back}}
    } \
    {Echo a string back} {
```

```
    return [list SimpleEchoResult $TestString]
}
```

```
::WS::Server::ServiceProc \
    wsExamples \
    {ComplexEcho {type echoReply comment {Requested Echo -- text
 and timestamp}}} \
    {
        TestString      {type string comment {The text to echo
 back}}
    } \
    {Echo a string and a timestamp back} {
```

```
    set timeStamp [clock format [clock seconds] -format
 {%Y-%m-%dT%H:%M:%SZ} -gmt yes]
    return [list ComplexEchoResult [list echoBack $TestString
 echoTS $timeStamp]  ]
}
```

CLIENTSEITE

```
package require WS::Client
```

```
##
## Definition der angebotenen Dienste holen
##
::WS::Client::GetAndParseWsdl
 http://localhost:8015/service/wsExamples/wsdl
```

```
set testString "This is a test"
set inputs [list TestString $testString]
```

```
##
## Synchron aufrufen
##
puts stdout "Calling SimpleEcho via DoCalls!"
set results [::WS::Client::DoCall wsExamples SimpleEcho $inputs]
puts stdout "\t Received: {$results}"
```

```
puts stdout "Calling ComplexEcho via DoCalls!"
set results [::WS::Client::DoCall wsExamples ComplexEcho $inputs]
puts stdout "\t Received: {$results}"
```

```
##
## Stubs erzeugen und für die Aufrufe verwenden
##
::WS::Client::CreateStubs wsExamples
puts stdout "Calling SimpleEcho via Stubs!"
set results [::wsExamples::SimpleEcho $testString]
puts stdout "\t Received: {$results}"
```

```
puts stdout "Calling ComplexEcho via Stubs!"
set results [::wsExamples::ComplexEcho $testString]
puts stdout "\t Received: {$results}"
```

```
##
## Asynchron aufrufen
##
proc success {service operation result} {
    global waitVar
```

```
    puts stdout "A call to $operation of $service was successful
 and returned $result"
    set waitVar 1
}
```

```
proc hadError {service operation errorCode errorInfo} {
    global waitVar
```

```
    puts stdout "A call to $operation of $service was failed with
 {$errorCode} {$errorInfo}"
    set waitVar 1
}
```

```
set waitVar 0
puts stdout "Calling SimpleEcho via DoAsyncCall!"
::WS::Client::DoAsyncCall wsExamples SimpleEcho $inputs \
        [list success wsExamples SimpleEcho] \
        [list hadError wsExamples SimpleEcho]
vwait waitVar
```

```
puts stdout "Calling ComplexEcho via DoAsyncCall!"
::WS::Client::DoAsyncCall wsExamples ComplexEcho $inputs \
        [list success wsExamples SimpleEcho] \
        [list hadError wsExamples SimpleEcho]
vwait waitVar
```

```
exit
```

Tcl_Web_Service_Math_Example

Stichwörter: Tcl_Web_Service_Math_Example

Mathematikbeispiel eines Tcl-Webdienstes

SERVERSEITE

Das Folgende steht in httpdthread.tcl:

```
package require WS::Server
package require WS::Utils
```

Das Folgende steht in eine Datei im Verzeichnis custom:

```
 ##
 ## Den Dienst definieren
 ##
 ::WS::Server::Service \
     -service wsMathExample \
     -description  {Tcl Web Services Math Example} \
     -host         $::Config(host):$::Config(port)

 ##
 ## Besondere Typen definieren
 ##
 ::WS::Utils::ServiceTypeDef Server wsMathExample Term {
```

```
       `coef         {type float}
        powerTerms   {type PowerTerm()}
    }
    ::WS::Utils::ServiceTypeDef Server wsMathExample PowerTerm {
        var          {type string}
        exponet      {type float}
    }
    ::WS::Utils::ServiceTypeDef Server wsMathExample Variables {
        var          {type string}
        value        {type float}
    }

   ##
   ## Die verfügbaren Operationen definieren
   ##
   ::WS::Server::ServiceProc \
        wsMathExample \
        {EvaluatePolynomial {type float comment {Result of evaluating
    a polynomial}}} \
        {
            varList       {type Variables() comment {The variables to
    be substitued into the polynomial}}
            polynomial    {type Term() comment {The polynomial}}
        } \
        {Evaluate a polynomial} {
        set equation {0 }
        foreach varDict $varList {
            set var [dict get $varDict var]
            set val [dict get $varDict value]
            set vars($var) $val
        }
        foreach term $polynomial {
            if {[dict exists $term coef]} {
                set coef [dict get $term coef]
            } else {
                set coef 1
            }
            append equation "+ ($coef"
            foreach pow [dict get $term powerTerms] {
                if {[dict exists $pow exponet]} {
                    set exp [dict get $pow exponet]
                } else {
                    set exp 1
                }
                append equation [format { * pow($vars(%s),%s} [dict
    get $pow var] $exp]
```

```
            }
            append equation ")"
        }
        set result [expr $equation]
        return [list SimpleEchoResult $result]
    }
```

CLIENTSEITE

```
    package require WS::Client
    ##
    ## Definition der angebotenen Dienste holen
    ##
    ::WS::Client::GetAndParseWsdl
    http://localhost:8015/service/wsMathExamples/wsdl

    dict set term var X
    dict set term value 2.0
    dict lappend varList $term
    dict set term var Y
    dict set term value 3.0
    dict lappend varList $term

    set term {}
    set powerTerm {}
    dict set powerTerm coef 2.0
    dict set term var X
    dict set term pow 2.0
    dict lappend terms $term
    dict set term var Y
    dict set term pow 3.0
    dict lappend terms $term
    dict set powerTerm powerTerms $terms

    dict set powerTerm coef -2.0
    dict set term var X
    dict set term pow 3.0
    dict lappend terms $term
    dict set term var Y
    dict set term pow 2.0
    dict lappend terms $term
    dict set powerTerm powerTerms $terms
    dict lappend polynomial powerTerms $powerTerm

    dict set input [list varList $varList polynomial $polynomial]
    ##
```

```
    ## Dienst aufrufen
    ##
    puts stdout "Calling EvaluatePolynomial wiht {$input}"
    set resultsDict [::WS::Client::DoCall wsMathExample
    EvaluatePolynomial $input]
    puts stdout "Results are {$resultsDict}"
```

Using_Options

Stichwörter: Using_Options

Optionen eines Webdienstes verwenden

Inhalt

- Überblick
- Das Paket Webservices Utilities laden
- Zugriffsroutine
- Optionen
  - genOutAttr - Attribute an ausgehenden Tags erzeugen
  - includeDirectory - Plattenverzeichnis für XSD-Includes, wenn sie nicht über das Web erreichbar sind.
  - nsOnChangeOnly - Namespace-Präfix nur setzen, wenn sich Namespaces ändern
  - parseInAttr - Attribute an eingehenden Tags zerlegen
  - queryTimeout - Zeitlimit für http(s)-Abrufe setzen.
  - StrictMode - Verarbeitungsmodus für WSDL
  - UseNS - Namespaces an Feld-Tags setzen
  - useTypeNs - das Namespace-Präfix des Typs als Präfix der Elemente verwenden
  - suppressNS - einen bestimmten Namespace-Präfix nicht setzen

Überblick

Die Pakete Webservices Client und Server verwenden die folgenden Optionen:

- UseNS, wenn gesetzt, werden die folgenden Optionen wirksam:
- StrictMode
- parseInAttr
- genOutAttr
- includeDirectory
  - useTypeNs
  - nsOnChangeOnly
  - suppressNS

Die Attribute lassen sich mit ::WS::Utils::SetOption abfragen und setzen.

Beim Aufruf von der Client-Seite werden die folgenden Optionen bei jedem Aufruf überschrieben und wiederhergestellt:

- genOutAttr
- nsOnChangeOnly
- parseInAttr
- suppressNS

- UseNS
- useTypeNs
- valueAttrCompatiblityMode

Das Paket Webservices Utilities laden

Das Serverpaket für Webservices wird so geladen:

```
package require WS::Utils
```

Dieser Befehl lädt die Hilfsprogramme nur beim ersten Mal, es schadet also nicht, ihn in jede Datei zu setzen, die die Hilfsprogramme verwendet.

Zugriffsroutine

Prozedurname: ::WS::Utils::SetOption

Beschreibung: Eine Option abfragen oder setzen

Argumente:

```
    option - name of the option
    value - value of the option (optional)
```

Gibt zurück: Der Wert der Option

Nebenwirkungen: Keine

Ausnahmebedingungen: Keine

Voraussetzungen: Keine

---

genOutAttr - Attribute an ausgehenden Tags erzeugen

Die Option *genOutAttr* wandelt, wenn sie auf einen Wert "true" gesetzt ist, alle Dictionary-Schlüssel des Eintrags für ein gegebenes Feld-Tag in Attribut-Wert-Paare des Tags im ausgehenden XML um. Für Attribute in der URL "http://www.w3.org/2001/XMLSchema-instance" ist der Schlüssel der Attributname mit zwei vorangestellten Doppelpunkten (zum Beispiel **::type**), und der Wert ist der Wert des Attributs. Für Attribute außerhalb der URL " http://www.w3.org/2001/XMLSchema-instance" ist der Schlüssel der Attributname, und der Wert ist der Wert des Attributs. Der Wert des Tags erhält einen Schlüssel, den valueAttrCompatiblityMode bestimmt.

Der Vorgabewert "0" schaltet diese Option aus.

---

includeDirectory - Plattenverzeichnis für XSD-Includes, wenn sie nicht über das Web erreichbar sind.

Die Option *includeDirectory* weist TclWs an, wenn sie gesetzt ist, im angegebenen Verzeichnis nach XSD-Includes zu suchen, die über das Web nicht gefunden werden.

Der Vorgabewert "{}" schaltet diese Option aus.

---

nsOnChangeOnly - Namespace-Präfix nur setzen, wenn sich Namespaces ändern

Die Option *nsOnChangeOnly* setzt, wenn sie auf einen Wert "true" gesetzt ist, Namespace-Präfixe nur, wenn sich die Namespaces ändern.

Diese Option wirkt nur, wenn die Option UseNS gesetzt ist.

Der Vorgabewert "0" schaltet diese Option aus.

---

parseInAttr - Attribute an eingehenden Tags zerlegen

Die Option *parseInAttr* wandelt, wenn sie auf einen Wert "true" gesetzt ist, alle Attribute eingehender Feld-Tags in Dictionary-Einträge für dieses Tag um. Für Attribute in der URL "http://www.w3.org/2001/XMLSchema-instance" ist der Schlüssel der Attributname mit zwei vorangestellten Doppelpunkten (zum Beispiel **::type**), und der Wert ist der Wert des Attributs. Für Attribute außerhalb der URL "http://www.w3.org/2001/XMLSchema-instance" ist der Schlüssel der Attributname, und der Wert ist der Wert des Attributs. Der Wert des Tags erhält einen Schlüssel, den valueAttrCompatiblityMode bestimmt.

Der Vorgabewert "0" schaltet diese Option aus.

---

queryTimeout - Zeitlimit für http(s)-Abfragen setzen

Zeitlimit für jede Netzabfrage in ms. Das Paket auf der Client-Seite hat eine Option mit derselben Funktion, die verwendet wird, wenn ein Aufruf einen Optionskontext hat.

Vorgabewert: 60000 (1 Minute).

StrictMode - Verarbeitungsmodus für WSDL

Die Option *StrictMode* legt fest, was geschieht, wenn beim Zerlegen einer WSDL ein Fehler erkannt wird. Die zulässigen Werte sind:

- **debug**
- **warning**
- **error**

Wenn *StrictMode* auf **debug** oder **warning** gesetzt ist, wird eine Meldung mit dem Paket ::log auf dieser Stufe protokolliert, und der Fehler wird danach ignoriert.

Wenn *StrictMode* auf einen anderen Wert als **debug** oder **warning** gesetzt ist, wird eine Meldung mit dem Paket ::log auf der Stufe **error** protokolliert, und eine Ausnahme wird erzeugt.

Der Vorgabewert ist **error**.

Eine wichtige Verwendung ist, Namespace-Importe in einer WDSL zu ignorieren, die tatsächlich keine Definitionen importieren.

UseNS - Namespaces an Feld-Tags setzen

Die Option *UseNS* setzt, wenn sie auf einen Wert "true" gesetzt ist, einen Namespace-Alias an alle Feld-Tags.

Der Vorgabewert "1" schaltet diese Option ein.

useTypeNs - das Namespace-Präfix des Typs als Präfix der Elemente verwenden

Die Option *useTypeNs* verwendet, wenn sie auf einen Wert "true" gesetzt ist, das Präfix des Namespace des Typs statt des Präfixes des Namespace des Elements.

Diese Option wirkt nur, wenn die Option UseNS gesetzt ist.

Der Vorgabewert "0" schaltet diese Option aus.

---

suppressNS - einen gegebenen Namespace-Präfix nicht setzen.

Die Option *suppressNS* bewirkt, wenn sie gesetzt ist, dass der gegebene Namespace nie als Präfix verwendet wird (Tags, die das gegebene Präfix sonst hätten, haben dann kein Präfix).

Diese Option wirkt nur, wenn die Option UseNS gesetzt ist.

Der Vorgabewert "{}" schaltet diese Option aus.

---

valueAttrCompatiblityMode - den Dictionary-Schlüssel für den Wert angeben, wenn Attribute verwendet werden

Diese Option ist nur sinnvoll, wenn die Option *parseInAttr* oder *genOutAttr* auf einen Wert "true" gesetzt ist. Bei einem Wert "true" hat der Wert des Tags den Schlüssel der leeren Zeichenkette (also **{}**). Bei einem Wert "false" hat der Wert des Tags den Schlüssel **::value**.

Der Vorgabewert "0" schaltet diese Option aus.

---

index

Stichwörter: index

Tcl-Webdienste

ZUSAMMENFASSUNG

Die Distribution bietet sowohl den Zugriff auf Webdienste von der Client-Seite als auch das Erzeugen von Webdiensten auf der Server-Seite. Derzeit werden auf der Client-Seite nur document/literal und rpc/encoded mit HTTP-Soap-Transport unterstützt. Die Server-Seite arbeitet derzeit nur mit TclHttpd oder eingebettet in eine Anwendung. Die Server-Seite stellt alle Dienste als document/literal über HTTP-Soap-Transport bereit. Die Dokumentation des Pakets einschließlich der Beispiele steht hier.

- Einen Webdienst aus Tcl aufrufen
- Einen Tcl-Webdienst erzeugen
- Einen Webdienst-Typ erzeugen
- Antwort im REST-Stil
- Dictionary-Darstellung von XML-Arrays
- Optionen eines Webdienstes verwenden
- Einen Webdienst in eine Anwendung einbetten
- Beispiel eines Tcl-Webdienstes
- Mathematikbeispiel eines Tcl-Webdienstes

Der Client arbeitet nachweislich mit Webdiensten auf der Basis von #C und Java (das Ergebnis kann abweichen).

LIZENZ
Standard-BSD.

WEBSERVER
Die Server-Seite arbeitet mit den folgenden Webservern:

- TclHttpd
- Rivet
- AOLserver
- WUB
- wibble
- Eingebetteter Modus (siehe Einbetten)

BENÖTIGTE PAKETE
Die folgenden Pakete werden verwendet:

- Tcl 8.6
- tdom 0.8.1
- tcltls oder TWAPI 4.4.0 für TLS-Unterstützung von Client und eingebettetem Server
- log aus TclLib
- uri aus TclLib
- struct::set aus TclLib
- http aus Tcl selbst
- yajl-tcl von flightaware github (nur für Anforderungen im Rest-Geschmack)

Wenn TclHttpd unter Windows läuft, ist die Erweiterung iocpsock sehr zu empfehlen.

Im Modus des eingebetteten Servers werden zusätzlich die folgenden Pakete verwendet:

- base64 aus TclLib (auch Channel-Server)
- html aus TclLib (auch Channel-Server)
- ncgi aus TclLib
- fileutil aus TclLib

tdbc

tdbc
Stichwörter: tdbc, SQL, database, connectivity, connection, resultset, statement

NAME
tdbc - Datenbankanbindung für Tcl

ÜBERSICHT
package require tdbc 1.0
package require tdbc::driver version

tdbc::driver::connection create *db* ?*-option value* ...?

BESCHREIBUNG

Tcl Database Connectivity (TDBC) ist eine gemeinsame Schnittstelle, über die Tcl-Programme auf SQL-Datenbanken zugreifen. Umgesetzt wird sie durch eine Reihe von Datenbank-*Treibern*: getrennte Module, von denen jedes Tcl an die Schnittstelle eines bestimmten Datenbanksystems anpasst. Alle Treiber stellen eine gemeinsame Folge von Befehlen bereit, mit denen die Datenbank bearbeitet wird. Diese Befehle werden sämtlich dynamisch benannt, weil sie Objekte im Datenbanksystem darstellen. Dazu gehören **Verbindungen,** die Verbindungen zu einer Datenbank darstellen, **Anweisungen,** die SQL-Anweisungen darstellen, und **Ergebnismengen,** die die Mengen der Zeilen darstellen, die aus dem Ausführen von Anweisungen entstehen. Jede dieser Arten hat eine eigene Handbuchseite; aufgeführt sind sie unter **SIEHE AUCH**.

Daneben enthält TDBC selbst einige Dienstprozeduren, die vor allem für Autoren von Treibern von Bedeutung sind. **SIEHE AUCH** zählt auch sie auf.

COPYRIGHT

Copyright (c) 2008 by Kevin B. Kenny.

tdbc_connection

Stichwörter: tdbc_connection, TDBC, SQL, database, connectivity, connection, resultset, statement

NAME

tdbc::connection - TDBC-Objekt für eine Verbindung

ÜBERSICHT

package require tdbc 1.0
package require tdbc::driver version

tdbc::driver::connection create *db* ?*-option value* ...?

db **configure** ?*-option value* ...?
db **close**
db **foreignkeys** ?*-primary tableName*? ?*-foreign tableName*?
db **prepare** *sql-code*
db **preparecall** *call*
db **primarykeys** *tableName*
db **statements**
db **resultsets**
db **tables** ?*pattern*?
db **columns** *table* ?*pattern*?
db **begintransaction**
db **commit**
db **rollback**
db **transaction** *script*
db **foreach** ?*-as lists|dicts*? ?*-columnsvariable name*? ?*--*? *varName sqlcode* ?*dictionary*? *script*

.in 14 *db* **allrows** ?**-as lists**|**dicts**? ?**-columnsvariable** *name*? ?**--**? *sql-code* ?*dictionary*?

BESCHREIBUNG

Jeder Datenbanktreiber für TDBC (Tcl DataBase Connectivity) setzt ein Objekt *Verbindung* um, das eine Verbindung zu einer Datenbank darstellt. Nach üblicher Vereinbarung entsteht dieses Objekt durch den Befehl **tdbc::***driver***::connection create**. Dieser Befehl nimmt den Namen eines Tcl-Befehls entgegen, der die Verbindung darstellen wird, und eine mögliche Menge von Optionen (siehe **KONFIGURATIONSOPTIONEN**). Er stellt eine Verbindung zur Datenbank her und gibt den Namen des neu erzeugten Tcl-Befehls zurück.

Der Objektbefehl **configure** auf einer Datenbankverbindung gibt, ohne Argumente aufgerufen, eine Liste abwechselnder Schlüsselwörter und Werte zurück, die die aktuelle Konfiguration der Verbindung darstellen. Mit einem einzelnen Argument *-option* gibt er den konfigurierten Wert der angegebenen Option zurück. Andernfalls muss er eine gerade Anzahl von Argumenten erhalten, abwechselnd Optionen und Werte. Die angegebenen Optionen erhalten die angegebenen Werte, und der Befehl gibt nichts zurück.

Der Objektbefehl **close** auf einer Datenbankverbindung schließt die Verbindung. Alle aktiven Anweisungen und Ergebnismengen auf der Verbindung werden geschlossen. Jede nicht festgeschriebene Transaktion wird zurückgesetzt. Der Objektbefehl wird gelöscht.

Der Objektbefehl **prepare** auf einer Datenbankverbindung bereitet eine SQL-Anweisung zur Ausführung vor. Das Argument *sql-code* muss eine einzelne SQL-Anweisung enthalten, die ausgeführt werden soll. Gebundene Variablen dürfen vorkommen. Der Rückgabewert ist ein neu erzeugter Tcl-Befehl, der die Anweisung darstellt. **tdbc::statement** erörtert ausführlicher das SQL, das der Objektbefehl **prepare** akzeptiert, und die Schnittstelle, die eine Anweisung akzeptiert.

Bei einer Datenbankverbindung, deren zugrunde liegende Datenbank und deren Treiber gespeicherte Prozeduren unterstützen, bereitet der Objektbefehl **preparecall** einen Aufruf einer gespeicherten Prozedur zur Ausführung vor. Die Syntax des Aufrufs der gespeicherten Prozedur ist:

```
?resultvar =? procname(?arg ?, arg...?)
```

Der Rückgabewert ist ein neu erzeugter Tcl-Befehl, der die Anweisung darstellt. **tdbc::statement** beschreibt die Schnittstelle, die eine Anweisung akzeptiert.

Der Objektbefehl **statements** gibt eine Liste der Anweisungen zurück, die durch Anweisungen **prepare** und **preparecall** gegen die gegebene Verbindung erzeugt wurden und noch nicht geschlossen sind.

Der Objektbefehl **resultsets** gibt eine Liste der Ergebnismengen zurück, die durch Ausführen von Anweisungen entstanden sind, die über die gegebene Verbindung vorbereitet wurden, und die noch nicht geschlossen sind.

Der Objektbefehl **tables** erlaubt dem Programm, die Verbindung nach den Namen der Tabellen zu fragen, die in der Datenbank vorhanden sind. Der optionale Parameter *pattern* ist ein Muster, auf das der Name einer Tabelle passen soll. Er darf die SQL-Platzhalterzeichen "**%**" und "**_**" enthalten. Der Rückgabewert ist ein Dictionary, dessen Schlüssel Tabellennamen und dessen Werte untergeordnete Dictionarys sind. Die Deutung der Werte steht in der Dokumentation des einzelnen Datenbanktreibers.

Der Objektbefehl **columns** erlaubt dem Programm, die Verbindung nach den Namen der Spalten zu fragen, die in einer gegebenen Tabelle vorhanden sind. Der optionale Parameter **pattern** ist ein Muster, auf das der Name einer Spalte passen soll. Er darf die SQL-Platzhalterzeichen "**%**"

und "_" enthalten. Der Rückgabewert ist ein Dictionary, dessen Schlüssel Spaltennamen und dessen Werte Dictionarys sind. Jedes der untergeordneten Dictionarys enthält mindestens die folgenden Schlüssel und Werte (und darf weitere enthalten, deren Verwendung ein bestimmter Datenbanktreiber festlegt).

type
Enthält den Datentyp der Spalte und wird im Allgemeinen aus der Menge **bigint**, **binary**, **bit**, **char**, **date**, **decimal**, **double**, **float**, **integer**, **longvarbinary**, **longvarchar**, **numeric**, **real**, **time**, **timestamp**, **smallint**, **tinyint**, **varbinary** und **varchar** gewählt. (Hat die Spalte einen Typ, der sich nicht als einer der genannten darstellen lässt, enthält **type** eine treiberabhängige Beschreibung des Typs.)

precision
Enthält die Präzision der Spalte in Bit, Dezimalziffern oder die Breite in Zeichen, je nach Typ.

scale
Enthält die Skala der Spalte (die Anzahl der Ziffern nach dem Radixpunkt) für Typen, die dieses Konzept kennen.

nullable
Enthält 1, wenn die Spalte NULL-Werte enthalten kann, und andernfalls 0. Der Objektbefehl **primarykeys** erlaubt dem Programm, die Verbindung nach den Primärschlüsseln zu fragen, die zu einer gegebenen Tabelle gehören. Der Parameter *tableName* bezeichnet die abgefragte Tabelle. Das Ergebnis ist eine Liste von Dictionarys, die die Schlüssel aufzählen (in einem ähnlichen Format wie die Liste, die *$connection* **allrows -as dicts** zurückgibt). Die Schlüssel des Dictionary dürfen mindestens die folgenden umfassen. Werte, die NULL sind oder in einer gegebenen Datenbank ohne Bedeutung sind, werden weggelassen.

tableCatalog
Name des Katalogs, in dem die Tabelle erscheint.

tableSchema
Name des Schemas, in dem die Tabelle erscheint.

tableName
Name der Tabelle, der der Primärschlüssel gehört.

constraintCatalog
Name des Katalogs, in dem das Primärschlüssel-Constraint erscheint. In manchen Datenbanksystemen muss das nicht derselbe Katalog sein wie der der Tabelle.

constraintSchema
Name des Schemas, in dem das Primärschlüssel-Constraint erscheint. In manchen Datenbanksystemen muss das nicht dasselbe Schema sein wie das der Tabelle.

constraintName
Name des Primärschlüssel-Constraints,

columnName
Name einer Spalte, die Teil des Primärschlüssels ist.

ordinalPosition
Ordnungsposition der Spalte innerhalb des Primärschlüssels. Diesen Spalten dürfen weitere hinzugefügt werden, die für ein bestimmtes Datenbanksystem spezifisch sind. Der Objektbefehl **foreignkeys** erlaubt dem Programm, die Verbindung nach Fremdschlüsselbeziehungen zu fragen, die für eine bestimmte Tabelle gelten. Die

Beziehungen lassen sich auf die Schlüssel einschränken, die in einer bestimmten Tabelle erscheinen (**-foreign** *tableName*), auf die Schlüssel, die auf eine bestimmte Tabelle verweisen (**-primary** *tableName*), oder auf beides. Mindestens eines von **-primary** und **-foreign** sollte angegeben werden, obwohl manche Treiber alle Fremdschlüssel im aktuellen Katalog aufzählen, wenn beide Optionen entfallen. Das Ergebnis des Objektbefehls **foreignkeys** ist eine Liste von Dictionarys mit einem Listenelement je Schlüssel (in einem ähnlichen Format wie die Liste, die *$connection* **allrows -as dicts** zurückgibt). Die Schlüssel des Dictionary dürfen mindestens die folgenden umfassen. Werte, die NULL sind oder in einer gegebenen Datenbank ohne Bedeutung sind, werden weggelassen.

foreignConstraintCatalog
Katalog, in dem das Fremdschlüssel-Constraint erscheint.

foreignConstraintSchema
Schema, in dem das Fremdschlüssel-Constraint erscheint.

foreignConstraintName
Name des Fremdschlüssel-Constraints.

primaryConstraintCatalog
Katalog, der das Primärschlüssel-Constraint (oder das Constraint eines eindeutigen Schlüssels) auf der Spalte hält, auf die der Fremdschlüssel verweist.

primaryConstraintSchema
Schema, das das Primärschlüssel-Constraint (oder das Constraint eines eindeutigen Schlüssels) auf der Spalte hält, auf die der Fremdschlüssel verweist.

primaryConstraintName
Name des Primärschlüssel-Constraints (oder des Constraints eines eindeutigen Schlüssels) auf der Spalte, auf die der Fremdschlüssel verweist.

updateAction
Aktion, die auszuführen ist, wenn eine Anweisung UPDATE das Constraint ungültig macht. Der Wert ist **CASCADE**, **SET DEFAULT**, **SET NULL**, **RESTRICT** oder **NO ACTION**.

deleteAction
Aktion, die auszuführen ist, wenn eine Anweisung DELETE das Constraint ungültig macht. Der Wert ist **CASCADE**, **SET DEFAULT**, **SET NULL**, **RESTRICT** oder **NO ACTION**.

primaryCatalog
Name des Katalogs, in dem die Primärtabelle erscheint (diejenige, auf die der Fremdschlüssel verweist).

primarySchema
Name des Schemas, in dem die Primärtabelle erscheint (diejenige, auf die der Fremdschlüssel verweist).

primaryTable
Tabellenname der Primärtabelle (derjenigen, auf die der Fremdschlüssel verweist).

primaryColumn
Name der Spalte, auf die der Fremdschlüssel verweist.

foreignCatalog
Name des Katalogs, in dem die Tabelle erscheint, die den Fremdschlüssel enthält.

foreignSchema
Name des Schemas, in dem die Tabelle erscheint, die den Fremdschlüssel enthält.

foreignTable
Name der Tabelle, die den Fremdschlüssel enthält.

foreignColumn
Name der Spalte, die im Fremdschlüssel erscheint.

ordinalPosition
Position der Spalte im Fremdschlüssel, wenn der Schlüssel ein zusammengesetzter Schlüssel ist. Der Objektbefehl **begintransaction** auf einer Datenbankverbindung beginnt eine Transaktion auf der Datenbank. Unterstützt die zugrunde liegende Datenbank keine atomaren, konsistenten, isolierten und dauerhaften Transaktionen, gibt der Objektbefehl **begintransaction** einen Fehler zurück, der diesen Umstand meldet. Ebenso wird ein Fehler zurückgegeben, wenn mehrere Befehle **begintransaction** ohne einen dazwischenliegenden Befehl **commit** oder **rollback** ausgeführt werden, es sei denn, die zugrunde liegende Datenbank unterstützt verschachtelte Transaktionen. Der Objektbefehl **commit** auf einer Datenbankverbindung beendet die jüngste durch **begintransaction** begonnene Transaktion und schreibt die Änderungen in der Datenbank fest. Der Objektbefehl **rollback** auf einer Datenbankverbindung setzt die jüngste durch **begintransaction** begonnene Transaktion zurück. Der Zustand der Datenbank ist so, als wäre während der Transaktion nichts geschehen. Der Objektbefehl **transaction** auf einer Datenbankverbindung bietet einen einfachen Weg, eine Datenbanktransaktion zusammenzufassen. Er beginnt eine Transaktion und wertet das übergebene Argument *script* als Tcl-Skript im Geltungsbereich des Aufrufers aus. Endet *script* normal oder durch **break**, **continue** oder **return**, wird die Transaktion festgeschrieben (und jede von **break**, **continue** oder **return** verlangte Aktion findet statt). Schlägt das Festschreiben aus irgendeinem Grund fehl, wird der Fehler beim Festschreiben als Fehler im *script* behandelt. Bei einem Fehler in *script* oder beim Festschreiben wird die Transaktion zurückgesetzt und der Fehler erneut gemeldet. Jeder abweichende Rückgabecode des Skripts setzt die Transaktion zurück und wird danach erneut gemeldet. Der Objektbefehl **allrows** bereitet eine SQL-Anweisung vor (gegeben durch den Parameter *sql-code*), die gegen die Datenbank ausgeführt werden soll. Danach führt er sie aus (Einzelheiten bei **tdbc::statement**), wobei der optionale Parameter *dictionary* gebundene Variablen angibt. Anschließend verwendet er den Objektbefehl *allrows* auf der Ergebnismenge (siehe **tdbc::resultset**), um eine Liste der Ergebnisse aufzubauen. Anschließend werden Ergebnismenge und Anweisung geschlossen. Der Rückgabewert ist die Liste der Ergebnisse. Der Objektbefehl **foreach** bereitet eine SQL-Anweisung vor (gegeben durch den Parameter *sql-code*), die gegen die Datenbank ausgeführt werden soll. Danach führt er sie aus (Einzelheiten bei **tdbc::statement**), wobei der optionale Parameter *dictionary* gebundene Variablen angibt. Anschließend verwendet er den Objektbefehl *foreach* auf der Ergebnismenge (siehe **tdbc::resultset**), um das gegebene *script* für jede Ergebniszeile auszuwerten. Anschließend werden Ergebnismenge und Anweisung geschlossen, auch wenn das gegebene *script* ein **return**, einen Fehler oder einen ungewöhnlichen Rückgabecode ergibt.

KONFIGURATIONSOPTIONEN

Die Konfigurationsoptionen, die beim Erzeugen der Verbindung und beim Objektbefehl **configure** der Verbindung akzeptiert werden, umfassen die folgenden und dürfen weitere umfassen, die für einen Datenbanktreiber spezifisch sind.

-encoding *name*

Legt die Kodierung fest, die für die Verbindung zur Datenbank verwendet wird. *name* soll einer der Namen sein, die der Befehl **encoding** akzeptiert. Diese Option ist gewöhnlich unnötig; die meisten Datenbanktreiber ermitteln die verwendete Kodierung selbst.

-isolation *level*
Legt die Isolationsstufe fest, die Transaktionen auf der Datenbank brauchen. Die zulässigen Werte für *level* stehen unter **ISOLATIONSSTUFEN VON TRANSAKTIONEN**.

-timeout *ms*
Legt die Höchstdauer fest, die auf eine Operation der Datenbank-Engine gewartet wird, bevor dem Aufrufer ein Fehler gemeldet wird. Das Argument *ms* gibt die Höchstdauer in Millisekunden an. Der Wert null (die Vorgabe) legt fest, dass der aufrufende Prozess unbegrenzt auf Datenbankoperationen wartet.

-readonly *flag*
Legt fest, dass die Verbindung die Datenbank nicht verändert (wenn der boolesche Parameter *flag* wahr ist) oder dass sie die Datenbank verändern darf (wenn *flag* falsch ist). Ist *flag* wahr, kann diese Option bewirken, dass die Isolationsstufe auf *readonly* angehoben wird.

ISOLATIONSSTUFEN VON TRANSAKTIONEN

Die zulässigen Werte für die Konfigurationsoption **-isolation** sind:

readuncommitted
Erlaubt der Transaktion, "dirty", also unbestätigte Daten zu lesen. Diese Isolationsstufe kann die Datenintegrität gefährden, garantiert nicht, dass Fremdschlüssel oder Eindeutigkeitsbedingungen erfüllt sind, und garantiert allgemein keine Datenkonsistenz.

readcommitted
Verbietet der Transaktion, "dirty"-Daten zu lesen, garantiert aber keine wiederholbaren Lesevorgänge. Liest eine Transaktion zu einem Zeitpunkt eine Zeile der Datenbank, gibt es keine Garantie, dass dieselbe Zeile später in derselben Transaktion noch verfügbar ist.

repeatableread
Garantiert, dass jede einmal gelesene Zeile der Datenbank für die Dauer einer Transaktion dieselben Werte behält. "Phantom-Reads" bleiben zulässig (neu hinzugekommene Zeilen, die erscheinen, wenn eine Tabelle ein zweites Mal abgefragt wird).

serializable
Die strengste (und teuerste) Isolationsstufe. Jede Abfrage an die Datenbank liefert, wird sie wiederholt, für die Dauer der Transaktion genau dieselben Ergebnisse, so als wäre die Transaktion der einzige Benutzer der Datenbank.

readonly
Verhält sich wie **serializable**, insofern der Transaktion nur Ergebnisse sichtbar sind, die vor dem Beginn der Transaktion festgeschrieben wurden, verbietet der Transaktion aber, die Datenbank zu verändern. Eine Datenbank, die eine dieser Isolationsstufen nicht umsetzt, verwendet stattdessen die nächststrengere Isolationsstufe. Lässt sich die angegebene Isolationsstufe nicht erreichen, meldet die Datenbankschnittstelle einen Fehler, der diesen Umstand nennt. Die voreingestellte Isolationsstufe ist **readcommitted**. Ein Skript soll die Isolationsstufe nicht ändern, während eine Transaktion läuft.

COPYRIGHT

Copyright (c) 2008 by Kevin B. Kenny.

tdbc_mapSqlState

Stichwörter: tdbc_mapSqlState, TDBC, SQL, database, state

NAME

tdbc::mapSqlState - SQLSTATE einer Fehlerklasse zuordnen

ÜBERSICHT

package require tdbc 1.0

tdbc::mapSqlState *sqlstate*

BESCHREIBUNG

Der Befehl **tdbc::mapSqlState** nimmt eine Zeichenkette entgegen, die ein fünf Zeichen langes "SQL State" sein soll, wie es eine SQL-Datenbank bei einem Fehler zurückgibt. Er prüft die ersten beiden Zeichen der Zeichenkette und gibt eine Fehlerklasse als für Mensch und Maschine lesbaren Namen zurück (zum Beispiel **FEATURE_NOT_SUPPORTED**, **DATA_EXCEPTION** oder **INVALID_CURSOR_STATE**).

Die TDBC-Spezifikation verlangt, dass Datenbanktreiber bei einem Fehler eine Beschreibung des Fehlers im Fehlercode zurückgeben. Die Beschreibung ist eine Zeichenkette mit mindestens vier Elementen: "**TDBC** *errorClass sqlstate driverName details...*". Der Befehl **tdbc::mapSqlState** gibt einem TDBC-Treiber einen bequemen Weg, das Element *errorClass* aus dem von der Datenbank zurückgegebenen SQL State zu bilden.

COPYRIGHT

Copyright (c) 2009 by Kevin B. Kenny.

tdbc_resultset

Stichwörter: tdbc_resultset, TDBC, SQL, database, connectivity, connection, resultset, statement, bound variable, stored procedure, call

NAME

tdbc::resultset - TDBC-Objekt für eine Ergebnismenge

ÜBERSICHT

package require tdbc 1.0
package require tdbc::driver version

tdbc::driver::connection create *db* ?*-option value* ...?

set *stmt* **[db prepare** *sql-code]*
set *resultset* **[$stmt execute** ?*args...?]*?

$resultset **columns**
$resultset **rowcount**
$resultset **nextrow** ?*-as lists|dicts*? ?**--**? *varname*
$resultset **nextlist** *varname*
$resultset **nextdict** *varname*
$resultset **nextresults**
$resultset **foreach** ?*-as lists|dicts*? ?*-columnsvariable name*? ?**--**? *varname script*

$resultset **close**

.in 14 *$resultset* **allrows** ?**-as lists|dicts**? ?**-columnsvariable** *name*? ?**--**?

BESCHREIBUNG

Jeder Datenbanktreiber für TDBC (Tcl DataBase Connectivity) setzt ein Objekt *Ergebnismenge* um, das die Ergebnisse darstellt, die das Ausführen einer SQL-Anweisung in einer Datenbank zurückgibt. Instanzen dieses Objekts entstehen, indem der Objektbefehl **execute** auf einem Anweisungsobjekt ausgeführt wird.

Der Objektbefehl **columns** gibt eine Liste der Namen der Spalten in der Ergebnismenge zurück. Die Spalten erscheinen in derselben Reihenfolge, in der sie in der SQL-Anweisung standen, die die Datenbankabfrage ausgeführt hat. Gibt die SQL-Anweisung keine Menge von Spalten zurück (zum Beispiel, wenn es eine Anweisung INSERT, UPDATE oder DELETE ist), gibt der Befehl **columns** eine leere Liste zurück.

Der Objektbefehl **rowcount** gibt die Anzahl der Zeilen in der Datenbank zurück, die das Ausführen einer Anweisung INSERT, UPDATE oder DELETE betroffen hat. Für eine Anweisung SELECT ist die Zeilenzahl unbestimmt.

Der Objektbefehl **nextlist** setzt die durch *varname* gegebene Variable im Geltungsbereich des Aufrufers auf die nächste Ergebniszeile, ausgedrückt als Liste von Spaltenwerten. NULL-Werte werden durch leere Zeichenketten ersetzt. Die Spalten der Ergebniszeile erscheinen in derselben Reihenfolge, in der sie in der Anweisung SELECT standen. Die Rückgabe von **nextlist** ist **1**, wenn die Operation gelungen ist, und **0**, wenn das Ende der Ergebnismenge erreicht wurde.

Der Objektbefehl **nextdict** setzt die durch *varname* gegebene Variable im Geltungsbereich des Aufrufers auf die nächste Ergebniszeile, ausgedrückt als Dictionary. Die Schlüssel des Dictionary sind Spaltennamen, die Werte sind die Werte dieser Spalten in der Zeile. Ist der Wert einer Spalte in der Zeile NULL, entfällt ihr Schlüssel im Dictionary. Die Schlüssel erscheinen im Dictionary in derselben Reihenfolge, in der die Spalten in der Anweisung SELECT standen. Die Rückgabe von **nextdict** ist **1**, wenn die Operation gelungen ist, und **0**, wenn das Ende der Ergebnismenge erreicht wurde.

Der Objektbefehl **nextrow** ist genau gleichwertig zum Objektbefehl **nextdict** oder **nextlist**, je nachdem, ob **-as dicts** (die Vorgabe) oder **-as lists** angegeben ist.

Manche Datenbanken kennen die Vorstellung einer einzelnen Anweisung, die mehrere Ergebnismengen zurückgibt. Der Objektbefehl **nextresults** wird ausgeführt, gewöhnlich nachdem der Objektbefehl **nextlist** oder **nextdict 0** zurückgegeben hat, um zur nächsten Ergebnismenge vorzurücken. Er gibt **1** zurück, wenn eine weitere Ergebnismenge zu verarbeiten ist, und **0**, wenn die gerade verarbeitete Ergebnismenge die letzte war. Nach dem Aufruf von **nextresults** mit dem Rückgabewert **1** darf der Aufrufer erneut **columns** aufrufen, um die Spaltenbeschreibungen der nächsten Ergebnismenge zu erhalten, und danach wieder **nextdict** oder **nextlist** aufrufen, um die Zeilen der nächsten Ergebnismenge zu verarbeiten. Es ist ein Fehler, **columns**, **nextdict**, **nextlist** oder **nextrow** aufzurufen, nachdem **nextresults 0** zurückgegeben hat.

Der Objektbefehl **allrows** setzt die durch die Option **-columnsvariable** bezeichnete Variable (falls vorhanden) auf das Ergebnis des Objektbefehls **columns**. Danach führt er den Objektbefehl **nextrow** wiederholt aus, bis das Ende der Ergebnismenge erreicht ist. Gibt **nextresults** einen von null verschiedenen Wert zurück, führt er die beiden genannten Schritte

aus (**columns**, gefolgt von wiederholten Aufrufen von **nextrow**), solange weitere Ergebnisse vorliegen. Die von **nextrow** zurückgegebenen Zeilen werden zu einer Tcl-Liste zusammengesetzt und werden der Rückgabewert des Befehls **allrows**; der letzte von **columns** zurückgegebene Wert ist das, was die Anwendung in **-columnsvariable** sieht.

Der Objektbefehl **foreach** setzt die durch die Option **-columnsvariable** bezeichnete Variable (falls vorhanden) auf das Ergebnis des Objektbefehls **columns**. Danach führt er den Objektbefehl **nextrow** wiederholt aus, bis das Ende der Ergebnismenge erreicht ist, und speichert die aufeinanderfolgenden Zeilen in der durch *varName* bezeichneten Variable. Für jede Zeile führt er das gegebene *script* aus. Endet das Skript mit einem Fehler, meldet der Befehl **foreach** den Fehler, und die Wiederholung endet. Führt das Skript eine Operation **break** aus, endet die Wiederholung vorzeitig. Führt das Skript eine Operation **continue** aus, setzt die Wiederholung mit der nächsten Zeile fort. Führt das Skript ein **return** aus, sind die Ergebnisse dieselben, als hätte ein Skript außerhalb der Steuerung von **foreach** zurückgegeben. Jeder andere ungewöhnliche Rückgabecode beendet die Wiederholung und wird von **foreach** gemeldet.

Sobald **nextrow 0** zurückgibt, versucht der Objektbefehl **foreach**, mit **nextresults** zur nächsten Ergebnismenge vorzurücken. Gibt **nextresults 1** zurück, werden die genannten Schritte (**columns** und **nextrow**, mit Aufruf des Skripts) wiederholt, solange weitere Ergebnismengen bleiben. Das *script* sieht die richtige Beschreibung der Spalten der aktuellen Ergebnismenge stets in der durch **-columnsvariable** bezeichneten Variable. Am Ende des Aufrufs enthält die durch **-columnsvariable** bezeichnete Variable die Beschreibung der Spalten der letzten Ergebnismenge.

Der Objektbefehl **close** löscht die Ergebnismenge und gibt alle zugehörigen Systemressourcen frei.

COPYRIGHT

Copyright (c) 2008 by Kevin B. Kenny.

tdbc_statement

Stichwörter: tdbc_statement, TDBC, SQL, database, connectivity, connection, resultset, statement, bound variable, stored procedure, call

NAME

tdbc::statement - TDBC-Objekt für eine Anweisung

ÜBERSICHT

package require tdbc 1.0
package require tdbc::driver version

tdbc::driver::connection create *db* ?*-option value* ...?

set *stmt* **[db prepare** *sql-code]*
set *stmt* **[db preparecall** *call]*

$stmt **params**
$stmt **paramtype** ?*direction*? *type* ?*precision*? ?*scale*?
$stmt **execute** ?*dict*?
$stmt **resultsets**

$stmt **foreach** ?*-as lists|dicts*? ?*-columnsvariable name*? ?**--**? *varName* ?*dict*? *script*
$stmt **close**

.in 14 *$stmt* **allrows** ?**-as lists|dicts**? ?**-columnsvariable** *name*? ?**--**? ?*dict*

BESCHREIBUNG

Jeder Datenbanktreiber für TDBC (Tcl DataBase Connectivity) setzt ein Objekt *Anweisung* um, das eine SQL-Anweisung in einer Datenbank darstellt. Instanzen dieses Objekts entstehen, indem der Objektbefehl **prepare** oder **preparecall** auf einer Datenbankverbindung ausgeführt wird.

Der Objektbefehl **prepare** auf der Verbindung nimmt beliebigen SQL-Code entgegen, der gegen die Datenbank ausgeführt werden soll. Der SQL-Code darf *gebundene Variablen* enthalten. Das sind Zeichenketten aus alphanumerischen Zeichen oder Unterstrichen (das erste Zeichen der Zeichenkette darf nicht numerisch sein), denen ein Doppelpunkt (**:**) vorangestellt ist. Erscheint eine gebundene Variable in der SQL-Anweisung und steht sie weder in einer durch einfache oder doppelte Anführungszeichen abgesetzten Zeichenkette noch in einem mit **--** eingeleiteten Kommentar, wird sie zu einem Wert, der beim Ausführen der Anweisung eingesetzt wird. Eine gebundene Variable wird in der entstehenden Anweisung zu einem einzelnen Wert (Zeichenkette oder numerisch). *Die Treiber haben dafür zu sorgen, dass der Mechanismus zum Binden von Variablen SQL-Injection verhindert.*

Der Objektbefehl **preparecall** auf der Verbindung nimmt eine stilisierte Anweisung in der Form entgegen:

```
procname (?:varname? ?,:varname...?)
```

oder

```
varname = procname (?:varname? ?,:varname...?)
```

Diese Anweisung stellt einen Aufruf einer gespeicherten Prozedur *procname* in der Datenbank dar. Der Variablenname links vom Gleichheitszeichen (falls vorhanden) und alle Variablennamen, die Parameter innerhalb der Klammern sind, werden zu gebundenen Variablen.

Die Methode **params** auf einem Anweisungsobjekt zählt die gebundenen Variablen auf, die in der Anweisung vorkommen. Das von der Methode **params** zurückgegebene Ergebnis ist ein Dictionary, dessen Schlüssel die Namen der gebundenen Variablen sind (aufgeführt in der Reihenfolge, in der die Variablen zuerst in der Anweisung erscheinen) und dessen Werte Dictionarys sind. Die untergeordneten Dictionarys enthalten mindestens die folgenden Schlüssel (Datenbanktreiber dürfen weitere Schlüssel hinzufügen, die nicht in dieser Liste stehen).

direction
: Enthält eines der Schlüsselwörter **in**, **out** oder **inout**, je nachdem, ob die Variable eine Eingabe für die Anweisung oder eine Ausgabe aus ihr ist. Nur Aufrufe gespeicherter Prozeduren haben Parameter **out** oder **inout**.

type
: Enthält den Datentyp der Spalte und wird im Allgemeinen aus der Menge **bigint**, **binary**, **bit**, **char**, **date**, **decimal**, **double**, **float**, **integer**, **longvarbinary**, **longvarchar**, **numeric**, **real**, **time**, **timestamp**, **smallint**, **tinyint**, **varbinary** und **varchar** gewählt. (Hat die Variable einen Typ, der sich nicht als einer der genannten darstellen lässt, enthält **type** eine treiberabhängige Beschreibung des Typs.)

precision
Enthält die Präzision der Spalte in Bit, Dezimalziffern oder die Breite in Zeichen, je nach Typ.

scale
Enthält die Skala der Spalte (die Anzahl der Ziffern nach dem Radixpunkt) für Typen, die dieses Konzept kennen.

nullable
Enthält 1, wenn die Spalte NULL-Werte enthalten kann, und andernfalls 0. Der Objektbefehl **paramtype** erlaubt dem Skript, Typ und Richtung der Parameterübertragung einer Variable in einer Anweisung festzulegen. (Manche Datenbanken bieten keine Methode, diese Information automatisch zu ermitteln, und legen die Last dafür auf den Aufrufer.) Die Argumente *direction* , *type*, *precision*, *scale* und *nullable* haben dieselbe Bedeutung wie die entsprechenden Dictionary-Werte im Objektbefehl **params**. Der Objektbefehl **execute** führt die Anweisung aus. Vor dem Ausführen der Anweisung werden Werte für die gebundenen Variablen bereitgestellt, die in ihr vorkommen. Ist der Parameter *dict* angegeben, wird darin nach einem Schlüssel gesucht, dessen Name zum Namen der gebundenen Variable passt. Ist der Schlüssel vorhanden, wird sein Wert zur eingesetzten Variable. Andernfalls wird der Wert der eingesetzten Variable ein SQL-NULL. Ist der Parameter *dict nicht* angegeben, sucht der Objektbefehl **execute** im Geltungsbereich des Aufrufers nach einer Variable, deren Name zum Namen der gebundenen Variable passt. Wird eine gefunden, wird ihr Wert zum Wert der gebundenen Variable. Wird keine gefunden, erhält die gebundene Variable ein SQL-NULL als Wert. Sobald die Ersetzung abgeschlossen ist, wird die entstehende Anweisung ausgeführt. Der Rückgabewert ist ein Ergebnismengenobjekt (Einzelheiten stehen bei **tdbc::resultset**). Die Methode **resultsets** gibt eine Liste aller Ergebnismengen zurück, die das Ausführen der Anweisung geliefert hat und die noch nicht geschlossen wurden. Der Objektbefehl **allrows** führt die Anweisung wie der Objektbefehl **execute** aus und nimmt einen optionalen Parameter *dict* entgegen, der gebundene Variablen angibt. Nach dem Ausführen der Anweisung verwendet er den Objektbefehl *allrows* auf der Ergebnismenge (siehe **tdbc::resultset**), um eine Liste der Ergebnisse aufzubauen. Anschließend wird die Ergebnismenge geschlossen. Der Rückgabewert ist die Liste der Ergebnisse. Der Objektbefehl **foreach** führt die Anweisung wie der Objektbefehl **execute** aus und nimmt einen optionalen Parameter *dict* entgegen, der gebundene Variablen angibt. Nach dem Ausführen der Anweisung verwendet er den Objektbefehl *foreach* auf der Ergebnismenge (siehe **tdbc::resultset**), um das gegebene *script* für jede Ergebniszeile auszuwerten. Anschließend wird die Ergebnismenge geschlossen, auch wenn das gegebene *script* ein **return**, einen Fehler oder einen ungewöhnlichen Rückgabecode ergibt. Der Objektbefehl **close** entfernt eine Anweisung und alle Ergebnismengen, die sie erzeugt hat. Alle den Objekten zugeordneten Systemressourcen werden freigegeben.

BEISPIELE

Der folgende Code schlägt eine Telefonnummer in einem Verzeichnis nach, ein passendes SQL-Schema vorausgesetzt:

```
package require tdbc::sqlite3
tdbc::sqlite3::connection create db phonebook.sqlite3
set statement [db prepare {
    select phone_num from directory
```

```
    where first_name = :firstname and last_name = :lastname
}]
set firstname Fred
set lastname Flintstone
$statement foreach row {
    puts [dict get $row phone_num]
}
$statement close
db close
```

COPYRIGHT

Copyright (c) 2008 by Kevin B. Kenny.

tdbc_tokenize

Stichwörter: tdbc_tokenize, TDBC, SQL, database, tokenize

NAME

tdbc::tokenize - SQL-Tokenizer für TDBC

ÜBERSICHT

package require tdbc 1.0

tdbc::tokenize *string*

BESCHREIBUNG

Als Hilfe für Datenbanktreiber stellt Tcl Database Connectivity (TDBC) einen Befehl bereit, der SQL-Code in Token zerlegt, damit gebundene Variablen leicht erkannt und eingesetzt werden können.

Der Befehl **tdbc::tokenize** nimmt als Parameter eine Zeichenkette entgegen, die eine oder mehrere SQL-Anweisungen enthalten soll. Er gibt eine Liste von Teilzeichenketten zurück; aneinandergereiht ergeben diese Teilzeichenketten wieder die ursprüngliche Zeichenkette. Jede Teilzeichenkette ist eines der Folgenden:

[1]
Eine gebundene Variable, die mit einem der Zeichen "**:**", "**@**" oder "**$**" beginnt. Der Rest der Zeichenkette ist der Variablenname und besteht aus alphanumerischen Zeichen und Unterstrichen. (Das führende Zeichen ist nicht numerisch.)

[2]
Ein Semikolon, das zwei SQL-Anweisungen trennt.

[3]
Etwas anderes in einer SQL-Anweisung. Der Tokenizer versucht nicht, SQL zu parsen; er erkennt lediglich gebundene Variablen (und unterscheidet sie von ähnlichen Zeichenketten innerhalb von Anführungszeichen oder Kommentaren) und die Trennzeichen von Anweisungen.

COPYRIGHT

Copyright (c) 2008 by Kevin B. Kenny.

tdbcmysql

tdbc_mysql

Stichwörter: tdbc_mysql, TDBC, SQL, MySQL, database, connectivity, connection

NAME

tdbc::mysql - TDBC-MYSQL-Brücke

ÜBERSICHT

package require tdbc::mysql 1.0
tdbc::mysql::connection create *db* ?*-option value* ...?
tdbc::mysql::connection new ?*-option value* ...?
tdbc::mysql::datasources ?**-system**|**-user**?
tdbc::mysql::drivers
tdbc::mysql::datasource *command driverName* ?*keyword-value?* ...?

BESCHREIBUNG

Der Treiber **tdbc::mysql** stellt eine Datenbankschnittstelle bereit, die Tcl DataBase Connectivity (TDBC) entspricht, und erlaubt einem Tcl-Skript, sich mit einer MySQL-Datenbank zu verbinden.

Die Verbindung zu einer MYSQL-Datenbank entsteht durch den Aufruf von **tdbc::mysql::connection create**. Übergeben werden der Name, der dem Datenbankhandle gegeben werden soll, und eine Menge von Paaren *-option-value*. Die verfügbaren Optionen sind unter **VERBINDUNGSOPTIONEN** weiter unten aufgezählt. Als Alternative darf **tdbc::mysql::connection new** eine Datenbankverbindung mit automatisch vergebenem Namen erzeugen. Der Rückgabewert von **tdbc::mysql::connection new** ist der Name, der für das Verbindungshandle gewählt wurde.

Die Nebenwirkung von **tdbc::mysql::connection create** ist, eine neue Datenbankverbindung zu erzeugen. **tdbc::connection(n)** beschreibt, wie die Verbindung zum Bearbeiten einer Datenbank verwendet wird.

VERBINDUNGSOPTIONEN

Der Objektbefehl **tdbc::mysql::connection create** unterstützt die Optionen **-encoding**, **-isolation**, **-readonly** und **-timeout**, die allen TDBC-Treibern gemeinsam sind. Die Option **-encoding** schlägt stets fehl, es sei denn, die Kodierung ist **utf-8**; die Datenbankverbindung verwendet stets die Kodierung UTF-8, um beliebige Unicode-Zeichen übertragen zu können. Die Option **-readonly** muss **0** sein, weil MySQL keine nur lesenden Verbindungen anbietet.

Zusätzlich werden die folgenden Optionen erkannt:

-host *hostname*
: Verbindet mit dem durch *hostname* angegebenen Host. Diese Option muss beim ersten Erzeugen der Verbindung gesetzt werden; nach dem Verbinden lässt sie sich nicht mehr ändern. Vorgegeben ist die Verbindung zum lokalen Host.

-port *number*
: Verbindet mit einem MySQL-Server, der auf dem durch *number* angegebenen Port lauscht. Diese Option lässt sich nach dem Verbinden nicht mehr ändern. Sie wird nur verwendet, wenn *host* angegeben ist und nicht **localhost** ist.

-socket *path*
Verbindet mit einem MySQL-Server, der auf dem Unix-Socket oder der benannten Pipe lauscht, die *path* angibt. Diese Option lässt sich nach dem Verbinden nicht mehr ändern. Sie wird nur verwendet, wenn *-host* nicht angegeben ist oder **localhost** ist.

-user *name*
Legt *name* dem MySQL-Server als Benutzernamen vor. Vorgegeben ist die aktuelle Benutzerkennung.

-passwd *password*
...siehe nächstes...

-password *password*
Diese beiden Optionen sind gleichbedeutend. Sie legen das gegebene *password* dem MySQL-Server als Passwort des Benutzers vor. Vorgegeben ist, kein Passwort vorzulegen.

-database *name*
...siehe nächstes...

-db *name*
Diese beiden Optionen sind gleichbedeutend. Sie legen den gegebenen *name* als Namen der voreingestellten Datenbank vor, die in MySQL-Abfragen verwendet wird. Ist nichts angegeben, wird die voreingestellte Datenbank des aktuellen Benutzers verwendet.

-interactive *flag*
Der Wert *flag* muss ein boolescher Wert sein. Ist er **true** (oder ein gleichwertiger Wert), wird das voreingestellte Zeitlimit für einen interaktiven Benutzer gesetzt, andernfalls das voreingestellte Zeitlimit für einen Benutzer im Stapelbetrieb. Diese Option ist nur bei der ersten Verbindung von Bedeutung. Wird die Methode **configure** auf einer MySQL-Verbindung verwendet, setzt die Option **-timeout** das gewünschte Zeitlimit.

-ssl_ca *string*
...siehe nächstes...

-ssl_capath *string*
...siehe nächstes...

-ssl_cert *string*
...siehe nächstes...

-ssl_cipher *string*
...siehe nächstes...

-ssl_key *string*
Diese fünf Optionen setzen die Zertifizierungsstelle, den Suchpfad der Zertifizierungsstelle, das SSL-Zertifikat, die Übertragungschiffre und den SSL-Schlüssel auf die gegebenen Argumente *string*. Diese Optionen dürfen nur bei der ersten Verbindung zu einer Datenbank angegeben werden, nicht in der Methode **configure** einer bestehenden Verbindung. Vorgegeben ist, SSL nicht zu verwenden.

BEISPIELE

```
tdbc::mysql::connection -user joe -passwd sesame -db joes_database
```

Verbindet mit dem MySQL-Server auf dem lokalen Host über die voreingestellte Verbindungsmethode, legt die Benutzerkennung "joe" und das Passwort "sesame" vor. Verwendet "joes_database" als voreingestellten Datenbanknamen.

WEITERE VERBINDUNGSMETHODEN

Zusätzlich zu den üblichen Methoden des Objekts tdbc::connection(n) unterstützen Verbindungen zu einer MySQL-Datenbank eine weitere Methode:

$connection* evaldirect *sqlStatement

Diese Methode nimmt die gegebene *sqlStatement* entgegen, deutet sie als MySQL-eigenen SQL-Code und wertet sie aus, ohne sie vorzubereiten. Die Anweisung darf keine Variablenersetzungen enthalten. Die Ergebnismenge wird als Liste von Listen zurückgegeben; jede Unterliste ist die Liste der Spalten einer Ergebniszeile, formatiert als Zeichenketten. Die Formatierung der Zeichenketten übernimmt MySQL und nicht Tcl, sodass Einzelheiten wie das Aussehen von Gleitkommazahlen abweichen können. *Dieser Befehl ist nicht zu empfehlen* für alles, bei dem die üblichen Methoden *prepare* oder *preparecall* richtig arbeiten. Er liegt vor, damit Anweisungen der Datenverwaltungssprache ausgeführt werden können, die in der API für vorbereitete Anweisungen von MySQL nicht umgesetzt sind, etwa **CREATE DATABASE** oder **CREATE PROCEDURE**.

COPYRIGHT

Copyright (c) 2009 by Kevin B. Kenny.

Copyright (c) 2008 by Kevin B. Kenny.

tdbcodbc

tdbc_odbc

Stichwörter: tdbc_odbc, TDBC, SQL, ODBC, database, connectivity, connection

NAME

tdbc::odbc - TDBC-ODBC-Brücke

ÜBERSICHT

package require tdbc::odbc 1.0
tdbc::odbc::connection create *db connectionString* ?*-option value* ...?
tdbc::odbc::connection new *connectionString* ?*-option value* ...?
tdbc::odbc::datasources ?**-system|-user**?
tdbc::odbc::drivers
tdbc::odbc::datasource *command driverName* ?*keyword-value?* ...?

BESCHREIBUNG

Der Treiber **tdbc::odbc** stellt eine Datenbankschnittstelle bereit, die Tcl DataBase Connectivity (TDBC) entspricht, und erlaubt einem Tcl-Skript, sich mit jeder SQL-Datenbank zu verbinden, die eine ODBC-Schnittstelle anbietet. Er liegt außerdem als ausgearbeitetes Beispiel dafür vor, wie ein Datenbanktreiber in C geschrieben wird, damit Autoren von Treibern einen Ausgangspunkt für die weitere Entwicklung haben.

Die Verbindung zu einer ODBC-Datenbank entsteht durch den Aufruf von **tdbc::odbc::connection create**. Übergeben werden der Name, der als Verbindungshandle dienen soll, und danach eine übliche ODBC-Verbindungszeichenkette. Als Alternative darf **tdbc::odbc::connection new** eine Datenbankverbindung mit automatisch vergebenem Namen

erzeugen. Der Rückgabewert von **tdbc::odbc::connection new** ist der Name, der für das Verbindungshandle gewählt wurde.

Die Verbindungszeichenkette enthält mindestens ein Schlüsselwort **DRIVER** oder **DSN** und darf weitere enthalten, die ein bestimmter ODBC-Treiber festlegt. (Unterstützt das lokale ODBC-System eine grafische Benutzeroberfläche, kann die Option **-parent** (siehe unten) erlauben, **tdbc::odbc::connection create** mit einer leeren Verbindungszeichenkette aufzurufen.)

Die Nebenwirkung von **tdbc::odbc::connection create** ist, eine neue Datenbankverbindung zu erzeugen. **tdbc::connection(n)** beschreibt, wie die Verbindung zum Bearbeiten einer Datenbank verwendet wird.

Zusätzlich zu einer üblichen TDBC-Schnittstelle unterstützt **tdbc::odbc** drei weitere Befehle. Der erste davon, **tdbc::odbc::datasources**, gibt eine Tcl-Liste zurück, die die benannten Datenquellen aufzählt, die dem Programm zur Verfügung stehen (für die Verbindung mit dem Schlüsselwort **DSN** in der Verbindungszeichenkette). Das Ergebnis von **tdbc::odbc::datasources** lässt sich auf nur Systemdatenquellen oder nur Benutzerdatenquellen einschränken, indem die Optionen **-system** bzw. **-user** angegeben werden.

Der Befehl **tdbc::odbc::drivers** gibt ein Dictionary zurück, dessen Schlüssel die Namen der Treiber sind, die für das Schlüsselwort **DRIVER** in der Verbindungszeichenkette zur Verfügung stehen, und dessen Werte Beschreibungen der Treiber sind.

Der Befehl **tdbc::odbc::datasource** erlaubt das Konfigurieren benannter Datenquellen auf den Systemen, die die Programmierschnittstelle des ODBC-Installers unterstützen. Er nimmt ein *command* entgegen. Dieses *command* legt fest, welche Operation ausgeführt wird. Dazu kommen der Name eines *driver* für die betreffende Datenbank und eine Menge von Schlüsselwort-Wert-Paaren, die der gegebene Treiber deutet. *command* muss eines der Folgenden sein:

add
: Fügt eine Benutzerdatenquelle hinzu. Die Schlüsselwort-Wert-Paare müssen mindestens eine Option **DSN** enthalten, die die Datenquelle benennt.

add_system
: Fügt eine Systemdatenquelle hinzu. Die Schlüsselwort-Wert-Paare müssen mindestens eine Option **DSN** enthalten, die die Datenquelle benennt.

configure
: Konfiguriert eine Benutzerdatenquelle. Die Schlüsselwort-Wert-Paare enthalten gewöhnlich eine Option **DSN**, die die Datenquelle benennt. Manche Treiber unterstützen weitere Optionen, etwa die Option **CREATE_DB** des Microsoft-Access-Treibers unter Windows.

configure_system
: Konfiguriert eine Systemdatenquelle.

remove
: Entfernt eine Benutzerdatenquelle. Die Schlüsselwort-Wert-Paare müssen eine Option **DSN** enthalten, die die zu entfernende Datenquelle angibt.

remove_system
: Entfernt eine Systemdatenquelle. Die Schlüsselwort-Wert-Paare müssen eine Option **DSN** enthalten, die die zu entfernende Datenquelle angibt.

VERBINDUNGSOPTIONEN

Der Objektbefehl **tdbc::odbc::connection create** unterstützt die Optionen **-encoding**, **-isolation**, **-readonly** und **-timeout**, die allen TDBC-Treibern gemeinsam sind. Die Option **-encoding** gelingt nur, wenn die verlangte Kodierung dieselbe ist wie die Systemkodierung; **tdbc::odbc** versucht nicht, einem ODBC-Treiber abweichende Kodierungen vorzugeben. (Manche Treiber akzeptieren Kodierungsangaben in der Verbindungszeichenkette.)

Ist zusätzlich Tk im anfordernden Interpreter vorhanden und unterstützt der ODBC-Treibermanager des lokalen Systems eine grafische Benutzeroberfläche, unterstützt der Objektbefehl **tdbc::odbc::connection create** eine Option **-parent**, deren Wert der Pfadname eines Tk-Fensters ist. Ist diese Option angegeben und nennt eine Verbindungszeichenkette nicht alle Angaben, die für die Verbindung zu einer Schnittstelle nötig sind, zeigt der ODBC-Treibermanager einen Dialog an, der die zusätzlich nötigen Angaben erfragt. Der anfordernde Interpreter blockiert, bis der Benutzer den Dialog schließt; dann wird die Verbindung hergestellt.

BEISPIELE

Da ODBC-Verbindungszeichenketten vom Treiber abhängen, ist die Dokumentation, die zum Zusammensetzen nötig ist, oft schwer zu finden. Die folgenden Beispiele arbeiten auf den meisten Windows-Systemen und zeigen zumindest einige nützliche Dinge, die ein Programm tun kann.

```
tdbc::odbc::connection create db \
    "DSN={PAYROLL};UID={aladdin};PWD={Sesame}"
```

Verbindet mit einer benannten Datenquelle "PAYROLL", gibt "aladdin" als Benutzernamen und "Sesame" als Passwort an. Verwendet **db** als Namen der Verbindung.

```
set connString {DRIVER={Microsoft Access Driver (*.mdb)};}
append connString {FIL={MS Access}\;}
append connString {DBQ=} \
    [file nativename [file normalize $fileName]]
tdbc::odbc::connection create db2 -readonly 1 $connString
```

Öffnet eine Verbindung zu einer Microsoft-Access-Datenbankdatei, deren Dateiname in *$fileName* steht. Die Datenbank wird nur zum Lesen geöffnet. Die entstehende Verbindung heißt "db2".

```
tdbc::odbc::connection create db3 \
    "DRIVER=SQLite3;DATABASE=$fileName"
```

Öffnet eine Verbindung zu einer SQLite3-Datenbank, deren Name in "$fileName" steht.

```
tdbc::odbc::datasource add \
    {Microsoft Access Driver (*.mdb)} \
    DSN=MyTestDatabase \
    DBQ=[file native [file normalize $fileName]]
```

Erzeugt eine neue Benutzerdatenquelle mit dem Namen "MyTestDatabase", gebunden an eine Microsoft-Access-Datei, deren Pfadname in "$fileName" steht. Eine Verbindung zur Datenquelle entsteht erst, wenn das Programm **tdbc::odbc::connection create** aufruft.

```
tdbc::odbc::datasource configure \
    {Microsoft Access Driver (*.mdb)} \
    CREATE_DB=[file native [file normalize $fileName]] \
    General
```

Erzeugt eine neue, leere Microsoft-Access-Datenbank in der durch "$fileName" bezeichneten Datei. Eine Verbindung zur Datenbank entsteht erst, wenn das Programm **tdbc::odbc::connection create** aufruft.

WEITERE VERBINDUNGSMETHODEN

Zusätzlich zu den üblichen Methoden des Objekts tdbc::connection(n) unterstützen Verbindungen zu einer ODBC-Datenbank eine weitere Methode:

$connection* evaldirect *sqlStatement

Diese Methode nimmt den gegebenen treibereigenen SQL-Code *sqlStatement* entgegen und wertet ihn aus, ohne ihn vorzubereiten. Die Anweisung wird nicht in Token zerlegt und darf keine Variablenersetzungen enthalten. Das Auswerten von *sqlStatement* erzeugt eine Ergebnismenge mit null oder mehr Zeilen. Das Ergebnis des Befehls ist eine Liste von Dictionarys mit einem Listenelement je Zeile in der Ergebnismenge (in einem ähnlichen Format wie die Liste, die *$connection allrows -as dicts* zurückgibt). *Dieser Befehl ist nicht zu empfehlen* für alles, bei dem die üblichen Methoden *prepare* oder *preparecall* richtig arbeiten. Er liegt vor, damit Anweisungen der Datenverwaltungssprache ausgeführt werden können, die die API für vorbereitete Anweisungen des Treibers nicht umsetzt (etwa **CREATE DATABASE** oder **CREATE PROCEDURE**), oder die Zeichen enthalten, die der Tokenizer reserviert.

COPYRIGHT

Copyright (c) 2008 by Kevin B. Kenny.

tdbcpostgres

tdbc_postgres

Stichwörter: tdbc_postgres, TDBC, SQL, Postgres, database, connectivity, connection

NAME

tdbc::postgres - TDBC-POSTGRES-Brücke

ÜBERSICHT

package require tdbc::postgres 1.0
tdbc::postgres::connection create *db* ?*-option value* ...?
tdbc::postgres::connection new ?*-option value* ...?

BESCHREIBUNG

Der Treiber **tdbc::postgres** stellt eine Datenbankschnittstelle bereit, die Tcl DataBase Connectivity (TDBC) entspricht, und erlaubt einem Tcl-Skript, sich mit einer Postgres-Datenbank zu verbinden.

Die Verbindung zu einer POSTGRES-Datenbank entsteht durch den Aufruf von **tdbc::postgres::connection create**. Übergeben werden der Name, der dem Datenbankhandle

gegeben werden soll, und eine Menge von Paaren *-option-value*. Die verfügbaren Optionen sind unter **VERBINDUNGSOPTIONEN** weiter unten aufgezählt. Als Alternative darf **tdbc::postgres::connection new** eine Datenbankverbindung mit automatisch vergebenem Namen erzeugen. Der Rückgabewert von **tdbc::postgres::connection new** ist der Name, der für das Verbindungshandle gewählt wurde.

Die Nebenwirkung von **tdbc::postgres::connection create** ist, eine neue Datenbankverbindung zu erzeugen. **tdbc::connection(n)** beschreibt, wie die Verbindung zum Bearbeiten einer Datenbank verwendet wird.

VERBINDUNGSOPTIONEN

Der Objektbefehl **tdbc::postgres::connection create** unterstützt die Optionen **-encoding**, **-isolation**, **-readonly** und **-timeout**, die allen TDBC-Treibern gemeinsam sind. Die Option **-timeout** wirkt sich nur auf den Verbindungsvorgang aus; einmal verbunden, wird dieser Wert ignoriert und lässt sich nach dem Verbinden nicht mehr ändern.

Zusätzlich werden die folgenden Optionen erkannt (diese Optionen müssen beim ersten Erzeugen der Verbindung gesetzt werden; nach dem Verbinden lassen sie sich nicht mehr ändern):

-host *hostname*
: Verbindet mit dem durch *hostname* angegebenen Host. Vorgegeben ist die Verbindung über einen lokalen Unix-Domain-Socket.

-hostaddr *address*
: Verbindet mit dem Host, den die gegebene IP-Adresse *address* angibt. Sind sowohl **-host** als auch **-hostaddr** angegeben, wird der Wert von **-host** ignoriert. Vorgegeben ist die Verbindung über einen lokalen Unix-Domain-Socket.

-port *number*
: Verbindet mit einem Postgres-Server, der auf dem durch *number* angegebenen Port lauscht. Die Option wird nur verwendet, wenn *host* oder *hostaddr* angegeben ist.

-user *name*
: Legt *name* dem Postgres-Server als Benutzernamen vor. Vorgegeben ist die aktuelle Benutzerkennung.

-passwd *password*
: ...siehe nächstes...

-password *password*
: Diese beiden Optionen sind gleichbedeutend. Sie legen das gegebene *password* dem Postgres-Server als Passwort des Benutzers vor. Vorgegeben ist, kein Passwort vorzulegen.

-database *name*
: ...siehe nächstes...

-db *name*
: Diese beiden Optionen sind gleichbedeutend. Sie legen den gegebenen *name* als Namen der voreingestellten Datenbank vor, die in Postgres-Abfragen verwendet wird. Ist nichts angegeben, wird die voreingestellte Datenbank des aktuellen Benutzers verwendet.

-options *opts*
: Setzt *opts* als zusätzliche Kommandozeilenoptionen, die an den Server gesendet werden.

-tty *file*

Diese Option wird auf neueren Servern ignoriert. Früher legte sie fest, wohin die Debug-Ausgabe gesendet wird. Die Option bleibt aus Kompatibilität mit älteren Servern erhalten.

-sslmode *mode*
Diese Option legt fest, ob und mit welcher Priorität eine SSL-Verbindung mit dem Server ausgehandelt wird. Es gibt vier *modes*: **disable** versucht nur eine unverschlüsselte SSL-Verbindung; **allow** handelt aus und versucht zuerst eine Verbindung ohne SSL, danach, wenn das scheitert, eine SSL-Verbindung; **prefer** (die Vorgabe) handelt aus und versucht zuerst eine SSL-Verbindung, danach, wenn das scheitert, eine gewöhnliche Verbindung ohne SSL; **require** versucht nur eine SSL-Verbindung. Ist PostgreSQL ohne SSL-Unterstützung übersetzt, verursacht die Option **require** einen Fehler, und die Optionen **allow** und **prefer** werden geduldet, der Treiber kann aber keine SSL-Verbindung aushandeln.

-requiressl *flag*
Diese Option ist zugunsten der Einstellung **-sslmode** veraltet. Der Wert *flag* muss ein boolescher Wert sein. Ist er **true** (oder ein gleichwertiger Wert), verweigert der Treiber die Verbindung, wenn der Server keine SSL-Verbindung annimmt. Der voreingestellte Wert ist **false** (oder ein gleichwertiger Wert) und wirkt wie **-sslmode preffered**

-service *name*
Legt fest, welcher Dienstname *name* in der Datei pg_service.conf gilt. Die Datei enthält weitere Verbindungsparameter. Dadurch können Anwendungen nur einen Dienstnamen angeben, sodass die Verbindungsparameter zentral gepflegt werden können. Einzelheiten stehen in der PostgreSQL-Dokumentation oder in der Datei PREFIX/share/pg_service.conf.sample.

BEISPIELE

```
tdbc::postgres::connection -user joe -passwd sesame -db joes_database
```

Verbindet mit dem Postgres-Server auf dem lokalen Host über die voreingestellte Verbindungsmethode, legt die Benutzerkennung "joe" und das Passwort "sesame" vor. Verwendet "joes_database" als voreingestellten Datenbanknamen.

COPYRIGHT

Copyright (c) 2009 by Slawomir Cygan

tdbcsqlite3

tdbc_sqlite3

Stichwörter: tdbc_sqlite3, TDBC, SQL, SQLite3, database, connectivity, connection

NAME

tdbc::sqlite3 - TDBC-Treiber für die Datenbankverwaltung SQLite3

ÜBERSICHT

package require tdbc::sqlite3 1.0
tdbc::sqlite3::connection create *db fileName* ?*-option value* ...?

BESCHREIBUNG

Der Treiber **tdbc::sqlite3** stellt eine Datenbankschnittstelle bereit, die Tcl DataBase Connectivity (TDBC) entspricht, und erlaubt einem Tcl-Skript, sich mit einer SQLite3-Datenbank zu verbinden. Er liegt außerdem als ausgearbeitetes Beispiel dafür vor, wie ein Datenbanktreiber in Tcl geschrieben wird, damit Autoren von Treibern einen Ausgangspunkt für die weitere Entwicklung haben.

Die Verbindung zu einer SQLite3-Datenbank entsteht durch den Aufruf von **tdbc::sqlite3::connection create**. Übergeben werden eine Zeichenkette, die als Verbindungshandle dienen soll, und danach der Dateiname der Datenbank. Die Nebenwirkung von **tdbc::sqlite3::connection create** ist, eine neue Datenbankverbindung zu erzeugen. Als Alternative darf **tdbc::sqlite::connection new** eine Datenbankverbindung mit automatisch vergebenem Namen erzeugen. Der Rückgabewert von **tdbc::sqlite::connection new** ist der Name, der für das Verbindungshandle gewählt wurde. **tdbc::connection(n)** beschreibt, wie die Verbindung zum Bearbeiten einer Datenbank verwendet wird.

KONFIGURATIONSOPTIONEN

Die Standard-Konfigurationsoptionen **-encoding**, **-isolation**, **-readonly** und **-timeout** werden sämtlich erkannt, sowohl bei **tdbc::sqlite3::connection create** als auch bei der Methode **configure** der entstehenden Verbindung.

Da die Kodierung einer SQLite3-Datenbank stets bekannt ist, akzeptiert die Option **-encoding** nur **utf-8** als Kodierung und gibt für eine Kodierung stets **utf-8** zurück. Die tatsächliche Kodierung lässt sich beim Anlegen einer neuen Datenbank mit einer SQLite3-Anweisung **PRAGMA** setzen.

Umgesetzt sind nur die Isolationsstufen **readuncommitted** und **serializable**. Andere Isolationsstufen werden auf **serializable** angehoben.

Der Schalter **-readonly** ist nicht umgesetzt. Ein boolescher Wert **false** wird stillschweigend akzeptiert, jeder andere Wert meldet einen Fehler.

Der Schalter **-keepcase** ist besonders und steuert, wie Tabellen- und Spaltennamen in den verschiedenen Methoden gemeldet werden, die Metadaten der Datenbank abrufen. Vorgegeben ist der Schalter mit null, das heißt, Tabellen- und Spaltennamen werden durchgängig in Kleinbuchstaben gemeldet. Andernfalls werden Tabellen- und Spaltennamen so gemeldet, wie sie in der Datenbank definiert sind.

FEHLER

Ist ein Spaltenname unter den Spalten einer Ergebnismenge nicht eindeutig, fehlen in den Ergebnissen der Rückgaben von **-as dicts** alle bis auf die am weitesten rechts stehende der doppelten Spalten. Diese Einschränkung lässt sich umgehen, indem **SELECT**-Anweisungen passende **AS**-Klauseln erhalten, sodass alle zurückgegebenen Spaltennamen eindeutig sind. Geplant ist, diesen Fehler durch eine C-Umsetzung des Treibers zu beheben; das wird auch die Leistung deutlich verbessern.

COPYRIGHT

Copyright (c) 2008 by Kevin B. Kenny.

tdom

dom

Stichwörter: dom, XML, document, node, parsing

NAME

dom - Aus XML einen DOM-Baum im Speicher anlegen

ÜBERSICHT

package require tdom

dom *method* ?*arg arg* ...?

BESCHREIBUNG

Dieser Befehl legt DOM-Bäume im Speicher an. Im üblichen Fall wird eine Zeichenkette mit einem XML-Dokument gelesen und in einen DOM-Baum umgewandelt. Andere Eingaben können HTML oder JSON sein. *method* nennt den Unterbefehl.

Die gültigen Methoden sind:

dom parse ?*options*? ?*data*? ?objvar?

Liest *data* und baut daraus im Speicher einen DOM-Baum. Dazu gibt es einen Tcl-Objektbefehl für dieses DOM-Dokument. Beispiel:

```
dom parse $xml doc
$doc documentElement root
```

Das liest die XML-Daten in der Variablen xml, legt den DOM-Baum im Speicher an, erzeugt einen Verweis auf das Dokumentobjekt, in Tcl sichtbar als Dokument-Objektbefehl, und weist diesen neuen Objektbefehl der Variablen doc zu. Verlässt die Variable doc den Geltungsbereich, werden der DOM-Baum und das zugehörige Tcl-Befehlsobjekt (Dokument und alle Knotenobjekte) von selbst freigegeben.

Statt des Arguments *objvar* kann der zurückgegebene Objektbefehl verwendet werden:

```
set document [dom parse $xml]
set root     [$document documentElement]
```

Das legt einen dauerhaften DOM-Baum an, der nicht von selbst freigegeben wird. Damit der DOM-Baum und die zugehörigen Tcl-Objektbefehle (Dokument, Knoten und Fragmentknoten) frei werden, muss der Dokument-Objektbefehl ausdrücklich gelöscht werden, mit:

```
$document delete
```

or

```
rename $document ""
```

Voreingestellt werden keine Textknoten eingefügt, die nur aus Leerraum bestehen. Dabei gilt das Attribut *xml:space*="*preserve*", und innerhalb eines solchen Teilbaums stellt *xml:space*="*default*" die Vorgabe wieder her.

Die gültigen Optionen sind:

-simple

Mit *-simple* wird ein einfacher, schneller Parser verwendet. Er entspricht der XML-Empfehlung

nicht vollständig. Einlesen und Aufbau des DOM sollten damit etwa doppelt so schnell sein. Die Kodierung der Daten wird im Parser nicht umgewandelt. Der einfache Parser beachtet keine Kodierungsangabe in der XML-Deklaration. Er überspringt die interne DTD-Teilmenge und ignoriert, was darin steht. Vorgabewerte von Attributen kommen deshalb nicht in den Baum, auch wenn die zugehörige Attributdeklaration in der internen Teilmenge steht. Interne oder externe Entitätsverweise werden nicht aufgelöst, außer den vordefinierten Entitäten und Zeichenverweisen.

-html

Mit *-html* wird ein schneller HTML-Parser verwendet, der auch fehlerhaftes HTML in einen DOM-Baum zu lesen versucht. Hat das übergebene HTML-Dokument kein einzelnes Wurzelelement, was bis HTML 4.01 zulässig war, und fehlt die Option -forest, wird ein html-Knoten als Dokumentelement eingefügt. Die Elemente der obersten Ebene der HTML-Eingabe werden seine Kinder.

-html5

Diese Option gibt es nur, wenn tDOM mit --enable-html5 gebaut wurde. Ob das eingebaut ist, sagt die Methode *featureinfo*. Mit *-html5* baut der HTML5-Parser der Bibliothek gumbo (https://github.com/google/gumbo-parser) den DOM-Baum. Soweit das reicht, kennt er XML-Namensräume. Alle HTML-Elemente liegen dann im Namensraum html5. Das ist oft nicht gewollt und in vielen Fällen nur Ballast. *-html5* lässt sich mit *-ignorexmlns* kombinieren. Dann liegen alle Knoten und Attribute des DOM-Baums in keinem XML-Namensraum. Alle Element- und Attributnamen im DOM-Baum werden klein geschrieben, auch fremde Elemente außerhalb der Namensräume xhtml, svg oder mathml. Der DOM-Baum kann Knoten enthalten, die der Parser ergänzt hat, weil der Zusammenhang sie verlangt, etwa <head> und <tbody>. Eingaben länger als 4 GByte unterstützt der zugrunde liegende gumbo-Parser nicht.

-json

Mit *-json* muss *data* eine gültige JSON-Zeichenkette nach RFC 7159 sein. Der Befehl gibt ein gewöhnliches DOM-Dokument zurück. Verschachtelung in den JSON-Daten wird zur Baumhierarchie. Ist ein JSON-Array-Wert selbst ein Objekt oder ein Array, werden Container-Elementknoten in den Baum eingefügt, in einem Vorgabebau arraycontainer beziehungsweise objectcontainer. Die JSON-Serialisierung dieses Dokuments, mit der domDoc-Methode *asJSON*, ist dieselbe JSON-Information wie *data*. JSON-Datentypen bleiben erhalten, Member-Namen von Objekten dürfen mehrfach vorkommen und behalten ihre Reihenfolge, und der ganze Wertebereich von JSON-Zeichenketten bleibt. Die JSON-Datentypen hängen als zusätzliche Eigenschaft an den Dokument- und Baumknoten. Diese Eigenschaft steht nicht in einer XML-Serialisierung des Dokuments. Sollen die JSON-Daten eines Dokuments gespeichert werden, wird die JSON-Serialisierung gespeichert und von dort wieder gelesen. Abgesehen von dieser JSON-Typangabe ist der zurückgegebene Dokumentbefehl oder das Handle ein gewöhnliches DOM-Dokument. Es lässt sich mit dem vollen Umfang der Dokument- und Knotenmethoden untersuchen und ändern. Elementnamen und Textknotenwerte im Baum können außerhalb dessen liegen, was die passenden XML-Produktionen erlauben.

-jsonroot <document element name>

Ist die Option angegeben, wird der angegebene Elementname das Dokumentelement des entstehenden Dokuments. Der gelesene Inhalt der JSON-Zeichenkette wird zu den Kindern dieses Dokumentelementknotens.

-jsonmaxnesting *integer*
Diese Option wirkt nur zusammen mit *-json*. Die vorliegende Umsetzung verwendet einen rekursiv absteigenden JSON-Parser. Damit der Stapelspeicher nicht überläuft, gilt JSON-Eingabe mit mehr als einer bestimmten Verschachtelungstiefe als ungültig. Die voreingestellte größte Tiefe ist 2000. Die Option -jsonmaxnesting stellt diesen Wert ein.

--
Die Option -- markiert das Ende der Optionen. Sie ist nicht zwingend nötig, auch beim Lesen von JSON, dessen gültige Daten mit "-" beginnen können. Beim Lesen von JSON wird das vorletzte oder letzte Argument als JSON-Daten gelesen, wenn es mit "-" beginnt und kein bekannter Optionsname ist.

-keepEmpties
Mit *-keepEmpties* gehören Textknoten, die nur aus Leerraum bestehen, zum entstehenden DOM-Baum. Voreingestellt, also ohne *-keepEmpties*, werden diese leeren Textknoten beim Lesen entfernt.

-keepCDATA
Mit *-keepCDATA* kommen CDATA-Abschnitte nicht als Textknoten in den Baum und werden nicht, wie ohne diese Option, nötigenfalls mit benachbarten Textknoten zu einem Textknoten zusammengezogen. Sie kommen als CDATA_SECTION_NODE in den Baum. Der entstehende Baum ist nicht für XPath-Auswahlen vorbereitet und nicht als Quelle oder Stylesheet einer XSLT-Transformation. Ohne *-keepEmpties* kommen nur CDATA-Abschnitte in den DOM-Baum, die nicht ausschließlich aus Leerraum bestehen.

-channel <*channel-ID*>
Mit *-channel <channel-ID>* wird die zu lesende Eingabe aus dem angegebenen Kanal gelesen. Die Kodierung des Kanals, gesetzt mit fconfigure -encoding, wird beachtet. Die aus dem Kanal gelesenen Daten werden vor dem Lesen gemäß dieser Kodierung nach UTF-8 gewandelt.

-baseurl <*baseURI*>
Mit *-baseurl <baseURI>* ist die baseURI die Basis-URI des Dokuments. Verweise auf externe Entitäten im Dokument werden relativ zu dieser Basis-URI aufgelöst. Diese Basis-URI wird auch im DOM-Baum gespeichert.

-feedbackAfter <*#bytes*>
Mit *-feedbackAfter <#bytes>* wird der Tcl-Befehl aus *-feedbackcmd* beim ersten Elementanfang im Dokument oder in einer externen Entität ausgewertet, nach dem Beginn des Dokuments oder der externen Entität oder nach dem letzten solchen Aufruf nach #bytes. Aus Gründen der Verträglichkeit wird, wenn -feedbackcmd fehlt, aber eine Tcl-Prozedur ::dom::domParseFeedback existiert, diese Prozedur als -feedbackcmd verwendet. Gibt es diese Prozedur nicht und wird -feedbackAfter verwendet, ist es ein Fehler, -feedbackcmd nicht ebenfalls zu verwenden. Löst das aufgerufene Skript einen Fehler aus, bricht das Lesen ab. Der Aufruf *dom parse* gibt den Fehler zurück, mit der Fehlermeldung des Skripts. Führt das aufgerufene Skript *return -code break* aus, bricht das Lesen ab, und der Aufruf *dom parse* gibt die leere Zeichenkette zurück.

-feedbackcmd <*script*>
Mit *-feedbackcmd <script>* wird das Skript *script* beim ersten Elementanfang im Dokument oder in einer externen Entität ausgewertet, nach dem Beginn des Dokuments oder der externen Entität oder nach dem letzten solchen Aufruf nach dem Wert #bytes aus der Option *-feedbackAfter*. Fehlt *-feedbackAfter*, hat diese Option keine Wirkung. Löst das aufgerufene

Skript einen Fehler aus, bricht das Lesen ab. Der Aufruf *dom parse* gibt den Fehler zurück, mit der Fehlermeldung des Skripts. Führt das aufgerufene Skript *return -code break* aus, bricht das Lesen ab, und der Aufruf *dom parse* gibt die leere Zeichenkette zurück.

-externalentitycommand <*script*>

Mit *-externalentitycommand <script>* wird das angegebene Tcl-Skript aufgerufen, um externe Entitäten des Dokuments aufzulösen. Der tatsächlich ausgewertete Befehl besteht aus dieser Option und drei Argumenten: der Basis-URI, dem Systembezeichner der Entität und dem öffentlichen Bezeichner der Entität. Die Basis-URI und der öffentliche Bezeichner können die leere Liste sein. Das Skript gibt eine Tcl-Liste aus drei Elementen zurück. Das erste Element sagt, wie die externe Entität an den Prozessor zurückgegeben wird. Derzeit sind die Typen "string" und "channel" erlaubt. Das zweite Element muss die absolute Basis-URI der zu lesenden externen Entität sein. Das dritte Element sind die Daten: bei Typ "string" die bereits gelesenen Daten der externen Entität als Zeichenkette, bei Typ "channel" der Name eines Tcl-Kanals. Gibt das Skript einen Tcl-Kanal zurück, schließt der Prozessor ihn nicht. Er muss getrennt geschlossen werden, wenn er nicht mehr gebraucht wird.

-useForeignDTD <*boolean*>

Ist <boolean> wahr und hat das Dokument keine externe Teilmenge, ruft der Parser das Skript -externalentitycommand mit leeren Werten für die Argumente systemId und publicID auf. Hat das Dokument auch keine interne Teilmenge, werden die Skripte -startdoctypedeclcommand und -enddoctypedeclcommand, sofern gesetzt, nicht aufgerufen.

-paramentityparsing <*always*|*never*|*notstandalone*>

Die Option *-paramentityparsing* steuert, ob der Parser beim Aufbau des DOM-Baums die externen Entitäten des Dokuments aufzulösen versucht, einschließlich der externen DTD-Teilmenge. *-paramentityparsing* braucht ein Argument, entweder "always", "never" oder "notstandalone". Der Wert "always" heißt: der Parser versucht, alle externen Entitäten der XML-Quelle aufzulösen, auch rekursiv. Das ist die Vorgabe, wenn *-paramentityparsing* fehlt. Der Wert "never" heißt: nur die angegebene XML-Quelle wird gelesen, keine externe Entität und nicht die externe Teilmenge wird aufgelöst und gelesen. Der Wert "notstandalone" heißt: alle externen Entitäten werden aufgelöst und gelesen, außer bei Dokumenten, die in der XML-Deklaration ausdrücklich standalone="yes" angeben.

-forest

Mit dieser Option ist keine einzelne Wurzel nötig. Jede Folge wohlgeformter, ausbalancierter Teilbäume wird in einen DOM-Baum gelesen. Das gilt für den expat-DOM-Aufbau, den einfachen XML-Parser mit *-simple* und den einfachen HTML-Parser mit *-html*. Zusammen mit *-json* oder *-html5* wird die Option ignoriert.

-ignorexmlns

Diese Option sollte nur zusammen mit *-html5* verwendet werden. Ist sie angegeben, wird kein Knoten des angelegten DOM-Baums intern als in einem XML-Namensraum stehend markiert, auch wenn für Elemente ohne Präfix ein Vorgabe-Namensraum gilt oder das Element einen Namensraum-Präfix hat. Eine Folge: XPath-Knotenausdrücke auf einem solchen DOM-Baum arbeiten nicht wie erwartet. Elementknoten mit Präfix lassen sich nicht naiv auswählen, und Elementknoten ohne Präfix sieht XPath so, als lägen sie in keinem Namensraum, auch wenn sie in einem Vorgabe-Namensraum liegen sollten. Ein Präfix in einem XPath-Ausdruck wird mit der '%'-Schreibweise gesetzt, beschrieben in der Dokumentation der der

domNode

Befehlsmethode **selectNodes**.

-billionLaughsAttackProtectionMaximumAmplification *<float>*
<URL: https://en.wikipedia.org/wiki/Billion_laughs_attack> "" Diese Option steuert zusammen mit *-billionLaughsAttackProtectionActivationThreshold* die Parsergrenzen gegen Billion-Laughs-Angriffe (). Erwartet wird eine Gleitkommazahl >= 1.0. Die Option sollte nie nötig sein, die Vorgabe 100.0 reicht für wirkliche Daten. Muss der Wert für eine Nutzlast erhöht werden, die kein Angriff ist, bitte melden.

-billionLaughsAttackProtectionActivationThreshold *<long>*
<URL: https://en.wikipedia.org/wiki/Billion_laughs_attack> "" Diese Option steuert zusammen mit *-billionLaughsAttackProtectionMaximumAmplification* die Parsergrenzen gegen Billion-Laughs-Angriffe (). Erwartet wird eine positive ganze Zahl. Die Option sollte nie nötig sein, die Vorgabe 8388608 reicht für wirkliche Daten. Muss der Wert für eine Nutzlast erhöht werden, die kein Angriff ist, bitte melden.

-setReparseDeferralEnabled *<boolean>*
Expat schützt sich gegen quadratische Lesezeit durch lange Token in böswillig gebauten Dokumenten. Als Nebenwirkung kann ein Leseereignis nicht direkt nach dem Ende des betreffenden Tokens ausgelöst werden, sondern erst, nachdem der Parser weitere Bytes der Eingabe verarbeitet hat. Diese Option schaltet dieses Verhalten ab.

-allocTrackerActivationThreshold *<float>*
Diese Option steuert zusammen mit *-allocTrackerMaximumAmplification* die Parsergrenzen gegen unverhältnismäßig viel dynamischen Speicher durch böswillig gebaute Dokumente. Erwartet wird eine positive ganze Zahl. Die Option sollte nie nötig sein, die Vorgabe 100.0 reicht für wirkliche Daten. Muss der Wert für eine Nutzlast erhöht werden, die kein Angriff ist, bitte melden.

-allocTrackerMaximumAmplification *<long>*
Diese Option steuert zusammen mit *-allocTrackerActivationThreshold* die Parsergrenzen gegen unverhältnismäßig viel dynamischen Speicher durch böswillig gebaute Dokumente. Erwartet wird eine positive ganze Zahl. Die Option sollte nie nötig sein, die Vorgabe 67108864, das sind 64 MiB, reicht für wirkliche Daten. Muss der Wert für eine Nutzlast erhöht werden, die kein Angriff ist, bitte melden.

dom createDocument *docElemName*?*objVar*?
Legt ein neues DOM-Dokumentobjekt mit einem Elementknoten des Knotennamens *docElemName* an. *objVar* steuert die Speicherbehandlung wie oben beschrieben.

dom createDocumentNS *uri docElemName*?*objVar*?
Legt ein neues DOM-Dokumentobjekt mit einem Elementknoten des Knotennamens *docElemName* an. *Uri* gibt den Namensraum des anzulegenden Dokumentelements an. *objVar* steuert die Speicherbehandlung wie oben beschrieben.

dom createDocumentNode ?*-jsonType jsonType*? ?*objVar*?
Legt ein neues leeres DOM-Dokumentobjekt ohne Elementknoten an. *objVar* steuert die Speicherbehandlung wie oben beschrieben. Ist die Option -jsonType angegeben, hat der angelegte Dokumentknoten den angegebenen JSON-Typ.

dom createFromTypedList *typedList*?*objVar*?
Legt aus dem Argument *typedList* ein neues DOM-Dokument an. Das Argument *objVar* steuert die Speicherbehandlung wie oben beschrieben.

Das Argument *typedList* muss eine Tcl-Liste sein und dem Format der Ausgabe der Dokumentbefehlsmethode *asTypedList* folgen, siehe dort.

dom createNodeCmd*?-returnNodeCmd??-tagName name??-jsonType jsonType??-namespace URI??-noNamespacedAttributes??-notempty?(element|comment|text|cdata|pi)Node commandName*

Diese Methode legt Tcl-Befehle an, die wiederum tDOM-Knoten anlegen. Die so angelegten Tcl-Befehle gibt es nur in einem Skript, das den domNode-Methoden *appendFromScript* oder *insertBeforeFromScript* übergeben wird. Wird ein mit *createNodeCmd* angelegter Befehl in einem anderen Zusammenhang aufgerufen, kommt ein Fehler zurück. Der angelegte Befehl *commandName* ersetzt einen vorhandenen Befehl oder eine vorhandene Prozedur dieses Namens. Enthält *commandName* Tcl-Namensraum-Qualifizierer, wird er in diesem Namensraum angelegt. Die Option *-tagName* ist nur für den Typ elementNode erlaubt. Die Option *-jsonType* ist nur für die Typen elementNode und textNode erlaubt.

Wird ein solcher Befehl in einem Skript aufgerufen, das der domNode-Methode *appendFromScript* oder *insertBeforeFromScript* als Argument übergeben wurde, legt er einen neuen Knoten an und hängt ihn ans Ende der Kindliste des aufrufenden Elementknotens. War die Option *-returnNodeCmd* angegeben, gibt der Befehl den angelegten Knoten als Tcl-Befehl zurück. Fehlt die Option, gibt der Befehl nichts zurück. Jeder Befehl legt immer denselben Knotentyp an. Welcher Typ entsteht, bestimmt das erste Argument von *createNodeCmd*. Die Syntax des angelegten Befehls hängt vom Typ des Knotens ab, den er anlegt.

Ist der anzulegende Befehlstyp *elementNode*, legt der Befehl beim Aufruf einen Elementknoten an. Wurde beim Anlegen des Knotenbefehls die Marke *-notempty* verwendet, entsteht der Elementknoten nur, wenn er nicht leer ist. Ohne die Option *-tagName* ist der Elementname des angelegten Knotens *commandName* ohne Tcl-Namensraum-Qualifizierer. War *-tagName* angegeben, haben die angelegten Elemente den Wert dieser Option als Elementnamen. War *-jsonType* angegeben, haben die angelegten Elementknoten den angegebenen JSON-Typ. Ist *-namespace* angegeben, liegt der angelegte Elementknoten im XML-Namensraum dieser Option. Der Elementname bleibt wörtlich so, wie ihn der Befehlsname oder, falls angegeben, die Option *-tagname* vorgibt. Eine passende XML-Namensraumdeklaration wird von selbst ergänzt. Sie bindet den Präfix, wenn der Elementname einen hat, oder den Vorgabe-Namensraum, wenn der Elementname keinen Präfix hat, an den Namensraum, sofern eine solche Bindung nicht gilt.

Die Syntax des angelegten Befehls ist:

```
elementNodeCmd ?attributeName attributeValue ...? ?script?
elementNodeCmd ?-attributeName attributeValue ...? ?script?
elementNodeCmd ?name_value_list? ?script?
```

Die Befehlssyntax erlaubt drei Arten, die Attribute des entstehenden Elements anzugeben. Als Argumentpaare *attributeName attributeValue*, im Optionsstil als Argumentpaare *-attriubteName attributeValue*, wobei das Zeichen '-' nur syntaktischer Zucker ist und abgeschnitten wird, oder als Tcl-Liste, deren Elemente als Attributname und zugehöriger Attributwert gelesen werden. Die Attributnamen in der Liste dürfen mit '-' beginnen, das dann abgeschnitten wird. Wird *elementNodeCmd* mit nur einem Argument aufgerufen, gilt das als Inhaltsskript.

Hat *attributeName* einen Präfix, wird der Präfix voreingestellt zuerst in der Präfix-URI-Liste *selectNodesNamespaces* gesucht und, falls nicht gefunden, im XML-Namensraum des Knotens. Löst sich der Präfix nicht auf, wird ein Fehler ausgelöst. Löst er sich auf, entsteht das Attribut in

diesem Namensraum, nötigenfalls mit XML-Namensraumdeklarationen. Wurde *elementNodeCmd* mit der Marke *-noNamespacedAttributes* angelegt, gibt es keine Präfixsuche. Das Attribut entsteht mit dem angegebenen Namen und in keinem XML-Namensraum. XML-Namensraumdeklarationen sollten damit nicht angelegt werden.

Jeder *elementNodeCmd* nimmt als letztes Argument ein optionales Tcl-Skript an. Dieses Skript wird als rekursives *appendFromScript*-Skript ausgewertet. Der von *elementNodeCmd* angelegte Knoten ist der Elter aller Knoten, die das Skript anlegt.

Ist das erste Argument der Methode *textNode*, legt der Befehl einen Textknoten an. War die Option *-jsonType* angegeben, hat der angelegte Textknoten diesen JSON-Typ. Die Syntax des angelegten Befehls ist:

```
textNodeCmd ?-disableOutputEscaping? ?data?
```

Ist der JSON-Typ des angelegten Textknotens NULL, TRUE oder FALSE, ist das Argument *data* optional. Sonst muss dieses Argument angegeben werden.

Ist der JSON-Typ des angelegten Textknotens der virtuelle Typ BOOLEAN und erkennt Tcl_GetBooleanFromObj() den Textwert als boolesch, wird der JSON-Wert der entsprechende JSON-Boolesche. Versteht Tcl den Textknotenwert nicht als boolesch, wird der Wert als JSON-Zeichenkette geschrieben.

Mit der optionalen Marke *-disableOutputEscaping* entfällt die Maskierung des Und-Zeichens (&) und der linken spitzen Klammer (<) in den Daten. Diese Marke ist mit Vorsicht zu verwenden.

Ist das erste Argument der Methode *commentNode* oder *cdataNode*, legt der Befehl einen Kommentarknoten oder einen CDATA-Abschnittknoten an. Die Syntax des angelegten Befehls ist:

```
nodeCmd data
```

Ist das erste Argument der Methode *piNode*, legt der Befehl einen Verarbeitungsanweisungsknoten an. Die Syntax des angelegten Befehls ist:

```
piNodeCmd target data
```

Neben den mit **dom createNodeCmd** angelegten Knotenbefehlen gibt es zwei weitere Befehle, die in einem **appendFromScript**-Skript Knoten von selbst in den Baum einfügen.

```
tdom::fsnewNode ?-jsonType <jsonType>? ?-namespace <namespace>?
    ?-noNamespacedAttributes? -notempty tagName ?attributes? ?script?
```

Im Zusammenhang fromScript legt dieser Befehl einen neuen Knoten *tagName* an, im XML-Namensraum *namespace*, wenn *-namespace* angegeben war, und mit dem JSON-Typ *jsonType*, wenn *-jsonType* angegeben war, und hängt ihn ans Ende der Kindliste des aufrufenden Elementknotens. Die übrigen Marken wirken wie beim Anlegen eines Befehls, der Elemente anlegt, siehe dort. Die Argumente *attributes* und *script* werden ebenso verarbeitet wie bei einem Knotenbefehl, der Elemente anlegt. Außerhalb eines fromScript-Zusammenhangs löst dieser Befehl einen Fehler aus.

```
tdom::fsinsertNode node
```

Im Zusammenhang fromScript hängt dieser Befehl den als Argument angegebenen Knoten ans Ende der Kindliste des aufrufenden Elementknotens, statt einen neuen Knoten anzulegen. Der Knoten wird von seinem bisherigen Ort gelöst. Außerhalb eines fromScript-Zusammenhangs löst

dieser Befehl einen Fehler aus.

dom fromScriptContext ?*objVar*?
Im Zusammenhang fromScript gibt diese Methode den Knoten zurück, an dessen Kindliste der Skriptzusammenhang Knoten anhängt. Sonst kommt ein Fehler zurück.

Werden Knotenbefehle mit -notempty angelegt, kann der zurückgegebene Knoten außerhalb des Auswertungszusammenhangs verschwunden sein, wenn er am Ende der Auswertung noch leer ist.

dom setStoreLineColumn*?boolean*?
Ist das eingeschaltet, enthalten die DOM-Knoten nach dem Lesen Zeile und Spalte im ursprünglichen XML-Dokument. Voreingestellt werden Zeile und Spalte nicht gespeichert.

dom setNameCheck*?boolean*?
Ist NameCheck wahr, prüft jede Methode, die einen XML-Namen, einen voll qualifizierten Namen oder ein Ziel einer Verarbeitungsanweisung erwartet, ob die angegebene Zeichenkette nach ihrer Produktionsregel gültig ist. Bei Befehlen, die die Methode *createNodeCmd* für den Zusammenhang *appendFromScript* anlegt, entscheidet der Stand der Marke zur Anlegezeit. Ist NameCheck zur Anlegezeit wahr, prüft der Befehl seine Argumente, sonst nicht. *setNameCheck* setzt diese Marke. Zurück kommt der aktuelle Stand der Marke NameCheck. Die Vorgabe für NameCheck ist wahr.

dom setTextCheck*?boolean*?
Ist TextCheck wahr, prüft jeder Befehl, der XML-Zeichen, einen Kommentar, einen CDATA-Abschnittwert oder einen Wert einer Verarbeitungsanweisung erwartet, ob die angegebene Zeichenkette nach ihrer Produktionsregel gültig ist. Bei Befehlen, die die Methode *createNodeCmd* für den Zusammenhang *appendFromScript* anlegt, entscheidet der Stand der Marke zur Anlegezeit. Ist TextCheck zur Anlegezeit wahr, prüft der Befehl seine Argumente, sonst nicht. Die Methode *setTextCheck* setzt diese Marke. Zurück kommt der aktuelle Stand der Marke TextCheck. Die Vorgabe für TextCheck ist wahr.

dom setObjectCommands ?*(automatic|token|command)*?
Steuert, ob Dokumente und Knoten als Tcl-Befehle entstehen oder als Token für die Befehle domNode und domDoc. Im Modus 'automatic' legen Methoden an Tcl-Befehlen Tcl-Befehle an, und Methoden an Dokument- oder Knoten-Token legen Token an. Im Modus 'command' entstehen immer Tcl-Befehle. Im Modus 'token' entstehen immer Token. Die Methode gibt den aktuellen Modus zurück. Die Methode ist eine experimentelle Schnittstelle.

dom isName *name*
Gibt 1 zurück, wenn *name* nach Produktion 5 der XML-1.0-Empfehlung ein gültiger XML-Name ist. Dann ist *name* ein gültiger XML-Element- oder Attributname. Sonst kommt 0 zurück.

dom isPIName *name*
Gibt 1 zurück, wenn *name* nach Produktion 17 der XML-1.0-Empfehlung ein gültiges Ziel einer XML-Verarbeitungsanweisung ist. Sonst kommt 0 zurück.

dom isNCName *name*
Gibt 1 zurück, wenn *name* nach Produktion 4 der der Empfehlung Namespaces in XML ein gültiger NCName ist. Sonst kommt 0 zurück.

dom isQName *name*
Gibt 1 zurück, wenn *name* nach Produktion 6 der der Empfehlung Namespaces in XML ein gültiger QName ist. Sonst kommt 0 zurück.

dom isCharData *string*
Gibt 1 zurück, wenn jedes Zeichen in *string* nach Produktion 2 der XML-1.0-Empfehlung ein gültiges XML-Char ist. Sonst kommt 0 zurück.

dom isHTML5CustomName *string*
Gibt 1 zurück, wenn *string* ein nach HTML5 gültiger Name eines Custom Elements ist, nach https://html.spec.whatwg.org/#valid-custom-element-name vom 26. Juni 2024. Das ist ein lebender Standard. Sonst kommt 0 zurück.

dom clearString ?*-replace*? *string*
Gibt die als Argument angegebene Zeichenkette zurück, bereinigt um alle Zeichen, die als geparste XML-Zeichendaten nicht erlaubt sind. Mit der Option *-replace* werden solche Zeichen nicht entfernt, sondern durch \uFFFD ersetzt.

dom isBMPCharData *string*
Gibt 1 zurück, wenn jedes Zeichen in *string* ein gültiges XML-Char mit einem Unicode-Codepunkt in der Basic Multilingual Plane ist. Das heißt, jedes Zeichen der Zeichenkette ist höchstens 3 Bytes lang. Sonst kommt 0 zurück.

dom isComment *string*
Gibt 1 zurück, wenn *string* nach Produktion 15 der XML-1.0-Empfehlung ein gültiger Kommentar ist. Sonst kommt 0 zurück.

dom isCDATA *string*
Gibt 1 zurück, wenn *string* nach Produktion 20 der XML-1.0-Empfehlung gültig ist. Sonst kommt 0 zurück.

dom isPIValue *string*
Gibt 1 zurück, wenn *string* nach Produktion 16 der XML-1.0-Empfehlung gültig ist. Sonst kommt 0 zurück.

dom jsonEscape *string*
Gibt das Argument *string* so maskiert zurück, dass ein konformer JSON-Parser die zurückgegebene Zeichenkette, wörtlich in ein JSON-Dokument gesetzt, als die ursprüngliche Zeichenkette liest.

dom featureinfo *feature*
Diese Methode nennt die verwendeten Bauoptionen und die expat-Version. Gültige Werte für das Argument *feature* sind:

expatversion
Gibt die Version des zugrunde liegenden expat als Zeichenkette zurück, etwa "exapt_2.1.0". Das ist die Rückgabe der expat-API-Funktion XML_ExpatVersion().

expatmajorversion
Gibt die Hauptversion des zur Bauzeit verwendeten expat als ganze Zahl zurück.

expatminorversion
Gibt die Nebenversion des zur Bauzeit verwendeten expat als ganze Zahl zurück.

expatmicroversion
Gibt die Mikroversion des zur Bauzeit verwendeten expat als ganze Zahl zurück.

dtd
Gibt als booleschen Wert zurück, ob mit *--enable-dtd* gebaut wurde.

ns
Gibt als booleschen Wert zurück, ob mit *--enable-ns* gebaut wurde.

unknown
Gibt als booleschen Wert zurück, ob mit *--enable-unknown* gebaut wurde.

tdomalloc
Gibt als booleschen Wert zurück, ob mit *--enable-tdomalloc* gebaut wurde.

lessns
Gibt als booleschen Wert zurück, ob mit *--enable-lessns* gebaut wurde.

TCL_UTF_MAX
Gibt den Wert TCL_UTF_MAX des Tcl-Kerns als ganze Zahl zurück, mit dem tDOM gebaut wurde.

html5
Gibt als booleschen Wert zurück, ob mit *--enable-html5* gebaut wurde.

versionhash
Gibt den Versions-Hash des Fossil-Repositoriums zurück.

pullparser
Gibt als booleschen Wert zurück, ob der Befehl pullparser eingebaut ist.

schema
Gibt als booleschen Wert zurück, ob die Schema-Funktionen von tDOM eingebaut sind.

largedata
Gibt als booleschen Wert zurück, ob die Konfiguration der Bibliothek expat PCDATA-Inhalt eines XML-Elements größer als 2 GByte erlaubt. Wahr kann das nur sein, wenn mit Tcl 9 für eine 64-Bit-Architektur gebaut wurde. Dann hängt es davon ab, ob das eingebaute expat mit Vorgabekonfiguration verwendet wird oder ob die verwendete Systembibliothek expat das kann. In der Regel nicht.

domDoc

Stichwörter: domDoc, DOM node creation, document element

NAME

domDoc - Eine Instanz eines DOM-Dokumentobjekts bearbeiten

ÜBERSICHT

```
domDocObjCmd method ?arg arg ...?
```

BESCHREIBUNG

Dieser Befehl bearbeitet eine bestimmte Instanz eines Dokumentobjekts. *method* bezeichnet eine bestimmte Methode der Dokumentklasse. Diese Methoden sollen sich eng an die W3C-Empfehlung "Document Object Model (Core) Level 1" (http://www.w3.org/TR/REC-DOM-Level-1/level-one-core.html) halten. Diese Dokumente beschreiben die Funktion genauer.

Die gültigen Methoden sind:

documentElement ?*objVar*?
Gibt das oberste Element im Dokument zurück, das Wurzelelement.

getElementsByTagName *name*
Gibt eine Liste aller Elemente im Dokument zurück, die im Glob-Stil auf *name* passen.

getElementsByTagNameNS *uri localname*
Gibt eine Liste aller Elemente im Teilbaum zurück, die im Glob-Stil auf *localname* passen und den angegebenen Namensraum *uri* haben.

createElement *tagName*?*objVar*?
Legt einen neuen Elementknoten mit dem Knotennamen *tagName* an, hängt ihn an die versteckte Fragmentliste im Dokumentobjekt an und gibt das Knotenobjekt zurück. Ist *objVar* angegeben, wird das neue Knotenobjekt in dieser Variable gespeichert.

createElementNS *url tagName*?*objVar*?
Legt einen neuen Elementknoten in einem Namensraum an, mit *uri* als URI und dem Knotennamen *tagName*, der den Namensraumpräfix enthalten kann, hängt ihn an die versteckte Fragmentliste im Dokumentobjekt an und gibt das Knotenobjekt zurück. Ist *objVar* angegeben, wird das neue Knotenobjekt in dieser Variable gespeichert.

createTextNode *text*?*objVar*?
Legt einen neuen Textknoten mit dem Knotenwert *text* an, hängt ihn an die versteckte Fragmentliste im Dokumentobjekt an und gibt das Knotenobjekt zurück. Ist *objVar* angegeben, wird das neue Knotenobjekt in dieser Variable gespeichert.

createComment *text*?*objVar*?
Legt einen neuen Kommentarknoten mit dem Wert *text* an, hängt ihn an die versteckte Fragmentliste im Dokumentobjekt an und gibt das Knotenobjekt zurück. Ist *objVar* angegeben, wird das neue Kommentarknotenobjekt in dieser Variable gespeichert.

createCDATASection *data*?*objVar*?
Legt einen neuen CDATA-Knoten mit dem Knotenwert *data* an, hängt ihn an die versteckte Fragmentliste im Dokumentobjekt an und gibt das Knotenobjekt zurück. Ist *objVar* angegeben, wird das neue Knotenobjekt in dieser Variable gespeichert.

createProcessingInstruction *target data*?*objVar*?
Legt eine Verarbeitungsanweisung an, hängt sie an die versteckte Fragmentliste im Dokumentobjekt an und gibt das Knotenobjekt zurück. Ist *objVar* angegeben, wird das neue Knotenobjekt in dieser Variable gespeichert.

delete
Löscht das Dokument ausdrücklich, einschließlich der zugehörigen Tcl-Objektbefehle für Knoten, Fragment- und neue Knoten und das Dokumentobjekt selbst, und den zugrunde liegenden DOM-Baum.

getDefaultOutputMethod
Gibt die Vorgabe-Ausgabemethode des Dokuments zurück. Das ist gewöhnlich das Ergebnis einer XSLT-Transformation.

asXML?-indent none/tabs/1..8??-channel channelId??-escapeNonASCII?
?-doctypeDeclaration <boolean>? -xmlDeclaration <boolean>? -encString <string>
?-escapeAllQuot??-indentAttrs??-nogtescape??-noEmptyElementTag??-escapeCR?
?-escapeTab?
Gibt den DOM-Baum als optional eingerückte XML-Zeichenkette zurück oder sendet die Ausgabe direkt an die angegebene channelId.

Die Option -indent verlangt als Wert "no", "none", "tabs" oder eine natürliche Zahl zwischen 0 und 8, beide eingeschlossen. Mit den Werten "no" oder "none" wird außerhalb des Markups kein zusätzlicher Leerraum zur Serialisierung hinzugefügt. I. Sonst ist es eine "pretty-print"-Serialisierung, weil zwischen dem Ende-Tag und dem nächsten Start-Tag

entsprechend der Schachtelungstiefe Leerraum eingefügt wird. Die Breite der Einrückung je Ebene gibt die Zahl an. Ist der Wert "tabs", erfolgt die Einrückung mit Tabulatoren, ein Tabulator je Ebene.

Ist die Option *-escapeNonASCII* angegeben, wird jedes Zeichen außerhalb von 7-Bit-ASCII in Attributwerten oder im PCDATA-Inhalt von Elementen als Zeichenreferenz in dezimaler Darstellung geschützt.

Die Markierung *-doctypeDeclaration* bestimmt, ob vor dem ersten Knoten des Dokuments eine DOCTYPE-Deklaration ausgegeben wird. In der Vorgabe wird sie nicht ausgegeben. Der DOCTYPE-Name ist immer der Elementname des Dokumentelements. Eine Deklaration der externen Entität der externen Teilmenge wird nur ausgegeben, wenn das Dokument einen Systembezeichner hat.

Die Markierung *-xmlDeclaration* bestimmt, ob vor allem anderen eine XML-Deklaration und ein Zeilenumbruch ausgegeben werden. In der Vorgabe wird keine ausgegeben. Ist diese Markierung mit einem wahren Argument angegeben, dann

-encString setzt den Kodierungswert in der XML-Deklaration. Sonst wird diese Option übergangen. Diese Option ergänzt nur die Zeichenketten-Darstellung der erzeugten XML-Deklaration um eine Kodierungsangabe, nicht mehr. Die Kodierung beim Schreiben in einen Kanal oder beim erneuten Lesen bleibt Sache des Aufrufers.

Ist die Option *-escapeAllQuot* angegeben, werden Anführungszeichen auch im Textinhalt von Elementen mit " geschützt.

Ist die Option *-indentAttrs* angegeben, wird jedes Attribut mit Zeilenumbrüchen getrennt und auf dieselbe Ebene wie der Elternknoten eingerückt, zuzüglich des als Argument von *-indentAttrs* angegebenen Werts (0..8).

Ist die Option *-nogtescape* angegeben, wird das Zeichen '>' in Attributwerten und im Textinhalt von Elementen nicht geschützt. In der Vorgabe wird dieses Zeichen geschützt.

Ist die Option *-noEmptyElementTag* angegeben, wird keine Syntax für leere Tags verwendet. Hat ein Element leeren Inhalt, wird es mit einem Element-Start-Tag und einem unmittelbar folgenden Element-Ende-Tag serialisiert.

Ist die Option *-escapeCR* angegeben, wird das Zeichen '\r' in Attributwerten und im Textinhalt von Elementen als Zeichenreferenz geschützt. In der Vorgabe geschieht das nicht.

Ist die Option *-escapeTab* angegeben, wird das Zeichen '\t' in Attributwerten und im Textinhalt von Elementen als Zeichenreferenz geschützt. In der Vorgabe geschieht das nicht.

asCanonicalXML?-channel channelId??-comments <boolean>?

Gibt den DOM-Baum als kanonische XML-Zeichenkette nach der "Canonical XML Version 1.0 W3C Recommendation 15 March 2001" zurück oder sendet die Ausgabe direkt an die angegebene channelId.

Soll die kanonische XML-Serialisierung der XML-Datei entstehen, aus der der DOM-Baum gelesen wurde, gelten ein paar Voraussetzungen. Die XML-Daten müssen mit der Option *-keepEmpties* gelesen werden. Enthalten die XML-Daten eine DTD, die Attributvorgaben oder Verweise auf externe gelesene Entitäten definiert, ist der expat-Parser nötig, nicht der mit *-simple*. Enthalten die XML-Daten bei einer unterstützten Tcl-Version kleiner als 9.0 Zeichen außerhalb der BMP, ist ein Tcl-Bau mit TCL_UTF_MAX gleich 6 und ein tDOM-Bau mit diesem Tcl nötig.

Ist die Option -channel angegeben, geht die Ausgabe direkt an den als Argument angegebenen

Tcl-Kanal. Der Aufrufer muss sicherstellen, dass der Kanal richtig konfiguriert ist. Fehlt diese Option, gibt der Befehl die Serialisierung als Zeichenkette zurück.

Ist die Option *-comments* mit einem wahren Wert angegeben, enthält die Serialisierung Kommentare nach den Regeln der Empfehlung. Ist der Wert falsch oder fehlt die Option, werden Kommentare aus der Serialisierung entfernt.

asHTML?-channel channelId??-escapeNonASCII??-htmlEntities??-doctypeDeclaration <boolean>??-breakLines??-onlyContents?

Gibt den nach HTML-Regeln serialisierten DOM-Baum als Zeichenkette zurück oder sendet die Ausgabe direkt an die angegebene channelId. HTML-Elemente werden unabhängig von der Großschreibung erkannt, leere HTML-Elemente ohne Ende-Tags und so weiter.

Ist die Option *-escapeNonASCII* angegeben, wird jedes Zeichen außerhalb von 7-Bit-ASCII in Attributwerten oder im PCDATA-Inhalt von Elementen als Zeichenreferenz in dezimaler Darstellung geschützt.

Ist die Option *-htmlEntities* angegeben, wird ein Zeichen oder ein Zeichenpaar mit seiner HTML-5-Zeichenentität geschrieben, wenn es eine hat. Manche HTML-5-Zeichenentitäten kodieren dasselbe Zeichen oder dieselben Codepunkte. Von den möglichen Entitätsnamen wird der kürzeste gewählt. Gibt es mehr als einen kürzesten Namen und unterscheiden sich diese Namen nur in der Großschreibung, wird die kleingeschriebene Variante gewählt. Sonst wird der erste Name in lexikalischer ASCII-Reihenfolge gewählt. Es gibt eine HTML5-Entität (ThickSpace), die zwei Zeichen schützt, für die das erste und das zweite Zeichen selbst einen Entitätsnamen haben. Sollen die zwei Zeichen serialisiert werden, wird die Entität ThickSpace für zwei Zeichen gewählt.

Ist die Option *-breakLines* angegeben, gibt die Serialisierung für die öffnenden Tags von Elementen "\n>" statt ">" aus.

Ist die Option *-onlyContents* angegeben, werden nur alle Kindknoten serialisiert. Dokumentknoten übergehen diese Option.

Ist die Markierung *-doctypeDeclaration* angegeben, wird vor dem ersten Knoten des Dokuments eine DOCTYPE-Deklaration ausgegeben. In der Vorgabe geschieht das nicht. Der DOCTYPE-Name ist immer der Elementname des Dokumentelements ohne Normalisierung der Großschreibung. Eine Deklaration der externen Entität der externen Teilmenge wird nur ausgegeben, wenn das Dokument einen Systembezeichner hat. Die Doctype-Deklaration wird aus den vorhandenen Angaben geschrieben, ohne Prüfung, ob das eine bekannte HTML-Versionsangabe des W3C ist oder ob das Dokument der angegebenen HTML-Version entspricht. Alle Knotentypen außer Dokumentknoten übergehen diese Option.

asText

Die Methode asText gibt den Baum zurück, indem sie den Zeichenkettenwert jedes Textknotens in Dokumentreihenfolge ohne jeden Schutz serialisiert. Das ist genau das, was die XSLT-Ausgabemethode "text" tut, Empfehlung XSLT 1.0, Abschnitt 16.3.

asJSON?-indent none/0..8??-channel channelId?

Die Methode asJSON serialisiert den Baum in eine gültige JSON-Datenzeichenkette. Im Allgemeinen kann diese Serialisierung verlustbehaftet sein. Für diese Serialisierung werden alle Kommentare, Zeichendatenabschnitte und Knoten von Verarbeitungsanweisungen, alle Attribute und alle XML-Namensräume übergangen. In der erzeugten JSON-Serialisierung können sich nur Element- und Textknoten niederschlagen. Passende JSON-Datentypangaben

eines Knotens werden beachtet.

Hat ein Elementknoten den JSON-Typ OBJECT, wird jedes Elementknoten-Kind dieses Elements als Member dieses Objekts serialisiert, mit dem Knotennamen des Kindes als Member-Namen und den maßgeblichen Kindern dieses Kindes als Wert. Alle anderen Kindknoten werden übergangen.

Hat ein Elementknoten den JSON-Typ ARRAY, werden die Text- und Elementknoten-Kinder dieses Elementknotens als die aufeinanderfolgenden Werte des Arrays serialisiert. Elementknoten-Kinder eines ARRAY-Elements sind Container-Knoten für verschachtelte Werte ARRAY oder OBJECT.

Textknoten mit den JSON-Typen TRUE, FALSE oder NULL werden zum entsprechenden JSON-Token serialisiert, ohne den Wert des Textknotens anzusehen. Ein Textknoten ohne JSON-Typ wird immer als JSON-Zeichenketten-Token serialisiert. Ein Textknoten mit dem JSON-Typ NUMBER wird als JSON-Zahl-Token serialisiert, wenn der Wert des Textknotens tatsächlich eine gültige JSON-Zahl ist, und sonst als JSON-Zeichenkette.

Hat ein Elementknoten keinen JSON-Typ, bestimmen die folgenden Regeln die Serialisierung seiner Kinder:

Maßgeblich sind nur Text- und Elementknoten-Kinder. Ist der zu serialisierende Elementknoten der Member eines JSON-Objekts und gibt es keinen maßgeblichen Kindknoten, ist der Wert dieses Members eine leere JSON-Zeichenkette. Ist der einzige maßgebliche Kindknoten dieses Elementknotens ein Textknoten, ist der JSON-Wert dieses Textknotens der Wert des Objekt-Members. Hat das Element mehr als einen maßgeblichen Kindknoten und ist der erste ein Textknoten, werden die maßgeblichen Kinder als JSON-Array serialisiert. Ist der einzige maßgebliche Kindknoten ein Elementknoten oder ist das erste maßgebliche Kind ein Elementknoten und gleicht der Knotenname dieses einzigen oder ersten maßgeblichen Kindes nicht dem Namen des Array-Container-Knotens, werden alle Elementknoten-Kinder als die Member eines JSON-Objekts serialisiert, wobei eingestreute Textknoten übergangen werden. Ist das einzige oder erste maßgebliche Kind ein Elementknoten und gleicht der Knotenname dieses Kindes dem Namen des Array-Container-Elements, werden alle maßgeblichen Kinder als die Werte eines JSON-Arrays serialisiert.

Ist das zu serialisierende Element ein Wert eines JSON-Arrays und gleicht der Knotenname dieses Elements nicht dem Namen des Array-Container-Knotens, gilt dieses Element als Container-Knoten für ein JSON-Objekt, und alle Elementknoten-Kinder werden als die Member dieses Arrays serialisiert, wobei Textknoten-Kinder übergangen werden. Ist das zu serialisierende Element ein Wert eines JSON-Arrays und gleicht der Knotenname dieses Elements dem Namen des Array-Container-Knotens, werden alle maßgeblichen Kinder als JSON-Array serialisiert.

Ist die Option *-channel* angegeben, kommt die Serialisierung nicht als Zeichenkette zurück, sondern geht direkt an den Kanal, der der Option als Argument angegeben ist.

Ist die Option *-indent* angegeben und ist das Argument dieser Option nicht "none", wird die zurückgegebene JSON-Zeichenkette "pretty-printed". Das numerische Argument dieser Option bestimmt die Anzahl der Leerzeichen je Einrückungsebene. In der Vorgabe wird kein zusätzlicher Leerraum ausgegeben.

asTclValue?typevariable?
: Enthält der DOM-Baum JSON-Typangaben, gibt diese Methode die JSON-Daten als verschachtelte Tcl-Datenstruktur zurück.

Der zurückgegebene Wert kann ein Tcl-Dict, eine Tcl-Liste oder eine Zeichenkette sein. Ist das optionale Argument *typevariable* angegeben, wird die Variable dieses Namens auf den Wert *dict*, *list* oder *string* gesetzt und zeigt so den Typ des Ergebnisses an.

Ein JSON-Objekt kommt als Tcl-Dict zurück, ein JSON-Array als Liste, und JSON-Zeichenketten und Zahlen sowie die symbolischen JSON-Werte null, true und false kommen als Zeichenkette zurück, mit den Zeichenketten null, true und false für das jeweilige JSON-Symbol. Der Wert eines Members eines JSON-Objekts kann ebenfalls ein Tcl-Dict, eine Tcl-Liste oder eine Zeichenkette sein, und die Elemente einer JSON-Array-Liste können ein Tcl-Dict, eine Tcl-Liste oder eine Zeichenkette sein.

asTypedList
: Enthält der DOM-Baum JSON-Typangaben, gibt diese Methode die JSON-Daten als verschachtelte Tcl-Liste zurück.

Das erste Element jeder dieser Listen beschreibt den Typ des Werts. Die Typen sind OBJECT, ARRAY, STRING, NUMBER, TRUE, FALSE oder NULL.

Ist der Typ NUMBER oder STRING, ist das zweite und letzte Element der Wert. Ist der Typ NULL, TRUE oder FALSE, hat die Liste keine weiteren Elemente.

Ist der Typ OBJECT, ist der zweite Wert eine Tcl-Liste aus Paaren von Eigenschaftsname und Wert, das zweite Element lässt sich also als Dict verwenden. Der Wert ist eine Tcl-Liste, aufgebaut nach den Regeln der Methode *asTypedList*.

Ist der Typ ARRAY, ist der zweite Wert eine Tcl-Liste der JSON-Array-Werte, jeder einzelne aufgebaut nach den Regeln der Methode *asTypedList*.

publicId*?publicId?*
: Gibt den öffentlichen Bezeichner der Doctype-Deklaration des Dokuments zurück, wenn es einen gibt, sonst die leere Zeichenkette. Ist der Methode ein Wert angegeben, wird der öffentliche Bezeichner des Dokuments auf diesen Wert gesetzt.

systemId*?systemId?*
: Gibt den Systembezeichner der Doctype-Deklaration des Dokuments zurück, wenn es einen gibt, sonst die leere Zeichenkette. Ist der Methode ein Wert angegeben, wird der Systembezeichner des Dokuments auf diesen Wert gesetzt.

internalSubset*?internalSubset?*
: Gibt die interne Teilmenge der Doctype-Deklaration des Dokuments zurück, wenn es eine gibt, sonst die leere Zeichenkette. Ist der Methode ein Wert angegeben, wird die interne Teilmenge des Dokuments auf diesen Wert gesetzt. Keine der Lesemethoden bewahrt die interne Teilmenge eines Dokuments. Ein frisch gelesenes Dokument hat immer eine leere interne Teilmenge. Die Methode prüft eine angegebene interne Teilmenge auch nicht syntaktisch.

cdataSectionElements *(?URI:?localname|*) ?<boolean>?*
: Diese Methode steuert, für welche Elementknoten die Textknoten-Kinder als CDATA-Abschnitte serialisiert werden. Das betrifft nur die Serialisierung mit der Methode asXML, kein Textknoten wird von dieser Methode irgendwie verändert. Wird die Methode mit einem Elementnamen als erstem Argument und einem booleschen Wert wahr als zweitem Argument aufgerufen, wird jedes Textknoten-Kind jedes Elementknotens im Dokument mit demselben Namen wie das erste Argument als CDATA-Abschnitt serialisiert. Ist das zweite Argument ein boolescher Wert falsch, werden alle Textknoten aller Elemente mit demselben Namen wie das erste Argument wie üblich serialisiert. Elementnamen mit Namensraum müssen in der Form

namespace_URI:localname angegeben werden, nicht in der sonst üblichen Form prefix:localname. Mit zwei Argumenten aufgerufen gibt die Methode den verwendeten booleschen Wert zurück. Wird die Methode nur mit einem Elementnamen aufgerufen, gibt sie einen booleschen Wert zurück, der anzeigt, dass die Textknoten-Kinder aller Elemente dieses Namens im Dokument als CDATA-Abschnitt-Elemente serialisiert werden (Rückgabewert 1) oder nicht (Rückgabewert 0). Wird die Methode mit nur einem Argument aufgerufen und ist dieses Argument ein Stern ('*'), gibt die Methode eine ungeordnete Liste aller Elementnamen des Dokuments zurück, für die die Textknoten-Kinder als CDATA-Abschnitt-Knoten serialisiert werden.

selectNodesNamespaces?prefixUriList?

Diese Methode steuert eine dokumentweite Zuordnung von Präfix zum Namensraum-URI. Sie wird für Aufrufe der Methode selectNodes verwendet, am Dokument und an allen Knoten, die zum Dokument gehören, sofern die Option -namespaces der Methode selectNodes sie nicht überschreibt. Jeder Namensraumpräfix in einem XPath-Ausdruck wird zuerst gegen diese Liste aufgelöst. Bindet die Liste denselben Präfix an verschiedene Namensräume, gilt die erste Bindung. Lässt sich ein Präfix nicht gegen die dokumentweite Liste aus Präfix und Namensräumen auflösen, werden wie üblich die Namensraumdefinitionen im Geltungsbereich des Kontextknotens verwendet. Ist das optionale Argument *prefixUriList* angegeben, wird die globale Liste aus Präfix und Namensraum auf diese Liste gesetzt, und die Methode gibt sie zurück. Ohne das optionale Argument gibt die Methode die aktuelle Liste zurück. Die Vorgabe ist die leere Liste.

xslt?-parameters parameterList??-ignoreUndeclaredParameters??-maxApplyDepth int??-xsltmessagecmd script? *stylesheet?outputVar?*

Wendet eine XSLT-Transformation auf das ganze Dokument des Knotenobjekts an, mit dem XSLT-*stylesheet*, angegeben als domDoc. Gibt ein Dokumentobjekt mit dem Ergebnisdokument der Transformation zurück und speichert dieses Dokumentobjekt in der optionalen *outputVar*, falls sie angegeben ist.

Die optionale Option *-parameters* setzt <xsl:param> der obersten Ebene auf Zeichenkettenwerte. Die *parameterList* muss eine Tcl-Liste aus Paaren von Parametername und Wert sein.

Ist die Option *-ignoreUndeclaredParameters* angegeben, werden Parameternamen in der *parameterList* der Option *-parameters*, die im Stylesheet nicht als Parameter der obersten Ebene deklariert sind, still übergangen. Ohne diese Option wird ein Fehler ausgelöst, wenn ein Parameter der obersten Ebene gesetzt wird, der im Stylesheet nicht deklariert ist.

Die Option *-maxApplyDepth* erwartet als Argument eine positive ganze Zahl. In der Vorgabe erlaubt die XSLT-Maschine, dass sich XSLT-Templates bis zu 3000 Ebenen schachteln, und löst einen Fehler aus, wenn sie tiefer schachteln. Diese Grenze lässt sich mit der Option *-maxApplyDepth* setzen.

Die Option *-xsltmessagecmd* setzt eine Rückruffunktion für Elemente xslt:message im Stylesheet. Der tatsächliche Befehl besteht aus dem Skript, das der Option als Argument angegeben ist, angehängt das XML-Fragment aus der Instanziierung des Elementinhalts von xsl:message als Zeichenkette, als wäre die XPath-Funktion string() auf das XML-Fragment angewandt worden, und eine Markierung, die anzeigt, ob xsl:message ein Attribut "terminate" mit dem Wert "yes" hat. Gibt das aufgerufene Skript etwas anderes als TCL_OK zurück, wird die XSLT-Transformation abgebrochen und ein Fehler zurückgegeben. Gibt das aufgerufene Skript

-code break zurück, ist die Fehlermeldung leer, sonst wird der Ergebniscode gemeldet. Bei abgebrochener Transformation wird outputVar, falls angegeben, auf die leere Zeichenkette gesetzt.

toXSLTcmd ?*objVar*?
: Stellt der DOM-Baum ein gültiges XSLT-Stylesheet dar, wandelt diese Methode den DOM-Baum in einen XSLT-Befehl um, sonst gibt sie einen Fehler zurück. Der angelegte xsltCmd wird zurückgegeben und in der *objVar* gespeichert, wenn ein Variablenname angegeben war. Eine erfolgreiche Umwandlung des DOM-Baums in einen xsltCmd entfernt den Befehl domDoc und alle nodeCmds des Dokuments.

Die Syntax des angelegten xsltCmd ist:

```
xsltCmd method ?arg ...?
```

Die gültigen Methoden sind:

transform?-parameters parameterList??-ignoreUndeclaredParameters? ?-maxApplyDepth int??-xsltmessagecmd script? *domDoc?outputVar?*
: Wendet die XSLT-Transformation auf das Dokument *domDoc* an. Gibt ein Dokumentobjekt mit dem Ergebnisdokument dieser Transformation zurück und speichert es in der optionalen *outputVar*.

Die optionale Option *-parameters* setzt <xsl:param> der obersten Ebene auf Zeichenkettenwerte. Die *parameterList* muss eine Tcl-Liste aus Paaren von Parametername und Wert sein.

Ist die Option *-ignoreUndeclaredParameters* angegeben, werden Parameternamen in der *parameterList* der Option *-parameters*, die im Stylesheet nicht als Parameter der obersten Ebene deklariert sind, still übergangen. Ohne diese Option wird ein Fehler ausgelöst, wenn ein Parameter der obersten Ebene gesetzt wird, der im Stylesheet nicht deklariert ist.

Die Option *-maxApplyDepth* erwartet als Argument eine positive ganze Zahl. In der Vorgabe erlaubt die XSLT-Maschine, dass sich XSLT-Templates bis zu 3000 Ebenen schachteln, und löst einen Fehler aus, wenn sie tiefer schachteln. Diese Grenze lässt sich mit der Option *-maxApplyDepth* setzen.

Die Option *-xsltmessagecmd* setzt eine Rückruffunktion für Elemente xslt:message im Stylesheet. Der tatsächliche Befehl besteht aus dem Skript, das der Option als Argument angegeben ist, angehängt das XML-Fragment aus der Instanziierung des Elementinhalts von xsl:message als Zeichenkette, als wäre die XPath-Funktion string() auf das XML-Fragment angewandt worden, und eine Markierung, die anzeigt, ob xsl:message ein Attribut "terminate" mit dem Wert "yes" hat.

delete
: Löscht den xsltCmd und räumt alle verwendeten Ressourcen auf. Ist das erste Argument eines xsltCmd ein domDoc oder beginnt es mit "-", wird der Befehl genauso verarbeitet wie *<xsltCmd> transform*.

normalize*?-forXPath?*
: Bringt alle Textknoten im Dokument in eine "normale" Form, in der nur Struktur, zum Beispiel Elemente, Kommentare, Verarbeitungsanweisungen und CDATA-Abschnitte, Textknoten trennt. Es gibt also weder benachbarte Textknoten noch leere Textknoten. Ist die Option *-forXPath*

angegeben, werden alle CDATA-Abschnitte in den Knoten als erster Schritt vor der Normalisierung in Textknoten umgewandelt.

nodeType
Gibt den Knotentyp des Dokumentknotens zurück. Das ist immer DOCUMENT_NODE.

getElementById *id*
Gibt den Knoten zurück, der ein id-Attribut mit dem Wert *id* hat, oder die leere Zeichenkette, wenn kein Knoten ein id-Attribut mit diesem Wert hat.

firstChild?objVar?
Gibt den ersten Knoten der obersten Ebene des Dokuments zurück.

lastChild?objVar?
Gibt den letzten Knoten der obersten Ebene des Dokuments zurück.

appendChild *newChild*
Hängt *newChild* an das Ende der Liste der Knoten der obersten Ebene des Dokuments an.

removeChild *child*
Entfernt *child* aus der Liste der Knoten der obersten Ebene des Dokuments. *child* ist nach dieser Operation Teil der Dokumentfragmentliste. Er wird nicht physikalisch gelöscht.

hasChildNodes
Gibt 1 zurück, wenn das Dokument Knoten im Baum hat. Sonst kommt 0 zurück.

childNodes
Gibt eine Liste der Knoten der obersten Ebene des Dokuments zurück.

ownerDocument?domObjVar?
Gibt das Dokument selbst zurück.

insertBefore *newChild refChild*
Fügt *newChild* vor *refChild* in die Liste der Knoten der obersten Ebene des Dokuments ein. Ist *refChild* die leere Zeichenkette, wird *newChild* an das Ende der Knoten der obersten Ebene eingefügt.

replaceChild *newChild oldChild*
Ersetzt *oldChild* durch *newChild* in der Liste der Kinder dieses Knotens. Der Knoten *oldChild* ist nach dieser Operation Teil der Dokumentfragmentliste.

appendFromList *list*
Liest *list*, legt einen entsprechenden DOM-Teilbaum an und hängt diesen Teilbaum an das Ende der aktuellen Liste der Knoten der obersten Ebene des Dokuments an.

appendXML *XMLstring*
Liest *XMLstring*, legt einen entsprechenden DOM-Teilbaum an und hängt diesen Teilbaum an das Ende der aktuellen Liste der Knoten der obersten Ebene des Dokuments an.

selectNodes?-namespaces prefixUriList??-cache <boolean>??-list? *xpathQuery* ?typeVar?
Gibt das Ergebnis der Anwendung der XPath-Abfrage *xpathQuery* auf das Dokument zurück. Der Kontextknoten der Abfrage ist der Wurzelknoten im Sinn der XPath-Empfehlung, nicht das Dokumentelement. Das Ergebnis kann eine Zeichenkette oder ein Wert, eine Liste von Zeichenketten, eine Liste von Knoten oder eine Liste aus Paaren von Attributname und Wert sein. Ist *typeVar* angegeben, wird der Name des Ergebnistyps in dieser Variable gespeichert: empty, bool, number, string, nodes, attrnodes oder mixed.

Die genaue Beschreibung der Argumente steht in der Dokumentation der der

domNode
Befehlsmethode **selectNodes**.

baseURI*?URI?*
Gibt die aktuelle baseURI des Dokuments zurück. Ist das optionale Argument URI angegeben, wird die Basis-URI des Dokuments auf die angegebene URI gesetzt.

appendFromScript *tclScript*
Hängt die Knoten, die das *tclScript* mit Tcl-Funktionen anlegt, die mit *dom createNodeCmd* gebaut wurden, an das Ende der aktuellen Liste der Knoten der obersten Ebene des Dokuments an.

insertBeforeFromScript *tclScript refChild*
Fügt die Knoten, die im *tclScript* mit Tcl-Funktionen angelegt werden, die mit *dom createNodeCmd* gebaut wurden, vor *refChild* in die Liste der Knoten der obersten Ebene des Dokuments ein. Ist *refChild* die leere Zeichenkette, werden die neuen Knoten angehängt.

deleteXPathCache*?xpathQuery?*
Ohne das optionale Argument werden alle zwischengespeicherten XPath-Ausdrücke des Dokuments freigegeben. Mit dem optionalen Argument *xpathQuery* wird diese einzelne XPath-Abfrage aus dem Zwischenspeicher entfernt, wenn sie dort liegt. Die Methode gibt immer eine leere Zeichenkette zurück. Wird sonst ein unbekannter Methodenname angegeben, wird versucht, den Befehl desselben Namens wie die angegebene Methode im Namensraum **::dom::domDoc** auszuführen. Das erlaubt schnelles Ergänzen von Methoden auf Tcl-Ebene. Neu angelegte Knoten werden an eine versteckte Fragmentliste angehängt. Werden sie nicht in den Baum verschoben, werden sie automatisch gelöscht, sobald das ganze Dokument gelöscht wird.

domNode

Stichwörter: domNode, XML, DOM, document, node, parsing

NAME

domNode - Eine Instanz eines DOM-Knotenobjekts bearbeiten

ÜBERSICHT

```
$nodeObject method  arg arg ...
```

BESCHREIBUNG

Dieser Befehl bearbeitet eine bestimmte Instanz eines DOM-Knotenobjekts. *method* nennt eine Methode der Knotenklasse. Die Methoden sollen sich eng an die W3C-Empfehlung "Document Object Model (Core) Level 1" (http://www.w3.org/TR/REC-DOM-Level-1/level-one-core.html) halten und an Teile des W3C-Entwurfs "XML Pointer Language (XPointer)" (http://www.w3.org/TR/1998/WD-xptr-19980303). Die XPointer-Methoden sind veraltet. Statt ihrer sind DOM-Methoden oder XPath-Ausdrücke zu verwenden.

Die Methode selectNodes setzt die W3C-Empfehlung "XML Path Language (XPath) Version 1.0" vom 16. November 1999 um (http://www.w3.org/TR/1999/REC-xpath-19991116). Diese Dokumente beschreiben die Funktion genauer.

Die gültigen Methoden sind:

nodeType
Gibt den Knotentyp dieses Knotenobjekts zurück. Das kann sein: ELEMENT_NODE, TEXT_NODE, CDATA_SECTION_NODE, COMMENT_NODE oder PROCESSING_INSTRUCTION_NODE.

nodeName
Gibt den Knotennamen dieses Knotenobjekts zurück. Das ist der Elementname für Elementknoten (Typ ELEMENT_NODE), das Ziel der Verarbeitungsanweisung, "#text" für einen Textknoten, "#comment" für Kommentarknoten oder "#cdata" für CDATA-Abschnittknoten.

nodeValue*?newValue?*
Gibt den Wert dieses Knotenobjekts zurück. Das ist der Text oder die Daten für Knoten der Typen TEXT_NODE, COMMENT_NODE, PROCESSING_INSTRUCTION_NODE oder CDATA_SECTION_NODE). Sonst ist der Wert leer. Ist der Knoten ein TEXT_NODE, COMMENT_NODE oder PROCESSING_INSTRUCTION_NODE und ist das optionale Argument *newValue* angegeben, wird der Knoten auf diesen Wert gesetzt.

hasChildNodes
Gibt 1 zurück, wenn der Knoten Kinder hat. Sonst kommt 0 zurück.

parentNode?objVar?
Gibt den Elternknoten zurück.

childNodes
Gibt eine Liste der direkten Kindknotenobjekte zurück.

childNodesLive
Gibt ein lebendiges nodeList-Objekt der Kindknoten des Knotens im Sinne der DOM-Empfehlung zurück. Lebendig heißt: Änderungen an den Kindern des Knotenobjekts, aus dem die Liste entstand, zeigen sich sofort in den Knoten, die die Zugriffe der NodeList liefern. Es ist keine stehende Aufnahme des Knoteninhalts. Die beiden Zugriffe des nodeList-Objekts sind "item <index>", der den Eintrag an der Stelle index in der Sammlung zurückgibt, und "length", der die Anzahl der Knoten in der Liste zurückgibt.

firstChild?objVar?
Gibt das erste Kind als Knotenobjekt zurück.

lastChild?objVar?
Gibt das letzte Kind als Knotenobjekt zurück.

nextSibling?objVar?
Gibt den nächsten Geschwisterknoten zum aktuellen Knoten als Knotenobjekt zurück.

previousSibling?objVar?
Gibt den vorigen Geschwisterknoten zum aktuellen Knoten als Knotenobjekt zurück.

getElementsByTagName *name*
Gibt eine Liste aller Elemente im Teilbaum zurück, die auf *name* passen (Glob-Muster).

getElementsByTagNameNS *uri localname*
Gibt eine Liste aller Elemente im Teilbaum zurück, die auf *localname* passen (Glob-Muster) und den angegebenen Namensraum *uri* haben.

getElementById *id*
Gibt den Knoten zurück, dessen id-Attribut den Wert *id* hat, oder die leere Zeichenkette, wenn kein Knoten ein id-Attribut mit diesem Wert hat.

hasAttribute *attributeName*

Gibt 1 zurück, wenn der Objektknoten ein Attribut des Namens *attributeName* enthält. Sonst kommt 0 zurück.

getAttribute *attributeName ?defaultValue?*
Gibt den Wert des Attributs *attributeName* zurück. Fehlt das Attribut, kommt *defaultValue* zurück.

setAttribute *attributeName newValue ?attributeName newValue ...?*
Setzt den Wert eines oder mehrerer Attribute. Jeder *attributeName* wird auf den zugehörigen *newValue* gesetzt. Fehlt zu einem *attributeName* das Attribut, wird es angelegt. Von Attributen, die wie XML-Namensraumdeklarationen aussehen, ist abzuraten.

removeAttribute *attributeName*
Entfernt das Attribut *attributeName*.

hasAttributeNS *uri localName*
Gibt 1 zurück, wenn der Objektknoten ein Attribut mit dem lokalen Namen *localName* im Namensraum *uri* enthält. Sonst kommt 0 zurück.

getAttributeNS *uri localName ?defaultValue?*
Gibt den Wert des Attributs mit dem lokalen Namen *localName* im Namensraum-URI *uri* zurück. Hat der Knoten dieses Attribut nicht, kommt *defaultValue* zurück.

setAttributeNS *uri qualifiedName newValue ?uri qualifiedName newValue ...?*
Setzt den Wert eines oder mehrerer voll qualifizierter Attribute. Jedes Attribut *qualifiedName* mit dem Namensraum-URI *uri* wird auf *newValue* gesetzt. Fehlt das Attribut, wird es angelegt. Ein Attribut in einem Namensraum braucht den Attributnamen mit Präfix, auch wenn ein vorhandenes Attribut einen neuen Wert bekommen soll. Bei der Suche, ob das Attribut schon existiert, zählen nur der angegebene *uri* und der lokale Name von *qualifiedName*.

```
$node setAttributeNS "http://some.uri.com/wow" prefix:attr1 attrValue
```

Ist der URI die leere Zeichenkette und hat der Attributname keinen Präfix, wirkt diese Methode wie **setAttribute**.

```
$node setAttributeNS "" attri "some Value"
```

Außer bei den besonderen Präfixen "xmlns" und "xml" muss *uri* nicht leer sein, wenn *qualifiedName* einen Präfix hat. Von XML-Namensraumdeklarationen ist abzuraten. Die Wirkungen sind verwickelt und nicht immer offensichtlich, bis hin zu Serialisierungen, die nach weiterer Verarbeitung nicht wohlgeformt sind.

removeAttributeNS *uri localName*
Entfernt das Attribut mit dem lokalen Namen *localName* im Namensraum *uri*.

attributes?attributeNamePattern?
Gibt Angaben zu den Attributen zurück, die auf *attributeNamePattern* passen. Fehlt *attributeNamePattern*, kommen Angaben zu allen Attributen zurück. Der Rückgabewert ist eine Tcl-Liste. Bei Attributen ohne Namensraum ist das Element nur der Attributname, bei Attributen mit Namensraum eine Unterliste aus drei Elementen. Bei einem gewöhnlichen Attribut mit Namensraum sind die Elemente der Unterliste {<localname> <prefix> <namespace_uri>}. Bei einer XML-Namensraumdeklaration ist es {<der definierte Präfix> <localname> ""}.

attributeNames?attributeNamePattern?
Gibt eine flache Liste aller Attributnamen zurück, wie sie in der XML-Quelle stehen und auf

attributeNamePattern passen. Fehlt *attributeNamePattern*, kommen alle Attributnamen als Tcl-Liste zurück.

appendChild *newChild*
Hängt *newChild* ans Ende der Kindliste des Knotens.

insertBefore *newChild refChild*
Fügt *newChild* vor *refChild* in die Kindliste des Knotens ein. Ist *refChild* die leere Zeichenkette, wird *newChild* ans Ende der Kindliste dieses Knotens gesetzt.

replaceChild *newChild oldChild*
Ersetzt *oldChild* durch *newChild* in der Kindliste dieses Knotens. Der Knoten *oldChild* gehört nach diesem Vorgang zur Dokumentfragmentliste.

removeChild *child*
Nimmt *child* aus der Kindliste dieses Knotens. *child* gehört nach diesem Vorgang zur Dokumentfragmentliste.

delete
Löscht den angegebenen Knoten und seinen gesamten Kindbaum und gibt den gesamten internen Speicher frei. Die betroffenen Knoten sind über die Dokumentfragmentliste nicht erreichbar.

cloneNode?-deep?
Klont diesen Knoten und nimmt den neu angelegten Knoten in die Dokumentfragmentliste auf. Mit der Option *-deep* werden auch alle Nachfahren geklont.

ownerDocument?domObjVar?
Gibt das Dokumentobjekt des Dokuments zurück, zu dem dieser Knoten gehört.

find *attrName attrVal*?objVar?
Sucht im Teilbaum ab dem aktuellen Knoten den Knoten mit dem Attributnamen *attrName* und dem Attributwert *attrVal*.

child *number*|*all* type attrName attrValue
(XPointer) Kind

descendant *number*|*all* type attrName attrValue
(XPointer) Nachfahre

ancestor *number*|*all* type attrName attrValue
(XPointer) Vorfahr

fsibling *number*|*all* type attrName attrValue
(XPointer) folgender Geschwisterknoten

psibling *number*|*all* type attrName attrValue
(XPointer) voriger Geschwisterknoten

root objVar
(XPointer) Wurzel

text
Gibt alle Textknoten-Kinder des aktuellen Knotens aneinandergehängt als eine Zeichenkette zurück.

target
Bei einem Verarbeitungsanweisungsknoten kommt der Zielteil zurück. Sonst wird ein Fehler erzeugt.

data

Bei einem Verarbeitungsanweisungsknoten kommt der Datenteil zurück. Bei einem Textknoten, Kommentarknoten oder CDATA-Abschnittknoten kommt der Wert zurück. Sonst wird ein Fehler erzeugt.

prefix
Gibt den Namensraum-Präfix zurück.

namespaceURI
Gibt den Namensraum-URI zurück.

localName
Gibt den localName aus dem Elementnamen des angegebenen Knotens zurück.

selectNodes?-namespaces prefixUriList??-cache <boolean>??-list? *xpathQuery* ?typeVar?
Gibt das Ergebnis der XPath-Abfrage *xpathQuery* auf den Teilbaum zurück. Das Ergebnis kann eine Zeichenkette oder ein Wert sein, eine Liste von Zeichenketten, eine Liste von Knoten oder eine Liste von Paaren aus Attributname und Wert. Ist das Argument *typeVar* angegeben, wird der Name des Ergebnistyps in diese Variable geschrieben (empty, bool, number, string, nodes, attrnodes oder mixed).

Das Argument *xpathQuery* muss ein gültiger XPath-1.0-Ausdruck sein. Von dieser Regel gibt es wenige Ausnahmen. Tcl-Variablenverweise, in der üblichen Tcl-Syntax $varname, dürfen im XPath-Ausdruck an jeder Stelle stehen, an der die XPath-Syntax eine XPath-Variable erlaubt. Abgesehen von den XPath-Syntaxregeln darf der Tcl-Variablenname jeder zulässige Tcl-Variablenname sein: lokale Variablen, globale Variablen, Array-Einträge und so weiter. Die xpath-Maschine sieht den Wert immer als Zeichenkettenliteral. Der Wert lässt sich bei Bedarf mit den passenden xpath-Funktionen number() und boolean() ausdrücklich in einen anderen Datentyp wandeln.

Ähnlich wie sich Literale über Tcl-Variablenverweise sicher in den XPath-Ausdruck setzen lassen, gibt es eine Schreibweise, um Elementnamen aus Tcl-Variablen einzusetzen. An jeder Stelle, an der die XPath-Syntax einen Knotentest erlaubt, kann ein Tcl-Variablenverweis in jeder Form stehen, nur das führende $ ist durch % ersetzt. So lassen sich Knoten mit seltsamen Knotennamen auswählen, die nach der passenden XML-Produktion ungültig sind. Das kann bei JSON-Daten nötig sein.

Die Option *-namespaces* erwartet als Argument eine Tcl-Liste aus Paaren von Präfix und Namensraum. Fehlt die Option, wird jeder Namensraum-Präfix im xpath-Ausdruck zuerst gegen die Liste der Präfix-Namensraum-Paare aufgelöst, die die Methode selectNodesNamespaces für das Dokument gesetzt hat, zu dem der Knoten gehört. Gelingt das nicht, gelten die Namensraumdefinitionen im Geltungsbereich des Kontextknotens. Ist die Option angegeben, wird jeder Namensraum-Präfix im xpath-Ausdruck zuerst gegen diese Liste aufgelöst, und die dokumentweite Präfix-Namensraum-Liste wird ignoriert. Bindet die Liste denselben Präfix an verschiedene Namensräume, gilt die erste Bindung. Gelingt das nicht, gelten wie üblich die Namensraumdefinitionen im Geltungsbereich des Kontextknotens.

Ist die Option *-cache* mit einem wahren Wert angegeben, wird *xpathQuery* in einem dokumenteigenen Cache gesucht. Wird die Abfrage gefunden, wird die gespeicherte vorkompilierte Abfrage verwendet. Wird sie nicht gefunden, wird sie kompiliert und für weitere Aufrufe im Cache gespeichert. Die als Zeichenkette angegebene *xpathQuery* ist der Schlüssel des Cache. Gleiche XPath-Ausdrücke, die sich nur im Leerraum unterscheiden, sind verschiedene

Cache-Einträge. Besondere Vorsicht ist nötig, wenn der XPath-Ausdruck Namensraum-Präfixe oder Verweise auf Tcl-Variablen enthält. Beides wird nach den XML-Präfix-Namensraum-Zuordnungen und den Tcl-Variablenwerten zur Kompilierzeit des Ausdrucks aufgelöst. Wird derselbe XPath-Ausdruck später in einem Zusammenhang mit anderen XML-Präfix-Namensraum-Zuordnungen oder anderen Werten der verwendeten Tcl-Variablen benutzt, muss der kompilierte Ausdruck zuerst mit der Methode **deleteXPathCache** aus dem Cache genommen werden, damit er neu kompiliert wird. Ohne die Option *-cache* ist diese Überlegung nie nötig.

Ist die Option *-list* angegeben, muss das Argument *xpathQuery* eine Tcl-Liste von XPath-Abfragen sein. Jede XPath-Abfrage in dieser Liste außer der letzten muss eine Knotenmenge liefern. Mit denselben Regeln für Kontext und Namensraumauflösung wie ohne *-list* läuft die erste Abfrage der Liste. Jeder Knoten ihrer Ergebnismenge ist der Kontextknoten der nächsten XPath-Abfrage der Liste, und so weiter. Zurück kommen die Ergebnismengen der letzten Abfrage der Liste, aneinandergehängt.

Beispiele:

```
set paragraphNodes [$node selectNodes
    {chapter[3]//para[@type='warning' or @type='error'} ]
foreach paragraph $paragraphNodes {
    lappend  values [$paragraph selectNodes attribute::type]
}

set doc [dom parse {<doc
    xmlns="http://www.defaultnamespace.org"><child/></doc>}]
set root [$doc documentElement]
set childNodes [$root selectNodes -namespaces {default
    http://www.defaultnamespace.org} default:child]
```

getLine
Gibt die Zeilennummer dieses Knotens im ursprünglich gelesenen XML zurück. Die Zählung beginnt bei 1.

getColumn
Gibt die Spaltennummer dieses Knotens im ursprünglich gelesenen XML zurück. Die Zählung beginnt bei 0.

getByteIndex
Gibt die Byteposition dieses Knotens im ursprünglich gelesenen XML zurück. Die Zählung beginnt bei 0.

asList
Gibt den DOM-Teilbaum ab dem aktuellen Knoten als verschachtelte Tcl-Liste zurück.

asXML?-indent none/tabs/1..8??-channel channelId??-escapeNonASCII? ?-doctypeDeclaration <boolean>? -xmlDeclaration <boolean>? -encString <string> ?-escapeAllQuot??-indentAttrs??-nogtescape??-noEmptyElementTag??-escapeCR? ?-escapeTab?
Gibt den DOM-Teilbaum ab dem aktuellen Knoten als Wurzel des Ergebnisses als optional eingerückte XML-Zeichenkette zurück oder sendet die Ausgabe direkt an die angegebene channelId.

Die genaue Beschreibung der Argumente steht in der Dokumentation des

domDoc
Befehls **asXML**.

asCanonicalXML?-channel channelId??-comment?
Gibt den DOM-Baum als kanonische XML-Zeichenkette nach der "Canonical XML Version 1.0 W3C Recommendation 15 March 2001" zurück oder sendet die Ausgabe direkt an die angegebene channelId.

Die genaue Beschreibung der Argumente steht in der Dokumentation des

domDoc
Befehls **asCanonicalXML**.

asHTML?-channel channelId? b?-escapeNonASCII??-htmlEntities??-doctypeDeclaration <boolean>??-breakLines??-onlyContents?
Gibt den DOM-Teilbaum ab dem aktuellen Knoten als Wurzel des Ergebnisses nach HTML-Regeln serialisiert zurück. HTML-Elemente werden unabhängig von der Schreibung erkannt, leere HTML-Elemente ohne Endmarken und so weiter. Das Ergebnis ist eine Zeichenkette, oder die Ausgabe geht direkt an die angegebene channelId.

Die genaue Beschreibung der Argumente steht in der Dokumentation der

domDoc
Methode **asHTML**.

asText
Bei ELEMENT_NODE gibt die Methode asText den Zeichenkettenwert jedes Textknoten-Nachfahren des Knotens in Dokumentreihenfolge ohne Maskierung aus. Bei jedem anderen Knotentyp gibt die Methode den XPath-Zeichenkettenwert dieses Knotens aus.

asJSON?-indent none/0..8??-channel channelId?
Die Methode asJSON serialisiert den Teilbaum ab dem Knoten, an dem die Methode aufgerufen wurde, in eine gültige JSON-Datenzeichenkette.

Die genaue Beschreibung der Methode steht in der Dokumentation der

domDoc
Methode **asJSON**.

asTclValue?typevariable?
Enthält der Teilbaum ab dem Knoten, an dem die Methode aufgerufen wurde, JSON-Typangaben, gibt diese Methode die JSON-Daten als verschachtelte Tcl-Datenstruktur zurück.

Die genaue Beschreibung der Methode steht in der Dokumentation der

domDoc
Methode **asTclValue**.

appendFromList *list*
Liest *list*, legt einen entsprechenden DOM-Teilbaum an und hängt ihn an den aktuellen Knoten.

appendFromScript *tclScript*
Hängt die Knoten, die im *tclScript* von Tcl-Funktionen angelegt werden, die mit *dom*

createNodeCmd gebaut wurden, an den angegebenen Knoten.

insertBeforeFromScript *tclScript refChild*
Fügt die Knoten, die im *tclScript* von Tcl-Funktionen angelegt werden, die mit *dom createNodeCmd* gebaut wurden, vor *refChild* in die Kindliste des Knotens ein. Ist *refChild* die leere Zeichenkette, werden die neuen Knoten angehängt.

appendXML *XMLstring*
Liest *XMLstring*, legt einen entsprechenden DOM-Teilbaum an und hängt ihn an den aktuellen Knoten.

simpleTranslate *outputVar specifications*
Überträgt den Teilbaum ab dem Objektknoten nach den Angaben in *specifications* und schreibt das Ergebnis in die Variable *outputVar*. Die Übertragung ähnelt stark dem Modus Cost Simple.

toXPath*?-legacy?*
Gibt einen XPath zurück, der den angegebenen Knoten in seinem Dokument genau bezeichnet. Dieser XPath gilt nur, solange der DOM-Baum danach nicht geändert wird. Mit der Option -legacy kommen andere XPath-Ausdrücke zurück, die nicht in allen Fällen funktionieren.

getBaseURI
Gibt die baseURI des Knotens zurück. Diese Methode ist veraltet, an ihrer Stelle steht die Methode *baseURI*.

baseURI*?URI?*
Gibt die aktuelle baseURI des Knotens zurück. Ist das optionale Argument URI angegeben, setzt es die Basis-URI des Knotens und aller seiner Kindknoten aus derselben Entität wie der Knoten auf den angegebenen URI.

disableOutputEscaping*?boolean?*
Diese Methode gilt nur für Textknoten. Bei jedem anderen Knoten kommt ein Fehler zurück. Ohne das optionale Argument gibt sie zurück, ob die Ausgabemaskierung aus ist. Der Rückgabewert 0 heißt: die Zeichen des Textknotens werden maskiert, damit beim Serialisieren gültiges XML entsteht. Das ist die Vorgabe für jeden gelesenen oder angelegten Textknoten, außer Textknoten in einem Ergebnisbaum einer XSLT-Transformation, für die das Stylesheet die abgeschaltete Ausgabemaskierung ausdrücklich verlangt hat. Der Rückgabewert 1 heißt: die Ausgabemaskierung ist für diesen Textknoten aus. Wird ein solcher Textknoten serialisiert, mit asXML oder asHTML, wird er wörtlich geschrieben, ohne Maskierung der besonderen XML-Zeichen. Ist der optionale boolesche Wert *boolean* angegeben, wird die Marke entsprechend gesetzt. Die Marke sollte nur auf 1 gesetzt werden, wenn die Wirkung bekannt ist.

precedes *refnode*
Vergleicht die relative Ordnung des Knotens und von *refnode*. Beide Knoten müssen zum selben Dokument gehören und dürfen nicht in der Fragmentliste des Dokuments stehen. Gibt wahr zurück, wenn der Knoten in Dokumentreihenfolge im Sinne der XPath-1.0-Empfehlung vor *refnode* steht, sonst falsch.

normalize*?-forXPath?*
Bringt alle Textknoten in der ganzen Tiefe des Teilbaums unter diesem Knoten in eine normale Form, in der nur Struktur Textknoten trennt, also Elemente, Kommentare, Verarbeitungsanweisungen und CDATA-Abschnitte. Es gibt dann weder benachbarte noch

leere Textknoten. Mit der Option *-forXPath* werden alle CDATA-Abschnitte in den Knoten zuerst in Textknoten gewandelt, vor der Normalisierung.

xslt?-parameters parameterList??-ignoreUndeclaredParameters??-maxApplyDepth int??-xsltmessagecmd script? *stylesheet?outputVar?*

Wendet eine XSLT-Transformation auf das Dokument an, mit dem XSLT-*stylesheet* als domDoc. Gibt ein Dokumentobjekt mit dem Ergebnisdokument dieser Transformation zurück und speichert es in der optionalen *outputVar*.

Die optionale Option *-parameters* setzt <xsl:param> der obersten Ebene auf Zeichenkettenwerte. *parameterList* muss eine Tcl-Liste aus Paaren von Parametername und Wert sein.

Mit der Option *-ignoreUndeclaredParameters* werden Parameternamen in der *parameterList* der Option *-parameters* still ignoriert, die im Stylesheet nicht als Parameter der obersten Ebene deklariert sind. Ohne diese Option wird ein Fehler ausgelöst, wenn ein Parameter der obersten Ebene gesetzt werden soll, den das Stylesheet nicht deklariert.

Die Option *-maxApplyDepth* erwartet eine positive ganze Zahl. Voreingestellt erlaubt die XSLT-Maschine eine Verschachtelung von XSLT-Vorlagen bis zu 3000 Ebenen und löst einen Fehler aus, wenn sie tiefer verschachteln. Diese Grenze setzt die Option *-maxApplyDepth*.

Die Option *-xsltmessagecmd* setzt eine Rückruffunktion für xslt:message-Elemente im Stylesheet. Der tatsächliche Befehl besteht aus dem Skript, das der Option als Argument übergeben wird, angehängt der XML-Fragment-Zeichenkette aus der Instanzierung des Inhalts des Elements xsl:message, so als wäre die XPath-Funktion string() auf das XML-Fragment angewendet worden, und einer Marke, die angibt, ob xsl:message ein Attribut "terminate" mit dem Wert "yes" hat. Gibt das aufgerufene Skript etwas anderes als TCL_OK zurück, bricht die XSLT-Transformation ab und gibt einen Fehler zurück. Gibt das aufgerufene Skript -code break zurück, ist die Fehlermeldung leer, sonst wird der Ergebniscode gemeldet. Bei abgebrochener Transformation wird outputVar, falls angegeben, auf die leere Zeichenkette gesetzt.

@attrName

Gibt den Wert des Attributs *attrName* zurück. Kurzform für *getAttribute*.

jsonType*?(OBJECT|ARRAY|NONE)|(STRING|NUMBER|TRUE|FALSE|NULL|NONE)?*

Nur Element- und Textknoten können einen JSON-Typ haben, und nur diese Knotentypen kennen die Methode *jsonType*. Die anderen Knotentypen geben einen Fehler zurück, wenn die Methode an ihnen aufgerufen wird. Gibt den jsonType des Knotens zurück. Ist das optionale Argument angegeben, wird der JSON-Typ des Knotens auf den angegebenen Typ gesetzt und zurückgegeben. Gültige Typargumente für Elementknoten sind OBJECT, ARRAY und NONE. Gültige Typargumente für Textknoten sind STRING, NUMBER, TRUE, FALSE, NULL und NONE. Ist sonst ein unbekannter Methodenname angegeben, wird versucht, den Befehl dieses Namens im Namensraum **::dom::domNode** auszuführen. So lassen sich auf Tcl-Ebene schnell Methoden ergänzen.

expat

Stichwörter: expat, SAX, push, pushparser

NAME

expat - Eine Instanz eines expat-Parserobjekts anlegen

ÜBERSICHT

package require tdom

expat ?*parsername*? ?*-namespace*? ?*arg arg..*?

xml::parser ?*parsername*? ?*-namespace*? ?*arg arg..*?

BESCHREIBUNG

Der mit *expat* oder *xml::parser* angelegte Parser, nur ein anderer Name desselben Befehls in einem eigenen Namensraum, kann jede Art von wohlgeformtem XML lesen. Die Parser sind stromorientierte XML-Parser. Vor dem Lesen werden Handler-Skripte beim Parser angemeldet. Diese Skripte werden aufgerufen, wenn der Parser die zugehörigen Strukturen im gelesenen Dokument findet. Eine Startmarke ist ein Beispiel für eine Struktur, für die sich ein Handler-Skript anmelden lässt.

Die Parser prüfen die Eingabe immer auf XML-Wohlgeformtheit und melden einen Fehler, wenn die Eingabe nicht wohlgeformt ist. Sie lesen die interne DTD und auf Wunsch die externe DTD und externe Entitäten, wenn der Bezeichner der externen Entitäten mit dem Skript -externalentitycommand aufgelöst wird, siehe dort. Mit der Option -validateCmd, siehe dort, wird die Eingabe zusätzlich geprüft.

Zusätzlich stellt der Tcl-Erweiterungscode dieses Befehls eine API bereit, um Handler in C einzuhängen. Bislang gibt es den Parser-Erweiterungsbefehl "tdom". Der von dieser Erweiterung eingebaute Handlersatz baut beim Lesen der Eingabe einen "tDOM"-DOM-Baum im Speicher.

Für die meisten Ereignisse lassen sich beliebig viele verschiedene Handler-Skripte und Handler in C anmelden. Tritt das Ereignis ein, werden sie der Reihe nach aufgerufen.

BEFEHLSOPTIONEN

-namespace
: Schaltet das Lesen von Namensräumen ein. Diese Option muss beim Anlegen des Parsers mit **expat** oder **xml::parser** angegeben werden. Mit **<parserobj> configure ...** lässt sich das Lesen von Namensräumen weder ein- noch ausschalten.

-namespaceseparator *char*
: Diese Option wirkt nur zusammen mit *-namespace*. Sie bestimmt das Zeichen, das zwischen Namensraum-URI und lokalem Namen eingefügt wird, wenn ein XML-Elementname an ein Handler-Skript gemeldet wird. Die Vorgabe ist das Zeichen ':'. Der Wert muss eine Zeichenkette aus einem Zeichen kleiner oder gleich \u00FF sein, vorzugsweise ein 7-Bit-ASCII-Zeichen, oder die leere Zeichenkette. Ist der Wert die leere Zeichenkette, auch wenn der Wert \x00 ist, werden Namensraum-URI und lokaler Name ohne Trenner aneinandergehängt.

-final *boolean*
: Diese Option sagt, ob die als Nächstes an die Methode parse übergebenen Dokumentdaten der letzte Teil des Dokuments sind. Der Wert "0" heißt, dass weitere Daten erwartet werden. Der Wert "1" heißt, dass nichts weiter erwartet wird. Die Vorgabe ist "1".

Steht die Option auf "0", meldet der Parser bestimmte Fehler nicht, wenn die XML-Daten am Ende der Eingabe nicht wohlgeformt sind, etwa nicht geschlossene oder unausgeglichene Start- oder Endmarken. Stattdessen kann der Parser Daten bis zum nächsten Aufruf der Methode parse aufheben und die Meldung eines Teils der Daten aufschieben.

Steht die Option auf "1", erzeugen Dokumente, die am Ende der Eingabe nicht wohlgeformt sind, einen Fehler.

-validateCmd *<tdom schema cmd>*
Diese Option erwartet den Namen eines tDOM-Schemabefehls. Ist sie angegeben, wird die Eingabe auch geprüft. Hat der Schemabefehl kein reportcmd gesetzt, beendet der erste Prüffehler das weitere Lesen, wie ein Wohlgeformtheitsfehler.

-baseurl *url*
Meldet dem Parser die Basis-URL des Dokuments.

-fastcall *boolean*
Voreingestellt ist diese Option 0, also aus. Ist sie an, wird jeder Handler, den die Optionen *-elementstartcommand*, *-elementendcommand* und *-characterdatacommand* von diesem Zeitpunkt an einbauen, so eingebaut, dass der Aufruf weniger Aufwand hat als üblich. Die Option darf zwischen dem Einbauen der Rückruffunktionen nach Wunsch umgeschaltet werden.

Dafür gelten Bedingungen. Die Handler-Prozedur muss definiert sein, bevor sie als Rückruffunktion verwendet wird. Sonst wird die Rückruffunktion wie üblich eingebaut. Die Handler-Prozedur darf nicht entfernt oder neu definiert werden, solange sie als Rückruffunktion dient. Geschieht das, stürzt das Programm wahrscheinlich ab oder führt beliebigen Code aus.

Rückruffunktionen, die eingebaut werden, während diese Option an ist, werden beim Ausführen nicht verfolgt.

-keepTextStart *boolean*
Voreingestellt ist diese Option 0, also aus. Ist sie an, bleibt die Positionsangabe des Anfangs eines Text- oder CDATA-Knotens erhalten, während die von expat manchmal stückweise gelieferten cdata-Teile gesammelt werden. Mit dieser Option sind die Positionsangaben richtig, die der Parser in einer Prozedur -characterdatacommand zurückgibt, sonst nicht. In allem anderen Handler-Code gibt der Parser immer die richtigen Positionen zurück, unabhängig vom Wert dieser Option. Sie ist voreingestellt aus, weil sie selten gebraucht wird und so ein paar CPU-Takte spart.

-elementstartcommand *script*
Legt einen Tcl-Befehl fest, der zur Startmarke eines Elements gehört. Der tatsächliche Befehl besteht aus dieser Option und mindestens zwei Argumenten: dem Elementtypnamen und der Attributliste.

Die Attributliste ist eine Tcl-Liste aus Name-Wert-Paaren, geeignet für den Tcl-Befehl array set.

Beispiel:

```
proc HandleStart {name attlist} {
    puts stderr "Element start ==> $name has attributes $attlist"
}

$parser configure -elementstartcommand HandleStart

$parser parse {<test id="123"></test>}
```

Dabei wird dieser Befehl aufgerufen:

```
HandleStart text {id 123}
```

-elementendcommand *script*
Legt einen Tcl-Befehl fest, der zur Endmarke eines Elements gehört. Der tatsächliche Befehl besteht aus dieser Option und mindestens einem Argument, dem Elementtypnamen. Ist zusätzlich die Option -reportempty gesetzt, kann der Befehl mit der Konfigurationsoption -empty aufgerufen werden, die angibt, ob das Element leer ist. Ein Beispiel steht in der Beschreibung der Option -reportempty.

Beispiel:

```
proc HandleEnd {name} {
    puts stderr "Element end ==> $name"
}

$parser configure -elementendcommand HandleEnd

$parser parse {<test id="123"></test>}
```

Dabei wird dieser Befehl aufgerufen:

```
HandleEnd test
```

-characterdatacommand *script*
Legt einen Tcl-Befehl fest, der zu Zeichendaten im Dokument gehört, also zu Text. Der tatsächliche Befehl besteht aus dieser Option und einem Argument, dem Text.

Anders als bei der C-API des expat-Parsers ist zugesichert, dass Zeichendaten der Anwendung in einem einzigen Aufruf dieses Befehls übergeben werden.

Beispiel:

```
proc HandleText {data} {
    puts stderr "Character data ==> $data"
}

$parser configure -characterdatacommand HandleText

$parser parse {<test>this is a test document</test>}
```

Dabei wird dieser Befehl aufgerufen:

```
HandleText {this is a test document}
```

-processinginstructioncommand *script*
Legt einen Tcl-Befehl fest, der zu Verarbeitungsanweisungen im Dokument gehört. Der tatsächliche Befehl besteht aus dieser Option und zwei Argumenten: dem Ziel der Verarbeitungsanweisung und ihren Daten.

Beispiel:

```
proc HandlePI {target data} {
    puts stderr "Processing instruction ==> $target $data"
}

$parser configure -processinginstructioncommand HandlePI

$parser parse {<test><?special this is a processing
    instruction?></test>}
```

Dabei wird dieser Befehl aufgerufen:

```
HandlePI special {this is a processing instruction}
```

-notationdeclcommand *script*
Legt einen Tcl-Befehl fest, der zu einer Notationsdeklaration im Dokument gehört. Der tatsächliche Befehl besteht aus dieser Option und vier Argumenten: dem Notationsnamen, der Basis-URI des Dokuments, also dem, was die Option -baseurl gesetzt hat, dem Systembezeichner und dem öffentlichen Bezeichner. Der Notationsname ist nie leer, die anderen Argumente können es sein.

-externalentitycommand *script*
Legt einen Tcl-Befehl fest, der zu Verweisen auf externe Entitäten im Dokument gehört. Der tatsächliche Befehl besteht aus dieser Option und drei Argumenten: der Basis-URI, dem Systembezeichner der Entität und dem öffentlichen Bezeichner der Entität. Die Basis-URI und der öffentliche Bezeichner können die leere Liste sein.

Dieses Handler-Skript gibt eine Tcl-Liste aus drei Elementen zurück. Das erste Element sagt, wie die externe Entität an den Prozessor zurückgegeben wird. Derzeit sind die Typen "string", "channel" und "filename" erlaubt. Das zweite Element muss die absolute Basis-URI der zu lesenden externen Entität sein. Das dritte Element sind die Daten: bei Typ "string" die bereits gelesenen Daten der externen Entität als Zeichenkette, bei Typ "channel" der Name eines Tcl-Kanals, bei Typ "filename" der Pfad der zu lesenden externen Entität. Hinter den Kulissen wird die externe Entität, auf die der zurückgegebene Tcl-Kanal, die Zeichenkette oder der Dateiname zeigt, mit einem expat-Parser für externe Entitäten gelesen, mit denselben Handlersätzen wie der Hauptparser. Schlägt das Lesen der externen Entität fehl, stoppt das gesamte Lesen mit einer Fehlermeldung. Kann ein als externalentitycommand angemeldeter Tcl-Befehl eine externe Entität nicht auflösen, darf er TCL_CONTINUE zurückgeben. Dann versucht der Wrapper den nächsten angemeldeten externalentitycommand. Kann keiner die externe Entität behandeln, stoppt das Lesen mit einem Fehler.

Beispiel:

```
proc externalEntityRefHandler {base systemId publicId} {
    if {![regexp {^[a-zA-Z]+:/} $systemId]}  {
        regsub {^[a-zA-Z]+:} $base {} base
        set basedir [file dirname $base]
        set systemId "[set basedir]/[set systemId]"
    } else {
        regsub {^[a-zA-Z]+:} $systemId systemId
    }
```

```
    if {[catch {set fd [open $systemId]}]} {
        return -code error \
                -errorinfo "Failed to open external entity $systemId"
    }
    return [list channel $systemId $fd]
}

set parser [expat -externalentitycommand externalEntityRefHandler \
                  -baseurl "file:///local/doc/doc.xml" \
                  -paramentityparsing notstandalone]
$parser parse {<?xml version='1.0'?>
<!DOCTYPE test SYSTEM "test.dtd">
<test/>}
```

Dabei wird dieser Befehl aufgerufen:

```
externalEntityRefHandler file:///local/doc/doc.xml test.dtd {}
```

Externe Entitäten werden über dieses Handler-Skript nur aufgelöst, wenn es nötig ist. Externe Parameterentitäten lösen diesen Handler nur aus, wenn -paramentityparsing mit dem Argument "always" verwendet wird oder wenn -paramentityparsing mit dem Argument "notstandalone" verwendet wird und das Dokument nicht als standalone gekennzeichnet ist.

-unknownencodingcommand *script*
Auf Tcl-Ebene nicht umgesetzt.

-startnamespacedeclcommand *script*
Legt einen Tcl-Befehl fest, der zum Beginn des Geltungsbereichs von Namensraumdeklarationen im Dokument gehört. Der tatsächliche Befehl besteht aus dieser Option und zwei Argumenten: dem Namensraum-Präfix und dem Namensraum-URI. Bei einem Attribut xmlns ist der Präfix die leere Liste. Bei einem Attribut xmlns="" ist der URI die leere Liste. Die Aufrufe der Handler für Start- und Endelement liegen zwischen den Aufrufen der Handler für Beginn und Ende der Namensraumdeklaration.

-endnamespacedeclcommand *script*
Legt einen Tcl-Befehl fest, der zum Ende des Geltungsbereichs von Namensraumdeklarationen im Dokument gehört. Der tatsächliche Befehl besteht aus dieser Option und dem Namensraum-Präfix als Argument. Bei einem Attribut xmlns ist der Präfix die leere Liste. Die Aufrufe der Handler für Start- und Endelement liegen zwischen den Aufrufen der Handler für Beginn und Ende der Namensraumdeklaration.

-commentcommand *script*
Legt einen Tcl-Befehl fest, der zu Kommentaren im Dokument gehört. Der tatsächliche Befehl besteht aus dieser Option und einem Argument, den Kommentardaten.

Beispiel:

```
proc HandleComment {data} {
    puts stderr "Comment ==> $data"
}
```

```
$parser configure -commentcommand HandleComment

$parser parse {<test><!-- this is <obviously> a comment --></test>}
```

Dabei wird dieser Befehl aufgerufen:

```
HandleComment { this is <obviously> a comment }
```

-notstandalonecommand *script*
Dieser Tcl-Befehl wird aufgerufen, wenn das Dokument nicht eigenständig ist. Es hat eine externe Teilmenge oder einen Verweis auf eine Parameterentität, aber kein standalone="yes". Der Aufruf hat keine weiteren Argumente.

-startcdatasectioncommand *script*
Legt einen Tcl-Befehl fest, der zum Beginn eines CDATA-Abschnitts gehört. Der Aufruf hat keine weiteren Argumente.

-endcdatasectioncommand *script*
Legt einen Tcl-Befehl fest, der zum Ende eines CDATA-Abschnitts gehört. Der Aufruf hat keine weiteren Argumente.

-elementdeclcommand *script*
Legt einen Tcl-Befehl fest, der zu Elementdeklarationen gehört. Der tatsächliche Befehl besteht aus dieser Option und zwei Argumenten: dem Namen des Elements und dem Inhaltsmodell. Das Argument des Inhaltsmodells ist eine Tcl-Liste aus vier Elementen. Das erste Listenelement nennt den Typ des XML-Elements. Die sechs möglichen Typen werden als "MIXED", "NAME", "EMPTY", "CHOICE", "SEQ" oder "ANY" gemeldet. Das zweite Listenelement nennt den Quantor des Inhaltsmodells in XML-Syntax ("?", "*" oder "+") oder ist die leere Liste. Ist der Typ "MIXED", ist der Quantor "{}", das heißt ein Element nur mit PCDATA, oder "*", und die Elemente, die sich mit PCDATA mischen dürfen, stehen als Tcl-Liste im vierten Argument. Ist der Typ "NAME", ist der Name das dritte Argument, sonst ist das dritte Argument die leere Liste. Ist der Typ "CHOICE" oder "SEQ", enthält das vierte Argument eine Liste von Inhaltsmodellen, die wie dieses aufgebaut sind. Die Typen "EMPTY", "ANY" und "MIXED" kommen nur auf der obersten Ebene vor.

Beispiele:

```
proc elDeclHandler {name content} {
     puts "$name $content"
}

set parser [expat -elementdeclcommand elDeclHandler]
$parser parse {<?xml version='1.0'?>
<!DOCTYPE test [
<!ELEMENT test (#PCDATA)>
]>
<test>foo</test>}
```

Dabei wird dieser Befehl aufgerufen:

```
test {MIXED {} {} {}}

$parser reset
$parser parse {<?xml version='1.0'?>
<!DOCTYPE test [
<!ELEMENT test (a|b)>
]>
<test><a/></test>}
```

Dabei wird dieser Befehl aufgerufen:

```
elDeclHandler test {CHOICE {} {} {{NAME {} a {}} {NAME {} b {}}}}
```

-attlistdeclcommand *script*
Legt einen Tcl-Befehl fest, der zu Attlist-Deklarationen gehört. Der tatsächliche Befehl besteht aus dieser Option und fünf Argumenten. Der Handler der Attlist-Deklaration wird für *jedes* Attribut aufgerufen. Eine einzelne Attlist-Deklaration mit mehreren Attributen erzeugt also mehrere Aufrufe dieses Handlers. Die Argumente sind der Elementname, zu dem das Attribut gehört, der Name des Attributs, der Typ des Attributs, der Vorgabewert, der die leere Liste sein kann, und eine Marke required. Ist diese Marke wahr und der Vorgabewert nicht die leere Liste, ist das eine Vorgabe "#FIXED".

Beispiel:

```
proc attlistHandler {elname name type default isRequired} {
    puts "$elname $name $type $default $isRequired"
}

set parser [expat -attlistdeclcommand attlistHandler]
$parser parse {<?xml version='1.0'?>
<!DOCTYPE test [
<!ELEMENT test EMPTY>
<!ATTLIST test
          id      ID      #REQUIRED
          name    CDATA   #IMPLIED>
]>
<test/>}
```

Dabei werden diese Befehle aufgerufen:

```
attlistHandler test id ID {} 1
attlistHandler test name CDATA {} 0
```

-startdoctypedeclcommand *script*
Legt einen Tcl-Befehl fest, der zum Beginn der DOCTYPE-Deklaration gehört. Dieser Befehl wird aufgerufen, bevor eine DTD oder die interne Teilmenge gelesen wird. Der tatsächliche Befehl besteht aus dieser Option und vier Argumenten: dem Doctype-Namen, dem Systembezeichner, dem öffentlichen Bezeichner und einem Booleschen, das angibt, ob der DOCTYPE eine interne Teilmenge hat.

-enddoctypedeclcommand *script*

Legt einen Tcl-Befehl fest, der zum Ende der DOCTYPE-Deklaration gehört. Dieser Befehl wird nach der Verarbeitung einer externen Teilmenge aufgerufen. Der Aufruf hat keine weiteren Argumente.

-paramentityparsing *never*|*notstandalone*|*always*
"never" schaltet das Auflösen von Parameterentitäten aus, "always" löst immer auf, und "notstandalone" nur, wenn das Dokument nicht "standalone='no'" ist. Die Vorgabe ist "never".

-entitydeclcommand *script*
Legt einen Tcl-Befehl fest, der zu jeder Entitätsdeklaration gehört. Der tatsächliche Befehl besteht aus dieser Option und sieben Argumenten: dem Entitätsnamen, einem Booleschen, das Parameterentitäten kennzeichnet, dem Wert der Entität, der Basis-URI, dem Systembezeichner, dem öffentlichen Bezeichner und dem Notationsnamen. Je nach Art der Entitätsdeklaration können einige dieser Argumente die leere Liste sein.

-ignorewhitecdata *boolean*
Ist diese Marke gesetzt, wird Elementinhalt, der nur aus Leerraum besteht, nicht mit **-characterdatacommand** gemeldet.

-ignorewhitespace *boolean*
Ein anderer Name für *-ignorewhitecdata*, siehe dort.

-handlerset *name*
Diese Option setzt den Geltungsbereich des Tcl-Handlersatzes für die configure-Optionen. Jedes Options-Wert-Paar, das in demselben Aufruf des Parsers auf diese Option folgt, ändert den genannten Tcl-Handlersatz. Ohne diese Option wird der Vorgabe-Handlersatz "default" geändert.

-noexpand *boolean*
Normalerweise versucht der Parser, Verweise auf Entitäten aufzulösen, die in der internen Teilmenge definiert sind. Steht diese Option auf wahr, werden diese Entitäten nicht aufgelöst, sondern wörtlich über den Vorgabe-Handler gemeldet. **Warnung:** Wird diese Option auf wahr gesetzt und ist nicht für jeden Handlersatz des Parsers ein Vorgabe-Handler mit der Option -defaultcommand eingebaut, gehen alle internen Entitäten für die Handlersätze ohne Vorgabe-Handler still verloren.

-useForeignDTD <*boolean*>
Ist <boolean> wahr und hat das Dokument keine externe Teilmenge, ruft der Parser das Skript -externalentitycommand mit leeren Werten für die Argumente systemId und publicID auf. Diese Option muss gesetzt sein, bevor das erste Stück Daten gelesen wird. Sie nach Beginn des Lesens zu setzen, hat keine Wirkung. Voreingestellt wird keine fremde DTD verwendet. Die Vorgabe wird nach einem Zurücksetzen des Parsers wiederhergestellt. Ein Wert "never" von -paramentityparsing, das ist die Vorgabe, unterdrückt jeden Aufruf des Skripts -externalentitycommand. Hat das Dokument auch keine interne Teilmenge, werden die Skripte -startdoctypedeclcommand und enddoctypedeclcommand, sofern gesetzt, nicht aufgerufen.

-billionLaughsAttackProtectionMaximumAmplification <*float*>
<URL: https://en.wikipedia.org/wiki/Billion_laughs_attack> "" Diese Option steuert zusammen mit *-billionLaughsAttackProtectionActivationThreshold* die Parsergrenzen gegen Billion-Laughs-Angriffe (). Erwartet wird eine Gleitkommazahl >= 1.0. Die Option sollte nie nötig sein, die Vorgabe 100.0 reicht für wirkliche Daten. Muss der Wert für eine Nutzlast erhöht werden, die kein Angriff ist, bitte melden.

-billionLaughsAttackProtectionActivationThreshold *<long>*
<URL: https://en.wikipedia.org/wiki/Billion_laughs_attack> "" Diese Option steuert zusammen mit *-billionLaughsAttackProtectionMaximumAmplification* die Parsergrenzen gegen Billion-Laughs-Angriffe (). Erwartet wird eine positive ganze Zahl. Die Option sollte nie nötig sein, die Vorgabe 8388608 reicht für wirkliche Daten. Muss der Wert für eine Nutzlast erhöht werden, die kein Angriff ist, bitte melden.

-setReparseDeferralEnabled *<boolean>*
Expat schützt sich gegen quadratische Lesezeit durch lange Token in böswillig gebauten Dokumenten. Als Nebenwirkung kann ein Leseereignis nicht direkt nach dem Ende des betreffenden Tokens ausgelöst werden, sondern erst, nachdem der Parser weitere Bytes der Eingabe verarbeitet hat. Diese Option schaltet dieses Verhalten ab.

-allocTrackerActivationThreshold *<float>*
Diese Option steuert zusammen mit *-allocTrackerMaximumAmplification* die Parsergrenzen gegen unverhältnismäßig viel dynamischen Speicher durch böswillig gebaute Dokumente. Erwartet wird eine positive ganze Zahl. Die Option sollte nie nötig sein, die Vorgabe 100.0 reicht für wirkliche Daten. Muss der Wert für eine Nutzlast erhöht werden, die kein Angriff ist, bitte melden.

-allocTrackerMaximumAmplification *<long>*
Diese Option steuert zusammen mit *-allocTrackerActivationThreshold* die Parsergrenzen gegen unverhältnismäßig viel dynamischen Speicher durch böswillig gebaute Dokumente. Erwartet wird eine positive ganze Zahl. Die Option sollte nie nötig sein, die Vorgabe 67108864, das sind 64 MiB, reicht für wirkliche Daten. Muss der Wert für eine Nutzlast erhöht werden, die kein Angriff ist, bitte melden.

BEFEHLSMETHODEN

parser configure *option value ?option value?*
Setzt Konfigurationsoptionen des Parsers. Jede Befehlsoption außer *-namespace* lässt sich mit dieser Methode setzen oder ändern.

parser cget*?-handlerset name? option*
Gibt den aktuellen Konfigurationswert der Option für den Parser zurück.

Mit der Option -handlerset kommt die Konfiguration des genannten Handlersatzes zurück.

parser currentmarkup
Gibt das aktuelle Markup zurück, wie es im XML steht, wenn der Aufruf aus einem Handler-Skript für Markup-Ereignisse kommt, also -elementstartcommand, -elementendcommand, -commentcommand und -processinginstructioncommand. Sonst kommt die leere Zeichenkette zurück.

parser delete
Löscht den Parser und den Parserbefehl. Ein Parser lässt sich nicht aus einer seiner Handler-Rückruffunktionen löschen, weder direkt noch indirekt. In dem Fall wird ein Tcl-Fehler ausgelöst.

parser free
Ein anderer Name für den Aufruf der Methode *delete*, siehe dort.

parser get
-specifiedattributecount|-idattributeindex|-currentbytecount|-currentlinenumber|-currentcolumn

-specifiedattributecount
Gibt die Anzahl der Attribut-Wert-Paare zurück, die beim letzten Aufruf von elementstartcommand übergeben wurden und in der Startmarke angegeben waren, nicht als Vorgabe. Jedes Attribut-Wert-Paar zählt als 2. Das entspricht einem Index in die Attributliste, die an elementstartcommand übergeben wird.

-idattributeindex
Gibt den Index des ID-Attributs zurück, das beim letzten Aufruf von XML_StartElementHandler übergeben wurde, oder -1, wenn es kein ID-Attribut gibt. Jedes Attribut-Wert-Paar zählt als 2. Das entspricht einem Index in die Attributliste, die an elementstartcommand übergeben wird.

-currentbytecount
Gibt die Anzahl der Bytes im aktuellen Ereignis zurück. Kommt 0 zurück, liegt das Ereignis in einer internen Entität.

Bei dieser Option ist zu prüfen, ob die Parseroption *-keepTextStart* gebraucht wird.

-currentlinenumber
Gibt die Zeilennummer der aktuellen Lesestelle zurück.

Bei dieser Option ist zu prüfen, ob die Parseroption *-keepTextStart* gebraucht wird.

-currentcolumnnumber
Gibt die Spaltennummer der aktuellen Lesestelle zurück.

Bei dieser Option ist zu prüfen, ob die Parseroption *-keepTextStart* gebraucht wird.

-currentbyteindex
Gibt den Byteindex der aktuellen Lesestelle zurück.

Bei dieser Option ist zu prüfen, ob die Parseroption *-keepTextStart* gebraucht wird.

Es kann immer nur ein Wert erfragt werden.

parser parse *data*
Liest die XML-Zeichenkette *data*. Die Ereignis-Rückrufskripte werden aufgerufen, wenn ihre auslösenden Ereignisse eintreten. Diese Methode lässt sich nicht aus einer Rückruffunktion des zu verwendenden Parsers aufrufen, weder direkt noch indirekt. In dem Fall wird ein Fehler ausgelöst.

parser parsechannel *channelID*
Liest die XML-Daten aus dem Tcl-Kanal *channelID*, ab der aktuellen Zugriffsposition, ohne Seek, bis zum Dateiende, und liest diese Daten. Die Kodierung des Kanals wird beachtet. Zum Öffnen einer Datei für diese Methode dient die Hilfsprozedur tDOM::xmlOpenFile aus der Skriptbibliothek von tDOM. Diese Methode lässt sich nicht aus einer Rückruffunktion des zu verwendenden Parsers aufrufen, weder direkt noch indirekt. In dem Fall wird ein Fehler ausgelöst.

parser parsefile *filename*
Liest die XML-Daten direkt aus der Datei *filename* und liest diese Daten. Das geschieht mit Dateioperationen auf niedriger Ebene. Die XML-Daten müssen in US-ASCII, ISO-8859-1, UTF-8 oder UTF-16 kodiert sein. Wo das passt, ist das der schnellste Weg, XML-Daten zu lesen. Diese Methode lässt sich nicht aus einer Rückruffunktion des zu verwendenden Parsers aufrufen, weder direkt noch indirekt. In dem Fall wird ein Fehler ausgelöst.

parser reset
Setzt den Parser zurück, damit ein weiteres Dokument gelesen werden kann. Ein Parser lässt

sich nicht aus einer seiner Handler-Rückruffunktionen zurücksetzen, weder direkt noch indirekt. In diesen Fällen wird ein Tcl-Fehler ausgelöst.

RÜCKGABECODES DER RÜCKRUFE

Ein Skript, das für einen der Parser-Rückrufbefehle aufgerufen wird, etwa -elementstartcommand oder -elementendcommand, darf einen anderen Fehlercode als "ok" oder "error" zurückgeben. Alle Rückruffunktionen dürfen zusätzlich "break" oder "continue" zurückgeben.

Gibt ein Rückrufskript den Fehlercode "error" zurück, wird die Verarbeitung des Dokuments beendet und der Fehler wie üblich weitergereicht.

Gibt ein Rückrufskript den Fehlercode "break" zurück, unterbleibt jede weitere Verarbeitung jedes Handler-Skripts dieses Tcl-Handlersatzes für das weitere Lesen. Andere Handlersätze sind davon nicht betroffen.

Gibt ein Rückrufskript den Fehlercode "continue" zurück, endet die Verarbeitung des aktuellen Elements und seiner Kinder für jedes Handler-Skript dieses Tcl-Handlersatzes, und die Verarbeitung geht mit dem nächsten Geschwisterelement weiter. Andere Handlersätze sind davon nicht betroffen.

Gibt ein Rückrufskript den Fehlercode "return" zurück, wird das Lesen insgesamt abgebrochen, ohne dass ein Fehler ausgelöst wird.

expatapi

Stichwörter: expatapi, C handler set

NAME

CheckExpatParserObj, CHandlerSetInstall, CHandlerSetRemove, CHandlerSetCreate, CHandlerSetGetUserData, GetExpatInfo - Funktionen, um Erweiterungen von expat-Parserobjekten anzulegen, einzurichten und zu entfernen.

ÜBERSICHT

#include <tclexpat.h>

int
CheckExpatParserObj (*interp*, *nameObj*)

int
CHandlerSetInstall (*interp*, *expatObj*, *handlerSet*)

int
CHandlerSetRemove (*interp*, *expatObj*, *handlerSetName*)

CHandlerSet*
CHandlerSetCreate (*handlerSetName*)

CHandlerSet*
CHandlerSetGet (*interp*, *expatObj*, *handlerSetName*)

void*
CHandlerSetGetUserData (*interp*, *expatObj*, *handlerSetName*)

TclGenExpatInfo*
GetExpatInfo (*interp*, *expatObj*)

ARGUMENTE

Tcl_Interp *_interp_ (in)
Interpreter mit dem expat-Parserobjekt.

Tcl_Obj *_expatObj_ (in)
Ein Tcl-Objekt mit dem Befehlsnamen des abzufragenden oder zu ändernden expat-Parserobjekts.

char *_handlerSetName_ (in)
Bezeichner der Handler-Menge.

CHandlerSet *_handlerSet_ (in)
Zeiger auf eine C-Handler-Menge.

Tcl_Obj *_nameObj_
Ein Tcl-Objekt mit dem Namen eines expat-Parserobjekts.

BESCHREIBUNG

Die hier beschriebenen Funktionen erlauben es, einem tDOM-Tcl-expat-Parserobjekt auf C-Ebene geschriebene Ereignis-Handler hinzuzufügen. Ein tDOM-Tcl-expat-Parserobjekt kann zu einem bestimmten Ereignis mehrere Tcl-Skripte und C-Funktionen haben. Tritt das Ereignis ein, werden zuerst die Tcl-Skripte und dann die dem Ereignis zugeordneten C-Funktionen der Reihe nach aufgerufen.

Eine tDOM-Tcl-expat-Parsererweiterung ist eine gewöhnliche Tcl-Erweiterung und wird wie jede andere Tcl-Erweiterung geladen. Mindestens ein neuer Befehl der Tcl-Ebene muss eingerichtet werden, der ein tDOM-Tcl-expat-Parserobjekt bearbeitet.

Eine Handler-Menge auf C-Ebene ist eine Datenstruktur wie diese:

```
typedef struct CHandlerSet {
    struct CHandlerSet *nextHandlerSet;
    char *name;                     /* refname of the handler set */
    int ignoreWhiteCDATAs;          /* ignore 'white' CDATA sections */

    void *userData;                 /* Handler set specific Data
    Structure;
                                       the C handler set extension has
    to
                                       malloc the needed structure in
    his
                                       init func and has to provide a
                                       cleanup func (to free it). */

    CHandlerSet_userDataReset        resetProc;
    CHandlerSet_userDataFree         freeProc;

    /* C func for element start */
    XML_StartElementHandler          elementstartcommand;
    /* C func for element end */
    XML_EndElementHandler            elementendcommand;
```

```
    /* C func for character data */
    XML_CharacterDataHandler         datacommand;
    /* C func for namespace decl start */
    XML_StartNamespaceDeclHandler    startnsdeclcommand;
    /* C func for namespace decl end */
    XML_EndNamespaceDeclHandler      endnsdeclcommand;
    /* C func for processing instruction */
    XML_ProcessingInstructionHandler picommand;
    /* C func for default data */
    XML_DefaultHandler               defaultcommand;
    /* C func for unparsed entity declaration */
    XML_NotationDeclHandler          notationcommand;
    /* C func for external entity */
    XML_ExternalEntityRefHandler     externalentitycommand;
    /* C func for unknown encoding */
    XML_UnknownEncodingHandler       unknownencodingcommand;
    /* C func for comments */
    XML_CommentHandler               commentCommand;
    /* C func for "not standalone" docs */
    XML_NotStandaloneHandler         notStandaloneCommand;
    /* C func for CDATA section start */
    XML_StartCdataSectionHandler     startCdataSectionCommand;
    /* C func for CDATA section end */
    XML_EndCdataSectionHandler       endCdataSectionCommand;
    /* C func for !ELEMENT decl's */
    XML_ElementDeclHandler           elementDeclCommand;
    /* C func for !ATTLIST decl's */
    XML_AttlistDeclHandler           attlistDeclCommand;
    /* C func for !DOCTYPE decl's */
    XML_StartDoctypeDeclHandler      startDoctypeDeclCommand;
    /* C func for !DOCTYPE decl ends */
    XML_EndDoctypeDeclHandler        endDoctypeDeclCommand;
    /* C func for !ENTITY decls's */
    XML_EntityDeclHandler            entityDeclCommand;
} CHandlerSet;
```

Die Typen und die Argumente der Ereignisfunktionen (XML_*) sind genau dieselben wie die Handler-Funktionen der Bibliothek expat und in expat.h im Einzelnen beschrieben, siehe dort. Die Erweiterung muss nur die benötigten Handler-Funktionen bereitstellen. Ungenutzte Handler-Felder dürfen bleiben, wie sie sind. CHandlerSetCreate setzt sie auf NULL.

Der *name* dieser Struktur dient zur Kennzeichnung der Handler-Menge.

Ist die Markierung *ignoreWhiteCDATAs* gesetzt, wird Elementinhalt, der nur aus Leerraum besteht, nicht mit datacommand gemeldet.

Das Element *userData* zeigt auf die für die Handler-Menge eigenen Daten. Die Ereignis-Handler-Funktionen werden mit diesem Zeiger als Argument userData aufgerufen.

resetProc und *freeProc* müssen Argumente haben, die zum Typ passen

```
void (Tcl_Interp *interp, void *userData)
```

resetProc wird aufgerufen, wenn der Parser mit **<parserObj> reset** zurückgesetzt wird, und soll die nötige Aufräumarbeit und Neuinitialisierung tun, damit die C-Handler-Menge auf ein neues XML-Dokument vorbereitet ist. *freeProc* wird aufgerufen, wenn der Parser gelöscht wird, und soll Speicher aufräumen.

CHandlerSetCreate legt eine C-Handler-Menge an, gibt ihr den Namen *name* und setzt jedes andere Element auf NULL.

CHandlerSetInstall fügt die *handlerSet* dem Parser *expatObj* hinzu. Der Parser muss ein tDOM-Tcl-expat-Parserobjekt im Interpreter *interp* sein. Der Name der C-Handler-Menge muss für den Parser eindeutig sein. Bei Erfolg kommt 0 zurück. Ist *expatObj* im Interpreter *interp* kein Parserobjekt, kommt 1 zurück. Hat dieser Parser schon eine C-Handler-Menge mit dem Namen der handlerSet, kommt 2 zurück.

CHandlerSetRemove entfernt die über *handlerSetName* bezeichnete C-Handler-Menge vom Parser *expatObj*. Der Parser muss ein tDOM-Tcl-expat-Parserobjekt im Interpreter *interp* sein. *CHandlerSetRemove* ruft die Funktion freeProc der Struktur der C-Handler-Menge auf, entfernt die Handler-Menge aus der Liste der C-Handler-Mengen und gibt die Handler-Struktur frei. Bei Erfolg kommt 0 zurück. Ist *expatObj* im Interpreter *interp* kein Parserobjekt, kommt 1 zurück. Hat dieser Parser keine C-Handler-Menge namens *handlerSetName*, kommt 2 zurück.

CheckExpatParserObj gibt 1 zurück, wenn *nameObj* im Interpreter *interp* ein tDOM-Tcl-expat-Parserobjekt ist, sonst 0. Beispiel:

```
int
TclExampleObjCmd(dummy, interp, objc, objv)
     ClientData dummy;
     Tcl_Interp *interp;
     int objc;
     Tcl_Obj *const objv[];
{
    char          *method;
    CHandlerSet   *handlerSet;
    int            methodIndex, result;
    simpleCounter *counter;


    static char *exampleMethods[] = {
        "enable", "getresult", "remove",
        NULL
    };
    enum exampleMethod {
        m_enable, m_getresult, m_remove
    };

    if (objc != 3) {
```

```
        Tcl_WrongNumArgs (interp, 1, objv, example_usage);
        return TCL_ERROR;
    }

    if (!CheckExpatParserObj (interp, objv[1])) {
        Tcl_SetResult (interp, "First argument has to be a expat
    parser object", NULL);
        return TCL_ERROR;
    }
    /* ... */
```

CHandlerSetGet gibt einen Zeiger auf die über den Namen *handlerSetName* bezeichnete C-Handler-Menge des Parserobjekts *expatObj* zurück. *expatObj* muss ein expat-Parserobjekt im Interpreter *interp* sein. Bei einem Fehler kommt NULL zurück.

CHandlerSetGetUserData gibt einen Zeiger auf die userData der über den Namen *handlerSetName* bezeichneten C-Handler-Menge des Parserobjekts *expatObj* zurück. *expatObj* muss ein expat-Parserobjekt im Interpreter *interp* sein. Bei einem Fehler kommt NULL zurück.

GetExpatInfo ist eine Hilfsfunktion, die einen Zeiger auf die Struktur TclGenExpatInfo des tDOM-Tcl-expat-Parserobjekts *expatObj* zurückgibt. Das *expatObj* muss ein tDOM-Tcl-expat-Parserobjekt im Interpreter *interp* sein. Das ist vor allem nützlich, um den Anwendungsstatus des Parserobjekts zu setzen.

Ein einfaches, aber voll funktionsfähiges Beispiel steht im Verzeichnis extensions/example, ein umfangreicheres Beispiel tnc im Verzeichnis extensions/tnc, eine einfache Erweiterung zur DTD-Prüfung.

pullparser

Stichwörter: pullparser, XML, pull, parsing

NAME

tdom::pullparser - Einen XML-Pull-Parser-Befehl anlegen

ÜBERSICHT

package require tdom

tdom::pullparser *cmdName* ?*-ignorewhitecdata*?

BESCHREIBUNG

Dieser Befehl legt XML-Pull-Parser-Befehle mit einer einfachen API an, in der Art eines einfachen StAX-Parsers. Nach dem Anlegen muss eine Eingabequelle gesetzt werden, damit der Pull-Parser etwas Nützliches tut. Dazu dienen die Methoden *input*, *inputchannel* und *inputfile*.

Der Parser hat immer einen *state*. Das XML wird bis zu einem nächsten Zustand gelesen, die nötige Arbeit wird dort erledigt, und es geht weiter zum nächsten Zustand. Wohlgeformtheitsfehler unterwegs werden als TCL_ERROR gemeldet, mit zusätzlichen Angaben in der Fehlermeldung.

Die Pull-Parser folgen externen Entitäten nicht und kennen nur XML 1.0, von XML-Namensräumen wissen sie nichts. Tags und Attributnamen kommen wie in der Quelle.

Kommentare, Verarbeitungsanweisungen und externe Entitäten werden nicht gemeldet und still übergangen. CDATA-Abschnitte werden behandelt, als wäre ihr Inhalt ohne CDATA-Abschnitt geliefert worden.

Zeichenentitäten und Attributvorgaben in der internen Teilmenge werden beachtet, weil expat der zugrunde liegende Parser ist. Das ist vermutlich etwas schneller als eine vergleichbare Umsetzung mit der SAX-Schnittstelle. Das Programmiermodell ist angenehm, die Schnittstelle ist schlank.

Ist die Option *-ignorewhitecdata* angegeben, übergeht der angelegte XML-Pull-Parser-Befehl jeden nur aus Leerraum bestehenden Textinhalt (' ', \t, \n und \r) zwischen START_TAG und START_TAG oder END_TAG. Der Parser hält bei solcher Eingabe nicht an und legt TEXT-Zustandsereignisse nur für Text an, der nicht nur aus Leerraum besteht.

Nicht jede Methode ist in jedem Zustand gültig. Der Parser löst TCL_ERROR aus, wenn eine Methode in einem Zustand aufgerufen wird, für den sie nicht gilt. Die gültigen Methoden der angelegten Befehle sind:

state
Diese Methode ist in allen Parserzuständen gültig. Die möglichen Rückgabewerte und ihre Bedeutung:

- *READY* - Der Parser ist angelegt oder zurückgesetzt, aber es ist keine Eingabe gesetzt.
- *START_DOCUMENT* - Die Eingabe ist gesetzt, der Parser ist bereit, mit dem Lesen zu beginnen.
- *START_TAG* - Der Parser hat das Lesen an einem Start-Tag angehalten.
- *END_TAG* - Der Parser hat das Lesen an einem Ende-Tag angehalten.
- *TEXT* - Der Parser hat das Lesen angehalten, um Text zwischen Tags zu melden.
- *END_DOKUMENT* - Der Parser hat das Lesen ohne Fehler beendet.
- *PARSE_ERROR* - Der Parser hat das Lesen bei einem XML-Fehler in der Eingabe angehalten.

input *data*
Diese Methode ist nur im Zustand *READY* gültig. Sie bereitet den Parser darauf vor, *data* als zu lesende XML-Eingabe zu verwenden, und schaltet den Parser in den Zustand START_DOCUMENT.

inputchannel *channel*
Diese Methode ist nur im Zustand *READY* gültig. Sie bereitet den Parser darauf vor, die zu lesende XML-Eingabe aus *channel* zu lesen, und schaltet den Parser in den Zustand START_DOCUMENT.

inputfile *filename*
Diese Methode ist nur im Zustand *READY* gültig. Sie öffnet *filename* und bereitet den Parser darauf vor, die zu lesende XML-Eingabe aus dieser Datei zu lesen. Die Methode gibt TCL_ERROR zurück, wenn die Datei nicht im Lesemodus geöffnet werden kann. Sonst schaltet

sie den Parser in den Zustand START_DOCUMENT.

next
Diese Methode ist in den Zuständen *START_DOCUMENT*, *START_TAG*, *END_TAG* und *TEXT* gültig. Sie setzt das Lesen der XML-Eingabe bis zum nächsten Ereignis fort und gibt dieses zurück.

tag
Diese Methode ist nur in den Zuständen *START_TAG* und *END_TAG* gültig. Sie gibt den Tagnamen des aktuellen Start- oder Ende-Tags zurück.

attributes
Diese Methode ist nur im Zustand *START_TAG* gültig. Sie gibt alle Attribute des Elements in einer Liste aus Namen und Werten zurück.

text
Diese Methode ist nur im Zustand *TEXT* gültig. Sie gibt die Zeichendaten des Ereignisses zurück. Zwischen START_TAG und dem nächsten Ereignis START_TAG oder END_TAG gibt es immer höchstens ein TEXT-Ereignis.

skip
Diese Methode ist nur im Zustand *START_TAG* gültig. Sie springt zum zugehörigen Ende-Tag, übergeht unterwegs alle Ereignisse außer XML-Lesefehlern und gibt den neuen Zustand END_TAG zurück.

find-element ? tagname | -names tagnames ?
Diese Methode ist nur in den Zuständen *START_DOCUMENT*, *START_TAG* und *END_TAG* gültig. Sie springt vorwärts bis zum nächsten Element-Start-Tag mit dem Tagnamen *tagname* und gibt den neuen Zustand START_TAG zurück. Ist mit der Option *-names* eine Liste von Tagnamen angegeben, passt jeder der *tagnames*. Gibt es kein solches Element, hält der Parser am Ende der Eingabe an und gibt END_DOCUMENT zurück.

reset
Diese Methode ist in allen Parserzuständen gültig. Sie setzt den Parser in den Zustand READY zurück und gibt diesen zurück.

delete
Diese Methode ist in allen Parserzuständen gültig. Der Parserbefehl wird gelöscht. Weitere Methoden:

line
Diese Methode ist in allen Parserzuständen außer READY und TEXT gültig. Sie gibt die Zeilennummer der Leseposition zurück. Die Zeilenzählung beginnt bei 1.

column
Diese Methode ist in allen Parserzuständen außer READY und TEXT gültig. Sie gibt den Abstand der Leseposition vom Anfang der aktuellen Zeile zurück. Die Spaltenzählung beginnt bei 0.

schema

Stichwörter: schema, Validation, Postvalidation, DOM, SAX

NAME

tdom::schema - Einen Befehl zur Schema-Prüfung anlegen

ÜBERSICHT

package require tdom

tdom::schema ?*create*? *cmdName*

BESCHREIBUNG

Jeder Aufruf dieses Befehls legt einen neuen Prüfbefehl an. Ein Prüfbefehl hat Methoden, um ein Schema zu definieren, und kann XML-Daten prüfen oder einen tDOM-DOM-Baum nachträglich gegen dieses Schema prüfen, in gewissem Maß auch andere hierarchische Daten.

Ein Prüfbefehl kann außerdem als Argument der Option *-validateCmd* der Befehle *dom parse* und *expat* dienen, damit zusätzlich zu ihrer übrigen Arbeit geprüft wird.

Die Methoden der angelegten Befehle sind:

prefixns*?prefixUriList?*
: Diese Methode steuert die Zuordnung von Präfix oder Kurzform zum Namensraum-URI. Wo die Methoden des Schemabefehls ein Namensraum-Argument erwarten, kann der Präfix statt des Namensraum-URI stehen. Bildet die Liste denselben Präfix auf verschiedene Namensraum-URIs ab, gilt der erste. Gibt es den Präfix nicht, wird das Namensraum-Argument wörtlich als Namensraum-URI verwendet. Ohne Argument gibt die Methode die aktuelle prefixUriList zurück. Mit der leeren Zeichenkette werden alle Namensraum-URI-Argumente wörtlich verwendet. Das ist die Vorgabe.

defelement *name?namespace? <definition script>*
: Diese Methode definiert das Element *name* im Schema, optional im Namensraum *namespace*. Das *definition script* wird ausgewertet und definiert das Inhaltsmodell des Elements. Ist das Argument *namespace* angegeben, werden Verweise *element* oder *ref* im Definitionsskript, die nicht in einem Befehl *namespace* stehen, in diesem Namensraum aufgelöst. Gibt es für die Kombination aus Name und Namensraum schon eine Elementdefinition, löst der Befehl einen Fehler aus.

defelementtype *typename?namespace? <definition script>*
: Diese Methode definiert den Elementtyp *typename* im Schema, optional im Namensraum *namespace*. Wird der Elementtyp in einem Definitionsskript mit dem Schemabefehl element verwendet, erwartet die Prüfmaschine einen Elementinhalt nach dem Inhaltsmodell *definition script*. Elementtypen zu definieren und zu verwenden ist nur sinnvoll, wenn Elemente denselben Namen und Namensraum, aber verschiedene Inhaltsmodelle haben. Das *definition script* wird ausgewertet und definiert das Inhaltsmodell des Elements, dem es zugeordnet wird. Ist das Argument *namespace* angegeben, werden Verweise *element* oder *ref* im Definitionsskript, die nicht in einem Befehl *namespace* stehen, in diesem Namensraum aufgelöst. Gibt es für die Kombination aus Typname und Namensraum schon eine Elementtypdefinition, löst der Befehl einen Fehler aus. Das Dokumentelement eines zu prüfenden XML kann kein mit *defelementtype* definiertes Element sein.

defpattern *name?namespace? <definition script>*
: Diese Methode definiert einen, möglicherweise komplexen, Inhaltspartikel mit dem *name* im Schema, optional im Namensraum *namespace*, zur Verwendung in anderen Definitionsskripten mit dem Definitionsbefehl *ref*. Das *definition script* wird ausgewertet und definiert das Inhaltsmodell des Inhaltspartikels. Ist das Argument *namespace* angegeben, werden Verweise *element* oder *ref* im Definitionsskript, die nicht in einem Befehl *namespace* stehen, in diesem Namensraum aufgelöst. Gibt es für die Kombination aus Name und

Namensraum schon eine Musterdefinition, löst der Befehl einen Fehler aus.

deftexttype *name <constraint script>*

Diese Methode definiert ein Bündel von Texteinschränkungen, auf das sich bei Einschränkungen von Textelementen oder Attributwerten mit *name* verweisen lässt. Gibt es schon eine Texttypdefinition dieses Namens, löst der Befehl einen Fehler aus. Auf einen Texttyp darf verwiesen werden, bevor er im Schema definiert ist. Ist ein verwiesener Texttyp nirgends im Schema definiert, passt bei der Prüfung jeder Text auf diesen Typ.

start *documentElement?namespace?*

Diese Methode definiert Name und Namensraum des Wurzelelements eines zu prüfenden Baums. Wird die Methode verwendet, muss das Wurzelelement für die Gültigkeit passen. Fehlt *start*, darf jedes mit *defelement* definierte Element die Wurzel eines gültigen Dokuments sein. Die Methode *start* darf während der Lebensdauer eines Prüfbefehls mehrmals mit wechselnden Argumenten verwendet werden. Wird der Befehl nur mit der leeren Zeichenkette und ohne Namensraum-Argument aufgerufen, entfällt die Prüfeinschränkung für das Wurzelelement, und jedes definierte Element ist als Wurzel eines zu prüfenden Baums gültig.

define *<definition script>*

Diese Methode definiert mit einem Aufruf mehrere Elemente oder Muster oder ein ganzes Schema, indem sie das *definition script* auswertet. Alle bisherigen Methoden des Schemabefehls, *prefixns*, *defelement*, *defelementtype*, *defpattern*, *deftexttype* und *start*, sind auf der obersten Ebene des *definition script* erlaubt. Die Methode *define* selbst ist nicht rekursiv erlaubt.

event *(start|end|text)?event specific data?*

Diese Methode prüft hierarchische Daten gegen die Inhaltseinschränkungen des Prüfbefehls.

start *name ?attributes? ?namespace?*

Prüft, ob der aktuelle Prüfzustand hier den Beginn des Elements *name* im *namespace* erlaubt. Sonst wird ein Fehler ausgelöst.

end

Prüft, ob das aktuell innerste offene Element im aktuellen Zustand dort enden darf, ohne Prüfeinschränkungen zu verletzen. Sonst wird ein Fehler ausgelöst.

text *text*

Prüft, ob der aktuelle Prüfzustand den angegebenen Textinhalt erlaubt. Sonst wird ein Fehler ausgelöst.

validate ?*options*? *<XML string>?objVar?*

Gibt wahr zurück, wenn *<XML string>* gültig ist, sonst falsch. Ist die Prüfung fehlgeschlagen und das optionale Argument *objVar* angegeben, wird die Variable dieses Namens auf eine Prüffehler-Meldung gesetzt. Ist die XML-Zeichenkette gültig und das optionale Argument *objVar* angegeben, bleibt die Variable unberührt.

Die gültigen Optionen sind:

-baseurl *<baseURI>*

Mit *-baseurl <baseURI>* ist die baseURI die Basis-URI des Dokuments. Verweise auf externe Entitäten im Dokument werden relativ zu dieser Basis-URI aufgelöst. Diese Basis-URI wird auch im DOM-Baum gespeichert.

-externalentitycommand *<script>*

Mit *-externalentitycommand <script>* wird das angegebene Tcl-Skript aufgerufen, um externe

Entitäten des Dokuments aufzulösen. Die Vorgabe ist "::tdom::extRefHandler", ein einfacher Auflöser für Datei-URLs, definiert im Skriptteil des Pakets. Der leere Zeichenkettenwert schaltet das Auflösen externer Entitäten aus. Der tatsächlich ausgewertete Befehl besteht aus dieser Option und drei Argumenten: der Basis-URI, dem Systembezeichner der Entität und dem öffentlichen Bezeichner der Entität. Die Basis-URI und der öffentliche Bezeichner können die leere Liste sein. Das Skript gibt eine Tcl-Liste aus drei Elementen zurück. Das erste Element sagt, wie die externe Entität an den Prozessor zurückgegeben wird. Derzeit sind die Typen "string" und "channel" erlaubt. Das zweite Element muss die absolute Basis-URI der zu lesenden externen Entität sein. Das dritte Element sind die Daten: bei Typ "string" die bereits gelesenen Daten der externen Entität als Zeichenkette, bei Typ "channel" der Name eines Tcl-Kanals. Gibt das Skript einen Tcl-Kanal zurück, schließt der Prozessor ihn nicht. Er muss getrennt geschlossen werden, wenn er nicht mehr gebraucht wird.

-paramentityparsing <*always*|*never*|*notstandalone*>
Die Option *-paramentityparsing* steuert, ob der Parser beim Aufbau des DOM-Baums die externen Entitäten des Dokuments aufzulösen versucht, einschließlich der externen DTD-Teilmenge. *-paramentityparsing* braucht ein Argument, entweder "always", "never" oder "notstandalone". Der Wert "always" heißt: der Parser versucht, alle externen Entitäten der XML-Quelle aufzulösen, auch rekursiv. Das ist die Vorgabe, wenn *-paramentityparsing* fehlt. Der Wert "never" heißt: nur die angegebene XML-Quelle wird gelesen, keine externe Entität und nicht die externe Teilmenge wird aufgelöst und gelesen. Der Wert "notstandalone" heißt: alle externen Entitäten werden aufgelöst und gelesen, außer bei Dokumenten, die in der XML-Deklaration ausdrücklich standalone="yes" angeben.

-useForeignDTD <*boolean*>
Ist <boolean> wahr und hat das Dokument keine externe Teilmenge, ruft der Parser das Skript -externalentitycommand mit leeren Werten für die Argumente systemId und publicID auf. Hat das Dokument auch keine interne Teilmenge, werden die Skripte -startdoctypedeclcommand und -enddoctypedeclcommand, sofern gesetzt, nicht aufgerufen.

validatefile ?*options*? *filename?objVar?*
Gibt wahr zurück, wenn der Inhalt von *filename* gültig ist, sonst falsch. Die Datei geht als Binärstrom an expat. Deshalb arbeiten mit dieser Methode nur Daten in US-ASCII, ISO-8859-1, UTF-8 oder UTF-16. Ist die Prüfung fehlgeschlagen und das optionale Argument *objVar* angegeben, wird die Variable dieses Namens auf eine Prüffehler-Meldung gesetzt. Sind die XML-Daten gültig und das optionale Argument *objVar* angegeben, bleibt die Variable unberührt. Die erlaubten Optionen und ihre Bedeutung sind dieselben wie bei der Methode *validate*, die Beschreibung steht dort.

validatechannel ?*options*? *channel?objVar?*
Gibt wahr zurück, wenn der aus dem Tcl-Kanal *channel* gelesene Inhalt gültig ist, sonst falsch. Daten aus einem Tcl-Kanal sind UTF-8-kodiert. Eine irreführende Kodierungsdeklaration am Anfang der Daten führt deshalb zu Fehlern. Schlägt die Prüfung fehl und ist das optionale Argument *objVar* angegeben, wird die Variable dieses Namens auf eine Prüffehler-Meldung gesetzt. Sind die XML-Daten gültig und das optionale Argument *objVar* angegeben, bleibt die Variable unberührt. Die erlaubten Optionen und ihre Bedeutung sind dieselben wie bei der Methode *validate*, die Beschreibung steht dort.

domvalidate *domNode?objVar?*
Gibt wahr zurück, wenn das erste Argument ein gültiger Baum ist, sonst falsch. Ist die Prüfung

fehlgeschlagen und das optionale Argument *objVar* angegeben, wird die Variable dieses Namens auf eine Prüffehler-Meldung gesetzt. Ist der DOM-Baum gültig und das optionale Argument *objVar* angegeben, wird die Variable dieses Namens auf die leere Zeichenkette gesetzt.

reportcmd*?cmd?*
Diese Methode erwartet den Namen eines Tcl-Befehls, der bei einem Prüffehler aufgerufen wird. Der Befehl wird mit zwei angehängten Argumenten aufgerufen: dem Schemabefehl, der den Prüffehler auslöst, und einem Prüffehlercode.

Die möglichen Fehlercodes sind:

MISSING_ELEMENT
... siehe den folgenden Eintrag ...

MISSING_TEXT
... siehe den folgenden Eintrag ...

UNEXPECTED_ELEMENT
... siehe den folgenden Eintrag ...

UNEXPECTED_ROOT_ELEMENT
... siehe den folgenden Eintrag ...

UNEXPECTED_TEXT
... siehe den folgenden Eintrag ...

UNKNOWN_ROOT_ELEMENT
... siehe den folgenden Eintrag ...

UNKNOWN_ATTRIBUTE
... siehe den folgenden Eintrag ...

MISSING_ATTRIBUTE
... siehe den folgenden Eintrag ...

INVALID_ATTRIBUTE_VALUE
... siehe den folgenden Eintrag ...

DOM_KEYCONSTRAINT
... siehe den folgenden Eintrag ...

DOM_XPATH_BOOLEAN
... siehe den folgenden Eintrag ...

INVALID_KEYREF
... siehe den folgenden Eintrag ...

INVALID_VALUE
... siehe den folgenden Eintrag ...

UNKNOWN_GLOBAL_ID
... siehe den folgenden Eintrag ...

UNKNOWN_ID
Genaueres steht im Abschnitt Wiederaufnahme.

delete
Diese Methode löscht den Prüfbefehl.

info*?args?*
Diese Methode bündelt Methoden, die den Zustand und Einzelheiten des Schemabefehls

abfragen.

validationstate
Diese Methode gibt den Zustand des Prüfbefehls hinsichtlich der Prüfung zurück. Die möglichen Rückgabewerte und ihre Bedeutung:

READY
Der Prüfbefehl ist bereit, mit der Prüfung zu beginnen.

VALIDATING
Der Prüfbefehl prüft gerade die Eingabe.

FINISHED
Die Prüfung ist beendet, weitere Ereignisse werden nicht erwartet.

vstate
Diese Methode ist ein kürzerer Alias für validationstate, siehe dort.

line
Prüft der Schemabefehl gerade, gibt diese Methode den Zeilenanteil der Leseposition zurück, sonst die leere Zeichenkette. Prüft der Schemabefehl gerade einen DOM-Baum nach, kann an manchen oder allen Knoten keine Positionsangabe gespeichert sein. In diesen Fällen kommt die leere Zeichenkette zurück.

column
Prüft der Schemabefehl gerade, gibt diese Methode den Spaltenanteil der Leseposition zurück, sonst die leere Zeichenkette. Prüft der Schemabefehl gerade einen DOM-Baum nach, kann an manchen oder allen Knoten keine Positionsangabe gespeichert sein. In diesen Fällen kommt die leere Zeichenkette zurück.

byteIndex
Prüft der Schemabefehl gerade, gibt diese Methode die Byteposition der Leseposition zurück, sonst die leere Zeichenkette. Prüft der Schemabefehl gerade einen DOM-Baum nach, kann an manchen oder allen Knoten keine Positionsangabe gespeichert sein. In diesen Fällen kommt die leere Zeichenkette zurück.

domNode
Prüft der Schemabefehl gerade keinen DOM-Baum nach, gibt diese Methode die leere Zeichenkette zurück. Wartet der Schemabefehl sonst, während er sich von einem Prüffehler erholt, auf das Ende des Skripts reportcmd, kommt der Knoten zurück, den die Prüfmaschine gerade ansieht, wenn der Knoten ein ELEMENT_NODE ist, sonst sein Elternknoten. Von dieser Methode ist abzuraten. Wird sie doch verwendet, bleibt der DOM-Baum unangetastet und wird nur gelesen.

nrForwardDefinitions
Gibt zurück, wie viele Elemente, Elementtypen und ref-Muster verwiesen sind, die noch nicht definiert sind, als Summe.

definedElements
Gibt die definierten Elemente der Grammatik als Liste zurück, ohne bestimmte Reihenfolge. Hat ein Element einen Namensraum, ist sein Listeneintrag selbst eine Liste aus zwei Elementen, zuerst der Name, dann der Namensraum.

definedElementtypes
Gibt die definierten Elementtypen der Grammatik als Liste zurück, ohne bestimmte Reihenfolge. Hat ein Elementtyp einen Namensraum, ist sein Listeneintrag selbst eine Liste

aus zwei Elementen, zuerst der Name, dann der Namensraum.

definedPatterns
Gibt die definierten benannten Muster der Grammatik als Liste zurück, ohne bestimmte Reihenfolge. Hat ein benanntes Muster einen Namensraum, ist sein Listeneintrag selbst eine Liste aus zwei Elementen, zuerst der Name, dann der Namensraum.

expected
Gibt alle möglichen nächsten Ereignisse als Liste zurück, ohne bestimmte Reihenfolge, seit der letzten erfolgreichen Ereignisübereinstimmung, falls es eine gab. Hat ein Element einen Namensraum, ist sein Listeneintrag selbst eine Liste aus zwei Elementen, zuerst der Name, dann der Namensraum. Ist Text ein mögliches nächstes Ereignis, ist der Listeneintrag eine Liste aus zwei Elementen, zuerst #text und dann die leere Zeichenkette. Ist eine any-Element-Einschränkung möglich, ist der Listeneintrag eine Liste aus zwei Elementen, zuerst <any> und dann die leere Zeichenkette. Ist eine any-Element-Einschränkung in einem bestimmten Namensraum möglich, ist der Listeneintrag eine Liste aus zwei Elementen, zuerst <any> und dann der Namensraum. Ist das Elementende ein mögliches Ereignis, ist der Listeneintrag eine Liste aus zwei Elementen, zuerst <elementend> und dann die leere Zeichenkette.

definition name ?namespace?
Gibt den Code zurück, der das angegebene Element definiert. Der Befehl löst einen Fehler aus, wenn es keine Definition dieses Elements gibt.

typedefinition name ?namespace?
Gibt den Code zurück, der die angegebene Elementtypdefinition definiert. Der Befehl löst einen Fehler aus, wenn es keine Definition dieses Elements gibt.

patterndefinition name ?namespace?
Gibt den Code zurück, der die angegebene Musterdefinition definiert. Der Befehl löst einen Fehler aus, wenn es keine Definition eines Musters mit diesem Namen und, falls angegeben, diesem Namensraum gibt.

vaction ?name|namespace|text?
Diese Methode gibt nur dann nützliche Angaben zurück, wenn der Schemabefehl auf das Ende des Skripts reportcmd wartet, während er sich von einem Prüffehler erholt. Sonst gibt sie NONE zurück.

Ohne das optionale Argument sind die möglichen Rückgabewerte und ihre Bedeutung:

NONE
Der Schemabefehl erholt sich gerade nicht von einem Prüfereignis.

MATCH_ELEMENT_START
Ereignis Elementanfang, einschließlich der Suche nach fehlenden oder unbekannten Attributen.

MATCH_ELEMENT_END
Ereignis Elementende.

MATCH_TEXT
Prüfung von Text zwischen Tags.

MATCH_ATTRIBUTE_TEXT
Prüfung der Werteinschränkung eines Attributtexts.

MATCH_GLOBAL

Prüfung globaler IDs.

MATCH_DOM_KEYCONSTRAINT
Prüfung der Einschränkung domunique.

MATCH_DOM_XPATH_BOOLEAN
Prüfung der Konstante domxpathboolean. Mit einem der möglichen optionalen Argumente gibt der Befehl je nach aktueller Aktion Einzelheiten zurück.

name
Gibt bei MATCH_ELEMENT_START den Namen des Elements zurück, das passen muss. Gibt bei MATCH_ELEMENT_END den Namen des geschlossenen Elements zurück. Gibt bei MATCH_ATTRIBUTE_TEXT den Namen des Attributs zurück. Gibt bei MATCH_TEXT den Namen des Elternelements zurück.

namespace
Gibt bei MATCH_ELEMENT_START den Namensraum des Elements zurück, das passen muss. Gibt bei MATCH_ELEMENT_END den Namensraum des geschlossenen Elements zurück. Gibt bei MATCH_ATTRIBUTE_TEXT den Namensraum des Attributs zurück. Gibt bei MATCH_TEXT den Namensraum des Elternelements zurück.

text
Gibt bei MATCH_TEXT den Text zurück, der passen muss. Gibt bei MATCH_ATTRIBUTE_TEXT den Wert des Attributs zurück.

stack top|inside|associated
In Tcl-Skripten, die die Prüfung auswertet, gibt diese Methode Angaben zum aktuellen Prüfstapel. Außerhalb dieses Zusammenhangs gibt die Methode die leere Zeichenkette zurück.

top
Gibt das Element zurück, dessen Inhalt gerade geprüft wird, also das in diesem Augenblick offene Element-Tag.

inside
Gibt alle gerade offenen Elemente als Liste zurück.

associated
Gibt die Daten zurück, die dem aktuell obersten Inhaltspartikel des Stapels zugeordnet sind, oder die leere Zeichenkette, wenn es keine gibt.

reset
Diese Methode setzt den Prüfbefehl in den Zustand READY zurück und behält die definierte Grammatik.

SCHEMA-DEFINITIONSSKRIPTE

Schema-Definitionsskripte sind gewöhnliche Tcl-Skripte, die im Namensraum tdom::schema ausgewertet werden. Die unten aufgeführten Befehle der Schemadefinition in diesem Tcl-Namensraum erlauben die Definition sehr verschiedener Dokumentstrukturen. Jeder Befehl der Schemadefinition stellt eine Prüfeinschränkung an den Inhalt, der passen muss oder optional sein muss, damit der Inhalt als gültig gilt. Weiterer, nicht passender Inhalt ist ein Prüffehler. Nur aus Leerraum bestehender Text, im XML-Sinn von Leerraum, zwischen verschiedenen Tags wird übergangen, außer bei reinen Textelementen, bei denen auch nur aus Leerraum bestehender Text als bedeutsamer Inhalt gilt.

Die Befehle der Schemadefinition sind:

element *name?quant? (?<definition script>|*(lqtype*(lq typename)?*
: Ist weder das optionale Argument *definition script* noch die Zeichenkette "type" und ein *typename* angegeben, verweist dieser Befehl auf das mit *defelement* definierte Element mit dem Namen *name* im aktuellen Kontextnamensraum.

Sind die Zeichenkette "type" und ein *typename* angegeben, beschreibt das mit *defelementtype* definierte Inhaltsmodell mit dem Namen *typename* im aktuellen Kontextnamensraum den Inhalt des Elements.

Ist das Argument *defelement script* angegeben, erwartet die Prüfeinschränkung ein Element mit dem Namen *name* im aktuellen Namensraum, dessen Inhalt das *definition script* örtlich definiert. Vorwärtsverweise auf noch nicht definierte Elemente oder Muster oder andere örtliche Definitionen desselben Namens innerhalb des *definition script* sind erlaubt. Ist ein vorwärts verwiesenes Element bis zur Prüfung nicht definiert, passt nur ein leeres Element mit dem Namen *name*, dem Namensraum *namespace* und ohne Attribute.

ref *name?quant?*
: Dieser Befehl verweist auf den mit *defpattern* definierten Inhaltspartikel mit dem Namen *name* im aktuellen Kontextnamensraum. Vorwärtsverweise auf ein noch nicht definiertes Muster und rekursive Verweise sind erlaubt. Ist ein vorwärts verwiesenes Muster bis zur Prüfung nicht definiert, wird keinerlei Inhalt erwartet, ein leerer Treffer.

group*?quant? <definition script>*
: Diese Methode gruppiert eine Folge von Inhaltspartikeln, die das *definition script* definiert und die in dieser Reihenfolge passen müssen.

choice*?quant? <definition script>*
: Diese Schemaeinschränkung passt, wenn einer der obersten Inhaltspartikel passt, die das *definition script* definiert. Ist einer dieser obersten Inhaltspartikel optional, passt diese Einschränkung auf den leeren Treffer.

interleave*?quant? <definition script>*
: Diese Schemaeinschränkung passt, nachdem jeder der erforderlichen obersten Inhaltspartikel, die das *definition script* definiert, gepasst hat, und optional einige oder alle übrigen, in beliebiger Reihenfolge.

mixed*?quant? <definition script>*
: Diese Schemaeinschränkung passt auf jeden Text, auch den leeren, und auf jeden obersten Inhaltspartikel, den das *definition script* definiert, mit dem Vorgabequantor *.

text*?<constraint script>|*(lqtype*(lq typename?*
: Ohne das optionale Einschränkungsskript passt diese Prüfeinschränkung auf jede Zeichenkette, auch die leere. Mit *constraint script* oder mit einem angegebenen Texttyp-Argument wird ein Text erwartet, der auf dieses Skript oder den Texttyp passt.

any ?*options*??<namespace list>??quant?*
: Ohne Argumente passt der Befehl any auf jedes Element. Ist das Argument <*namespace list*> angegeben, passt dies auf jedes Element in einem Namensraum aus dieser Liste. Die leere Zeichenkette meint Elemente ohne Namensraum. Ist zusätzlich die Option *-not* angegeben, passt dies auf jedes Element mit einem Namensraum, der nicht in der Liste steht. Die einzige weitere erkannte Option ist --, sie markiert das Ende aller Optionen. Fehlt das Argument *namespace*, verbrauchen die Quantoren * und + alle Elemente, bis das umschließende

Element endet. Sieht ein Namensraum wie ein gültiger tDOM-Schema-Quantor aus, müssen immer beide Argumente ausgeschrieben werden.

attribute *name?quant? (?<constraint script>|*(lqtype*(lq typename?)*

Der Befehl attribute definiert ein Attribut ohne Namensraum am umschließenden Element. Die erste Definition von *name* innerhalb einer Elementdefinition gilt, spätere Definitionen desselben Namens werden still übergangen. Nach dem Argument *name* kann einer der Quantoren ? oder! stehen. Steht er dort, wird er verwendet. Sonst ist das Attribut erforderlich und muss in der XML-Quelle vorhanden sein. Gibt es ein Argument mehr, wird dieses Argument als Einschränkungsskript ausgewertet und definiert die Werteinschränkungen des Attributs. Gibt es sonst zwei Argumente mehr und ist das erste das bloße Wort "type", wird das folgende Argument als Name eines Texttyps verwendet. Dieser Befehl ist nur auf der obersten Ebene im Definitionsskript eines defelement- oder element-Skripts erlaubt.

nsattribute *name namespace?quant? (?<constraint script>|*(lqtype*(lq typename?)*

Dieser Befehl tut dasselbe wie der Befehl *attribute*, für das Attribut *name* im Namensraum *namespace*.

namespace *URI <definition script>*

Wertet das *definition script* mit dem Kontextnamensraum *URI* aus. Jeder Name eines Befehls element, eines Elementtyps oder von ref wird im Namensraum *URI* nachgeschlagen, und örtlich definierte Elemente liegen in diesem Namensraum. Die leere Zeichenkette als *URI* meint keinen Namensraum.

tcl *tclcmd?arg arg ...?*

Wertet das Tcl-Skript *tclcmd arg arg ...* aus. Dieser Prüfbefehl ist nur in streng sequenziellem Zusammenhang erlaubt, nicht in choice, mixed und interleave. Ist der Rückgabecode etwas anderes als TCL_OK, ist das ein Fehler, den reportcmd nicht abfängt und nicht meldet.

self

Gibt den Schemabefehl zurück.

associate *data*

Dieser Befehl ist nur auf der obersten Ebene innerhalb von Definitionsskripten der Inhaltspartikel element, elementtype, pattern oder interleave erlaubt. Er ordnet die als Argument angegebenen *data* dem gerade definierten Inhaltspartikel zu. In Skripten, die beim Prüfen des Inhalts dieses Partikels ausgewertet werden, lassen sie sich mit dem Methodenaufruf *info stack associated* des Schemabefehls abfragen.

domunique *selector fieldlist?name??*(lqIGNORE_EMPTY_FIELD_SET*(lq|(\ *(lqEMPTY_FIELD_SET_VALUE*(lq emptyFieldSetValue)?*

Wird ein DOM-Baum nicht mit *domvalidate* nachträglich geprüft, passt diese Einschränkung immer. Beim Nachprüfen entspricht sie dem Mechanismus xsd key/keyref. Das Argument *selector* darf jeder gültige XPath-Ausdruck sein, ohne die Grenzen von xsd. Mehrere Befehle *domunique* innerhalb einer Elementdefinition sind erlaubt und werden in der Definitionsreihenfolge geprüft. Der Argumentname steht im Wiederaufnahme-Skript über *info vaction name* zur Verfügung. Wählt die *fieldlist* für einen Knoten der Ergebnismenge des *selector* nichts aus, ist der Schlüsselwert in der Vorgabe die leere Zeichenkette. Sind die Argumente *EMPTY_FIELD_SET_VALUE <value>* angegeben, hat eine leere Knotenmenge den Schlüsselwert *value*. Ist stattdessen die Markierung *IGNORE_EMPTY_FIELD_SET* angegeben, hat das Ergebnis einer leeren Knotenmenge keinen Schlüsselwert.

domxpathboolean *XPath_expr?name?*

Wird ein DOM-Baum nicht mit *domvalidate* nachträglich geprüft, passt diese Einschränkung immer. Beim Nachprüfen wird das Argument *XPath_expr* ausgewertet, mit dem Knoten als Kontextknoten, der zum Schema-Elternteil des Befehls *domxpathboolean* passt. Die Einschränkung passt, wenn das Ergebnis dieses XPath-Ausdrucks nach den XPath-Regeln in einen booleschen Wert umgewandelt wahr ist. Mehrere Befehle *domxpathboolean* innerhalb einer Elementdefinition sind erlaubt und werden in der Definitionsreihenfolge geprüft.

Das ermöglicht Prüfungen, die von mehr als einem Element abhängen. Beispiel:

```
tdom::schema s
s define {
    defelement doc {
        element a ! text
        element b ! text
        element c ! text
        domxpathboolean "a * b * c >= 20000" volume
        domxpathboolean "a > b and b > c" sequence
    }
}
```

jsontype *JSON structure type*
Wird ein DOM-Baum nicht mit *domvalidate* nachträglich geprüft, passt diese Einschränkung immer. Beim Nachprüfen passt die Einschränkung, wenn das umschließende Element den JSON-Typ hat, der der Struktureinschränkung als Argument angegeben ist. Die möglichen JSON-Strukturtypen sind *NONE*, *OBJECT* und *ARRAY*. Diese Einschränkung ist nur als direktes Kind einer defelement-, defelementtype- oder örtlichen Elementdefinition erlaubt.

prefixns*?prefixUriList?*
Dies definiert eine Zuordnung von Präfix zum Namensraum-URI genau so, wie es *schemacmd prefixns* täte. Gedacht ist es als Befehl der obersten Ebene eines Skripts *schemacmd define*. Dieser Befehl darf nicht in einem anderen Befehl eines Definitionsskripts verschachtelt sein und löst einen Fehler aus, wenn der Aufruf dort steht.

defelement *name?namespace? <definition script>*
Dies definiert ein Element genau so, wie es ein Aufruf *schemacmd defelement* täte. Gedacht ist es als Befehl der obersten Ebene eines Skripts *schemacmd define*. Dieser Befehl darf nicht in einem anderen Befehl eines Definitionsskripts verschachtelt sein und löst einen Fehler aus, wenn der Aufruf dort steht.

defelementtype *typename?namespace? <definition script>*
Dies definiert einen Elementtyp genau so, wie es ein Aufruf *schemacmd defelementtype* täte. Gedacht ist es als Befehl der obersten Ebene eines Skripts *schemacmd define*. Dieser Befehl darf nicht in einem anderen Befehl eines Definitionsskripts verschachtelt sein und löst einen Fehler aus, wenn der Aufruf dort steht.

defpattern *name?namespace? <definition script>*
Dies definiert ein benanntes Muster genau so, wie es ein Aufruf *schemacmd defpattern* täte. Gedacht ist es als Befehl der obersten Ebene eines Skripts *schemacmd define*. Dieser Befehl darf nicht in einem anderen Befehl eines Definitionsskripts verschachtelt sein und löst einen Fehler aus, wenn der Aufruf dort steht.

deftexttype *name <constraint script>*

Dies definiert ein benanntes Bündel von Texteinschränkungen genau so, wie es ein Aufruf *schemacmd deftexttype* täte. Gedacht ist es als Befehl der obersten Ebene eines Skripts *schemacmd define*. Dieser Befehl darf nicht in einem anderen Befehl eines Definitionsskripts verschachtelt sein und löst einen Fehler aus, wenn der Aufruf dort steht.

start *name?namespace?*

Dieser Befehl arbeitet genau so, wie es ein Aufruf *schemacmd start* täte. Gedacht ist er als Befehl der obersten Ebene eines Skripts *schemacmd define*. Dieser Befehl darf nicht in einem anderen Befehl eines Definitionsskripts verschachtelt sein und löst einen Fehler aus, wenn der Aufruf dort steht.

QUANTOR

Mehrere Befehle der Schemadefinition erwarten einen Quantor als eines ihrer Argumente. Er bestimmt, wie oft der vom Befehl angegebene Inhaltspartikel erwartet wird. Die gültigen Werte für ein Argument *quant* sind:

!

Der Inhaltspartikel muss in gültigen Dokumenten genau einmal vorkommen.

?

Der Inhaltspartikel darf in gültigen Dokumenten nicht öfter als einmal vorkommen, der Partikel ist optional.

*****

Der Inhaltspartikel darf in gültigen Dokumenten nullmal oder öfter hintereinander vorkommen.

+

Der Inhaltspartikel darf in gültigen Dokumenten einmal oder öfter hintereinander vorkommen.

n

Der Inhaltspartikel muss in gültigen Dokumenten n-mal hintereinander vorkommen. Der Quantor muss eine ganze Zahl größer als null sein.

{n m}

Der Inhaltspartikel muss in gültigen Dokumenten mindestens n-mal und höchstens m-mal hintereinander vorkommen. Der Quantor muss eine Tcl-Liste mit zwei Elementen sein. Das erste Element dieser Liste muss eine ganze Zahl mit n >= 0 sein. Ist das zweite Listenelement das Zeichen *, gibt es keine obere Grenze. Sonst muss das zweite Listenelement eine ganze Zahl mit n < m sein. Fehlt ein optionaler Quantor, ist die Vorgabe * beim Befehl *mixed* und! bei allen anderen Befehlen.

TEXT-EINSCHRÄNKUNGSSKRIPTE

Text, von XML parsed character data genannt, muss manchmal von einer bestimmten Art sein oder bestimmten Regeln folgen, um gültig zu sein. Die Argumente des Texteinschränkungsskripts der Befehle text, attribute, nsattribute und deftexttype werden im Tcl-Namensraum *tdom::schema::text* ausgewertet und erlauben den folgenden Befehlen der Texteinschränkung, Text auf bestimmte Eigenschaften zu prüfen. Die Befehle sind im Tcl-Namensraum *tdom::schema::text* definiert. Ein Aufruf außerhalb eines Texteinschränkungsskripts löst einen Fehler aus.

Einige der folgenden Texttyp-Befehle stehen als allgemeine Tcl-Befehle zur Verfügung. Definiert sind sie im Namensraum tdom::type und werden wie unten beschrieben aufgerufen, mit dem zu

prüfenden Text an die Argumentliste angehängt. Der Rückgabewert ist logisch. Die Befehle müssen führenden oder abschließenden Leerraum nicht annehmen. Ob ein Befehl im Namensraum tdom::type zur Verfügung steht, steht in seiner Beschreibung.

Der Tcl-Text-Einschränkungsbefehl

Der Texteinschränkungsbefehl *tcl* gibt die Prüfung an einen beliebigen Tcl-Befehl weiter und ermöglicht so beliebige programmierbare Entscheidungsregeln.

tcl *tclcmd?arg arg ...?*

Wertet das Tcl-Skript *tclcmd arg arg ...* aus, mit dem zu prüfenden Text an die Argumentliste angehängt. Der Rückgabewert des Tcl-Befehls wird als boolescher Wert gelesen.

Grundlegende XML-Typen

name

<URL: https://www.w3.org/TR/xml/#NT-Name> "" Diese Texteinschränkung passt, wenn der Textwert auf die XML-Produktion name passt. Der Textwert muss also mit einem Buchstaben, einem Unterstrich (_) oder einem Doppelpunkt (:) beginnen und darf nur Buchstaben, Ziffern, Unterstriche (_), Doppelpunkte (:), Bindestriche (-) und Punkte (.) enthalten.

ncname

<URL: https://www.w3.org/TR/xml-names/#NT-NCName> "" Diese Texteinschränkung passt, wenn der Textwert auf die XML-Produktion ncname passt. Der Textwert muss also mit einem Buchstaben oder einem Unterstrich (_) beginnen und darf nur Buchstaben, Ziffern, Unterstriche (_), Bindestriche (-) und Punkte (.) enthalten. Der einzige Unterschied zur Einschränkung name ist, dass Doppelpunkte nicht erlaubt sind.

qname

<URL: https://www.w3.org/TR/xml-names/#NT-QName> "" Diese Texteinschränkung passt, wenn der Textwert auf die XML-Produktion qname passt. Der Textwert ist also entweder ein ncname oder zwei durch einen Doppelpunkt (:) verbundene ncnames.

nmtoken

<URL: https://www.w3.org/TR/xml/#NT-Nmtoken> "" Diese Texteinschränkung passt, wenn der Textwert auf die XML-Produktion nmtoken passt

nmtokens

<URL: https://www.w3.org/TR/xml/#NT-Nmtokens> "" Diese Texteinschränkung passt, wenn der Textwert auf die XML-Produktion nmtokens passt

Grundlegende Typtests

integer*?(xsd|tcl)?*

Diese Texteinschränkung passt, wenn der Textwert als ganze Zahl gelesen werden könnte. Ist das optionale Argument des Befehls *tcl*, passt alles, was TCL_OK zurückgibt, wenn es an Tcl_GetInt() geht. Ist das optionale Argument des Befehls *xsd*, passt die Einschränkung, wenn der Wert ein gültiges xsd:integer ist. Ohne Argument ist *xsd* die Vorgabe.

negativeInteger*?(xsd|tcl)?*

Diese Texteinschränkung passt auf dieselben Textwerte wie die Texteinschränkung *integer*, siehe dort, mit der zusätzlichen Einschränkung, dass der Wert < null sein muss.

nonNegativeInteger*?(xsd|tcl)?*

Diese Texteinschränkung passt auf dieselben Textwerte wie die Texteinschränkung *integer*, siehe dort, mit der zusätzlichen Einschränkung, dass der Wert >= null sein muss.

nonPositiveInteger*?(xsd|tcl)?*
Diese Texteinschränkung passt auf dieselben Textwerte wie die Texteinschränkung *integer*, siehe dort, mit der zusätzlichen Einschränkung, dass der Wert <= null sein muss.

positiveInteger*?(xsd|tcl)?*
Diese Texteinschränkung passt auf dieselben Textwerte wie die Texteinschränkung *integer*, siehe dort, mit der zusätzlichen Einschränkung, dass der Wert > null sein muss.

number*?(xsd|tcl)?*
Diese Texteinschränkung passt, wenn der Textwert als Zahl gelesen werden könnte. Ist das optionale Argument des Befehls *tcl*, passt alles, was TCL_OK zurückgibt, wenn es an Tcl_GetDouble() geht. Ist das optionale Argument des Befehls *xsd*, passt die Einschränkung, wenn der Wert ein gültiges xsd:decimal ist. Ohne Argument ist *xsd* die Vorgabe.

boolean*?(xsd|tcl)?*
Diese Texteinschränkung passt, wenn der Textwert als boolescher Wert gelesen werden könnte. Ist das optionale Argument des Befehls *tcl*, passt alles, was TCL_OK zurückgibt, wenn es an Tcl_GetBoolean() geht. Ist das optionale Argument des Befehls *xsd*, passt die Einschränkung, wenn der Wert ein gültiges xsd:boolean ist. Ohne Argument ist *xsd* die Vorgabe.

date
Diese Texteinschränkung passt, wenn der Textwert ein xsd:date ist, im Wesentlichen ein ISO-8601-Datum der Form YYYY-MM-DD, mit optionalem Zeitzonenanteil, entweder dem Buchstaben Z oder Plus (+) oder Minus (-), gefolgt von hh:mm, und mit der höchsten erlaubten positiven oder negativen Zeitzone 14:00. Für alle Daten gelten die Datumsregeln des gregorianischen Kalenders. Ein vorangestelltes Minuszeichen für Daten v. Chr. ist erlaubt. Ein Jahr 0 gibt es nicht. Das Jahr darf mehr als 4 Stellen haben, aber nur wenn nötig, ohne zusätzliche führende Nullen. Verfügbar als allgemeiner Tcl-Befehl tdom::type::date.

time
Diese Texteinschränkung passt, wenn der Textwert ein xsd:time ist, im Wesentlichen eine ISO-8601-Uhrzeit der Form hh:mm:ss mit optionalem Zeitzonenanteil. Der Zeitzonenanteil folgt den Regeln des Befehls *date*, siehe dort. Alle drei Teile des Zeitwerts, Stunden, Minuten und Sekunden, müssen mit 2 Stellen ausgeschrieben sein. Zusätzliche Sekundenbruchteile mit einem Punkt ('.') als Trenner sind erlaubt, nicht aber ein alleinstehender Punkt. Der Zeitwert 24:00:00 ohne Bruchteil ist erlaubt. Verfügbar als allgemeiner Tcl-Befehl tdom::type::time.

dateTime
Diese Texteinschränkung passt, wenn der Textwert ein xsd:dateTime ist, im Wesentlichen eine ISO-8601-Datumszeit der Form YYYY-MM-DDThh:mm:ss mit optionalem Zeitzonenanteil. Die Anteile Datum und Zeitzone folgen den Regeln der Befehle *date* und *time*, siehe dort. Der Zeitanteil einschließlich des kennzeichnenden Zeichens 'T' ist erforderlich. Verfügbar als allgemeiner Tcl-Befehl tdom::type::dateTime.

duration
Diese Texteinschränkung passt, wenn der Textwert eine xsd:duration ist, im Wesentlichen eine ISO-8601-Dauer der Form PnYnMnDTnHnMnS. Alle Teile außer dem beginnenden P und, wenn eines von H, M oder S angegeben ist, T sind optional. Ist der folgende Kennbuchstabe S, darf n eine Dezimalzahl sein, mit mindestens einer Ziffer vor und nach dem Punkt, sonst muss es eine positive ganze Zahl sein. Verfügbar als allgemeiner Tcl-Befehl tdom::type::duration.

base64
Diese Texteinschränkung passt, wenn der Text nach RFC 4648 gültig ist. Verfügbar als allgemeiner Tcl-Befehl tdom::type::base64.

hexBinary
Diese Texteinschränkung passt, wenn der Text eine Folge binärer Oktette in hexadezimaler Kodierung ist, wobei jedes binäre Oktett eine zweistellige Hexadezimalzahl ist. Klein- und Großbuchstaben A bis F sind erlaubt.

unsignedByte
Diese Texteinschränkung passt, wenn der Textwert ein xsd:unsignedByte ist. Das ist eine ganze Zahl zwischen 0 und 255, beide eingeschlossen, optional mit vorangestelltem Pluszeichen und führenden Nullen.

unsignedShort
Diese Texteinschränkung passt, wenn der Textwert ein xsd:unsignedShort ist. Das ist eine ganze Zahl zwischen 0 und 65535, beide eingeschlossen, optional mit vorangestelltem Pluszeichen und führenden Nullen.

unsignedInt
Diese Texteinschränkung passt, wenn der Textwert ein xsd:unsignedInt ist. Das ist eine ganze Zahl zwischen 0 und 4294967295, beide eingeschlossen, optional mit vorangestelltem Pluszeichen und führenden Nullen.

unsignedLong
Diese Texteinschränkung passt, wenn der Textwert ein xsd:unsignedLong ist. Das ist eine ganze Zahl zwischen 0 und 18446744073709551615, beide eingeschlossen, optional mit vorangestelltem Pluszeichen und führenden Nullen.

byte
Diese Texteinschränkung passt, wenn der Textwert ein xsd:byte ist. Das ist eine ganze Zahl zwischen -128 und 127, beide eingeschlossen, optional mit vorangestelltem Plus- oder Minuszeichen und führenden Nullen.

short
Diese Texteinschränkung passt, wenn der Textwert ein xsd:short ist. Das ist eine ganze Zahl zwischen -32768 und 32767, beide eingeschlossen, optional mit vorangestelltem Plus- oder Minuszeichen und führenden Nullen.

int
Diese Texteinschränkung passt, wenn der Textwert ein xsd:int ist. Das ist eine ganze Zahl zwischen -2147483648 und 2147483647, beide eingeschlossen, optional mit vorangestelltem Plus- oder Minuszeichen und führenden Nullen.

long
Diese Texteinschränkung passt, wenn der Textwert ein xsd:long ist. Das ist eine ganze Zahl zwischen -9223372036854775808 und 9223372036854775807, beide eingeschlossen, optional mit vorangestelltem Plus- oder Minuszeichen und führenden Nullen.

Logische Verknüpfungen

oneOf *<constraint script>*
Diese Texteinschränkung passt, wenn eine der im Argument *constraint script* definierten Texteinschränkungen auf den Text passt. Die Prüfung hält nach der ersten Übereinstimmung an und prüft die Texteinschränkungen in der Definitionsreihenfolge.

allOf *<constraint script>*
Diese Texteinschränkung passt, wenn alle im Argument *constraint script* definierten Texteinschränkungen auf den Text passen. Die Prüfung hält nach dem ersten Fehlschlag an und prüft die Texteinschränkungen in der Definitionsreihenfolge. Da auch der Schemadefinitionsbefehl *text* erwartet, dass alle Texteinschränkungen auf die Texteinschränkung passen, ist *allOf* vor allem zusammen mit dem Texteinschränkungsbefehl *oneOf* nützlich.

not *<constraint script>*
Diese Texteinschränkung passt, wenn keine der im Argument *constraint script* definierten Texteinschränkungen auf den Text passt. Die Prüfung hält nach der ersten passenden Einschränkung im *constraint script* an und meldet einen Prüffehler. Die Texteinschränkungen im *constraint script* werden in der Definitionsreihenfolge geprüft.

type *text type name*
Diese Texteinschränkung passt, wenn der als Argument angegebene Texttyp passt.

Einschränkungen des verarbeiteten Textwerts

whitespace *(preserve|replace|collapse) <constraint script>*
Dieser Texteinschränkungsbefehl normalisiert den Leerraum des Textwerts, #x20 (Leerzeichen, ' '), #x9 (Tabulator, \t), #xA (Zeilenvorschub, \n) und #xD (Wagenrücklauf, \r), und prüft den entstandenen Text mit den Texteinschränkungen des Arguments des Einschränkungsskripts. Die Normalisierungsmethode *preserve* lässt alles, wie es ist; das ist eine andere Art, *allOf* zu sagen. Die Normalisierungsmethode *replace* ersetzt jedes einzelne Leerraumzeichen, wie oben, durch ein Leerzeichen. Die Normalisierungsmethode *collapse* entfernt allen führenden und abschließenden Leerraum, und alle übrigen Folgen zusammenhängenden Leerraums werden durch ein einzelnes Leerzeichen ersetzt.

split*?type ?args??<constraint script>*
Dieser Texteinschränkungsbefehl zerlegt den zu prüfenden Text in eine Liste von Werten und prüft alle Elemente dieser Liste auf die Texteinschränkungen im ausgewerteten *constraint script*.

Die verfügbaren Typen sind:

whitespace
Der zu zerlegende Text wird am Anfang und am Ende von allem Leerraum befreit und an jedem aufeinanderfolgenden Leerraum in eine Liste zerlegt.

tcl tclcmd ?arg ...?
Der zu zerlegende Text geht an *tclcmd*, das auf globaler Ebene ausgewertet wird, mit jedem angegebenen arg angehängt und dem zu zerlegenden Text als letztem Argument. Dieser Aufruf muss eine gültige Tcl-Liste zurückgeben, deren Elemente geprüft werden. Fehlt das Argument des Zerlegungstyps, ist die Vorgabe *whitespace*.

strip *<constraint script>*
Dieser Texteinschränkungsbefehl prüft alle Texteinschränkungen im ausgewerteten *constraint script* mit dem zu prüfenden Text, befreit von allem Leerraum am Anfang und am Ende.

Weitere Zeichenketteneigenschaften

fixed *value*
Die Texteinschränkung passt nur, wenn der Textwert zeichengleich mit dem angegebenen

Wert ist.

enumeration *list*
Diese Texteinschränkung passt, wenn der Textwert einem Element des Arguments *list* gleicht, unter Beachtung von Groß- und Kleinschreibung und jedem Leerraum. *list* muss eine gültige Tcl-Liste sein.

match*?-nocase? glob_style_match_pattern>*
<URL: https://www.tcl.tk/man/tcl8.6/TclCmd/string.htm#M35> "" Diese Texteinschränkung passt, wenn der Textwert auf das als Argument angegebene Glob-Muster passt. Es gelten die Regeln des Tcl-Befehls [string match], siehe.

regexp *expression*
<URL: https://www.tcl.tk/man/tcl8.6/TclCmd/re_syntax.htm> "" Diese Texteinschränkung passt, wenn der Textwert auf den als Argument angegebenen regulären Ausdruck passt. beschreibt die Syntax des regulären Ausdrucks

length *length*
Diese Texteinschränkung passt, wenn die Länge des Textwerts, in Zeichen und nicht in Bytes, *length* ist. Das Argument length muss eine positive ganze Zahl oder null sein.

maxLength *length*
Diese Texteinschränkung passt, wenn die Länge des Textwerts, in Zeichen und nicht in Bytes, höchstens *length* ist. Das Argument length muss eine ganze Zahl größer als null sein.

minLength *length*
Diese Texteinschränkung passt, wenn die Länge des Textwerts, in Zeichen und nicht in Bytes, mindestens *length* ist. Das Argument length muss eine ganze Zahl größer als null sein.

id*?keySpace?*
Dieser Texteinschränkungsbefehl markiert den Text als dokumentweite ID, auf die ein idref verweisen kann. Jeder ID-Wert innerhalb eines Dokuments muss eindeutig sein. Es ist kein Fehler, wenn auf die ID im Dokument tatsächlich nicht verwiesen wird. Das optionale Argument *keySpace* tut all das für einen benannten Schlüsselraum. Der Schlüsselraum "" (die leere Zeichenkette) ist ein anderer Schlüsselraum als der Befehl *id* ohne Argument keySpace.

idref*?keySpace?*
Dieser Texteinschränkungsbefehl erwartet, dass der Text ein Verweis auf eine ID innerhalb des Dokuments ist. Die verwiesene ID darf im Dokument später stehen als der Verweis. Mehrere Verweise innerhalb des Dokuments auf eine ID sind möglich. Zum optionalen Argument *keySpace* siehe *id*.

jsontype *<JSON text type>*
Wird ein DOM-Baum nicht mit *domvalidate* nachträglich geprüft, passt diese Einschränkung immer. Beim Nachprüfen muss der zu prüfende TEXT_NODE den JSON-Texttyp haben, der dem Texteinschränkungsbefehl als Argument angegeben ist. Die möglichen Typen sind *NULL*, *TRUE*, *FALSE*, *STRING* und *NUMBER*.

LOKALE SCHLÜSSELEINSCHRÄNKUNGEN

Dokumentweite Eindeutigkeit und Fremdschlüssel-Einschränkungen stehen mit den Texteinschränkungsbefehlen id und idref zur Verfügung. Schlüsselräume erlauben eindeutige Werte und Fremdschlüssel-Einschränkungen, die auf einen Teilbaum beschränkt sind.

keyspace *<names list> <constraint script>*
Beliebig viele Schlüsselräume sind möglich. Ein Schlüsselraum ist entweder aktiv oder nicht.

Ein innerhalb eines *constraint script* aufgerufener keyspace mit demselben Namen tut nichts. Diese Texteinschränkungsbefehle arbeiten mit Schlüsselräumen:

key *<name>*
Ist der Schlüsselraum mit dem Namen <*name*> nicht aktiv, passt die Einschränkung immer. Ist der Schlüsselraum aktiv, wird ein Fehler gemeldet, wenn es schon einen Schlüssel mit dem Wert gibt. Sonst speichert sie den Wert als Schlüssel in diesem Schlüsselraum und passt.

keyref *<name>*
Ist der Schlüsselraum mit dem Namen <*name*> nicht aktiv, passt sie immer. Ist der Schlüsselraum aktiv, wird ein Fehler gemeldet, wenn es am Ende des Schlüsselraums <*name*> noch keinen Schlüssel mit dem Wert gibt. Sonst passt sie.

WIEDERAUFNAHME

In der Vorgabe hält die Prüfmaschine bei der ersten erkannten Prüfverletzung an und meldet diesen Befund. Wird der Schemabefehl selbst zur Prüfung der Eingabe verwendet, gibt sie dazu falsch zurück und setzt, falls angegeben, die Ergebnisvariable auf eine Fehlermeldung. Verwendet ein SAX-Parser oder der DOM-Parser den Schemabefehl, löst sie dazu einen Fehler aus.

Ist ein *reportcmd* gesetzt, wird dieser Befehl auf globaler Ebene aufgerufen, mit dem Schemabefehl und einem Fehlertyp als angehängten Argumenten, sobald eine Prüfverletzung erkannt wird. Danach erholt sich die Prüfung vom Fehler und läuft weiter. Bei manchen Prüffehlern lässt sich die Strategie der Wiederaufnahme mit dem Skriptergebnis von reportcmd bestimmen.

Mit einem *reportcmd*, solange der *reportcmd* beim Aufruf keinen Fehler auslöst, meldet die Prüfmaschine ihrem Aufrufer nie einen Prüffehlschlag. Die Prüfmaschine erholt sich, läuft weiter und meldet den nächsten Fehler, falls einer auftritt, und so weiter bis zum Ende der Eingabe. Der Schemabefehl gibt wahr zurück, und der SAX-Parser und der DOM-Aufbau arbeiten bis zum Ende der Eingabe normal, als hätte es keinen Prüffehler gegeben.

Das gilt nur für Prüffehler. Von Wohlgeformtheitsfehlern lässt sich nicht erholen. Ist die Eingabe nicht wohlgeformt, gibt der Schemabefehl falsch zurück und setzt, falls angegeben, die Ergebnisvariable auf eine Fehlermeldung über den Wohlgeformtheitsfehler.

Löst der *reportcmd* beim Aufruf durch die Prüfmaschine einen Fehler aus, hält die Prüfung an, und der Schemabefehl löst einen Fehler mit der Fehlermeldung des Skripts aus.

Beim Prüfen können im Wesentlichen drei Ereignisse eintreten: ein Element-Start-Tag muss passen, ein Textstück muss passen oder ein Element-Ende-Tag muss passen. Die Methode *info vaction*, aufgerufen im Wiederaufnahme-Skript oder in jedem von dort aufgerufenen Skriptcode, gibt zurück, welches Ereignis die Fehlermeldung ausgelöst hat, der Reihe nach MATCH_ELEMENT_START, MATCH_TEXT, MATCH_ELEMENT_END. Während der Befehl das Schema durchläuft und prüft, ob das Ereignis passt, können andere, datengetriebene Ereignisse eintreten, zum Beispiel die Prüfung, ob ein keyref innerhalb eines Schlüsselraums vorhanden ist.

Mehrere der Prüffehlercodes, die den Aufrufen von *reportcmd* als zweites Argument angehängt werden, können bei mehr als einer Art von Prüfereignis auftreten. Die Methode *info vaction* und ihre Unterbefehle geben Angaben zum aktuellen Prüfereignis, wenn sie aus dem Meldebefehl aufgerufen werden.

Tritt ein struktureller Prüffehler auf, ist die Vorgabe der Wiederaufnahme, jeden folgenden oder

fehlenden Inhalt innerhalb des aktuellen Teilbaums zu übergehen und mit dem Ereignis Elementende des Teilbaums weiterzumachen.

Die Rückgabe "ignore" aus dem Wiederaufnahme-Skript beim Fehlertyp MISSING_ELEMENT erholt sich, indem die fehlgeschlagene Einschränkung übergangen wird, und passt das Ereignis weiter gegen das Schema.

Die Rückgabe "vanish" aus dem Wiederaufnahme-Skript bei den Fehlertypen MISSING_ELEMENT und UNEXPECTED_ELEMENT erholt sich, indem das Ereignis übergangen wird.

BEISPIELE

<URL: > XML Schema Part 0: Primer Second Edition () beginnt mit diesem Beispielschema:

```
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">

  <xsd:annotation>
    <xsd:documentation xml:lang="en">
     Purchase order schema for Example.com.
     Copyright 2000 Example.com. All rights reserved.
    </xsd:documentation>
  </xsd:annotation>

  <xsd:element name="purchaseOrder" type="PurchaseOrderType"/>

  <xsd:element name="comment" type="xsd:string"/>

  <xsd:complexType name="PurchaseOrderType">
    <xsd:sequence>
      <xsd:element name="shipTo" type="USAddress"/>
      <xsd:element name="billTo" type="USAddress"/>
      <xsd:element ref="comment" minOccurs="0"/>
      <xsd:element name="items"  type="Items"/>
    </xsd:sequence>
    <xsd:attribute name="orderDate" type="xsd:date"/>
  </xsd:complexType>

  <xsd:complexType name="USAddress">
    <xsd:sequence>
      <xsd:element name="name"   type="xsd:string"/>
      <xsd:element name="street" type="xsd:string"/>
      <xsd:element name="city"   type="xsd:string"/>
      <xsd:element name="state"  type="xsd:string"/>
      <xsd:element name="zip"    type="xsd:decimal"/>
    </xsd:sequence>
    <xsd:attribute name="country" type="xsd:NMTOKEN"
                   fixed="US"/>
  </xsd:complexType>
```

```
  <xsd:complexType name="Items">
    <xsd:sequence>
      <xsd:element name="item" minOccurs="0" maxOccurs="unbounded">
        <xsd:complexType>
          <xsd:sequence>
            <xsd:element name="productName" type="xsd:string"/>
            <xsd:element name="quantity">
              <xsd:simpleType>
                <xsd:restriction base="xsd:positiveInteger">
                  <xsd:maxExclusive value="100"/>
                </xsd:restriction>
              </xsd:simpleType>
            </xsd:element>
            <xsd:element name="USPrice"  type="xsd:decimal"/>
            <xsd:element ref="comment"   minOccurs="0"/>
            <xsd:element name="shipDate" type="xsd:date"
    minOccurs="0"/>
          </xsd:sequence>
          <xsd:attribute name="partNum" type="SKU" use="required"/>
        </xsd:complexType>
      </xsd:element>
    </xsd:sequence>
  </xsd:complexType>

  <!-- Stock Keeping Unit, a code for identifying products -->
  <xsd:simpleType name="SKU">
    <xsd:restriction base="xsd:string">
      <xsd:pattern value="\d{3}-[A-Z]{2}"/>
    </xsd:restriction>
  </xsd:simpleType>

</xsd:schema>
```

Eine einfache Eins-zu-eins-Übertragung davon in ein tDOM-Schema-Definitionsskript wäre:

```
tdom::schema schema
schema define {

    # Purchase order schema for Example.com.
    # Copyright 2000 Example.com. All rights reserved.

    defelement purchaseOrder {ref PurchaseOrderType}

    foreach elm {comment name street city state product} {
        defelement $elm text
    }
```

```
    defpattern PurchaseOrderType {
        element shipTo ! {ref USAddress}
        element billTo ! {ref USAddress}
        element comment ?
        element items
        attribute orderDate date
    }

    defpattern USAddress {
        element name
        element street
        element city
        element state
        element zip ! {text number}
        attribute country {fixed "US"}
    }

    defelement items {
        element item * {
            element product
            element quantity ! {text positiveInteger}
            element USPrice ! {text number}
            element comment
            element shipDate ? {text date}
            attribute partNum {regexp "^\d{3}-[A-Z]{2}$"}
        }
    }
}
```

<URL: > Das RELAX NG Tutorial () beginnt mit diesem Beispiel:

```
Consider a simple XML representation of an email address book:

<addressBook>
  <card>
    <name>John Smith</name>
    <email>js@example.com</email>
  </card>
  <card>
    <name>Fred Bloggs</name>
    <email>fb@example.net</email>
  </card>
</addressBook>

The DTD would be as follows:

<!DOCTYPE addressBook [
```

```
<!ELEMENT addressBook (card*)>
<!ELEMENT card (name, email)>
<!ELEMENT name (#PCDATA)>
<!ELEMENT email (#PCDATA)>
]>

A RELAX NG pattern for this could be written as follows:

<element name="addressBook"
    xmlns="http://relaxng.org/ns/structure/1.0">
  <zeroOrMore>
    <element name="card">
      <element name="name">
        <text/>
      </element>
      <element name="email">
        <text/>
      </element>
    </element>
  </zeroOrMore>
</element>
```

Dieses Schema-Definitionsskript leistet dasselbe:

```
tdom::schema schema
schema define {
    defelement addressBook {
        element card *
    }
    defelement card {
        element name
        element email
    }
    foreach e {name email} {
        defelement $e text
    }
}
```

tdomcmd

Stichwörter: tdomcmd, DOM, SAX, C handler set

NAME

tdom - tdom ist eine Erweiterung eines expat-Parserobjekts und legt beim Lesen aus der Eingabe einen DOM-Baum im Speicher an.

ÜBERSICHT

package require tdom

set parser [expat]

tdom $parser enable

BESCHREIBUNG

tdom fügt einem Tcl-expat-Parserobjekt die C-Handler-Menge "tdom" hinzu. Diese Handler-Menge baut aus der vom Parser gelesenen Eingabe einen DOM-Baum im Speicher. Ein so angelegter DOM-Baum verhält sich genau wie ein mit dem Befehl "dom" angelegter DOM-Baum, siehe dort. tdom ist nur eine weitere Schnittstelle zur selben Funktion und nutzt den Code hinter dem Code von **dom**, um den DOM-Baum aufzubauen.

tdom *parserObj* enable
Fügt einem Tcl-expat-Parserobjekt die C-Handler-Menge tdom hinzu. Wenn der Parser das nächste Mal Eingabe liest, legen die C-Handler-Funktionen von tdom einen DOM-Baum im Speicher an.

tdom *parserObj* getdoc
Gibt den DOM-Baum als Objekt domDoc zurück, siehe dort.

tdom *parserObj* setStoreLineColumn ?*boolean*?
Die genaue Beschreibung steht bei der Methode **setStoreLineColumn** des Befehls **dom**.

tdom *parserObj* remove
Entfernt die C-Handler-Menge tdom vom Parserobjekt.

tdom *parserObj* keepEmpties
Die genaue Beschreibung steht bei der Option **-keepEmpties** des Befehls **dom**.

tdom *parserObj* setExternalEntityResolver *script*

tnc

Stichwörter: tnc, Validation, DTD

NAME

tnc - tnc ist eine Erweiterung eines expat-Parserobjekts, die den XML-Strom beim Lesen gegen die Dokument-DTD prüft.

ÜBERSICHT

package require tdom
package require tnc

set parser [expat]

tnc $parser enable

BESCHREIBUNG

tnc fügt einem Tcl-expat-Parserobjekt die C-Handler-Menge "tnc" hinzu. Diese Handler-Menge ist ein einfacher DTD-Prüfer. Erkennt der Prüfer einen Prüffehler, setzt er das Interpreter-Ergebnis, meldet einen Fehler und beendet das Lesen. Eine Wiederaufnahme nach einem Prüffehler gibt es nicht. Deshalb werden nur gültige Dokumente vollständig gelesen.

Diese Handler-Menge hat nur drei Methoden:

tnc *parserObj* enable
Fügt einem Tcl-expat-Parserobjekt die C-Handler-Menge tnc hinzu.

tnc *parserObj* remove
Entfernt den tnc-Prüfer vom Parser *parserObj* und gibt alle von ihm gespeicherten Angaben frei.

tnc *parserObj* getValidateCmd?validateCmdName?
Gibt einen neu angelegten Prüfbefehl zurück, wenn der Parserbefehl einen bereithält, sonst wird ein Fehler gemeldet. Der Name des Prüfbefehls ist *validateCmdName*, wenn dieses optionale Argument angegeben ist, sonst ein zufällig gewählter Name. Ein Prüfbefehl steht in einem Parserbefehl zur Verfügung, wenn der Parser mit eingeschaltetem tnc zuvor ein XML-Dokument gelesen hat, das eine gültige Doctype-Deklaration hat, eine gültige externe Teilmenge, falls die Doctype-Deklaration eine angibt, und eine gültige interne Teilmenge. Das übrige Dokument muss nicht gültig sein, damit der Prüfbefehl zur Verfügung steht. Der Prüfbefehl lässt sich vom Parserbefehl nur einmal empfangen. Der angelegte Prüfbefehl hat diese Syntax:

```
validationCmd method ?args?
```

Die gültigen Methoden sind:

validateDocument *domDocument?varName?*
Prüft, ob das angegebene domDocument gegen die DTD-Angaben gültig ist, die der Prüfbefehl darstellt. Gibt 1 zurück, wenn das Dokument gültig ist, sonst 0. Ist das Argument *varName* angegeben, wird die Variable dieses Namens auf den erkannten Grund des Prüffehlers gesetzt oder bei einem gültigen Dokument auf die leere Zeichenkette.

validateTree *elementNode?varName?*
Prüft, ob der angegebene Teilbaum mit *domNode* als Wurzelelement ein möglicher gültiger Teilbaum eines Dokuments ist, das den DTD-Angaben des Prüfbefehls entspricht. IDREF lässt sich bei der Prüfung nur eines Teilbaums nicht prüfen, aber es wird geprüft, dass jedes bekannte ID-Attribut im Teilbaum eindeutig ist. Gibt 1 zurück, wenn der Teilbaum in Ordnung ist, sonst 0. Ist das Argument *varName* angegeben, wird die Variable dieses Namens auf den erkannten Grund des Prüffehlers gesetzt oder bei einem gültigen Teilbaum auf die leere Zeichenkette.

validateAttributes *elementNode?varName?*
Prüft, ob es in der vom Prüfbefehl dargestellten DTD eine Elementdeklaration für den Namen des *elementNode* gibt und, wenn ja, ob die Attribute des *elementNode* den ATTLIST-Deklarationen für dieses Element in der DTD entsprechen. Gibt 1 zurück, wenn die Attribute und ihre Werttypen in Ordnung sind, sonst 0. Ist das Argument *varName* angegeben, wird die Variable dieses Namens auf den erkannten Grund des Prüffehlers gesetzt oder auf die leere Zeichenkette, wenn das Element alle erforderlichen Attribute hat, nur deklarierte Attribute und die Attributwerte ihrem Typ entsprechen.

delete
Löscht den Prüfbefehl und gibt den von ihm belegten Speicher frei. Gibt die leere Zeichenkette zurück.

BUGS

Die Meldungen von Prüffehlern könnten deutlich informativer und benutzerfreundlicher sein.

Der Prüfer erkennt mehrdeutige Inhaltsmodelle nicht, siehe XML-Empfehlung Abschnitt 3.2.1 und Anhang E. Die meisten Java-Prüfer erkennen sie ebenfalls nicht, behandeln solche Inhaltsmodelle

aber dennoch richtig. Tnc tut das nicht. Hat die DTD solche mehrdeutigen Inhaltsmodelle, lässt sich tnc nicht zur Prüfung von Dokumenten gegen solche, nicht vollständig der XML-Spezifikation entsprechenden DTDs verwenden.

XML-Dokumente mit standalone="yes" in der XML-Deklaration lassen sich nicht prüfen.

Verstöße gegen die Gültigkeitseinschränkungen Proper Group/PE Nesting und Proper Conditional Section/PE Nesting werden nicht erkannt. Solche Verstöße können nur innerhalb einer ungültigen DTD auftreten, nicht im Inhalt eines Dokuments.

xpathFunc

Stichwörter: xpathFunc, XPath

NAME

::dom::xpathFunc - XPath-Funktionen in Tcl

ÜBERSICHT

```
::dom::xpathFunc::funcName ctxNode pos nodeListType nodeList ?type arg
    type arg...?
::dom::xpathFunc::namespaceURL::funcName ctxNode pos nodeListType
    nodeList ?type arg type arg...?
```

BESCHREIBUNG

Die XPath-Maschine von tDOM unterstützt eigene XPath-Funktionen, die Tcl-Befehle ausführen. Trifft sie auf einen Funktionsaufruf mit unbekanntem Funktionsnamen ohne Namensraum-Qualifikator, sucht die XPath-Maschine im Namensraum **::dom::xpathFunc** einen Tcl-Befehl desselben Namens. Trifft sie auf einen unbekannten Funktionsaufruf mit Namensraum, sucht sie innerhalb von **::dom::xpathFunc** einen Tcl-Namensraum, dessen Name dem vollen Namensraum-URI der Funktion gleicht, und darin einen Tcl-Befehl, der nach dem lokalen Teil des Funktionsnamens benannt ist. Findet sie einen solchen Tcl-Befehl, wird er mit mindestens den folgenden Argumenten ausgeführt, die den aktuellen XPath-Kontext beschreiben:

ctxNode
: Der Objektbefehl **domNode** des XPath-Kontextknotens.

pos
: Die XPath-Kontextposition, nullbasiert. Der Rückgabewert der üblichen XPath-Funktion **position()** im Kontext wäre demnach **$pos - 1**.

nodeListType
: Der Typname für *nodeList*, wie bei *typeVar* der Methode **selectNodes**.

nodeList
: Die Liste des gerade bearbeiteten Ergebnisses des XPath-Schritts, in Dokumentreihenfolge, wie **selectNodes** sie zurückgäbe. *ctxNode* ist gleich **[lindex $nodeList $pos]**. Enthält der Funktionsaufruf Argumente, werden der Argumentliste des Befehls für jedes Argument zwei Argumente angehängt:

type
: Der Typname des Arguments, wie bei *typeVar* von **selectNodes**.

val
: Der Wert des Arguments, wie **selectNodes** ihn in der Ergebnismenge eines XPath-Ausdrucks

zurückgäbe. Der Befehl muss eine Tcl-Liste mit einem oder zwei Elementen zurückgeben, die das Ergebnis des XPath-Funktionsaufrufs liefert. Hat die Ergebnisliste zwei Elemente, ist das erste der XPath-Typname des Ergebnisses und das zweite ein passend kodierter Wert. Der Typname *attrnodes* wird nicht unterstützt:

bool
Tcl-Wahrheitswert, den **Tcl_GetBooleanFromObj** annimmt.

number
Tcl-Zahlenwert, den **Tcl_GetSizeIntFromObj** oder **Tcl_GetDoubleFromObj** annimmt.

string
Einfache Zeichenkette.

nodes
Tcl-Liste von Objektbefehlen **domNode**.

attrvalues
Alias für *string*. Hat die Ergebnisliste nur ein Element, gilt es als einfache Zeichenkette. Eine Ergebnisliste mit null Elementen, mit mehr als zwei Elementen oder eine ungültige Tcl-Liste ist ein Fehler, und es ist ein Fehler, wenn der *val* eines zweiteiligen Ergebnisses nicht den oben für seinen *type* beschriebenen Anforderungen entspricht.

BEISPIELE

Ein einfacher Nachbau der XPath-2/3-Funktion **fn:matches()**, die in einer XPath-1.0-Maschine wie der von tDOM sonst nicht zur Verfügung steht:

```
proc ::dom::xpathFunc::regmatch {
    ctxNode pos nodeListType nodeList
    inputType inputVal patternType patternVal
} {
    set input [::dom::xpathFuncHelper::coerce2string $inputType
    $inputVal]
    set pattern [::dom::xpathFuncHelper::coerce2string $patternType
    $patternVal]
    return [list bool [regexp -- $pattern $input]]
}
```

HILFSPROZEDUREN

Der Namensraum **::dom::xpathFuncHelper** enthält Hilfsprozeduren für XPath-Funktionen in Tcl:

coerce2number *type val*
Aus einem *type* und einem *val*, wie sie XPath-Funktionen in Tcl in ihrer Argumentliste bekommen, wird *val* auf eine für seinen *type* passende Weise in eine Zahl umgewandelt:

empty
Immer null.

number
Unverändert.

string
Unverändert. Es wird kein Fehler ausgelöst, wenn *val* nicht numerisch ist.

attrvalues
Das erste Element von *value*.

nodes
Das Ergebnis der XPath-Funktion **number()**, aufgerufen für den ersten Knoten der Liste.

attrnodes
Der Wert des einen Attributs, das in *value* angenommen wird. Es wird kein Fehler ausgelöst, wenn dieser Wert nicht numerisch ist.

coerce2string *type val*
Wie bei **coerce2number**, aber *val* wird in eine reine Zeichenkette umgewandelt:

empty
Die leere Zeichenkette.

number
Unverändert.

string
Unverändert.

attrvalues
Das erste Element von *value*.

nodes
Das Ergebnis der XPath-Funktion **string()**, aufgerufen für den ersten Knoten der Liste.

attrnodes
Der Wert des einen Attributs, das in *value* angenommen wird.

GRENZEN

Namen eigener XPath-Funktionen sind auf 200 Zeichen begrenzt, einschließlich eines Namensraum-URI und des Tcl-Namensraumtrenners **::** zwischen ihm und dem lokalen Teil. Funktionsaufrufe dürfen höchstens 22 Argumente haben. Die Argumentwerte selbst dürfen große Knotenmengen sein. Werden mehr gebraucht, lassen sich diese Grenzen zur Bauzeit über Defines anpassen. Tcl-Befehle, die mit der veralteten Schnittstelle **Tcl_CreateCommand** angelegt wurden, lassen sich nicht als XPath-Funktionen in Tcl verwenden.

Die DOM-Bäume, aus denen Knoten einer XPath-Funktion in Tcl als Argument übergeben werden, dürfen **nicht** verändert werden. Das gilt für das Argument ctxNode und für die Knoten im Argument nodeList. Sie werden streng nur gelesen. Wird das missachtet, sind beliebige unvorhersehbare Folgen möglich, einschließlich Speicherzugriffsfehlern oder Sicherheitsproblemen.

thread

thread

Stichwörter: thread, events, message passing, mutex, synchronization

NAME

thread - Erweiterung für den Skriptzugriff auf Tcl-Threading

ÜBERSICHT

package require Tcl 9.0
package require thread ?**3.0**?
thread::create ?*-joinable*? ?*-preserved*? ?*script*?
thread::preserve ?*id*?
thread::release ?*-wait*? ?*id*?
thread::id
thread::errorproc ?*procname*?
thread::cancel ?*-unwind*? *id* ?*result*?
thread::unwind
thread::exit ?*status*?
thread::names
thread::exists *id*
thread::send ?*-async*? ?*-head*? *id script* ?*varname*?
thread::broadcast *script*
thread::wait
thread::eval ?*-lock mutex*? *arg* ?*arg ...*?
thread::join *id*
thread::configure *id* ?*option*? ?*value*? ?*... ...*?
thread::transfer *id channel*
thread::detach *channel*
thread::attach *channel*
thread::mutex
thread::mutex create ?*-recursive*?
thread::mutex destroy *mutex*
thread::mutex lock *mutex*
thread::mutex unlock *mutex*
thread::rwmutex
thread::rwmutex create
thread::rwmutex destroy *mutex*
thread::rwmutex rlock *mutex*
thread::rwmutex wlock *mutex*
thread::rwmutex unlock *mutex*
thread::cond
thread::cond create
thread::cond destroy *cond*
thread::cond notify *cond*
thread::cond wait *cond mutex* ?*ms*?

BESCHREIBUNG

Die Erweiterung **thread** erzeugt Threads, die Tcl-Interpreter enthalten, und erlaubt, Skripte zur Auswertung an diese Threads zu senden. Zusätzlich bietet sie auf Skriptebene Zugang zu den grundlegenden Grundelementen der Thread-Synchronisation, etwa Mutexen und Bedingungsvariablen.

BEFEHLE

Dieser Abschnitt beschreibt Befehle, um Threads zu erzeugen und zu zerstören und Skripte zur

Auswertung an Threads zu senden.

thread::create ?-joinable? ?-preserved? ?script?

Dieser Befehl erzeugt einen Thread, der einen Tcl-Interpreter enthält. Der Tcl-Interpreter wertet entweder das optionale **script** aus, falls angegeben, oder er wartet in der Ereignisschleife auf Skripte, die über den Befehl **thread::send** eintreffen. Das Ergebnis des optionalen **script**, falls vorhanden, wird dem Aufrufer nie zurückgegeben. Das Ergebnis von **thread::create** ist die ID des Threads. Das ist das undurchsichtige Handle, das den neu erzeugten Thread für alle anderen Befehle des Pakets bezeichnet. Das Handle des Threads verlässt den Geltungsbereich automatisch, wenn der Thread zum Beenden vorgemerkt ist (siehe den Befehl **thread::release** unten). Enthält das optionale Argument **script** den Befehl **thread::wait**, betritt der Thread die Ereignisschleife. Wird ein solcher Befehl im **script** nicht gefunden, führt der Thread das **script** bis zum Ende aus und beendet sich. In dem Fall darf das Handle unbedenklich ignoriert werden, weil es auf einen Thread verweist, der zu dem Zeitpunkt, an dem der Befehl zurückgibt, nicht mehr besteht. Mit dem Schalter **-joinable** lässt sich ein joinbarer Thread erzeugen, also einer, auf dessen Ende mit dem Befehl **thread::join** gewartet werden kann. Wird ein mit dem Schalter **-joinable** erzeugter Thread nicht verbunden, entstehen Ressourcen- und Speicherlecks. Threads, die **thread::create** erzeugt, lassen sich nicht gewaltsam zerstören. Folglich gibt es keinen entsprechenden Befehl, um einen Thread zu zerstören. Ein Thread darf nur mit **thread::release** freigegeben werden, und wenn sein interner Referenzzähler auf null fällt, wird der Thread zum Beenden vorgemerkt. Das wirft den Thread aus der Bedienung der Ereignisschleife, und der Thread führt die im Argument **script** übergebenen Befehle weiter aus, die auf den Befehl **thread::wait** folgen. War das der letzte Befehl im Skript, wie es gewöhnlich der Fall ist, beendet sich der Thread. Es lässt sich eine Lage erzeugen, in der der Thread sich nicht mehr beenden lässt, etwa indem nach **thread::wait** eine Endlosschleife steht oder die Ereignisschleife durch einen Befehl der Art **vwait** erneut betreten wird. In solchen Fällen beendet sich der Thread vielleicht nie. Das gilt als schlechte Praxis und ist zu vermeiden, wo es geht. Das Beispiel unten zeigt das am deutlichsten:

```
# Das ist zu vermeiden ...
set tid [thread::create {
    package require Http
    thread::wait
    vwait forever ; # <-- diese Stelle!
}]
```

Der im Beispiel oben erzeugte Thread kann sich nie beenden. Nachdem er mit dem letzten passenden Aufruf von **thread::release** freigegeben wurde, springt der Thread aus **thread::wait** und führt die folgenden Befehle weiter aus. Er betritt den Befehl **vwait** und wartet endlos auf Ereignisse. Einen solchen Thread kann nichts beenden, das ist also zu vermeiden. Jeder neu erzeugte Thread hat seinen internen Referenzzähler auf 0 (null) gesetzt, das heißt, er ist nicht reserviert. Dieser Zähler wird durch einen Aufruf von **thread::preserve** erhöht und durch einen Aufruf des Befehls **thread::release** verringert. Diese beiden Befehle setzen ein einfaches, aber wirksames System zur Reservierung von Threads um und bieten eine vorhersagbare und steuerbare Beendigung von Threads. Anfangs reservierte Threads lassen sich erzeugen, indem der Schalter **-preserved** des Befehls **thread::create** verwendet wird. Threads, die mit diesem Schalter erzeugt werden, haben als Anfangswert des

Referenzzählers 1 (eins) und sind damit anfangs als reserviert gekennzeichnet.

thread::preserve ?id?
Dieser Befehl erhöht den Referenzzähler des Threads. Jeder Aufruf dieses Befehls erhöht den Referenzzähler um eins (1). Der Befehl gibt den Wert des Referenzzählers nach der Erhöhung zurück. Mit der optionalen Thread-**id** aufgerufen, reserviert der Befehl den gegebenen Thread. Andernfalls wird der aktuelle Thread reserviert. Mit der Referenzzählung lässt sich der kontrollierte Zugang zu einem gemeinsamen Tcl-Thread umsetzen. Durch das Erhöhen des Referenzzählers zeigt der Aufrufer an, dass er den Thread längere Zeit verwenden will. Durch das Verringern des Zählers zeigt der Aufrufer an, dass er die Verwendung des Threads beendet hat.

thread::release ?-wait? ?id?
Dieser Befehl verringert den Referenzzähler des Threads. Jeder Aufruf dieses Befehls verringert den Referenzzähler um eins (1). Mit der optionalen Thread-**id** aufgerufen, gibt der Befehl den gegebenen Thread frei. Andernfalls wird der aktuelle Thread freigegeben. Der Befehl gibt den Wert des Referenzzählers nach der Verringerung zurück. Erreicht der Referenzzähler null (0), wird der Ziel-Thread zur Beendigung vorgemerkt. Nachdem der Befehl **thread::release** null oder eine negative ganze Zahl zurückgibt, darf der Thread nicht mehr verwendet werden. Das Handle des Threads verlässt den Geltungsbereich und darf nicht weiter verwendet werden. Jeder folgende Verweis auf dasselbe Thread-Handle führt zu einem Tcl-Fehler. Der optionale Schalter **-wait** weist den aufrufenden Thread an, auf das Ende des Ziel-Threads zu warten, wenn die Wirkung des Befehls die Beendigung des Ziel-Threads zur Folge hätte, das heißt, wenn das Ergebnis null (0) wäre. Ohne den Schalter wartet der aufrufende Thread nicht auf das Ende des Ziel-Threads. Beim Verwenden von **-wait** ist Vorsicht nötig, weil das den aufrufenden Thread unbegrenzt blockieren kann. Diese Option wurde für einige besondere Verwendungen der Erweiterung umgesetzt und ist für den regelmäßigen Gebrauch veraltet. Für den regelmäßigen Gebrauch sind joinbare Threads mit der Option **-joinable** des Befehls **thread::create** zu erzeugen und mit **thread::join** auf das Ende des Threads zu warten.

thread::id
Dieser Befehl gibt die ID des aktuellen Threads zurück.

thread::errorproc ?procname?
Dieser Befehl setzt eine Behandlung für Fehler, die in Skripten auftreten, die asynchron, mit dem Schalter **-async** des Befehls **thread::send**, an andere Threads gesendet werden. Ist keine Behandlung angegeben, wird die aktuelle Behandlung zurückgegeben. Die leere Zeichenkette setzt die Behandlung auf den Vorgabewert (unbestimmt) zurück. Ein nicht abgefangener Fehler in einem Thread bewirkt, dass eine Fehlermeldung an den Standardfehlerkanal gesendet wird. Dieses voreingestellte Melden lässt sich ändern, indem eine Prozedur registriert wird, die aufgerufen wird, um den Fehler zu melden. *procname* wird in dem Interpreter aufgerufen, der den Befehl **thread::errorproc** aufgerufen hat. *procname* wird so aufgerufen:

```
myerrorproc thread_id errorInfo
```

thread::cancel ?-unwind? *id* ?result?
Dieser Befehl verlangt Tcl ab Version 8.6. Er bricht das Skript ab, das in dem durch den Parameter *id* gegebenen Thread ausgewertet wird. Ohne den Schalter **-unwind** wird der

Auswertungsstapel des Interpreters abgebaut, bis ein umschließender catch-Befehl gefunden wird oder keine weiteren Aufrufe des Interpreters auf dem Aufrufstapel bleiben. Mit dem Schalter **-unwind** wird der Auswertungsstapel des Interpreters ohne Rücksicht auf einen dazwischenliegenden catch-Befehl abgebaut, bis keine weiteren Aufrufe des Interpreters auf dem Aufrufstapel bleiben. Ist *result* vorhanden, wird es als Zeichenkette der Fehlermeldung verwendet; andernfalls wird eine voreingestellte Zeichenkette der Fehlermeldung verwendet.

thread::unwind

Die Verwendung dieses Befehls ist veraltet zugunsten des weiter entwickelten Systems zur Reservierung von Threads, das mit den Befehlen **thread::preserve** und **thread::release** umgesetzt ist. Die Unterstützung für den Befehl **thread::unwind** wird in einer künftigen Hauptversion der Erweiterung verschwinden. Dieser Befehl beendet ein vorangegangenes **thread::wait**. Die Ausführung des Skripts, das an den neu erzeugten Thread übergeben wurde, setzt am Befehl **thread::wait** fort. War **thread::wait** der letzte Befehl im Skript, beendet sich der Thread. Der Befehl gibt ein leeres Ergebnis zurück, kann aber in manchen Situationen einen Tcl-Fehler mit der Meldung "target thread died" auslösen.

thread::exit ?status?

Die Verwendung dieses Befehls ist veraltet zugunsten des weiter entwickelten Systems zur Reservierung von Threads, das mit den Befehlen **thread::preserve** und **thread::release** umgesetzt ist. Die Unterstützung für den Befehl **thread::exit** wird in einer künftigen Hauptversion der Erweiterung verschwinden. Dieser Befehl zwingt einen Thread, der im Befehl **thread::wait** feststeckt, bedingungslos zum Beenden. Der Beendigungsstatus des Threads ist voreingestellt 666 und lässt sich mit dem optionalen Argument *status* angeben. Die Ausführung des Befehls **thread::exit** hinterlässt den Programmspeicher garantiert in einem inkonsistenten Zustand, erzeugt Speicherlecks und beeinträchtigt andere Teilsysteme der Tcl-Anwendung auf unvorhersagbare Weise. Der Befehl gibt ein leeres Ergebnis zurück, kann aber in manchen Situationen einen Tcl-Fehler mit der Meldung "target thread died" auslösen.

thread::names

Dieser Befehl gibt eine Liste von Thread-IDs zurück. Das sind nur Threads, die über den Befehl **thread::create** erzeugt wurden. Erzeugt die Anwendung weitere Threads auf C-Ebene, meldet dieser Befehl sie nicht.

thread::exists *id*

Gibt wahr (1) zurück, wenn der durch den Parameter *id* gegebene Thread besteht, andernfalls falsch (0). Das gilt nur für Threads, die über den Befehl **thread::create** erzeugt wurden.

thread::send ?-async? ?-head? *id script* ?varname?

Dieser Befehl übergibt ein *script* an einen anderen Thread und wartet optional auf das Ergebnis. Ist der Schalter **-async** angegeben, wartet der Befehl nicht auf das Ergebnis und gibt die leere Zeichenkette zurück. Der Ziel-Thread muss seine Ereignisschleife betreten, um Skripte zu empfangen, die über diesen Befehl gesendet werden. Das geschieht voreingestellt für Threads, die ohne Startskript erzeugt wurden. Threads können die Ereignisschleife ausdrücklich betreten, indem sie **thread::wait** oder einen anderen passenden Tcl/Tk-Befehl aufrufen, etwa **update**, **vwait** und so weiter. Das optionale **varname** legt fest, in welcher Variable das Ergebnis des *script* gespeichert wird. Ohne den Schalter **-async** gibt der Befehl den Auswertungscode zurück, ähnlich dem üblichen Tcl-Befehl **catch**. Ist der Schalter **-async** jedoch angegeben, gibt der Befehl sofort zurück, und der Aufrufer kann später **vwait** auf ?varname? ausführen, um das Ergebnis des übergebenen *script* zu erhalten.

```
set t1 [thread::create]
set t2 [thread::create]
thread::send -async $t1 "set a 1" result
thread::send -async $t2 "set b 2" result
for {set i 0} {$i < 2} {incr i} {
    vwait result
}
```

Im Beispiel oben wurde zwei Threads Arbeit zugeführt, und beide wurden angewiesen, dieselbe Variable "result" im aufrufenden Thread zu signalisieren. Der Aufrufer betrat die Ereignisschleife zweimal, um beide Ergebnisse zu erhalten. Die Reihenfolge der empfangenen Ergebnisse kann jedoch schwanken, abhängig von der aktuellen Systemlast, der Art der geleisteten Arbeit und so weiter. Viele Threads können gleichzeitig Skripte zur Ausführung an den Ziel-Thread senden. Alle werden in die Ereigniswarteschlange des Ziel-Threads eingereiht und nach FIFO ausgeführt, vermischt mit optional anderen Ereignissen, die in der Ereigniswarteschlange des Ziel-Threads anstehen. Mit dem optionalen Schalter ?-head? lassen sich Skripte, die in die Ereigniswarteschlange des Threads gestellt werden, an den Kopf statt an das Ende der Warteschlange setzen und damit nach LIFO ausführen.

thread::broadcast *script*

Dieser Befehl übergibt ein *script* zur Ausführung an alle vom Paket erzeugten Threads. Er wartet auf keine Antwort von irgendeinem der Threads. Der sendende Thread (derjenige, der den Befehl thread::broadcast ausführt) sendet das Skript nicht an sich selbst. Jeder empfangende Thread wird angewiesen, die Auswertung des gesendeten Skripts an den Kopf seiner Ereigniswarteschlange zu setzen, vor alle anderen Ereignisse, die auf die Verarbeitung warten.

thread::wait

Damit wird die Ereignisschleife betreten, sodass ein Thread Nachrichten vom Befehl **thread::send** empfangen kann. Dieser Befehl darf nur innerhalb des Skripts verwendet werden, das an **thread::create** übergeben wird. Er soll der allerletzte Befehl im Skript sein. Ist das nicht der Fall, führt der sich beendende Thread die Skriptzeilen nach **thread::wait** weiter aus, was gewöhnlich nicht das Gewünschte und Erwartete ist.

```
set t1 [thread::create {
    #
    # Hier die Initialisierung
    #
    thread::wait ; # Ereignisschleife betreten
}]
```

thread::eval ?-lock mutex? *arg* ?arg ...?

Dieser Befehl reiht die übergebenen Argumente aneinander und wertet das entstehende Skript unter dem Schutz eines Mutex aus. Ist mit dem optionalen Argument ?-lock mutex? kein Mutex angegeben, wird der interne statische Mutex verwendet.

thread::join *id*

Dieser Befehl wartet, bis der Thread mit der ID *id* sich beendet, und gibt dann seinen Beendigungscode zurück. Für Threads, die nicht joinbar sind oder auf die bereits ein anderer Thread gewartet hat, werden Fehler zurückgegeben. Nach dem Verbinden hat das Handle des

Threads den Geltungsbereich verlassen und darf nicht weiter verwendet werden.

thread::configure *id* ?option? ?value? ?...?
Dieser Befehl konfiguriert verschiedene Aspekte auf niedriger Ebene des Threads mit der ID *id* , ähnlich wie der übliche Tcl-Befehl **fconfigure** einige Optionen eines Tcl-Kanals konfiguriert. Derzeit unterstützt werden die Optionen **-eventmark** und **-unwindonerror**. Wird **-eventmark** mit einem Wert größer als 0 (null) angegeben, ist dieser Wert die Höchstzahl asynchron abgegebener Skripte, die für den Thread anstehen dürfen. **thread::send -async** blockiert, bis die Zahl der anstehenden Skripte in der Ereignisschleife unter den Wert von **-eventmark** fällt. Wird **-unwindonerror** mit einem wahren Wert angegeben, bewirkt ein Fehlerergebnis in einem Skript, dass der Thread abgewickelt wird und keine weiteren Skripte mehr auswerten kann.

thread::transfer *id channel*
Damit wird der angegebene *channel* aus dem aktuellen Thread und Interpreter in den Hauptinterpreter des Threads mit der gegebenen *id* verschoben. Nach dem Verschieben hat der aktuelle Interpreter keinen Zugang mehr zum Kanal, der Hauptinterpreter des Ziel-Threads kann ihn aber von nun an verwenden. Der Befehl wartet, bis der andere Thread den Kanal aufgenommen hat. Dadurch ist es möglich, die beteiligten Threads zu verklemmen, indem der andere durch ein synchrones **thread::send** angewiesen wird, einen Kanal an uns zu übertragen. Das weitet sich leicht zu längeren Schleifen von Threads aus, die aufeinander warten. Weitere Einschränkungen: Der betreffende Kanal darf nicht unter mehreren Interpretern geteilt sein, die im sendenden Thread laufen. Das schließt die besonderen Kanäle für Standardeingabe, Standardausgabe und Standardfehler von selbst aus. Wegen der internen Umsetzung des Tcl-Kerns und der Einschränkung, geteilte Kanäle zu übertragen, sind besondere Maßnahmen nötig, wenn Socket-Kanäle übertragen werden, die durch das Annehmen der Verbindung aus den Rückruffunktionen des Befehls **socket** entstanden sind:

```
socket -server _Accept 2200
proc _Accept {s ipaddr port} {
    after idle [list Accept $s $ipaddr $port]
}
proc Accept {s ipaddr port} {
    set tid [thread::create]
    thread::transfer $tid $s
}
```

thread::detach *channel*
Damit wird der angegebene *channel* vom aktuellen Thread und Interpreter gelöst. Danach hat der aktuelle Interpreter keinen Zugang mehr zum Kanal. Der Kanal ist im geparkten Zustand, bis ein anderer (oder derselbe) Thread den Kanal mit **thread::attach** wieder anhängt. Einschränkungen: dieselben wie beim Übertragen geteilter Kanäle mit dem Befehl **thread::transfer**.

thread::attach *channel*
Damit wird der zuvor gelöste *channel* im aktuellen Thread und Interpreter angehängt. Für bereits bestehende Kanäle tut der Befehl nichts, das heißt, es ist kein Fehler, denselben Kanal mehr als einmal anzuhängen. Die erste Operation führt die Operation tatsächlich aus, alle folgenden Operationen tun nur nichts. Der Befehl löst einen Fehler aus, wenn der *channel* nicht in der Liste der gelösten Kanäle und/oder im aktuellen Interpreter gefunden wird.

thread::mutex

Mutexe sind die häufigsten Grundelemente der Thread-Synchronisation. Sie dienen dazu, den Zugang von zwei oder mehr Threads zu einer oder mehr gemeinsamen Ressourcen zu synchronisieren. Dieser Befehl bietet auf Skriptebene Zugang zu exklusiven und/oder rekursiven Mutexen. Exklusive Mutexe kann ein Thread nur einmal sperren, rekursive Mutexe kann derselbe Thread viele Male sperren. Bei rekursiven Mutexen muss die Zahl der Sperr- und Entsperroperationen übereinstimmen, andernfalls wird der Mutex nie freigegeben, was zu verschiedenen Verklemmungen führen würde. Beim Verwenden von Mutexen in einem Programm mit mehreren Threads ist Vorsicht nötig. Unsachgemäße Verwendung von Mutexen kann zu verschiedenen Verklemmungen führen, besonders bei exklusiven Mutexen. Der Befehl **thread::mutex** unterstützt die folgenden Unterbefehle und Optionen:

thread::mutex create ?-recursive?

Erzeugt den Mutex und gibt sein undurchsichtiges Handle zurück. Dieses Handle ist für jeden künftigen Verweis auf den neu erzeugten Mutex zu verwenden. Wurde kein optionales Argument ?-recursive? angegeben, erzeugt der Befehl den exklusiven Mutex. Mit dem Argument ?-recursive? erzeugt der Befehl einen rekursiven Mutex.

thread::mutex destroy *mutex*

Zerstört den *mutex*. Der Mutex soll vor dem Versuch zu zerstören entsperrt sein. Ist der Mutex gesperrt, löst der Befehl einen Tcl-Fehler aus.

thread::mutex lock *mutex*

Sperrt den *mutex*. Das Sperren des exklusiven Mutex kann einen Tcl-Fehler auslösen, wenn versucht wird, denselben Mutex zweimal aus demselben Thread zu sperren. Zwingt die Programmlogik dazu, denselben Mutex zweimal oder öfter aus demselben Thread zu sperren (das kann bei rekursiven Prozeduraufrufen geschehen), sind rekursive Mutexe in Betracht zu ziehen.

thread::mutex unlock *mutex*

Entsperrt den *mutex*, sodass ein anderer Thread ihn wieder sperren kann. Der Versuch, den bereits entsperrten Mutex zu entsperren, löst einen Tcl-Fehler aus.

thread::rwmutex

Dieser Befehl erzeugt Mutexe für viele Leser und einen Schreiber. Lese-/Schreib-Mutexe erlauben, den Zugang zu einer gemeinsamen Ressource günstiger zu serialisieren. Wird eine gemeinsame Ressource meist gelesen und selten verändert, lässt sich mit Lese-/Schreib-Mutexen statt exklusiver oder rekursiver Mutexe etwas an Leistung gewinnen. Zum Lesen der Ressource soll der Thread eine Lesesperre auf die Ressource erlangen. Die Lesesperre ist nicht exklusiv, das heißt, mehr als ein Thread kann eine Lesesperre auf dieselbe Ressource erlangen, ohne auf andere Leser zu warten. Zum Ändern der Ressource muss ein Thread jedoch eine exklusive Schreibsperre erlangen. Diese Sperre hindert alle Threads wirksam daran, die Lesesperre zu erlangen, während der Schreiber-Thread die Ressource verändert. Erst nachdem die Schreibsperre freigegeben wurde, darf die Ressource wieder lesend gesperrt werden. Der Befehl **thread::rwmutex** unterstützt die folgenden Unterbefehle und Optionen:

thread::rwmutex create

Erzeugt den Lese-/Schreib-Mutex und gibt sein undurchsichtiges Handle zurück. Dieses Handle ist für jeden künftigen Verweis auf den neu erzeugten Mutex zu verwenden.

thread::rwmutex destroy *mutex*
Zerstört den Lese-/Schreib-*mutex*. Ist der Mutex bereits gesperrt, löst der Versuch, ihn zu zerstören, einen Tcl-Fehler aus.

thread::rwmutex rlock *mutex*
Sperrt den *mutex* zum Lesen. Mehr als ein Thread darf denselben *mutex* gleichzeitig lesend sperren.

thread::rwmutex wlock *mutex*
Sperrt den *mutex* zum Schreiben. Nur ein Thread darf denselben *mutex* gleichzeitig schreibend sperren. Der Versuch, denselben *mutex* zweimal aus demselben Thread schreibend zu sperren, löst einen Tcl-Fehler aus.

thread::rwmutex unlock *mutex*
Entsperrt den *mutex*, sodass ein anderer Thread ihn wieder sperren kann. Der Versuch, einen bereits entsperrten *mutex* zu entsperren, löst einen Tcl-Fehler aus.

thread::cond
Dieser Befehl bietet auf Skriptebene Zugang zu Bedingungsvariablen. Eine Bedingungsvariable schafft eine sichere Umgebung, in der das Programm eine Bedingung prüfen, bei falschem Ergebnis darauf schlafen und geweckt werden kann, wenn sie wahr geworden sein könnte. Eine Bedingungsvariable wird stets zusammen mit einem exklusiven Mutex verwendet. Wird versucht, eine andere Art von Mutex zusammen mit der Bedingungsvariable zu verwenden, wird ein Tcl-Fehler ausgelöst. Der Befehl unterstützt die folgenden Unterbefehle und Optionen:

thread::cond create
Erzeugt die Bedingungsvariable und gibt ihr undurchsichtiges Handle zurück. Dieses Handle ist für jeden künftigen Verweis auf die neu erzeugte Bedingungsvariable zu verwenden.

thread::cond destroy *cond*
Zerstört die Bedingungsvariable *cond*. Äußerste Vorsicht ist nötig, dass niemand die Bedingungsvariable verwendet (das heißt, auf sie wartet), andernfalls können unerwartete Fehler geschehen.

thread::cond notify *cond*
Weckt alle Threads, die auf die Bedingungsvariable *cond* warten.

thread::cond wait *cond mutex* ?ms?
Dieser Befehl dient dazu, die Programmausführung anzuhalten, bis die Bedingungsvariable *cond* signalisiert wurde oder der optionale Timer abgelaufen ist. Der exklusive *mutex* muss beim Eintritt in diesen Befehl vom aufrufenden Thread gesperrt sein. Ist der Mutex nicht gesperrt, wird ein Tcl-Fehler ausgelöst. Während des Wartens auf *cond* gibt der Befehl *mutex* frei. Bevor er zum aufrufenden Thread zurückkehrt, erlangt der Befehl *mutex* wieder. Das Entsperren von *mutex* und das Warten auf die Bedingungsvariable *cond* geschehen atomar. Die Befehlsoption **ms**, falls angegeben, muss eine ganze Zahl sein, die das Zeitintervall in Millisekunden angibt, das der Befehl auf ein Signal wartet. Andernfalls wartet der Befehl unbegrenzt auf die Benachrichtigung der Bedingung. In Programmen mit mehreren Threads gibt es viele Situationen, in denen ein Thread auf ein Ereignis warten muss, bevor er fortfahren darf. Das wird gewöhnlich erreicht, indem eine Bedingung unter dem Schutz des Mutex wiederholt geprüft und auf die Bedingungsvariable gewartet wird, bis die Bedingung wahr ergibt:

```
    set mutex [thread::mutex create]
    set cond  [thread::cond  create]

    thread::mutex lock $mutex
    while {<some_condition_is_true>} {
        thread::cond wait $cond $mutex
    }
    # Arbeit unter dem Schutz des Mutex
    thread::mutex unlock $mutex
```

Die Bedingung muss wiederholt geprüft werden, weil die Bedingungsvariable signalisiert werden kann, ohne dass die Bedingung sich tatsächlich geändert hat (zum Beispiel unechtes Aufwecken von Threads).

DISKUSSION

Das grundlegende Threading-Modell in Tcl ist, dass es einen oder mehrere Tcl-Interpreter je Thread geben kann, jeder Tcl-Interpreter aber nur von dem einen Thread verwendet werden soll, der ihn erzeugt hat. Eine Abstraktion "gemeinsamer Speicher" ist in Tcl ungeschickt bereitzustellen, weil Tcl Annahmen über den Besitz von Variablen und Daten macht. Darum unterstützt diese Erweiterung eine einfache Form des Threading, bei der der Haupt-Thread mehrere Hintergrund- oder "Arbeits"-Threads verwalten kann. Ein ereignisgetriebener Server kann zum Beispiel Aufträge an Arbeits-Threads weitergeben und dann auf Antworten der Arbeits-Threads oder auf neue Client-Aufträge warten. Alles läuft durch die gemeinsame Tcl-Ereignisschleife, sodass die Nachrichtenübermittlung zwischen Threads natürlich mit ereignisgetriebener Ein-/Ausgabe, **vwait** auf Variablen und so weiter zusammenwirkt. Für die Übertragung großer Informationsmengen lassen sich Kanäle zwischen den Threads verschieben.

Für fortgeschrittene Skripte mit mehreren Threads wird auch der Zugang auf Skriptebene zu zwei grundlegenden Synchronisationsgrundelementen unterstützt, Mutex und Bedingungsvariablen.

tpool

Stichwörter: tpool, thread, threadpool

NAME

tpool - Teil der Tcl-Threading-Erweiterung für Pools von Arbeits-Threads

ÜBERSICHT

package require Tcl 9.0
package require thread ?**3.0**?
tpool::create ?*options*?
tpool::names
tpool::post ?*-detached*? ?*-nowait*? *tpool script*
tpool::wait *tpool joblist* ?*varname*?
tpool::cancel *tpool joblist* ?*varname*?
tpool::get *tpool job*
tpool::preserve *tpool*

tpool::release *tpool*
tpool::suspend *tpool*
tpool::resume *tpool*

BESCHREIBUNG

Dieses Paket erzeugt und verwaltet Pools von Arbeits-Threads. Es erlaubt, Aufträge an Arbeits-Threads zu geben und auf ihren Abschluss zu warten. Die Umsetzung des Thread-Pools kennt die Tcl-Ereignisschleife. Das heißt: Immer wenn ein Aufrufer auf ein Ereignis warten muss (ein Auftrag wird abgeschlossen, oder ein Arbeits-Thread wird untätig oder ist initialisiert), betritt die Umsetzung die Ereignisschleife und erlaubt die Bearbeitung anderer anstehender Datei-, Timer- oder sonst unterstützter Ereignisse.

BEFEHLE

tpool::create ?options?
Dieser Befehl erzeugt einen neuen Thread-Pool. Er nimmt mehrere Optionen als Schlüssel-Wert-Paare entgegen. Die Optionen dienen dazu, einige Parameter des Thread-Pools einzustellen. Der Befehl gibt die ID des neu erzeugten Thread-Pools zurück. Folgende Optionen werden unterstützt:

-minworkers *number*
Mindestzahl der Arbeits-Threads, die diese Instanz des Thread-Pools braucht. Beim Erzeugen des Thread-Pools legt die Umsetzung so viele Arbeits-Threads im Voraus an und hält mindestens diese Zahl während der Lebensdauer der Instanz am Leben. Der Vorgabewert dieses Parameters ist 0 (null). Das bedeutet, dass ein neuer Thread-Pool anfangs keine Arbeits-Threads hat. Alle Arbeits-Threads werden bei Bedarf von Aufrufern gestartet, die den Befehl **tpool::post** ausführen und Aufträge in die Auftragswarteschlange stellen.

-maxworkers *number*
Höchstzahl der Arbeits-Threads, die diese Instanz des Thread-Pools zulässt. Liegt ein neuer Auftrag an und ist kein untätiger Arbeits-Thread verfügbar, versucht die Umsetzung, einen neuen Arbeits-Thread zu erzeugen. Ist die Zahl der verfügbaren Arbeits-Threads kleiner als die angegebene Zahl, startet ein neuer Arbeits-Thread. Der Aufrufer betritt automatisch die Ereignisschleife und wartet, bis der Arbeits-Thread initialisiert ist. Ist die Zahl der verfügbaren Arbeits-Threads jedoch gleich der angegebenen Zahl, betritt der Aufrufer die Ereignisschleife und wartet, bis der erste Arbeits-Thread untätig und damit bereit ist, den Auftrag auszuführen. Der Vorgabewert dieses Parameters ist 4 (vier). Das bedeutet, dass die Instanz des Thread-Pools höchstens 4 Arbeits-Threads zulässt, die Aufträge ausführen oder untätig auf neue Aufträge in der Auftragswarteschlange warten.

-idletime *seconds*
Zeit in Sekunden, die ein untätiger Arbeits-Thread auf einen Auftrag in der Auftragswarteschlange wartet. Trifft in diesem Intervall kein Auftrag ein und läuft die Zeit ab, prüft der Arbeits-Thread die Zahl der gerade verfügbaren Arbeits-Threads und beendet sich, wenn die Zahl höher ist als die durch die Option **minthreads** gesetzte Zahl. Ist ein **exitscript** definiert, führt der sich beendende Arbeits-Thread zuerst das Skript aus und beendet sich dann. Fehler aus dem Beendigungsskript, falls vorhanden, werden ignoriert. Der untätige Arbeits-Thread bedient die Ereignisschleife nicht. Wird der Arbeits-Thread jedoch in die Ereignisschleife gelegt, indem **vwait** oder andere verwandte Tcl-Befehle ausgewertet werden, ist der Arbeits-Thread nicht untätig, und der Untätigkeits-Timer wird nicht berücksichtigt. Der

Vorgabewert dieser Option ist unbestimmt.

-initcmd *script*
Setzt ein Tcl-Skript, mit dem ein neuer Arbeits-Thread initialisiert wird. Das dient gewöhnlich dazu, im Worker Pakete und Befehle zu laden, Vorgabevariablen zu setzen, Namespaces zu erzeugen und Ähnliches. Läuft das übergebene Skript in einen Tcl-Fehler, wird der Worker nicht erzeugt, und der auslösende Befehl (**tpool::create** oder **tpool::post**) löst einen Fehler aus. Der Vorgabewert dieser Option ist unbestimmt, der Tcl-Interpreter des Arbeits-Threads enthält also nur den anfänglichen Satz von Tcl-Befehlen.

-exitcmd *script*
Setzt ein Tcl-Skript, das läuft, wenn der untätige Arbeits-Thread sich beendet. Das dient gewöhnlich dazu, den Zustand des Arbeits-Threads aufzuräumen, reservierte Ressourcen freizugeben, Speicher aufzuräumen und Ähnliches. Der Vorgabewert dieser Option ist unbestimmt, beim Beenden des Arbeits-Threads läuft also kein Tcl-Skript.

tpool::names
Dieser Befehl gibt eine Liste der IDs von Thread-Pools zurück, die mit dem Befehl **tpool::create** erzeugt wurden. Wurden keine Thread-Pools gefunden, gibt der Befehl eine leere Liste zurück.

tpool::post ?-detached? ?-nowait? *tpool script*
Dieser Befehl sendet ein *script* zur Ausführung an den Ziel-Thread-Pool *tpool*. Das Skript wird im ersten verfügbaren untätigen Arbeits-Thread ausgeführt. Ist kein untätiger Arbeits-Thread verfügbar, erzeugt der Befehl einen neuen, betritt die Ereignisschleife und bedient Ereignisse, bis der neu erzeugte Thread initialisiert ist. Ist die aktuelle Zahl der Arbeits-Threads gleich der Höchstzahl, wie sie beim Erzeugen des Thread-Pools festgelegt wurde, betritt der Befehl die Ereignisschleife und bedient Ereignisse, während er darauf wartet, dass einer der Arbeits-Threads untätig wird. Ist das optionale Argument ?-nowait? angegeben, legt der Befehl den Auftrag nur in die Auftragswarteschlange des Pools, ohne auf einen untätigen Worker zu warten oder einen neuen zu erzeugen, auch wenn der Pool noch Platz für weitere Threads hätte. Der Befehl gibt die ID des abgegebenen Auftrags zurück. Diese ID wird für die nachfolgenden Befehle **tpool::wait**, **tpool::get** und **tpool::cancel** verwendet, um auf das Ergebnis des abgegebenen Skripts zu warten und es abzuholen oder den abgegebenen Auftrag abzubrechen. Ist das optionale Argument ?-detached? angegeben, gibt der Befehl einen losgelösten Auftrag ab. Ein losgelöster Auftrag lässt sich nicht abbrechen und nicht abwarten und ist nicht durch die Auftrags-ID bezeichnet. Wird der Thread-Pool *tpool* nicht in der Liste der aktiven Thread-Pools gefunden, löst der Befehl einen Fehler aus. Der Fehler wird auch ausgelöst, wenn der neu erzeugte Arbeits-Thread sich nicht initialisieren lässt.

tpool::wait *tpool joblist* ?varname?
Dieser Befehl wartet auf einen oder viele Aufträge, deren Auftrags-IDs in *joblist* stehen, bis sie von den Arbeits-Threads verarbeitet sind. Ist keiner der angegebenen Aufträge fertig, betritt der Befehl die Ereignisschleife, bedient Ereignisse und wartet auf den ersten fertigen Auftrag. Der Befehl gibt die Liste der abgeschlossenen Auftrags-IDs zurück. Ist die optionale Variable ?varname? angegeben, wird sie auf die Liste der Aufträge in *joblist* gesetzt, die noch anstehen. Wird der Thread-Pool *tpool* nicht in der Liste der aktiven Thread-Pools gefunden, löst der Befehl einen Fehler aus.

tpool::cancel *tpool joblist* ?varname?
Dieser Befehl bricht die zuvor an den Pool *tpool* abgegebenen Aufträge ab, die *joblist* nennt.

Das Abbrechen gelingt nur für einen Auftrag, der noch auf die Verarbeitung wartet. Wird der Auftrag bereits von einem der Arbeits-Threads ausgeführt, wird er nicht abgebrochen. Der Befehl gibt die Liste der abgebrochenen Auftrags-IDs zurück. Ist die optionale Variable ?varname? angegeben, wird sie auf die Liste der Aufträge in *joblist* gesetzt, die nicht abgebrochen wurden. Wird der Thread-Pool *tpool* nicht in der Liste der aktiven Thread-Pools gefunden, löst der Befehl einen Fehler aus.

tpool::get *tpool job*

Dieser Befehl holt das Ergebnis des zuvor abgegebenen *job*. Nur Ergebnisse von Aufträgen, auf die mit dem Befehl **tpool::wait** gewartet wurde, lassen sich abholen. Führte die Ausführung des Skripts zu einem Fehler, löst der Befehl den Fehler aus und aktualisiert die Variablen **errorInfo** und **errorCode** entsprechend. Wird der Pool *tpool* nicht in der Liste der Thread-Pools gefunden, löst der Befehl einen Fehler aus. Ist der Auftrag *job* nicht zur Abholung bereit, weil ein Arbeits-Thread ihn gerade ausführt, löst der Befehl einen Fehler aus.

tpool::preserve *tpool*

Jeder Aufruf dieses Befehls erhöht den Referenzzähler des Thread-Pools *tpool* um eins (1). Der Befehl gibt den Wert des Referenzzählers nach der Erhöhung zurück. Durch das Erhöhen des Referenzzählers zeigt der Aufrufer an, dass er die Ressource längere Zeit verwenden will.

tpool::release *tpool*

Jeder Aufruf dieses Befehls verringert den Referenzzähler des Thread-Pools *tpool* um eins (1). Der Befehl gibt den Wert des Referenzzählers nach der Verringerung zurück. Erreicht der Referenzzähler null (0), wird der Thread-Pool *tpool* zur Beendigung vorgemerkt. Nachdem der Befehl **tpool::release** null zurückgibt, darf der Thread-Pool nicht mehr verwendet werden. Das Handle *tpool* verlässt den Geltungsbereich und darf nicht weiter verwendet werden. Jeder folgende Verweis auf dasselbe Handle des Thread-Pools führt zu einem Tcl-Fehler.

tpool::suspend *tpool*

Hält die Verarbeitung von Arbeit auf dieser Warteschlange an. Alle Worker des Pools werden pausiert, weitere Arbeit kann dem Pool aber hinzugefügt werden. Das Hinzufügen weiterer Arbeit erhöht die Zahl der Worker nicht dynamisch, weil die Verarbeitung des Pools angehalten ist. Die Zahl der Worker bleibt bei dem Stand, der vor dem Anhalten der Worker-Tätigkeit bestand. Muss eine bestimmte Zahl von Arbeits-Threads sichergestellt werden, dient die Option **minworkers** des Befehls **tpool::create**.

tpool::resume *tpool*

Nimmt die Verarbeitung von Arbeit auf dieser Warteschlange wieder auf. Alle pausierten (angehaltenen) Worker dürfen sich Arbeit aus dem Pool holen. Das Wiederaufnehmen des Poolbetriebs lässt nur bereits erzeugte Worker fortfahren. Es erzeugt keine zusätzlichen Arbeits-Threads, um die in die Arbeitswarteschlange des Pools gestellte Arbeit zu erledigen.

DISKUSSION

Der Thread-Pool ist eines der häufigsten Threading-Muster, wenn Serveranwendungen eine große Zahl verhältnismäßig kleiner Aufgaben bearbeiten. Ein sehr einfaches Modell für eine Serveranwendung würde bei jedem eintreffenden Auftrag einen neuen Thread erzeugen und den Auftrag in dem neuen Thread bedienen. Ein Nachteil dieses Vorgehens ist, dass der Aufwand, für jeden Auftrag einen neuen Thread zu erzeugen, erheblich ist. Ein Server, der für jeden Auftrag einen neuen Thread erzeugt, würde mehr Zeit und mehr Systemressourcen für das Erzeugen und Zerstören von Threads aufwenden als für die eigentlichen Benutzeraufträge. Neben dem Aufwand für das Erzeugen und Zerstören von Threads verbrauchen aktive Threads

Systemressourcen. Zu viele Threads können dazu führen, dass dem System der Speicher ausgeht oder es durch übermäßigen Speicherverbrauch zusammenbricht.

Ein Thread-Pool löst sowohl das Problem des Aufwands im Lebenszyklus von Threads als auch das Problem der Ressourcenüberlastung. Werden Threads für mehrere Aufgaben wiederverwendet, verteilt sich der Aufwand der Thread-Erzeugung auf viele Aufgaben. Als Zugabe entfällt die Verzögerung durch die Thread-Erzeugung, weil der Thread schon besteht, wenn ein Auftrag eintrifft. Der Auftrag kann also sofort bedient werden. Wird die Zahl der Threads im Thread-Pool passend eingestellt, lässt sich auch die Ressourcenüberlastung vermeiden, indem jeder Auftrag warten muss, bis ein Thread frei ist, ihn zu verarbeiten.

tsv

Stichwörter: tsv, locking, synchronization, thread shared data, threads

NAME

tsv - Teil der Tcl-Threading-Erweiterung für Daten, die Threads gemeinsam sind

ÜBERSICHT

package require Tcl 9.0
package require thread ?**3.0**?
tsv::names ?*pattern*?
tsv::object *varname element*
tsv::set *varname element* ?*value*?
tsv::get *varname element* ?*namedvar*?
tsv::unset *varname* ?*element*?
tsv::exists *varname element*
tsv::pop *varname element*
tsv::move *varname oldname newname*
tsv::incr *varname element* ?*count*?
tsv::append *varname element value* ?*value* ...?
tsv::lock *varname arg* ?*arg* ...?
tsv::handlers
tsv::lappend *varname element value* ?*value* ...?
tsv::linsert *varname element index value* ?*value* ...?
tsv::lreplace *varname element first last* ?*value* ...?
tsv::llength *varname element*
tsv::lindex *varname element* ?*index*?
tsv::lrange *varname element from to*
tsv::lsearch *varname element* ?*options*? *pattern*
tsv::lset *varname element index* ?*index* ...? *value*
tsv::lpop *varname element* ?*index*?
tsv::lpush *varname element* ?*index*?
tsv::array set *varname list*
tsv::array get *varname* ?*pattern*?
tsv::array names *varname* ?*pattern*?
tsv::array size *varname*
tsv::array reset *varname list*

tsv::array bind *varname handle*
tsv::array unbind *varname*
tsv::array isbound *varname*
tsv::keyldel *varname keylist key*
tsv::keylget *varname keylist key* ?*retvar*?
tsv::keylkeys *varname keylist* ?*key*?
tsv::keylset *varname keylist key value* ?*key value..*?

BESCHREIBUNG

Dieser Abschnitt beschreibt Befehle, die gemeinsame Thread-Variablen umsetzen. Eine gemeinsame Thread-Variable ähnelt einem Tcl-Array sehr, entsteht aber im Gegensatz zu einem Tcl-Array im gemeinsamen Speicher und kann von vielen Threads gleichzeitig verwendet werden. Eine wichtige Eigenschaft der gemeinsamen Thread-Variable ist, dass jeder Zugriff auf die Variable intern durch einen Mutex geschützt ist, sodass der Skriptautor die Variable nicht selbst sperren muss.

Gemeinsame Thread-Variablen sind an keinen Thread ausdrücklich gebunden. Das heißt: Beendet sich ein Thread, der gemeinsame Thread-Variablen erzeugt hat, werden die Variable und der zugehörige Speicher nicht gelöscht und nicht zurückgewonnen. Der Benutzer muss die Variable ausdrücklich löschen, um den von der Variable verbrauchten Speicher zurückzugewinnen.

ELEMENTBEFEHLE

tsv::names ?pattern?
Gibt die Namen gemeinsamer Variablen zurück, die auf das optionale ?pattern? passen, oder alle bekannten Variablen, wenn das Muster entfällt.

tsv::object *varname element*
Erzeugt einen Objektzugriffsbefehl für das *element* in der gemeinsamen Variable *varname*. Mit diesem Befehl lassen sich die meisten anderen Befehle für gemeinsame Variablen als Methoden des Element-Objektbefehls anwenden. Der Objektbefehl wird automatisch gelöscht, wenn das Element, auf das dieser Befehl zeigt, gelöscht wird.

```
% tsv::set foo bar "A shared string"
% set string [tsv::object foo bar]
% $string append " appended"
=> A shared string appended
```

tsv::set *varname element* ?value?
Setzt den Wert des *element* in der gemeinsamen Variable *varname* auf *value* und gibt den Wert an den Aufrufer zurück. *value* darf entfallen; dann gibt der Befehl den aktuellen Wert des Elements zurück. Kann das Element nicht gefunden werden, wird ein Fehler ausgelöst.

tsv::get *varname element* ?namedvar?
Holt den Wert des *element* aus der gemeinsamen Variable *varname*. Ist das optionale Argument *namedvar* angegeben, wird der Wert in der benannten Variable gespeichert. Der Rückgabewert des Befehls hängt vom Vorhandensein des optionalen Arguments *namedvar* ab. Entfällt das Argument und kann das angeforderte Element im gemeinsamen Array nicht gefunden werden, löst der Befehl einen Fehler aus. Ist das optionale Argument jedoch auf der Befehlszeile angegeben, gibt der Befehl wahr (1) zurück, wenn das Element gefunden wird,

oder falsch (0), wenn das Element nicht gefunden wird.

tsv::unset *varname* ?element?
Löscht das *element* aus der gemeinsamen Variable *varname*. Ist das optionale Element nicht angegeben, löscht der Befehl die Variable.

tsv::exists *varname element*
Prüft, ob das *element* in der gemeinsamen Variable *varname* besteht, und gibt wahr (1) zurück, wenn ja, oder falsch (0), wenn nicht.

tsv::pop *varname element*
Gibt den Wert des *element* in der gemeinsamen Variable *varname* zurück und löscht das Element, alles in einer atomaren Operation.

tsv::move *varname oldname newname*
Benennt das Element *oldname* in *newname* in der gemeinsamen Variable *varname* um. Das führt wirksam eine Folge von Holen, Löschen und Setzen aus, aber alles in einem atomaren Schritt.

tsv::incr *varname element* ?count?
Ähnlich dem üblichen Tcl-Befehl **incr**, erhöht aber den Wert des *element* in der gemeinsamen Variable *varname* statt den der Tcl-Variable.

tsv::append *varname element value* ?value ...?
Ähnlich dem üblichen Tcl-Befehl **append**, hängt aber einen oder mehrere Werte an das *element* in der gemeinsamen Variable *varname* an statt an die Tcl-Variable.

tsv::lock *varname arg* ?arg ...?
Dieser Befehl reiht die übergebenen Argumente aneinander und wertet das entstehende Skript unter dem Schutz des internen Mutex aus. Während der Auswertung des Skripts ist die gesamte gemeinsame Variable gesperrt. Für Befehle auf der gemeinsamen Variable innerhalb des Skripts ist die interne Sperre abgeschaltet, sodass keine Verklemmung entstehen kann. Es ist auch erlaubt, die gemeinsame Variable aus dem Skript heraus zu löschen. Die gemeinsame Variable wird automatisch erzeugt, wenn sie beim ersten Sperren nicht bestand.

```
% tsv::lock foo {
    tsv::lappend foo bar 1
    tsv::lappend foo bar 2
    puts stderr [tsv::set foo bar]
    tsv::unset foo
}
```

tsv::handlers
Gibt die Namen aller Handler für dauerhaften Speicher zurück, die zur Übersetzungszeit eingeschaltet waren. Einzelheiten stehen bei **ARRAYBEFEHLE**.

LISTENBEFEHLE

Diese Befehle ähneln dem gleichnamigen Tcl-Befehl. Der Unterschied ist, dass sie auf Elementen gemeinsamer Arrays arbeiten.

tsv::lappend *varname element value* ?value ...?
Ähnlich dem üblichen Tcl-Befehl **lappend**, hängt aber einen oder mehrere Werte an das *element* in der gemeinsamen Variable *varname* an statt an die Tcl-Variable.

tsv::linsert *varname element index value* ?value ...?

Ähnlich dem üblichen Tcl-Befehl **linsert**, fügt aber einen oder mehrere Werte an der Listenposition *index* in das *element* der gemeinsamen Variable *varname* ein statt in die Tcl-Variable.

tsv::lreplace *varname element first last* ?value ...?
Ähnlich dem üblichen Tcl-Befehl **lreplace**, ersetzt aber einen oder mehrere Werte zwischen den Positionen *first* und *last* im *element* der gemeinsamen Variable *varname* statt in der Tcl-Variable.

tsv::llength *varname element*
Ähnlich dem üblichen Tcl-Befehl **llength**, gibt aber die Länge des *element* in der gemeinsamen Variable *varname* zurück statt die der Tcl-Variable.

tsv::lindex *varname element* ?index?
Ähnlich dem üblichen Tcl-Befehl **lindex**, gibt aber den Wert an der Listenposition *index* des *element* aus der gemeinsamen Variable *varname* zurück statt aus der Tcl-Variable.

tsv::lrange *varname element from to*
Ähnlich dem üblichen Tcl-Befehl **lrange**, gibt aber die Werte zwischen den Listenpositionen *from* und *to* aus dem *element* in der gemeinsamen Variable *varname* zurück statt aus der Tcl-Variable.

tsv::lsearch *varname element* ?options? *pattern*
Ähnlich dem üblichen Tcl-Befehl **lsearch**, durchsucht aber das *element* in der gemeinsamen Variable *varname* statt die Tcl-Variable.

tsv::lset *varname element index* ?index ...? *value*
Ähnlich dem üblichen Tcl-Befehl **lset**, setzt aber das *element* in der gemeinsamen Variable *varname* statt die Tcl-Variable.

tsv::lpop *varname element* ?index?
Ähnlich dem üblichen Tcl-Befehl **lindex**, schneidet aber zusätzlich zum Zurückgeben den Wert in einer atomaren Operation aus dem *element* der gemeinsamen Variable *varname* heraus. Im Gegensatz zum Tcl-Befehl **lindex** gibt dieser Befehl dem Aufrufer keinen Wert zurück.

tsv::lpush *varname element* ?index?
Dieser Befehl tut das Gegenteil des Befehls **tsv::lpop**. Wie sein Gegenstück gibt er dem Aufrufer keinen Wert zurück.

ARRAYBEFEHLE

Dieser Befehl unterstützt die meisten Optionen des üblichen Tcl-Befehls **array**. Zusätzlich erlaubt er die Bindung einer gemeinsamen Variable an Datenbanken für dauerhaften Speicher. Derzeit unterstützt werden als dauerhafte Optionen das bekannte GNU Gdbm und LMDB. Diese Optionen müssen zur Übersetzungszeit des Pakets ausgewählt werden. Die Umsetzung bietet Einhängpunkte, um bei Bedarf weitere Schichten für die Dauerhaftigkeit zu definieren.

tsv::array set *varname list*
Tut dasselbe wie der übliche Tcl-Befehl **array set**.

tsv::array get *varname* ?pattern?
Tut dasselbe wie der übliche Tcl-Befehl **array get**.

tsv::array names *varname* ?pattern?
Tut dasselbe wie der übliche Tcl-Befehl **array names**.

tsv::array size *varname*

Tut dasselbe wie der übliche Tcl-Befehl **array size**.

tsv::array reset *varname list*
Tut dasselbe wie der übliche Tcl-Befehl **array set**, leert aber *varname* und setzt die neuen Werte aus der Liste atomar.

tsv::array bind *varname handle*
Bindet *varname* an den dauerhaften Speicher *handle*. Das Format von *handle* ist <handler>:<address>, wobei <handler> "gdbm" für GNU Gdbm und "lmdb" für LMDB ist und <address> der Pfad zur Datenbankdatei.

tsv::array unbind *varname*
Löst das gemeinsame *array* von seinem gebundenen dauerhaften Speicher.

tsv::array isbound *varname*
Gibt wahr (1) zurück, wenn das gemeinsame *varname* an einen dauerhaften Speicher gebunden ist, oder null (0), wenn nicht.

BEFEHLE FÜR SCHLÜSSELLISTEN

Die Befehle für Schlüssellisten sind dem Paket TclX entlehnt. Schlüssellisten bieten einen strukturierten Datentyp, der auf üblichen Tcl-Listen aufbaut. Das ist eine Funktionalität ähnlich den Structs in der Programmiersprache C.

Eine Schlüsselliste ist eine Liste, in der jedes Element ein Paar aus Schlüssel und Wert enthält. Diese Elementpaare werden selbst als Listen gespeichert, wobei der Schlüssel das erste Element der Liste ist und der Wert das zweite. Die Schlüssel-Wert-Paare heißen Felder. Das ist ein Beispiel einer Schlüsselliste:

```
    {{NAME  {Frank  Zappa}} {JOB {musician and composer}}}
```

Felder dürfen Unterfelder enthalten; das Trennzeichen ist `.'. Unterfelder sind tatsächlich Felder, deren Wert eine weitere Schlüsselliste ist. Die folgende Liste hat also die Felder der obersten Ebene ID und NAME und die Unterfelder NAME.FIRST und NAME.LAST:

```
    {ID 106} {NAME {{FIRST Frank} {LAST Zappa}}}
```

Die Rekursionstiefe der Unterfelder ist nicht begrenzt, sodass sich komplexe Datenstrukturen bauen lassen. Schlüssellisten werden über eine Reihe von Befehlen aufgebaut und verwendet. Alle Befehle zur Verwaltung von Schlüssellisten nehmen den Namen der Variable, die die Schlüsselliste enthält, als Argument (das heißt, als Verweis übergeben), statt die Liste unmittelbar zu übergeben.

tsv::keyldel *varname keylist key*
Löscht das durch *key* angegebene Feld aus der Schlüsselliste *keylist* in der gemeinsamen Variable *varname*. Das entfernt sowohl den Schlüssel als auch den Wert aus der Schlüsselliste.

tsv::keylget *varname keylist key* ?retvar?
Gibt den *key* zugeordneten Wert aus der Schlüsselliste *keylist* in der gemeinsamen Variable *varname* zurück. Ist das optionale *retvar* nicht angegeben, wird der Wert als Ergebnis des Befehls zurückgegeben. In diesem Fall entsteht ein Fehler, wenn der Schlüssel in der Liste nicht gefunden wird. Ist *retvar* angegeben und steht *key* in der Liste, wird der Wert in der Variable *retvar* zurückgegeben, und der Befehl gibt 1 zurück, wenn der Schlüssel in der Liste vorhanden war. Steht *key* nicht in der Liste, gibt der Befehl 0 zurück, und *retvar* bleibt

unverändert. Ist {} für *retvar* angegeben, wird der Wert nicht zurückgegeben, sodass der Tcl-Programmierer feststellen kann, ob ein *key* in einer Schlüsselliste vorhanden ist, ohne als Nebenwirkung eine Variable zu setzen.

tsv::keylkeys *varname keylist* ?key?
Gibt die Liste der Schlüssel in der Schlüsselliste *keylist* in der gemeinsamen Variable *varname* zurück. Ist *key* angegeben, ist es der Name eines Schlüsselfelds, dessen Unterfeldschlüssel geholt werden sollen.

tsv::keylset *varname keylist key value* ?key value..?
Setzt den *key* zugeordneten Wert in der Schlüsselliste *keylist* auf *value*. Besteht die *keylist* nicht, wird sie erzeugt. Steht *key* derzeit nicht in der Liste, wird es hinzugefügt. Besteht es bereits, ersetzt *value* den bestehenden Wert. Mehrere Schlüsselwörter und Werte dürfen angegeben werden, falls gewünscht.

DISKUSSION

Die aktuelle Umsetzung gemeinsamer Thread-Variablen erlaubt den leichten und bequemen Zugang zu Daten, die verschiedene Threads gemeinsam haben. Intern werden die Daten in Tcl-Objekten gespeichert, und alle Befehle des Pakets arbeiten auf der internen Datendarstellung, wodurch das Shimmering verringert und die Leistung verbessert wird. Besondere Sorgfalt galt der Zusicherung, dass alle Objektdaten richtig gesperrt und tief kopiert werden, wenn Objekte zwischen Threads verschoben werden.

Wegen des internen Entwurfs des Tcl-Kerns gibt es leider keine Möglichkeit, gemeinsame Variablen vollständig in die Tcl-Syntax einzubinden. Jeder Zugriff auf gemeinsame Daten muss mit den mitgelieferten Paketbefehlen erfolgen. Auch Variablen-Traces werden nicht unterstützt. Trotzdem überwiegen die Vorteile der leichten, einfachen und sicheren Bearbeitung gemeinsamer Daten die auferlegten Grenzen.

DANKSAGUNGEN

Gemeinsame Thread-Variablen sind von der Schnittstelle nsv im AOLserver angeregt, einem hoch skalierbaren Webserver von America Online.

ttrace

Stichwörter: ttrace, command tracing, introspection

NAME

ttrace - Trace-basierte Initialisierung von Interpretern

ÜBERSICHT

package require Tcl 9.0
package require thread ?**3.0**?
ttrace::eval *arg* ?*arg* ...?
ttrace::enable
ttrace::disable
ttrace::cleanup
ttrace::update ?*epoch*?
ttrace::getscript
ttrace::atenable *cmd arglist body*

ttrace::atdisable *cmd arglist body*
ttrace::addtrace *cmd arglist body*
ttrace::addscript *name body*
ttrace::addresolver *cmd arglist body*
ttrace::addcleanup *body*
ttrace::addentry *cmd var val*
ttrace::getentry *cmd var*
ttrace::getentries *cmd* ?*pattern*?
ttrace::delentry *cmd*
ttrace::preload *cmd*

BESCHREIBUNG

Dieses Paket schafft einen Rahmen, um den Zustand des Interpreters bei Bedarf über Threads einer Anwendung mit mehreren Threads hinweg nachzubilden. Es stützt sich auf die Mechanik der Tcl-Befehlstraces und auf den Tcl-Befehl **unknown** und dessen Mechanismus.

Das Paket verlangt die Tcl-Threading-Erweiterung, lässt sich aber auch eigenständig im AOLserver verwenden, einem skalierbaren Webserver von America Online.

Kurz gefasst ein kleines Beispiel, das die Verwendung von ttrace mit der Tcl-Threading-Erweiterung zeigt:

```
% package require ttrace
3.0.1

% set t1 [thread::create {package require ttrace; thread::wait}]
tid0x1802800

% ttrace::eval {proc test args {return test-[thread::id]}}
% thread::send $t1 test
test-tid0x1802800

% set t2 [thread::create {package require ttrace; thread::wait}]
tid0x1804000

% thread::send $t2 test
test-tid0x1804000
```

Wie oben zu sehen, dienen die Befehle **ttrace::eval** und **ttrace::update** dazu, eine für alle Threads geltende Definition einer einfachen Tcl-Prozedur zu erzeugen und diese Definition auf alle bereits bestehenden oder später erzeugten Threads zu übertragen.

BENUTZERBEFEHLE

Dieser Abschnitt beschreibt Befehle auf Benutzerebene. Skriptautoren können mit diesen Befehlen die Ausführung des Trace-Rahmens steuern.

ttrace::eval *arg* ?arg ...?

Dieser Befehl reiht die gegebenen Argumente aneinander und wertet den entstehenden Tcl-Befehl aus, wobei der Trace-Rahmen eingeschaltet ist. War die Ausführung des Befehls in Ordnung, unternimmt er die nötigen Schritte, um die Änderung der Trace-Epoche automatisch

an alle Threads in der Anwendung weiterzugeben. Beim AOLserver erhalten nur neu erzeugte Threads die Änderung der Epoche. Bei der Tcl-Threading-Erweiterung werden alle von der Erweiterung erzeugten Threads automatisch aktualisiert. Führte die Ausführung des Befehls zu einem Tcl-Fehler, findet keine Weitergabe des Zustands statt. Das ist der wichtigste Befehl des Pakets auf Benutzerebene, weil er die meisten der unten beschriebenen Befehle umschließt. Das vereinfacht die Sache erheblich, weil der Benutzer nur diesen einen Befehl lernen muss, um das Paket wirksam zu verwenden. Die anderen Befehle, wie unten beschrieben, stehen vor allem der Vollständigkeit halber dabei.

ttrace::enable
Aktiviert alle registrierten Rückruffunktionen im Rahmen und beginnt eine neue Trace-Epoche. Die Trace-Epoche umfasst alle Änderungen am Interpreter während der Zeit, in der die Traces aktiv sind.

ttrace::disable
Deaktiviert alle registrierten Rückruffunktionen im Rahmen und schließt die aktuelle Trace-Epoche.

ttrace::cleanup
Dient dazu, alle bei Bedarf geladenen Ressourcen im Interpreter aufzuräumen. Er bringt den Tcl-Interpreter wirksam in seinen ursprünglichen Zustand zurück.

ttrace::update ?epoch?
Dient dazu, den Zustand des Interpreters aufzufrischen, sodass er zur optionalen Trace-Epoche ?epoch? passt. Ist die optionale ?epoch? nicht angegeben, nimmt er die jüngste Trace-Epoche.

ttrace::getscript
Gibt ein zusammengesetztes Tcl-Skript zurück, das in jedem Interpreter per source geladen werden kann. Dieses Skript bereitet den Tcl-Befehl **unknown** so vor, dass er per Trace erfasste Ressourcen aus der Datenbank im Speicher laden kann. Gewöhnlich rufen andere Befehle höherer Ebene wie **ttrace::eval** und **ttrace::update** diesen Befehl automatisch auf.

RÜCKRUFFBEFEHLE

Vorweg: Das Paket enthält bereits Rückruffunktionen, um die folgenden Tcl-Befehle zu tracen: **proc**, **namespace**, **variable**, **load** und **rename**. Zusätzlich liegt ein Satz von Rückruffunktionen vor, um Ressourcen (Objekt, Klassen) von XOTcl ab Version 1.3.8 zu tracen, einer OO-Erweiterung von Tcl. Das gibt eine tragfähige Grundlage, die meisten Bedürfnisse der Praxis zu lösen, und dient als Beispiel für alle, die das Paket an ihre besonderen Bedürfnisse anpassen wollen.

Unten stehen Befehle, um Rückruffunktionen im Rahmen zu registrieren und Rückruffunktionskripte zu schreiben. Der Rahmen ruft diese Rückruffunktionen auf, um Zustandsänderungen des Interpreters zu sammeln, die Datenbank im Speicher aufzubauen, eigene Aufräumarbeiten auszuführen und verschiedene andere Aufgaben zu erledigen.

ttrace::atenable *cmd arglist body*
Registriert eine Tcl-Rückruffunktion, die bei **ttrace::enable** aktiviert wird. Registrierte Rückruffunktionen werden nach FIFO aktiviert. Die Definition der Rückruffunktion umfasst den Namen der Rückruffunktion, *cmd*, eine Liste der Argumente der Rückruffunktion, *arglist*, und den *body* der Rückruffunktion. Wirksam entspricht das der Aufrufschnittstelle des üblichen Tcl-Befehls **proc**.

ttrace::atdisable *cmd arglist body*
Registriert eine Tcl-Rückruffunktion, die bei **ttrace::disable** aktiviert wird. Registrierte Rückruffunktionen werden nach FIFO aktiviert. Die Definition der Rückruffunktion umfasst den Namen der Rückruffunktion, *cmd*, eine Liste der Argumente der Rückruffunktion, *arglist*, und den *body* der Rückruffunktion. Wirksam entspricht das der Aufrufschnittstelle des üblichen Tcl-Befehls **proc**.

ttrace::addtrace *cmd arglist body*
Registriert eine Tcl-Rückruffunktion, die aktiviert wird, um den Tcl-Befehl **cmd** zu tracen. Die Definition der Rückruffunktion umfasst den Namen des zu tracenden Tcl-Befehls, *cmd*, eine Liste der Argumente der Rückruffunktion, *arglist*, und den *body* der Rückruffunktion. Wirksam entspricht das der Aufrufschnittstelle des üblichen Tcl-Befehls **proc**.

ttrace::addscript *name body*
Registriert eine Tcl-Rückruffunktion, die aktiviert wird, um ein Tcl-Skript aufzubauen, das an andere Interpreter übergeben wird. Dieses Skript bereitet den Tcl-Befehl **unknown** vor. Registrierte Rückruffunktionen werden nach FIFO aktiviert. Die Definition der Rückruffunktion umfasst den Namen der Rückruffunktion, *name*, und den *body* der Rückruffunktion.

ttrace::addresolver *cmd arglist body*
Registriert eine Tcl-Rückruffunktion, die der überladene Tcl-Befehl **unknown** aktiviert. Registrierte Rückruffunktionen werden nach FIFO aktiviert. Diese Rückruffunktion dient dazu, die Ressource aufzulösen und sie im aktuellen Interpreter zu laden.

ttrace::addcleanup *body*
Registriert eine Tcl-Rückruffunktion, die von **trace::cleanup** aktiviert wird. Registrierte Rückruffunktionen werden nach FIFO aktiviert.

ttrace::addentry *cmd var val*
Fügt der benannten Datenbank im Speicher einen Eintrag hinzu.

ttrace::getentry *cmd var*
Gibt den Wert des Eintrags aus der benannten Datenbank im Speicher zurück.

ttrace::getentries *cmd* ?pattern?
Gibt die Namen aller Einträge aus der benannten Datenbank im Speicher zurück.

ttrace::delentry *cmd*
Löscht einen Eintrag aus der benannten Datenbank im Speicher.

ttrace::preload *cmd*
Registriert den Tcl-Befehl, der in den Interpreter geladen werden soll. Auf diese Weise registrierte Befehle sind stets Teil des Interpreters und werden nicht bei Bedarf durch den Tcl-Befehl **unknown** geladen.

DISKUSSION

Übliche Verfahren, den Zustand durch Introspektion nachzubilden, verwenden ein eigenes Tcl-Skript, um den laufenden Interpreter zu untersuchen und ein weiteres Tcl-Skript zu erzeugen, das diesen Zustand in einem anderen Interpreter nachbildet. Dieses Paket verwendet stattdessen Tcl-Befehlstraces. Befehlstraces werden auf ausgewählte Tcl-Befehle registriert, etwa **proc**, **namespace**, **load** und andere übliche (und/oder vom Benutzer definierte) Tcl-Befehle. Sind sie aktiv, bauen diese Traces eine Datenbank der erzeugten Ressourcen im Speicher auf. Diese Datenbank dient als Ressourcenlager für den (überladenen) Tcl-Befehl **unknown**, der die angeforderte Ressource im Interpreter bei Bedarf erzeugt. So können Benutzer nur einen

Interpreter (den Master) in einem Thread aktualisieren und diesen Interpreterzustand (oder einen Teil davon) auf andere Threads und Interpreter im Prozess übertragen.

Der unmittelbare Gewinn dieses Vorgehens ist der deutlich kleinere Speicherbedarf der Anwendung und die deutlich schnellere Erzeugung von Threads. Werden nicht alle nötigen Prozeduren (und anderen Ressourcen) in jedem Thread schon bei der Initialisierung des Threads geladen, sondern erst dann, wenn die Ressource tatsächlich verwendet wird, lassen sich Speicherverbrauch und Initialisierungszeit der Threads deutlich verbessern. Einige Tests haben gezeigt, dass der Speicherbedarf einer Tcl-Anwendung mit mehreren Threads auf weniger als ein Drittel sank und die Startzeit der Threads etwa auf ein Fünfzigstel zurückging. Die Ergebnisse können davon abweichen. Weitere Gewinne sind die feinere Steuerung, was (und wann) vom Master auf andere Tcl-Threads und Interpreter übertragen wird.

Lizenz

Deutsche Übersetzung © 2026 Gregor Ebbing.

Die Seiten wurden aus den Originalen nach Markdown umgewandelt und ins Deutsche übersetzt; Auszeichnung ohne Platz im Format (Verweise ohne Ziel, Layoutbilder) kann dabei wegfallen. Das sind die Änderungen; für Teile unter der Mozilla Public License 1.1 (tDOM) ist es die Beschreibung der Änderungen nach Abschnitt 3.3.

Die Übersetzung steht unter denselben Bedingungen wie das jeweilige englische Original. Diese Bedingungen folgen. Die Copyright-Angaben der Originalseiten stehen bei jeder Seite.

Lizenzbedingungen bwidget

BWidget ToolKit

Copyright (c) 1998-1999 UNIFIX.

Copyright (c) 2001-2002 ActiveState Corp.

The following terms apply to all files associated with the software
unless explicitly disclaimed in individual files.

The authors hereby grant permission to use, copy, modify, distribute,
and license this software and its documentation for any purpose, provided
that existing copyright notices are retained in all copies and that this
notice is included verbatim in any distributions. No written agreement,
license, or royalty fee is required for any of the authorized uses.
Modifications to this software may be copyrighted by their authors
and need not follow the licensing terms described here, provided that
the new terms are clearly indicated on the first page of each file where
they apply.

IN NO EVENT SHALL THE AUTHORS OR DISTRIBUTORS BE LIABLE TO ANY PARTY
FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES
ARISING OUT OF THE USE OF THIS SOFTWARE, ITS DOCUMENTATION, OR ANY
DERIVATIVES THEREOF, EVEN IF THE AUTHORS HAVE BEEN ADVISED OF THE

POSSIBILITY OF SUCH DAMAGE.
THE AUTHORS AND DISTRIBUTORS SPECIFICALLY DISCLAIM ANY WARRANTIES,
INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT. THIS SOFTWARE
IS PROVIDED ON AN "AS IS" BASIS, AND THE AUTHORS AND DISTRIBUTORS HAVE
NO OBLIGATION TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR
MODIFICATIONS.
GOVERNMENT USE: If you are acquiring this software on behalf of the
U.S. government, the Government shall have only "Restricted Rights"
in the software and related documentation as defined in the Federal
Acquisition Regulations (FARs) in Clause 52.227.19 (c) (2). If you
are acquiring the software on behalf of the Department of Defense, the
software shall be classified as "Commercial Computer Software" and the
Government shall have only "Restricted Rights" as defined in Clause
252.227-7013 (c) (1) of DFARs. Notwithstanding the foregoing, the
authors grant the U.S. Government and others acting in its behalf
permission to use and distribute the software in accordance with the
terms specified in this license.

Lizenzbedingungen expect

Expect

Written by: Don Libes, libes at nist.gov, NIST

Design and implementation of this program was paid for by U.S. tax dollars. Therefore it is public domain. However, the author and NIST would appreciate credit if this program or parts of it are used.

Lizenzbedingungen itcl

This is a mostly rewritten version of [incr Tcl], which is copyrighted by Arnulf P. Wiedemann (c) Copyright 2008. It is derived from a version written by Lucent Technologies, Inc., and other parties see that copyright below.

The rewritten version is copyrighted with BSD license or Public Domain at your choice.

The original version of this software is copyrighted by Lucent Technologies, Inc., and other parties. The following terms apply to all files associated with the software unless explicitly disclaimed in individual files.

The authors hereby grant permission to use, copy, modify, distribute, and license this software and its documentation for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions. No written agreement, license, or royalty fee is required for any of the authorized uses. Modifications to this software may be copyrighted by their authors and need not follow the licensing terms described here, provided that the new terms are clearly indicated on the first page of each file where they apply.

IN NO EVENT SHALL THE AUTHORS OR DISTRIBUTORS BE LIABLE TO ANY PARTY FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OF THIS SOFTWARE, ITS DOCUMENTATION, OR ANY DERIVATIVES THEREOF, EVEN IF THE AUTHORS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

THE AUTHORS AND DISTRIBUTORS SPECIFICALLY DISCLAIM ANY WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT. THIS SOFTWARE IS PROVIDED ON AN "AS IS" BASIS, AND THE AUTHORS AND DISTRIBUTORS HAVE NO OBLIGATION TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR MODIFICATIONS.

GOVERNMENT USE: If you are acquiring this software on behalf of the U.S. government, the Government shall have only "Restricted Rights" in the software and related documentation as defined in the Federal Acquisition Regulations (FARs) in Clause 52.227.19 (c) (2). If you are acquiring the software on behalf of the Department of Defense, the software shall be classified as "Commercial Computer Software" and the Government shall have only "Restricted Rights" as defined in Clause 252.227-7013 (c) (1) of DFARs. Notwithstanding the foregoing, the authors grant the U.S. Government and others acting in its behalf permission to use and distribute the software in accordance with the terms specified in this license.

Lizenzbedingungen itk

This software is copyrighted by Lucent Technologies, Inc., and other parties. The following terms apply to all files associated with the software unless explicitly disclaimed in individual files.

The authors hereby grant permission to use, copy, modify, distribute, and license this software and its documentation for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions. No written agreement, license, or royalty fee is required for any of the authorized uses. Modifications to this software may be copyrighted by their authors and need not follow the licensing terms described here, provided that the new terms are clearly indicated on the first page of each file where they apply.

IN NO EVENT SHALL THE AUTHORS OR DISTRIBUTORS BE LIABLE TO ANY PARTY FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OF THIS SOFTWARE, ITS DOCUMENTATION, OR ANY DERIVATIVES THEREOF, EVEN IF THE AUTHORS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

THE AUTHORS AND DISTRIBUTORS SPECIFICALLY DISCLAIM ANY WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT. THIS SOFTWARE IS PROVIDED ON AN "AS IS" BASIS, AND THE AUTHORS AND DISTRIBUTORS HAVE NO OBLIGATION TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR MODIFICATIONS.

GOVERNMENT USE: If you are acquiring this software on behalf of the U.S. government, the Government shall have only "Restricted Rights" in the software and related documentation as defined in the Federal Acquisition Regulations (FARs) in Clause 52.227.19 (c) (2). If you are acquiring the software on behalf of the Department of Defense, the software shall be classified as "Commercial Computer Software" and the Government shall have only "Restricted Rights" as defined in Clause 252.227-7013 (c) (1) of DFARs. Notwithstanding the foregoing, the authors grant the U.S. Government and others acting in its behalf permission to use and distribute the software in accordance with the terms specified in this license.

Lizenzbedingungen mclistbox

Copyright (c) 1999, Bryan Douglas Oakley All Rights Reserved.

This software is provided AS-IS with no waranty expressed or implied. This software may be used free of charge, though I would appreciate it if you give credit where credit is due and mention my name when you use it.

Lizenzbedingungen mpexpr

This software is copyrighted by Tom Poindexter and Talus Technologies, Inc., 1998. The following terms apply to all files associated with the software unless explicitly disclaimed in individual files.

The authors hereby grant permission to use, copy, modify, distribute, and license this software and its documentation for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions. No written agreement, license, or royalty fee is required for any of the authorized uses. Modifications to this software may be copyrighted by their authors and need not follow the licensing terms described here, provided that the new terms are clearly indicated on the first page of each file where they apply.

IN NO EVENT SHALL THE AUTHORS OR DISTRIBUTORS BE LIABLE TO ANY PARTY FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OF THIS SOFTWARE, ITS DOCUMENTATION, OR ANY DERIVATIVES THEREOF, EVEN IF THE AUTHORS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

THE AUTHORS AND DISTRIBUTORS SPECIFICALLY DISCLAIM ANY WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT. THIS SOFTWARE IS PROVIDED ON AN "AS IS" BASIS, AND THE AUTHORS AND DISTRIBUTORS HAVE NO OBLIGATION TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR MODIFICATIONS.

GOVERNMENT USE: If you are acquiring this software on behalf of the U.S. government, the Government shall have only "Restricted Rights" in the software and related documentation as defined in the Federal Acquisition Regulations (FARs) in Clause 52.227.19 (c) (2). If you are acquiring the software on behalf of the Department of Defense, the software shall be classified as "Commercial Computer Software" and the Government shall have only "Restricted Rights" as defined in Clause 252.227-7013 (c) (1) of DFARs. Notwithstanding the foregoing, the authors grant the U.S. Government and others acting in its behalf permission to use and distribute the software in accordance with the terms specified in this license.

Lizenzbedingungen tclhttpd

TCLHTTPD License

This software is copyrighted by Brent Welch, Sun Microsystems, Inc, Scriptics, and Panasas. The following terms apply to all files associated with the software unless explicitly disclaimed in individual files.

The authors hereby grant permission to use, copy, modify, distribute, and license this software and its documentation for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions. No written agreement, license, or royalty fee is required for any of the authorized uses. Modifications to this software may be copyrighted by their authors and need not follow the licensing terms described here,

provided that the new terms are clearly indicated on the first page of each file where they apply.

IN NO EVENT SHALL THE AUTHORS OR DISTRIBUTORS BE LIABLE TO ANY PARTY FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OF THIS SOFTWARE, ITS DOCUMENTATION, OR ANY DERIVATIVES THEREOF, EVEN IF THE AUTHORS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

THE AUTHORS AND DISTRIBUTORS SPECIFICALLY DISCLAIM ANY WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT. THIS SOFTWARE IS PROVIDED ON AN "AS IS" BASIS, AND THE AUTHORS AND DISTRIBUTORS HAVE NO OBLIGATION TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR MODIFICATIONS.

GOVERNMENT USE: If you are acquiring this software on behalf of the U.S. government, the Government shall have only "Restricted Rights" in the software and related documentation as defined in the Federal Acquisition Regulations (FARs) in Clause 52.227.19 (c) (2). If you are acquiring the software on behalf of the Department of Defense, the software shall be classified as "Commercial Computer Software" and the Government shall have only "Restricted Rights" as defined in Clause 252.227-7013 (c) (1) of DFARs. Notwithstanding the foregoing, the authors grant the U.S. Government and others acting in its behalf permission to use and distribute the software in accordance with the terms specified in this license.

Lizenzbedingungen tcltls

This software is copyrighted by Matt Newman <matt@novadigm.com> and other parties. The following terms apply to all files associated with the software unless explicitly disclaimed in individual files.

The authors hereby grant permission to use, copy, modify, distribute, and license this software and its documentation for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions. No written agreement, license, or royalty fee is required for any of the authorized uses. Modifications to this software may be copyrighted by their authors and need not follow the licensing terms described here, provided that the new terms are clearly indicated on the first page of each file where they apply.

IN NO EVENT SHALL THE AUTHORS OR DISTRIBUTORS BE LIABLE TO ANY PARTY FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OF THIS SOFTWARE, ITS DOCUMENTATION, OR ANY DERIVATIVES THEREOF, EVEN IF THE AUTHORS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

THE AUTHORS AND DISTRIBUTORS SPECIFICALLY DISCLAIM ANY WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT. THIS SOFTWARE IS PROVIDED ON AN "AS IS" BASIS, AND THE AUTHORS AND DISTRIBUTORS HAVE NO OBLIGATION TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR MODIFICATIONS.

GOVERNMENT USE: If you are acquiring this software on behalf of the U.S. government, the Government shall have only "Restricted Rights" in the software and related documentation as defined in the Federal Acquisition Regulations (FARs) in Clause 52.227.19 (c) (2). If you are acquiring the software on behalf of the Department of Defense, the software shall be classified as "Commercial Computer Software" and the Government shall have only "Restricted Rights" as defined in Clause 252.227-7013 (c) (1) of DFARs. Notwithstanding the foregoing, the authors

grant the U.S. Government and others acting in its behalf permission to use and distribute the software in accordance with the terms specified in this license.

Lizenzbedingungen tcludp

Copyright (c) 1999-2003 by Columbia University; all rights reserved Copyright (c) 2003-2006 Pat Thoyts <patthoyts@users.sourceforge.net>

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Lizenzbedingungen tclvfs

This software is copyrighted by the Vince Darley, and other parties. The following terms apply to all files associated with the software unless explicitly disclaimed in individual files.

The authors hereby grant permission to use, copy, modify, distribute, and license this software and its documentation for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions. No written agreement, license, or royalty fee is required for any of the authorized uses. Modifications to this software may be copyrighted by their authors and need not follow the licensing terms described here, provided that the new terms are clearly indicated on the first page of each file where they apply.

IN NO EVENT SHALL THE AUTHORS OR DISTRIBUTORS BE LIABLE TO ANY PARTY FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OF THIS SOFTWARE, ITS DOCUMENTATION, OR ANY DERIVATIVES THEREOF, EVEN IF THE AUTHORS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

THE AUTHORS AND DISTRIBUTORS SPECIFICALLY DISCLAIM ANY WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT. THIS SOFTWARE IS PROVIDED ON AN "AS IS" BASIS, AND THE AUTHORS AND DISTRIBUTORS HAVE NO OBLIGATION TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR MODIFICATIONS.

GOVERNMENT USE: If you are acquiring this software on behalf of the U.S. government, the Government shall have only "Restricted Rights" in the software and related documentation as defined in the Federal Acquisition Regulations (FARs) in Clause 52.227.19 (c) (2). If you are acquiring the software on behalf of the Department of Defense, the software shall be classified as "Commercial Computer Software" and the Government shall have only "Restricted Rights" as defined in Clause 252.227-7013 (c) (1) of DFARs. Notwithstanding the foregoing, the authors

grant the U.S. Government and others acting in its behalf permission to use and distribute the software in accordance with the terms specified in this license.

Lizenzbedingungen tclws

This software is copyrighted by the Gerald W. Leser and other parties. The following terms apply to all files associated with the software unless explicitly disclaimed in individual files.

The authors hereby grant permission to use, copy, modify, distribute, and license this software and its documentation for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions. No written agreement, license, or royalty fee is required for any of the authorized uses. Modifications to this software may be copyrighted by their authors and need not follow the licensing terms described here, provided that the new terms are clearly indicated on the first page of each file where they apply.

IN NO EVENT SHALL THE AUTHORS OR DISTRIBUTORS BE LIABLE TO ANY PARTY FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OF THIS SOFTWARE, ITS DOCUMENTATION, OR ANY DERIVATIVES THEREOF, EVEN IF THE AUTHORS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

THE AUTHORS AND DISTRIBUTORS SPECIFICALLY DISCLAIM ANY WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT. THIS SOFTWARE IS PROVIDED ON AN "AS IS" BASIS, AND THE AUTHORS AND DISTRIBUTORS HAVE NO OBLIGATION TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR MODIFICATIONS.

GOVERNMENT USE: If you are acquiring this software on behalf of the U.S. government, the Government shall have only "Restricted Rights" in the software and related documentation as defined in the Federal Acquisition Regulations (FARs) in Clause 52.227.19 (c) (2). If you are acquiring the software on behalf of the Department of Defense, the software shall be classified as "Commercial Computer Software" and the Government shall have only "Restricted Rights" as defined in Clause 252.227-7014 (b) (3) of DFARs. Notwithstanding the foregoing, the authors grant the U.S. Government and others acting in its behalf permission to use and distribute the software in accordance with the terms specified in this license.

Lizenzbedingungen tdbc, tdbcmysql, tdbcodbc

This software is copyrighted by Kevin B. Kenny, and by other parties. The following terms apply to all files associated with the software unless explicitly disclaimed in individual files.

The authors hereby grant permission to use, copy, modify, distribute, and license this software and its documentation for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions. No written agreement, license, or royalty fee is required for any of the authorized uses. Modifications to this software may be copyrighted by their authors and need not follow the licensing terms described here, provided that the new terms are clearly indicated on the first page of each file where they apply.

IN NO EVENT SHALL THE AUTHORS OR DISTRIBUTORS BE LIABLE TO ANY PARTY FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OF THIS SOFTWARE, ITS DOCUMENTATION, OR ANY DERIVATIVES THEREOF, EVEN IF THE AUTHORS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

THE AUTHORS AND DISTRIBUTORS SPECIFICALLY DISCLAIM ANY WARRANTIES, INCLUDING, BUT

NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT. THIS SOFTWARE IS PROVIDED ON AN "AS IS" BASIS, AND THE AUTHORS AND DISTRIBUTORS HAVE NO OBLIGATION TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR MODIFICATIONS.

GOVERNMENT USE: If you are acquiring this software on behalf of the U.S. government, the Government shall have only "Restricted Rights" in the software and related documentation as defined in the Federal Acquisition Regulations (FARs) in Clause 52.227.19 (c) (2). If you are acquiring the software on behalf of the Department of Defense, the software shall be classified as "Commercial Computer Software" and the Government shall have only "Restricted Rights" as defined in Clause 252.227-7013 (c) (1) of DFARs. Notwithstanding the foregoing, the authors grant the U.S. Government and others acting in its behalf permission to use and distribute the software in accordance with the terms specified in this license.

Lizenzbedingungen tdbcpostgres

This software is copyrighted by Slawomir Cygan, and by other parties. The following terms apply to all files associated with the software unless explicitly disclaimed in individual files.

The authors hereby grant permission to use, copy, modify, distribute, and license this software and its documentation for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions. No written agreement, license, or royalty fee is required for any of the authorized uses. Modifications to this software may be copyrighted by their authors and need not follow the licensing terms described here, provided that the new terms are clearly indicated on the first page of each file where they apply.

IN NO EVENT SHALL THE AUTHORS OR DISTRIBUTORS BE LIABLE TO ANY PARTY FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OF THIS SOFTWARE, ITS DOCUMENTATION, OR ANY DERIVATIVES THEREOF, EVEN IF THE AUTHORS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

THE AUTHORS AND DISTRIBUTORS SPECIFICALLY DISCLAIM ANY WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT. THIS SOFTWARE IS PROVIDED ON AN "AS IS" BASIS, AND THE AUTHORS AND DISTRIBUTORS HAVE NO OBLIGATION TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR MODIFICATIONS.

GOVERNMENT USE: If you are acquiring this software on behalf of the U.S. government, the Government shall have only "Restricted Rights" in the software and related documentation as defined in the Federal Acquisition Regulations (FARs) in Clause 52.227.19 (c) (2). If you are acquiring the software on behalf of the Department of Defense, the software shall be classified as "Commercial Computer Software" and the Government shall have only "Restricted Rights" as defined in Clause 252.227-7013 (c) (1) of DFARs. Notwithstanding the foregoing, the authors grant the U.S. Government and others acting in its behalf permission to use and distribute the software in accordance with the terms specified in this license.

Lizenzbedingungen tdbcsqlite3

This software is copyrighted by the Scriptics Corporation, and other parties. The following terms apply to all files associated with the software unless explicitly disclaimed in individual files.

The authors hereby grant permission to use, copy, modify, distribute, and license this software

and its documentation for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions. No written agreement, license, or royalty fee is required for any of the authorized uses. Modifications to this software may be copyrighted by their authors and need not follow the licensing terms described here, provided that the new terms are clearly indicated on the first page of each file where they apply.

IN NO EVENT SHALL THE AUTHORS OR DISTRIBUTORS BE LIABLE TO ANY PARTY FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OF THIS SOFTWARE, ITS DOCUMENTATION, OR ANY DERIVATIVES THEREOF, EVEN IF THE AUTHORS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

THE AUTHORS AND DISTRIBUTORS SPECIFICALLY DISCLAIM ANY WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT. THIS SOFTWARE IS PROVIDED ON AN "AS IS" BASIS, AND THE AUTHORS AND DISTRIBUTORS HAVE NO OBLIGATION TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR MODIFICATIONS.

GOVERNMENT USE: If you are acquiring this software on behalf of the U.S. government, the Government shall have only "Restricted Rights" in the software and related documentation as defined in the Federal Acquisition Regulations (FARs) in Clause 52.227.19 (c) (2). If you are acquiring the software on behalf of the Department of Defense, the software shall be classified as "Commercial Computer Software" and the Government shall have only "Restricted Rights" as defined in Clause 252.227-7013 (c) (1) of DFARs. Notwithstanding the foregoing, the authors grant the U.S. Government and others acting in its behalf permission to use and distribute the software in accordance with the terms specified in this license.

Lizenzbedingungen tdom

The tDOM specific source files come with the Mozilla Public License Version 1.1. If you want to distribute/use tDOM under a different license please feel free to contact the author (loewerj@hotmail.com).

For Expat XML parser package license look into the directory expat.

Mozilla Public License Version 1.1

1. Definitions.

1.0.1. "Commercial Use" means distribution or otherwise making the Covered Code available to a third party.

1.1. "Contributor" means each entity that creates or contributes to the creation of Modifications.

1.2. "Contributor Version" means the combination of the Original Code, prior Modifications used by a Contributor, and the Modifications made by that particular Contributor.

1.3. "Covered Code" means the Original Code or Modifications or the combination of the Original Code and Modifications, in each case including portions thereof.

1.4. "Electronic Distribution Mechanism" means a mechanism generally accepted in the software development community for the electronic transfer of data.

1.5. "Executable" means Covered Code in any form other than Source Code.

1.6. "Initial Developer" means the individual or entity identified as the Initial Developer in the

Source Code notice required by Exhibit A.

1.7. "Larger Work" means a work which combines Covered Code or portions thereof with code not governed by the terms of this License.

1.8. "License" means this document.

1.8.1. "Licensable" means having the right to grant, to the maximum extent possible, whether at the time of the initial grant or subsequently acquired, any and all of the rights conveyed herein.

1.9. "Modifications" means any addition to or deletion from the substance or structure of either the Original Code or any previous Modifications. When Covered Code is released as a series of files, a Modification is: Any addition to or deletion from the contents of a file containing Original Code or previous Modifications. Any new file that contains any part of the Original Code or previous Modifications.

1.10. "Original Code" means Source Code of computer software code which is described in the Source Code notice required by Exhibit A as Original Code, and which, at the time of its release under this License is not already Covered Code governed by this License.

1.10.1. "Patent Claims" means any patent claim(s), now owned or hereafter acquired, including without limitation, method, process, and apparatus claims, in any patent Licensable by grantor.

1.11. "Source Code" means the preferred form of the Covered Code for making modifications to it, including all modules it contains, plus any associated interface definition files, scripts used to control compilation and installation of an Executable, or source code differential comparisons against either the Original Code or another well known, available Covered Code of the Contributor's choice. The Source Code can be in a compressed or archival form, provided the appropriate decompression or de-archiving software is widely available for no charge.

1.12. "You" (or "Your") means an individual or a legal entity exercising rights under, and complying with all of the terms of, this License or a future version of this License issued under Section 6.1. For legal entities, "You" includes any entity which controls, is controlled by, or is under common control with You. For purposes of this definition, "control" means (a) the power, direct or indirect, to cause the direction or management of such entity, whether by contract or otherwise, or (b) ownership of more than fifty percent (50%) of the outstanding shares or beneficial ownership of such entity.

1. Source Code License.

2.1. The Initial Developer Grant. The Initial Developer hereby grants You a world-wide, royalty-free, non-exclusive license, subject to third party intellectual property claims:

a. under intellectual property rights (other than patent or trademark) Licensable by Initial Developer to use, reproduce, modify, display, perform, sublicense and distribute the Original Code (or portions thereof) with or without Modifications, and/or as part of a Larger Work; and b. under Patents Claims infringed by the making, using or selling of Original Code, to make, have made, use, practice, sell, and offer for sale, and/or otherwise dispose of the Original Code (or portions thereof). c. the licenses granted in this Section 2.1 (a) and (b) are effective on the date Initial Developer first distributes Original Code under the terms of this License. d. Notwithstanding Section 2.1 (b) above, no patent license is granted: 1) for code that You delete from the Original Code; 2) separate from the Original Code; or 3) for infringements caused by: i) the modification of the Original Code or ii) the combination of the Original Code with other software or devices.

2.2. Contributor Grant. Subject to third party intellectual property claims, each Contributor hereby grants You a world-wide, royalty-free, non-exclusive license

a. under intellectual property rights (other than patent or trademark) Licensable by Contributor, to use, reproduce, modify, display, perform, sublicense and distribute the Modifications created by such Contributor (or portions thereof) either on an unmodified basis, with other Modifications, as Covered Code and/or as part of a Larger Work; and b. under Patent Claims infringed by the making, using, or selling of Modifications made by that Contributor either alone and/or in combination with its Contributor Version (or portions of such combination), to make, use, sell, offer for sale, have made, and/or otherwise dispose of: 1) Modifications made by that Contributor (or portions thereof); and 2) the combination of Modifications made by that Contributor with its Contributor Version (or portions of such combination). c. the licenses granted in Sections 2.2 (a) and 2.2 (b) are effective on the date Contributor first makes Commercial Use of the Covered Code. d. Notwithstanding Section 2.2 (b) above, no patent license is granted: 1) for any code that Contributor has deleted from the Contributor Version; 2) separate from the Contributor Version; 3) for infringements caused by: i) third party modifications of Contributor Version or ii) the combination of Modifications made by that Contributor with other software (except as part of the Contributor Version) or other devices; or 4) under Patent Claims infringed by Covered Code in the absence of Modifications made by that Contributor.

1. Distribution Obligations.

3.1. Application of License. The Modifications which You create or to which You contribute are governed by the terms of this License, including without limitation Section 2.2. The Source Code version of Covered Code may be distributed only under the terms of this License or a future version of this License released under Section 6.1, and You must include a copy of this License with every copy of the Source Code You distribute. You may not offer or impose any terms on any Source Code version that alters or restricts the applicable version of this License or the recipients' rights hereunder. However, You may include an additional document offering the additional rights described in Section 3.5.

3.2. Availability of Source Code. Any Modification which You create or to which You contribute must be made available in Source Code form under the terms of this License either on the same media as an Executable version or via an accepted Electronic Distribution Mechanism to anyone to whom you made an Executable version available; and if made available via Electronic Distribution Mechanism, must remain available for at least twelve (12) months after the date it initially became available, or at least six (6) months after a subsequent version of that particular Modification has been made available to such recipients. You are responsible for ensuring that the Source Code version remains available even if the Electronic Distribution Mechanism is maintained by a third party.

3.3. Description of Modifications. You must cause all Covered Code to which You contribute to contain a file documenting the changes You made to create that Covered Code and the date of any change. You must include a prominent statement that the Modification is derived, directly or indirectly, from Original Code provided by the Initial Developer and including the name of the Initial Developer in (a) the Source Code, and (b) in any notice in an Executable version or related documentation in which You describe the origin or ownership of the Covered Code.

3.4. Intellectual Property Matters

(a) Third Party Claims If Contributor has knowledge that a license under a third party's

intellectual property rights is required to exercise the rights granted by such Contributor under Sections 2.1 or 2.2, Contributor must include a text file with the Source Code distribution titled "LEGAL" which describes the claim and the party making the claim in sufficient detail that a recipient will know whom to contact. If Contributor obtains such knowledge after the Modification is made available as described in Section 3.2, Contributor shall promptly modify the LEGAL file in all copies Contributor makes available thereafter and shall take other steps (such as notifying appropriate mailing lists or newsgroups) reasonably calculated to inform those who received the Covered Code that new knowledge has been obtained.

(b) Contributor APIs If Contributor's Modifications include an application programming interface and Contributor has knowledge of patent licenses which are reasonably necessary to implement that API, Contributor must also include this information in the LEGAL file.

(c) Representations. Contributor represents that, except as disclosed pursuant to Section 3.4 (a) above, Contributor believes that Contributor's Modifications are Contributor's original creation(s) and/or Contributor has sufficient rights to grant the rights conveyed by this License.

3.5. Required Notices. You must duplicate the notice in Exhibit A in each file of the Source Code. If it is not possible to put such notice in a particular Source Code file due to its structure, then You must include such notice in a location (such as a relevant directory) where a user would be likely to look for such a notice. If You created one or more Modification(s) You may add your name as a Contributor to the notice described in Exhibit A. You must also duplicate this License in any documentation for the Source Code where You describe recipients' rights or ownership rights relating to Covered Code. You may choose to offer, and to charge a fee for, warranty, support, indemnity or liability obligations to one or more recipients of Covered Code. However, You may do so only on Your own behalf, and not on behalf of the Initial Developer or any Contributor. You must make it absolutely clear than any such warranty, support, indemnity or liability obligation is offered by You alone, and You hereby agree to indemnify the Initial Developer and every Contributor for any liability incurred by the Initial Developer or such Contributor as a result of warranty, support, indemnity or liability terms You offer.

3.6. Distribution of Executable Versions. You may distribute Covered Code in Executable form only if the requirements of Sections 3.1, 3.2, 3.3, 3.4 and 3.5 have been met for that Covered Code, and if You include a notice stating that the Source Code version of the Covered Code is available under the terms of this License, including a description of how and where You have fulfilled the obligations of Section 3.2. The notice must be conspicuously included in any notice in an Executable version, related documentation or collateral in which You describe recipients' rights relating to the Covered Code. You may distribute the Executable version of Covered Code or ownership rights under a license of Your choice, which may contain terms different from this License, provided that You are in compliance with the terms of this License and that the license for the Executable version does not attempt to limit or alter the recipient's rights in the Source Code version from the rights set forth in this License. If You distribute the Executable version under a different license You must make it absolutely clear that any terms which differ from this License are offered by You alone, not by the Initial Developer or any Contributor. You hereby agree to indemnify the Initial Developer and every Contributor for any liability incurred by the Initial Developer or such Contributor as a result of any such terms You offer.

3.7. Larger Works. You may create a Larger Work by combining Covered Code with other code not governed by the terms of this License and distribute the Larger Work as a single product. In such a case, You must make sure the requirements of this License are fulfilled for the Covered

Code.

1. Inability to Comply Due to Statute or Regulation.

If it is impossible for You to comply with any of the terms of this License with respect to some or all of the Covered Code due to statute, judicial order, or regulation then You must: (a) comply with the terms of this License to the maximum extent possible; and (b) describe the limitations and the code they affect. Such description must be included in the LEGAL file described in Section 3.4 and must be included with all distributions of the Source Code. Except to the extent prohibited by statute or regulation, such description must be sufficiently detailed for a recipient of ordinary skill to be able to understand it.

1. Application of this License. This License applies to code to which the Initial Developer has attached the notice in Exhibit A and to related Covered Code.
2. Versions of the License.

6.1. New Versions Netscape Communications Corporation ("Netscape") may publish revised and/or new versions of the License from time to time. Each version will be given a distinguishing version number.

6.2. Effect of New Versions Once Covered Code has been published under a particular version of the License, You may always continue to use it under the terms of that version. You may also choose to use such Covered Code under the terms of any subsequent version of the License published by Netscape. No one other than Netscape has the right to modify the terms applicable to Covered Code created under this License.

6.3. Derivative Works If You create or use a modified version of this License (which you may only do in order to apply it to code which is not already Covered Code governed by this License), You must (a) rename Your license so that the phrases "Mozilla", "MOZILLAPL", "MOZPL", "Netscape", "MPL", "NPL" or any confusingly similar phrase do not appear in your license (except to note that your license differs from this License) and (b) otherwise make it clear that Your version of the license contains terms which differ from the Mozilla Public License and Netscape Public License. (Filling in the name of the Initial Developer, Original Code or Contributor in the notice described in Exhibit A shall not of themselves be deemed to be modifications of this License.)

1. DISCLAIMER OF WARRANTY COVERED CODE IS PROVIDED UNDER THIS LICENSE ON AN "AS IS" BASIS, WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, WITHOUT LIMITATION, WARRANTIES THAT THE COVERED CODE IS FREE OF DEFECTS, MERCHANTABLE, FIT FOR A PARTICULAR PURPOSE OR NON-INFRINGING. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THE COVERED CODE IS WITH YOU. SHOULD ANY COVERED CODE PROVE DEFECTIVE IN ANY RESPECT, YOU (NOT THE INITIAL DEVELOPER OR ANY OTHER CONTRIBUTOR) ASSUME THE COST OF ANY NECESSARY SERVICING, REPAIR OR CORRECTION. THIS DISCLAIMER OF WARRANTY CONSTITUTES AN ESSENTIAL PART OF THIS LICENSE. NO USE OF ANY COVERED CODE IS AUTHORIZED HEREUNDER EXCEPT UNDER THIS DISCLAIMER.
2. Termination

8.1. This License and the rights granted hereunder will terminate automatically if You fail to comply with terms herein and fail to cure such breach within 30 days of becoming aware of the breach. All sublicenses to the Covered Code which are properly granted shall survive any termination of this License. Provisions which, by their nature, must remain in effect beyond the termination of this License shall survive.

8.2. If You initiate litigation by asserting a patent infringement claim (excluding declatory judgment actions) against Initial Developer or a Contributor (the Initial Developer or Contributor against whom You file such action is referred to as "Participant") alleging that:

a. such Participant's Contributor Version directly or indirectly infringes any patent, then any and all rights granted by such Participant to You under Sections 2.1 and/or 2.2 of this License shall, upon 60 days notice from Participant terminate prospectively, unless if within 60 days after receipt of notice You either: (i) agree in writing to pay Participant a mutually agreeable reasonable royalty for Your past and future use of Modifications made by such Participant, or (ii) withdraw Your litigation claim with respect to the Contributor Version against such Participant. If within 60 days of notice, a reasonable royalty and payment arrangement are not mutually agreed upon in writing by the parties or the litigation claim is not withdrawn, the rights granted by Participant to You under Sections 2.1 and/or 2.2 automatically terminate at the expiration of the 60 day notice period specified above. b. any software, hardware, or device, other than such Participant's Contributor Version, directly or indirectly infringes any patent, then any rights granted to You by such Participant under Sections 2.1(b) and 2.2(b) are revoked effective as of the date You first made, used, sold, distributed, or had made, Modifications made by that Participant.

8.3. If You assert a patent infringement claim against Participant alleging that such Participant's Contributor Version directly or indirectly infringes any patent where such claim is resolved (such as by license or settlement) prior to the initiation of patent infringement litigation, then the reasonable value of the licenses granted by such Participant under Sections 2.1 or 2.2 shall be taken into account in determining the amount or value of any payment or license.

8.4. In the event of termination under Sections 8.1 or 8.2 above, all end user license agreements (excluding distributors and resellers) which have been validly granted by You or any distributor hereunder prior to termination shall survive termination.

1. LIMITATION OF LIABILITY UNDER NO CIRCUMSTANCES AND UNDER NO LEGAL THEORY, WHETHER TORT (INCLUDING NEGLIGENCE), CONTRACT, OR OTHERWISE, SHALL YOU, THE INITIAL DEVELOPER, ANY OTHER CONTRIBUTOR, OR ANY DISTRIBUTOR OF COVERED CODE, OR ANY SUPPLIER OF ANY OF SUCH PARTIES, BE LIABLE TO ANY PERSON FOR ANY INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES OF ANY CHARACTER INCLUDING, WITHOUT LIMITATION, DAMAGES FOR LOSS OF GOODWILL, WORK STOPPAGE, COMPUTER FAILURE OR MALFUNCTION, OR ANY AND ALL OTHER COMMERCIAL DAMAGES OR LOSSES, EVEN IF SUCH PARTY SHALL HAVE BEEN INFORMED OF THE POSSIBILITY OF SUCH DAMAGES. THIS LIMITATION OF LIABILITY SHALL NOT APPLY TO LIABILITY FOR DEATH OR PERSONAL INJURY RESULTING FROM SUCH PARTY'S NEGLIGENCE TO THE EXTENT APPLICABLE LAW PROHIBITS SUCH LIMITATION. SOME JURISDICTIONS DO NOT ALLOW THE EXCLUSION OR LIMITATION OF INCIDENTAL OR CONSEQUENTIAL DAMAGES, SO THIS EXCLUSION AND LIMITATION MAY NOT APPLY TO YOU.
2. U.S. government end users The Covered Code is a "commercial item," as that term is defined in 48 C.F.R. 2.101 (Oct. 1995), consisting of "commercial computer software" and "commercial computer software documentation," as such terms are used in 48 C.F.R. 12.212 (Sept. 1995). Consistent with 48 C.F.R. 12.212 and 48 C.F.R. 227.7202-1 through 227.7202-4 (June 1995), all U.S. Government End Users acquire Covered Code with only those rights set forth herein.
3. Miscellaneous This License represents the complete agreement concerning subject matter hereof. If any provision of this License is held to be unenforceable, such provision shall be

reformed only to the extent necessary to make it enforceable. This License shall be governed by California law provisions (except to the extent applicable law, if any, provides otherwise), excluding its conflict-of-law provisions. With respect to disputes in which at least one party is a citizen of, or an entity chartered or registered to do business in the United States of America, any litigation relating to this License shall be subject to the jurisdiction of the Federal Courts of the Northern District of California, with venue lying in Santa Clara County, California, with the losing party responsible for costs, including without limitation, court costs and reasonable attorneys' fees and expenses. The application of the United Nations Convention on Contracts for the International Sale of Goods is expressly excluded. Any law or regulation which provides that the language of a contract shall be construed against the drafter shall not apply to this License.

4. Responsibility for claims As between Initial Developer and the Contributors, each party is responsible for claims and damages arising, directly or indirectly, out of its utilization of rights under this License and You agree to work with Initial Developer and Contributors to distribute such responsibility on an equitable basis. Nothing herein is intended or shall be deemed to constitute any admission of liability.
5. Multiple-licensed code Initial Developer may designate portions of the Covered Code as "Multiple-Licensed". "Multiple-Licensed" means that the Initial Developer permits you to utilize portions of the Covered Code under Your choice of the MPL or the alternative licenses, if any, specified by the Initial Developer in the file described in Exhibit A.

Exhibit A - Mozilla Public License.

"The contents of this file are subject to the Mozilla Public License Version 1.1 (the "License"); you may not use this file except in compliance with the License. You may obtain a copy of the License at http://www.mozilla.org/MPL/

Software distributed under the License is distributed on an "AS IS" basis, WITHOUT WARRANTY OF ANY KIND, either express or implied. See the License for the specific language governing rights and limitations under the License.

The Original Code is _____.

The Initial Developer of the Original Code is _____. Portions created by _____ are Copyright (C) _____ _____. All Rights Reserved.

Contributor(s): _____.

Alternatively, the contents of this file may be used under the terms of the _____ license (the "[___] License"), in which case the provisions of [_____] License are applicable instead of those above. If you wish to allow use of your version of this file only under the terms of the [____] License and not to allow others to use your version of this file under the MPL, indicate your decision by deleting the provisions above and replace them with the notice and other provisions required by the [___] License. If you do not delete the provisions above, a recipient may use your version of this file under either the MPL or the [___] License."

NOTE: The text of this Exhibit A may differ slightly from the text of the notices in the Source Code files of the Original Code. You should use the text of this Exhibit A rather than the text found in the Original Code Source Code for Your Modifications.

Lizenzbedingungen thread

This software is copyrighted by the Regents of the University of California, Sun Microsystems,

Inc., Scriptics Corporation, and other parties. The following terms apply to all files associated with the software unless explicitly disclaimed in individual files.

The authors hereby grant permission to use, copy, modify, distribute, and license this software and its documentation for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions. No written agreement, license, or royalty fee is required for any of the authorized uses. Modifications to this software may be copyrighted by their authors and need not follow the licensing terms described here, provided that the new terms are clearly indicated on the first page of each file where they apply.

IN NO EVENT SHALL THE AUTHORS OR DISTRIBUTORS BE LIABLE TO ANY PARTY FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OF THIS SOFTWARE, ITS DOCUMENTATION, OR ANY DERIVATIVES THEREOF, EVEN IF THE AUTHORS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

THE AUTHORS AND DISTRIBUTORS SPECIFICALLY DISCLAIM ANY WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT. THIS SOFTWARE IS PROVIDED ON AN "AS IS" BASIS, AND THE AUTHORS AND DISTRIBUTORS HAVE NO OBLIGATION TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR MODIFICATIONS.

GOVERNMENT USE: If you are acquiring this software on behalf of the U.S. government, the Government shall have only "Restricted Rights" in the software and related documentation as defined in the Federal Acquisition Regulations (FARs) in Clause 52.227.19 (c) (2). If you are acquiring the software on behalf of the Department of Defense, the software shall be classified as "Commercial Computer Software" and the Government shall have only "Restricted Rights" as defined in Clause 252.227-7013 (c) (1) of DFARs. Notwithstanding the foregoing, the authors grant the U.S. Government and others acting in its behalf permission to use and distribute the software in accordance with the terms specified in this license.

Stichwortverzeichnis

A

Archetype 203, 208, 211, 215
ArrowButton 6
asynchronous I/O 231, 246

B

bind 231, 246
body 155
bound variable 309, 311
Button 12
ButtonBox 15
BWidget 9

C

C handler set 361, 388
call 309, 311
callback 165
Calling_a_Web_Service 262
certificate 231
channel 231, 246
class 155, 157, 166, 172, 173, 174, 201, 213
code 165, 202
ComboBox 18
command tracing 411
component 176, 215
configbody 166
configure 166
connection 231, 246, 302, 303, 309, 311, 315, 317, 320, 322
connectivity 302, 303, 309, 311, 315, 317, 320, 322
contents 106
Creating_a_Tcl_Web_Service 275
Creating_a_Web_Service_Type 282

D

database 302, 303, 309, 311, 314, 315, 317, 320, 322
Defining_Types 285
delegation 177
delete 169
Dialog 22
Dictionary_Representation_of_XML_Arrays 288
document 324, 342
document element 333
DOM 342, 367, 388
dom 324
DOM node creation 333

domain 246
domain name 231
domDoc 333
domNode 342
DragSite 26
DropSite 27
DTD 389
DynamicHelp 30

E

Embedded_Web_Service 289
ensemble 169, 170
Entry 33
events 393
expat 350
expatapi 361
expect 118
extendedclass 176, 180, 189

F

file 251, 252, 257, 261
filesystem 251, 252, 257, 261
find 172
ftp 251, 261

H

host 231, 246
HTTP 229
http 251, 261
httpd 229
https 231

I

I/O 231, 246
import 172
index 108, 301
info 170
introspection 411
IP Address 231, 246
is 173
itcl 174, 191, 214
itclcomponent 176
itcldelegate 177
itclextendedclass 180
itcloption 189
itclvars 191
itclwidget 192
itk 203, 208, 211, 213, 214, 215
itkvars 214

L

Label 37
LabelEntry 42
LabelFrame 44
libexpect 145
ListBox 45
local 201
locking 406

M

MainFrame 52
mclistbox 216
mega-widget 174, 203, 208, 211, 213, 215
message passing 393
MessageDlg 57
metakit 251
method 177
mpexpr 221
mutex 393
MySQL 315

N

name 246
namespace 165, 169, 174, 202, 251, 261
navtree 108
network 231, 246
network address 231, 246
networking 246
node 324, 342
NoteBook 59

O

object 155, 157, 166, 172, 173, 174, 180, 192, 201, 213
object-oriented 157, 174, 180, 192, 213
ODBC 317
OpenSSL 231
option 177, 189
options 109

P

PagesManager 63
PanedWindow 65
PanelFrame 67
parsing 324, 342, 365
part 170
PasswdDlg 69
Postgres 320
Postvalidation 367

private 165
proc 169, 177
procedure 155, 201
ProgressBar 71
ProgressDlg 73
protected 165
public 165
pull 365
pullparser 365
push 350
pushparser 350

R

Rest_flavor_service_response 291
resultset 302, 303, 309, 311

S

SAX 350, 367, 388
schema 367
scope 165, 202
ScrollableFrame 76
ScrolledWindow 78
ScrollView 75
search 172
SelectColor 80
SelectFont 82
Separator 84
socket 231, 246
SpinBox 85
SQL 302, 303, 309, 311, 314, 315, 317, 320, 322
SQLite3 322
SSL 231
state 309
statement 302, 303, 309, 311
StatusBar 88
stored procedure 309, 311
synchronization 393, 406

T

tar 251, 261
Tcl_Web_Service_Example 292
Tcl_Web_Service_Math_Example 295
TclHttpd 229
TclTLS 231
TclUDP 246
TCP 231
TDBC 303, 309, 311, 314, 315, 317, 320, 322
tdbc 302

tdbc_connection 303
tdbc_mapSqlState 309
tdbc_mysql 315
tdbc_odbc 317
tdbc_postgres 320
tdbc_resultset 309
tdbc_sqlite3 322
tdbc_statement 311
tdbc_tokenize 314
tdomcmd 388
thread 393, 402
thread shared data 406
threadpool 402
threads 406
TitleFrame 89
tls 231
tnc 389
tokenize 314
Toplevel 203, 208
tpool 402
Tree 91
tsv 406
ttrace 411

U

udp 246
Using_Options 298
usual 215

V

Validation 367, 389
variable 166, 169, 202
variables 191, 214
vfs 251, 252, 257, 261
vfs-filesystems 251
vfs-fsapi 252
vfslib 251, 261

W

Web Server 229
webdav 251, 261
Widget 100, 203, 208, 211
widget 176, 189, 192
widgetadaptor 176, 189

X

XML 324, 342, 365
XPath 391
xpathFunc 391

Z

zip 251, 261